

计算机硬件从零到整机

从电、二极管、逻辑门和时钟到最小 CPU 与整机硬件

CPU Books

本书沿着器件、电平、逻辑、状态、时钟、算术、数据通路、控制器、最小 CPU、主存、总线和 I/O 的顺序，把一台计算机从最小可理解部件推导出来。

前言

这本书从硬件本身开始。全书首先建立底层约定：一根线上连续变化的电压怎样被稳定解释成 0 和 1；一个器件怎样让电流更容易沿某个方向通过；一个受控开关怎样让一个信号决定另一条电流路径是否导通。只有这些约定成立，后续的逻辑门、寄存器、ALU、CPU 和整机接口才具有可靠含义。

本书按照两部分六章展开。第一章把电平、器件、开关、逻辑门和组合电路放在同一条链路中讨论，说明 bit 如何从物理电平进入可计算的逻辑网络。第二章集中讨论状态、时钟与同步边界，说明硬件如何保存过去，并在清楚的时钟边界提交下一值。

第三章讨论数值解释、算术电路、ALU、数据通路和控制器。它把固定宽度 bit、补码、标志位、选择器、寄存器阵列和控制 trace（执行轨迹）统一起来，使读者能够从旧状态走读到新状态。第四章把这些部件合成为最小 CPU，集中说明 PC、IR、控制字、数据通路和指令 trace。第五章再把 CPU 接到寄存器、cache、TLB、DRAM、MMIO、DMA、SSD/硬盘、网络和其他 I/O 队列，说明 CPU 之外的硬件如何形成整机吞吐和可见性边界。第六章借助 6502、Z80、8051、8086、80386、80486/Pentium、PC 主板、显卡和现代 SoC 观察真实处理器与整机平台的演进。

这种组织方式刻意避免把 CPU 作为孤立名词提前给出。CPU 在本书中是前述硬件部件按同步边界组织后的结果：PC 保存下一条编码地址，译码逻辑产生控制线，数据通路执行计算和写回，外部接口把处理器连接到主存与设备。读完整本书后，读者应能把一台机器从电平、开关、门电路、组合逻辑、时序逻辑、运算部件、数据通路、控制器一路连到整机。

本书采用接近 CSAPP (Computer Systems: A Programmer's Perspective) 等经典系统教材的写法：每讲一个抽象，都同时说明它在机器层的对应实现。读逻辑门时同时对照真值表和开关路径；读组合逻辑时同时分析函数、延迟和毛刺；读寄存器时同时观察 bit 和采样边界；读 CPU 时同时跟踪指令、控制信号和数据通路 trace。章节末尾的“经典教材标注”不是附加材料，而是用来检验读者是否已经把抽象概念与机器行为联系起来。

这不是一本要求读者实际焊板、购买示波器或搭完整实验平台的书。文中的“实验”和“项目”主要指纸面推演、结构图、波形阅读、规格分析、故障定位和运行记录整理；如果读者有 FPGA、仿真器、逻辑分析仪或真实开发板，可以把这些任务落实到真实硬件上，但没有硬件也应能完成大部分训练。读者真正要掌握的不是某块板的接线，而是能把一次访问、一次计算、一次启动、一次中断或一次 DMA 写成可解释的状态边界。

扩写目录与路线图

当前书稿已经有一条从电平、逻辑门、状态、ALU、最小 CPU 到整机平台的主线，但它仍然更接近讲义，而不是完整教材。后续扩写不应继续在六章末尾堆专题，而应按经典教材的粒度拆成更多章节：每章有明确问题、结构图、案例、习题和纸面项目。下面是后续扩写采用的目标目录。

章	章节名	需要讲清楚的问题	章末项目
1	数字抽象与电气基础	电压、电流、噪声裕量、阈值、负载、电平标准、受控电流路径如何形成可靠 bit。	电平约定图、噪声裕量题
2	组合逻辑与布尔代数	真值表、布尔代数、NAND/NOR 完备性、译码器、编码器、mux、毛刺和传播延迟。	逻辑化简与毛刺案例
3	时序逻辑、寄存器和状态机	锁存器、触发器、setup/hold、复位、同步器、计数器、移位寄存器和 FSM。	状态机分析表
4	RTL 与硬件描述语言阅读	如何读 Verilog/SystemVerilog：组合块、时序块、非阻塞赋值、综合边界、testbench 思路。	RTL 走读题
5	数值表示与算术电路	无符号、补码、定点、浮点、BCD、加法器、减法器、乘除法、多精度和标志位。	边界向量表
6	ISA、汇编和机器码	指令格式、寄存器、寻址模式、立即数、条件码、调用约定、栈帧和机器码编码。	汇编到机器码 trace
7	数据通路与控制	寄存器堆、ALU、mux、控制字、硬连线控制、微程序控制和数据通路阶段。	控制信号表
8	单周期与多周期 CPU	IF/ID/EX/MEM/WB 的状态边界，单周期关键路径，多周期复用硬件和等待状态。	访存周期图
9	流水线、冒险和异常	结构冒险、数据冒险、控制冒险、转发、停顿、冲刷、精确异常和中断入口。	五级流水线题
10	cache 与存储层次	locality、cache line、tag/index/offset、替换、写策略、多级 cache 和一致性入门。	cache 命中/miss 推演
11	虚拟内存、TLB 和页表	虚拟地址、页表、TLB、权限、缺页、页错误、IOMMU 和地址空间隔离。	地址转换案例
12	DRAM/DDR 与内存控制器	SRAM 与 DRAM 差异、bank、row buffer、refresh、ECC、训练、调度和带宽。	DRAM 访问时序图
13	总线、互连和 PCIe	片选、valid/ready、AXI/APB 概念、PCIe 配置空间、BAR、TLP、MSI 和 AER。	PCIe 枚举路径

14	MMIO、中断、DMA 和 IOMMU	设备寄存器位语义、中断控制器、DMA 描述符、completion、cache 可见性和权限。	设备事务 trace
15	SSD、硬盘、网络与外设队列	块设备、NVMe、SATA、硬盘、网卡、描述符环、错误恢复、吞吐和尾延迟。	NVMe/网卡队列题
16	GPU、VRAM 和显示系统	GPU 执行模型、SIMD/SIMT、shader、VRAM、命令缓冲、fence、显示扫描和帧缓冲。	GPU 命令路径图
17	主板、固件、启动和平台管理	VRM、时钟、复位、UEFI/BIOS、PCH、DDR 训练、ACPI、EC、风扇和传感器。	启动 bring-up 日志
18	链接、装载和系统软件边界	目标文件、符号、重定位、ELF、动态链接、loader、系统调用和异常控制流。	ELF/loader 纸面项目
19	性能分析与定量方法	latency、throughput、CPI、带宽、Amdahl 定律、roofline、cache miss 代价和瓶颈定位。	性能分析题
20	综合纸面项目与故障案例集	从开机、读文件、执行代码、访问设备、GPU 显示到错误恢复的全链路 trace。	整机规格与排错报告

拆分原则

旧六章不会被简单复制成二十章。拆分时按问题边界移动内容：第一章和第二章保留电气与组合逻辑；当前状态机内容拆到第 3 章；数值和 ALU 拆到第 5、7 章；最小 CPU 拆到第 6、8、9 章；主存、cache、DMA、SSD、PCIe、GPU 和主板分别进入第 10 到第 17 章；目标文件、链接、装载和系统软件边界补成第 18 章；性能分析和综合项目补成第 19、20 章。

当前六章与规划章的对应关系如下，读者可以按右列判断某一主题会先在哪里出现、又会在哪里被完整展开：

当前章	拆入规划章
第 1 章电平、开关、逻辑门与组合电路	第 1、2 章（数字抽象与电气基础、组合逻辑与布尔代数）
第 2 章状态、时钟与同步边界	第 3 章（时序逻辑、寄存器和状态机）
第 3 章数值、ALU、数据通路与控制器	第 5、7 章（数值表示与算术电路、数据通路与控制器）
第 4 章最小 CPU 与控制 trace	第 6、8、9 章（ISA 与汇编、单周期与多周期、流水线与冒险）
第 5 章主存、总线、I/O 与 DMA	第 10-15 章（cache、虚拟内存、DRAM、总线互连、MMIO/DMA、外设队列）
第 6 章经典芯片、启动与平台演进	第 16、17 章（GPU 与显示、主板固件与平台管理），并吸收第 13 章 PCIe 内容
当前尚未覆盖	第 4 章 RTL 阅读、第 18 章链接装载、第 19 章性能分析、第 20 章综合项目

每章最低质量标准

每一章至少包含五类内容。第一，开头给出本章要解决的硬件问题，避免堆名词。第二，给出一到三张能说明机制的图，图必须区分数据路径、控制路径、状态边界

和错误路径。第三，至少包含一个真实或接近真实的案例，例如 8086 总线、RISC-V 指令、cache miss、NVMe 队列或 GPU fence。第四，章末习题要覆盖概念题、trace 题、边界题和故障定位题。第五，给出参考解答和答题要点，让读者知道答案需要落实到哪些机器细节。

扩写顺序

后续优先扩第 6 到第 15 章，因为这些内容决定本书能不能从“硬件入门讲义”变成“计算机底层教材”。建议顺序是：先补 ISA/汇编和数据通路，再补流水线和异常，然后补 cache/TLB/虚拟内存，接着补 DRAM、PCIe、MMIO/DMA、SSD/网络，最后补 GPU、主板启动、链接装载和性能分析。这样每一轮扩写都能增加一个完整知识块，而不是只增加页数。

目录

扩写目录与路线图	3
第一部分 从电平到数据通路	8
第 1 章 电平、开关、逻辑门与组合电路	10
一、从一根线得到可靠 bit	11
二、从器件到受控电流路径	21
三、从路径表得到逻辑门	32
四、门网络构成组合逻辑	38
五、延迟、负载、毛刺与检查	54
第 2 章 状态、时钟与同步边界	71
一、反馈、锁存器与保存一个 bit	71
二、触发器、时钟周期与采样窗口	76
三、关键路径、复位与跨边界输入	81
四、寄存器、计数器与移位结构	87
五、状态机把硬件动作切成阶段	94
第 3 章 数值、ALU、数据通路与控制	120
一、固定宽度 bit 的数值解释	120
二、从加法器到可复用 ALU	132
三、进位、移位、比较与乘法的硬件取舍	135
四、数据通路让值从旧状态走到新状态	141
五、控制器输出一组一致的控制线	144
第二部分 最小 CPU 与整机硬件	164
第 4 章 最小 CPU 与控制 trace	166
一、从动作编码到控制字组织	166
二、极小 ISA 与可接线数据通路	172
三、可见状态、条件码与调用约定	176
四、异常、中断与精确异常	183
五、整机实验、调试顺序与案例 trace	186
第 5 章 主存、总线、I/O 与 DMA	199
一、主存不是抽象数组	199
二、地址译码定义整机地图	207
三、MMIO 把设备寄存器放进地址空间	209
四、中断和 DMA 把外部事件接回 CPU	214
五、从教学总线到现代互连	217
第 6 章 经典芯片、启动与平台演进	238
一、早期微处理器：从 6502、Z80 到 8086	238
二、80386：32-bit、保护模式与地址转换	245
三、整机启动与经典芯片后的演进	249

四、从目标文件、固件到可启动机器	255
----------------------------	-----

从电平到数据通路

本部分导读

本部分集中回答一个连续问题：怎样从可控电流路径得到稳定的 0 和 1，再把多个 0/1 组合成可计算的逻辑网络；怎样让硬件保存状态、按时钟边界推进；怎样把数值解释、ALU、寄存器阵列、数据通路和控制信号组织成可重复的硬件动作。

电平、开关、逻辑门与组合电路

先从一个会出问题的场景开始。一块控制板上有一个启动按钮：按下时，信号线被接到电源；松开时，信号线哪里都不接。程序读者会以为松开就是 0，但真实电路不是这样——那根悬空的线可能停在旧电压上，也可能被旁边的信号线、人体或漏电流慢慢推走，接收端今天读到 0，明天读到 1。本章就从这类事故出发，一层层回答：一根真实导线上的电压，怎样变成 CPU 可以信赖的 bit 计算。

这条链路分三步展开：连续变化的电压先被解释成稳定的 0/1；二极管和受控开关提供可预测的电流路径；开关网络形成 NOT、NAND、NOR、XOR 等门；门再组合成判断器、加法器、选择器和译码器。读者应始终沿着同一条线索阅读本章：一个逻辑值必须能追溯到电压区间，一个门必须能追溯到上拉/下拉路径，一个组合输出必须能追溯到完整真值表、延迟和负载边界。这些名词现在听不懂没有关系，本章会逐个把它们变成可以检查的东西。

本章不展开完整的半导体物理，也不把内容处理成布尔代数速查表。它关注一个核心问题：一根真实导线上的电压，怎样逐步成为 CPU 可使用的 bit 计算。后续所有寄存器、ALU、数据通路、控制器、内存和 I/O，最终都必须满足这里建立的底层约定。

为避免把内容讲成分散概念，本章使用一个贯穿案例：一块简化的安全启动控制板。它接收启动按钮、保护盖、急停、温度报警和电流报警，输出启动允许、电机使能和故障灯。这个小系统不涉及软件，也不需要处理器；它只需要把输入电平可靠地解释成 bit，再用门网络形成输出。正因为它足够小，读者可以逐项检查每根线的默认状态、每个门的路径、每个输出的电平和每条组合路径的延迟。

信号	含义	需要检查的硬件问题
start_button	人按下启动按钮	松开时是否悬空，按下时是否可靠为 1
cover_closed	保护盖已经闭合	断线或接触不良时应回到安全默认值
emergency_ok	急停没有被按下	信号名为正逻辑，但实物开关可能采用低有效接法
temp_alarm	温度报警	任一报警都应阻止启动并点亮故障灯
current_alarm	电流报警	报警输入不能依赖偶然电平或悬空节点
allow_start	允许启动	必须由所有安全条件共同决定
motor_enable_cmd	电机使能命令	只能由经过检查确认的组合结果产生
fault_lamp	故障灯	需要汇总多个故障信号

表中的每个信号都不是孤立变量，而是一项硬件约定。输入信号要说明来源、默认值、有效极性、断线后果和进入逻辑前的调理方式；输出信号要说明由哪些条件决定、是否允许毛刺、何时被后级使用，以及是否需要驱动执行机构。后文反复回到这张表，是为了让抽象逻辑始终落在可检查的电气边界上。

核心理想

数字硬件的第一层抽象不是门，而是**带余量的电平**；数字硬件的第一层实现不是公式，而是**受控电流路径**。



本章所有例子都沿这条链路逐步检查：先确认电平可靠，再确认路径不悬空、不冲突，最后确认组合输出在后级采样前已经稳定。

图示：从电平到组合逻辑的抽象链。箭头表示“后一个抽象必须由前一个物理事实支撑”。

本章还提前引入几个芯片参照。它们暂时不作为完整处理器来讲，而是作为本章概念的落点：7400 系列把门电路封装成标准逻辑芯片，4000 系列 CMOS 展示互补路径带来的低静态功耗；8086 把门、寄存器、选择器、译码器和总线控制组合成早期完整 CPU；80386 用更多晶体管换来更宽数据通路和更复杂的系统控制；现代 CPU 则靠器件、布局、体系结构和存储层次，把同样的逻辑函数做得更快、更密、更低时延。真正芯片的数据手册还会给出**输入阈值、输出电流、传播延迟和扇出限制**——门不是纸面符号，而是带电气约定的器件。把这些案例提前放在这里，是为了避免读者把第一章误解成“只讲离散门电路”；本章讲的是所有芯片共同依赖的底层约定。

因此，本章的阅读重点不在于快速记住某个门符号，而在于形成一种固定检查法：先确认信号能否成为可靠 bit，再确认器件路径能否稳定地产生这个 bit，最后确认多个 bit 组成的组合网络能否在后级需要之前稳定下来。这个检查法会直接延伸到第二章的触发器、寄存器和状态机。

本章的学习范围与目标 本章保留“从电压到组合网络”的完整链路，是为了让读者先看到底层各层之间的支撑关系，而不是把电气、门级和系统边界割裂为互不相干的名词。阅读时可以把本章看成三层基础：第一层回答一根线怎样成为可靠 bit；第二层回答器件怎样形成受控电流路径；第三层回答多个 bit 怎样组合成可靠的逻辑结果。

这意味着第一章不追求把所有细节一次讲完，而是要求每个抽象都能追溯到具体的物理事实。后面讨论 cache 命中、PCIe 中断、GPU 显存访问或 SSD 控制器状态机时，仍然会回到同一问题：这个信号在电气上是否可靠，在时序上是否被正确采样，在逻辑上是否有完整定义。

读完本章后，读者至少应能掌握四类能力。第一，能为一个板外输入写出默认电平、保护边界、阈值整形和内部语义信号，而不是直接把外部触点当作可靠 bit。第二，能把一个二输入门写成真值表和上拉/下拉路径表，而不是只画 $Y=A \text{ AND } B$ 这类公式。第三，能把一个组合需求分解为门、选择器、译码器、编码器或比较器，并说明最长路径、扇出、毛刺和危险输出处理，而不是只证明稳定真值表正确。第四，能把同一套概念映射到标准逻辑芯片、早期微处理器和现代 CPU 的低时延路径上，而不是只罗列芯片名称。

一、从一根线得到可靠 bit

电路里说“一根线是 1”，完整含义不是“线上写着数字 1”，而是“这个节点相对于参考地的电压落在接收端承认的高电平区间内”。节点可以理解成多条金属连接汇合成的同一个电气位置；参考地是整块电路共同约定的 0V 参考点；电压是两个点之间的差值。数字电路里说某根信号线是 0V、3.3V 或 5V，实际都是在说它相对于地的电压。

电流说明电荷正在沿通路流动。若输出节点被一条通路接向电源，它可能被拉到高电平；若被一条通路接向地，它可能被拉到低电平。若两条通路都断开，节点不会自动变成 0，而是可能暂时停在旧电压附近，再被漏电流、干扰、后级输入或测

量探针慢慢推走。若上拉和下拉两条强通路同时打开，电源到地之间形成冲突电流，电平不可靠，还会增加功耗和发热。

需要先区分导线和程序变量。变量没有被赋值时，语言可以规定默认值；导线没有被驱动时，只剩物理过程。它可能因为寄生电容保存旧电压，表现为保留上一次状态；也可能被旁边切换的信号线、人体接近、探针输入阻抗或后级漏电流轻微影响。调试数字硬件时，不能把这种偶然稳定误认为设计正确。

以按钮输入为例。假设按钮按下表示“请求发生”。一种不完整的接法是按钮一端接信号线，另一端接电源：

```
button pressed -> signal connected to VCC
button released -> signal connected to nothing
```

按下时，信号线被电源路径拉高，该部分行为符合预期。松开时，信号线并没有被明确拉低，它只是断开了。这个节点可能保留先前电荷，也可能被旁边导线耦合出一定电压，还可能被后级输入漏电流拖向某个不确定位置。于是接收端可能读到 1、0 或过渡电压。问题的根源不是程序错误，而是电路没有为节点规定明确归宿。

正确的第一层想法是：任何要被后级判断的输出节点，都必须在每一种输入情况下被明确地拉向高电平或低电平。按钮松开时需要一条弱下拉路径把信号线拉向地；按钮按下时，按钮路径把它拉向电源。弱下拉不是为了在按钮按下时和按钮争夺电源，而是为了在按钮松开时给节点一个默认归宿。它太弱，节点下降慢、抗干扰差；它太强，按钮按下时浪费电流，甚至让高电平不够高。



图示：同一个按钮，两种结局。左图松开按钮后节点没有任何归宿，只能听天由命；右图多了一条弱下拉电阻：松开时它把节点拉向地（可靠 0），按下时按钮的强路径“赢过”弱下拉，把节点拉向电源（可靠 1）。

场景	高电平路径	低电平路径	结论
按下	按钮把节点接向电源	弱下拉仍存在	可靠高
松开	高电平路径断开	弱下拉把节点接向地	可靠低
接触抖动	高电平路径快速断续	弱下拉持续存在	需要消抖
导线断裂	按钮路径永远断开	若下拉还在，则为低	有默认值
没有下拉	高电平路径断开	低电平路径也断开	悬空

该表给出硬件读法的雏形。它不只问逻辑上应该是 0 还是 1，还问路径在哪里、谁驱动谁、异常时会回到哪个默认状态。后面讲复位、总线仲裁和设备中断时，同样要用这种检查法：每个状态都要有明确来源，每个默认值都要有物理路径支持。

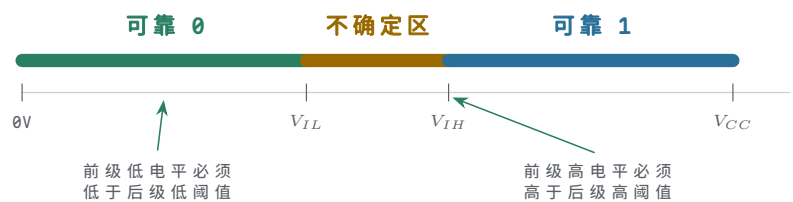
把按钮放回安全启动控制板中，可以得到更明确的安全要求。启动按钮松开时，系统不应因为线缆悬空而误认为有人按下；保护盖传感器断线时，系统不应默认保护盖闭合；报警线断开时，系统不应静默地继续允许运行。因此，“默认值”不是随意选择，而是安全策略的一部分。对人机输入而言，常见策略是让未动作、断线或未知状态对应不启动、不使能或报警一侧；对内部数据通路而言，策略可能不同，但仍必须明确。

输入线	不完整设计的风险	更稳妥的默认方向
启动按钮	松开时悬空，可能误触发	默认不启动，按下才变为启动请求

保护盖检测	断线时可能被误读为闭合	默认未闭合，需要传感器明确证明闭合
急停链路	接触不良时可能丢失急停信号	默认不允许运行，需要链路明确证明急停正常
报警输入	断线时可能丢失报警	根据安全需求选择默认报警或要求上层诊断

电平也不能理解成两个数学点。假设一块小电路用 5V 供电。线上的真实电压可能是 0V、0.17V、1.3V、2.6V、4.8V，也可能在切换过程中从 0V 慢慢升到 5V。硬件不能要求每一级都精确读出这个连续数值。更可靠的做法是规定两个稳定区间：低到足够接近地电位时解释为 0，高到足够接近电源电位时解释为 1，中间一段不作为长期稳定状态。

电压范围	逻辑解释	设计意义
接近地电位	逻辑 0	小噪声不会轻易把它推成 1
中间区域	不作为稳定状态	切换可以经过，但不能长期停留
接近电源电位	逻辑 1	小噪声不会轻易把它拉成 0



图示：电压不是两个数学点，而是带噪声余量的区间。中间区域允许切换经过，不允许作为稳定工作点。

可以用四个电压指标描述两级电路之间的约定。 V_{OH} 是输出端保证的高电平下限， V_{OL} 是输出端保证的低电平上限； V_{IH} 是输入端承认为高的最低电压， V_{IL} 是输入端承认为低的最高电压。若一个电路保证 V_{OH} 高于下一级需要的 V_{IH} ，中间差额就是高电平噪声余量；若 V_{OL} 低于下一级允许的 V_{IL} ，中间差额就是低电平噪声余量。

符号	含义	读法
V_{OH}	输出保证为高时的最差电压	前一级说：我给出的 1 至少高到这里
V_{IH}	输入承认为高的最低电压	后一级说：至少给到这里，我才认作 1
V_{OL}	输出保证为低时的最差电压	前一级说：我给出的 0 至多高到这里
V_{IL}	输入承认为低的最高电压	后一级说：低到这里以下，我才认作 0

例如，某一级输出保证高电平至少为 2.4V，后一级输入只要求达到 2.0V 就承认为高，那么高电平噪声余量为 0.4V。若同一输出在最坏负载、温度和工艺条件下只能到 1.9V，逻辑表达式仍然可能写着“输出为 1”，但后一级不会可靠承认这个 1。低电平也同理：前级输出低电平的最坏值必须低于后一级允许的低电平上限，并留出余量。

项目	前级保证	后级要求	结论
高电平	$V_{OH} \geq 2.4V$	$V_{IH} \geq 2.0V$	余量 0.4V
低电平	$V_{OL} \leq 0.4V$	$V_{IL} \leq 0.8V$	余量 0.4V

失败情形	实测高电平仅 1.9V	后级需要 2.0V	不能可靠读作 1
------	----------------	-----------	----------

把这个例子进一步写成定量算式，会更接近数据手册读法。假设某输出在最坏负载下保证 $V_{OH} = 3.0V$ 、 $V_{OL} = 0.25V$ ，后级输入要求 $V_{IH} = 2.1V$ 、 $V_{IL} = 0.7V$ ，则高电平噪声余量为 $3.0V - 2.1V = 0.9V$ ，低电平噪声余量为 $0.7V - 0.25V = 0.45V$ 。若板级串扰、地弹或线缆压降可能带来约 $0.3V$ 扰动，这个连接仍有余量；若某条长线在最坏情况下可能叠加 $0.6V$ 扰动，低电平余量已经不足，必须改变驱动、布线、阈值或输入调理方案。

检查项	算式	结果	解释
高电平余量	$3.0V - 2.1V$	0.9V	前级高电平高于后级高阈值
低电平余量	$0.7V - 0.25V$	0.45V	前级低电平低于后级低阈值
0.3V 干扰	余量仍为正	可接受	仍需用最坏条件确认
0.6V 干扰	低余量不足	不合格	可能把低电平推入不确定区

数字抽象并不是忽略模拟世界，而是把模拟世界约束在约定之内。只要所有连接都满足这些最差条件，后续讨论 bit、真值表和指令编码才有可靠基础。若这些条件不满足，系统层面可能出现间歇性现象：同一块板有时能启动、有时不能；按键有时触发两次；某条总线在高温下出错；更换更长排线后原本稳定的电路开始误判。这些现象不一定来自逻辑表达式错误，也可能来自电平约定被破坏。

输出还必须能驱动下一级。一个节点如果只能在空载时看起来像 1，一接上后级就被拖低，它不是可靠数字输出。后级输入、板级走线、封装和探针都有寄生电容。把一个节点从 0 拉到 1，本质上是在给这个电容充电；把它从 1 拉到 0，本质上是在放电。驱动路径越弱，等效电阻越大；负载越多，等效电容越大；上升和下降越慢。

larger load capacitance -> slower edge
weaker driver -> slower edge
slower edge -> less time margin before sampling

所以示波器上看到的真实信号不是理想直角。电压边沿有斜率，可能有过冲、振铃、平台和噪声。逻辑 0/1 的背后是一条随时间变化的电压曲线，接收端只是在某个时刻把它归类为 0 或 1。若这个时刻到来之前电压还没有进入可靠区间，最终会稳定也不够。

因此，一个 bit 被称为可靠，至少需要同时满足四项条件。第一，有明确驱动路径，不能悬空、保留旧值或被干扰改变；第二，电平进入接收端承认的稳定区间并留有噪声余量，最坏输出也要满足后级阈值；第三，驱动端能够带动实际负载，不能让边沿被电容拖慢、电压被拖低；第四，在后级采样或使用之前，信号已经稳定足够长时间——最终值正确但采样时刻错误，同样是失败。

外部输入还需要调理。按钮和机械触点会抖动，线缆会引入噪声，传感器输出可能是开漏或开集结构，板外信号还可能与板内电源域不同。第一章暂不展开完整接口电路，但必须建立原则：进入组合逻辑核心之前，外部输入应先变成带默认值、带驱动能力、带明确有效电平的内部信号。否则后面的真值表虽然正确，输入源本身却可能不可靠。

外部现象	进入组合逻辑前的处理	不处理的后果
按钮抖动	消抖或在后续同步边界过滤	一次按下可能产生多次请求

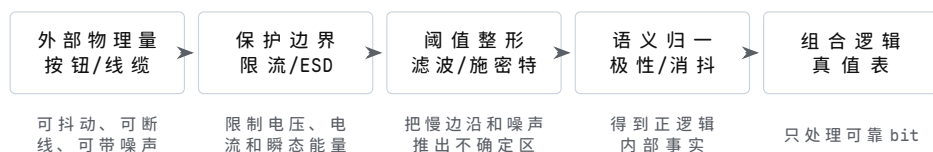
长线噪声	上拉/下拉、滤波、缓冲或屏蔽	误触发、边沿变慢、阈值附近摆动
开漏输出	外部上拉并限制总线负载	释放时没有可靠高电平
电源域不同	电平转换或隔离	后级阈值不匹配甚至损坏器件

板外输入还要有保护边界。按钮、传感器、线缆和连接器会把静电放电、误接电压、浪涌、长线耦合和地电位差带到芯片引脚附近。若把这些信号直接接入 CMOS 输入，逻辑上也许只是“读一个 0/1”，电气上却可能让输入保护二极管承受过大电流，或者让引脚在绝对最大额定值之外工作。保护电路的职责不是改变业务含义，而是在进入逻辑核心之前限制**电压、电流、带宽和边沿形态**。

保护措施	主要作用	设计时必须说明
串联电阻	限制异常电压下的输入电流，减小尖峰冲击	电阻不能大到让边沿过慢或阈值附近停留过久
RC 滤波	抑制低速按钮抖动和高频噪声	时间常数要匹配输入速度，不能吞掉有效脉冲
TVS 或 ESD 保护	为静电和瞬态浪涌提供旁路	器件选型要匹配工作电压、钳位电压和能量等级
输入钳位与限流	让引脚异常电压不直接伤害芯片内部结构	不能依赖片内保护长期承受外部错误连接
缓冲或施密特输入	恢复边沿、提供滞回、隔离前级负载	输出电平、传播延迟和驱动能力仍需逐项确认

因此，板外按钮或传感器不应被写成“直接连到门输入”。更严谨的写法是：连接器输入先经过默认上拉/下拉、限流、保护、滤波或施密特边界，再形成内部语义信号。这样即使后续表达式仍是 `allow_start = start_request AND covered AND emergency_ok AND no_fault`，读者也能看出 `start_request` 已经不是裸露引脚，而是经过电气边界整理后的可靠 bit。

从外部引脚到内部 bit 的五层边界 外部世界进入芯片内部之前，至少要经过五层边界。每一层都回答一个不同问题：连接器和线缆负责把物理事件带到板上；保护与限流负责让异常能量不直接伤害芯片；阈值整形负责把连续电压变成稳定 0/1；语义归一化负责把低有效、反相、断线默认和消抖结果整理成内部名字；后级逻辑只应依赖最后得到的内部语义信号。



图示：外部输入进入组合逻辑前的五层边界。图中的箭头不是简单连线，而是逐层收窄不确定性的过程。

这张图能避免两个常见错误。第一，不能把 `start_button_raw` 直接代入真值表，因为它还包含抖动、噪声、线缆和保护问题。第二，不能把保护电路当作逻辑表达式的一部分，因为保护电路的职责是限制物理风险，而不是决定业务语义。一个严谨的规格应分别写出 `start_button_raw`、`start_button_pin`、`start_button_clean` 和 `start_request`，并说明每个名字跨越了哪一层边界。

上拉电阻和 RC 滤波可以用一个数量级例题理解。若开漏信号采用 $4.7\text{k}\Omega$ 上拉到 3.3V ，低电平被拉下时的电流约为 $3.3\text{V}/4.7\text{k}\Omega \approx 0.7\text{mA}$ ，后级通常较容易承受；若同一节点总电容约 100pF ，则时间常数约为 $4.7\text{k}\Omega \times 100\text{pF} = 0.47\mu\text{s}$ ，低速按钮或报警线通常可以接受。若把上拉改成 $100\text{k}\Omega$ ，同样 100pF 下时间常数变成 $10\mu\text{s}$ ，边沿会明显变慢；若后级很快采样，或者输入没有施密特滞回，信号就可能在阈值附近停留太久。

参数选择	数量级结果	设计含义
4.7kΩ 上拉到 3.3V	拉低电流约 0.7mA	要确认开漏器件能可靠吸收该电流
4.7kΩ 与 100pF	时间常数约 0.47μs	适合低速输入，仍需看采样时刻
100kΩ 与 100pF	时间常数约 10μs	边沿显著变慢，抗噪声能力下降
RC 消抖 10ms	能过滤机械抖动	不适合需要微秒级响应的输入

这些数值不是固定推荐值，而是提醒读者把“能不能读到 1”拆成电流、边沿和时间三个问题。上拉过强会增加拉低电流，上拉过弱会让释放后的高电平上升太慢；RC 过小无法过滤抖动，RC 过大又会吞掉有效边沿。真正设计时应以器件数据手册、输入速度、阈值、功耗和安全响应时间共同决定。

上拉/下拉电阻的阻值权衡 “上拉取 10kΩ”常被当作口诀背下来，但阻值其实是一笔可以算清的账。先看电阻太大的一端。后级输入并非完全不取电流，CMOS 输入存在微安量级以下的漏电流，而这股电流必须流过上拉电阻，在电阻两端形成压降 $V = IR$ ，于是高电平被拉低。以 5V 系统、10kΩ 上拉、输入漏电流 1μA 为例：压降为 $1\mu A \times 10k\Omega = 10mV$ ，实测高电平约 4.99V，几乎无损。若为了“省电”把上拉换成 1MΩ，同样 1μA 就产生 1V 压降，高电平只剩 4.0V；若高温下漏电升到 10μA，压降将达到 10V 量级——节点根本到不了高电平。阻值越大，默认电平越怕漏电，噪声裕量越薄。

再看电阻太小的一端：按键或开漏器件把节点拉低时，上拉电阻直接从电源向地通电。若上拉只有 100Ω，按键按下期间持续流过 $5V/100\Omega = 50mA$ ，电阻上的功耗为 $I^2R = 0.05^2 \times 100 = 0.25W$ ，已超过常见 1/8W 贴片电阻的额定功率，按着不放就会发烫，拉低它的器件也必须灌得进这 50mA。换成 10kΩ，同样动作只消耗 0.5mA，普通逻辑输出和开漏器件都能承受。

但阻值也不能无限大。默认电平靠电阻“拽住”，阻值越大节点越“轻”，同样的耦合电荷产生的电压扰动越大，抗干扰越差；同时电阻与节点电容构成 RC 充电回路，100kΩ 配 50pF 走线电容的时间常数已达 $100k\Omega \times 50pF = 5\mu s$ 。综合起来，阻值由灌/拉电流能力、噪声裕量、功耗三者权衡，常用范围在 1kΩ 到 100kΩ 之间：需要强抗干扰或较快边沿时偏向下端，电池供电、信号很慢时偏向上端。

情形	算式	结果	说明
10kΩ 上拉，漏电 1μA	$1\mu A \times 10k\Omega$	压降 10mV	5V 高电平约 4.99V，裕量几乎无损
1MΩ 上拉，漏电 1μA	$1\mu A \times 1M\Omega$	压降 1V	高电平只剩 4.0V，高温下更差
100Ω 上拉，按键接地	$5V/100\Omega, I^2R$	50mA、0.25W	超过 1/8W 电阻额定，按着不放会发烫
10kΩ 上拉，按键接地	$5V/10k\Omega$	0.5mA	功耗可忽略，开漏器件也灌得起
100kΩ 上拉，节点 50pF	$100k\Omega \times 50pF$	$\tau = 5\mu s$	边沿明显变慢，抗扰动力下降

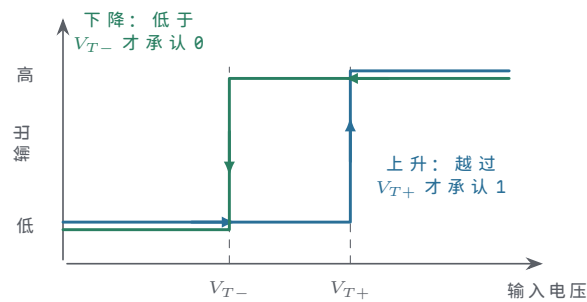
这张表也说明为什么“先看数据手册再定阻值”：输入漏电、开漏器件的灌电流能力和节点电容分别决定阻值的上限和下限，口诀只是在常见参数下的折中。

由此可见，书写硬件规格时应区分“原始外部信号”和“内部逻辑信号”。原始外部信号属于板外世界，可能带有抖动、噪声、线缆压降和接插件故障；内部逻辑信号属于组合逻辑世界，必须已经具有明确的 0/1 含义。若把二者混成一个名字，读者会误以为真值表直接约束了外部物理现象。

信号层次	典型名称	设计要求
------	------	------

外部触点	<code>start_button_raw</code>	允许抖动，但必须有默认路径和保护边界
输入调理后	<code>start_button_clean</code>	在逻辑核心看来只能是稳定 0 或稳定 1
语义信号	<code>start_request</code>	明确表示“本周期有启动请求”这一逻辑事实
安全汇总	<code>allow_start</code>	由全部安全条件共同决定，不能依赖原始触点

施密特触发输入是常见的边界处理方式。普通输入通常只有一个判断阈值附近的过渡区；当信号边沿很慢或带噪声时，电压可能在阈值附近反复穿越，导致接收端多次翻转。施密特触发器使用两个不同阈值：上升时必须超过较高阈值才承认为 1，下降时必须低于较低阈值才承认为 0。两个阈值之间的间隔称为滞回。滞回不是改变逻辑功能，而是让输入在噪声环境中更稳定地跨过边界。

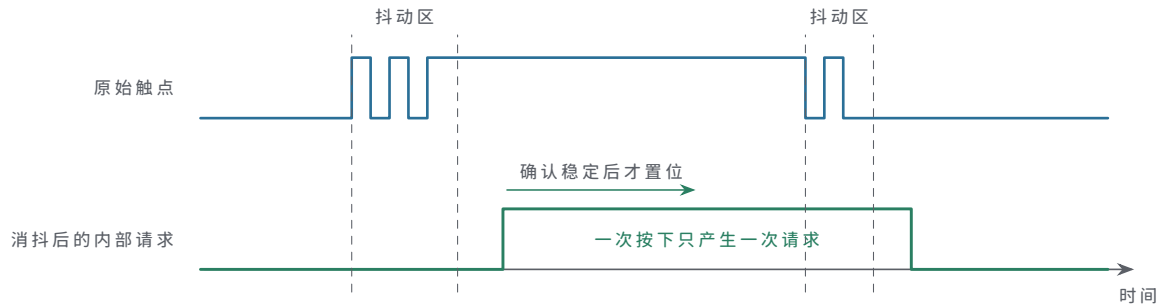


图示：施密特输入的传输特性曲线。普通输入只有一个模糊阈值，噪声会让输出在阈值附近反复横跳；施密特把“上升承认”和“下降承认”拉开成两个阈值，中间这段滞回区间就是噪声的容身之地——信号必须真正越过 V_{T+} 才算 1，真正跌回 V_{T-} 以下才算 0。

输入处理	适用情形	对可靠 bit 的贡献
上拉或下拉	按钮、开漏输出、断线默认值	给悬空节点明确归宿
限流和保护	板外连接、误插、瞬态干扰	避免异常电压直接进入逻辑核心
滤波或消抖	机械触点、低速传感器	把多次物理跳变收敛为一次逻辑事件
施密特触发	慢边沿、噪声叠加的输入	用滞回减少阈值附近的反复翻转
电平转换	1.8V、3.3V、5V 等混合系统	让输出约定匹配输入约定

按钮消抖尤其适合作为入门反例。人只按下一次，触点却可能在几毫秒内多次接通、断开。如果后级把每一次跳变都看成一次请求，一个启动按钮就可能产生多个启动脉冲。消抖电路或后续时序逻辑的职责，是在确认输入持续稳定后才更新内部请求信号。

时间段	原始触点	内部请求	解释
按下前	稳定断开	0	默认下拉给出明确低电平
刚按下	断续跳变	保持 0	尚未接受为一次稳定请求
稳定按下	持续接通	1	已满足消抖条件
刚松开	断续跳变	保持 1 或等待释放确认	不把抖动解释为多次按键
稳定松开	持续断开	0	回到默认状态



图示：人只按了一次，触点却在几毫秒内多次通断（抖动区）。消抖的逻辑是“连续稳定一段时间才算数”：内部请求比真实按下晚一点点，代价可忽略，换来的是它只跳变一次。

这组处理说明一个原则：逻辑核心应当面对已经被整理过的事实，而不是直接面对物理噪声。安全启动控制板中，`allow_start` 不应直接依赖 `start_button_raw`，而应依赖 `start_request`；故障汇总不应直接依赖线缆上的未定义电平，而应依赖经过默认值、阈值和驱动能力检查后的报警信号。后面进入触发器和时钟边界后，还会继续把这种“边界整理”推广到同步采样。

可靠 bit 还必须考虑上电和复位。电源从 0V 上升到额定电压需要时间，芯片内部阈值、外部上拉下拉、时钟源和输入调理电路不一定同时进入正常状态。在这段时间里，若某个输出直接控制电机、继电器或写使能，就不能假设它已经是安全值。正式硬件设计通常要求复位链路或电源监控电路在电源稳定之前保持关键输出禁止。

阶段	硬件状态	对关键输出的要求
断电	节点可能通过漏电或外部连接获得微弱电压	不应驱动执行机构
电源爬升	阈值、时钟和输入调理尚未完全有效	复位或禁止链路保持有效
复位保持	内部状态被强制到约定默认值	<code>motor_enable_cmd=0</code> ，写使能关闭
复位释放	输入和时钟达到有效条件	组合逻辑开始按规格产生输出
正常运行	所有电平约定和时序约定成立	后级只使用已经稳定的信号

这也是“默认安全”的电气含义。它不是在文档里写一句“默认不启动”，而是要能指出：电源未稳时谁拉住使能，复位未释放时谁关闭输出，输入断线时谁把条件解释为不满足，组合路径未稳定时谁阻止后级采样。若这些问题回答不出来，系统可能在最危险的上电瞬间出现短促误动作。

最后，可靠 bit 还要能被实际测量确认。万用表可以检查静态电压，却看不见窄脉冲和抖动；示波器能观察边沿、过冲、振铃和噪声；逻辑分析仪适合记录长时间数字事件，但它同样依据自己的阈值把模拟电压解释成 0/1。测量工具不是抽象之外的附属品，而是把电平约定落实到实物上的手段。

工具或手段	能说明什么	不能单独证明什么
原理图审查	是否有默认路径、保护和驱动边界	实物焊接和噪声环境是否合格
万用表	静态高低电平是否大致正确	边沿速度、毛刺和短时抖动
示波器	波形、阈值穿越、过冲和振铃	长时间低概率事件一定不会发生
逻辑分析仪	数字事件顺序和持续时间	模拟电平余量是否足够
数据手册参数	最坏阈值、延迟、负载能力	板级实现是否满足这些边界

同一根按钮输入线，可以用三类工具得到互补的测量结果。假设 `start_button`

`_raw` 采用上拉输入，按下时接地，内部逻辑再把它反相为 `start_request`。万用表能确认松开和按下时的静态电压方向；示波器能观察按下瞬间的触点抖动、边沿速度和阈值穿越；逻辑分析仪能记录内部请求是否只产生一次事件。三者关注的问题不同，不能互相替代。

测量手段	在按钮案例中应看到什么	若结果异常应怀疑什么
万用表	松开为稳定高电平，按下为稳定低电平	上拉/下拉错误、接线反向、触点失效
示波器	按下和松开有有限抖动，最终越过阈值并稳定	抖动过长、边沿太慢、噪声跨越阈值
逻辑分析仪	消抖后只出现一次 <code>start_request</code>	消抖不足、阈值设置不当、采样边界错误
数据手册	输入阈值、滞回、电流和最大额定值满足设计	电平约定只在实验样品上偶然成立

把这类测量结果写入设计文档后，第一章的抽象就不再停留在“按钮产生 0/1”。更完整的说法是：外部触点通过默认路径得到静态归宿，通过输入调理穿过阈值，通过消抖变成一次语义事件，并在后级使用前满足电平和时间条件。后面的内存读写、总线请求和中断输入，也应按同样方式处理边界。

万用表与示波器各自能看见什么 万用表直流电压档测出的是一段时期的平均值。静态节点上，松开约 5V、按下约 0V，读数直接可信；但一个 0V/5V、占空比 50% 的时钟信号，直流档读数约为 2.5V——既不是 0 也不是 1，无法据此判断波形好坏。窄毛刺、触点几毫秒的抖动、一次几纳秒的过冲，在平均值里几乎不留痕迹。所以万用表适合回答“默认电平对不对、供电对不对”，不适合回答“信号干不干净”。

示波器把电压随时间的曲线显示出来，才能看到边沿时间、过冲、振铃和阈值穿越次数。读边沿时要看 10% 到 90% 电平之间的行程时间，而不是只看最终稳定值；按钮消抖是否成功、复位释放是否单调、开漏上拉是否太慢，都要在波形上确认。两类工具回答的是不同问题，不能互相替代。

还要记住：测量仪器接入后本身就是被测电路的一部分。万用表和示波器的输入电阻通常约为 $10M\Omega$ ，去测一个靠 $1M\Omega$ 上拉维持的节点，相当于给节点并联了一个 $10M\Omega$ 电阻，分压后读数约为 $5V \times 10M\Omega / (1M\Omega + 10M\Omega) \approx 4.55V$ ——比真实值低了约 0.45V，可能把一个勉强合格的高电平误判成故障。示波器探头还带有电容： $\times 10$ 探头约 10 到 15pF， $\times 1$ 探头可达上百 pF；把 15pF 加到 $100k\Omega$ 上拉的节点上，时间常数就多了 $100k\Omega \times 15pF = 1.5\mu s$ ，边沿会被明显拖慢。测量会改变被测对象，在高阻、高速节点上尤其明显。

工具	能读出的内容	读不出或会引入的问题
万用表直流档	静态高低电平、供电电压、慢变化平均值	看不到毛刺、抖动和边沿，快速波形只剩平均值
示波器	边沿时间、过冲、振铃、阈值穿越、瞬态过程	探头电容与输入电阻会改变高阻节点的读数
测量行为本身	$10M\Omega$ 级输入电阻、pF 级探头电容	高阻节点被分压，边沿被附加电容拖慢

因此读数之前要先问两个问题：仪器接入是否改变了这个节点，以及读数代表的是哪一段时间内的电压。

还要补上板级物理边界。数字信号不是只沿着“信号线”单独传播；每一次电流变化都需要回流路径，供电和地也有电阻、电感和寄生电容。若多个输出同时翻转，瞬时电流会让地线或电源线上出现小幅电压变化，表现为地弹、供电跌落或参考点移动。若两根长线靠得太近，快速边沿还可能通过寄生电容或电感耦合到邻线，表现为串扰。它们不会改变布尔表达式，却会改变接收端看到的真实电压。

板级现象	物理来源	对可靠 bit 的影响
回流路径过长	信号电流的返回路径绕远或不连续	边沿振铃、辐射增加、邻线耦合增强
地弹	多个输出同时切换，地线上产生瞬时压降	接收端参考地移动，噪声余量被压缩
供电跌落	瞬时电流超过局部供电支撑能力	输出驱动变弱，边沿变慢或阈值判断不稳
串扰	相邻走线之间存在寄生电容和互感	静止信号上出现错误脉冲或阈值附近扰动
长线振铃	走线具有传输线特性且终端不匹配	接收端可能多次穿越阈值
去耦不足	芯片附近缺少局部电荷支撑	切换时电源/地噪声增大

因此，电平约定至少隐含三类物理条件。第一，信号源和接收端必须共享可接受的参考地，或者通过隔离/差分/电平转换重新定义边界。第二，供电必须能在切换瞬间提供局部电流，去耦电容的职责就是在芯片附近提供短时间电荷支撑。第三，长线和外部连接器不能只按“理想导线”理解，必要时要考虑终端、屏蔽、滤波、缓冲和同步采样。第一章不展开 PCB 设计，但必须让读者知道：真值表正确并不自动保证板级物理边界正确。

输入模型和输出模型：引脚不是理想变量 为了把原理讲清楚，可以把每个数字引脚暂时看成一个等效模型。输入端不是完全不取电流的数学变量，它有输入漏电、输入电容、阈值和绝对最大额定值；输出端也不是理想电压源，它有等效导通电阻、最大源电流/灌电流、传播延迟和短路风险。把这些模型放进脑中，很多“逻辑式没错但板子不稳”的问题就能被提前解释。

引脚模型	关键参数	影响的问题
输入漏电	输入端即使不主动驱动，也可能有微小电流	上拉/下拉过弱时，默认电平可能被漏电和噪声改变
输入电容	输入门电容、封装和走线电容共同形成负载	边沿速度、RC 时间常数和阈值穿越时间
输入阈值	V_{IH} 、 V_{IL} 、滞回和电源域	电平余量、慢边沿误触发和电平转换需求
输出电阻	上拉/下拉晶体管导通后仍有等效电阻	负载越重，输出电平越偏离理想电源或地
输出电流	源电流、灌电流和总封装电流有限	扇出、LED/继电器驱动、总线冲突和发热
输出延迟	输入变化到输出越过阈值需要时间	组合路径最长延迟、毛刺宽度和采样窗口

因此，扇出不是“一个输出能画多少根线”的绘图问题，而是负载、电流和延迟问题。一个输出同时驱动十个 CMOS 输入，静态电流也许很小，但输入电容会叠加，边沿会变慢；同一个输出若还要驱动 LED、长线或继电器线圈，就已经超出普通逻辑门职责，通常需要缓冲器、晶体管、MOSFET、驱动芯片或隔离器。工程文档应明确区分“逻辑输出”和“功率驱动输出”。

连接方式	不能只看逻辑式的原因	更稳妥的检查问题
一个门驱动多个输入	负载电容叠加，边沿变慢	最坏负载下是否按时越过阈值
逻辑门直接点亮 LED	LED 电流可能超过输出额定值	是否需要限流、电流预算和专用驱动

普通输出接长线	长线可能振铃、串扰或引入地电位差	是否需要缓冲、终端、滤波或差分传输
两个输出并接	一高一低时会形成强冲突电流	是否改用开漏、三态仲裁或总线协议
跨电源域连接	阈值和最大额定值可能不匹配	是否需要电平转换或隔离

读数据手册时，也应把这些模型翻译成检查清单：输入端看阈值、漏电、电容和最大额定值；输出端看高低电平保证、驱动电流、传播延迟、上升下降时间和总功耗限制。只要这张清单没有填完，布尔式就还不能直接等同于可工作的硬件。

二、从器件到受控电流路径

二极管可以先理解成方向性很强的器件。在合适偏置下，它比较容易让电流沿一个方向通过；反向偏置时，它基本阻止电流通过。这个模型不需要一开始展开半导体物理，却已经足够说明硬件设计的一条基本路线：利用器件物理特性，制造可预测的电流路径。

二极管的“单向”不是理想数学开关。正向导通通常需要一定压差，导通后还有压降；反向并非绝对没有电流，而是漏电很小；电压过高时还可能击穿。初读数字硬件时可以先用理想模型建立方向感，但不能忘记真实器件会吃掉电压余量、引入延迟和功耗。

把数量级写下来，后面的讨论才有抓手。常见硅二极管的正向压降约为 0.6V 到 0.7V，锗二极管约为 0.2V 到 0.3V，肖特基二极管约为 0.2V 到 0.4V；反向漏电流通常在微安量级以下，普通小功率管的反向耐压从几十伏到几百伏不等。本章估算一律取硅管 0.7V：一路 5V 信号经过一只二极管后，高电平只剩约 4.3V；再串一级，就只剩约 3.6V。若后级输入高阈值是 2.0V，单看一级还剩 2.3V 余量，串到第三级时余量只剩 1.6V——这就是“级联多了电平余量越来越紧”的定量含义。

级数	输出高电平（输入 5V）	距 2.0V 阈值的余量
0 级（直连）	5.0V	3.0V
1 级二极管	$5.0 - 0.7 = 4.3V$	2.3V
2 级二极管	$4.3 - 0.7 = 3.6V$	1.6V
3 级二极管	$3.6 - 0.7 = 2.9V$	0.9V

二极管事实	对数字电路的影响	读书时要追问
有方向性	可以限制电流路径	这条路径在何种输入下导通
有正向压降	输出电平会损失一部分余量	级联后高电平还够不够高
有漏电和极限	不能把反向阻断看成无限好	极端温度、电压下是否仍可靠
不能主动放大	不能恢复变弱的电平	下一级是否需要缓冲或开关恢复

整流、钳位与续流：二极管的三个工程角色 在把二极管用进逻辑之前，先看它在电源与保护中的三个更常见的工程角色。这三种用法都只依赖“单向导通”这一条性质，解决的却是三类完全不同的事故。

第一个角色是整流：把交变或极性不定的电压整理成单向电流。接法上，可以单管串联做半波整流，也可以用四只二极管组成桥式全波整流，再配合电容和稳压得到直流。电源入口串一只二极管同时也是最低成本的防反接：电源装反时它不导通，电路只是不工作，而不是烧毁。失效后果是直接的一整流二极管接反或击穿短路后，反向电压或交流分量直接加到直流母线上，后级电解电容和稳压器可能损坏。

第二个角色是钳位保护：把引脚电压限制在电源轨附近。接法是用两只二极管从信号引脚分别“反并”到电源和地：引脚电压高于 $V_{CC} + 0.7V$ 时上管导通，把电流引

入电源轨；低于 $-0.7V$ 时下管导通。引脚因此被钳在电源轨上下各一个二极管压降的范围之内。芯片引脚内部的 ESD 保护正是这种结构。没有钳位时，人体静电动辄几千伏，一次触碰就可能击穿片内结构；但钳位也不是无限保险——它只为瞬态小能量设计，若把 12V 长期误接到 5V 引脚上，持续大电流会先烧掉钳位二极管，再烧掉芯片。

第三个角色是续流：给电感电流留一条退路。电感电流不能突变。继电器线圈由晶体管驱动，导通时电流 100mA；晶体管约在 $1\mu s$ 内关断，线圈仍要维持电流，两端感应电压可按 $V = L \cdot \Delta I / \Delta t$ 估算：10mH 电感在 $1\mu s$ 内切断 100mA，理想估算达 $10mH \times 100mA / 1\mu s = 1000V$ ，实际虽被寄生电容和器件击穿限制，仍常达几十到上百伏，远超小信号晶体管的耐压。接法是在线圈两端反并一只二极管：正常供电时它反偏不工作，关断瞬间线圈电流改经二极管环流、慢慢衰减，反峰被钳在约 0.7V。没有续流二极管的继电器驱动，常见现象是晶体管工作一段时间后“莫名其妙”损坏。

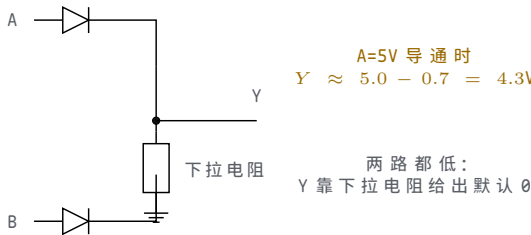
角色	典型接法	缺少它的失效后果
整流与防反接	单管串联或四管桥，串在电源入口	反向或交变电压直达直流母线，电容与稳压器受损
钳位保护	两只二极管把引脚反并到电源轨和地	静电与瞬态直接打入片内结构；长期过压同样会烧毁钳位
续流	与继电器或电感线圈反并联	关断反峰击穿驱动晶体管，表现为多次动作后失效

这三个角色都不参与逻辑判断，却决定了逻辑电路能否在真实电源和真实负载下长期存活。

二极管逻辑可以表达某些简单关系。若有两路输入想共同点亮一个指示节点，任意一路为高时都能通过二极管把输出节点拉高，输出就呈现 OR 行为：

```
if A can drive through a diode:
    Y may be pulled high
if B can drive through a diode:
    Y may be pulled high
```

该例能够说明方向性逻辑，但也暴露出局限。第一，二极管会带来压降，输出高电平可能比输入高电平低一截；级联多了，电平余量会越来越紧。第二，二极管只能提供方向性，不能主动把一个偏弱的高电平重新恢复成强高电平。第三，它不方便实现取反，而取反是构造通用逻辑必须具备的能力。



图示：二极管 OR 的真实样子：任意一路为高，该路二极管导通，把 Y 拉高——但 Y 比输入低一个正向压降（硅管约 0.7V）；两路都低时没有驱动源，靠下拉电阻给出默认 0。方向性由二极管保证，电平却不会被恢复：这就是它不能无限级联的原因。

以安全启动控制板的故障汇总为例，`temp_alarm` 和 `current_alarm` 都可能点亮 `fault_lamp`。二极管 OR 可以让任一报警输入把故障节点拉高，但如果这个故障节点还要继续驱动后面的禁止逻辑、记录逻辑和指示灯，问题就会扩大。每经过一级二极管，电平余量都可能被吃掉一部分；每增加一个负载，边沿都会变慢。最终出现的不是“公式错了”，而是高电平不够高、上升不够快、后级读不稳。

二极管 OR 用法	可以解决的问题	不能解决的问题
-----------	---------	---------

汇总多个报警源	任一报警能提供一条拉高路径	不能主动恢复被削弱的高电平
隔离多个输入源	一个输入不容易反向灌入另一个输入	正向压降会消耗电平余量
形成简单有线关系	结构直观，适合早期理解	不适合无限级联成通用逻辑网络

取反为什么重要？因为许多条件天然需要反向解释。传感器未触发时允许运行，触发时禁止运行；按钮未按下时保持，按下时改变。若电路只能表达“有一路导通”，却能把 1 变成 0、把 0 变成 1，那么可组合的逻辑很快就受限。要继续向前，需要一个信号能控制另一条电流路径是否导通。

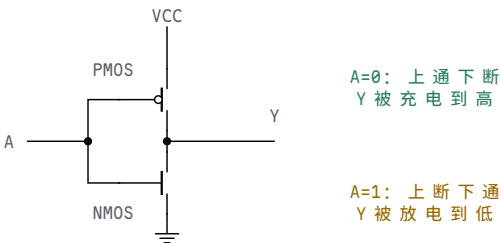
晶体管的细节很深，但作为第一层硬件模型，可以先把它当成受控开关。控制端不是直接给输出供电，而是决定输出节点到某条电源路径之间是否接通。受控开关和普通机械开关最大的差别，是控制信号本身也是电信号。按钮需要人按，晶体管可以由前一级逻辑输出控制。于是硬件有了“信号控制信号”的递归能力：一个门的输出可以控制下一批开关，下一批开关再形成新的输出。

在 CMOS 数字电路中，可以先用极简模型区分两类受控开关。NMOS 更适合把节点拉向地，输入控制为高时容易导通；PMOS 更适合把节点拉向电源，输入控制为低时容易导通。真实器件还有阈值电压、体效应、漏电、沟道电阻和寄生电容等细节，但第一章暂时只抓住一件事：上拉路径和下拉路径由互补器件协作，才能把输出恢复为强 0 或强 1。

器件视角	在门电路中的典型职责	需要避免的误解
NMOS 下拉	给输出提供到地的受控路径	不是“产生 0”这个符号，而是放电路径
PMOS 上拉	给输出提供到电源的受控路径	不是“产生 1”这个符号，而是充电路径
互补配对	一个负责拉低，另一个负责拉高	不能让两条强路径长期同时导通
输入过渡	两类器件可能都部分导通	过渡区只应短暂经过，不应长期停留

反相器可以作为第一种完整逻辑门来理解。输入低时，上拉路径导通、下拉路径断开，输出被拉高；输入高时，上拉路径断开、下拉路径导通，输出被拉低。注意这里不是“输入低所以输出自动高”，而是“输入低使某个高电平路径导通”。把每一句逻辑描述翻译成路径描述，能避免很多硬件误解。

输入	上拉路径	下拉路径	输出
低电平	导通	断开	被拉高
高电平	断开	导通	被拉低
过渡中	可能部分导通	可能部分导通	暂时不应采样



图示：反相器的开关模型。输入 A 同时接到两个控制端：PMOS 控制端带小圆圈，表示“输入低才导通”；NMOS 没有圆圈，表示“输入高才导通”。这就是 NOT 的全部——但它同时完成了二极管做不到的事：不管输入边沿多差，输出都被

重新驱动成强 0 或强 1，电平被恢复了。

把这张路径表压缩成真值表，就是 NOT 的全部定义：

A	Y	路径解释
0	1	上拉导通，输出被拉到高
1	0	下拉导通，输出被拉到低

第三行很重要。真实输入从低到高不是瞬间跳变，在过渡区间里，上拉和下拉可能都不处于理想开/关状态。短时间内出现电源到地的电流并不奇怪，这就是动态功耗和短路功耗的一部分。只要过渡足够快、采样避开中间区域，数字抽象仍然成立。若输入边沿太慢，电路可能在中间区域停留过久，功耗和误判风险都会上升。

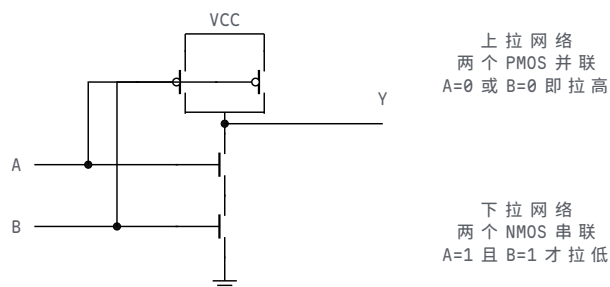
把开关串联，可以表达“两个条件都满足才有通路”；把开关并联，可以表达“任意一个条件满足就有通路”。以下拉网络为例，两个受控开关串联，只有 A 和 B 都让开关导通时，输出节点才会被拉向低电平；两个开关并联，只要 A 或 B 有一个导通，输出节点就会被拉低。上拉网络通常要和下拉网络互补，保证输出在稳定输入下要么被拉高，要么被拉低，而不是两边都断或两边都强通。这种“互补路径”思想，后面会自然走到 CMOS 逻辑。

互补路径可以用两条规则记住。第一，输出应该被明确驱动：下拉网络不导通时，上拉网络应当负责把输出拉高；下拉网络导通时，上拉网络应当关闭，避免电源到地的强冲突。第二，稳定输入下不应依赖中间电压：如果上拉和下拉都处于部分导通状态，输出可能落在不可靠区间，功耗也会上升。CMOS 逻辑之所以适合作为数字门的基础，关键就在于它把“拉高”和“拉低”组织成互补关系，并在稳定状态下尽量避免静态直通电流。

路径状态	输出后果	设计判断
上拉开、下拉关	输出被拉向高电平	合法稳定状态
上拉关、下拉开	输出被拉向低电平	合法稳定状态
上拉关、下拉关	输出悬空，可能保留旧电压	需要补默认路径或改逻辑
上拉开、下拉开	电源到地形成强通路	功耗高，电平不可靠
两边部分导通	输出可能处于过渡区	只允许短时间经过，不应长期停留

把这条规则应用到 NAND 和 NOR，可以看到 CMOS 门并不是把 AND、OR 符号直接画在硅片上，而是把上拉网络 and 下拉网络组织成互补关系。二输入 NAND 的下拉网络由两个 NMOS 串联：只有 A、B 都为 1 时，输出才被拉向地；它的上拉网络由两个 PMOS 并联：只要 A 或 B 有一个为 0，就有一条上拉路径把输出拉高。二输入 NOR 则相反：下拉网络由两个 NMOS 并联，只要 A 或 B 为 1 就能拉低；上拉网络由两个 PMOS 串联，只有 A、B 都为 0 时才拉高。

CMOS 门	下拉网络	上拉网络
NAND	NMOS 串联，A=1 且 B=1 时完整导通	PMOS 并联，A=0 或 B=0 时至少一路导通
NOR	NMOS 并联，A=1 或 B=1 时至少一路导通	PMOS 串联，A=0 且 B=0 时完整导通



图示：二输入 NAND 的晶体管级结构。输出为 0 的唯一情形是 $A=B=1$ ：两个串联 NMOS 同时导通，下拉路径完整；其余三行至少一只 PMOS 导通，上拉把输出充到高。真值表的每一行都能在这张图上找到对应的路径状态。（输入 B 的连线跨过 A 的线但不连接：没有连接点的交叉不算接通。）

这张表把逻辑表和器件路径接在一起。NAND 输出为 0 的唯一稳定条件，对应“下拉串联路径完整”；NAND 输出为 1 的三个稳定条件，对应“至少一条上拉路径存在”。NOR 输出为 1 的唯一稳定条件，对应“上拉串联路径完整”；NOR 输出为 0 的三个稳定条件，对应“至少一条下拉路径存在”。如果只背真值表，读者很难解释为什么 NAND/NOR 常成为门级实现的基础；如果只看器件路径，又容易失去逻辑函数的目的。两者必须同时保留。

CSAPP 一类系统教材常把“布尔函数”和“硬件实现”分层书写：先说明抽象函数，再说明门级和时序代价。这里的 CMOS NAND/NOR 正好提供一个小型范例。稳定逻辑值来自互补路径，传播延迟来自器件和连线，功耗来自节点切换和漏电；同一个门符号背后同时存在功能、时间和能量三个维度。

从功耗角度看，数字门并非“只有 0 和 1，没有模拟代价”。稳定状态下，如果只有上拉或只有下拉导通，静态电流可以很小；切换时，输出节点的寄生电容需要充电或放电，形成动态功耗；输入经过过渡区时，上拉和下拉可能短时间同时导通，形成短路功耗。后面讨论频率、负载和关键路径时，这些现象会继续出现。

这也是 CMOS 成为主流数字逻辑基础的重要原因。它不是没有功耗，而是在稳定 0 和稳定 1 时尽量避免持续直通电流，把主要能量消耗集中到节点切换和短暂过渡中。随着芯片集成度提高，设计者更关心哪些节点频繁翻转、哪些长线负载过大、哪些门需要更强驱动，以及哪些路径应由寄存器边界切分。由此可见，第一章的受控路径模型会直接延伸到后面的频率、功耗和时序预算。

功耗来源	物理原因	设计含义
静态功耗	漏电或稳定状态下仍存在电流路径	器件越小越不能忽略漏电和待机功耗
动态功耗	输出节点和连线电容反复充放电	高翻转率、长线和大扇出都会增加能耗
短路功耗	输入过渡时上拉和下拉短暂同时导通	慢边沿不仅影响时序，也会增加功耗
驱动损耗	输出级推动外部负载或长线	指示灯、总线和连接器常需要专用驱动级

BJT 与 MOSFET 开关对照 本章的受控开关主要以 MOS 管出现，但值得把它和另一类器件——双极型晶体管（BJT）——放在一起对照，因为两者的差别正好解释了现代数字电路的工艺选择。最核心的差别在控制端：BJT 是电流驱动器件，集电极电流要靠持续的基极电流维持，工程估算常取 $I_B \geq I_C/\beta$ ， β 约几十到几百；MOSFET 是电压驱动器件，栅极与沟道之间隔着绝缘氧化层，稳定状态下几乎没有栅极电流，能量只在翻转瞬间用于给栅电容充放电。

作开关使用时，两者的导通损耗来源也不同。BJT 在截止与饱和之间切换，导通

时呈现近似恒定的饱和压降 $V_{CE(sat)}$ (约 $0.2V$)，损耗为压降乘以电流；MOSFET 在夹断与导通之间切换，导通时表现为沟道电阻 $R_{DS(on)}$ ，损耗按 I^2R 计算。算一笔账：驱动一个 $100mA$ 的继电器，BJT 取 $\beta = 100$ ，基极要持续供出约 $1mA$ ，导通损耗约 $0.2V \times 100mA = 20mW$ ；换用 $R_{DS(on)} = 50m\Omega$ 的 MOSFET，损耗只有 $0.1^2 \times 0.05 = 0.5mW$ ，且栅极在稳定期间几乎不取电流。

对比项	BJT	MOSFET
驱动方式	电流驱动，基极需持续电流	电压驱动，栅极只取充放电电流
开关状态	截止与饱和	夹断与导通
导通损耗来源	$V_{CE(sat)} \times I$ ，近似恒定压降	$I^2 \times R_{DS(on)}$ ，随电流平方上升
100mA 驱动例	基极约 $1mA$ ，损耗约 $20mW$	栅极静态近零，损耗约 $0.5mW$ ($50m\Omega$)
静态功耗 仍活跃の場合	基极电流贯穿整个导通期 大电流驱动、线性放大、分立高压	稳定状态下几乎为零 数字集成与功率开关的主流

由此可以理解为什么现代数字电路主流是 MOS。数字芯片里有数十亿个开关，若每个开关的控制端都要持续电流，静态功耗将完全无法接受；MOS 栅极不取静态电流，NMOS 与 PMOS 两种极性又容易在同一工艺中做成互补对，器件尺寸还能持续缩小——静态功耗和集成密度因此都站在 MOS 一边。BJT 并未消失：在大电流驱动、线性放大和某些高压分立场合，它的线性区特性和电流处理能力仍然合适，只是不再是构成通用数字逻辑的器件。

开漏或开集结构是理解共享线路的重要例子。一个输出器件只负责把线拉低，释放时并不主动拉高；高电平由公共上拉电阻提供。多个器件可以共享同一根线，只要任意一个器件拉低，整根线就是低电平；只有所有器件都释放，线上才被上拉为高电平。这种结构常用于中断请求、故障汇总或简单总线。

共享线状态	物理路径	逻辑后果
所有输出释放	上拉电阻把线拉高	线为 1，但上升速度取决于上拉和负载
任一输出拉低	该输出提供到地路径	线为 0，多个输出不会互相强冲突
上拉过弱 上拉过强	高电平上升慢 拉低时电流大	采样前可能尚未达到高阈值 功耗增加，输出器件负担加重

这说明“多个来源接到一根线”并非永远错误，关键在于是否有协议和电气结构保证同一时刻不会出现强驱动冲突。开漏共享线允许多个设备共同拉低，是因为它们只竞争“是否提供到地路径”，不竞争“谁主动拉高”。普通推挽输出若直接并在一起，一个输出拉高、另一个输出拉低，就会形成强冲突。

三态输出是另一类共享线路方案。普通数字输出只有主动拉高和主动拉低两种驱动状态；三态输出还可以进入高阻态。高阻态不是逻辑 0，也不是逻辑 1，而是输出驱动器基本断开，让这一路暂时不参与驱动总线。若多块设备共享数据总线，控制逻辑必须保证同一时刻最多只有一个设备打开输出驱动，其余设备都处于高阻态。否则，一个设备驱动 1、另一个设备驱动 0，仍会发生总线争用。

输出方式	驱动状态	适合表达的问题
推挽输出	主动拉高或主动拉低	单一来源驱动一条内部信号
开漏输出	只主动拉低，释放靠上拉	多个来源共同提出低有效请求

三态输出 拉高、拉低或高阻 多个来源按仲裁结果轮流占用总线

开漏和三态都要求协议。开漏协议通常是“任一设备可以拉低，所有设备释放后才为高”；三态协议通常是“仲裁者只允许一个设备驱动，其他设备必须释放”。这两类结构后来会在中断线、片选、数据总线和 I/O 控制中反复出现。现在先记住硬件判断标准：多来源共享一根线时，必须说明每个来源什么时候能驱动、什么时候必须释放，以及释放后由谁给出默认电平。

把这些器件观点收束起来，可以得到本章的第二层抽象：数字门不是画在纸上的符号，而是一组受输入控制的上拉、下拉、释放和恢复路径。二极管提供方向性，但不恢复电平；晶体管提供受控路径；互补开关网络提供明确的高低驱动；缓冲和三态驱动器提供负载隔离和共享边界。后面所有更大的组合电路，仍然要回到这些路径检查。

器件族之间还存在电平兼容问题。即使两个器件都叫“数字逻辑”，也不表示它们可以任意直连。一个 5V 供电器件的输出、一个 3.3V CMOS 输入、一个 TTL 兼容输入和一个开漏传感器，可能有不同的高低阈值、输入电流和输出驱动能力。设计者必须把数据手册中的最坏输出和最坏输入放在同一张表里比较，而不是只看“都是 0/1”。

连接检查	要问的问题	典型风险
电源电压	前级输出高电平是否超过后级输入上限	过压损坏或长期可靠性下降
输入阈值	前级最差高/低输出是否满足后级阈值	电平看似变化，后级不承认
输入电流	前级能否提供或吸收后级所需电流	电平被拖偏，余量不足
输出驱动	前级能否驱动全部负载和走线电容	边沿变慢，时序失败
上拉配置	开漏/开集输出是否有合适上拉	释放后没有可靠高电平

5V 与 3.3V 系统互连的电平转换 上表回答的是“怎么查”，本专题回答“查出不匹配之后怎么办”。先把两个方向各自卡在哪里算清楚。5V 输出进 3.3V 输入：5V 超过多数 3.3V 器件的最大输入额定值，长期直连可能损坏输入结构，除非该输入明确标注 5V 容忍。反方向的问题更隐蔽：3.3V 系统的最差高电平可能只有 2.4V 左右，而 74HC 类 CMOS 输入在 5V 供电时要求 $V_{IH} \geq 0.7V_{CC} = 3.5V$ ，电平看似翻转了，后级却不承认。同一 5V 供电下，74HCT 系列把输入阈值做成 TTL 兼容： $V_{IH} = 2.0V$ 、 $V_{IL} = 0.8V$ ，2.4V 的高电平仍有 0.4V 余量。74HC 与 74HCT 的分工由此而来：同一电源域内部级联用 HC，需要承接 TTL 电平或 3.3V 输出的场合用 HCT。

真正需要转换时，常见做法有三种，代价各不相同。第一种是电阻分压：上臂 1.7kΩ、下臂 3.3kΩ，可把 5V 分成 $5V \times 3.3 / (1.7 + 3.3) = 3.3V$ 。它只适合从高到低、单向、低速的信号：分压网络的等效内阻（本例约 1.1kΩ）会拖慢边沿， $5V / 5k\Omega = 1mA$ 的静态电流常流不断，而且电阻只会衰减不会放大，3.3V 到 5V 方向完全用不了。第二种是专用电平转换芯片：双向、多通道，阈值和速度在数据手册中有明确规格；代价是多一片器件、两组电源脚和相应的布板面积。第三种是开漏加目标电源上拉：输出级只做开漏，上拉电阻接到对方电源—器件只负责拉低，高电平天然由目标电源域提供，电平自动匹配；I2C 总线正是靠这个结构允许不同电压域挂在同一条线上。代价是速度受上拉 RC 限制，且只有开漏或开集结构的信号才能这样接。

做法	适用方向与速度	代价与限制
----	---------	-------

电阻分压 (1.7kΩ+3.3kΩ)	仅 5V 到 3.3V、单向、低速	约 1mA 静态电流，约 1.1kΩ 等效内阻拖慢边沿，不能反向
电平转换芯片	双向、多通道，速度有手册规格	增加器件、两组电源和布板面积
开漏加目标域上拉	双向共享线，适合总线与中断请求	速度受上拉 RC 限制，仅开漏结构可用
选对输入阈值（74HC 与 74HCT）	3.3V 到 5V 单方向最省事	只是选型手段，不解决 5V 到 3.3V 过压

三种做法都不改变逻辑内容，只重新匹配两侧的电平约定。选型顺序是先定方向、速度和通道数，再决定用分压、专用芯片还是开漏结构。

数据手册里的参数要按“最坏组合”阅读。若前级在轻载、常温下能输出漂亮的 3.3V，不代表它在最大负载、高温、低电源电压下仍有同样余量；若后级典型阈值较低，也不代表所有批次、所有温度下都按典型值判断。经典教材强调抽象层次，但并不鼓励忽略边界条件。抽象的意义是把边界写清楚，然后在边界内使用更高层概念。

读取标准逻辑芯片数据手册时，可以采用固定顺序。先看供电范围，确认器件是否处在允许电源电压内；再看输入阈值，确认前级输出能被后级可靠承认；再看输出电流能力，确认它能驱动实际后级和上拉/下拉网络；再看传播延迟，确认多级组合路径能在采样前稳定；最后看封装、引脚、温度范围和绝对最大额定值，确认板级使用没有越界。这个顺序比直接查“某个门是什么功能”更接近硬件设计的实际检查顺序。

读取顺序	要确认的参数	与本章概念的关系
供电条件	工作电压范围、绝对最大额定值	电平约定必须先处在器件允许边界内
输入条件	V_{IH} 、 V_{IL} 、输入电流、滞回类型	后级如何把真实电压承认为 0/1
输出条件	V_{OH} 、 V_{OL} 、输出源/灌电流	前级能否给出足够强的高低电平
时间条件	传播延迟、上升/下降时间、使能/禁止延迟	组合路径多久后才可被使用
负载条件	扇出、输出电容、推荐上拉、总线负载	真实连接是否拖慢边沿或压缩余量
环境和封装	温度范围、封装引脚、ESD 和焊接约束	板级实现是否仍满足器件边界

例如同样是“用一个 NAND 门”，74LS、74HC、4000 CMOS 和高速 CMOS 的电气约定并不相同。某个系列可能适合 5V TTL 兼容输入，另一个系列可能要求 CMOS 阈值，某些高速器件对输入边沿和去耦要求更严格。正式设计不能只写“接一个 NAND”，还要写清器件族、供电电压、输入输出阈值、负载和时序要求。否则，逻辑表达式相同，实物板卡仍可能在温度、批次或负载变化时失效。

从这个角度看，缓冲器、电平转换器和驱动器并不是“逻辑上多余”的器件。缓冲器可以恢复电平和增强驱动；电平转换器让两个电源域的约定重新匹配；驱动器让小逻辑信号控制更大的负载。它们是否改变真值表不是唯一问题；它们是否让真实节点满足电平、负载和时序约定，才是硬件设计要回答的问题。

标准逻辑芯片把这些约束以产品形式暴露出来。以常见的 7400 系列为例，一个封装中可以放入若干个 NAND 门。教材里的 NAND 真值表只说明稳定逻辑行为，而芯片数据手册还会说明每个输入能承认的高低电平、输出能提供或吸收的电流、传播延迟范围、工作电压范围和扇出建议。换言之，标准门芯片把“布尔函数”和“电气约定”绑定在一起出售。

从 7400 类门芯片读到的项目	对应本章概念	设计时的含义
输入高低阈值	可靠 bit	前级输出必须落入后级承认区间
输出电流能力	驱动能力和扇出	一个门不能无限驱动后级
传播延迟	组合路径时序	多级门相连后要累计最坏延迟
工作电压范围	电平约定边界	不同电源域不能随意直连
封装引脚	物理边界	逻辑门最终仍要通过真实引脚连接

4000 系列 CMOS 逻辑则更适合说明互补路径和功耗。CMOS 门在稳定 0 或稳定 1 时，理想情况下没有从电源到地的持续强通路，静态功耗可以很低；切换时，输出电容充放电，形成动态功耗。现代处理器虽然远比 4000 系列复杂，但仍然继承了这个核心事实：频率越高、切换节点越多、负载越大，动态功耗越高；门越小、线越短、负载越低，延迟越容易下降。

比较 7400 TTL 和 4000 CMOS 时，不应把它们只看成“都能实现 NAND/NOR”。TTL 器件常用于说明输入电流、扇出和标准逻辑电平；CMOS 器件更适合说明极高输入阻抗、低静态功耗和输入悬空风险。一个 CMOS 输入若没有明确上拉或下拉，可能因为极小漏电和耦合电荷保持在不确定电平；这与“输入阻抗高”并不矛盾，反而正是高阻输入必须定义默认值的原因。

芯片族视角	教材中应观察的事实	对本章的补充意义
7400 TTL	输入电流、输出灌/拉电流、传播延迟和扇出限制	门芯片是带电气约定的布尔函数
4000 CMOS	高输入阻抗、低静态功耗、宽电源范围和较慢边沿选型	默认路径和输入调理不能省略
高速 CMOS	更低延迟、更强驱动、更严格边沿和布局要求	速度提高后，负载、布线和毛刺更敏感

体二极管与栅极保护 高输入阻抗这条性质还有更深的器件根源，值得补两句。第一句关于体二极管。功率 MOSFET 的源极与衬底在制造时短接，衬底与漏极之间天然留下一个 PN 结，称为体二极管。对源极接地的 NMOS，它的方向是从源极指向漏极：正常工作时漏极电位高于源极，二极管反偏不导通；一旦漏极被拉到比源极低约 0.7V——例如走线电感上的负尖峰或电源接反——它就绕过栅极自行导通。栅极控制不了这只二极管，所以 MOSFET 开关挡不住反向电流；桥式驱动里它有时被有意用作续流路径，但长期依赖它导通会带来额外损耗和可靠性风险。

第二句关于栅氧击穿。MOS 栅极是一块被氧化层绝缘的电容极板，氧化层只有纳米量级厚，常见器件的栅源额定电压仅约 $\pm 20V$ ；而人体摩擦积累的静电轻松达到几千伏。栅极又是高阻节点，电荷没有泄放路径，一次静电触碰就可能把氧化层击穿，器件当场失效或留下隐患。这正是 CMOS 输入必须配保护结构、未用输入必须接确定电平、插拔与焊接要防静电的物理原因。

两个廉价元件能化解其中大部分风险。其一是在驱动输出与栅极之间串一只电阻：它与栅电容构成 RC，既限制瞬态进入栅极的电流，也阻尼快速边沿引起的振铃；取值按速度定， $1k\Omega$ 串阻配 $5pF$ 小信号栅电容的时间常数约为 $1k\Omega \times 5pF = 5ns$ ，对低速输入毫无影响，驱动功率 MOS 时则常用几欧到几十欧换取边沿与开关损耗的平衡。其二是给栅极配一只到地或到电源的大阻值电阻，例如 $100k\Omega$ ：驱动断开、复位期间或连接器拔出时，栅极不再悬空，器件保持确定关断。这就是前面上拉/下拉原则在器件级的再次出现——高阻节点必须有默认归宿。

风险	物理来源	常用对策
----	------	------

体二极管误导通	漏源电压反向超过约 0.7V	抑制负压尖峰，必要时串肖特基二极管或按反峰设计
栅氧击穿	静电在无路可泄的高阻栅极上积累	输入保护结构、串联电阻、防静电操作
悬空误开通	驱动断开时栅极电荷随机	栅极上拉或下拉给出确定电平
边沿振铃	栅电容与走线电感形成谐振	串联栅电阻阻尼，按速度取值

这些措施都不改变真值表，却决定了开关器件在第一块板子上能否活着工作。

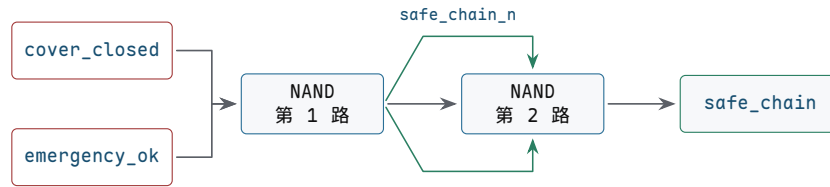
读一片标准 NAND 芯片，至少要同时读三层信息。第一层是逻辑：两个输入同时为 1 时输出为 0，其余情况输出为 1。第二层是电气：输入高低阈值、输出电流、工作电压和负载能力说明它能否与前后级可靠连接。第三层是时间：传播延迟说明输入变化后多久输出才可用。若只读第一层，就会把真实芯片误当成理想布尔符号；若只读第二、三层，又会失去门的功能目的。

把逻辑式映射到芯片引脚时，还要完成一次 **表达式到封装** 的翻译。以 74HC00 或传统 7400 类四路二输入 NAND 芯片为例，一个封装中包含四个独立 NAND 门。设计者不能只写“使用一个 NAND”，还要指定哪两个信号接到某一路 NAND 的输入脚，该路输出脚驱动哪个后级，芯片的 VCC 和 GND 接到哪个电源域，电源脚附近是否放置去耦电容，未使用门的输入是否被接到确定电平。

落地项目	需要写清的内容	常见错误
电源脚	VCC、GND、工作电压和去耦电容位置	只接信号脚，忽略供电噪声和地参考
输入脚	A、B 两个输入分别来自哪些内部语义信号	把低有效或未调理信号直接接入门
输出脚	输出驱动哪个门、缓冲器或后级输入	未检查扇出、负载和传播延迟
未用门	未使用输入接到确定高或低，输出可悬空不用	CMOS 输入悬空导致随机翻转和额外功耗
封装方向	引脚编号、缺口方向、封装类型和焊接方向	原理图正确但实物引脚接反

例如要把 `safe_chain = cover_closed AND emergency_ok` 映射到 74HC00，可先用一路 NAND 得到反相信号，再用另一路 NAND 自反相恢复为 AND。文字规格可以写成：第 1 路 NAND 输入接 `cover_closed` 与 `emergency_ok`，输出命名为 `safe_chain_n`；第 2 路 NAND 两个输入都接 `safe_chain_n`，输出命名为 `safe_chain`。这种写法让中间反相信号、芯片资源占用和调试测点都可审查。

```
safe_chain_n = cover_closed NAND emergency_ok
safe_chain   = safe_chain_n NAND safe_chain_n
```



第二路 NAND 的两个输入接同一个反相信号，等价于把第 1 路输出再反相。图中分成两条回路线，是为了避免箭头穿过文字。

图示：用两路 NAND 实现 AND。图的命题是：功能等价必须能对应到具体芯片资源和中间测点。

芯片引脚级说明还必须保留物理约束。每片芯片电源脚附近通常需要本地去耦电容，以降低门切换时的电源压降；输入不能超过允许电压范围；输出不能超出源电流或灌电流能力；同一输出驱动多个输入时要重新计算扇出；跨连接器或长线时还要考虑保护、终端和边沿质量。把布尔式映射到芯片，不是把公式抄到封装上，而是把功能、电平、时间、负载和装配边界同时写进规格。

若读者希望把安全启动控制板搭成一块低速演示电路，可以采用标准 CMOS 逻辑芯片完成第一版。这个版本不追求工业安全认证，只用于验证本章的电平、路径、组合逻辑和测量方法。输入侧使用按键或拨码开关表示 `start_request`、`cover_closed`、`emergency_ok`、`temp_alarm` 和 `current_alarm`；每个输入都有明确上拉或下拉，必要时经施密特输入整形；组合核心用 74HC00、74HC04、74HC08、74HC32 或同类芯片实现；输出侧用缓冲器或三极管/MOSFET 驱动 LED，而不是让普通门输出承担全部指示灯电流。

模块	可选器件或结构	检查重点
输入默认值	10kΩ 量级上拉/下拉、电阻网络或拨码开关模块	松开、断线、未接时不悬空，默认方向符合安全策略
慢边沿整形	74HC14 或同类施密特反相器	慢按钮边沿和噪声不能在阈值附近反复翻转
基本门	74HC00、74HC08、74HC32、74HC04 或等价标准门	每个门的输入阈值、输出驱动和传播延迟满足连接要求
故障汇总	OR 门、NAND-only 网络或二极管加缓冲结构	任一故障都点亮故障灯并禁止启动，双故障有明确行为
输出指示	74HC244/74HC125 缓冲、三极管或小 MOSFET 驱动 LED	指示灯负载不拖垮内部逻辑电平
供电与去耦	5V 或 3.3V 稳定电源，每片芯片近旁 0.1μF 去耦	切换时电源和地不出现影响阈值判断的扰动

搭建顺序应从静态边界开始，而不是直接运行全部逻辑。第一步只接电源、地和去耦，确认芯片供电正确。第二步只接输入默认网络，用万用表确认每个输入在松开、按下、断线时的电平。第三步逐个验证单门行为，例如 NAND、反相器和 OR 的四行真值表。第四步再连接 `fault_lamp` 和 `no_fault`，确认任一报警都会禁止启动。第五步连接 `safe_chain`、`request_ok` 和 `allow_start`，遍历正常允许、单条件缺失、单故障、双故障和断线默认。最后才连接 LED 驱动或执行机构模拟负载，并确认负载加入后内部逻辑电平仍满足后级阈值。

阶段	操作	进入下一阶段的条件
1	只检查电源、地、去耦和芯片方向	供电电压正确，芯片不发热，电源脚附近无明显跌落
2	接入输入默认网络，不接组合核心	每个输入在所有机械状态下都有确定电平
3	单独验证每片门芯片的一路门	真值表正确，输出能驱动一个后级输入或测试点

4	接入故障汇总和无故障反相信号	报警输入任一为 1 时 <code>no_fault=0</code>
5	接入安全链和启动允许逻辑	只有所有允许条件同时成立时 <code>allow_start=1</code>
6	接入缓冲、LED 或模拟执行负载	负载不改变内部逻辑结果，边沿和电平仍可测量

这个低速演示电路也能暴露真实工程问题。若未用 CMOS 输入悬空，LED 可能随机闪烁；若按钮没有消抖，逻辑分析仪可能看到多个启动请求；若把 LED 直接接到弱输出上，门输出的高电平可能被拖低；若两片推挽输出误接到同一节点，一高一低时会产生冲突电流；若所有芯片共用长电源线而缺少去耦，多个输出同时翻转时可能出现误触发。它们都不是布尔代数错误，却都属于第一章必须掌握的硬件错误。

三、从路径表得到逻辑门

门电路的第一步不是画符号，而是把输入组合列完整。两个输入 A、B 只有四种组合。电路必须对每一种组合给出明确输出，而且每一行都要满足前面的要求：输出节点不能悬空，不能上下冲突，必须能在规定时间内进入可靠电平区间。

从需求到门电路，建议固定采用五步。第一，给每个输入和输出写清楚硬件含义，特别注意别把低有效信号误读成高有效。第二，列出完整真值表，不跳过“看起来不会发生”的行——只按自然语言想象少数情况，是最常见的漏项来源。第三，标出输出为 1 的行，或者标出输出为 0 的禁止行，不遗漏边界输入和异常组合。第四，把每一行翻译成开关路径：哪些条件使上拉导通，哪些条件使下拉导通，不写只有公式、不看电流路径。第五，检查每一行是否有明确驱动、是否存在冲突、是否满足电平和延迟约束——只证明逻辑等价，不算证明硬件可靠。

以安全允许灯为例。A 表示保护盖合上，B 表示急停未按下。允许灯亮的条件很严格：任一条件不满足，输出都必须为 0。

A	B	允许灯	为什么
0	0	0	盖子没合上，急停也不是可运行状态
0	1	0	急停条件满足，但盖子没合上
1	0	0	盖子合上了，但急停条件不满足
1	1	1	两个安全条件同时满足

该表对应 AND 的行为。它不是抽象数学定义，而是“两个条件都满足才允许”的硬件约束。若某个实现让 A=1、B=0 时灯也亮，那就不是 AND 近似得不够好，而是安全行为错了。真值表的价值在于防止“只凭自然语言写电路”。“两个条件都满足”看起来简单，但实现时很容易漏掉某一行。

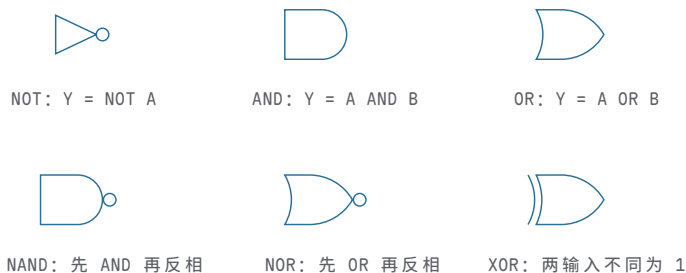
再看故障灯。A 表示温度报警，B 表示电流报警。故障灯的规则反过来：只要有一个坏消息出现，就要亮。

A	B	故障灯	为什么
0	0	0	没有任何故障报警
0	1	1	电流报警已经足够触发故障
1	0	1	温度报警已经足够触发故障
1	1	1	两个故障同时存在，仍应报警

该表对应 OR 的行为。AND 和 OR 的差别不在符号长什么样，而在需求不同：一个要求所有条件同时成立，一个要求任一条件成立即可。NOT 只有一个输入，但同样要列完整。若 A 表示“禁止信号”，输出 Y 表示“允许动作”，那么 A=0 时 Y=1，A=1 时 Y=0。它解决的是反向解释：输入出现时，输出反而关闭。

到这里六个基本门都已经出场，把它们的工程符号一次认全。前面刻意只用方框

和路径表，是为了先建立“门 = 电流路径”的读法；从此刻起，本书改用标准符号：



图示：六个基本门的标准符号（美式画法）。注意输出端的小圆圈：它表示反相——NAND 就是 AND 加一个小圆圈，NOR 就是 OR 加一个小圆圈。这个小圆圈和晶体管开关模型里 PMOS 控制端的圆圈是同一个约定：有圈即取反。

不要把“0 是假、1 是真”机械套到每个硬件信号上。某些信号是低有效，例如 `reset_n`、`chip_select_n`，低电平才表示动作发生。读这类信号时，先写真值表，再谈门电路。

低有效信号值得单独处理。名字中的 `_n` 常表示 active low，即低电平有效。若信号名为 `emergency_ok_n`，那么 0 可能表示“急停链路正常允许运行”，1 反而表示“不允许”。这种命名并不适合作为入门案例的默认读法，但真实硬件中很常见，因为开漏输出、有线汇总、复位链路和片选信号都可能倾向于低有效形式。读低有效信号时，不能先把 1 写成“发生”、0 写成“未发生”，必须先读信号约定。

信号名	有效电平	读法
<code>reset_n</code>	0	低电平时复位动作发生
<code>chip_select_n</code>	0	低电平时芯片被选中
<code>output_enable_n</code>	0	低电平时输出驱动器打开
<code>fault_n</code>	0	低电平可能表示故障请求有效，需看约定

低有效并不只是命名习惯，它会改变推理方向。假设外部急停链路使用低有效输入 `emergency_stop_n`：链路正常释放时为 1，急停动作、断线或安全继电器打开时为 0。若内部逻辑需要的是正逻辑 `emergency_ok`，就必须先做一次语义转换：

```
emergency_ok = emergency_stop_n
stop_request = NOT emergency_stop_n
```

这里 `emergency_ok` 和 `stop_request` 都可以是正确内部信号，但它们表达的事实不同。前者适合放进 `allow_start` 的 AND 链；后者适合放进故障或停机请求的 OR 链。若在同一个表达式里混用低有效原始信号和正逻辑语义信号，很容易写出方向相反的门网络。正式硬件文档通常会在信号表里明确有效电平，就是为了防止这种错误。

真值表说清楚“应该怎样”，路径表说明“为什么会这样”。用受控开关看 AND，最自然的结构是串联。两个开关串在同一条路径上，只有 A 和 B 都让自己的开关闭合时，整条路径才通。OR 更像并联。A 控制一条路径，B 控制另一条路径。只要任意一路闭合，输出就能被拉向目标电平。

A	B	串联路径状态	AND 输出为什么成立或不成立
0	0	两个开关都断开	没有完整导通路径
0	1	A 断开，B 闭合	串联路径仍被 A 切断
1	0	A 闭合，B 断开	串联路径仍被 B 切断

1

1

两个开关都闭合

从源到输出的路径完整

如果只画一条“让输出为 1 的路径”，还没有完成门电路设计。输出节点不能靠悬空表示 0。因此每一行真值表还要补一张路径表，说明高电平路径和低电平路径的状态。一个合法门在稳定输入下，不应该让输出悬空，也不应该让上拉和下拉强路径同时导通。

从开关网络看，NAND 和 NOR 往往比 AND 和 OR 更像基础积木。原因是它们天然带反相输出，更容易形成互补的上拉和下拉结构：当下拉网络导通时输出被拉低；当下拉网络不导通时，上拉网络负责把输出拉高。

A	B	NAND 输出	开关直觉
0	0	1	下拉串联路径断开，上拉负责拉高
0	1	1	A 断开，下拉路径不完整
1	0	1	B 断开，下拉路径不完整
1	1	0	A、B 都闭合，下拉路径完整，输出被拉低

NOR 可以用同样方式读。NOR 的输出只有在 $A=0$ 且 $B=0$ 时为 1；只要 A 或 B 为 1，输出就为 0。若以下拉网络看，A、B 并联，只要任一路输入为 1，就能把输出拉低；只有两路都不导通时，上拉网络把输出拉高。

NAND 和 NOR 还可以帮助理解 De Morgan 定律。逻辑上， $\text{NOT}(A \text{ OR } B)$ 等价于 $(\text{NOT } A) \text{ AND } (\text{NOT } B)$ ；硬件上，这句话的含义是：只要 A 或 B 有一路故障报警，输出就应被拉低；只有 A 和 B 都没有故障报警，输出才允许被拉高。对于安全启动控制板，若 $\text{fault} = \text{temp_alarm OR current_alarm}$ ，那么“没有故障”就是 NOT fault ，也就是 $(\text{NOT temp_alarm}) \text{ AND } (\text{NOT current_alarm})$ 。这不是代数游戏，而是把“任一故障禁止启动”和“所有故障都不存在才允许启动”写成同一硬件约束的两种形式。

```
fault = temp_alarm OR current_alarm
no_fault = NOT fault
no_fault = (NOT temp_alarm) AND (NOT current_alarm)
```

只用 NAND 就能构造 NOT、AND、OR：

```
NOT A = A NAND A
A AND B = (A NAND B) NAND (A NAND B)
A OR B = (A NAND A) NAND (B NAND B)
```

这说明 NAND 在功能上完备，不说明所有实现都应该只用 NAND。真实设计会在功能正确的前提下，考虑门级数量、路径延迟、面积、功耗和负载。逻辑式等价，只说明稳定输出行为相同；它不保证延迟、功耗和驱动能力相同。

例如安全允许信号可以先写成：

```
allow_start = start_button AND cover_closed
              AND emergency_ok AND no_fault
```

若只用 NAND 实现，需要把多输入 AND 拆成若干 NAND 与反相结构。每拆一次，逻辑功能仍可保持，但门级层数、负载和延迟都会变化。因此“用 NAND 可以实现”只是功能完备性结论；真正进入硬件设计时，还要继续检查最长路径和每一级扇出。

XOR 的输出在两个输入不同的时候为 1，相同的时候为 0。它解决的是“检测两个 bit 是否不同”这个具体问题。

A	B	A XOR B	解释
0	0	0	两个输入相同
0	1	1	两个输入不同

1	0	1	两个输入不同
1	1	0	两个输入相同

可以把 XOR 拆成两种会输出 1 的情况： $A=1$ 且 $B=0$ ，或者 $A=0$ 且 $B=1$ ：

$$A \text{ XOR } B = (A \text{ AND NOT } B) \text{ OR } (\text{NOT } A \text{ AND } B)$$

这条式子不要只当代数记忆。它对应两个乘积项：一项在“只有 A 为 1”时成立，另一项在“只有 B 为 1”时成立，最后的 OR 把两项合并。若两个输入都为 0，两项都不成立；若两个都为 1，两个“只有一个为 1”的项同样都不成立。XOR 后面会反复出现：两个 bit 相加时，和位就是“两个输入是否不同”；比较两个 bit 时，XOR 可以告诉我们它们是否不同；接一个 NOT，就能得到“两个输入是否相同”。

XOR 也说明了“常用门”和“便宜门”不一定一致。在纸面表达式里，XOR 是一个符号；在门级实现里，它往往比 AND、OR、NOT 更复杂，路径延迟也可能更长。因此，读到加法器、比较器、校验位或地址译码时，不能只数逻辑符号数量。若关键路径里连续经过多个 XOR，速度代价可能比同样数量的简单门更高。经典教材在讲门延迟时强调这一点，是为了让读者把布尔函数和实现代价同时放进脑中。

XOR 与 XNOR：门级构成与三个经典用途 XOR 既然比基本门贵，就值得先看清它贵在哪里。若器件库只有二输入 NAND，XOR 最少可以用 4 个 NAND 实现。办法不是凭空想的，而是把与或式逐步改写成 NAND 形式：

```
n = A NAND B
p = A NAND n      -- 等价于 (NOT A) OR B
q = B NAND n      -- 等价于 A OR (NOT B)
Y = p NAND q      -- 展开后正是 (A AND NOT B) OR (NOT A AND B)
```

按信号流向数级数： n 在第一级， p 、 q 在第二级， Y 在第三级——一个二输入 XOR 需要三级门延迟。若改用与或式直接搭建，需要 2 个反相器、2 个 AND、1 个 OR 共 5 个门，信号同样要依次经过反相、AND、OR 三级。两种搭法殊途同归：XOR 无论怎么实现，门数和级数都比 AND、OR、NAND 多。XNOR 只需在 XOR 之后再取反一次，门级成本再加一级反相。这就是前文“常用门不等于便宜门”的具体含义：表达式里一个清爽的符号，在门级就是三级延迟。

值得付出这个代价，是因为 XOR 回答的是算术与校验中最常问的问题：“两个 bit 是否不同”。三个经典用途几乎贯穿本书后续所有章节。

第一个用途是奇偶校验。8 位数据的偶校验位就是把 8 个 bit 全部异或起来：1 的个数为偶数时结果为 0，为奇数时结果为 1。8 个输入两两分组，用 7 个二输入 XOR 排成 3 层树（第一层 4 个，第二层 2 个，第三层 1 个）；若图省事排成一条链，则要 7 级延迟。树形结构门数不变、级数从 7 降到 3，这是“同一函数、不同拓扑、不同延迟”的又一个实例。接收端把收到的 8 位再算一次同一棵树，与随数据传来的校验位比较，任何单个 bit 出错都会让两边不一致——XOR 树因此是通信和存储系统里最小的检错硬件。

第二个用途是加法器的和位。前文全加器的 $S = A \text{ XOR } B \text{ XOR } C_{in}$ ，问的正是“A、B、 C_{in} 中是否有奇数个 1”；而进位 C_{out} 问的是“是否至少两个 1”，即三传感器判断器。一位全加器因此可以读成一个奇偶网络加一个多数网络：XOR 负责“奇数个”，AND/OR 负责“至少两个”。

第三个用途是相等比较。XOR 输出 1 表示“不同”，把它反过来：XNOR 输出 1 表示“相同”。比较两个 4 位数是否相等，就是逐位 XNOR，再把四路结果用 AND 归并：

```
eq0 = A0 XNOR B0
eq1 = A1 XNOR B1
eq2 = A2 XNOR B2
eq3 = A3 XNOR B3
equal = eq0 AND eq1 AND eq2 AND eq3
```

归并的 AND 同样可以排成两级树而不是四输入单门，以控制扇入。后文数值比较器还会看到：相等信号 `eq_i` 不只用于判断全等，还是大小比较的裁决基础。

用途	回答的问题	关键结构	检查点
奇偶校验	1 的个数是奇还是偶	n bit 用 n-1 个 XOR 排成 $\lceil \log_2 n \rceil$ 级树	排成链则延迟随位数线性增长
加法和位	是否有奇数个 1	两级 XOR: <code>A XOR B</code> 再 <code>XOR Cin</code>	和位与进位是两条不同延迟的路径
相等比较	每一位是否都相同	XNOR 逐位加 AND 归并树	归并树分级，避免单门扇入过大

这张表也预告了一个反复使用的读图方法：看到 XOR，先问它在数“不同”还是数“奇偶”；看到 XOR 树，先数层数而不是门数。

真值表还可以用“最小项”来读。所谓最小项，就是与一行输入组合对应的完整乘积项。例如三输入 A、B、C 中，只有 A=1、B=0、C=1 这一行输出为 1，则对应的乘积项是 `A AND (NOT B) AND C`。把所有输出为 1 的行用 OR 合并，就得到一种直接实现。它不一定最简，但它有一个优点：每一项都能追溯到真值表中的具体一行。

A B C	输出 Y	对应乘积项
0 1 1	1	<code>(NOT A) AND B AND C</code>
1 0 1	1	<code>A AND (NOT B) AND C</code>
1 1 0	1	<code>A AND B AND (NOT C)</code>
1 1 1	1	<code>A AND B AND C</code>

若把这些最小项直接相加，就能实现“三个输入至少两个为 1”的判断。但前文给出的表达式 `(A AND B) OR (A AND C) OR (B AND C)` 更短，因为它把若干行合并成“任意两项同时成立”的条件。这里体现出两个层次：最小项保证从真值表机械得到正确表达式，化简则减少门数和路径深度。化简以后仍要确认没有改变低有效含义、默认安全策略和毛刺风险。

卡诺图的核心可以用“相邻最小项”的文字规则理解。若两个最小项只在一个输入上不同，且其余输入完全相同，那么这个不同输入对输出为 1 并不是必要条件，可以被消去。例如：

$$(A \text{ AND } B \text{ AND } C) \text{ OR } (A \text{ AND } B \text{ AND NOT } C) = A \text{ AND } B$$

左边两项分别覆盖 C=1 和 C=0 两行；既然 C 的两种取值都让输出为 1，最终表达式就不必再依赖 C。卡诺图的格子相邻关系，本质上就是帮助设计者系统地寻找这种“只差一个输入”的合并机会。合并得当可以减少门数、缩短路径和降低负载；但合并之后必须重新检查低有效语义、非法状态和过渡行为。

文字规则必须落实到格子才算学会。以“三个输入至少两个为 1”为例，把 A 放在行、BC 放在列，每个格子填一行的输出：

		BC 消去 A: BC			
		00	01	11	10
A	0	0	0	1	0
	1	0	1	1	1
		消去 B: AC		消去 C: AB	

图示：三变量卡诺图（注意列头顺序 00、01、11、10 是 Gray 码：相邻两列永远只差一个 bit，这样“相邻”才有消元意义）。三个圈各合并两个相邻的 1：每个圈里那个“两种取值都出现”的输入被消去，得到 $Y = AB \text{ OR } AC \text{ OR } BC$ ——与后文三传感器判断器用最小项硬推出的结果一致，但快得多。

化简还可能改变毛刺形态。某些冗余项对稳定真值表没有贡献，却能在输入翻转时提供连续路径。后文会看到， $(A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$ 在 $B=1$ 、 $C=1$ 且 A 翻转时可能短暂掉到 0；增加共识项 $B \text{ AND } C$ 不改变稳定真值表，却能减少这类静态毛刺风险。因此，安全相关组合逻辑不应把“项数最少”当作唯一目标。形式上最简的表达式，未必是危险输出最稳妥的实现。

有时真值表里会出现“不关心”项。所谓不关心，不是设计者懒得定义，而是某些输入组合在系统约定中不会被使用，或者该组合出现时输出 0/1 都不会影响可观察行为。例如一个 2-bit 故障编码只允许 00、01、10，保留 11 作为非法状态；若后级在 11 时一定进入停机诊断，那么某个普通指示灯在 11 下亮或灭可能都可以接受。化简逻辑时可以利用不关心项减少门数，但必须谨慎：一旦所谓“不可能”的输入来自外部连接器、未初始化状态或故障状态，它就不应轻易被当作不关心。

输入组合类别	可以怎样处理	必须确认的问题
正常组合	严格定义输出	是否覆盖了所有业务行为
保留组合	可定义为诊断、停机或不关心	后级是否真的不会误用该输出
非法组合	通常导向安全默认值	硬件上是否可能短暂出现
外部故障组合	不应随意当作不关心	断线、噪声、上电瞬间如何表现

对于安全启动控制板，保守做法是：任何未确认安全的组合，都不应让 `allow_start` 为 1。即使某个组合在正常操作流程中不会出现，只要它可能由断线、上电初态、输入调理失败或编码错误产生，就应进入“不启动”或“故障”一侧。组合逻辑的化简必须服从系统约定，而不是反过来让系统约定迁就最短表达式。

逻辑化简还要保留可审查性。对教学和安全控制而言，表达式稍长但能清楚对应需求，往往比极度压缩的表达式更容易审查。若某个输出必须由四个安全条件共同允许，那么把表达式保留为四个语义项，有助于后续检查每个条件来自哪里、默认值是什么、是否经过输入调理。化简可以减少门数和延迟，但不能让规格意图变得不可见。

四种常见表达方式各有取舍。逐项语义表达式容易对应需求和安全审查，但门数和层数通常不是最少；最小项展开可以机械追溯到真值表行，表达式却可能很长、延迟不一定好；代数化简式可能减少门数和路径层数，却可能隐藏低有效、默认值和故障策略；器件库映射式最接近实际可实现结构，但必须重新检查负载、扇入和时序。选择哪一种，取决于这段逻辑更需要可审查性还是更少的门级资源。

因此，正式设计常保留多个层次的描述：需求层写“所有安全条件满足才允许启动”；逻辑层写 `allow_start = start_request AND cover_closed AND emergency_ok AND no_fault`；实现层再说明由哪些门、缓冲和输出驱动器组成；验证层记录电平、延迟、毛刺和异常输入的检查结果。四个层次互相对应，才能避免“公式正确但系统含义错误”。

处理器控制逻辑中常见的 PLA，可以提前理解为“可规则化实现的组合逻辑”。PLA 的基本思想是先用一组 AND 项识别输入条件，再用 OR 项合成输出控制信号。例如指令译码可以把操作码、状态位和时序阶段转成若干控制线：选择哪一路进入 ALU、是否写寄存器、是否读内存、下一步进入哪个控制状态。第一章不需要展开 PLA 版图，但读者应看出它和最小项方法的关系：先识别条件，再汇总结果。

PLA 视角	本章对应概念	在处理器中的用途
输入条件	真值表行、最小项、状态位	操作码、标志位、时序阶段
AND 平面	乘积项	某条指令或某类条件成立
OR 平面	乘积项汇总	产生写寄存器、读内存、选择 ALU 等控制线

8086 这类早期微处理器内部就包含大量译码和控制逻辑。它不是一堆孤立门的随意堆叠，而是把操作码解释、微操作顺序、总线周期和算术逻辑组合成可重复执行的机器动作。后面讲整机时会展开这些结构；在第一章，只需要先建立一个判断：处理器内部那些看似高级的控制信号，本质上也必须由可靠电平、门网络、组合路径和时序边界支撑。

8086 常被放在“完整 CPU”章节讲，但在第一章也能提前借用它的局部结构。它把取指相关的总线接口工作和执行相关的内部运算工作分开，并通过指令预取队列尝试让取指与执行重叠。这个案例说明：低时延并不只来自某个门更快，也来自**把等待路径重叠**。不过预取队列不能消除所有等待；分支改变控制流、总线访问受阻或指令消费速度超过预取速度时，执行仍会遇到空洞。

80386 的案例则说明另一件事：晶体管增加以后，芯片不只是把原有电路做得更快，还会把更多系统功能搬到片内。32-bit 数据通路、保护机制、地址转换和更复杂控制逻辑，都意味着更多组合判断、更多寄存器边界和更复杂的关键路径管理。处理器越复杂，越不能把“时延”只理解成单个门的传播时间；控制路径、数据路径、存储路径和外部接口路径都要分别分析。

还可以把几个经典芯片作为后续整机章节的预告。它们不需要在第一章展开完整微架构，但可以帮助读者把“门网络、组合控制、寄存器边界和片上集成”放到真实产品谱系中理解。

芯片	先观察的结构特征	与本章概念的关系
MOS 6502	早期微处理器，外部总线与内部控制紧密配合	指令执行依赖译码、数据通路选择和 控制时序
Zilog Z80	经典 8-bit CPU，寄存器组和总线控制较丰富	组合译码与状态机共同组织内存和 I/O 访问
Intel 8051	单片机把 CPU、存储器、I/O 和定时器放到同一芯片	片上集成减少板级连接，使控制信号 更局部
Intel 80486	在处理器内加入更强流水线、片上 cache 和浮点方向的集成	常用路径靠近执行核心，外部等待被 部分削减
Pentium	双流水线、分支预测和更复杂执行控制成为重点	低时延不仅靠门快，还靠预测、并行 和路径重叠

这些案例覆盖了从“CPU 依赖外部存储器和总线”到“更多功能进入片内”的演进。6502 和 Z80 让读者看到早期微处理器怎样把控制、数据通路和外部总线组织成机器周期；8051 说明 CPU 不一定孤立存在，片上 I/O 与定时器会把整机控制的一部分直接收进芯片；80486 与 Pentium 则预告 cache、流水线和预测将成为降低常见等待的重要工具。后面讲整机时会展开这些芯片；第一章只要求先建立共识：无论芯片多经典，内部仍由可靠电平、门网络、组合路径和时序边界支撑。

四、门网络构成组合逻辑

组合逻辑的特点很直接：输出只由当前输入决定，不保存过去。它像一个硬件函数，但这个函数不是一步一步执行的过程，而是一张由门和连线组成的网络。输入变化后，信号沿多条路径同时传播；等延迟过去，输出稳定到当前输入对应的值。

设计组合逻辑时，可以把前一节的五步具体化为一条工作顺序。先列出输入输出，让每个 0/1 的含义明确；再列出行为，用真值表或条件表覆盖所有输入组合；然后写出逻辑表达式，让输出为 1 或 0 的每个条件都对应一条可实现路径；接着合成网络，用基本门、选择器和译码器明确每个输出的驱动者和选择者；最后检查物理限制—延迟、扇出、毛刺、电平余量，并说明后级在什么时候使用这个输出。这个顺序不要求一开始就写出最简表达式，而是先把需求完整展开，再逐步收缩为硬件网络。

先做一个三传感器判断器。输入 A、B、C 表示三个传感器，要求至少两个传感器为 1 时输出 1。这个需求不能只凭直觉画线，先列出行为：

A	B	C	Y	原因
0	0	0	0	一个条件都没有
1	0	0	0	只有一个条件
0	1	0	0	只有一个条件
0	0	1	0	只有一个条件
1	1	0	1	A 和 B 同时满足
1	0	1	1	A 和 C 同时满足
0	1	1	1	B 和 C 同时满足
1	1	1	1	三个都满足

从表中可以直接得到一个门网络：

$$Y = (A \text{ AND } B) \text{ OR } (A \text{ AND } C) \text{ OR } (B \text{ AND } C)$$

这条表达式包含三个乘积项。第一项在 A、B 同时为 1 时成立；第二项在 A、C 同时为 1 时成立；第三项在 B、C 同时为 1 时成立。最后的 OR 不是任意合并，而是在检查三个条件中是否至少有一个成立。按这种方式阅读，门网络就不只是公式变形，而是把真值表中每一种输出为 1 的原因接成硬件。

两个 bit 相加是第二个基本组合逻辑例子。输出不止一个 bit，因为 1+1 得到二进制 10，需要一个和位 S 和一个进位 C。

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

观察表格可以看到，S 在 A、B 不同时为 1，C 在 A、B 同时为 1：

$$\begin{aligned} S &= A \text{ XOR } B \\ C &= A \text{ AND } B \end{aligned}$$

这就是半加器。它已经完成了一位相加，但它还缺一个输入：来自低位的进位。只要要做多位加法，最低位之外的每一位都不能只看 A 和 B，还要看低一位是否进上来。

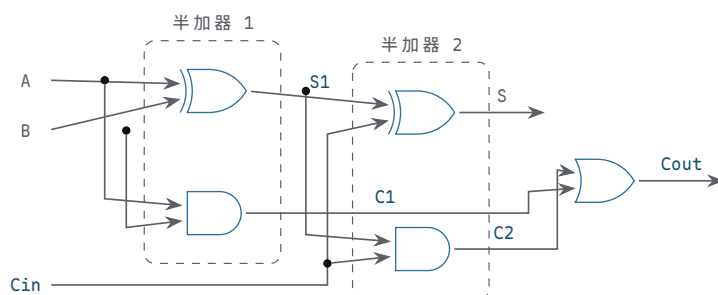
全加器输入为 A、B、Cin，输出为 S、Cout。可以先把 A 和 B 用半加器相加，得到临时和 S1 和进位 C1；再把 S1 和 Cin 用第二个半加器相加，得到最终和 S 和第二个进位 C2；最后把两个进位合并。

$$\begin{aligned} S1 &= A \text{ XOR } B \\ C1 &= A \text{ AND } B \\ S &= S1 \text{ XOR } Cin \\ C2 &= S1 \text{ AND } Cin \\ Cout &= C1 \text{ OR } C2 \end{aligned}$$

先别急着看电路，把三输入八行真值表列全。加法本质是“数 1 的个数”：一个 1 都没有，和为 0；恰好一个 1，和为 1 不进位；两个 1，和位变 0、产生进位；三个 1，和位为 1 且进位：

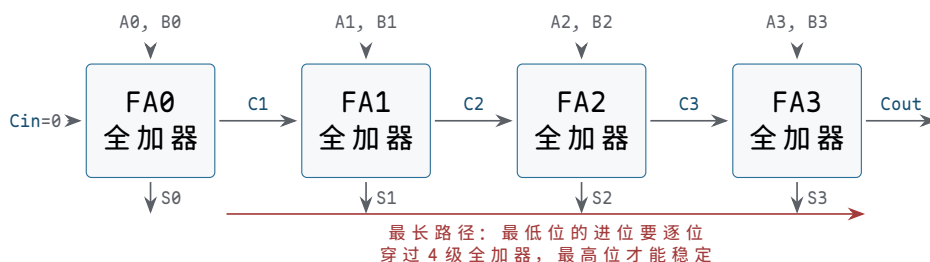
A	B	Cin	S	Cout	解释
0	0	0	0	0	没有 1
0	0	1	1	0	只有一个 1（来自进位输入）
0	1	0	1	0	只有一个 1
0	1	1	0	1	两个 1：本位写 0，向高位进 1
1	0	0	1	0	只有一个 1
1	0	1	0	1	两个 1
1	1	0	0	1	两个 1
1	1	1	1	1	三个 1：和位 1，再进 1

对照表格验证上面的式子：S 在“恰好一个 1”或“三个 1”时为 1，正是 $A \oplus B \oplus Cin$ ；Cout 在“至少两个 1”时为 1，正是三传感器判断器的结果。再把式子落成门，就是两个半加器加一个 OR：



图示：全加器 = 两个半加器 + 一个 OR。第一个半加器算 $A+B$ ，第二个半加器把临时和 $S1$ 再与进位输入 Cin 相加；两处的进位输出经 OR 合并——只要有一处产生进位，就向高位进 1。把这张图的每个输出和上面八行真值表逐行对一遍，比背公式可靠得多。

把全加器一位一位串起来，就得到串行进位加法器。它的逻辑很清楚：第 0 位算出进位，传给第 1 位；第 1 位再把进位传给第 2 位。问题也很清楚：若最低位产生的进位一路传到最高位，最高位必须等前面所有进位稳定后才能稳定。这是组合逻辑里第一次真正遇到“逻辑正确”和“速度足够”的差别。真值表保证最后结果正确，传播延迟决定结果多久之后才可靠。



图示：4 位串行进位加法器。每位的和位 S 只依赖本位输入，很快；但 $C4$ 依赖 $C3$ 、 $C3$ 依赖 $C2$ ……进位像接力一样从低位向高位逐级传播，红色线段就是决定全机速度的那条最长路径。加法器位数越多，这条链越长——这正是第三章要解决的矛盾。

全加器也可以用生成与传播来阅读。若 A 和 B 同时为 1，本位一定产生进位，称为生成；若 A 和 B 恰好一个为 1，那么本位不会自己产生进位，但会把输入进位传到输出，称为传播。

```
generate = A AND B
propagate = A XOR B
Cout = generate OR (propagate AND Cin)
S = propagate XOR Cin
```

这种写法为后面的加法器优化留下接口。串行进位把进位逐位传递，结构简单但最长路径随位数增长；更快的加法器会提前计算若干位的生成和传播关系，减少等待层数。本章只要求读者看清一个事实：组合电路不仅有“算什么”，还有“最慢路径从哪里穿过”。算术电路尤其如此，因为进位路经常常决定 ALU 的速度上限。

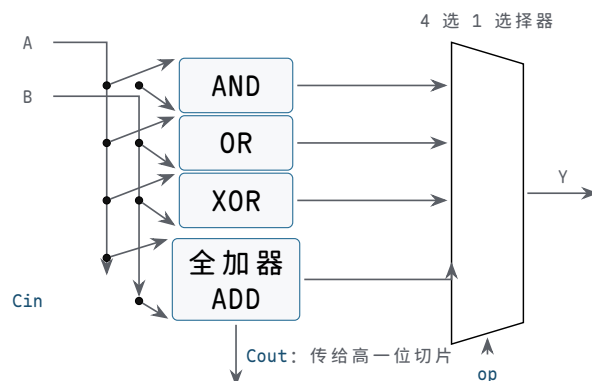
把全加器再向前推进一步，就得到 1-bit ALU 切片的雏形。ALU 不是不可分解的黑箱，而是许多位宽相同的切片并排工作：每一位接收本位的 A、B、来自低位的进位，以及操作选择信号；它同时准备若干候选结果，例如 AND、OR、XOR 和 ADD；最后由选择器决定哪一种候选结果成为本位输出（选择器是由控制信号从多路输入中选一路输出的组合电路，下一小节马上讲到）。进位输出则继续传给高一位或交给更快的进位逻辑处理。

```
and_result = A AND B
or_result  = A OR B
xor_result = A XOR B

sum_result = A XOR B XOR Cin
Cout = (A AND B) OR ((A XOR B) AND Cin)

Y = mux(op, and_result, or_result, xor_result, sum_result)
```

这段行为式只是说明逻辑关系，不表示硬件按行执行。真实电路中，AND、OR、XOR 和加法候选路径可以同时传播；op 控制选择器，让其中一路成为输出。于是，一个 1-bit ALU 切片至少包含三类组合结构：基本逻辑门、全加器、选择器。多个切片并排以后，AND/OR/XOR 这类逐位运算的延迟主要取决于本位门和选择器；ADD 则还受进位路径影响。



图示：1-bit ALU 切片：四路候选同时传播，op 控制选择器让其中一路成为本位输出 Y；进位不进选择器，沿进位链直接传给高位切片。把这种切片并排 16 个就是 8086 的加法/逻辑单元雏形，并排 32 个、64 个，就逼近 80386 和现代执行单元——结构没变，只是宽度和进位优化在变。

减法通常不需要另造一套独立减法器。二进制补码中， $A - B$ 可以写成 $A + (\text{NOT } B) + 1$ 。硬件上常见做法是在 B 输入前放置一组可控反相器或 XOR 门：做加法时让 B 原样进入全加器，做减法时让 B 翻转，同时把最低位进位输入置为 1。这样同一条加法进位链既可服务 ADD，也可服务 SUB。

```
B_eff = B XOR B_invert
sum_result = A XOR B_eff XOR Cin
Cout = (A AND B_eff) OR ((A XOR B_eff) AND Cin)
```

操作	B_invert	最低位 Cin	本位输出含义
ADD	0	0	$A + B$ 的和位
SUB	1	1	$A + (\text{NOT } B) + 1$ ，即补码减法

AND	不使用或置 0	不使用	输出 $A \text{ AND } B$ ，进位链不作为结果来源
OR	不使用或置 0	不使用	输出 $A \text{ OR } B$ ，进位链不作为结果来源
XOR	不使用或置 0	不使用	输出 $A \text{ XOR } B$ ，进位链不作为结果来源

这张表说明**控制信号本身也是硬件约定**。若 `op` 选择 SUB，但 `B_invert` 或最低位 `Cin` 没有同步改变，结果会变成错误的加法变体；若多位减法要生成借位、进位、零标志、符号标志或溢出标志，还要明确每个标志按哪一种数值解释计算。第一章不展开完整标志逻辑，只强调复用加法器并不意味着控制可以随意；控制位、数据路径和标志解释必须一起确定。

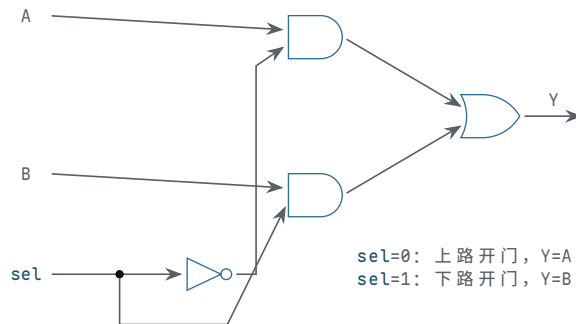
逐位逻辑操作路径最短：AND、OR、XOR 的本位输出只经过一两个门，主要受负载和选择器影响；它们适合逐位屏蔽、合并标志位、差异检测和翻转控制。加法路径则由全加器和进位链组成，最长路径常由进位传播决定。选择器一层看似透明，却是最容易被忽视的约定：`op` 必须稳定，不能让错误候选短暂输出到危险后级。这个切片模型提前解释了 CPU 数据通路里的三个常见事实。第一，位宽增加不只是“多写几个 bit”，而是复制更多切片，并重新处理进位、布线和负载。第二，控制信号不是附属注释；`op`、进位选择、结果写回选择都会直接改变哪条硬件路径有效。第三，ALU 的延迟不是单个门延迟，而是由候选路径、进位路径、选择器和输出负载共同决定。8086 的 16-bit 数据通路、80386 的 32-bit 数据通路，以及现代 64-bit 执行单元，都可以从这个切片观点继续放大的理解。

多路选择器解决“多个输入选一个输出”。2 选 1 选择器的行为是：

```
if sel = 0, Y = A
if sel = 1, Y = B
```

上述 `if` 只是描述行为，电路本身没有顺序执行。它的门级表达式是：

$Y = (\text{NOT } \text{sel} \text{ AND } A) \text{ OR } (\text{sel} \text{ AND } B)$



图示：2 选 1 选择器的门级实现：`sel` 直接控制下路 AND，反相后控制上路 AND——任何瞬间两路不会同时开门，输出永远有明确来源。选择器的“选择”不是程序分支，而是两扇由互补信号控制的门。

选择器的作用很基础。若一条输出线可能来自加法器，也可能来自传感器，也可能来自某个寄存器，那么必须有一个明确的选择信号决定哪一路接到输出。没有选择器，多路来源同时驱动同一条线就会冲突。

选择器的控制信号本身也需要检查。若 `sel` 悬空，输出可能在 A 和 B 之间随机变化；若选择信号来自毛刺严重的组合网络，输出可能短暂选择错误来源；若 A、B 属于不同电源域或不同稳定时间，选择器输出还会继承这些边界问题。因此，选择器不是“安全地把两根线接在一起”，而是“在一个可靠控制信号约束下，只让一路值成为输出”。

选择器检查项	问题	失败后果
选择信号默认值	上电或复位前选哪一路	输出来源不确定

数据输入稳定性	被选中的输入是否已稳定	选择了正确来源但值仍不可靠
未选输入行为	未选输入是否允许变化	不应通过内部耦合影响输出
输出负载	选择器是否能驱动后级	边沿变慢或电平不足

译码器做相反方向的事：把一个二进制编号展开成 one-hot 信号。2-bit 输入可以选中 4 条输出中的一条。

输入	输出
00	Y0=1, Y1/Y2/Y3=0
01	Y1=1, 其他为 0
10	Y2=1, 其他为 0
11	Y3=1, 其他为 0

选择器把多路收成一路，译码器把编号展开成多路选择。后面无论是寄存器阵列、数据通路，还是存储器地址选择，都会反复看到这两个结构。

选择器还可以直接实现布尔函数。若一个函数有两个输入 A、B，可以用 A 作为选择信号，选择器的两个数据输入分别接“当 A=0 时 Y 应该等于什么”和“当 A=1 时 Y 应该等于什么”。例如 XOR 中，当 A=0 时，Y=B；当 A=1 时，Y=NOT B。因此一个 2 选 1 选择器加一个反相器即可实现 XOR：

```
if A = 0: Y = B
if A = 1: Y = NOT B
Y = mux(A, B, NOT B)
```

这种读法对后面的数据通路很重要。ALU 输入选择、写回选择、下一 PC 选择，本质上都可以看成“控制信号选择哪一路值成为输出”。选择器不是附属小部件，而是把多种候选硬件动作收束成一个当前动作的基本结构。

多路复用器：树形扩展与任意函数实现 一个 2 选 1 选择器只是一粒种子。回看它的门级结构：一个反相器、两个 AND、一个 OR——成本固定。把 2 选 1 当积木按树形堆叠，就能得到任意规模的选择器。4 选 1 用 3 个 2 选 1 排成两级：第一级两个，由选择信号低位 S0 分别在 D0/D1、D2/D3 中各选一路；第二级一个，由高位 S1 在两路胜者中选最终输出：

```
w0 = mux(S0, D0, D1)
w1 = mux(S0, D2, D3)
Y = mux(S1, w0, w1)
```

逐一代入可验证：S1S0=00 时 Y=D0，01 时 Y=D1，10 时 Y=D2，11 时 Y=D3。同一规律继续放大：8 选 1 用 7 个 2 选 1 排成三级，16 选 1 用 15 个排成四级。一般地， 2^n 选 1 需要 $2^n - 1$ 个 2 选 1，共 n 级：

规模	2 选 1 个数	级数	结构读法
2 选 1	1	1	最小单元
4 选 1	3	2	第一级两个并行，第二级一个汇总
8 选 1	7	3	第一级四个并行，逐级减半
16 选 1	15	4	数据输入指数增长，级数只线性增长

这张表藏着一个工程读法：选择器规模翻倍，数据输入数量翻倍，但延迟只增加一级——树形结构把指数增长的数据宽度换成了线性增长的级数。布线时还有一个讲究：最晚稳定的那一路选择信号应放在最后一级，让先到的数据先在低级门里传播。这与前文把晚到信号放在 NAND 链最后一级是同一个原则。

多路复用器的第二个身份是通用函数发生器。一个 n 变量函数，可以把全部 n 个变量接到 2^n 选 1 的选择端，再把真值表逐行抄到数据端：第 k 个数据端接第

k 行的输出值。这样实现函数不需要任何化简——选择器本身就是一张接死的真值表。更省的办法是只把 $n-1$ 个变量接选择端，剩下的一个变量以四种形态之一出现在数据端：0、1、变量本身、变量的反。以三变量奇校验函数 $F(A,B,C)$ (A、B、C 中有奇数个 1 时输出 1) 为例，先用 8 选 1 实现，选择端 S2S1S0 接 A、B、C：

A B C	最小项	F	数据端接法
0 0 0	m0	0	D0 接 0
0 0 1	m1	1	D1 接 1
0 1 0	m2	1	D2 接 1
0 1 1	m3	0	D3 接 0
1 0 0	m4	1	D4 接 1
1 0 1	m5	0	D5 接 0
1 1 0	m6	0	D6 接 0
1 1 1	m7	1	D7 接 1

再省一级：只把 A、B 接 4 选 1 的选择端，数据端改成 C 的函数。逐行检查每个 A、B 组合下 F 随 C 的变化：

A B	C=0 时 F	C=1 时 F	数据端接法
0 0	0	1	D0 接 C
0 1	1	0	D1 接 NOT C
1 0	1	0	D2 接 NOT C
1 1	0	1	D3 接 C

例如 A=0、B=1 时：C=0 对应最小项 m2 输出 1，C=1 对应 m3 输出 0，F 恰好等于 NOT C，于是 D1 接反相后的 C——其余三行同理。一片 4 选 1 加一个反相器，就装下了整张八行真值表。

这个身份的意义远超节省门数。可编程逻辑器件中的查找表，本质就是把数据端换成可写存储单元的多路复用器：改写存储单元，就改写了函数。读法上也要注意代价的另一面：用选择器实现函数是机械的，它不利用函数内部结构——八行真值表和八个随机值，对 8 选 1 来说是同一个价格。化简的机会被换成了实现的整齐。

选择器用集中的逻辑裁决“谁成为输出”，前提是所有候选来源都汇聚到一处。当候选来源分散在不同芯片、不同板卡上时，还有另一种组织方式：让每个来源挂在同一条线上，各自决定何时驱动、何时松手。接下来的两个专题分别考察这两种方式的两种输出级：三态与开漏。

三态输出与总线争用 到目前为止，每个输出级都遵循同一约定：任何时刻要么强驱动高电平，要么强驱动低电平，绝不允许悬空，也绝不允许上下同时导通。三态输出在这个约定上增加第三个合法状态：高阻态。使能端无效时，输出级的上拉管和下拉管同时关断，输出端既不供出电流也不吸收电流，对外表现为“像没接一样”——它不是 0，也不是 1，而是退出。使能端有效时，它就是一个普通的推挽输出。按惯例，使能常做成低有效，记为 OE_n ，与前文低有效信号约定一致：

OE_n	D	Y	输出级状态
0	0	0	下拉管导通，强驱动低电平
0	1	1	上拉管导通，强驱动高电平
1	任意	Z	两管都关断，高阻，退出驱动

高阻态的价值在“共享”。若加法器、寄存器、外部接口都要把结果送到同一条数据线上，集中式选择器要求所有来源汇聚到一点；而给每个来源配一个三态输出，让它们平时处于高阻、轮到自己时才使能，同一条线就能被多个模块分时使用——这就是总线。总线能成立，靠的是一条硬约定：同一时刻最多一个驱动者。使能信号必须互斥，通常由译码器产生的 one-hot 或专门的仲裁逻辑保证。

违反这条约定的后果值得用电流算一遍。设两个三态驱动器同时使能，一个想推高、一个想拉低：电流从电源出发，经前者的上拉管、总线、后者的下拉管直到地，形成一条低阻通路。以每个输出级导通电阻约 25Ω 估算，这条通路上的电流约为 $5V/(25\Omega+25\Omega)=100\text{mA}$ ——比正常信号电流大一个数量级以上。此时总线电压停在中点附近，接收端读到的既不是合法高电平也不是合法低电平；大电流持续流过输出级，带来发热，反复或长时间争用会损坏器件。即使两个驱动器想驱动同一电平，超出额定的并联驱动也会使电平和边沿偏离数据手册的保证值。这就是总线争用：它不是逻辑错误——逻辑式里两个驱动者的值可以都正确；它是电气事故。

争用是一个方向的故障，另一个方向的故障是无人驱动。若某一时刻所有使能都无效，总线悬空，电压会被漏电和耦合慢慢推走——这正是本章开篇悬空按钮问题的总线版本。工程上常用总线保持器兜底：它是挂在总线上的一个弱反馈锁存，最后一个驱动器留下什么电平，它就以微弱电流维持什么电平；新驱动器接管时，其强驱动轻易盖过这个弱保持，总线正常翻转。注意分工：总线保持器解决“无人驱动时漂移”，完全不解决“多人驱动时争用”——使能互斥仍是设计必须证明的性质，不能交给器件去兜底。

关键问题	预期结果
使能是否互斥	任何时刻至多一个 OE_n 有效，由译码器 one-hot 或仲裁逻辑保证
驱动器切换间隙 无人驱动时	旧驱动器先进入高阻，新驱动器再使能，留出交断时间 总线保持器或上拉/下拉电阻给出确定电平，不悬空
上电与复位期间	所有 OE_n 默认无效，总线默认空闲，不误导后级

后文讲整机时会看到，存储器、外设和 CPU 的数据引脚几乎都是三态的；一张原理图上几十根数据线的和平共处，靠的就是这里写下的互斥约定。

开漏输出与线与 三态输出的思路是“要么全力驱动，要么彻底退出”。开漏输出走另一条路：它只保留下拉管，干脆没有上拉管。输出管导通时，线被强拉到 0；输出管关断时，驱动器松手——注意不是输出 1，而是什么都不做。高电平由线路上唯一的公共上拉电阻提供：所有驱动器都松手后，电流经电阻向线路电容充电，电压回升到电源。

这个结构天然允许一条线挂多个驱动器。任何一个驱动器拉低，全线就是 0；只有全部松手，线才回到 1。把“松手”读成赞成、“拉低”读成否决，线上的电平就是全体的 AND——没有门的实体，连线本身完成了与运算，所以叫线与。与三态方案对比，差别很清楚：三态要求使能互斥，违反就是电源到地的争用事故；开漏不存在这个事故，最坏情况只是多个驱动器同时拉低，线依然可靠为 0。代价集中在高电平一侧：上升沿靠电阻慢慢充电，快不起来；拉低期间电阻上持续流过电流，白白耗电。

I2C 总线的 SDA 和 SCL 就是这个模型。每个设备都只能拉低或松手，起始、停止、应答全部是“谁在何时拉低”的约定；从机处理不过来时按住 SCL 不放，主机发现时钟线升不回去，就知道必须等待——这就是时钟延展。这类总线的动作电平因此都是低有效，与前文低有效信号约定出自同一个电气原因：低电平是有源强驱动，可靠且谁都能发起；高电平是电阻慢慢充出来的，只配当默认态。

上拉电阻的取值是开漏设计的主要权衡，它同时卡在两个约束之间。上升沿一侧：时间常数 $\tau = R \cdot C_{\text{bus}}$ ，电阻越大，回到高电平越慢。拉低一侧：电流 $I = V_{\text{DD}}/R$ ，电阻越小，拉低时耗电越多，且驱动器的吸收电流能力有限，电流超限会把低电平抬高、侵蚀低电平余量。以 $V_{\text{DD}} = 3.3\text{V}$ 、总线电容 100pF 为例：

上拉电阻	拉低电流 V_{DD}/R	时间常数 $\tau = R \cdot C$	适用场合
10 k Ω	0.33 mA	1000 ns	低速、省电优先的场合
4.7 k Ω	0.70 mA	470 ns	I2C 标准模式 (100 kHz) 常见取值

2.2 kΩ	1.5 mA	220 ns	I2C 快速模式 (400 kHz)
			常见取值
1.0 kΩ	3.3 mA	100 ns	边沿更快，但已接近常见的 3 mA 拉低限值

电压升到约 70% 需要 1.2τ ：4.7 kΩ 时约 560 ns，尚在标准模式对上升沿的容限之内；总线电容更大或速率更高，就必须减小电阻，或者改用有源上拉电路。读表的方向不要反：不是“电阻越小越好”，而是在上升沿速度和拉低电流之间取交点。

到这里，三种输出级形态可以放在一起读：推挽输出任何时候都独占线路；三态输出靠互斥使能分时共享；开漏输出靠“只能拉低”让多驱动者和平共存。它们回答的是同一个问题——谁驱动这条线、何时驱动、没人驱动时谁来兜底。后文译码器产生的互斥 one-hot 信号，正是三态总线最常见的值班表；而开漏线把“兜底”直接做进了电气结构里。

译码器则常用于“编号变成单独使能”。例如 2-bit 编号选择 4 个寄存器中的一个写入，译码器输出 one-hot 写使能：同一时刻只允许一个目标被写。若译码器错误地产生两个 1，就可能同时写两个寄存器；若所有输出都是 0，本周期写操作会消失。后面寄存器阵列和控制器都会反复依赖这个性质。

结构	典型用途	必须保证
选择器	多个来源选一个输出	同一时刻只有一路被选中成为结果
译码器	编号展开成 one-hot 使能	合法编号只激活一个目标
编码器	多个请求压缩成编号	多请求同时有效时要有优先级或报错
比较器	判断相等、大小或条件成立	输入解释方式必须明确

编码器和比较器也是常见组合电路。编码器把 one-hot 或优先级请求压缩成编号；比较器判断两个值是否相等，或者按某种解释比较大小。安全启动控制板可以用简单编码器把故障来源压缩成故障码：没有故障为 00，温度故障为 01，电流故障为 10，若两者同时故障则输出优先级最高的一项或输出“多故障”码。关键在于规则必须写清楚，不能让两个输入同时为 1 时输出处于未定义状态。

temp_alarm	current_alarm	故障码	说明
0	0	00	无故障
1	0	01	温度故障
0	1	10	电流故障
1	1	11	多故障或最高优先级故障

若系统选择优先级编码，就要把优先级写进规格。例如温度故障比电流故障优先，则两个报警同时为 1 时输出温度故障码；若系统更重视诊断完整性，则两个报警同时为 1 时输出多故障码。二者没有绝对优劣，但不能在电路里含糊。

编码策略	规则	适合场景
优先级编码	多个请求同时有效时选择最高优先级	中断控制、快速响应最重要的请求
多故障编码	多个请求同时有效时输出专用多故障码	故障诊断和维护记录
非法状态报警	非 one-hot 输入触发错误输出	希望尽早暴露编码器前级错误

译码器、编码器与优先编码器 前文把译码器和编码器各用一段话带过，这两种

结构值得展开，因为它们是“编号”与“选择”之间往返转换的标准件。

译码方向先一般化： n 位输入、 2^n 条输出，第 k 条输出正好是最小项 m_k ——它只在输入等于 k 时为 1。前文的 2 到 4 译码器是 $n=2$ 的特例；常见的 3 到 8 译码器（74HC138 一类）还带使能端，使能无效时所有输出一律无效。使能端给了译码器两个能力：级联，以及给片选加一个总开关。译码器的关键性质是输出互斥：合法输入下恰好一条输出有效。这个 one-hot 性质正好满足三态总线和寄存器写使能要求的“唯一驱动者”——译码器因此是“地址变片选”的通用件，存储器选体、外设选择、寄存器堆写入都靠它。

编码器走反方向： 2^n 条 one-hot 输入，输出 n 位编号，即“第几条线有效”。普通编码器要求输入严格 one-hot；若两条输入同时有效，输出没有定义。这不是电路的缺陷，而是输入约定被打破——编码器只承诺在约定之内给出正确编号。

优先编码器把这条约定放宽：允许多个输入同时有效，优先级最高者胜出，并额外给出一个标志 G ，表示“当前至少有一个请求有效”。以 4 到 2 优先编码器为例，约定 I_3 优先级最高、 I_0 最低：

I_3	I_2	I_1	I_0	Y_1	Y_0	G	说明
0	0	0	0	0	0	0	无请求， $G=0$
0	0	0	1	0	0	1	只有 I_0 请求
0	0	1	X	0	1	1	I_1 在场， I_0 被压住
0	1	X	X	1	0	1	I_2 在场，低两位被压住
1	X	X	X	1	1	1	I_3 一票定音

表中 X 表示该输入不影响本行输出。逐列读出布尔式：

```
Y1 = I3 OR I2
Y0 = I3 OR (NOT I2 AND I1)
G  = I3 OR I2 OR I1 OR I0
```

验算两行： $I_2=1$ 、 $I_3=0$ 时， $Y_1=1$ 、 $Y_0=0$ ，编号 10，且无论 I_1 、 I_0 如何都不变——这正是“压住”的含义；只有 $I_1=1$ 时， $Y_1=0$ 、 $Y_0=1$ ，编号 01。 G 与编号相互独立：全 0 输入时编号也是 00，全靠 G 区分“有请求、编号为 00”和“根本没有请求”。

优先编码器是中断请求排队的雏形。多个设备共用一路请求通道时，同时请求是常态而不是异常；固定优先级的编码器先报出最高优先级设备的编号， G 则直接充当“有待处理请求”的总信号。后文中断控制器会在此基础上增加屏蔽、嵌套和可编程优先级，但裁决的第一步就是这里这几行式子。

两种结构的级联也值得对照。译码器靠使能端扩展：两个 3 到 8 译码器，地址最高位 A_3 直接接高位片的使能、反相后接低位片的使能，就得到 4 到 16 译码，输出仍然互斥。优先编码器靠 G 串联扩展：高位片的 G 决定低位片的编号是否被采用，全局优先级仍然唯一。选择器的树形扩展、译码器的使能级联、优先编码器的 G 串联，是同一个思想的三种形态：用少量控制端，把“同一时刻只有一个胜者”的性质扩展到大网络。

比较器也必须说明数值解释。同样的 bit 模式，用无符号数解释和用补码有符号数解释，大小关系可能不同。以 4-bit 为例，1111 作为无符号数是 15；作为补码有符号数是 -1。若比较器输出 less_than，规格必须说明它比较的是无符号大小、有符号大小，还是只比较 bit 模式是否相等。

比较类型	示例	说明
相等比较	$A == B$	只关心每一位是否相同，通常不涉及数值解释
无符号大小	$1111 > 0001$	1111 表示 15
有符号大小	$1111 < 0001$	1111 表示 -1，0001 表示 1
条件比较	报警级别是否超过阈值	必须说明编码范围和非法值处理

数值比较器与四变量卡诺图案例 相等比较只问“每位是否相同”，数值比较器还要多问一句“谁大”。裁决规则与字典序相同：从最高位开始逐位比较，第一对不同的位决定全局结果，更低的位不再有机会发言。以 4 位无符号数为例，先逐位求相等信号 $eq_i = A_i \text{ XNOR } B_i$ ，再让高位的一票定音权逐级传递：

```
gt = (A3 AND NOT B3)
    OR (eq3 AND A2 AND NOT B2)
    OR (eq3 AND eq2 AND A1 AND NOT B1)
    OR (eq3 AND eq2 AND eq1 AND A0 AND NOT B0)

eq = eq3 AND eq2 AND eq1 AND eq0
lt = 与 gt 对称，A、B 互换即可
```

读 gt 的每一项：第一项说“最高位就分出胜负”；第二项说“最高位打平，则由次高位裁决”，依此类推。 eq_i 在这里身兼两职——既是相等比较的输出，又是大小比较中“把裁决权交给下一位”的通行证。这与逐位进位链在结构上同构：都是高位优先、逐级传递，级数同样随位宽线性增长。

单片 4 位比较器（74HC85 一类）还带三个级联输入： $I(A>B)$ 、 $I(A=B)$ 、 $I(A<B)$ 。它们的作用是：当本片四位全部打平时，比较结果不由本片决定，而是透传级联输入。把低位片的三个输出接到高位片的级联输入，两片就串成 8 位比较器，四片串成 16 位——裁决权从最低位一路传递到最高位，与式子里的 eq 链严格对应。有符号与无符号的差别集中在最高有效位的解释上（补码的最高位是负权），级联结构本身不变。

比较器用到的“相等信号加传递链”是一种标准手法。下面换一个完整演算，把前文只在文字层面讲过的卡诺图，在一个含无关项的四变量函数上走完全程。题目：检测 8421 BCD 码是否大于等于 5。输入 A、B、C、D（A 为最高位），按约定只会出现 0 到 9；10 到 15 是无关项，记为 d。

先列完整真值表，一行都不跳：

十进制	A B C D	F	说明
0	0 0 0 0	0	小于 5
1	0 0 0 1	0	小于 5
2	0 0 1 0	0	小于 5
3	0 0 1 1	0	小于 5
4	0 1 0 0	0	小于 5
5	0 1 0 1	1	大于等于 5
6	0 1 1 0	1	大于等于 5
7	0 1 1 1	1	大于等于 5
8	1 0 0 0	1	大于等于 5
9	1 0 0 1	1	大于等于 5
10	1 0 1 0	d	无关项：约定不会出现
11	1 0 1 1	d	无关项
12	1 1 0 0	d	无关项
13	1 1 0 1	d	无关项
14	1 1 1 0	d	无关项
15	1 1 1 1	d	无关项

再填四变量卡诺图。A、B 沿行方向按 Gray 码排列，C、D 沿列方向按 Gray 码排列，每个格子填入对应最小项的函数值：

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	1	1

11	d	d	d	d
10	1	1	d	d

圈组的规则只有两条：每个圈必须是 2^k 个相邻格子组成的矩形，且每个 1 至少被一个圈覆盖；无关项可以当 1 圈进去，也可以当 0 留在圈外，取舍的唯一标准是能否把圈撑得更大。按这个标准可以圈出三组。第一组：A=1 的下面两行共 8 格（m8 到 m15），六个无关项全部圈入，得到单文字项 A—B、C、D 在圈内部取遍两种值，全部消去。第二组：B=1 且 C=1 的四格（m6、m7、m14、m15），其中两格是无关项，得到 BC。第三组：B=1 且 D=1 的四格（m5、m7、m13、m15），同样借两个无关项撑大，得到 BD。三个圈合起来：

$$F = A \text{ OR } (B \text{ AND } C) \text{ OR } (B \text{ AND } D) = A \text{ OR } (B \text{ AND } (C \text{ OR } D))$$

逐行验算：5=0101 由 BD 覆盖，6=0110 由 BC 覆盖，7 同时落在三个圈里（重复覆盖不改变函数），8、9 由 A 覆盖；0 到 4 各行 A=0，且 B、C、D 不满足任何一项，输出 0。全部符合真值表。

为什么这样选圈，而不是逐对合并？对照不用无关项的化简就清楚了。若把 d 全部当 0，m8、m9 只能两两合并成 $A \text{ AND } (\text{NOT } B) \text{ AND } (\text{NOT } C)$ ，m5、m7 合并成 $(\text{NOT } A) \text{ AND } B \text{ AND } D$ ，m6、m7 合并成 $(\text{NOT } A) \text{ AND } B \text{ AND } C$ ——同样是三个乘积项，但每项有三个文字，合计 9 个文字；借入无关项后，三项合计只有 5 个文字，门更小、扇入更低。无关项的价值就在这里：它不增加任何需要覆盖的 1，却允许把已有的 1 圈进更大的组里。

最后必须重申前文对不关心项的警告。这个化简把 10 到 15 实现成了 1——在“输入只会是合法 BCD 码”的约定下无害；一旦输入可能来自外部连接器或故障状态，这个约定就可能被打破，届时 d 必须改回确定值重新化简。化简服从系统约定，而不是相反。

现在把贯穿案例写成一个完整组合网络。安全启动控制板至少需要两个输出：fault_lamp 和 allow_start。故障灯由报警信号决定；启动允许由启动按钮、安全条件和无故障共同决定。

```
fault_lamp = temp_alarm OR current_alarm
no_fault = NOT fault_lamp
allow_start = start_button AND cover_closed
              AND emergency_ok AND no_fault
```

这组表达式可以继续展开成门网络。第一层 OR 汇总故障信号；第二层 NOT 得到无故障条件；第三层多输入 AND 汇总启动按钮和安全条件。若硬件库里没有四输入 AND，可以用二输入 AND 逐级合成：

```
safe_chain = cover_closed AND emergency_ok
request_ok = start_button AND no_fault
allow_start = safe_chain AND request_ok
```

这种重写在逻辑上等价，但硬件上改变了门级分组。若 cover_closed 和 emergency_ok 来自同一个连接器，先合成 safe_chain 可能便于布线；若 fault_lamp 驱动多个后级，no_fault 前后可能需要缓冲。组合逻辑的表达式只是第一层结果，门级分组、负载和时序仍要继续检查。

若器件库只允许二输入 NAND，还可以把同一逻辑展开为 NAND-only 网络。每个 AND 由“一次 NAND 得到反相信号，再用 NAND 自反相恢复”为正逻辑。中间的反相信号必须命名清楚，否则很容易在后续维护中把 safe_chain_n 当作 safe_chain 使用。

```
n1 = cover_closed NAND emergency_ok
safe_chain = n1 NAND n1
```

```
n2 = start_request NAND no_fault
request_ok = n2 NAND n2

n3 = safe_chain NAND request_ok
allow_start = n3 NAND n3
```

NAND-only 写法证明了功能可实现，但不自动证明它就是最好的实现。线性串接、平衡二输入树和 NAND-only 分解在稳定真值表上可以等价，门级层数、局部负载、中间节点数量和毛刺形态却可能不同。平衡树通常缩短最长逻辑层数，线性结构可能更便于让某个晚到信号放在最后一级，NAND-only 结构可能更符合某些标准门芯片资源。正式规格应保留可审查的安全表达式，再单独说明实际门级映射，避免把实现技巧误写成需求本身。

顺这条链再检查一遍内部信号：`fault_lamp` 表示任一报警成立，OR 路径要能驱动指示灯和后级逻辑；`no_fault` 是它的反相，必须由反相器恢复为强 0/1；`safe_chain` 要求保护盖和急停都满足，任一断开时默认禁止；`request_ok` 要求有启动请求且无故障，按钮抖动不能传到后级；`allow_start` 是全部条件的汇合，后级何时采样、有无毛刺风险都要写明；`motor_enable_cmd` 是发给执行机构的命令，必须经过时序边界或专用安全链路，不能直接把组合输出接出去。

组合逻辑设计到这里仍然没有引入“步骤执行”。所有输入同时作用于网络，所有中间信号同时传播。若 `temp_alarm` 从 0 变 1，`fault_lamp` 会在 OR 路径延迟后变为 1，`no_fault` 会在反相器延迟后变为 0，`allow_start` 会在 AND 路径延迟后关闭。这个传播链条不是程序调用栈，而是多条电气路径的延迟叠加。

`motor_enable_cmd` 需要比 `allow_start` 更谨慎。前者是给外部执行机构的命令，后者只是内部组合判断。直接令 `motor_enable_cmd = allow_start` 在最小示例里看似合理，但正式系统通常会增加时序边界、复位保持、驱动级和独立停机链路。原因很简单：组合判断可以短暂过渡，执行机构不应响应这种过渡。

```
allow_start = start_request AND cover_closed
              AND emergency_ok AND no_fault

motor_enable_cmd = registered_allow_start AND power_stage_ready
```

上式中的 `registered_allow_start` 表示已经在时钟边界保存过的允许结果，`power_stage_ready` 表示功率级自身已准备好并未报告故障。第一章还没有正式引入触发器，所以这里只把它作为边界预告：组合逻辑负责给出当前条件下的判断，执行机构命令通常还需要由时序逻辑和驱动电路接管。这样组织，能把“判断是否允许”和“真正驱动负载”分成两个可独立检查的层次。

可以把安全启动控制板整理成一份组合规格。规格不必一开始就写门级细节，但必须把输入语义、输出语义和异常策略写清楚：输入前提是板外触点已经完成默认值、电平和消抖处理，原始触点不直接进入安全组合式；允许条件是启动请求、保护盖闭合、急停正常、无故障四者缺一不可；故障输出要求温度或电流报警都点亮故障灯，同时报警要有明确编码策略；默认安全要求未知、断线、非法编码一律倒向禁止启动，化简逻辑不得破坏这条策略；使用时刻要求后级只在信号稳定后使用，异步使能必须额外做毛刺检查。这样做的好处是，之后无论用分立门、可编程逻辑，还是把同一逻辑放进更大的控制器，行为边界都保持一致。

下面给出这类小组合模块的完整书写顺序。它看起来比直接写一个公式更长，但每一步都对应一种常见错误：信号没定义、非法状态没处理、公式和需求不一致、实现后电气条件不满足。第一步，列出原始输入、内部语义信号和输出，不把物理触点直接当作可靠 bit。第二步，说明每个信号的有效电平和默认值，避免低有效、断线和上电默认错误。第三步，写出正常行为和异常行为，不让非法组合默默进入允许状态。第四步，写逻辑表达式或真值表，让需求可以机械检查。第五步，映射到门、选择器、译码器或编码器，明确驱动来源和选择关系。第六步，检查延迟、负载、毛刺、电平余量，证明实物能按时产生可靠输出。第七步，记录测量或仿真结

果，让设计从文字规格变成可验证的对象。

把这七步用于 `allow_start`，可以得到更严格的文字规格：输入必须已经完成调理；`start_request` 为 1 表示一次经过消抖或同步确认的启动请求，缺失时不启动；`cover_closed` 为 1 表示保护盖闭合且传感器链路有效，缺失时不启动；`emergency_ok` 为 1 表示急停链路处于允许运行状态，缺失时不启动或停机；`no_fault` 为 1 表示没有温度或电流报警，缺失时不启动并报警；组合路径已越过最坏延迟，输出才算稳定，此前后级暂不采样、也不驱动执行机构。

若某个条件缺失却仍能启动，就是需求错误或实现错误；若输出尚未稳定就驱动执行机构，就是时序边界错误。组合逻辑的严肃性正在于此：它没有软件中的异常处理栈，只有电路在每一种输入情况下实际产生的电平。

为了让贯穿案例形成闭环，可以把安全启动控制板从外部物理输入一路写到执行命令。该表不是新增功能，而是把前面分散的原则合成一条可审查链路：原始线缆先进入电气边界，调理后得到内部语义信号，组合网络只处理已经可靠的 bit，危险输出再经过时序和驱动边界。

层次	典型对象	约定要求
原始输入	<code>start_button_raw</code> 、传感器触点、报警线	允许抖动和噪声，但必须有默认路径、保护和极性说明
输入调理	上拉/下拉、滤波、施密特触发、电平转换	把板外现象整理成稳定内部电平，不让悬空和慢边沿直接进入核心
语义信号	<code>start_request</code> 、 <code>cover_closed</code> 、 <code>emergency_ok</code>	每个 0/1 含义清楚，低有效信号先转换为可审查语义
组合判断	<code>fault_lamp</code> 、 <code>no_fault</code> 、 <code>allow_start</code>	真值表覆盖正常、异常、非法和默认状态，化简不破坏安全策略
时序边界	<code>registered_allow_start</code> 、复位保持、采样时刻	只在组合路径稳定后保存或使用结果，避免毛刺进入危险动作
输出驱动	<code>motor_enable_cmd</code> 、指示灯驱动、功率级使能	具备足够驱动能力，上电和复位期间保持禁止，故障时进入安全状态
验证记录	原理图、数据手册、示波器、逻辑分析仪、测试矩阵	说明电平、延迟、负载、毛刺和异常输入处理都满足约定

这张闭环表也说明为什么第一章要同时讲电平、器件、门和组合逻辑。如果只看组合表达式，会把 `start_request` 当作天然可靠的变量；如果只看器件，又难以说明为何这些输入最终必须合成为一个 `allow_start`。正式硬件说明应当让两者互相校验：每个语义信号都能追溯到电气边界，每个组合输出都能追溯到真值表和电流路径，每个执行命令都能追溯到时序和驱动电路。

端到端示例：用两片 74HC00 实现安全启动控制板 前面所有片段—真值表、NAND-only 展开、芯片引脚、噪声余量—现在可以合成一次完整演练。目标是让 `allow_start` 从自然语言需求一路走到可焊接的引脚分配。

第一步，需求表达式（安全语义层，保持可审查）：

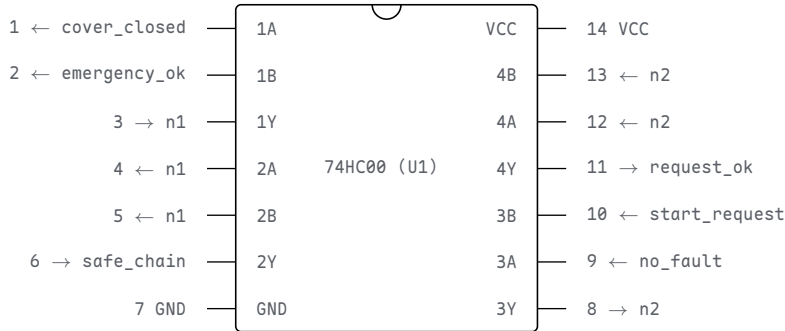
```
fault_lamp = temp_alarm OR current_alarm
no_fault = NOT fault_lamp
allow_start = start_request AND cover_closed
              AND emergency_ok AND no_fault
```

第二步，NAND-only 展开（实现层，每个反相中间信号命名清楚）。`fault_lamp` 与 `no_fault` 用一路 OR 芯片（如 74HC32）或二极管 OR 加缓冲实现即可，这里聚

焦 AND 链：

```
n1 = cover_closed NAND emergency_ok
safe_chain = n1 NAND n1
n2 = start_request NAND no_fault
request_ok = n2 NAND n2
n3 = safe_chain NAND request_ok
allow_start = n3 NAND n3
```

第三步，数门。共 6 个二输入 NAND；一片 74HC00 有 4 路，需要两片。74HC00 的封装是 14 脚双列直插：左边 1-7 脚，右边 14-8 脚，缺口朝上时左下角是 1 脚。



图示：第一片 74HC00 的引脚分配：门 1 产生 `n1`，门 2 把 `n1` 自反相得到 `safe_chain`，门 3 产生 `n2`，门 4 把 `n2` 自反相得到 `request_ok`。缺口（上方半圆）决定引脚编号方向—原理图和实物靠它对应。

第二片（U2）完成最后两级：门 1 计算 `n3 = safe_chain NAND request_ok`，门 2 自反相得到 `allow_start`；门 3、门 4 未使用。未使用的 CMOS 输入必须接确定电平（例如都接 VCC），否则悬空输入会让输出级随机翻转、白白耗电。

资源	分配	检查点
U1 门 1-2	<code>cover_closed</code> 、 <code>emergency_ok</code> 合成 <code>safe_chain</code>	中间信号 <code>n1</code> 是反相电平，不可误用
U1 门 3-4	<code>start_request</code> 、 <code>no_fault</code> 合成 <code>request_ok</code>	<code>no_fault</code> 来自故障汇总级
U2 门 1-2	<code>safe_chain</code> 、 <code>request_ok</code> 合成 <code>allow_start</code>	输出送时序边界，不直接驱动执行机构
U2 门 3-4	未使用	输入接确定电平，输出悬空不理
电源	14 脚 VCC、7 脚 GND，两片各自就近 $0.1\mu\text{F}$ 去耦	去耦电容贴近电源脚

第四步，电气参数核算。以 74HC00 数据手册在 4.5V 供电下的保证值为例：输出高电平最差 $V_{OH} = 4.4\text{V}$ （额定负载电流内），输出低电平最差 $V_{OL} = 0.33\text{V}$ ；输入高阈值 $V_{IH} = 3.15\text{V}$ ，输入低阈值 $V_{IL} = 1.35\text{V}$ 。逐级核算：

检查项	算式	结论
高电平余量	$4.4\text{V} - 3.15\text{V}$	1.25V，合格
低电平余量	$1.35\text{V} - 0.33\text{V}$	约 1.0V，合格
级联三级后	每级输出都重新驱动到 $4.4\text{V}/0.33\text{V}$	CMOS 恢复电平，余量不逐段衰减

注意第三行与二极管 OR 的本质差别：二极管每级吃掉 0.7V，三级后余量所剩无几；CMOS 门每级都把电平重新驱动到接近电源和地，余量被恢复——这就是为什么通用逻辑网络必须用有源门，而不是二极管逻辑。

至此，`allow_start` 可以回答本章开头的全部追问：需求来自真值表，实现落在两片具体芯片的 12 个引脚上，电气有余量核算，时序上它只送到时序边界而不

是执行机构。这个从自然语言到引脚的完整链条，就是本章要达到的目标。

端到端示例二：三级报警优先权电路 上一例回答的是“所有安全条件同时成立才允许动作”，还有另一类同样常见的需求：多个请求可能同时到来，必须按优先级裁决，高优先级抢占输出并屏蔽低优先级。本例用一片 74HC04、一片 74HC00 和一片 74HC32，把一台设备的三级报警面板从需求走到芯片分配，流程与上一例完全相同。

需求如下。面板有三个请求源：火警 `fire`（最高优先级）、故障 `fault`、提示 `info`（最低优先级），对应三盏指示灯 `fire_lamp`、`fault_lamp`、`info_lamp`。任一时刻只允许点亮当前最高优先级的那一盏：高优先级请求出现时立即抢占，并把低优先级输出强制为 0。注意屏蔽不是“盖住显示”，而是让低优先级输出在逻辑上就是 0——这样无论后级接的是灯、蜂鸣器还是记录电路，看到的都是被裁决过的结果。全部请求消失时三盏都熄灭。

先列完整真值表。三个请求共八种组合，裁决结果如下：

请求 (fire, fault, info)	输出 (fire/fault/info_lamp)	裁决结果
0, 0, 0	0, 0, 0	无请求，全部熄灭
0, 0, 1	0, 0, 1	只有提示
0, 1, 0	0, 1, 0	故障点亮，提示未请求
0, 1, 1	0, 1, 0	故障屏蔽提示
1, 0, 0	1, 0, 0	火警独占
1, 0, 1	1, 0, 0	火警屏蔽提示
1, 1, 0	1, 0, 0	火警屏蔽故障
1, 1, 1	1, 0, 0	火警屏蔽全部

从真值表直接读出化简结果。`fire_lamp` 在 `fire=1` 的四行全为 1，与另两个输入无关，所以 `fire_lamp = fire`——最高优先级输出不经过任何门，路径最短，这正符合它的安全地位。`fault_lamp` 只在 (0,1,0) 与 (0,1,1) 两行为 1，化简得 `fault AND (NOT fire)`：火警的反相信号作为一个输入项，它一来，故障灯就被逻辑本身关断。`info_lamp` 只在 (0,0,1) 一行为 1，即 `info AND (NOT fire) AND (NOT fault)`；用德摩根律改写成 `info AND NOT(fire OR fault)`，可以省下一级三输入 AND：

```

fire_n      = NOT fire
m1          = fault NAND fire_n    -- 74HC00 门1
fault_lamp  = NOT m1               -- 74HC04
any_hi      = fire OR fault        -- 74HC32 门1
any_hi_n    = NOT any_hi           -- 74HC04
m2          = info NAND any_hi_n   -- 74HC00 门2
info_lamp   = NOT m2              -- 74HC04
any_alarm   = any_hi OR info       -- 74HC32 门2，校验输出
fire_lamp   = fire                -- 直连，不经门

```

芯片分配如下。74HC04 (U1) 用四路反相器，产生 `fire_n`、`fault_lamp`、`any_hi_n`、`info_lamp`；74HC00 (U2) 用两路 NAND，产生 `m1`、`m2`；74HC32 (U3) 用两路 OR，产生 `any_hi`、`any_alarm`。U1 门 5-6、U2 门 3-4、U3 门 3-4 未使用，输入一律接确定电平。其中 `any_alarm` 是故意多算的校验输出：它在逻辑上应恒等于 `fire OR fault OR info`，也应恒等于三盏灯输出的 OR——同一个事实沿两条独立路径各算一遍，上电测试时逐行比对，就是一种廉价的一致性检查。

资源	分配	检查点
U1 (74HC04) 门 1-4	<code>fire_n</code> 、 <code>fault_lamp</code> 、 <code>any_hi_n</code> 、 <code>info_lamp</code>	门 5-6 未用，输入接 确定电平

U2 (74HC00)	产生 m1、m2	门 3-4 未用，输入接确定电平
门 1-2		
U3 (74HC32)	产生 any_hi、any_alarm	门 3-4 未用，输入接确定电平
门 1-2		
fire 直连	fire_lamp 不经门	最高优先级路径最短；灯的电流由驱动级承担，不经过逻辑门
裁决延迟	fire 到 fault_lamp 三级门、到 info_lamp 四级门	延迟沿路径逐级累加，定量核算见第五节
电源	每片 14 脚 VCC、7 脚 GND，就近 0.1 μ F 去耦	去耦电容贴近电源脚，理由见第五节专题
一致性	any_alarm 与三灯输出逐行比对	两条独立路径的结果必须一致

与上一例对照可以看到两点。第一，优先级不是额外的器件，而是写在真值表里的屏蔽关系：高优先级输入以反相形式出现在低优先级的乘积项里，它一来，低优先级输出就被逻辑本身关断。第二，优先级越高的输出经过的门越少——fire_lamp 零级、fault_lamp 最多三级、info_lamp 最多四级——“越重要的信号越快”在这里是自然结果，不需要刻意安排。至于三级门到底慢到什么程度，正是第五节要算的账。

把同一思路放大到 8086 和 80386，就能看出早期处理器为什么需要大量组合模块。以 Intel 历史速查表所列参数为例，8086 于 1978 年引入，约 29,000 个晶体管，采用约 3 微米制造技术；Intel386 DX 于 1985 年引入，约 275,000 个晶体管，制造技术从约 1.5 微米进入后续 1 微米级别。这些数字本身不是本章重点，重点在于它们对应的结构变化：晶体管数量增加以后，可以容纳更宽的数据通路、更复杂的译码、更丰富的寄存器和更强的总线控制；工艺尺寸缩小以后，门和连线的寄生电容下降，单位距离连接更短，许多路径的延迟随之降低。

芯片	年代与规模	本章可观察的硬件变化	对时延的意义
8086	1978 年，约 29,000 晶体管	16-bit 数据通路、指令译码、总线接口	已经需要大量组合控制与寄存器边界配合
80386	1985 年，约 275,000 晶体管	32-bit 数据通路、更复杂保护和地址机制	更多片上逻辑减少外部协作等待，关键路径需重新组织
现代 CPU	数十亿级晶体管	多级缓存、乱序执行、预测、片上互连	不只让门更快，还减少取数和等待的次数

这里要避免一个误解：现代芯片低时延不是因为“逻辑门不再有延迟”。门仍然有延迟，连线仍然有电容，存储器仍然需要访问时间。现代芯片做的是把延迟压低、隐藏、分层和局部化——这四个动词的具体含义放在本章第五节末尾展开，这里先记住结论：低时延来自让常见情况少走远路，而不是让所有路径同样快。

五、延迟、负载、毛刺与检查

门电路在纸上像瞬间给出输出的函数，真实硬件却至少有四个限制。第一，输入变化后，输出需要传播延迟才稳定，组合逻辑层数越多，稳定越慢。第二，一个输出能驱动的后级输入数量有限，后级太多会让切换变慢，甚至让电平余量变差，大网络需要缓冲器和分层布线。第三，门的输出必须重新形成清楚的 0/1，不能只靠一个偏弱路径勉强改变节点电压。第四，多条路径延迟不同，输入变化过程中可能出现短暂毛刺，后级不能在毛刺期间采样。

考虑一个反例。若把十几个门的输入全接到同一个弱输出上，真值表仍然可以保

持正确，但物理电路可能切换缓慢。输出从 0 变 1 时，要给所有后级输入电容充电；如果下一级在它尚未进入可靠高电平区间时读取，就可能看到旧值或不确定值。因此，分析组合逻辑时不能只数门的个数，还要考察最长路径、负载和稳定时间。

扇出问题可以放在安全启动控制板里理解。若 `fault_lamp` 只驱动一个小逻辑门，边沿可能很快；若它同时驱动指示灯缓冲、记录电路、禁止启动网络和外部连接器，等效负载明显增加。正确做法通常不是让一个弱节点直接驱动所有后级，而是分层缓冲：一个内部故障信号先驱动若干缓冲器，再由缓冲器分别驱动不同区域。缓冲器不改变逻辑含义，却改变驱动能力和负载隔离。同理，长线不能直接接到敏感输入上，容易耦合噪声和振铃，需要加强驱动、终端处理或同步采样；普通逻辑输出也不能直接承担指示灯负载，要用驱动级把指示灯的电流需求隔在逻辑电平之外。

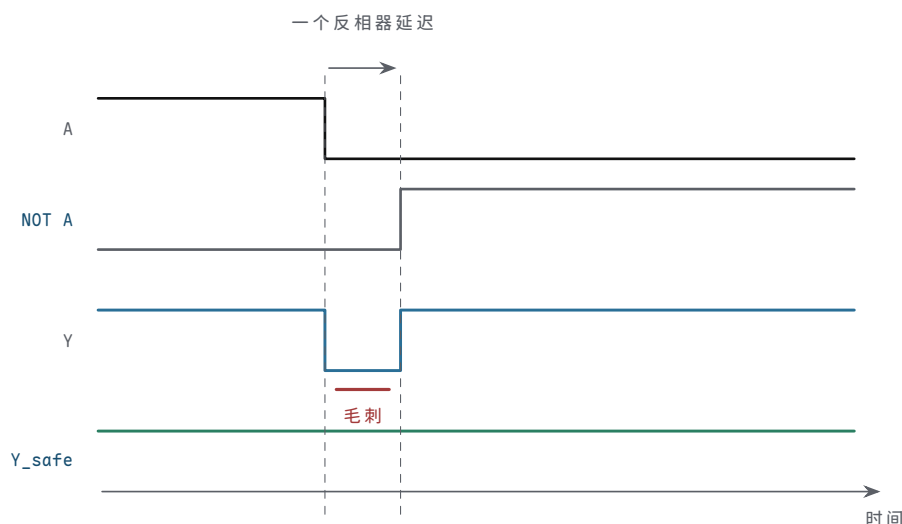
短暂毛刺也很常见。假设一个输出由多条逻辑路径合成，输入同时变化时，每条路径稳定的时间可能不同。真值表只告诉我们变化前和变化后的稳定结果，却不保证中间不会短暂出现错误电平。如果后级只是另一个组合门，毛刺可能很快消失；如果后级在毛刺期间被时钟采样，错误就可能被保存下来。后面讲触发器和时钟时，会把这个问题放大讨论。

一个典型毛刺表达式是：

$$Y = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$$

假设 $B=1$ 、 $C=1$ 。稳定状态下，不论 $A=1$ 还是 $A=0$ ， Y 都应为 1： $A=1$ 时第一项成立， $A=0$ 时第二项成立。问题出现在 A 翻转的瞬间。若 A 从 1 变 0，第一条路径可能先关断，而 $\text{NOT } A$ 经过反相器后才让第二条路径打开，中间可能出现一个短暂间隙，使 Y 从 1 短暂掉到 0。真值表看不见这个瞬间，因为真值表只描述稳定输入和稳定输出。

时刻	路径状态	Y 的风险
A=1 稳定	A AND B 成立	Y=1
A 正在翻转	第一条路径可能已关， 第二条路径尚未开	Y 可能短暂为 0
A=0 稳定	NOT A AND C 成立	Y=1



图示： $B=C=1$ 时， Y 的稳定值本应从 1 到 1 不变。但 A 翻转到 $\text{NOT } A$ 生效之间隔着一个反相器延迟：上路先关、下路未开， Y 出现一个真值表看不见的窄 0 脉冲。加入共识项 $B \text{ AND } C$ 后 (Y_{safe})，有一条不依赖 A 的路径全程把输出钉在 1。

若这个 Y 只是驱动另一个组合网络，并且后级只在更晚的时钟边界采样，毛刺可能不会造成状态错误；若 Y 直接作为写使能、复位、片选或异步控制信号，短暂毛刺就可能触发真实动作。安全启动控制板里的 `allow_start` 尤其需要谨慎：即使

最终会回到正确值，也不能让一个窄脉冲误开电机使能。

上面的毛刺可以用增加共识项来消除。表达式 $Y = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$ 在 $B=1$ 、 $C=1$ 且 A 翻转时有静态 1 毛刺风险。加入第三项 $B \text{ AND } C$ 后， A 翻转期间仍有一条不依赖 A 的路径保持 Y 为 1：

$$Y_{\text{safe}} = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C) \text{ OR } (B \text{ AND } C)$$

这不是为了改变稳定真值表。因为当 $B \text{ AND } C$ 为 1 时，原表达式在 $A=0$ 或 $A=1$ 下本来也为 1；新增项只是在过渡期间提供连续路径。代价是多一个门和更多负载。是否值得加入，取决于这个输出是否会直接驱动危险控制线，或者是否会在毛刺窗口内被采样。

危险信号需要分级处理。并非所有组合输出都必须做成完全无毛刺：普通数据位通常可以容忍短暂过渡，只要后级在稳定的时钟边界采样；状态指示灯容忍度更高，短暂闪动人眼看不见，主要关心驱动能力；写使能和片选容忍度低，应避免异步毛刺，必要时寄存后再使用；复位、停机和电机使能容忍度极低，需要安全默认值、路径冗余和严格的时序检查。对这些信号，常见策略包括：把组合结果先送入触发器，在时钟边界后再使用；避免用毛刺风险高的组合式直接产生异步控制；必要时加入共识项或专用使能门；对外部执行机构再增加独立保护链路。

组合路径还需要时间预算。假设某个输出要在 20ns 内稳定，而路径包含输入缓冲、两级门、一级选择器和输出缓冲，就要把最坏延迟相加。下面的数字只是说明这种预算方法，不代表某个工艺或芯片规格。

路径段	最坏延迟	说明
输入缓冲	2ns	外部信号进入内部逻辑
第一层门	3ns	例如报警 OR 或安全条件 AND
第二层门	4ns	例如 no_fault 与请求合并
选择器或缓冲	5ns	输出选择或驱动增强
布线和负载	3ns	走线、电容、扇出影响
合计	17ns	若要求 20ns 内稳定，则余量 3ns

负载也应纳入同一份时间预算。假设一个输出驱动 8 个 CMOS 输入，每个输入电容按 5pF 估算，走线和封装再贡献约 20pF，则总负载约为 60pF。若输出级等效电阻按 100Ω 粗估，时间常数约为 6ns；若后级要求在 10ns 内越过阈值，这条连接已经没有什么余量。若通过缓冲把 8 个后级分成两组，每个缓冲只承担较小负载，边沿和电平余量通常更容易满足。这个例子不替代仿真和数据手册，但它说明扇出不是“能接几个门”的静态数字，而是电流、电容、边沿和采样时刻共同形成的约束。

估算项	数值	结论
输入电容	$8 \times 5\text{pF}$	后级越多，总电容越大
走线和封装	约 20pF	板级和封装不是零负载
总负载	约 60pF	边沿速度需要重新估算
等效 RC	$100\Omega \times 60\text{pF} \approx 6\text{ns}$	采样窗口紧时需要缓冲或分层驱动

扇出估算：每个后级输入都是一只小电容 上表把负载当成一个给定的 60pF，这个数字从哪里来？答案是一条经验估算规则：每个 CMOS 输入贡献约 3–5pF（栅电容加引脚封装），每厘米走线对地平面再贡献约 1pF。一个输出驱动 10 个输入、走线 15cm，负载约为 $10 \times 4\text{pF} + 15 \times 1\text{pF} \approx 55\text{pF}$ 。驱动这样一个输出，本质上就是给这只总电容充电和放电，扇出问题的全部后果都从这一只电容展开。

边沿快慢可以用一阶 RC 粗估。从 74HC 数据手册的输出压降可以反推输出级等效电阻：额定 4mA 负载下 V_{OL} 约 0.2V，对应约 50Ω。于是时间常数 $\tau = R \cdot C = 50\Omega \times 55\text{pF} \approx 2.8\text{ns}$ ，10%–90% 边沿约为 $2.2\tau \approx 6\text{ns}$ 。若扇出加倍到 20 个输入、走线

20cm, 负载约 100pF, 边沿拖到约 11ns——已经与一级门自身的传播延迟相当, 等于白白多串了一级门, 而这级“门”不贡献任何逻辑功能。

估算项	规则与算式	结果
后级输入电容	每输入 3–5pF, $10 \times 4\text{pF}$	约 40pF
走线电容	约 1pF/cm, $15\text{cm} \times 1\text{pF}$	约 15pF
合计负载	$40 + 15$	约 55pF
边沿时间	$2.2 \times 50\Omega \times 55\text{pF}$	约 6ns
扇出加倍后	$2.2 \times 50\Omega \times 100\text{pF}$	约 11ns, 堪比一级门延迟

高扇出还直接推高功耗。每次翻转搬运的电荷是 $Q = C \cdot V$, 动态功耗为 $P = C \cdot V^2 \cdot f$: 55pF 在 5V、1MHz 下约 1.4mW, 一块板上有几十根这样的重负载信号线, 就是几十毫瓦白白发热。更隐蔽的是第二重代价: 边沿变慢以后, 后级输入在阈值附近停留的时间变长, 接收门内部上下管同时微导通的窗口随之拉长, 产生额外的短路功耗——慢边沿不仅自己慢, 还让接收它的每一扇门更耗电。

缓冲器解决的正是这个物理问题。它不改变逻辑值, 却把一个重负载拆成几个轻负载: 先由普通输出驱动缓冲器, 再由缓冲器分组驱动各片后级, 每条分支电容小、边沿快, 局部延迟反而更可控。代价是缓冲器自身的传播延迟要计入路径, 所以缓冲层级不是越多越好——这又是一次功能、负载与延迟之间的取舍, 与“逻辑上保持原值”并不矛盾。

若后续系统在 15ns 时就使用这个输出, 稳定真值表仍然不够; 输出可能还在传播或毛刺阶段。第二章要引入的触发器和时钟边界, 正是为了规定“什么时候保存”。组合逻辑在边界之间准备结果, 时钟边界只应采样已经稳定、满足建立时间要求的信号。

时间预算要使用最坏情况, 而不是典型情况。最小延迟说明某条路径最快可能多久变化, 用来检查保持时间、窄脉冲和过早到达; 典型延迟是常见条件下的大致延迟, 便于估算, 但不能作为设计边界; 最大延迟是不利条件下最晚多久稳定, 用来检查时钟周期和采样时刻是否足够; 偏斜是不同路径或不同信号到达时间的差, 用来解释毛刺、竞争和采样风险。若一条路径在实验室常温下 12ns 稳定, 并不能证明它在低电压、高温、最大负载下也能在 15ns 内稳定。正式规格通常同时给出最小、典型、最大值; 做时序分析时应以满足系统要求的最坏组合为准。

为了衔接下一章, 还需要先建立几个时序词汇。建立时间指数据在采样时钟边沿到来前必须提前稳定的最短时间——结果不能刚到边沿才变化; 保持时间指数据在采样时钟边沿之后仍必须继续稳定的最短时间——结果不能过早被下一条路径改写; 时钟到输出指触发器被时钟采样后输出变化所需的时间, 它是新数据进入下一段组合逻辑的起点; 时钟偏斜指不同触发器看到同一时钟边沿的时间差, 它会压缩某些路径的有效时序余量。

完整同步时序检查将在第二章展开。本章只使用最基本的判断: 组合路径的最大延迟不能侵占建立时间, 组合路径的最小延迟不能破坏保持时间, 时钟偏斜不能被假定为零。换言之, 组合逻辑不是“算完就行”, 而是必须在规定的时钟边界之前算完, 并且在边界附近保持稳定。

传播延迟沿路径累加: 一条五级门的核算 前面时间预算表里的数字是示意性的, 现在用真实器件把同一条账重算一遍。回到安全启动控制板: 从板级连接器到 `allow_start` 的最长路径, 经过输入调理反相 (一片 74HC04) 和四级 NAND (两片 74HC00 依次产生 `n1`、`safe_chain`、`n3`、`allow_start`), 共五级门。链式结构有一个简单却关键的性质: 前一级的输出越过阈值后, 后一级的延迟才开始计时, 所以总延迟近似等于各级传播延迟之和。

查 74HC 系列数据手册 (4.5V–5V 供电、50pF 负载电容、25°C 附近的标称条件), 这一档门的单级传播延迟典型值约 9ns; 手册同时给出保证最大值约 20ns, 覆盖工艺离散、全温度范围和电压波动的最不利组合。不同厂商、不同温度档次的具体数字略有出入, 正式设计必须以手头器件的手册为准, 这里取代表值是为了说明算法。

路径级	器件与作用	典型	最大
第 1 级	74HC04, 输入调理反相	9ns	20ns
第 2 级	74HC00, 产生 <code>n1</code>	9ns	20ns
第 3 级	74HC00, 自反相得 <code>safe_chain</code>	9ns	20ns
第 4 级	74HC00, 汇合得 <code>n3</code>	9ns	20ns
第 5 级	74HC00, 自反相得 <code>allow_start</code>	9ns	20ns
合计	五级串联	45ns	100ns

两个合计数回答两个不同的问题。45ns 回答“台架上大概会看到什么”：实验台恰好常温、额定 5V、轻载，手头的芯片又只是少量样本，所以示波器上的实测值通常落在典型值附近——测量常见典型值，是因为实验室条件恰好全部偏有利。100ns 回答“设计必须容忍什么”：任何一片合格芯片都允许慢到最大值，换一批料、盛夏高温、电源跌到下限、负载加重，都可能把延迟推向 100ns。若采样边界按典型值设计，比如 60ns 就读取，台架上一切正常，量产后最不利的那批板子却会读进旧值。这就是为什么建立时间一类的检查必须用最大值核算，而典型值只能用来估量级。

还有两条细则。第一，数据手册把上升翻转 (t_{PLH}) 和下降翻转 (t_{PHL}) 分开给出，两者一般不相等，链式相加时每一级都应取两者中较大的那个。第二，50pF 是手册规定的测试条件；若实际负载更重，每级还要按扇出估算中的 RC 法则追加延迟——门的固有延迟和负载造成的延迟从来不是两个无关的量。

现在可以更系统地回答“现代芯片为什么时延低”。第一层原因来自器件和工艺。更小的晶体管通常意味着更小的栅电容、更短的沟道、更短的局部连接和更低的单个门负载；同样的逻辑函数，在更小、更近的器件中传播，很多局部路径的绝对时间会下降。但这不是无条件成立。长连线、全芯片时钟、供电网络和散热会逐渐成为新的限制，所以现代芯片必须同时优化**晶体管**和**连线**。

第二层原因来自片上集成。8086 时代，处理器与内存、外设和许多支撑逻辑之间存在大量片外通信；片外总线距离长、电容大、引脚数量有限，等待代价高。8086 已经用总线接口单元和执行单元的分工，以及指令预取队列，尝试把取指和执行重叠起来。到 80386，更多地址转换、保护和控制逻辑进入处理器内部，32-bit 数据通路和更丰富的片上控制减少了对外部协作的依赖。再往后，cache、浮点单元、内存控制器、图形或 I/O 控制器逐步进入同一颗芯片或同一封装，许多原本必须跨板级总线的路径被缩短为片上路径。

第三层原因来自存储层次。主存容量大，但距离核心远、访问路径长；寄存器和 L1 cache 容量小，却离执行单元近。现代芯片通过多级 cache 把最常用的数据和指令放在更近的位置。一次 L1 命中不是因为 DRAM 变得同样快，而是因为本次访问根本没有走到 DRAM。时延降低来自“常见情况走短路径”，不是“所有路径都同样短”。

第四层原因来自流水线和并行。把一条很长的组合路径拆成多个较短阶段，每个阶段之间放入寄存器，就能提高时钟频率。这样做降低了每个时钟周期需要容纳的组合延迟，但并不等于每条指令的总周期数一定减少。现代处理器还会用分支预测、乱序执行、多个执行单元和内存预取，把将来可能需要的工作提前准备，把等待主存或分支结果的空洞尽量填满。它们降低的是常见执行路径上的可见等待。

因此，现代芯片的低时延是一组工程选择的结果。器件缩小降低局部门延迟；门级库、缓冲和布局缩短关键路径；寄存器边界把长组合逻辑切成阶段；cache 把常用数据放近；预测和乱序把等待隐藏；片上集成减少跨芯片通信。若某次访问落在**DRAM**、分支预测失败、cache 未命中或跨核心通信，时延仍会显著上升。严肃的硬件分析必须同时说明**最快常见路径**和**最坏慢路径**。这些机制在本书后续章节（ISA、流水线、cache 与虚拟内存，见扩写目录）和第六章经典芯片里会逐个展开成可检验的结构；第一章只需要先建立判断框架：低时延来自让常见情况少走远路。

现代 CPU 的一个典型低时延路径是：操作数已经在寄存器或 L1 cache 中，分支预测正确，所需执行单元空闲，结果能在相邻流水线阶段之间传递。此时，芯片用局部电路、短路径和寄存器边界把等待压到很低。相反，若数据不在 cache 中、分支预测错误或需要跨核心同步，同一条指令流会进入慢路径。现代芯片的工程目标不是让所有路径同样快，而是让高频路径短、让慢路径少发生，并在慢路径发生时尽量隐藏等待。

还要区分**时延**和**吞吐率**。流水线可以让每个周期都有不同指令处在不同阶段，从而提高吞吐率；但一条具体指令从进入流水线到完成，仍然要经过多个阶段。多执行单元可以同时处理多条互不依赖的操作，但无法让一条有严格依赖链的操作跳过必要计算。cache 可以显著降低常见访问时延，但不能让未命中的访问变成命中。理解现代芯片时，必须同时问两个问题：单次操作最快多久返回，连续大量操作每单位时间能完成多少。

还要区分反相器和缓冲器。反相器改变逻辑含义，缓冲器通常保持逻辑含义但增强驱动能力或隔离负载。很多时候，插入缓冲器不是为了“多一个门”，而是为了让一个信号能推得动更长的线或更多的后级。软件读者容易觉得保持值不变的缓冲器没有意义，但在硬件里，驱动能力本身就是功能的一部分。

最后还要检查扇入。扇出关心一个输出驱动多少后级，扇入关心一个门接收多少输入。四输入、八输入甚至更多输入的门在逻辑表达式里写起来很自然，但在硬件实现中可能被拆成多级结构；输入越多，门内部路径、电容和延迟也可能越大。把 `allow_start` 写成一个五输入 AND 并不意味着实际芯片里一定存在一个理想五输入 AND。实现可能把它拆成树形结构，也可能先合成若干中间信号。拆分方式会改变最长路径和毛刺形态。

```
flat idea:
    allow_start = A AND B AND C AND D AND E

tree implementation:
    left  = A AND B
    right = C AND D
    mid   = left AND right
    allow_start = mid AND E
```

树形实现通常比线性串接更平衡，但会引入中间节点和不同路径长度。若 E 很晚才稳定，把它放在最后一级可能有利于减少无效切换；若 A、B 来自同一连接器，先合成 `left` 可能便于局部布线。这里没有脱离功能的“最佳写法”，只有在功能、负载、延迟、布局和安全策略之间作出的实现选择。

对组合逻辑的最终检查，不应只停留在“真值表正确”。功能上，稳定输入下输出要符合真值表，否则是逻辑行为错误；默认状态上，悬空、断线、复位前的输入解释要有定义，否则异常时会误启动或误使能；驱动能力上，每个输出要能带动实际负载，否则电平被拖低、边沿过慢；路径延迟上，最长组合路径要能在后级使用前稳定，否则后级采到旧值或过渡值；毛刺风险上，多路径汇合不能产生危险窄脉冲，否则写使能、复位、片选会被误触发；低有效信号上，信号命名和真值表要一致，否则使能和禁止方向会接反；物理边界上，长线、外部输入和按钮抖动都要处理，否则干扰和抖动会进入逻辑核心。

对安全启动控制板，可以把这些检查整理成矩阵。矩阵的价值在于把“正常输入”“边界输入”“异常输入”和“时间条件”放在一起，防止只验证最容易通过的几行真值表。

检查类别	要施加的条件	期望结果
正常允许	请求有效、盖闭合、急停正常、无故障	<code>allow_start</code> =1，执行命令等待时序边界

正常禁止	任一安全条件为 0	<code>allow_start=0</code>
故障禁止	温度或电流报警为 1	<code>fault_lamp=1, allow_start=0</code>
断线默认	保护盖、急停或报警链路断开	进入禁止或报警一侧
上电过程	电源未稳或复位保持	<code>motor_enable_cmd=0</code>
按钮抖动	原始按钮快速断续	内部请求不产生多次启动事件
最坏负载	故障灯、后级逻辑和连接器同时连接	电平仍满足后级阈值
最坏时序	输入在不利延迟组合下变化	后级只在稳定后采样或使用

常见失败也应明确记录。许多硬件问题并不是因为设计者不知道 AND、OR、NOT，而是因为把抽象层次接错：悬空输入让系统偶尔误启动或误报警，根源是没有默认上拉/下拉或输入调理；低有效误读让复位、片选或急停方向相反，根源是信号约定未先于公式建立；电平不兼容让单独测试正常的电路接入后失效，根源是前后级阈值和驱动能力不匹配；毛刺误触发让最终值正确的电路出现短促动作，根源是危险输出直接使用了组合信号；扇出过大让边沿变慢、高电平不够高，根源是一个输出承担了过多负载；时序乐观让常温可用的电路在高温或低压下失效，根源是用典型值代替最坏值做核算；不关心误用让异常状态进入允许输出，根源是把可能发生的故障组合当作无关项。

若要把本章内容带入后续 CPU 数据通路，读者应保留两种同时成立的视角。第一，抽象视角：逻辑门实现布尔函数，组合网络实现加法、选择、译码和比较。第二，物理实现视角：每个输出都来自具体电流路径，每条路径都有延迟、负载和阈值，每个危险输出都需要明确的使用时刻。缺少前者，无法构造大系统；缺少后者，系统只是在纸上正确。

本章结束时，读者应能整理出一份小型硬件设计包。它不要求购买昂贵仪器，也不要求完成工业级认证；它要求把抽象、实现和验证记录放在同一份文档里：一张信号表，列出原始输入、内部语义信号、组合输出和危险输出及其有效电平；一张电平表，列出前级 V_{OH}/V_{OL} 、后级 V_{IH}/V_{IL} 、噪声余量和负载条件，全部用最坏值；一张路径表，写出关键门在每行输入下的上拉、下拉、悬空和冲突状态；一张实现表，写明使用的门芯片、缓冲器、上拉/下拉、保护和输出驱动；一张时序表，写明最长路径、最小路径、毛刺风险、后级使用时刻和危险输出策略；一张测试表，写明施加条件、期望结果、测量工具、测点和波形记录；外加几张案例卡，把 7400/4000/8086/80386/现代 CPU 与本章概念真正关联起来。若后续章节要把这个组合逻辑接入寄存器、CPU 或总线，这份设计包就是接口约定。

对安全启动控制板的检查也应按执行顺序展开，而不是只看最终结果。第一步在断电和上电爬升阶段确认 `motor_enable_cmd` 被保持为 0；第二步测量每个原始输入在松开、按下、断线和报警状态下的默认电平；第三步用示波器确认按钮抖动、传感器边沿和故障输入不会在阈值附近长期停留；第四步用逻辑分析仪或等效测试记录内部语义信号是否只产生规定事件；第五步施加正常允许、单故障、双故障和断线条件，确认组合输出满足真值表；第六步在最坏负载和最坏时序假设下确认后级只在稳定后使用结果。

实验记录应当能让第三方复查。只写“测试通过”不够；至少要记录条件、预期结果、测量数据和结论。下面的模板适合本章的小组合模块，也适合后续寄存器和总线实验继续扩展。

测试项	施加条件	期望结果	实测电压或波形	结论
上电默认	电源爬升、复位保持	<code>motor_enable_cmd=0</code>	记录关键节点电压与复位释放时间	合格或失败
按钮输入	松开、按下、抖动、断线	内部请求只产生规定事件	记录阈值穿越和消抖后信号	合格或失败

故障禁止	温度或电流报警有效	<code>fault_lamp=1,</code> <code>allow_start=0</code>	记录报警输入与组合输出延迟	合格或失败
最坏负载	连接全部后级和指示灯驱动	电平余量和边沿仍满足规格	记录高低电平、上升/下降时间	合格或失败
毛刺检查	输入按不利时序翻转	危险输出不响应窄脉冲	记录示波器或逻辑分析仪波形	合格或失败

若使用逻辑分析仪，还应记录采样率、触发条件和被测信号名称；若使用示波器，还应记录探头倍率、带宽限制、地线接法和触发边沿。测量工具本身也会影响电路：长地线可能引入额外振铃，高电容探头可能拖慢边沿。因此，实验记录不仅证明电路行为，也证明测量方法没有把问题掩盖或放大。

实际测量时，探头应尽量接在接收端附近，而不只接在驱动端。驱动端波形漂亮，不代表穿过连接器、长线和负载后的接收端仍有足够余量。示波器观察快速边沿时应优先使用短地弹簧或低感抗接地方式，避免探头地线把额外振铃带入测量结果；逻辑分析仪只给出阈值后的数字结果，不能证明边沿是否干净；万用表只能证明低速或静态电压，不能证明毛刺和建立时间。正式记录应注明测点、探头、阈值、触发条件和环境条件——只证明常温空载，不等于证明最坏条件。

最后预告一个第二章会正式处理的问题：异步输入进入时钟边界时可能出现亚稳态。亚稳态指触发器在采样边沿附近遇到变化中的输入，内部节点可能暂时停在不能立刻判定为 0 或 1 的状态。第一章只要求记住边界原则：外部按钮、传感器、中断线和跨时钟域信号，不应未经同步就直接控制内部状态或危险输出。组合逻辑可以整理当前事实，但“何时保存”必须由后续时序电路给出约定。

到这里，本章完成了从电平到组合逻辑的闭环：节点要有明确驱动路径；电平要有噪声余量；二极管提供方向性但不能恢复电平；受控开关让信号控制电流路径；路径表和真值表共同定义门；门网络构成加法器、选择器、译码器等组合电路；组合电路虽然没有记忆，却有延迟、负载和毛刺。下一章开始讨论另一件事：怎样让硬件保存状态，并按时钟边界一步一步推进。

专题：把电平特性从“能亮”推进到“有裕量” 前面已经把电平、开关和门电路连成一条主线。真正做硬件时，还必须把“能看到 0/1”推进到“在误差、噪声、负载和温度变化下仍然可靠”。一个输入脚不只是在某个理想电压上突然改变含义，而是由输入低阈值、输入高阈值、输出低电平、输出高电平共同定义可用范围。输出端能给出的高电平必须高于下一级认可的高阈值，输出端能拉到的低电平必须低于下一级认可的低阈值，中间差额就是噪声裕量。

参数	含义	设计时要回答的问题
<code>VOH</code>	输出为 1 时保证达到的最低高电平	在最大负载下是否仍高于下一级 <code>VIH</code>
<code>VOL</code>	输出为 0 时保证不超过的最高低电平	在灌电流时是否仍低于下一级 <code>VIL</code>
<code>VIH</code>	输入被识别为 1 的最低电压	外部信号、上拉和噪声是否能跨过阈值
<code>VIL</code>	输入被识别为 0 的最高电压	慢边沿和串扰是否会停在不确定区
噪声裕量	输出保证值到输入阈值之间的余量	板级噪声、地弹和电源纹波是否小于余量

这张表比“LED 亮不亮”更接近工程真实。LED 亮只能说明某个输出能驱动肉眼可见负载，不能证明它满足数字输入阈值；示波器上看到高电平接近电源，也不等于在最大扇出、最坏温度和最长连线下仍可靠。教材应让读者从第一章就建立习惯：每个数字信号都要同时看电平范围、驱动能力、边沿速度和负载。

noise margin check:


```
high_margin = VOH_min - VIH_min
low_margin  = VIL_max - VOL_max
pass only if expected_noise < both margins
```

CMOS 反相器的关键不是“一个管上、一个管下” CMOS 反相器常被画成上拉 PMOS 和下拉 NMOS。这个图很有用，但只说“输入 0 时上管开、下管关”还不够。实际电路要处理三个非理想问题：输出节点有电容——栅电容、扩散电容、连线电容和下级输入电容累加，翻转需要充放电时间，这就是传播延迟的来源；输入在阈值附近缓慢变化时，上下管可能同时部分导通形成短路电流，慢边沿因此既增加功耗也增加噪声；驱动多个输入或长线时等效电容更大，边沿更慢，延迟和裕量都被压缩。同一张清单上还有两条板级事实：多个输出同时翻转产生的瞬态电流会让地弹或电源跌落，输入阈值附近可能被误触发；高阻输入容易被漏电和耦合影响，必须用上拉、下拉或明确驱动源给出归宿。

因此，第一章的“开关”要升级为“受控电流路径”。当输出从 0 变 1 时，上拉网络不是瞬间把节点变成电源，而是向负载电容充电；当输出从 1 变 0 时，下拉网络把负载电容放电。电容越大、电阻越大，边沿越慢。延迟并不是后面时序章节才出现的概念，它从第一颗门的输出节点就已经存在。

扇出、上拉和开漏输出要按电流读 许多初学错误来自只看电压不看电流。一个输出脚能否驱动多个输入，要看它在保持合法 V_{OH}/V_{OL} 时能提供或吸收多少电流。TTL、CMOS、开漏、三态输出的驱动方式不同，不能只按“都是 5V 逻辑”混接。

输出形式	电流路径	常见用途和风险
推挽输出	上拉和下拉都由有源器件驱动	边沿快，但两个推挽输出不能直接并联
开漏/开集	只能主动拉低，高电平靠上拉电阻	适合线与、总线仲裁，但上升沿较慢
三态输出	可驱动高、低或断开	总线共享，但必须保证同一时刻唯一驱动器
弱上拉/下拉	用较大电阻给默认电平	适合防悬空，不适合驱动大负载

开漏输出尤其适合解释“电压是结果，电流路径才是原因”。当任意设备拉低总线时，电流从电源经上拉电阻流入该设备到地，总线电压变为低；当所有设备都断开时，上拉电阻慢慢给总线电容充电，电压回到高。上拉电阻太大，上升沿慢；太小，拉低时电流大。I2C、按键输入和许多中断线都能用这个模型理解。

```
open-drain line:
any driver pulls low -> line = 0
all drivers released -> pull-up charges line to 1
rise time depends on pull-up R and bus capacitance C
```

去耦电容是本地瞬态电源 门翻转的电流从哪里来？不是从稳压器“立刻”流过来——稳压器的响应以微秒计，而门的边沿以纳秒计。输出翻转时对负载电容充放电的电荷，加上内部上下管瞬间同时微导通的贯穿电流，都只能先从电源脚附近的储能元件取得。如果附近没有储能，电荷只能沿电源走线从远处赶来，而走线不是理想导线：每厘米约贡献 5–10nH 电感（含回流路径），电感上的电压满足 $v = L \cdot di/dt$ ，电流变化越快，走线上感应出的压降越大。

代入一组典型数字。8 路输出同时翻转，负载各 50pF，搬运的总电荷为 $Q = 8 \times 50\text{pF} \times 5\text{V} = 2\text{nC}$ ；在 5ns 内完成，意味着瞬态电流约 $2\text{nC}/5\text{ns} = 0.4\text{A}$ 。若这 0.4A 要穿过约 20nH 的走线电感，感应电压为 $20\text{nH} \times 0.4\text{A}/5\text{ns} = 1.6\text{V}$ ——电源脚瞬时跌落、地脚瞬时抬高（后者俗称地弹），整片芯片看到的“5V”在剧烈抖动，输入阈值跟着晃，邻近信号可能被误判。地弹不是器件坏了，而是瞬态电流与走线电感的必然乘积。

每颗芯片就近放一只 $0.1\mu\text{F}$ 瓷片电容，就是给瞬态电流修一座本地水库：尖峰电流先由它供给，走线电感只需在事后把电荷慢慢补回。同样 2nC 改由 $0.1\mu\text{F}$ 供给，电压跌落只有 $\Delta V = Q/C = 2\text{nC}/0.1\mu\text{F} = 20\text{mV}$ ，比 1.6V 低约两个数量级。“贴近电源脚”是电气要求而非工艺美观：电容与芯片之间每多一毫米走线，水库与芯片之间就多串一段电感，本地储能的效果就打一分折扣。所以布局规则是紧贴电源脚、短焊盘、短回流路径。

$0.1\mu\text{F}$ 只管得住快而小的口袋，整板还需要分层：电源入口的 $10\text{--}100\mu\text{F}$ 大电容应付更大、更慢的电流波动（继电器吸合、数码管扫描、背光开启），稳压器负责直流供给。按时间尺度分层供电，与后面存储层次按速度分层是同一个思想：快的层级小而近，慢的层级大而远，各自接住上一层来不及响应的那一段。

走线多短才算“短”：信号完整性初步 信号在走线上的传播速度是有限的：FR4 板材上约为 15cm/ns ，大约是光速的一半。只要走线延迟远小于信号边沿时间，线上各点就来不及显现分布特性，可以把整条走线当作一只集中电容——这就是“短”线。常用的经验法则是：走线延迟小于边沿时间的六分之一（有的教材取四分之一），即可按集中参数处理。前面的扇出估算把走线只算作每厘米 1pF ，依据正是这个近似。

74HC 的边沿约 $5\text{--}8\text{ns}$ ，代入法则：临界长度 $\approx 5\text{ns} \times 15\text{cm/ns} \div 6 \approx 12\text{cm}$ 。本章的面板板和实验连线都在这个量级之下，所以把走线当集中电容是站得住的——一本书低速实验大多安全地待在“短”线区。反过来，一旦边沿变快（现代器件可低于 1ns ）或距离变长（跨板、背板、电缆），短线近似就会失效，同一根线必须改按传输线看待。

长线上的核心现象是反射。阻抗不连续的地方，行波会部分折返：无端接的长线末端近似开路，入射波几乎全反射回来，与入射波叠加后，接收端出现过冲、振铃甚至回沟，一个边沿可能被看成多次穿越阈值——接收端误以为信号翻转了好几回。端接的思路是为行波安排能量归宿：末端并联电阻到地或电源，或源端串联电阻，使反射系数接近零，波形一次到位。反射不是噪声玄学，它的幅度和极性都可以从阻抗差算出。

串扰则来自相邻走线之间的互容与互感。两条长平行线中，一条快速翻转会在另一条上感应出窄毛刺；平行段越长、间距越近、边沿越快，耦合越强。处理办法是拉开间距、缩短平行段、在敏感信号之间夹一根地线。这些现象在 HC 实验里只会以“边沿有点圆、偶尔带振铃”的形式露面，但同一条物理规律决定了高速主板为什么讲究受控阻抗与端接电阻——先在这里建立直觉，后续章节再定量。

毛刺不是玄学，而是路径延迟不一致 组合逻辑里不同输入路径有不同延迟。当多个输入几乎同时变化时，中间节点可能短暂经过错误组合，输出形成窄脉冲，这就是毛刺。若毛刺只被后级组合逻辑看到，可能不影响最终稳定值；若毛刺被锁存器、计数器、异步复位或外部设备采样，就会变成真实错误。

毛刺最常见的来源有四种。重收敛路径：同一输入经不同延迟路径再汇合，处理办法是改写逻辑、增加共识项，或让后级只在时钟边界采样。译码器切换：地址多位同时变化，片选可能短暂重叠或空洞，处理办法是地址先锁存、片选加使能窗口。机械按键：触点弹跳导致多次高低变化，处理办法是 RC、施密特、同步去抖或状态机去抖。异步输入：外部事件不按本机时钟变化，处理办法是同步器和边沿检测分开设计。

这也解释了为什么本书反复强调“边界必须可检查”。组合逻辑允许在稳定前经历暂态，时序元件只应在规定窗口采样稳定结果。若把组合毛刺直接接到计数器时钟、异步清零或设备写使能，系统会表现得像随机错误。第一章的门延迟和毛刺，直接通向第二章的时钟边界和后面整机的片选安全。

第一章实验应记录波形而不只记录现象 一块面包板、一个反相器、一只按键和一台示波器，就能做出高质量的第一章实验。目标不是炫耀器件数量，而是把电平、阈值、边沿、负载和毛刺变成可以直接观察的量。

实验

操作

必须记录

输入阈值扫描	用电位器缓慢改变输入电压	输出翻转点、滞回是否存在
负载电容实验	在输出加不同电容负载	上升/下降时间和传播延迟变化
上拉电阻实验	改变开漏线的上拉电阻	上升沿、低电平电流和噪声裕量
按键抖动观察	直接观察按键输入波形	抖动持续时间和次数
译码毛刺观察	让多位地址同时变化	片选是否短暂重叠或误触发

这些实验报告应写出测量条件：电源电压、器件型号、负载、电阻电容值、示波器探头倍率、时基和触发方式。没有测量条件的波形很难复现，也很难判断结论是否能迁移到另一块板。

章末练习

本节练习要求使用文字、真值表、路径表和检查表完成。重点不是记忆门符号，而是能说明某个 bit、某条路径、某个组合输出为什么在真实硬件条件下可靠成立。

练习按三层完成。基础层要求掌握电平、路径和真值表，能从每根线的 0/1 语义追溯到物理路径；设计层要求把组合逻辑实现为门、芯片和实验，能把表达式变成可工作的实现；扩展层要求把第一章概念映射到经典芯片和现代低时延机制，能把概念放进真实产品和测量环境。读者不应只做会写公式的题，也要能留下可复查的实验记录。

任务	要完成的产物	检查要点
按钮输入	为 <code>start_button_raw</code> 写出上拉或下拉方案、有效电平、断线默认值和消抖策略	松开、按下、抖动、断线四种状态都有明确内部语义
电平约定	给出一组前级 V_{OH}/V_{OL} 与后级 V_{IH}/V_{IL} ，计算高低噪声余量	说明余量为正、为零、为负时分别意味着什么
路径表	任选 NAND 或 NOR，列出四行真值表，并为每一行写出上拉与下拉路径状态	不出现稳定输入下的悬空或强冲突
低有效信号	将 <code>reset_n</code> 、 <code>chip_select_n</code> 或急停链路转换为正逻辑语义信号	说明原始电平、内部语义和默认安全值之间的关系
故障汇总	为 <code>temp_alarm</code> 与 <code>current_alarm</code> 设计故障灯和故障码逻辑	单故障、双故障、无故障和非法输入都有定义
1-bit ALU	写出 AND、OR、XOR、ADD、SUB 的候选结果、 <code>op</code> 、 <code>B_invert</code> 和最低位 <code>Cin</code> 规则	区分逐位逻辑路径、加减法进位路径和标志解释
NAND-only 实现	把 <code>allow_start</code> 的安全表达式改写成二输入 NAND 网络，并命名所有反相中间信号	稳定真值表等价，同时说明门级层数、负载和毛刺风险
芯片引脚映射	任选 74HC00 或 7400 类芯片，把一个逻辑式映射到电源脚、输入脚、输出脚和未用门处理	说明去耦、未用输入默认值、扇出和封装方向
输入保护	为一个板外按钮或传感器写出限流、滤波、ESD/TVS、施密特输入或缓冲方案	说明保护不改变语义，但限制异常电压、电流和噪声

五层边界图	任选 <code>start_button</code> 、 <code>cover_closed</code> 或 <code>temp_alarm</code> ，画出外部物理量、保护边界、阈值整形、语义归一和组合逻辑五层	每层都要写出输入、输出、风险和验证方法，不能把原始引脚直接当作内部 bit
引脚等效模型	为一个输入和一个输出分别列出漏电、电容、阈值、驱动电流、传播延迟或绝对最大额定值	能解释为什么逻辑变量模型不足以判断扇出、长线、LED 驱动和跨电源域连接
毛刺分析	分析 $Y=(A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$ 在 $B=1$ 、 $C=1$ 、 A 翻转时的风险	能说明共识项为什么不改变稳定真值表，却改变过渡行为
安全闭环	按“原始输入、输入调理、语义信号、组合判断、时序边界、输出驱动、验证记录”重写安全启动控制板规格	每一层都能追溯到上一层，危险输出不能直接依赖未经检查的组合结果
芯片案例	分别用 7400、4000 CMOS、8086、80386 和现代 CPU 解释本章一个概念；6502、Z80、8051、80486、Pentium 作为选做，等学完第六章经典芯片后回看	每个案例都要指出对应的电平、路径、组合逻辑、片上集成或低时延机制
测量计划	为按钮输入或故障输出写出万用表、示波器、逻辑分析仪各自要证明的内容	不把静态电压测量误认为完整的时序和毛刺检查
时序词汇	用一条组合路径解释建立时间、保持时间、时钟到输出和时钟偏斜	能说明最大延迟、最小延迟和采样边界的关系
实验记录	按测试项、条件、期望结果、实测波形和结论整理一次安全启动控制板实验	记录足以让第三方复查，不只写“通过”
板级边界	说明参考地、回流路径、去耦、串扰或长线振铃如何影响某个输入	能把布尔错误和物理边界错误区分开
快慢路径	为现代 CPU 写出一个快路径和一个慢路径案例	能说明低时延来自常见路径短，而不是所有路径同样短
上拉阻值	为一条 5V 开漏总线（总线电容 100pF，驱动器灌电流上限 4mA）选择上拉电阻，要求从低电平上升到 $V_{IH}=3.5V$ 的时间不超过 $1\mu s$	同时算出阻值上界和下界，并验证所选标称值两头都满足
三态总线互斥	为共享一条数据总线的两个三态驱动器写出使能逻辑，并分析选择信号切换瞬间的风险	任何时刻至多一个驱动器使能，切换要先断后通
选择器实现函数	用 8 选 1 选择器实现 $F(A,B,C)=\sum m(1,3,5,6)$ ，再改用 4 选 1 选择器、以 A 作数据输入实现同一函数	每个数据输入的常量或函数都要与真值表逐行一致
延迟链核算	一条 4 级 74HC 路径，每级典型 9ns、最大 22ns，要求信号在采样前 90ns 稳定	分别用典型值和最大值核算，说明哪一个才能作为设计依据

去耦电容选择	为一块 8 路输出可能同时翻转的 74HC 小板选择去耦方案，估算本地电容上的电压跌落	说明就近原则与板级大电容的分工，跌落估算要有数值
电平转换	3.3V CMOS 输出（最差 $V_{OH} = 3.0V$ ）驱动 5V 74HC 输入（ $V_{IH} = 3.15V$ ），给出可行方案	先核算直接连接的余量，再给出器件级方案并重算余量

完成这些练习后，读者应能把一个组合电路从自然语言需求推进到可检查的规格：先定义信号含义，再列真值表和路径表，然后检查电平、负载、延迟、毛刺、默认状态和测量记录。这个顺序将直接用于后续寄存器、数据通路、控制器和整机接口。

参考解答与检查要点

本节给出参考答案。实际工程方案允许存在不同器件和参数选择，但结论必须有电平、路径、时序和测量结果支撑。若答案只给出布尔式而没有默认值、负载、毛刺或测量边界，不能视为完整解答。

任务	参考解答	必须保留的检查点
按钮输入	可采用上拉输入：松开时 <code>start_button_raw=1</code> ，按下接地为 0，再经反相、消抖和同步得到 <code>start_request=1</code> ；也可采用下拉输入，关键是松开、按下、断线都要有定义。	不允许松开或断线时悬空；按钮抖动不能形成多次启动事件。
电平约定	例如 $V_{OH} = 3.0V$ 、 $V_{IH} = 2.1V$ ，高余量 0.9V； $V_{OL} = 0.25V$ 、 $V_{IL} = 0.7V$ ，低余量 0.45V。余量为正表示最坏条件仍可识别，余量为零表示没有抗扰空间，余量为负表示电平约定失败。	使用最坏值，不能用典型值替代核算。
路径表	以 NAND 为例， $A=B=1$ 时下拉串联路径完整、输出为 0；其余三行至少一个下拉开关断开，上拉网络提供高电平。NOR 则对应并联下拉和串联上拉。	每一行都要说明上拉、下拉、悬空和冲突。
低有效信号	<code>reset_n=0</code> 表示复位有效，可转换为 <code>reset_active = NOT reset_n</code> ； <code>chip_select_n=0</code> 表示选中，可转换为 <code>chip_selected = NOT chip_select_n</code> 。	名称必须反映极性，默认安全状态要明确。
故障汇总	<code>fault_lamp = temp_alarm OR current_alarm</code> ； <code>no_fault = NOT fault_lamp</code> ；双故障可输出专用多故障码 11。	任一故障都禁止启动，非法或未知组合不应进入允许状态。
1-bit ALU	<code>AND=A AND B</code> ， <code>OR=A OR B</code> ， <code>XOR=A XOR B</code> ；ADD 使用 <code>B_invert=0, Cin=0</code> ；SUB 使用 <code>B_invert=1, Cin=1</code> ，即 <code>A+(NOT B)+1</code> 。	<code>op</code> 、 <code>B_invert</code> 、最低位 <code>Cin</code> 必须一致，标志位要说明无符号或补码解释。

NAND-only 实现	可写为 <code>n1=cover_closed NAND emergency_ok、 safe_chain=n1 NAND n1、 n2=start_request NAND no_fault、 request_ok=n2 NAND n2、 n3=safe_chain NAND request_ok、 allow_start=n3 NAND n3。</code>	反相中间信号必须命名清楚，NAND-only 等价不代表延迟和毛刺也相同。
芯片引脚映射	以 74HC00 为例，一路 NAND 接 <code>cover_closed/emergency_ok</code> 得到 <code>safe_chain_n</code> ，第二路 NAND 两输入并接 <code>safe_chain_n</code> 得到 <code>safe_chain</code> ；VCC/GND 接正确电源并就近去耦，未用 CMOS 输入接确定电平。	检查封装方向、引脚编号、未用门、扇出、输出驱动和电源去耦。
输入保护	板外按钮可采用串联电阻、上拉/下拉、RC 滤波、TVS/ESD 保护和施密特输入；高速输入不能使用过大 RC。	保护电路限制异常电压和电流，但不能破坏有效边沿和响应时间。
五层边界图	以 <code>start_button</code> 为例， <code>raw</code> 表示触点和线缆；保护层包含限流、ESD 和默认上拉/下拉；整形层包含 RC 或施密特输入；语义层把低有效、消抖和反相整理成 <code>start_request</code> ；组合层只使用 <code>start_request</code> 。	每层输出都要比上一层更接近可靠 bit，不能跳过保护和阈值整形环节。
引脚等效模型	输入端可写成阈值比较器加输入电容、漏电和保护结构；输出端可写成受控上拉/下拉开关加等效电阻、电流限制和延迟。这个模型能解释上拉太弱、负载过重、长线振铃、输出并接和跨电源域损坏。	数据手册最坏值优先；不要用“仿真里是 1”替代引脚电气检查。
毛刺分析	对 $Y=(A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$ ，当 $B=C=1$ 且 A 翻转时，两条路径延迟不同， Y 可能短暂掉到 0；加入共识项 $B \text{ AND } C$ 可保持静态 1。	稳定真值表不变，但过渡行为改变；危险输出要寄存或专门处理。
安全闭环	规格应写成：原始输入先经保护、默认值和消抖，得到语义信号；组合逻辑形成 <code>fault_lamp/no_fault/allow_start</code> ；危险输出经时序边界和驱动级形成 <code>motor_enable_cmd</code> 。	原始触点不能直接驱动执行机构；复位和断线默认必须安全。
芯片案例	7400 展示门芯片电气约定；4000 CMOS 展示高输入阻抗和低静态功耗；6502/Z80 展示外部总线微处理器；8051 展示片上 I/O；8086 展示预取和复用总线；80386 展示 32-bit 和保护；80486/Pentium 展示 cache、流水线和预测。	每个案例都要绑定到本章概念，不能只列名称。
测量计划	万用表确认静态高低电平；示波器观察抖动、边沿、振铃和毛刺；逻辑分析仪记录内部语义事件和时序关系。	三类工具不能互相替代，测点应靠近接收端。

时序词汇	建立时间是采样沿前必须稳定的时间；保持时间是采样沿后必须继续稳定的时间；时钟到输出是触发器输出延迟；时钟偏斜是不同触发器看到时钟沿的差异。	最大延迟影响建立，最小延迟影响保持。
实验记录	记录测试项、条件、期望结果、实测电压或波形、逻辑分析仪结果和结论；注明探头倍率、触发条件、采样率、电源电压和负载。	只写“测试通过”不算结论，记录必须能让第三方复查。
板级边界	地弹会压缩噪声余量；回流路径不连续会增加环路面积；去耦不足会造成供电跌落；串扰和振铃会让接收端误判。	区分布尔函数错误和物理边界错误。
快慢路径	快路径：操作数在寄存器或 L1，分支预测正确，旁路可用；慢路径：cache miss、TLB miss、分支错误、跨核心同步或 DRAM 访问。	低时延来自常见路径短，不代表所有路径都短。
上拉阻值	上界由上升时间决定： $t = R \cdot C \cdot \ln \frac{V_{CC}}{V_{CC} - V_{TH}}$ 代入得 $t \approx R \times 100\text{pF} \times 1.20$ 由 $t \leq 1\mu\text{s}$ 得 $R \leq 8.3\text{k}\Omega$ ；下界由低电平电流决定：拉低时电阻电流 $(5 - 0.4)\text{V}/R \leq 4\text{mA}$ ，得 $R \geq 1.15\text{k}\Omega$ 。取标称 $4.7\text{k}\Omega$ ：上升时间约 $4.7\text{k}\Omega \times 100\text{pF} \times 1.20 \approx 0.57\mu\text{s}$ ，低电平电流约 0.98mA ，两关都过。	只算上升时间会漏掉拉低电流这一关；ln 因子来自电容充电曲线，不是凑出来的系数。
三态总线互斥	使能由同一选择信号经独热译码产生，并在时序边界上先全部断开、下一拍再接通新驱动者（先断后通）。若用 $\text{EN_B} = \text{NOT EN_A}$ 组合派生，反相器延迟窗口内会出现两者同通或同断；同通时两个推挽输出并联，可能一个拉高一个拉低，形成冲突电流。	使能信号属于危险输出，要按毛刺与时序边界管理；分析切换瞬间必须写出电流路径。
选择器实现函数	8 选 1：选择端接 A、B、C，数据端 $D1=D3=D5=D6=1$ ，其余接 0。4 选 1：选择端接 B、C，逐行对照得 $D0=0$ ($m0$ 、 $m4$ 均为 0)、 $D1=1$ ($m1$ 、 $m5$ 均为 1)、 $D2=A$ ($m2=0$ 、 $m6=1$)、 $D3=\text{NOT } A$ ($m3=1$ 、 $m7=0$)。	每个数据输入都要能追溯到真值表的两行；选择端变量顺序一旦接反，整张映射全错。
延迟链核算	典型合计 $4 \times 9\text{ns} = 36\text{ns}$ ；最大合计 $4 \times 22\text{ns} = 88\text{ns}$ ，仍满足 90ns 的要求，但余量只有 2ns 。若误用典型值，会误以为余量 54ns 。设计依据只能是最大值： $88\text{ns} \leq 90\text{ns}$ 。	建立类检查用最大值；典型值只用于估量级，不能写进设计边界。

去耦电容选择	8 路同时翻转搬运电荷 $Q = 8 \times 50\text{pF} \times 5\text{V} = 2\text{nC}$ ；由就近的 $0.1\mu\text{F}$ 供给，跌落 $\Delta V = Q/C = 20\text{mV}$ 。 若无本地电容，约 0.4A ($2\text{nC}/5\text{ns}$) 的瞬态电流穿过约 20nH 走线电感， 将引起约 $L \cdot di/dt = 1.6\text{V}$ 的扰动。方 案：每片 $0.1\mu\text{F}$ 瓷片紧贴电源脚， 电源入口再放 $10\text{--}47\mu\text{F}$ 。	就近是电气要求而非工 艺美观；只写“加 $0.1\mu\text{F}$ ”而没有位置与数 值估算不算完整。
电平转换	直接连接的高电平余量为 $3.0 - 3.15 = -0.15\text{V}$ ，必然失败。方案 之一：用一片 74HCT (5V 供电、TTL 输入阈值 $V_{IH} = 2.0\text{V}$) 作第一级整形， 新余量 $3.0 - 2.0 = 1.0\text{V}$ ，输出已是标 准 5V CMOS 电平；也可用开漏输出 加上拉到 5V，或专用电平转换芯片。 反向 (5V 驱动 3.3V) 可用串联电阻 加钳位二极管，或选用耐 5V 输入的 器件。	先证明原方案余量为 负，再谈转换；不能只 写“中间加一级”而不重 算新余量。

本章的掌握标准不是“会背 AND、OR、NOT、XOR 的真值表”，而是能把每个抽象还原为具体的硬件行为。

- 电平：前一级的最差高/低输出，必须落在后一级输入阈值允许的范围内，并留下噪声余量。
- 路径：任何输出节点都要能写成“高电平路径是否导通、低电平路径是否导通、是否存在冲突电流、是否存在悬空”四项。
- 门级：任选一个二输入门，列出 4 行真值表，再为每一行写出上拉网络和下拉网络状态。
- 组合：半加器的 Sum 是 XOR，Carry 是 AND；全加器是在同一张真值表中纳入进位输入后的组合电路。
- 延迟：同一个逻辑函数可以有不同门级网络，最长路径不同，稳定时间也不同。
- 负载：一个门的输出不是无限共享变量，后级数量、线长和缓冲策略会改变电气行为。
- 边界：外部输入、上电复位、开漏共享线、三态总线和电源域转换都必须先满足边界约定，才能进入普通组合逻辑。
- 安全：内部允许判断不等同于执行机构命令，危险输出必须说明默认状态、使用时刻和毛刺处理策略。
- 芯片案例：7400/4000 系列展示标准门芯片的电气约定；8086/80386 展示组合控制和数据通路如何扩展成处理器；现代 CPU 展示 cache、流水线、预测和片上集成如何降低常见路径的可见等待。
- 检查题：若组合网络输出短暂从 0 跳到 1 又回到 0，而最终值正确，它是否一定构成设计错误？答案取决于后级是否在毛刺期间采样。

经典系统教材常把抽象的位函数和它的物理实现放在同一页上看。只看位函数会漏掉电平、驱动和时序；只看器件又会失去逻辑行为。两者合在一起，才是硬件设计对象。

状态、时钟与同步边界

第一章讲的是组合逻辑：输入稳定后，输出由当前输入唯一决定。它能计算，却不能保存过去。真正的机器必须能把某个结果留下来，等下一个步骤继续使用；也必须能规定“什么时候更新状态”，否则反馈路径会让硬件在一个周期内连续变化，机器无法分步骤推进。

本章将反馈、锁存器、触发器、时钟、复位、寄存器、计数器和状态机合并讨论。读完后，应当能把“状态”理解成真实硬件边界：一些 bit 在时钟边界被提交，边界之间由组合逻辑准备下一值。后续 CPU 的 PC、指令寄存器、控制状态、流水线寄存器和中断入口，都将反复使用这一章的模型。

核心思想

组合逻辑负责准备下一值，状态元件负责在边界保存下一值。同步硬件就是把这两件事反复组织起来。

第一章已经把建立时间、保持时间、时钟到输出和时钟偏斜作为词汇预告。本章从这里正式接上：组合逻辑的输出并不是“最终正确即可”，它必须在目标触发器的采样窗口之前稳定；状态元件也不是理想变量，它采样后需要时间把新值送到 Q 端。同步设计的基本任务，就是把这些时间约定整理成一组可检查的时序关系：时钟到输出说明触发器被时钟触发后 Q 端多久开始并完成变化，它决定下一段组合逻辑最早什么时候开始传播；组合最大延迟说明 Q 端变化经过门和连线到达 D 端的最晚时间，它决定下一个时钟沿到来时结果是否已稳定；建立时间要求 D 端在采样沿之前提前稳定，否则目标触发器会采到过过渡值；保持时间要求 D 端在采样沿之后继续稳定，否则新值会过早冲进采样窗口；时钟偏斜说明源触发器和目标触发器看到时钟沿的时间差，它会压缩有效周期、甚至破坏保持窗口。

本章的版面组织遵循同一顺序：先说明反馈为什么能保存一个 bit，再说明锁存器和触发器如何定义写入边界，随后把多个触发器扩展成寄存器、计数器、移位寄存器和状态机，最后把复位、跨边界输入、实验板接线和数据手册参数整合成完整的工程方法。读者不应把这些小节看成独立名词表，而应把它们看成“状态如何被创建、提交、传播和验证”的连续链条。

一、反馈、锁存器与保存一个 bit

保存一个 bit 的起点是反馈。两个反相器首尾相接，会形成两个稳定状态。若左边节点为 0，右边经过反相后为 1；右边的 1 再反相回来，正好维持左边为 0。反过来，左边为 1、右边为 0 也能稳定。只要没有外部强制改变，这个状态会保持。

该结构说明，“保存一个 bit”对应电路中某些节点在反馈作用下停留于稳定物理状态。一个稳定状态解释为 0，另一个稳定状态解释为 1。状态的本质，是电路有能力在输入不再变化时仍然维持某个内部结果。

从电路角度看，反馈不是抽象箭头，而是输出重新影响输入。两个反相器互相连接时，每一侧都在“支持”另一侧的当前电平。若某个扰动让左侧电压略有上升，右侧反相器会把右侧电压压低，右侧的低电平又通过另一个反相器继续支持左侧高电平。这个正反馈过程让电路迅速落入两个稳定点之一。数字电路把这两个稳定点命名为 0 和 1，但底层仍然是晶体管、电容、电阻和电源共同决定的物理平衡。



两个反相器首尾相接后，任意一个稳定输出都会成为另一侧的输入；反馈线绕到下方，避免穿过门符号和标签。

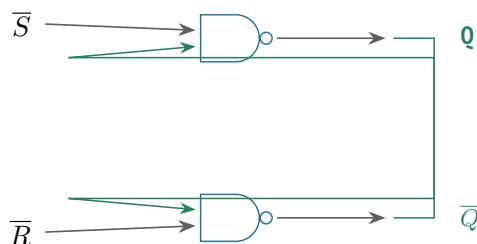
图示：两个反相器首尾相接形成双稳态： $X=0, Y=1$ 稳定； $X=1, Y=0$ 也稳定。

真实锁存结构通常不会直接暴露为两个反相器，而是用交叉耦合的 NAND 门或 NOR 门构成 SR 锁存器。SR 的含义是 set 与 reset: set 请求把状态置为 1, reset 请求把状态置为 0。它已经能表达“外部命令改变内部状态”，但也引入一个重要约束：不能同时给出相互冲突的命令。若 set 和 reset 同时有效，输出可能进入不符合设计语义的状态；当冲突解除时，最终稳定在 0 还是 1 取决于门延迟和释放顺序。这就是第一类状态设计约定：不仅要说明哪个状态合法，还要说明哪些输入组合禁止出现。

把 SR 锁存器的四种输入组合逐行推一遍。两个输入 $\bar{S}$ 、 $\bar{R}$ 都是低有效（为 0 才表示请求），以 NAND 实现为例：

$\bar{S}$	$\bar{R}$	Q	逐行推导
0	0	禁用	两个 NAND 的输入都有 0, Q 和 $\bar{Q}$ 都被迫为 1, 互补关系被破坏；这是非法输入，不是“存了 1”
0	1	1	上侧 NAND 因 $\bar{S}=0$ 输出 1；下侧 NAND 两输入全 1 输出 0, 反馈回去维持 $Q=1$ 。set 生效
1	0	0	与上一行对称： $\bar{R}=0$ 把 $\bar{Q}$ 拉成 1, 反馈把 Q 拉成 0。reset 生效
1	1	保持	无外部请求：若 $Q=1$, 下侧输出 0 支持上侧输出 1；若 $Q=0$ 则反过来。反馈环自己维持旧值

禁行还要单独说一句“释放之后发生什么”。若 $\bar{S}$ 、 $\bar{R}$ 从 0,0 同时回到 1,1, 两个 NAND 的旧输出都是 1, 于是两个门都打算把输出改成 0——谁先翻转，反馈就支持谁，最终状态取决于两个门的实际延迟差。这就是为什么正文说“稳定在 0 还是 1 取决于门延迟和释放顺序”：它不是玄学，是禁行退出瞬间的一场不可预测的竞争。可靠设计要么不产生 0,0, 要么让两个输入不同时释放。



交叉反馈只连接到另一只 NAND 的第二输入，输入线、输出线和反馈线各走独立通道。

图示：由两个交叉耦合的 NAND 门构成的 SR 锁存器。 $\bar{S}$ 与 $\bar{R}$ 低有效；当两者都为高时，反馈维持原状态。

专题：NOR 版 SR 锁存器——同一结构，另一套输入约定

上面用 NAND 搭出的 SR 锁存器不是唯一的搭法。把两个 NAND 换成两个 NOR, 得到的仍是交叉耦合反馈环，存储机制完全一样；变化只发生在输入约定上。NAND 版的 $\bar{S}$ 、 $\bar{R}$ 是低有效：输入为 0 才表示请求，无请求时两者都停在 1。NOR 版的 S、R 是高有效：输入为 1 才表示请求，无请求时两者都停在 0。极性相反不是设计风格问题，而是门本身的函数决定的：NAND 只要有一个输入为 0 输出就是 1，所以

“请求”只能靠 0 来表达；NOR 只要有一个输入为 1 输出就是 0，所以“请求”只能靠 1 来表达。

把 NOR 版逐行推一遍。设上侧 NOR 输出 Q ，下侧 NOR 输出 $\bar{Q}$ ，S 接下侧、R 接上侧。S=1, R=0 时，下侧 NOR 因 S=1 输出 0，上侧 NOR 两个输入全为 0 输出 1， Q 被写成 1，置位生效；S=0, R=1 时对称， Q 被写成 0，复位生效；S=0, R=0 时，两个 NOR 各有一个输入来自对方输出：若 $Q=1$ ，下侧输出 0 正好支持上侧输出 1；若 $Q=0$ 则反过来，反馈环维持旧值；S=1, R=1 时，两个 NOR 都被迫输出 0， Q 与 $\bar{Q}$ 的互补关系被破坏，这就是 NOR 版的禁用组合。两版放在一起对照：

S (NOR)	R (NOR)	$\bar{S}$ (NAND)	$\bar{R}$ (NAND)	锁存器行为
0	0	1	1	无请求：反馈环保持旧值
1	0	0	1	置位： Q 被写成 1
0	1	1	0	复位： Q 被写成 0
1	1	0	0	禁用：set 与 reset 同时请求，互补输出被破坏

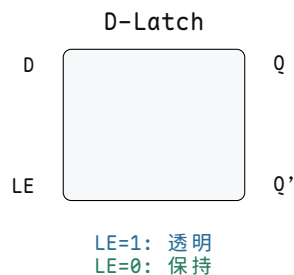
表面看，两版的禁用组合一个是 11、一个是 00，似乎完全不同；实际上它们表示同一件事——set 和 reset 被同时请求。禁用组合指向的从来不是某个二进制图案，而是“两条相互冲突的命令同时有效”这个语义。退出禁用组合时的竞争也一样：NOR 版从 11 同时回到 00，与 NAND 版从 00 同时回到 11 一样，两个门的旧输出都是 0，于是都打算把输出改成 1，谁先翻转反馈就支持谁，最终状态同样取决于门延迟差和释放顺序。

这个对照的意义在于把“结构”和“约定”分开。存储能力来自交叉耦合反馈，它在 NAND 版和 NOR 版里是同一个结构；低有效还是高有效、空闲电平停在 1 还是 0、禁用图案是 00 还是 11，只是套在结构外面的输入约定。数据手册里常见同一功能给出两种极性的型号，读表时应当先确认有效电平，再读功能行；顺序反了，就会把保持行误读成禁用行。

到这里，四种保存 1 bit 的结构可以排成一条线：双反相器反应用两个稳定点保存 1 bit，但没有独立写入控制；SR 锁存器允许用 set/reset 命令改变状态，代价是 set 与 reset 不能给出冲突请求；D 锁存器用一个数据输入和 enable 表达写入，代价是 enable 有效期间输入可以穿透到输出；D 触发器只在时钟边沿采样并保存，代价是必须满足建立时间、保持时间和复位要求。后面的讨论会逐个展开后三种。

单纯反馈还不足以形成可用存储单元。若无法可靠地把它从 0 改成 1，或从 1 改成 0，它只是一个稳定反馈环。因此需要写入控制。锁存器可以理解为带写使能的 1-bit 存储单元：写使能有效时，输入 D 可以影响输出 Q ；写使能无效时，反馈通路维持原来的 Q 。

写使能	数据输入	输出行为
有效	0 或 1	输出跟随输入变化
无效	任意	输出保持原值

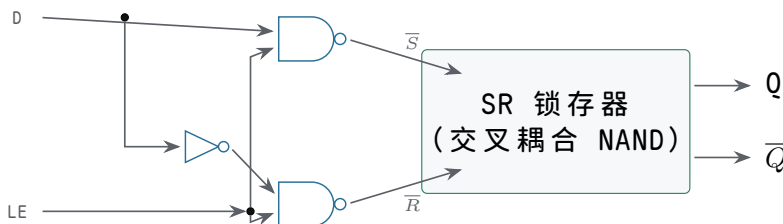


图示：D 锁存器方框符号：LE=1 时 Q 跟随 D（透明），LE=0 时反馈保持原值。

例如一个“按住才能设置”的电路中，enable 为 1 时，D=1 会把锁存器写成 1；enable 回到 0 后，即使 D 变回 0，Q 仍保持 1。这样，过去某一时刻的输入就被硬件保存下来了。

把 SR 锁存器变成 D 锁存器的意义，在于把两个可能冲突的命令合并为一个数据输入 D 和一个写使能 enable。enable 有效且 D=1 时，相当于请求 set；enable 有效且 D=0 时，相当于请求 reset；enable 无效时，set 和 reset 都不被触发，反馈保持旧值。这样，外部控制逻辑不再同时生成 set/reset 两条危险命令，而是生成“是否写入”和“写入什么值”两个更清晰的问题。

这个改造在门级只需要一个反相器和两个 NAND：

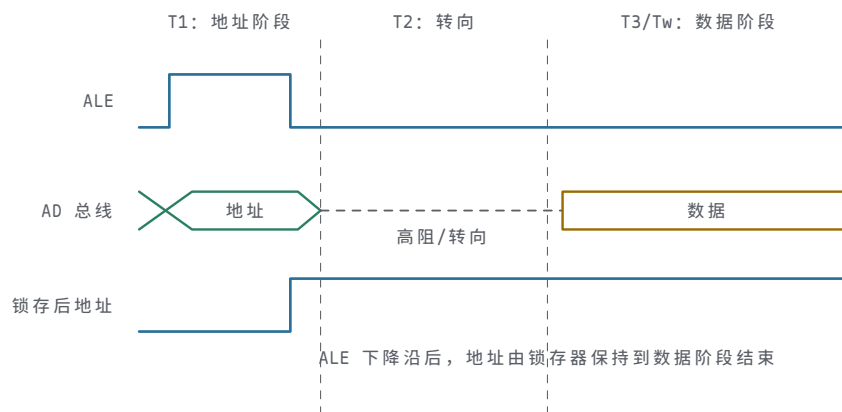


图示：D 锁存器的门级实现：LE=1 时，D 和它的反相信号分别推开两扇 NAND，一路 $\bar{S}$ 或 $\bar{R}$ 被拉低（写入）；LE=0 时两扇都关闭， $\bar{S}$ 、 $\bar{R}$ 同为高（保持）。0,0 禁行从结构上不可能再出现——这就是“把 set/reset 两条危险命令合并成 D 和 LE”在电路结构上的直接体现。

真实器件中的锁存器还会暴露输出使能、低有效清零、三态输出等控制脚。以 74HC373/74HC573 这类透明锁存器为例，锁存使能决定内部是否继续跟随输入，输出使能只决定引脚是否驱动外部总线；二者不能混淆。关闭输出并不等于内部状态消失，打开输出也不等于重新采样输入。后面讨论总线、寄存器堆和 I/O 芯片时，这个区分会反复出现：内部状态、外部驱动、写入边界是三件不同的事。

8086 是理解锁存器用途的经典案例。8086 的部分地址线与数据线复用，地址阶段先在总线上给出地址，随后同一组引脚又可能转为数据传输。外部存储器不能在数据阶段继续把这些引脚当作地址，因此系统板需要由 ALE 这类地址锁存允许信号驱动外部锁存器，把地址阶段的值保存下来。8086 时代常见的板级器件包括 8282/8283 或 74LS373；今天做低速教学实验时也常用 74HC373/74HC573 复现同一思想。这里的锁存器不是“多余缓冲器”，而是把一个总线相位中的地址变成后续相位仍然可用的状态。

总线阶段	锁存器动作	设计含义
地址有效	锁存窗口打开，地址进入锁存器	允许复用总线先表达地址
窗口关闭	内部反馈保持地址	外部存储器继续看到稳定地址
数据传输	总线引脚改作数据或状态用途	地址不会随复用引脚变化而丢失
输出使能控制	决定是否驱动外部总线或后级	不改变锁存器内部保存的值



图示：8086 复用总线的一个读周期：T1 时 ALE 打开锁存窗口，地址随 AD 总线进入锁存器；ALE 落下后窗口关闭，AD 总线转向传数据，而锁存器把地址保持到 T3 采样完成。锁存器把一个总线相位的值变成后续相位仍然可用的状态。

从这个案例可以看出，锁存器适合用在协议已经提供清晰电平窗口的地方，例如地址锁存、外部总线相位分离、显示输出暂存等。若协议没有给出可靠窗口，或者多个状态级之间存在组合反馈，透明锁存器反而会让调试变难。教材后面讲最小 CPU 时，PC、指令寄存器和控制状态一般优先使用边沿触发器；讲总线和 I/O 时，再把锁存器作为相位保持和输出隔离工具使用。

锁存器的问题在于透明窗口。只要 enable 有效，输入变化就可能一路穿透到输出。若多个锁存器共用同一个 enable，并且它们之间还接着组合逻辑，状态可能在一个打开窗口内穿过多级，边界变得模糊。对小电路来说，锁存器易于分析；对大规模同步机器来说，更清晰的做法是把写入集中到离散时钟边界。

透明窗口不是“锁存器不能用”的理由，而是要求设计者明确时间边界。两相锁存器（主、从两级各用半个时钟周期的锁存器）、流水线透明锁存、片内时钟门控单元都可能利用锁存器，但它们依赖严格的相位、非重叠时钟（两相高低时间不重叠的时钟）和时序分析。初学同步机器时，先使用边沿触发器更稳妥，因为每一级状态只在时钟沿提交，调试时可以把每个周期看成独立的一步：上一步的 Q 经过组合逻辑形成本步的 D，本步边界再把 D 提交为下一步的 Q。

专题：门控 D 锁存器的透明窗口——为什么计数器和状态机不用它

门控 D 锁存器在 LE 有效期间是透明的：D 的任何变化经过几级门延迟就出现在 Q 上。这不是缺陷描述，而是工作机制描述——锁存器保存的是窗口关闭那一刻的 D，窗口之内它只是一条带延迟的组合通路。由此推出两个直接后果。第一，晚到的数据会穿透：只要 D 在窗口关闭之前变化，新值就来得及进入并被保存。第二，毛刺也会穿透：D 上的短暂干扰会经过同样的延迟出现在 Q 上，下游看到的是一条和输入几乎一样乱的波形，只是晚了一点。

用一组具体数字感受两种元件的差别。设 LE 窗口宽 20ns ($t=0$ 打开， $t=20\text{ns}$ 关闭)，锁存器传播延迟约 2ns。D 在 $t=17\text{ns}$ 变化， $t=19\text{ns}$ 新值已经出现在 Q 上，窗口关闭时保存的就是这个新值——离关闭只剩 3ns 的变化仍然穿透。D 上一个 2ns 宽的毛刺出现在 $t=10\text{ns}$ ，约 $t=12\text{ns}$ 起 Q 上跟着出现一个毛刺。换成 D 触发器看同样的输入：触发器只关心采样沿前后那一小段窗口，设建立时间 0.5ns、保持时间 0.3ns，那么 $t=10\text{ns}$ 的毛刺和 $t=17\text{ns}$ 的变化只要不落在沿前后共 0.8ns 的窗口里，就不会被采进去。锁存器的敏感区是整个 20ns，触发器的敏感区是 0.8ns，相差 25 倍；这个倍数就是“透明”二字的代价。

输入事件	门控 D 锁存器 (LE 窗口 20ns)	D 触发器 (沿采样)
D 在窗口打开前已稳定	Q 早已跟随，窗口关闭时保存	沿到来时采样，沿后 Q 更新

D 在窗口中段才变化 (晚到数据)	Q 随之更新, 保存的是晚到 的值	距沿超过建立时间则正常 采样, 但 Q 只在沿后才 变
D 在窗口内出现毛刺	毛刺经传播延迟出现在 Q 上	毛刺不落在沿附近窗口内 则完全不可见
D 在窗口外任意变化	无影响	无影响

为什么计数器和状态机必须用触发器, 而不能用共用一个 LE 的透明锁存器? 关键在于这两类电路存在经过组合逻辑回到自身输入的反馈。计数器的下一值是当前值加一, 状态机的下一状态由当前状态和输入共同译码。若状态元件是透明锁存器, LE 打开的整个窗口里, 新状态会穿过组合逻辑再次改变 D, D 再穿透到 Q, 一个窗口内状态可能连续前进好几次, 前进步数由窗口宽度和路径延迟共同决定——这等于把时序问题换成了竞态问题。触发器把这个回路切成离散步: 一个沿只提交一次, 本沿采到的 D 要等下一个沿之后才可能再次影响 D, 于是每台状态机每个周期恰好前进一步。这就是同步设计可分析、可调试的根本原因。

这并不是说透明锁存器一无是处。前面 8086 地址锁存的案例已经说明, 协议给出清晰电平窗口的地方正是锁存器的用武之地; 下一节还会看到, 主从结构正是用两个窗口错开的锁存器拼出一个触发器。工程结论因此是分层而不是否决: 跨相位保持用锁存器, 状态回路的提交边界用触发器。

二、触发器、时钟周期与采样窗口

触发器解决的是“什么时候写入”的问题。它不像锁存器那样在一段时间内透明, 而是在时钟沿附近采样输入, 把采样值保存到输出。

```
on clock edge:
    Q = D
between clock edges:
    Q holds
```

这让大电路有了清楚节奏。一个时钟沿之后, 上一批触发器输出新状态; 这些状态经过组合逻辑传播, 形成下一批输入; 到下一个时钟沿, 目标触发器再统一采样。于是“计算”和“保存”被分开了。

以 74HC74 这类双 D 触发器为例, D 是待采样数据, CP 或 CLK 是采样边界, Q 是保存后的输出, 异步置位和异步清零用于把状态强制带到已知值。它的引脚命名迫使设计者区分三件事: 数据是否已经准备好, 时钟边沿是否合格, 异步控制脚是否处于安全状态。若 D 在边沿附近变化, 触发器不保证采样结果; 若清零脚悬空, 触发器可能在电源噪声中被误复位; 若时钟来自未消抖按钮, 单次按压可能被解释成多个边沿。

把 74HC74 接成最小实验电路时, 应把异步置位和异步清零接到确定无效电平或受控复位, D 端由开关、组合逻辑或前级触发器驱动, CP 端只能接经过整形的时钟。若用按钮作为单步时钟, 按钮必须先经过上拉/下拉、消抖和施密特整形; 否则一次机械按压可能产生多个有效边沿, 使 Q 连续翻转或连续采样。这个案例说明, **时钟边沿质量**与逻辑功能同等重要。

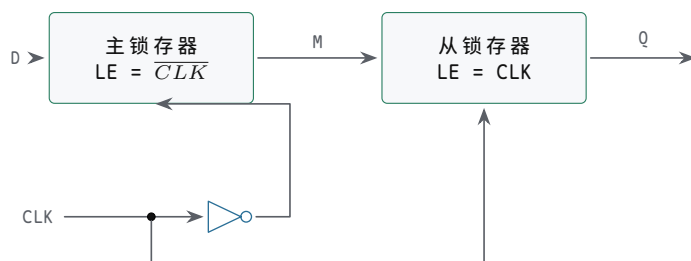
把这只触发器拆开看, 每个对象都有明确的读法。D 输入是下一个边界要保存的候选值, 但“只要最终正确、何时稳定都无所谓”是误解——它必须在采样窗口前后都稳定。CLK/CP 是状态提交的边界, 不是任意跳变都能当时钟, 边沿质量和电平窗口都有要求。Q 输出是已提交的当前状态, 它和 D 只在边界之后才相等, 不能认为随时相等。异步复位/置位不等时钟即可强制状态, 但不能悬空或随意接按钮。数据手册时序(建立、保持、脉宽、传播延迟)不是高频设计才需要读的东西, 低速实验同

样会因为不满足它而失败。



图示：D 触发器方框符号：只在时钟上升沿采样 D 并保存到 Q。

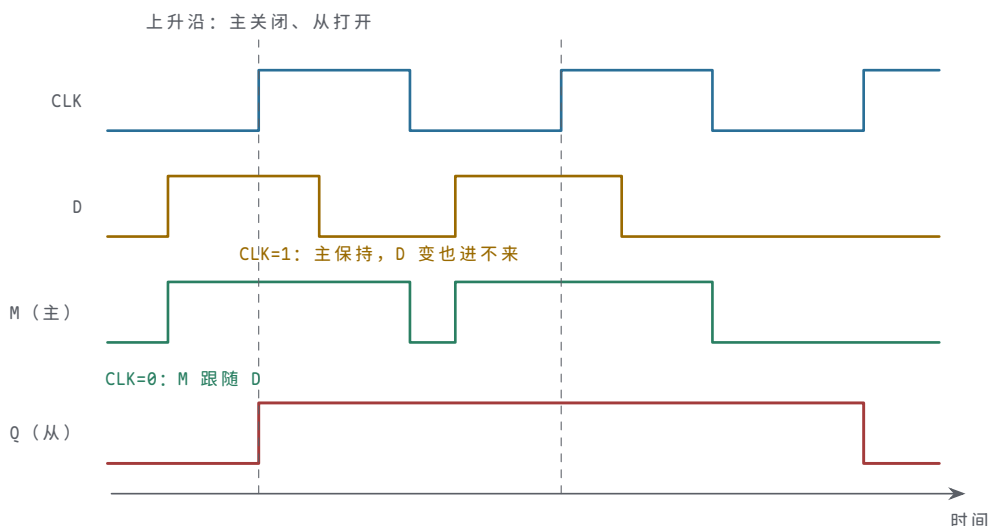
触发器为什么能做到“不像锁存器那样透明”？最常见的实现是主从结构：两个 D 锁存器串联，主锁存器的使能接反相后的时钟，从锁存器的使能接原时钟。CLK=0 时，主锁存器透明、从锁存器保持——D 的变化可以进入主，但到不了 Q；CLK 从 0 变 1 的瞬间，主锁存器关闭、从锁存器打开，主此刻保存的值被交给 Q；CLK=1 时，主保持、从透明——Q 跟随主保存的值，但主不再受 D 影响。所以 Q 只在上升沿那一刻获得新值。边沿触发不是一种新的存储元件，而是两个透明窗口错开半个周期的锁存器组合。



主锁存器在 CLK=0 时透明，从锁存器在 CLK=1 时透明：两个窗口永远错开

图示：主从 D 触发器：D 只能在 CLK=0 期间进入主锁存器；CLK 上升沿把主保存的值移交从锁存器，Q 才更新。

把主从两级放进同一张波形，“边沿”就不再是断言：



图示：主从两级的工作波形。M 在 CLK=0 期间跟随 D，CLK=1 期间保持——所以 D 在 CLK=1 里的翻转（本例 2.8 处）进不了主；Q 只在上升沿复制 M 当时的值。所谓“上升沿采样”，全部秘密就在这两条错开的透明窗口里。

专题：JK 与 T 触发器——从两条命令到一个翻转请求

D 触发器不是唯一的边沿触发器。中小规模器件里长期流行的还有 JK 触发器：它把 SR 的两条命令搬到时钟沿上，J 对应置位请求，K 对应复位请求，并且顺手解决了 SR 的禁用组合问题——J 和 K 同时有效不再是非法输入，而是表示“翻转”。逐行看它的约定：J=1，K=0 时，下一个有效沿把 Q 写成 1，与当前值无关；J=0，K=1

时写成 0；J=K=0 时没有请求，Q 保持；J=K=1 时，下一值被定义成当前值的反，于是每个有效沿 Q 翻转一次。

J	K	下一 Q	含义
0	0	Q	无请求：保持旧值
1	0	1	置位请求：沿后 Q=1
0	1	0	复位请求：沿后 Q=0
1	1	$\overline{Q}$	翻转请求：沿后 Q 取反

对照 SR 锁存器就能看出 JK 的价值：SR 的第四行是禁用组合，JK 的第四行是有明确定义的功能。同一个“两条命令同时有效”的情形，前者只能交给门延迟竞争，后者被约定成确定行为。这也说明触发器家族的差异不在存储机制——它们都在边沿保存 1 bit——而在输入命令集如何约定。

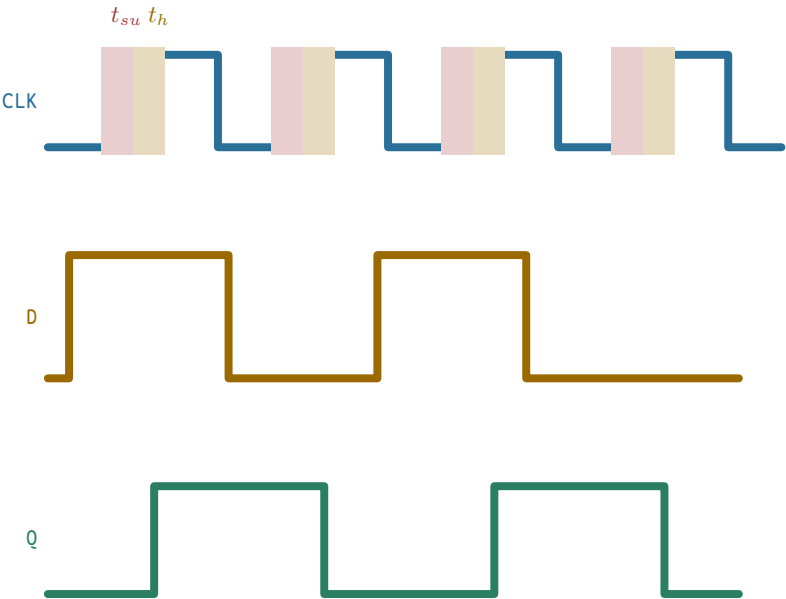
把 J 和 K 短接在一起，JK 触发器就退化成 T 触发器：T=0 相当于 J=K=0，保持；T=1 相当于 J=K=1，每个有效沿翻转一次。T 触发器几乎不单独出现，但它是计数与分频的基本单元。把 T 固定接 1，每个时钟沿 Q 翻转一次：输入每过两个完整周期，Q 才完成一次高低循环，所以 Q 的频率恰好是输入的一半——一只 T=1 的触发器就是一个二分频器。

把三级这样的触发器串起来，前一级的 Q 作为后一级的时钟，就得到一条纹波分频链：

级	时钟来源	输出翻转频率	相对输入时钟
第 1 级 Q ₀	输入时钟 CLK	f/2	二分频
第 2 级 Q ₁	Q ₀	f/4	四分频
第 3 级 Q ₂	Q ₁	f/8	八分频

代入数字：输入时钟 1MHz 时，Q₀ 为 500kHz，Q₁ 为 250kHz，Q₂ 为 125kHz；Q₂ 的周期是 8μs，恰好等于 8 个输入周期。换一个角度看同一条链：每经过 8 个输入沿，三级输出 (Q₂, Q₁, Q₀) 的组合就遍历 8 种编码回到起点，所以三级 T 链同时就是一个模 8 的 3-bit 计数器。分频和计数是同一个电路的两种读法：前者看单路输出的频率，后者看多路输出的编码。本章后面讲计数器时还会看到，纹波结构的代价是各级时钟到输出延迟逐级累加，Q₂ 要等三级延迟之后才稳定；位数一多，这条链本身就成为关键路径，这正是同步计数器存在的理由。

有了触发器的采样模型，建立/保持时间窗口就可以对照波形图逐项检查：



图示：建立/保持时间窗口都相对于上升采样沿：D 必须在上升沿之前 t_{su} 已经稳定，并在上升沿之后 t_h 继续保持；Q 在上升沿后经过 t_{CQ} 才更新。

最常见的同步路径可以写成：

```
source register -> combinational logic -> destination register
```

时钟沿到来后，源寄存器输出旧状态对应的新值。这个值进入组合逻辑，经过门延迟和布线延迟，最终到达目标寄存器输入。下一个时钟沿到来时，目标寄存器采样这个结果。因此频率不是越高越好。频率提高意味着周期缩短，留给组合逻辑稳定的时间变少。若周期短到最慢路径来不及稳定，目标寄存器就可能采到旧值、过渡值或错误值。

同步设计的关键，是区分**当前状态**和**下一状态**。当前状态存在于源寄存器 Q 端，下一状态存在于目标寄存器 D 端。组合逻辑可以在整个周期内反复变化，直到输入稳定、路径传播完成、D 端满足建立时间；但只有时钟沿到来，D 才会成为新的 Q。用软件赋值类比硬件时，最容易犯的误差是把 D 端的临时变化当成已经写入。硬件没有“变量立即更新”的全局瞬间，只有被状态元件定义出来的提交边界。

阅读触发器数据手册时，至少要把四类时间参数分开。第一，时钟到输出延迟说明源状态何时对后继可见。第二，建立时间和保持时间说明目标触发器允许 D 端怎样靠近采样沿。第三，时钟高电平、低电平和脉冲宽度说明怎样的波形才算合格时钟。第四，异步复位或置位的脉宽、恢复时间和释放时间说明复位不能随意撤销。即使低速面包板实验看起来“肉眼很慢”，按钮抖动、慢边沿和悬空输入仍然可能违反这些时间约定。

数据手册项目	应回答的问题	板上故障表现
时钟到输出延迟	Q 何时能作为后继输入使用	后继过早采样旧值或过渡值
建立/保持时间	D 在边沿前后要稳定多久	偶发采错、亚稳态或重复触发
最小时钟脉宽	CP 高低电平是否足够长	窄脉冲被忽略或产生异常采样
复位恢复时间	复位释放离时钟沿是否过近	复位后状态机随机进入非入口状态

触发器采样也不是数学瞬间。输入 D 必须在时钟沿到来前稳定一小段时间，这称为建立时间，违反它会采到错误值或不稳定值；时钟沿之后还要继续稳定一小段时间，这称为保持时间，违反它会让新旧值混入采样窗口。



图示：时钟周期预算：各段时间之和决定最小周期。

一条路径的周期预算可以写成：

```
clock period >= source clock-to-Q
                + combinational delay
                + wire delay
                + destination setup time
```

更严谨地写，还要把时钟偏斜和余量纳入公式。若目标触发器的时钟沿比源触发器更早到达，有效可用时间会变少；若目标时钟更晚到达，建立时间压力可能减轻，但保持时间压力可能增加。为了保持第一轮理解清晰，可以先把建立约束写成：

$$T_{clk} \geq t_{CQ_max} + t_{comb_max} + t_{wire_max} + t_{setup} + t_{skew} + margin$$

其中 t_{CQ_max} 是源触发器时钟到输出的最大时间， t_{comb_max} 是组合逻辑最大延迟， t_{wire_max} 是布线最大延迟， t_{setup} 是目标触发器建立时间， t_{skew} 表示不利时钟偏斜， $margin$ 是留给电压、温度、工艺、建模误差和老化的余量。建立约束回答“最慢结果能否赶上下一个采样沿”。

项目	示例数值	读法
t_{CQ_max}	1.2ns	源寄存器输出最晚 1.2ns 后有效
t_{comb_max}	5.8ns	组合逻辑最坏传播时间
t_{wire_max}	0.9ns	远距离连线 and 负载引入延迟
t_{setup}	0.6ns	目标寄存器采样前要求
$t_{skew}+margin$	0.5ns	偏斜和工程余量
合计	9.0ns	时钟周期必须不小于 9.0ns



图示：异步拉入、同步释放的复位电路。

若目标频率要求周期为 8ns，上表路径就不合格。修复方式不是在公式里删除余量，而是缩短组合逻辑、增加寄存器边界、改善布局布线、换用更快单元、降低频率，或重新安排微操作。同步设计的严肃性在于：功能真值表正确，仅能说明最终会算对；时序约束也满足，才说明能在规定边界前算对。

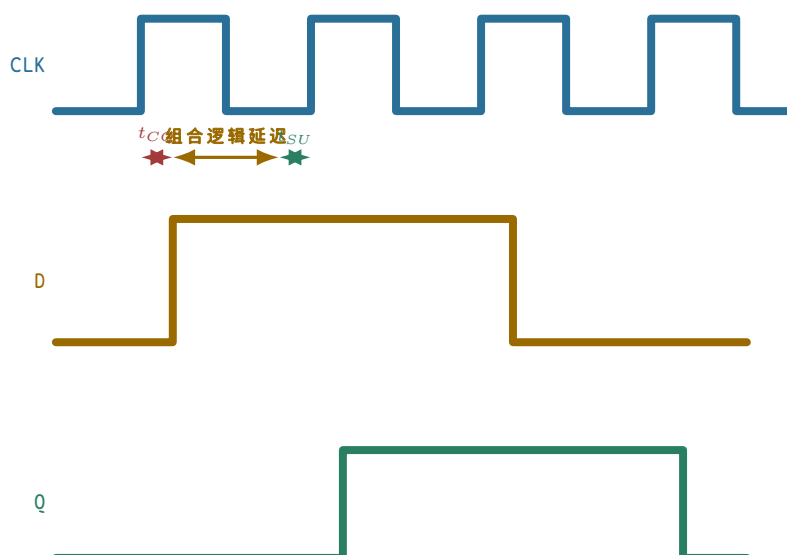
保持时间看起来更小，却同样危险。若源寄存器输出太快、路径太短，目标寄存器可能在同一个时钟沿之后立刻看到新值，从而破坏它本应采旧值的窗口。长路径会限制最高频率，短路径也可能破坏保持约束。

保持约束可以按最小延迟理解：

$$t_{CQ_min} + t_{comb_min} + t_{wire_min} \geq t_{hold} + unfavorable_skew$$

这条式子回答“新值是否来得太早”。例如源触发器最快 0.1ns 就改变 Q，路径中几乎没有组合逻辑，布线也很短，而目标触发器要求保持 0.3ns；若时钟偏斜又让目标采样沿相对更晚，那么新值可能在目标仍需要旧值稳定时抵达。修复保持问题的常见手段是插入小延迟、调整布线、改变时钟树或重新放置寄存器。它和建立问题方向相反：建立嫌路径太慢，保持嫌路径太快。

把这几类约束放在一起看，修复手段完全不同：建立约束失败说明最慢路径赶不上采样沿，要缩短组合逻辑、流水线化、降频或优化布局；保持约束失败说明最快路径太早冲进采样窗口，要增加最小延迟、调整时钟偏斜或改变布线；复位释放失败是状态元件退出复位不一致，要异步拉入、同步释放并限定复位域；跨时钟输入失败是采样沿附近输入不受控，要用两级同步、握手、异步 FIFO 或协议化采样。用建立的修法去修保持，或者用降频去解决复位和跨边界问题，都会把故障掩盖成偶发错误。



图示：同一个上升沿既是源寄存器的更新沿，也是目标寄存器的采样沿：源在 t_{CQ} 后改变 D，组合逻辑在下一上升沿 t_{SU} 前让 D 稳定，目标在该上升沿后 t_{CQ} 更新 Q。 $t_{CQ} + \text{组合逻辑延迟} + t_{SU} = \text{最小时钟周期}$ 。

三、关键路径、复位与跨边界输入

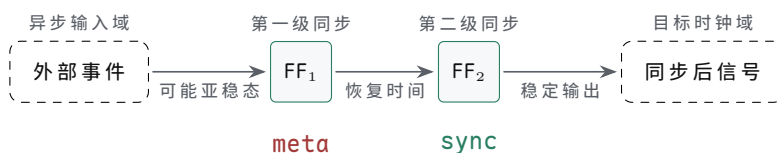
一块同步电路里会有很多寄存器到寄存器路径。最长、最慢、最限制周期的那条叫关键路径。它可能是一串加法器，也可能是多级选择器，也可能是远距离布线加上复杂译码。

例如一个 32-bit 串行进位加法器接在两个寄存器之间。最低位进位要一位一位传到最高位，最高位结果最后才稳定。如果把这个加法器放在一个周期里完成，那么周期至少要等最坏进位链走完。若把运算拆成更短的几段，中间插入寄存器，单个周期可以变短，但完成一次完整运算需要更多周期。

关键路径不只来自“算术看起来复杂”的地方。一个很宽的 mux、远距离控制线、解码后再进入选择器的路径、寄存器堆读口到 ALU 再到写回选择器的路径，都可能成为限制频率的对象。真实 CPU 中，很多优化不是改变逻辑功能，而是重新安排边界：把一次长动作拆成取指、译码、执行、访存、写回；把全局信号改成本地译码；把长总线切成更短的局部通路。时序分析让这些选择可以相互比较，而不是靠直觉判断“这条线应该很快”。

在整机视角下，8086、80386 和现代处理器都在反复处理同一个问题：哪些动作放在一个边界之间完成，哪些动作必须切成多个边界。8086 的外部总线周期把地址、控制 and 数据分成若干相位，用总线相位和等待状态分步完成长访问；80386 的 32-bit 数据通路、分页地址转换和总线接口让路径更宽、更复杂，因此必须更严格地区分内部状态、外部总线状态和等待状态；现代 CPU 则进一步把取指、译码、重命名、执行、访存和提交拆成流水阶段，用局部化、cache 层次和旁路网络缩短常见路径。FPGA/ASIC 数据通路里，宽 mux、长连线、加法器链和译码扇出也按同样思路处理：插入寄存器、重定时、局部译码和布局优化。它们规模不同，但工程判断相同：若一条路径太长，就要缩短组合逻辑、增加状态边界或改变协议等待方式。

这也解释了“一周完成”和“拆段插入寄存器”的取舍：一周完成长运算，控制简单、单条动作需要的边界少，代价是关键路径长、频率低；拆段插入寄存器让单周期路径变短、频率可能更高，代价是完成一次动作需要更多边界和更复杂的控制。



图示：两级触发器同步器结构。

触发器通电后的值不一定符合设计预期。若一组状态寄存器上电后取值各不相同，后面的组合逻辑可能立刻进入非法状态。复位信号的作用，是把关键状态单元带到已知点，例如计数器清零、状态机回到 idle、控制寄存器进入安全默认值。

并不是所有触发器都必须复位。控制状态、外部使能、写存储器信号、片选、错误标志和协议状态通常需要复位，因为它们会决定机器是否对外产生动作；纯数据缓冲如果有可靠写使能保护，未必需要每一位都在复位时清零。过度复位会增加布线、负载和时序压力，尤其在大规模芯片中会让复位网络本身变成困难对象。因此，复位设计不是“全清零”，而是列出哪些状态必须已知、哪些输出必须安全、哪些数据可以等第一次有效写入后再使用。

按这个原则分类：控制状态机必须复位，它要有唯一入口和安全输出；外部写使能和片选必须复位，上电和复位期间不能误写外设或存储器；计数器和超时器通常要复位，初始计数会影响协议等待和超时判断；流水数据寄存器视情况而定，若有 valid 位保护，数据位可以等首次有效写入；同步器和事件 pending 位必须复位，复位释放不能制造伪事件。复位本身也要讲边界。

若复位释放时，有的触发器先退出复位，有的后退出复位，状态机可能看到一半新状态、一半旧状态。常见处理是异步拉入复位以便快速进入安全状态，再同步释放，让所有相关触发器在清楚的时钟边界恢复工作。复位不能简化为“清零所有东西”。哪些状态需要复位，哪些数据寄存器可以不复位，哪些输出在复位期间必须安全，都要由硬件需求决定。

专题：异步复位与同步复位—立即生效与沿上生效的对照

触发器的复位有两条实现路径。异步复位使用触发器专用的 CLR 引脚，直接强制内部反馈环进入 0 状态，不问时钟：引脚一有效，Q 立刻被拉成 0，时钟是否在运行都无所谓。同步复位只是数据路径上的一个约定：复位有效时，D 端之前的选择逻辑把 0 送到触发器输入，Q 要等下一个有效沿才变成 0。前者改变存储元件本身的状态，后者改变准备提交给存储元件的下一值——这是两种复位最本质的区别。

两条路径各有代价。异步复位的优势是上电即有效：时钟还没起振、时钟被门控关闭、系统卡在未知状态时，只有不问时钟的复位才能保证把状态拉到已知点。它的代价在释放端：撤销复位同样是一个异步事件，若释放时刻落在时钟沿附近的恢复时间窗口内，触发器可能这一拍就响应新的 D，也可能仍停留在复位值，结果不可预知。同步复位的优势恰好相反：生效和释放都以时钟沿为界，普通的建立/保持分析就能覆盖，不会制造半个周期的歧义；它的代价是必须有时钟才起作用，并且复位选择逻辑串在 D 路径上，给关键路径增加一级延迟。

关键问题	异步复位	同步复位
何时生效	引脚有效立即生效，不问时钟	下一个有效沿才生效
无时钟时能否工作	能：上电、停振时都有效	不能：沿不来状态不变
数据路径代价	不占 D 路径，使用专用引脚	选择逻辑串入 D 路径，增加一级延迟
释放时刻	仍是异步事件，可能违反恢复时间	由建立/保持分析覆盖，时刻确定
引脚毛刺影响	毛刺直接清掉状态	只有沿附近的毛刺才可能被采入

用一个数量例子看释放风险。设触发器要求的恢复时间是 1.0ns，异步复位在某个沿之前 0.3ns 才撤销——离沿太近，这个触发器本次沿是否退出复位不可预知。若状态寄存器的两个 bit 因复位走线长度不同，一个看到释放离沿 0.3ns，另一个看到 1.2ns，前者行为不定、后者满足恢复时间正常退出，两个 bit 就可能错开一拍退出复位，状态机出现一个周期的混合状态，可能走进任何编码。这就是“异步拉

入、同步释放”成为标准做法的原因：拉入用异步路径，保证任何时刻都能进入安全状态；释放经过本节前面的两级触发器同步到本地时钟边界，让所有状态元件在同一拍恢复工作。两种复位不是互相替代的竞争方案，而是分别负责“进得去”和“出得来”两道工序。

跨时钟或来自外部世界的输入还会带来亚稳态。若输入 D 正好在时钟沿附近变化，触发器内部节点可能短时间落在不稳定区域，既不像明确 0，也不像明确 1。多数情况下，它会在一小段时间后恢复到明确的 0 或 1，但恢复到哪一边、需要多久，不能按普通组合逻辑精确保证。因此来自另一个时钟节奏或外部世界的信号，不能直接拿来控制大面积电路，通常要先经过同步结构，降低亚稳态扩散概率。

最常见的单 bit 同步结构是两级触发器串联。第一级直接采样异步输入，可能进入亚稳态；第二级在下一个时钟沿再采样第一级输出，给第一级更多时间恢复到明确 0 或 1。两级同步不能从数学上消灭亚稳态，但能显著降低亚稳态传播到后级控制逻辑的概率。多 bit 数据不能简单地每一位各接两级同步后直接使用，因为各位可能在不同周期被采到，形成混合值；多 bit 跨边界通常要使用握手、有效位、灰码计数器或异步 FIFO。

工程上常用 **平均无故障时间** (MTBF, Mean Time Between Failures) 描述亚稳态风险是否可接受。它不是功能真值表，而是可靠性指标：异步输入变化越频繁、目标时钟越快，触发器遇到采样窗口的机会越多；留给第一级同步器恢复的时间越长，亚稳态传播到第二级之后的概率越低。因此，提高同步级数、降低异步事件频率、给同步器留出完整周期、避免在同步器后面接复杂组合逻辑，都能改善可靠性。严肃设计不会声称“两级同步一定安全”，而是说明在目标频率、输入事件频率和器件参数下，失效率是否满足系统要求。

放一个数量级感受“恢复时间买可靠性”的杠杆有多大。亚稳态未恢复概率随恢复时间指数下降：恢复窗口每多一个器件时间常数 τ ，失败概率大约下降一个数量级。假设某触发器 $\tau = 0.1\text{ns}$ ，第一级留 1ns (约 10τ) 恢复窗口，单次采样的失败概率约 10^{-10} 量级；异步输入平均每秒变化 1000 次，则大约 10^7 秒 (近 4 个月) 才出现一次可见失败。若只留 0.3ns (约 3τ) ——比如同步器后面紧跟一条长组合路径再吃一级触发器——失败概率变成约 10^{-3} ，同样输入下每天失败几十次。这些数字是示意量级，真实参数必须查器件手册；但结论形态是普适的：**可靠性按指数购买，时钟越快、事件越频繁、恢复窗口越短，买到的越少。**

MTBF 影响因素	对可靠性的影响	设计含义
目标时钟频率	采样次数越多，遇到危险窗口机会越多	高频域更需要规范同步结构
异步事件频率	输入变化越频繁，触发亚稳态机会越多	高频事件应使用握手或 FIFO
恢复时间	两级触发器之间时间越充足，传播概率越低	同步器后不应接长组合路径再采样
器件特性	触发器恢复参数决定概率下降速度	应使用目标工艺或芯片数据手册参数

跨边界对象	推荐处理	不当处理的风险
单 bit 按钮/中断	两级同步后再进入状态机	亚稳态扩散或一次事件被识别多次
多 bit 数据总线	使用握手、valid/ready 或 FIFO	各 bit 来自不同采样时刻，形成错误编码
复位释放脉冲信号	异步拉入、同步释放 转换为电平握手或拉宽后同步	状态机部分触发器先运行 窄脉冲被漏采或重复采样

专题：跨时钟域的两种基本做法——单 bit 同步与多 bit 协议

上表把跨边界对象分了类，这里把背后的两类做法讲透。分类的依据不是信号数量本身，而是“接收方能否容忍采样时刻的不确定”。单 bit 信号只有一个自由度，采早采晚最多差一个周期，两级同步器把亚稳态概率降到可忽略量级之后，剩下的时机不确定性可以靠协议容忍。多 bit 数据的每一位各有自己的采样时刻，逐位同步可能让同一拍采到不同代的 bit，错的就不只是时机，而是数值本身，所以必须引入协议。

单 bit 做法本章已经展开：两级触发器串联，第一级留给亚稳态恢复，第二级输出干净电平，前面也已经算过 MTBF 的账——恢复窗口从 1ns 缩到 0.3ns，失败间隔就从近 4 个月缩成每天几十次。这里只补一条数量级推论：因为失败概率随恢复时间指数下降，MTBF 对余量极端敏感，余量每多一个器件时间常数 τ ，失败间隔大约拉长 10 倍。在两级之后再添第三级，恢复窗口近似翻倍，MTBF 随之再抬高若干个数量级，从“产品寿命内偶见”推到“远远超出产品寿命”；反过来，同步器后面紧接一条长组合路径再吃一级触发器，等于把恢复窗口削掉大半，MTBF 可能从年级跌回天级。结论很朴素：可靠性按指数购买，买多买少由恢复窗口决定，而恢复窗口几乎不花钱——只是触发器和布线。

多 bit 做法先看为什么不能逐位同步。设源域一个 4-bit 计数值从 0111 变成 1000，四个 bit 同时变化；若每位各接一条两级同步器，各路走线和器件延迟略有差异，接收域某一拍可能采到低三位的新值和最高位的旧值——于是接收方依次看到 0111、0000、1000，中间的 0000 是源从未发送过的幻影值。控制逻辑若拿幻影值做判断，机器就按一个不存在的状态行动。逐级加触发器也解决不了这个问题，因为它消除的是亚稳态，不是各位之间的采样时刻分歧。

跨域对象与做法	机制	代价与适用
单 bit 电平或事件：两级同步器	第一级吸收亚稳态，第二级给出稳定电平	延迟一两拍；适用按钮、中断、状态标志
多 bit 批量数据：请求/应答握手	源先放好数据再发请求，数据保持稳定直到应答返回	每次传输数拍；适用低速、非连续传输
多 bit 连续数据流：异步 FIFO	读写两侧各用自己的时钟，指针用格雷码跨域传递	结构复杂、占用资源；适用连续数据流

握手的要点是“数据稳定在先，请求在后，撤销在应答之后”。源域把数据总线放好并保持不变，然后发出 request；request 经两级同步进入接收域时，数据早已稳定了好几个周期，接收方采数据不存在时刻分歧；接收方采完后发 acknowledge，同样同步回源域；源域收到应答之后才允许改变数据。一次传输多花几拍，换来的是多 bit 值作为整体被采走。异步 FIFO 则把这套思想结构化和流水化：写指针在源域递增，读指针在接收域递增，两个指针必须跨域比较才能判断空满；指针采用格雷码，相邻计数值只差一个 bit，于是指针跨域时可以安全地逐位同步——最多把指针读旧一拍，不会读出幻影编码。格雷码在这里的角色值得单独记住：它不是检错手段，而是把“多 bit 同时变化”约束成“一次只变一个 bit”，让逐位同步重新合法化。

安全启动控制板中的 `start_request` 可以作为例子。按钮原始触点先经过默认电平、保护和消抖，得到板内稳定请求；若后级控制器是同步状态机，这个请求还要在控制器时钟域内同步，状态机只能使用同步后的 `start_request_sync`。这样第一章的“可靠 bit”与本章的“可靠采样边界”才真正接上：前者解决电平和路径，后者解决保存时刻。

用 74HC14 和 74HC74 可以搭出一个完整的单 bit 跨边界入口。

74HC14 负责把按钮或慢边沿信号整形成确定翻转的逻辑电平，74HC74 的两级触发器负责把该电平带入目标时钟域。若后级是计数器或状态机，只允许使用

第二级 Q；第一级 Q 是给亚稳态恢复留时间的内部边界，不应直接参与控制。这个小电路比“按钮直接接计数器 CP”更接近真实机器的输入入口。

专题：从外部事件到同步状态机

许多同步电路的故障并不发生在核心状态机内部，而是发生在外部事件进入状态机之前。按钮、传感器、限位开关、中断线、另一片芯片的 ready 信号，都可能在本时钟域的任意时刻变化。若把这些信号直接接到状态机输入，设计等于默认外部世界服从本时钟的建立时间和保持时间；这个默认在真实硬件中通常不成立。

可以把外部事件进入同步状态机的链路写成六段：

```
raw pin
-> electrical default and protection
-> filtering or debounce
-> level normalization
-> clock-domain synchronization
-> event detection or handshake
-> FSM consumption
```

第一段处理电气边界。原始引脚不能悬空，不能让过压、静电或长线干扰直接进入芯片内部；上拉、下拉、限流、钳位、RC 滤波和施密特输入都属于这一层。第二段处理物理抖动或噪声。按钮按下不是一个理想阶跃，可能在数毫秒内多次跳变；传感器线缆也可能受到干扰。第三段把电平极性转成内部语义，例如把低有效的 `start_n` 转成高有效的 `start_pressed`。

第四段才进入本章的核心：同步。一个单 bit 电平在消抖后仍然是异步输入，因为它的变化时刻不由目标时钟决定。两级同步器给第一级触发器留下恢复时间，降低亚稳态传播概率。同步后的信号仍然只是“本时钟域看到的电平”，若状态机需要的是—次事件，还必须再做边沿检测、事件保持或握手确认。

这条链上的每一段都不能替代另一段：默认电平与保护解决悬空、过压、静电和异常电流，但不能保证采样时刻安全；滤波与消抖解决机械抖动、窄噪声和线缆干扰，但不能消除跨时钟亚稳态；语义归一化解决低有效、高有效和断线默认的含义问题，但不能证明事件只发生—次；两级同步把单 bit 异步电平带进目标时钟域，但不能直接处理多 bit 一致性；边沿检测把同步电平转换成单周期事件，但不能保证目标状态机—定接受；握手或事件保持让事件持续到被消费并确认，但不能替代前面的电气保护。跳段的代价不是少—个步骤，而是某—层风险被悄悄漏过去。

以安全启动按钮为例，较完整的命名应当区分每一层信号：

```
start_button_raw      // 引脚上的原始电平
start_button_filtered // 经过默认电平、保护和消抖
start_pressed         // 归—化后的语义电平
start_pressed_meta    // 第一级同步器输出
start_pressed_sync    // 第二级同步器输出
start_event           // 本时钟域检测到的一次请求
start_pending         // 等待状态机消费的保持事件
```

这些名字不是形式主义，而是调试边界。若示波器看到 `start_button_raw` 抖动，问题属于电气或消抖；若 `start_pressed_sync` 偶发异常，重点检查同步器、时钟域和亚稳态扩散；若 `start_event` 出现两次，重点检查边沿检测和消抖窗口；若 `start_event` 只有—次但状态机没有启动，重点检查 `start_pending`、状态转移条件和消费确认。

事件保持尤其重要。假设同步后的 `start_event` 只有—个时钟周期，而状态机当前处在 `RESET_WAIT` 或 `FAULT`，这个事件可能被漏掉。更稳妥的做法是让事件进入 `start_pending` 寄存器，直到状态机在允许启动的状态下消费它，再由 `start_ack` 清除。

```

on clock edge:
    if reset:
        start_pending = 0
    else if start_event:
        start_pending = 1
    else if start_ack:
        start_pending = 0

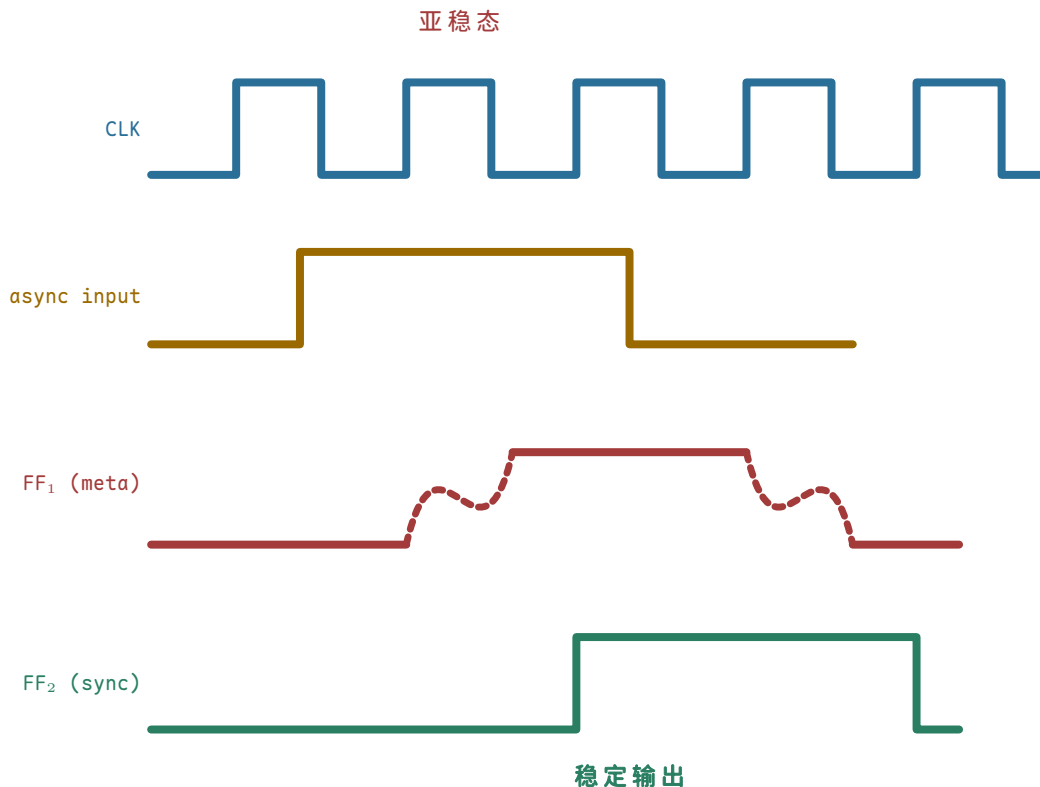
```

这段逻辑说明，外部事件不是直接驱动危险输出，而是先转化为本时钟域内的状态。状态机可以在 `IDLE` 中消费 `start_pending`，在 `FAULT` 中忽略或保留它，在复位期间清除它。这样，事件、状态和安全策略都由时钟边界定义，而不是由一根不受控的外部线决定。

两级同步器只适合单 bit 控制电平或事件标志。若外部输入是 8-bit 数据、地址、计数值或编码状态，逐位接两级同步器不能保证各位来自同一个源时刻。多 bit 跨域必须使用握手、有效信号加稳定保持、格雷码计数器或异步 FIFO 等协议。

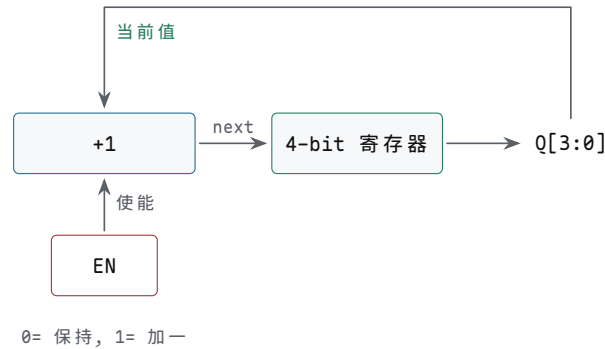
检查一条外部事件链路，可以分四个方面逐项确认。第一，电气处理：默认电平、保护、电压范围和抖动处理，缺失它就会出现悬空、抖动和多次触发。第二，同步处理：进入哪个时钟域、几级同步、复位后同步器处于什么默认状态，缺失它就会出现亚稳态扩散和复位后伪事件。第三，事件处理：电平如何变成一次事件，事件是否保持到被消费，缺失它就会出现窄脉冲漏采或重复消费。第四，状态机接口：哪些状态接受事件，哪些状态拒绝事件，拒绝时事件清除还是保留，缺失它就会出现故障状态误启动或请求丢失。

这个专题把第一章的电气可靠性、本章的同步边界和后续控制器的状态消费连接在一起。对硬件工程而言，“外部输入已经变成 0/1”不是终点；只有当它被目标时钟域可靠采样、被事件协议保持、被状态机按安全条件消费之后，它才成为可用于控制机器动作的内部事实。




```
next = current + 1
on clock edge:
    current = next
```

该结构里有反馈，但反馈不是直接绕回去，而是经过加法器，并且被时钟边界切开。若 `current` 为 3，本周期组合逻辑算出 `next` 为 4；只有下一个时钟沿到来，寄存器才保存 4。随后组合逻辑再从 4 算出 5。如果没有寄存器边界，`current+1` 的结果会立刻回到输入，再加一，再回到输入，电路无法表达“每个周期只增加一次”。



图示：4-bit 同步计数器。

把 3、4、5、6、7 的计数过程画成波形，”整体提交”和”逐位爬”的区别就一目了然：



图示：4-bit 计数器波形：每个上升沿，寄存器把“当前值 +1”整体提交一次，总线带上的值从 3 整齐跳到 4、5、6、7。注意 Q0——它每个周期翻转，正好是 CLK 的二分频；Q1 再慢一半。计数器的每个 bit 都是比它低一位的半速时钟，这是后面分频和定时器的基础。

74HC161/74HC163 这类 4-bit 计数器把这种结构做成了可直接使用的芯片。它们通常有时钟、计数使能、并行装载、清零、进位输出和 4 个 Q 输出。计数使能决定是否执行 `current+1`，并行装载允许在边界写入外部给定值，进位输出用于把多个计数器级联成更宽的计数器。读这类芯片的数据手册时，不能只看“它会加一”，还要查清楚清零是同步还是异步、装载优先级如何、使能脚无效时是否保持、进位输出何时有效。

计数器引脚/功能	对应的状态问题	典型用途
CP/CLK	哪个边界提交新计数值	单步时钟、系统时钟、分频后时钟
ENP/ENT 等使能	本周期是否执行加一	PC 加一、循环计数、定时等待
并行装载	是否用外部值覆盖加一结果	跳转地址、预置计数、状态恢复
清零/复位	初始状态如何确定	上电入口、错误恢复、实验复位

进位输出

更宽计数器何时进位

级联 8-bit、16-bit 或更宽计数器

把两个 4-bit 计数器级联成 8-bit 计数器时，低 4 位的进位不能被理解为普通数据输出。它是高 4 位是否在同一边界加一的控制条件，必须满足高位计数器的使能时序。若把低位 Q 经过随意组合逻辑再驱动高位时钟，就把同步计数器退化成脉动计数器，时序关系会变得难以分析。同步级联的要点是：所有位共享同一个时钟，进位只作为下一状态选择条件。

专题：纹波计数器与同步计数器对照

上一段提醒过：级联计数器时若把低位 Q 经过随意组合逻辑去驱动高位时钟，同步结构就退化成了脉动计数器。脉动计数器也叫**纹波计数器**（ripple counter），它和同步计数器完成同一个功能，但时钟的组织方式完全不同，值得放在一起对照。

纹波计数器的做法极为节省：每个触发器接成翻转模式（把 D 触发器的 $\bar{Q}$ 接回 D），第一级由外部时钟驱动，之后每一级的时钟直接取自前一级的 Q 输出。第 0 位在时钟沿之后先翻转；第 1 位看到第 0 位变化后才跟着翻转；如此逐级传递，第 n 位要等前面各级依次翻完才轮到本级，延迟随位号一级一个 t_{CQ} 累加。变化像波浪一样从低位传到高位，这正是“纹波”二字的来历。

这个逐级延迟不是理论细节。以 4-bit 纹波计数器从 0111 计到 1000 为例，设每级 $t_{CQ} = 1\text{ns}$ ：

时钟沿后时刻	已完成的动作	总线上读到的值
0ns	外部时钟沿到达，各级尚未更新	0111（旧值 7）
1ns	第 0 位翻转完成	0110（读数 6，中间值）
2ns	第 1 位跟随翻转	0100（读数 4，中间值）
3ns	第 2 位跟随翻转	0000（读数 0，中间值）
4ns	第 3 位最后翻转	1000（读数 8，最终值）

在最终值稳定之前，总线上依次出现了 6、4、0 三个并不存在于计数序列中的中间值。若后级只是低速显示驱动，这几纳秒毫无影响；但若后级是译码逻辑——例如“计数值等于 8 时打开某写使能”——中间值 0 就可能让另一路译码输出意外有效，制造窄毛刺。纹波计数器的风险不在计数错误，而在读取窗口内的值不可信。

同步计数器走另一条路：所有触发器共享同一个时钟，每一位的下一值由组合逻辑根据当前各位统一算出（ $\text{next} = \text{current} + 1$ ），在同一个时钟边界整体提交。前文 4-bit 计数器波形里“整体提交、不是一位一位爬”说的就是这件事。代价是每个触发器前面多了下一状态逻辑，位数越宽进位链越长；收益是时钟沿之后只有一级 t_{CQ} 加组合路径的延迟，各位同时稳定，任何时刻读到的都是真实计数值。

速度差异可以用数字算出来。设 $t_{CQ} = 1\text{ns}$ ：纹波结构 8 位时最高位要 $8 \times 1 = 8\text{ns}$ 才稳定，时钟周期必须大于 8ns ，上限约 125MHz ，还未计任何译码余量；同步结构同样 8 位，若下一状态组合逻辑最大延迟 2ns 、目标建立时间 0.4ns ，周期下限约为 $1 + 2 + 0.4 = 3.4\text{ns}$ ， 5ns 周期（ 200MHz ）仍有富余。位数越多，纹波结构的级数惩罚线性增长，同步结构的组合延迟只缓慢增加。

结构	时钟接法	稳定延迟	读数行为	适用场合
纹波计数器	每级时钟取自前一级 Q	低位 1 个 t_{CQ} 起，逐级累加	稳定前出现短暂中间值	低速分频链、定时链、读数不被同域逻辑消费的场所

同步计数器	所有触发器共享时钟	一级 t_{CQ} 加组合路径	边界后各位同时稳定	计数值要被同域译码、比较、控制的场合
-------	-----------	-------------------	-----------	--------------------

选择原则因此很直接：只把计数器当分频器用、中间值无人读取，纹波结构简单省电；计数值要被同一时钟域的逻辑读取、译码或比较，就必须用同步计数器。这解释了为什么 74HC161/163 这类通用计数芯片全部做成同步结构，而纹波结构更多出现在专用分频链和异步预分频器里。

专题：任意模数计数器—复位法与预置法

4-bit 计数器自然按模 16 循环，但十进制显示、BCD 运算和秒分计时需要的是模 10。把模 16 改成任意模 N ，工程上有两种标准做法，差别在“如何让第 N 个状态消失”。

第一种是复位法：让计数器照常加一，另用一个译码门监视计数值，一旦检出 N 就立刻异步清零，不等下一个时钟沿。以模 10 为例，译码门检出 1010（十进制 10），清零端把各级直接拉回 0000：

时钟沿序号	沿后稳定值	说明
0	0000	初始值
1	0001	正常加一
...	...	依次递增
9	1001	最后一个合法值（十进制 9）
10	0000	加一结果 1010 短暂出现，译码门检出后立即异步清零

注意第 10 拍：1010 并非完全不出现，它作为瞬态存在几纳秒——正是这段瞬态触发了清零。这带来复位法的固有弱点：清零脉冲的宽度只有“译码门延迟加清零响应”那么长，若各级触发器清零速度不齐，最快的一级先清掉会使译码条件消失、清零脉冲提前撤除，最慢的一级可能还没清完，计数器从一个错误值重新起步。复位法的优点是只要一个译码门，缺点是把可靠性押在各级清零延迟一致上。

第二种是预置法：不消灭第 N 个状态，而是根本不让它出现。译码门检出的是最后一个合法值 $N-1$ （模 10 时为 1001），检出后拉起同步装载控制，下一个时钟沿把初值 0000 装入，覆盖掉本来的加一结果：

时钟沿序号	沿后稳定值	说明
0	0000	初始值
1	0001	正常加一
...	...	依次递增
9	1001	译码门检出末态，本拍内拉起装载控制
10	0000	时钟沿把初值装入，1010 从头到尾不出现

预置法的每一次变化都发生在时钟边界上，没有任何瞬态；装载控制有整整一个周期的时间稳定，只需像普通同步输入一样满足建立时间。代价是每级前面多一级装载选择逻辑，且译码门在整个 1001 周期内都必须稳定有效，不能带毛刺。

做法	译码的状态	动作方式	有无瞬态	关键问题
复位法	检出 N （如 1010）	异步清零，不等时钟沿	有， N 短暂出现	清零脉冲仅数纳秒，各级清零不齐时可能清不干净

预置法	检出 $N-1$ (如 1001)	同步装载, 等下一 时钟沿	无	装载控制须整拍稳定 并满足建立时间, 多 用一级选择逻辑
-----	----------------------	------------------	---	------------------------------------

两种做法在芯片族里都有对应型号: 74HC160/161 用异步清零, 74HC162/163 用同步清零。同一族芯片同时提供两种清零方式, 正说明这个差别是真实的设计选择, 而不是教科书里的文字游戏。BCD 计数器(模 10)只是最常用的一例; 秒针的模 60、地址回卷的模 2^k 分区, 用的都是同一对方法。

专题: 格雷码、环形与约翰逊计数器

二进制顺序不是唯一的计数方式。改变“下一状态是什么”的规则, 可以得到三类各有专长的计数器, 它们的取舍恰好说明: 状态编码本身就是设计。

格雷码计数器让相邻状态只差一位。3-bit 格雷码的顺序是 000、001、011、010、110、111、101、100, 随后回到 000。普通二进制从 0111 计到 1000 时四位同时变化, 异步采样器在变化途中可能读到任意混合值; 格雷码每步只变一位, 采样器最坏读到旧值或新值, 绝不可能读出第三种组合。本章前面讲两级同步器时提过“多 bit 跨域要用格雷码计数器”, 原因就在这里: 异步 FIFO 的读写指针用格雷码编码, 指针跨域采样时的多值风险被结构性消除。代价是下一状态逻辑比二进制加一复杂, 且状态顺序不再是数值顺序, 不能直接当地址用。

环形计数器把移位寄存器的最后一位接回串行输入, 形成一个环。4 级环以种子值 0001 起步, 依次经过 0001、0010、0100、1000, 再回到 0001。 N 个触发器只得到 N 个状态, 状态利用率很低, 但换来一个独特性质: 每个触发器的输出本身就是状态标志, 译码连一个门都不用——想知道“是否处于第 2 态”, 看第 2 根输出线即可。独热编码的状态机正是这个结构的推广: 把带种子值的环形计数器当作状态寄存器, 一个触发器对应一个状态; 独热编码就是环形计数器的种子值版本。前文状态编码讨论里说的“one-hot 译码直接”, 说的就是这件事。

约翰逊计数器(扭环计数器)只改一处: 接回串行输入的不是最后一位本身, 而是它的反相值。4 级约翰逊从 0000 起步, 依次经过 0000、0001、0011、0111、1111、1110、1100、1000, 再回到 0000—— N 个触发器得到 $2N$ 个状态, 比环形多一倍。更妙的是译码: 每一步仍只有一位变化, 且每个状态都可以由唯一一对相邻位(含首尾相接的一对)识别, 译码一个状态只需一个两输入门。例如状态 0000 由“ $Q_3 = 0$ 且 $Q_0 = 0$ ”唯一确定, 0111 由“ $Q_3 = 0$ 且 $Q_2 = 1$ ”确定。译码门少、每步只变一位, 使约翰逊计数器的译码输出干净无毛刺, 常用于多相时钟生成、步进电机相序和简单分频。

结构	4 级状态数	每步变化位数	译码代价	典型用途
二进制	16	最多 4 位同时变	全译码, 变化途中可能出中间值	通用计数、地址生成
格雷码	16	恒为 1 位	译码无中间态	跨时钟域指针、角度与位置编码
环形	4	2 位(一位退、一位进)	无需译码门, Q 直接当状态标志	独热状态机、简单轮转控制
约翰逊	8	恒为 1 位	每态一个两输入门	多相时钟、步进电机相序、分频

三类结构共用同一个提醒: 未在序列里的编码必须有恢复路径。环形计数器若因上电扰动出现两个 1, 就会在错误序列里一直循环; 可靠设计要在上电时置入种子值, 或加自检逻辑把非法编码拉回主循环——这和状态机“未使用编码要有默认去向”是同一条约定。

专题: LFSR 伪随机计数器

把移位寄存器的某几位抽出来做异或, 把异或结果接回串行输入, 就得到线性反

馈移位寄存器 (LFSR)。抽头位置用多项式记号表示： $x^4 + x^3 + 1$ 表示 4 级结构中取第 4 位和第 3 位（即 Q_3 、 Q_2 ）做异或反馈。设状态写作 $Q_3Q_2Q_1Q_0$ ，每个时钟沿各位左移一格，反馈值 $f = Q_3 \oplus Q_2$ 装入 Q_0 ：

```
f = Q3 XOR Q2
on clock edge:
  Q3 = Q2; Q2 = Q1; Q1 = Q0; Q0 = f
```

这组抽头选得恰当（ $x^4 + x^3 + 1$ 是 4 级的本原多项式之一），LFSR 会遍历全部 $2^4 - 1 = 15$ 个非零状态才重复。从种子 0001 起步，前 8 拍为：

拍	状态 $Q_3Q_2Q_1Q_0$	反馈 $f = Q_3 \oplus Q_2$	说明
0	0001	0	种子值，必须非零
1	0010	0	
2	0100	1	
3	1001	1	
4	0011	0	
5	0110	1	
6	1101	0	
7	1010	1	之后继续遍历其余 7 态， 第 15 拍回到 0001

把 15 个状态写成十进制是 1、2、4、9、3、6、13、10、5、11、7、15、14、12、8，随后精确重复。这个序列“伪”在完全确定、可由种子重现，“随机”在局部统计性质接近真随机序列：任一输出位上 0 和 1 的出现次数只差一次，游程分布也与随机序列一致，因此得名伪随机。

LFSR 的价值在于用极少的逻辑产生长序列。与二进制计数器相比，它没有进位链：每个触发器的下一状态只是一根移位线，唯一的一级组合逻辑是那个异或门，因此同样位宽下更快、更省。三类经典用途：一是伪随机测试，芯片内建自测 (BIST) 用 LFSR 给被测电路灌激励，再压缩响应做签名分析；二是定时器和模数计数，凡是只问“数满多少个周期”、不问数值顺序的场合，LFSR 都比二进制计数器划算；三是扰码，通信链路把数据流与 LFSR 序列异或，打散长连 0 和长连 1，让频谱和时钟恢复更友好。

必须记住两个约束。第一，全零是死状态： $f = 0 \oplus 0 = 0$ ，一旦进入 0000 就永远停在那里，所以上电必须置入非零种子，或加全零检测逻辑把序列拉回。第二，状态顺序不是数值顺序，需要“第几号状态”的场合要查表，不能直接当地址计数器用。另外，LFSR 的内核正是一个移位寄存器——接下来就来正式看移位结构本身。

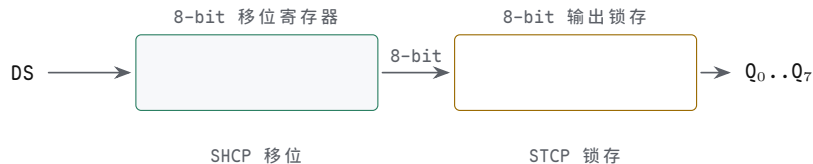
移位寄存器把一组 bit 向左或向右移动。它可以用于串行输入、串行输出、简单延迟线和位级变换。

```
next[3] = current[2]
next[2] = current[1]
next[1] = current[0]
next[0] = serial_in
```

假设 current 为 1010，serial_in 为 1。下一个时钟沿后，寄存器变成 0101。注意，四个位不是按顺序一个一个搬。组合逻辑同时准备好四个 next 输入，时钟沿到来时一起写入。该例说明状态更新不一定是加法，也可以是重新排列、选择、清零、保持或装载外部输入。

移位结构在整机里非常常见。串行外设把一个引脚上的 bit 流变成内部并行字节，显示驱动把内部字节逐位送到外部，流水线把阶段之间的状态向前推进，调试链把片内状态一位一位移出。74HC595 的移位寄存器与输出锁存器分开，正好说明两级状态边界：移位时钟改变内部移位状态，锁存时钟才改变外部可见输出。这个结构避免了外部 LED 或后级芯片在移位过程中看到中间状态乱跳。

74HC595 边界	发生的状态动作	读者应观察到的现象
SHCP 边沿	串行 bit 进入内部移位寄存器	外部输出可保持不变
连续 8 次移位	内部移位状态逐步形成一个字节	中间状态不应直接驱动负载语义
STCP 边沿	内部字节提交到输出锁存器	Q0 到 Q7 一起更新
OE _n /MR _n	控制输出驱动和清零默认状态	低有效脚必须处于确定电平



图示：74HC595：移位寄存器与输出锁存器分开。串行数据经 DS 在 SHCP 边沿逐位移入，外部输出全程不动；STCP 边沿才把整字节一次性提交到 Q0..Q7。

这一类器件提前引出后续流水线寄存器的思想。流水线寄存器保存的不是单个数，而是一组需要一起前进的状态：数据、控制位、valid 位、异常标志和目的寄存器编号。若只有数据向前移动而控制位没有同步移动，后级就会把一个周期的数据解释成另一个周期的操作。移位寄存器是最简单的“状态随边界推进”模型；CPU 流水线是同一思想在更宽状态集合上的应用。

专题：串并转换与动态扫描显示

74HC595 的两级边界前面已经画过。把它真正用到板子上，还要把每个引脚的职责和一种典型负载——多位数码管——连起来看，才能理解为什么这颗芯片几乎成了“输出扩展”的代名词。

先看引脚职责。SER（部分手册记作 DS）是串行数据输入，给出当前要移入的那一位；SRCLK（SHCP）是移位时钟，每个有效沿把 SER 的位移入内部移位寄存器，内部状态前进一格，外部引脚纹丝不动；RCLK（STCP）是锁存时钟，有效沿把整个移位寄存器的内容一次性抄进输出锁存器，8 个输出脚同时更新；OE（低有效）是输出使能，决定锁存器内容是否真正驱动引脚，无效时输出呈高阻。此外还有 SRCLR（MR，低有效）异步清空移位寄存器，以及 Q7' 级联输出，把移出的位交给下一片的 SER。

引脚	职责	关键问题
SER (DS)	串行数据输入，给出待移入的一位	该位必须在 SRCLK 有效沿前稳定
SRCLK (SHCP)	移位时钟，驱动内部移位寄存器	移位期间外部输出不变，这正是设计目的
RCLK (STCP)	锁存时钟，把整字节提交到输出	8 次移位完成后才给，过早会输出半成品字节
OE (低有效)	输出使能，无效时输出高阻	上电期间保持无效，避免显示乱闪
Q7'	最高位移出，级联下一片 SER	多片串联时移位次数按总位数计

两级寄存器为什么输出不抖，值得再说透一层。移位过程必然要经历 8 个中间状态，若这些中间状态直接驱动 LED 或数码管，用户会看到一片乱闪。595 把“内部状态推进”和“外部可见提交”拆到两个时钟沿上：SRCLK 管内部怎么动，RCLK 管外部什么时候看。外部负载在任何时刻看到的要么是完整的旧字节，要么是完整的新字节，绝不会看到移了一半的混合字节——这就是“寄存器是原子提交”在接口芯片

上的一次应用。

最典型的负载是多位数码管。N 位数码管通常段线并联、位线分开：所有位的 a 段接同一根线，每位的公共端各有位选。要显示不同的数字，只能分时复用：每一小段时间只选通一位，同时段码线送出这一位的图案；下一小段时间选通下一位，段码同步换成下一位的图案。只要整帧刷新率超过约 50Hz（工程上常取 100Hz 以上），人眼视觉暂留会把轮流点亮看成同时常亮。

数字算一遍：4 位数码管，每位点亮 2ms，一帧 8ms，帧率 125Hz，每位每秒被刷新 125 次，远在闪烁阈值之上。代价是每位占空比只有 1/4，亮度随之下降，需要在器件允许的脉冲电流范围内提高段电流来补偿。扫描顺序上有一条经验：先消隐、再换码、后点亮——若先开下一位再改段码，上一位的段码会短暂串到新位上，形成鬼影。用 595 驱动段码时这条天然成立：新段码在移位期间被锁存器挡在门内，移完后一拍 RCLK 原子切换，剩下的只是位选时序。

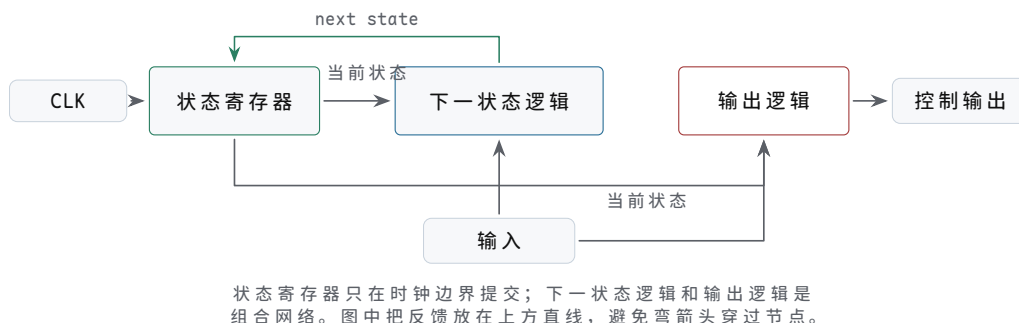
扫描步骤	动作	检查点
消隐	关闭当前位选（或令 OE 无效）	位选与段码不能同时切换
送段码	段码线（或 595）给出下一位图案	段码必须在位选打开前稳定
选通保持	打开下一位位选 维持点亮约 1 至 2ms	同一时刻只有一位被选通 整帧周期小于 20ms，即帧率大于 50Hz
循环	推进到下一位，周而复始	扫描由定时器驱动，不能被长任务卡住

串并转换和动态扫描合起来，说明同一个原则：外部接口的引脚永远比内部状态少，解决办法是把“哪些位先走、哪些位后走”变成明确的时间约定，并用寄存器边界保证每一拍对外都是完整的。

把本节几种结构放在一起看，各自的设计要点不同：普通寄存器保存 ALU 结果、地址和控制字，要写使能只在目标周期打开；计数器用于 PC 加一、循环计数和超时计数，要使能、清零、装载优先级明确；移位寄存器用于串并转换、延迟线和输出扩展，要移位边界和输出锁存边界分开；标志寄存器保存 zero/carry/error 等条件，要说明哪些指令或事件会覆盖标志。

五、状态机把硬件动作切成阶段

有限状态机由三部分组成：当前状态、输入条件、下一状态逻辑。当前状态保存在寄存器里，组合逻辑根据当前状态和输入条件决定下一状态。



图示：有限状态机结构：状态寄存器 + 下一状态逻辑 + 输出逻辑。

设计状态机时，第一步不是写状态名字，而是列出机器必须区分的阶段。若一个控制器要等待请求、装载数据、运行运算、等待外设完成、提交结果，那么这些阶段是否能合并，取决于它们是否需要不同的控制输出和不同的等待条件。第二步才是给阶段编码，并写出每个状态在每种关键输入下的下一状态。第三步是列出输出：

哪些输出只由状态决定，哪些输出还依赖输入，哪些输出必须寄存后再送往外部。

一个完整状态机说明至少包含五张表。第一张是状态含义表，说明每个状态代表哪个硬件阶段——状态命名漂亮但动作不明确，等于没有状态。第二张是转移表，说明当前状态遇到关键输入后进入哪里，等待、错误和默认分支都不能遗漏。第三张是输出表，说明每个状态打开哪些控制线，写使能、片选、ALU 选择、驱动使能不能在错误周期打开。第四张是复位和非法状态表，说明上电、复位释放和异常编码如何处理，入口状态、默认安全输出和恢复路径缺一不可。第五张是波形检查表，说明哪些信号必须在同一周期出现——最终结果正确但中间周期错误，同样是失败。缺少任何一张，状态机都容易退化成“名字看起来对，但边界不清楚”的草率描述。

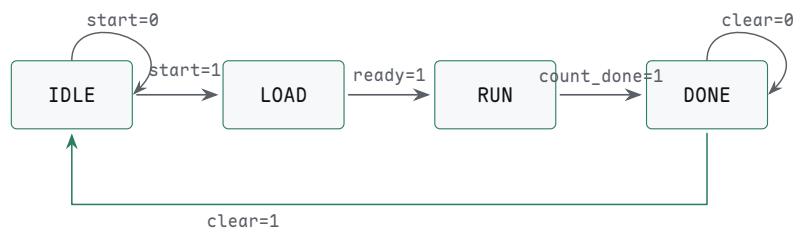
以一个三阶段硬件控制器为例：

当前状态	条件	下一状态
IDLE	start=1	LOAD
LOAD	ready=1	RUN
RUN	count_done=1	DONE
DONE	clear=1	IDLE

若当前状态是 IDLE 且 start=0，状态保持 IDLE；若 start=1，下一状态逻辑准备 LOAD，等时钟沿到来后状态寄存器才真正变成 LOAD。状态机因此不会在一个周期里连续越过 IDLE、LOAD、RUN、DONE，它每次只跨过一个明确边界。

上表还不够完整，因为它只写了“去哪里”，没有写“做什么”。若 LOAD 状态要把外部数据装入寄存器，就必须说明 load_reg_we 在哪个周期为 1；若 RUN 状态要让计数器递减，就必须说明计数使能、ALU 选择和完成条件；若 DONE 状态要向外部报告结果，就必须说明输出是否寄存、ready 信号保持多久、clear 到来前是否允许覆盖结果。状态机设计的核心不是列出状态名称，而是让状态名称、控制输出和被控制的状态元件逐周期对应。

状态	本周动作	关键输出	离开条件
IDLE	等待同步后的启动请求	关闭写使能与外部驱动	start_pending=1
LOAD	装载输入寄存器或初始计数	load_we=1	输入 ready 或装载完成
RUN	执行运算、计数或移位	run_en=1	count_done=1
DONE	保持结果并等待清除	done=1，输出稳定	clear=1



主路径水平推进，返回路径独占下方通道，自环标签放在节点上方，避免和返回线相交。

图示：四状态硬件控制器状态图：IDLE 到 LOAD 到 RUN 到 DONE 再回到 IDLE。每个保持条件画成自环——状态机不是只会前进，更多时候是在等条件。

把这个状态机真正做完一次。第二步是编码：四个状态用两个 bit，IDLE=00、LOAD=01、RUN=10、DONE=11，编码刚好用完，没有非法空洞。第三步写下一状态逻辑和输出逻辑：

```
case (S):
```

```

IDLE: next = start      ? LOAD : IDLE
LOAD: next = ready      ? RUN  : LOAD
RUN:  next = count_done ? DONE : RUN
DONE: next = clear      ? IDLE : DONE
default: next = IDLE

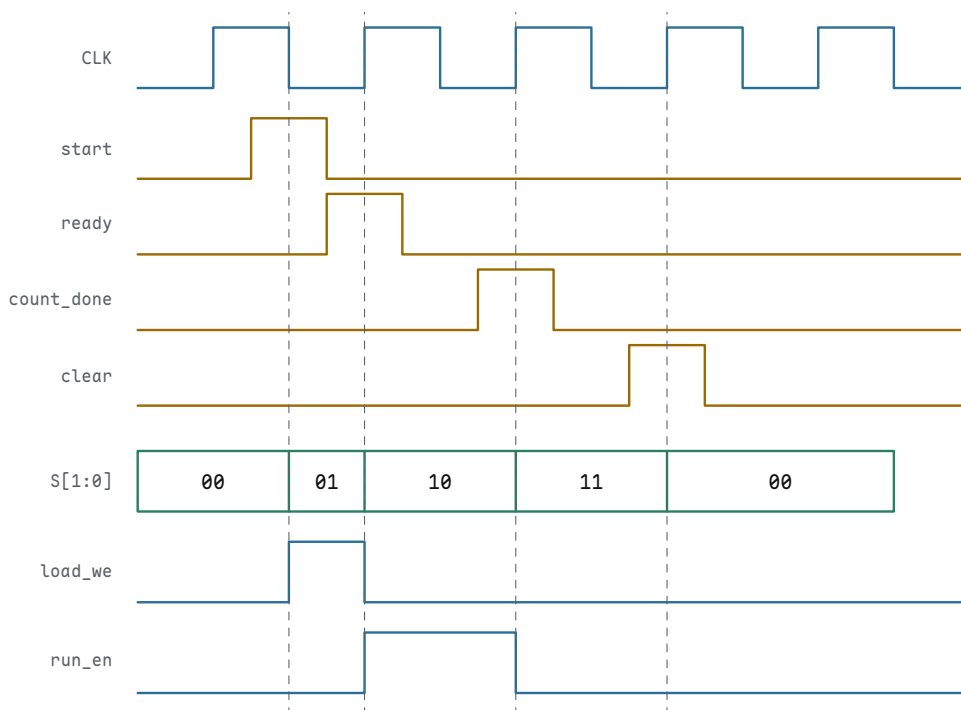
load_we = (S == LOAD)
run_en  = (S == RUN)
done    = (S == DONE)

```

写成布尔式也能机械推导。以 `next_LOAD` 为例：进入 `LOAD` 的条件是“当前 `IDLE` 且 `start=1`”，留在 `LOAD` 的条件是“当前 `LOAD` 且 `ready=0`”。

```
next_LOAD = (S == IDLE AND start) OR (S == LOAD AND NOT ready)
```

第四张表是复位：上电和复位期间状态寄存器强制为 `IDLE=00`，`load_we/run_en/done` 全为 0。第五张表是波形检查：把一次完整运行按时钟对齐画出来，检查每个控制线是否只在规定周期有效。



图示：一次完整运行的对齐波形：输入条件在两个上升沿之间给出，状态只在上升沿跳转；`load_we` 只在 `LOAD` 的那一个周期为高，`run_en` 只在 `RUN` 的两个周期为高。检查一台状态机，就是对照这张图逐周期核对：控制线有没有早一拍、晚一拍或粘在错误的周期。

状态机输出可以只由当前状态决定，也可以同时依赖输入。前者输出更稳定，后者反应更快但更容易受输入毛刺影响。基本原则是：状态机不是若干状态名称的集合，而是“状态寄存器 + 下一状态组合逻辑 + 输出组合逻辑”的硬件结构。

Moore 型输出只由当前状态决定，常用于写使能、片选、锁存时钟、功率使能等需要稳定边界的控制线。Mealy 型输出同时依赖状态和输入，响应可能少等一个周期，但输入毛刺也可能被带到控制线。工程上并不是简单规定只能用哪一种，而是按后果分类：危险输出、外部驱动和写存储器信号倾向于寄存或由稳定状态产生；纯内部、已同步且后级还会采样的条件信号，可以在确认时序余量后使用组合依赖。

专题：Moore 与 Mealy 输出对照

上一段把输出分成“只由状态决定”和“同时依赖输入”两类，并给出了按后果分类的原则。这里用同一个功能把两类做实：检测已同步输入 `sig` 的上升沿，发现后输出一个脉冲 `pulse`。

Moore 型实现需要三个状态。`S_LOW` 表示上一拍 `sig` 为 0, `S_EDGE` 表示刚采到上升沿 (`pulse` 只在此状态为 1), `S_HIGH` 表示 `sig` 已稳定为 1、等待回落:

当前状态	含义	pulse 输出	sig=1 去向	sig=0 去向
<code>S_LOW</code>	上一拍 <code>sig</code> 为 0	0	<code>S_EDGE</code>	<code>S_LOW</code>
<code>S_EDGE</code>	刚采到上升沿	1	<code>S_HIGH</code>	<code>S_LOW</code>
<code>S_HIGH</code>	<code>sig</code> 已稳定为 1	0	<code>S_HIGH</code>	<code>S_LOW</code>

Mealy 型实现只用两个状态, 把“是否刚采到边沿”交给输入参与判断: `pulse` 在“当前状态为 `S_LOW` 且 `sig=1`”的那个周期直接为 1。

当前状态	sig	pulse (本拍)	下一状态
<code>S_LOW</code>	0	0	<code>S_LOW</code>
<code>S_LOW</code>	1	1	<code>S_HIGH</code>
<code>S_HIGH</code>	0	0	<code>S_LOW</code>
<code>S_HIGH</code>	1	0	<code>S_HIGH</code>

对照两种实现的时序。Moore 版在采到边沿的下一拍才进入 `S_EDGE`, 脉冲晚一个周期出现, 但宽度恰为一个完整时钟周期, 且输出只是状态译码, 触发器输出之后最多一级门, 干净稳定。Mealy 版的 `pulse = (state == S_LOW) AND sig`, 在采到边沿的同一个周期内就升高, 抢回了一个周期; 但这条输出路径上有输入 `sig` 的组合分量: 若 `sig` 未经同步、带有毛刺, 毛刺会原样穿到 `pulse` 上; 即使 `sig` 本身已是寄存器输出, `pulse` 也要等沿后 t_{CQ} 加一级门延迟才稳定, 脉冲的头尾不像 Moore 版那样整齐。

维度	Moore	Mealy
本例状态数	3	2
输出决定因素	仅当前状态	当前状态加输入
响应时刻	采到条件后下一拍	采到条件的当拍
输出稳定度	整拍稳定, 沿后至多一级门	受输入路径影响, 沿后 t_{CQ} 加组合延迟
毛刺风险	输出无输入直接路径	输入毛刺可穿透到输出
适用输出	写使能、片选、外部驱动	同域内部、后级会再采样的条件信号

选择原则于是回到前文的按后果分类。输出要驱动外部器件、写存储器、控制功率, 选 Moore, 或者在 Mealy 输出后再加一级寄存器——后者把 Mealy 的早判断和 Moore 的干净边界拼在一起, 代价是又回到晚一拍。输出只在同一时钟域内部被后级再次采样、且功能上必须抢那一拍, Mealy 可以放心用, 前提是输入已经同步、组合路径有时序余量。状态数上 Mealy 常常更省, 因为它把一部分状态信息转交给了输入条件; 但省下的触发器换来的是输出上的组合路径, 这笔交换必须用上面这张对照表逐项想清楚。

一个状态表必须覆盖默认情况。若状态是 `IDLE`、`LOAD`、`RUN`、`DONE`, 而编码使用两个 bit, 则四个编码刚好用完; 若状态更多或编码留有空洞, 未使用编码不能留给综合器或门延迟“自己决定”。安全写法是给下一状态逻辑设置默认恢复路径, 例如进入 `FAULT`、`RESET_WAIT` 或 `IDLE`, 同时让写使能、外部驱动、片选和功率使能默认为关闭。这样, 即使上电扰动、复位释放问题或外部干扰让状态寄存器进入非法编码, 电路也不会在非法状态下驱动危险动作。

把状态机连接到后续 CPU 时, 可以把每条控制线都问成一个状态问题: PC 什么时候写入, 指令寄存器什么时候装载, 寄存器堆写使能在哪个周期打开, ALU 选择哪种运算, 存储器读写请求何时有效, 外设未 ready 时机器停在哪个状态。若这些问题只用“连线会变成正确值”回答, 就没有真正形成控制器; 只有把它们逐一对应

到状态、条件、下一状态和输出时序上，CPU 才能按周期推进。

以最小多周期 CPU 为例，状态机通常至少要区分取指、译码、执行、访存和写回。取指状态打开存储器读请求 `mem_read`，选择 `pc_to_addr`，并在存储器 ready 后装载指令寄存器 (`ir_we`)；译码状态读取寄存器堆并准备立即数，但不写回任何目的寄存器；执行状态选择 ALU 运算 `alu_op` 或计算地址，分支条件和 ALU 结果必须在采样前稳定；访存状态等待数据存储器 ready，期间保持地址和控制线不变；写回状态打开寄存器堆写使能 `rf_we`，且只在目标指令的目标寄存器上打开。若存储器没有 ready，状态机不能假设数据已经存在，而应保持在等待状态并关闭不该打开的写使能。这样，CPU 的每个周期都遵循本章模型：当前状态和寄存器输出经过组合控制，形成下一状态和下一批写入。

这种写法也能解释为什么现代芯片会引入更复杂的控制状态和流水阶段。它们不是为了把教材概念复杂化，而是为了缩短关键路径、隐藏等待、保持高吞吐，并让异常、中断、cache miss、分支错误等事件有可恢复边界。无论是 8086 的总线等待状态、80386 的系统控制状态，还是现代 CPU 的提交队列和异常入口，本质上都要回答同一个问题：某个动作在哪个状态被允许，结果在哪个边界对外可见，失败时回到哪个安全状态。

现在硬件已经能够保存一组 bit、按周期加一、按周期移动、按条件切换阶段。下一步要把这些能力用于更大的动作组织：哪些寄存器在这个周期写入，ALU 做哪种运算，哪一路数据被选择，下一阶段去哪里。这就是控制器的基础。

专题：时钟、复位、状态编码与验证闭环

前面已经把同步边界讲成寄存器到寄存器的路径，但真实工程中还必须处理四个经常被低估的问题：时钟如何送达，复位如何跨越边界，状态编码如何影响安全性，验证如何证明边界确实成立。这四件事不改变“组合逻辑准备下一值、状态元件保存下一值”的基本模型，却决定一个设计能否从纸面状态表进入可靠机器。

首先区分**写使能**和**门控时钟**。写使能的做法是让时钟继续到达触发器，但在时钟沿到来时决定是否接收新 D 值；若写使能为 0，寄存器保持原值。门控时钟则是让某段时钟停止翻转，以降低功耗或隔离无用活动。两者在功能描述上都可能表现为“保持不变”，但硬件风险完全不同。普通数据通路优先使用写使能；真正需要门控时钟时，应使用专用 clock gating 单元，并保证 enable 在安全窗口内被锁存。不能随意用一个 AND 门把 `clk` 和 `enable` 相与后送给触发器，因为 `enable` 的毛刺或变化时刻可能制造短脉冲、窄时钟或额外时钟沿。

把几种保持状态的写法排开看：写使能让时钟照常到达，触发器按使能选择写入或保持，使能信号必须满足目标触发器时序；数据 mux 保持用 `D=mux(we,new,Q)` 写入旧值实现保持，代价是 mux 延迟进入数据关键路径；专用时钟门控用门控单元生成局部时钟，要保证 enable 被锁存、无毛刺、满足时钟树约束；普通门电路直接切时钟最容易产生短脉冲和不可控偏斜，通常禁止使用。

其次，复位也有“域”。

一个时钟域内部可以规定复位异步拉入、同步释放；两个时钟域之间则不能假设复位释放同时发生。若控制状态机已经退出复位，而它依赖的外设状态机、同步器或计数器仍在复位中，就可能读到无效状态。复位域设计应回答三个问题：哪些状态元件属于同一复位域，哪些跨域信号在复位期间必须保持安全，复位释放后第一个有效事件如何产生。

按对象写复位策略：控制状态要复位到 `RESET_WAIT` 或 `IDLE`，上电后进入未定义状态等于没有控制器；数据寄存器只复位影响控制或外部可见安全的那些，大面积无意义复位只会增加布线和时序压力；跨域复位释放要在每个时钟域内部同步释放，并等待依赖域 ready，否则一个域先运行、另一个域仍无效；复位后的第一个事件要特别小心，同步器和 pending 位要进入不会伪触发的默认值，否则复位释放本身就会产生假启动或假中断。

第三，状态机编码不是纯粹的排版选择。二进制编码使用较少触发器，但下一状态译码可能较复杂；one-hot 编码为每个状态使用一个触发器，译码直接、常适合较高速度或 FPGA 结构，但触发器数量更多；格雷编码让相邻状态尽量只改变一位，

常用于跨域计数指针或减少多位同时变化带来的采样风险。无论使用哪种编码，都要定义未使用编码的下一状态和输出默认值。安全相关控制器还应考虑显式安全编码：为非法编码、错误记录和恢复路径预留设计，而不是指望编码本身永远不出错。

第四，验证必须看波形上的边界。只看最终寄存器值，可能错过“某个周期写使能提前打开”“复位释放产生伪事件”“同步器第一级短暂不稳定”“pending 位被错误清除”等问题。一个同步小系统的波形检查至少要同时观察 `clk`、`reset`、源寄存器 `Q`、目标寄存器 `D`、目标写使能、状态编码、同步器两级输出、事件 `pending` 位和 `ack` 信号。

每个对象要检查的问题不同：`reset` 的释放是否在本时钟域边界生效，否则状态机会半周期启动或产生伪事件；`Q -> D` 路径上 `D` 是否是在采样沿前稳定，否则目标寄存器采到过渡值；`write_enable` 是否只在允许写入的周期有效，提前一拍就是寄存器被提前覆盖；同步器的第二级输出是否作为唯一后级输入，后级直接使用 `raw` 或 `meta` 信号等于没有同步；`pending/ack` 的事件是否保持到被消费，窄事件丢失、重复消费和错误清除都是真实失败；非法状态编码是否关闭危险输出并恢复，未使用状态不能驱动写使能或外部输出。

这些检查会直接连接到后续 CPU 章节。PC 是寄存器，指令寄存器是状态，流水线寄存器是阶段边界，控制器是状态机，异常和中断入口依赖同步事件，写回使能必须在正确周期打开。若第二章的边界概念不清楚，后面的数据通路和最小 CPU 就会退化成“线连起来就能跑”的误解。真正的 CPU 不是一张组合逻辑图，而是一组由时钟、复位、状态编码、控制使能和波形验证共同约束的同步系统。

经典案例：交通灯控制器的完整设计

前面的状态机例子只有四五个抽象状态，现在用一个带真实时间需求的案例，把“需求、状态列表、转移表、输出表”完整走一遍。设计一个十字路口的交通灯控制器：东西向绿灯亮 30 秒，转黄灯 5 秒，然后双向全红 2 秒；接着南北向绿灯 30 秒、黄灯 5 秒，再全红 2 秒，随后回到东西向绿灯，如此循环。全红段不是装饰：在一个方向放行之前，两个方向都必须先看到红灯，给已经进入路口的车辆留出清空时间。这 2 秒是用一个专门状态换来的安全余量，删掉它，状态机更简单，路口更危险。

设计的第一步是把时间从状态机里剥离出去。30、5、2 都是计数值，而状态机不应该自己数数。让一个独立的定时器——一个带装载值的递减计数器——负责计数：状态机进入每个状态时，把该阶段的秒数装载进去并启动定时器；定时器减到 0，发出一个周期宽的 `timer_done` 信号；状态机看到它就走向下一阶段，并装载下一阶段的秒数。分工之后，状态机只回答“现在在哪个阶段、下一阶段是什么”，定时器只回答“这个阶段结束了没有”。将来把 30 秒改成 40 秒，只动装载值，状态机结构一行不改。若系统时钟是 1Hz，定时器就是一个最大装载 30 的 5-bit 递减计数器；若系统时钟更快，先分频得到 1Hz 使能，定时器每秒钟才减一次。

状态列表按灯色阶段划分，共六个状态。注意两个全红状态不能合并：它们的灯色输出完全相同，但下一状态不同——一个之后放行南北向，一个之后放行东西向。状态划分由“未来行为是否相同”决定，而不是由“当前输出是否相同”决定；输出相同而去向不同的两个阶段，必须是两个状态。

状态	含义	装载值
<code>EW_GREEN</code>	东西向绿灯，南北向红灯	30s
<code>EW_YELLOW</code>	东西向黄灯，南北向红灯	5s
<code>ALL_RED_EW</code>	东西向刚结束，双向全红	2s
<code>NS_GREEN</code>	南北向绿灯，东西向红灯	30s
<code>NS_YELLOW</code>	南北向黄灯，东西向红灯	5s
<code>ALL_RED_NS</code>	南北向刚结束，双向全红	2s

转移表只有一类条件：`timer_done` 是否为 1。为 0 时保持原状态，下表只列出为 1 时的转移行。

当前状态	条件	下一状态	动作
EW_GREEN	timer_done=1	EW_YELLOW	装载 5s, 重启定时器
EW_YELLOW	timer_done=1	ALL_RED_EW	装载 2s, 重启定时器
ALL_RED_EW	timer_done=1	NS_GREEN	装载 30s, 重启定时器
NS_GREEN	timer_done=1	NS_YELLOW	装载 5s, 重启定时器
NS_YELLOW	timer_done=1	ALL_RED_NS	装载 2s, 重启定时器
ALL_RED_NS	timer_done=1	EW_GREEN	装载 30s, 重启定时器

输出是典型 Moore 型：灯色只由当前状态决定，与 `timer_done` 无关，因此在整个状态期间稳定，输入信号上的毛刺不可能让灯闪一下。

状态	东西向	南北向
EW_GREEN	绿	红
EW_YELLOW	黄	红
ALL_RED_EW	红	红
NS_GREEN	红	绿
NS_YELLOW	红	黄
ALL_RED_NS	红	红

还有两处工程细节值得写明。第一，复位状态应选全红状态而不是某个绿灯状态：上电后的第一件事是让所有方向看到红灯，而不是让某个方向侥幸先走。第二，一个完整循环是 74 秒，但状态机的硬件成本与秒数无关——计时全部由定时器承担。这个案例再次印证本章模型：定时器是“寄存器加递减组合逻辑的反馈”，状态机是“寄存器加下一状态逻辑和输出逻辑”，两者靠 `timer_done` 和装载、启动三条线连接，所有变化都发生在时钟边界上。

专题：状态编码的三种选择

交通灯控制器有六个状态，状态寄存器该用几个触发器、每个状态编成什么二进制值，并不是排版偏好。编码直接决定三件事：触发器用掉多少、下一状态和输出译码的组合逻辑有多深、状态翻转瞬间译码输出会不会出毛刺。三种经典选择是二进制编码、格雷编码和独热编码，下表给出交通灯六状态的三种编法。

状态	二进制 (3 位)	格雷 (3 位)	独热 (6 位)
EW_GREEN	000	000	000001
EW_YELLOW	001	001	000010
ALL_RED_EW	010	011	000100
NS_GREEN	011	010	001000
NS_YELLOW	100	110	010000
ALL_RED_NS	101	100	100000

二进制编码按状态顺序编号，用 $\lceil \log_2 6 \rceil = 3$ 个触发器，最省存储元件；代价是下一状态逻辑和输出译码都要看全部 3 位，逻辑级数最深。格雷编码让循环中相邻的两个状态只差一位：本例刻意选取 000、001、011、010、110、100 这条六状态环，连从末态 100 回到首态 000 的闭环边也只翻最高一位。独热编码为每个状态分配一个触发器，某一位为 1 就表示“正处于这个状态”，译码常常只需看一位，最浅最直接；代价是触发器从 3 个变成 6 个。

编码	触发器数	译码逻辑	适用场合
二进制	3	最深，需综合全部编码位	状态很多、触发器昂贵的 ASIC

格雷	3	深度与二进制相当	相邻转移无中间码，跨时钟域计数指针常用
独热	6	最浅，常常一级	FPGA 触发器丰富，常是默认选择

格雷码的价值要放在“多位同时翻转”的背景下看。状态从 011 变到 100 时三位都要翻，而物理上三位不可能精确同时到达，译码器在翻转窗口里可能瞬间看到 000 或 111；若这个中间码恰好对应某个有效输出，输出线上就冒出一条毛刺。格雷码每次转移只翻一位，不存在中间码，译码窗口天然干净——这也是异步 FIFO 的读写指针跨时钟域传递时一律用格雷码的原因。

独热编码在 FPGA 上流行有结构上的理由：FPGA 的每个逻辑单元都自带触发器，触发器几乎随用随有，而宽输入的译码逻辑要占用查找表层级、拉长关键路径。用 6 个“免费”的触发器换最浅的译码，往往反而更容易满足时序。ASIC 的情形相反，触发器占面积，状态一多就倾向二进制编码。无论选哪种，未使用编码都必须有去处：二进制和格雷各有 110、111 或 101、111 两个空洞，独热更有 $2^6 - 6 = 58$ 个非法编码（多位同时为 1 或全为 0），下一状态逻辑应把它们一律导向全红状态并关闭所有绿灯——非法状态恢复是编码设计的收尾，不是附加题。

经典案例：1101 序列检测器（允许重叠）

第二类经典状态机不控制外部设备，而是识别数据流：每个时钟沿采样 1 位输入 x ，当最近收到的 4 位恰好是 1101 时，输出 y 为 1。要求允许重叠：一次匹配的末尾几位如果同时是下一次匹配的开头，必须被复用，不能匹配一次就从头再来。

设计的关键是给每个状态一个明确含义：“到目前为止，输入串的尾巴与模式 1101 的前缀匹配了多长”。状态不需要记住整段历史，只需记住这个长度——未来能否完成匹配，只取决于它。

状态	含义	已匹配前缀
S0	没有可用前缀（起始或刚失配）	（空）
S1	尾巴是 1	1
S2	尾巴是 11	11
S3	尾巴是 110	110

采用 Mealy 型，输出标在转移上（写法为“下一状态/输出”）：

当前状态	$x=0$	$x=1$
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S3/0	S2/0
S3	S0/0	S1/1

逐行核对这张表，能看出构造方法是完全机械的。在 S2 收到 1：已匹配串变成 111，它不是 1101 的前缀，但其尾巴 11 仍是最长可用前缀，所以留在 S2，不退回起点。在 S3 收到 1：恰好完成 1101，输出 1；同时这一位 1 是下一个匹配的开头，所以退到 S1 而不是 S0。在 S3 收到 0：得到 1100，其尾巴 0、00、100、1100 没有一个是 1101 的前缀，只能回 S0。一般规则是：失配时把“已匹配前缀加上新输入”的尾巴从长到短逐个试，第一个仍是模式前缀的，就是下一状态。

```
input x : 1 1 0 1 1 0 1
state  : S0 S1 S2 S3 S1 S2 S3 S1
output y: 0 0 0 1 0 0 1
```

重叠检测正是靠“匹配后退回 S1 而非 S0”实现的。以输入 11011 为例：第 4 位到达时检测到第一个 1101，机器停在 S1；第 5 位的 1 随即把它推进 S2——第二

个匹配的前缀 11 已经备好。若输入继续 01，即完整序列 1101101，第二个 1101 在第 7 位被认出，它与第一个匹配共享了第 4 位的那个 1。倘若错误地让机器匹配后回 S0，这个共享位就被白白丢弃，1101101 中的第二个 1101 将被漏检。

最后说明为什么用 Mealy 型：检测输出依赖“状态加当前输入”，比 Moore 型少一个 DONE 状态，同拍给出结果，响应快一个周期。代价是 y 直接跟着 x 走， x 上的毛刺会原样出现在 y 上。若 y 只被后级在时钟沿采样，提前一拍是净收益；若 y 要驱动写使能这类危险控制线，应当再寄存一拍，用稳定性换掉这一拍的提前量。四个状态用 2 位二进制编码恰好用完，没有空洞编码。

六、从时序约束到检查方法

同步设计不能仅以“这个状态机功能正确”作为结论。功能正确说明下一状态逻辑在稳定输入下给出正确答案；时序正确说明这些答案能在规定边界前到达，并且不会过早冲入保持窗口。二者缺一不可。分析一条关键路径，应当把它写成四个对象：源状态元件，即哪个触发器在发射沿后改变 Q ，不能只写信号名而不写状态边界；数据路径，即结果经过哪些门、mux、加法器、译码和连线，不能只看逻辑级数而忽略布线和负载；目标状态元件，即哪个触发器在捕获沿采样 D ，不能没有采样窗口；时钟关系，即周期、偏斜、抖动和工程余量，不能用理想同相时钟代替真实时钟树。

建立约束和保持约束检查的是同一条路径的两个方向。建立约束关心最慢数据是否赶得上下一个采样沿；保持约束关心最快数据是否过早进入同一个采样沿后的保持窗口。工程上常见的错误，是只盯着最高频率而忽略保持时间。事实上，降低频率可以缓解建立问题，却不一定修复保持问题；保持问题往往需要改变最小延迟、时钟偏斜或布局。

可以用一个寄存器到寄存器路径作为分析模板。若源寄存器 R_{src} 的 $tCQ_{max}=0.8ns$ ，组合逻辑最大延迟为 $3.6ns$ ，连线最大延迟为 $0.7ns$ ，目标寄存器建立时间为 $0.4ns$ ，不利偏斜和余量合计 $0.5ns$ ，则周期至少为 $6.0ns$ 。若系统要求 $200MHz$ ，即周期 $5.0ns$ ，该路径就不满足建立约束。正确处理方式是把组合逻辑拆段、优化 mux 和加法路径、调整布局或降低频率，而不是把余量从时序预算中删除。

```
required period = 0.8ns + 3.6ns + 0.7ns + 0.4ns + 0.5ns
                = 6.0ns
target period   = 5.0ns
result          = setup violation
```

保持检查则使用最小延迟。若 $tCQ_{min}=0.08ns$ ，组合逻辑最小延迟为 $0.02ns$ ，连线最小延迟为 $0.03ns$ ，目标保持时间为 $0.20ns$ ，再叠加不利偏斜 $0.08ns$ ，则数据最早 $0.13ns$ 到达，而目标要求至少 $0.28ns$ 后才允许变化。这条路径存在保持违例。修复方法通常是在数据路径上增加受控延迟、调整寄存器相对位置、修改时钟树或让工具插入保持缓冲。它不是“频率太高”的问题。

现象	优先检查	不应直接假设
高频下偶发错误	建立约束、关键路径、温度和电压角落	逻辑表达式一定写错
任何频率都偶发错误	保持约束、异步输入、复位释放、亚稳态	降低频率一定有效
复位后偶发失控	复位释放同步、非法状态恢复、安全默认输出	只要复位电平正确就足够
外部按钮偶发多次触发	消抖、同步、脉冲宽度、状态机边界	按钮已经是数字信号

复位要求同样要写成明确的约定。首先列出哪些寄存器必须复位：控制状态、计数器、使能输出、错误状态、协议状态通常需要复位；纯数据缓冲如果写使能受控，未必每一位都需要复位。其次说明复位期间输出是否安全，例如电机使能、写存储

器使能、片选信号和外部驱动必须进入禁止态。最后说明复位如何释放：异步拉入可以快速进入安全状态，同步释放可以避免同一状态机内不同触发器在不同边界开始运行。

按对象核对默认值：控制状态机复位到 IDLE 或 RESET_WAIT，只能有一个合法入口状态；写使能复位为 0，复位期间不能写任何寄存器或存储器；外部驱动进入禁止输出或高阻安全态，不能误启动执行机构；错误状态被清除或进入诊断态，要能区分复位清除和真实故障；跨域同步器进入不会触发事件的状态，释放后不能产生伪脉冲。

跨时钟边界要按对象分类。单 bit 电平可以使用两级同步；窄脉冲应先转换为持续足够长的电平事件，或使用握手协议；多 bit 数据不能逐位同步后直接合并，因为各位可能来自不同采样边界；计数器跨域常用格雷码，异步 FIFO 则用读写指针、同步后的指针和空满判断来建立跨域约定。这里的关键不是背某一种结构，而是承认两个时钟域之间没有共同的“同时”概念，必须让协议定义何时有效、何时可采样、何时完成。

74HC74 这类双 D 触发器可以作为本章的门槛案例：它把 D、CLK、Q、复位和置位引脚明确暴露出来，迫使读者区分数据输入、采样边界和异步控制。74HC161 这类同步计数器则展示“当前状态经加法路径反馈到下一状态”的标准结构。74HC164、74HC595 等移位寄存器进一步说明，多 bit 状态可以按固定连线和时钟边界逐步移动。使用这些芯片时，必须检查电源去耦、输入默认电平、时钟边沿质量、复位极性和输出负载；否则纸面上的状态表无法保证板上行为。

专题：把本章改写成纸面设计审查

本章概念可以用常见 74HC/74HCT 逻辑芯片搭成低速实验板，但本书不要求读者实际搭建。更适合本书目标的训练，是把这些器件当作可审查的真实案例：根据数据手册写出电源、输入默认值、复位极性、时钟边界、输出负载和故障表现。若读者没有面包板和仪器，也可以完成同样的纸面设计审查；若读者选择实际搭建，则必须限制在低压直流逻辑范围内，例如 3.3V 或 5V 数字电路，不得把这些小信号电路直接接到市电、电机、继电器线圈或其他大功率负载。LED 必须串联限流电阻，CMOS 输入不能悬空，每片芯片的 VCC 与 GND 之间应靠近芯片放置约 0.1uF 去耦电容，实验板和信号源必须共地。

数据手册是搭电路时的依据。正文用信号名说明连接关系，而不依赖某一种封装的脚号；实际搭建时必须以手中芯片和封装的数据手册为准。低有效输入常写作 reset_n、MR_n、OE_n 或 RD_n，含义是拉低有效、拉高无效。若把低有效复位误当成高有效复位，电路会长期处于复位或完全失去复位保护。

器件	本章对应概念	搭建时的重点
74HC14	施密特输入、慢边沿整形、按钮消抖前后边界	输出反相；若内部语义需要同相，可串联两级反相器
74HC74	正沿 D 触发器、两级同步器、事件保持位	异步置位/复位脚必须接到确定无效电平或受控复位
74HC161/74HC163	4-bit 同步计数器、当前状态加一、进位级联	区分异步复位和同步复位，计数使能与并行装载不能悬空
74HC157	数据选择器、写使能保持、D=mux(we,new,Q)	让时钟连续运行，用数据路径决定写入或保持
74HC595	串入并出移位寄存器、输出锁存、三态输出	区分移位时钟和锁存时钟，输出负载要受限
NE555 或晶振模块	低速实验时钟	时钟边沿应再经过施密特整形，按钮时钟必须消抖

如果只做纸面训练，不需要准备物料；需要准备的是三张表：器件功能表、引脚电平表和故障定位表。器件功能表说明每颗芯片在本章概念中扮演什么角色；引脚电平表说明每个输入脚在复位、空闲和动作时处于什么电平；故障定位表说明某个现象优先怀疑哪条边界。下面保留实际搭建所需物料，是为了让这些纸面表格有真实约束来源，而不是暗示读者必须做实物实验。

具体到本章实验，常用芯片包括 74HC14、74HC74、74HC161 或 74HC163、74HC157、74HC595；常用阻容范围包括 330R 到 1k 的 LED 限流电阻、4.7k 到 10k 的上拉或下拉电阻、100nF 去耦电容，以及 100nF 到 1uF 的按钮滤波电容。逻辑分析仪或示波器不是第一步必需品，但一旦要判断“按一次为什么计了两次”“复位释放为什么产生假事件”“锁存时钟为什么没有更新输出”，它们就比肉眼观察 LED 更可靠。

物料	用途	检查要点
稳定直流电源	给所有逻辑芯片提供同一电源域	电压在芯片允许范围内，实验板各处共地
0.1uF 去耦电容	抑制芯片开关瞬间的局部电源扰动	每片芯片靠近 VCC/GND 放置一颗
上拉/下拉电阻	给按钮、复位和未驱动输入确定默认值	松开按钮或开关断开时节点不悬空
LED 与限流电阻	低速观察 Q、同步级和计数输出	LED 电流不超过芯片输出能力
按钮与 RC 滤波	产生人工事件和复位请求	按钮边沿进入 74HC14 前已有默认电平
逻辑分析仪或示波器	观察边沿、抖动、短脉冲和时序关系	能同时看到 raw/pressed/sync/clock/reset_n

上电前应先按连接检查表逐项确认。许多低速实验失败不是因为状态机概念错误，而是因为 VCC/GND 接反、低有效输入悬空、输出直接短接、不同电压域硬连或 LED 没有限流。对 CMOS 逻辑而言，“未使用输入不接”不是中性状态，而是让输入节点暴露给噪声和漏电流；它可能让芯片耗电增大、输出随机变化，甚至把后级状态机带入错误状态。

检查项	必须确认	常见后果
电源脚	每片芯片的 VCC、GND 与数据手册一致	芯片发热、无输出或永久损坏
去耦	每片芯片电源脚旁有 0.1uF 电容	时钟沿或输出翻转时产生偶发错误
低有效脚	reset_n/MR_n/OE_n 等处于确定有效或无效电平	电路长期复位、输出高阻或随机启动
未用输入	接到确定 0 或 1，不能悬空	LED 随机闪烁，计数器误跳变
输出负载	LED 串电阻，不能多个推挽输出硬短接	输出电流过大或总线冲突
电压域	3.3V 与 5V 芯片互连满足输入阈值和绝对最大额定值	高电平不被识别，或 5V 损伤 3.3V 输入

搭建顺序也应分层进行。第一步只接电源、去耦、一个 LED 和限流电阻，确认电源轨、极性和 LED 方向。第二步只接 74HC14 与按钮，验证按钮松开和按下时输出有确定变化。第三步接 74HC74 两级同步器，用低速时钟观察 button_meta 和 button_sync 的周期差。第四步再接 74HC161/74HC163 或 74HC595。若一次把所有芯片都插上，故障会混在一起，读者很难判断问题属于电源、按钮、时钟、复位还是状态逻辑。

第一个可搭电路是“按钮进入同步状态机”的前半段。按钮一端接 GND，另一端通

过约 10k 电阻上拉到 VCC，形成低有效原始输入 `button_raw_n`。在按钮节点到 GND 之间可并联 100nF 左右电容作为低速消抖起点；随后进入 74HC14。由于 74HC14 是反相施密特触发器，经过一级后得到边沿更陡但极性反转的信号，经过第二级后可得到与内部语义一致的 `button_pressed`。这个电路对应外部事件链路中的“默认电平、滤波、语义归一化”。

然后用 74HC74 搭两级同步器。第一触发器的 D 接 `button_pressed`，CP 接系统时钟，异步置位保持无效，异步复位接受控的 `reset_n`；第一触发器的 Q 记为 `button_meta`。第二触发器的 D 接 `button_meta`，CP 接同一个系统时钟，Q 记为 `button_sync`。状态机、计数器或后级逻辑只能使用 `button_sync`，不能直接使用 `button_raw_n` 或 `button_meta`。若要观察现象，可把 `button_pressed`、`button_meta`、`button_sync` 分别通过较大限流电阻接到 LED；在 1Hz 到 5Hz 的慢时钟下，读者能看到同步输出只在时钟边界变化。

```
button_raw_n -> 74HC14 -> 74HC14 -> button_pressed
```

```
74HC74 stage 1:
```

```
  D = button_pressed
```

```
  CP = clk
```

```
  Q = button_meta
```

```
74HC74 stage 2:
```

```
  D = button_meta
```

```
  CP = clk
```

```
  Q = button_sync
```

该实验的观察重点不是“按键能点亮 LED”这么简单，而是要记录四个事实。第一，按钮松开时原始节点有确定默认电平。第二，施密特输入之后的边沿不再长时间停留在中间电压。第三，`button_meta` 可能比 `button_sync` 早一个周期变化，后级不得使用它。第四，复位拉低后两个触发器进入不会伪触发的默认值，复位释放后输出只能在时钟沿重新进入有效状态。

第二个可搭电路是 74HC161 或 74HC163 计数器。把 CP 接低速时钟，计数使能接有效电平，并行装载保持无效，复位接到受控 `reset_n`，Q0 到 Q3 分别通过限流电阻接 LED。此时四个 LED 显示的是同一个时钟边界保存的 4-bit 状态，而不是四个彼此独立的灯。每来一个有效时钟沿，内部寄存器一起更新，组合加一逻辑准备下一个值。若使用 74HC161，应特别注意其复位口是异步复位；若教材实验要强调“同步释放”，可以让外部复位先进入本时钟域同步器，再驱动后级控制逻辑，或选用同步复位方式更清楚的计数器。

这个计数器实验可以验证三件事。第一，把时钟停住后，Q0 到 Q3 保持不变，说明状态由触发器保存。第二，把计数使能关掉后，时钟仍然可以继续进入芯片，但状态保持，说明写使能和门控时钟不是一回事。第三，若把并行装载打开并在 D0 到 D3 上给出固定值，计数器会在规定边界装载该值，说明状态元件可以执行“保持、加一、装载、清零”等不同下一状态选择。

第三个可搭电路是 74HC595 输出链路。它内部有移位寄存器和输出锁存器，通常使用 `DS` 作为串行数据，`SHCP` 作为移位时钟，`STCP` 作为输出锁存时钟，`MR_n` 作为低有效清零，`OE_n` 作为低有效输出使能。每给 `SHCP` 一个有效边沿，串行 bit 进入移位寄存器；连续移入 8 bit 后，再给 `STCP` 一个边沿，八个输出 Q0 到 Q7 才一起更新。这个器件非常适合说明“内部状态移动”和“外部可见输出提交”是两个边界。

```
for each bit:
  set DS stable
  pulse SHCP
```

```
after 8 bits:
  pulse STCP
```


Q0..Q7 update together

若用按钮手动提供 DS、SHCP 或 STCP，这些按钮信号仍然要先经过上拉/下拉、消抖和施密特整形。未经消抖的手动时钟可能一次按压产生多个边沿，使移位寄存器多移几位。若用微控制器驱动 74HC595，要保证电压域兼容；5V 逻辑输出不能随意接到只允许 3.3V 的输入。Q0 到 Q7 可以驱动小电流 LED 作为观察负载，若要控制继电器、电机或更大负载，必须增加合适的驱动器件、续流保护和隔离策略，不能把 74HC595 输出脚当作功率开关。

第四个可搭电路用 74HC157 和 74HC74 说明写使能。把 74HC74 的 Q 反馈到 74HC157 的一个输入，把新数据 new_bit 接到另一个输入，用 write_enable 选择哪一路送到 74HC74 的 D。时钟始终送到 74HC74；当 write_enable=0 时，D 得到旧 Q，下一个边界写回旧值，表现为保持；当 write_enable=1 时，D 得到 new_bit，下一个边界写入新值。这个实验应当与“用 74HC08 把时钟和 enable 相与”明确区分。后者即使在低速实验中似乎能工作，也会把 enable 的毛刺、传播延迟和不对称边沿带进时钟路径；它适合用逻辑分析仪作为反例观察，不适合作为后级状态机的正常时钟。

实验	观察信号	预期现象
按钮同步器	raw/pressed/meta/sync/reset_n/clock	后级只在时钟边界看到同步后的事件
4-bit 计数器	clock/reset_n/enable/Q0..Q3	关使能时状态保持，开使能时每周期加一
74HC595 输出链路	DS/SHCP/STCP/MR_n/OE_n/Q0..Q7	移位期间输出不乱跳，锁存边界后一起更新
写使能寄存器	new_bit/write_enable/D/Q/clock	时钟连续存在，是否更新由数据路径决定

故障定位应从可观察事实开始，而不是先猜逻辑表达式。若计数器按一次按钮跳多格，优先检查按钮抖动、手动时钟是否经过 74HC14、是否把按钮直接当 CP 使用。若 LED 随机闪烁，优先检查 CMOS 输入是否悬空、复位或输出使能是否处于确定电平、面包板电源轨是否断开。若 74HC595 的 Q0 到 Q7 始终不变，应检查 OE_n 是否允许输出、MR_n 是否保持无效、STCP 是否真的收到锁存边沿，而不是只检查 SHCP。若复位后出现一次假启动，应检查同步器和 pending 位的复位默认值，以及复位释放是否在本时钟域边界发生。

失败现象	优先检查	修复方向
按一次计多次	按钮抖动、手动时钟、74HC14 整形	增加 RC 消抖，整形后再进时钟域
LED 随机闪烁	悬空输入、复位脚、输出使能脚、电源轨	给所有输入确定默认电平并检查共地
复位后假事件	同步器复位值、pending 位默认值、释放边界	异步拉入、同步释放，事件位复位为 0
74HC595 输出不变	OE_n/MR_n/STCP/SHCP 与数据稳定时间	区分移位时钟和锁存时钟，固定控制脚电平
写使能保持失败	mux 选择脚、Q 反馈路径、D 采样边界	观察 write_enable/D/Q/clock 的周期关系
芯片明显发热	电源极性、输出短路、总线冲突、负载电流	立即断电，按数据手册逐脚复查

专题：从数据手册到可靠接线

真实芯片不能只按逻辑符号连接，还要按数据手册给出的电气边界连接。数据手

册通常有两类容易混淆的表。**绝对最大额定值** (absolute maximum ratings) 是器件不应超过的损伤边界，不是推荐工作点；长期按这个边界运行并不可靠。**推荐工作条件** (recommended operating conditions) 才是设计应采用的电源、电压、温度、输入上升下降时间和时钟条件范围。实验报告应记录使用的是哪一种条件，而不能把“没有立刻烧毁”当成合格。

数据手册项目	读法	设计用途
绝对最大额定值	超过可能损伤器件	用来划红线，不用来定正常工作点
推荐工作条件	厂商承诺正常工作的范围	决定 VCC、电压域、温度和输入边沿要求
VIH/VIL	输入被承认为 1 或 0 的电压门限	判断 3.3V 输出能否驱动 5V 输入
VOH/VOL	在给定输出电流下仍能保证的输出电平	判断下一级是否能可靠识别高低电平
输出电流	单脚和整片芯片的源/灌电流限制	决定 LED 电阻、负载数量和是否需要驱动器
传播延迟	输入变化到输出有效的最坏时间	计入时序预算，不能只看典型值
最小时钟脉宽	CP、SHCP、STCP 等高/低电平至少持续多久	判断按钮、NE555 或逻辑脉冲是否太窄
最高工作频率	在规定电压、负载和温度下的频率上限	面包板实验应留出明显余量

电压兼容要用最坏条件判断。若一个 3.3V 微控制器输出接到 5V 供电的 74HC 输入，不能只看示波器上有 3.3V 高电平，还要查 5V 条件下该输入要求的 `VIH_min`。如果 `VIH_min` 高于 3.3V 输出在负载下能保证的 `VOH_min`，这个连接就没有静态高电平余量，即使短时间看似能工作，也可能随温度、电源和芯片批次变化而失败。74HCT 系列常用于 5V 系统接收 TTL 电平标准的输入，是因为它的输入门限更适合较低高电平；但 74HCT 输出仍由自身 VCC 决定，不能把 5V 输出直接送入不耐 5V 的 3.3V 输入。

```
static high-level margin:
    driver VOH_min >= receiver VIH_min + margin

static low-level margin:
    driver VOL_max <= receiver VIL_max - margin
```

扇出与负载也要按电流和电平一起看。**扇出** (fan-out) 指一个输出能可靠驱动多少个输入。CMOS 输入静态电流很小，所以低速实验中一个 74HC 输出常能驱动多个 CMOS 输入；但这不等于它能随意驱动多个 LED、长线或电容负载。LED 会消耗直流输出电流，电流过大时 `VOH` 会下降或 `VOL` 会升高，后级输入余量随之变小。长杜邦线和多个输入会增加电容负载，使边沿变慢；边沿太慢又可能让普通 CMOS 输入在门限附近停留过久，造成多次翻转或额外功耗。因此，观察 LED 应使用较大限流电阻，真实负载应使用专门驱动器或晶体管/MOSFET 级。

负载类型	风险	正确处理
多个 CMOS 输入	电容变大，边沿变慢	降低频率、缩短连线，必要时加缓冲器
LED 观察负载	输出电流过大拉低高电平或抬高电平	串较大限流电阻，优先低电流观察
长杜邦线	串扰、反射、接触不良和地线回路	低速实验、短线、共地、必要时示波器确认

继电器或电机	电流和反灌电压远超逻辑芯片能力	使用驱动管、续流二极管、独立供电和隔离策略
多个推挽输出相连	一个拉高、一个拉低会形成短路	只用三态总线或开漏/开集加上拉结构

时钟质量决定状态边界是否真实存在。一个合法时钟不仅要有频率，还要满足最小高电平时间、最小低电平时间、上升/下降时间和边沿单调性。NE555 可以产生低速时钟，但输出边沿、占空比和电源扰动仍需检查；按钮可以产生人工事件，但未经消抖不应直接作为计数器 CP；晶振模块边沿质量较好，但频率可能高到面包板布线、LED 观察和手工调试都跟不上。手工实验建议先用 1Hz 到 5Hz 的可观察时钟验证状态，再逐步提高频率，并在提高频率前去掉重负载 LED 或改用缓冲观察点。

时钟来源	适合用途	额外检查
按钮单步	教学观察、单周期推进	必须消抖并经施密特整形，不直接驱动 CP
NE555	低速自动运行	检查占空比、电源去耦、边沿和最小脉宽
晶振模块	稳定周期时钟	频率、电压域、输出负载和面包板布线余量
微控制器 GPIO	可编程时钟或测试脉冲	电压兼容、脉宽、地线和软件抖动

面包板适合低速验证概念，不等同于可量产硬件。面包板触点有接触电阻和寄生电容，长杜邦线会带来额外电感和串扰，电源轨可能在中间断开，地线回路会让几个芯片看到不同的瞬时参考电位。这些问题在 1Hz LED 实验里可能不明显，在更快边沿和更多芯片同时翻转时就会变成偶发故障。将来进入 CPU 数据通路、总线和存储器章节时，读者应记住：面包板证明“逻辑关系能被观察”，不能证明“任意频率和任意规模都可靠”。

安全边界也要明确。本章实验只验证低压逻辑信号，不验证功率驱动。若要让逻辑电路控制继电器、电机、灯带或其它外部负载，必须增加驱动晶体管或 MOSFET、栅极/基极电阻、续流二极管或 TVS、独立电源退耦、共地或隔离策略，并重新评估电流、热、反灌电压和失效状态。把 74HC595、74HC74 或 74HC161 的输出脚直接当作功率开关，是把逻辑输出误用为能量输出。

下面是一份完整故障报告样例。现象：用按钮给 74HC161 提供单步时钟，按一次按钮时计数器从 3 跳到 5。定位步骤一，逻辑分析仪同时观察按钮原始节点 `button_raw_n`、74HC14 输出 `button_pressed` 和计数器 CP；发现一次按压期间 CP 出现两个上升沿。定位步骤二，断开计数器，只观察按钮链路；发现 RC 时间常数过小且按钮释放也产生窄脉冲。修复动作：增大消抖电容，保证按钮先进入 74HC14，再由整形后的信号产生单步请求；若该请求进入状态机，则继续经过 74HC74 同步。复测结果：一次按压期间 CP 只有一个有效边沿，计数器从 3 变为 4。结论：故障不是 74HC161 的加一逻辑错误，而是手动时钟没有形成唯一边界。

实验报告应像一份小型工程记录，而不是只写“现象正确”。每个实验至少记录芯片型号和系列、电源电压、时钟来源、复位极性、每个低有效控制脚的默认电平、异步输入如何进入同步器、观察到的信号、失败现象和修复动作。若使用 74HCT 与 74HC 混合，还要记录电平兼容依据。一般而言，5V 输出不应直接接入只允许 3.3V 的输入；3.3V 输出能否可靠驱动 5V 供电的 74HC 输入，也不能凭“看起来能亮”判断，而应查数据手册中的输入高电平阈值。74HCT 的输入阈值更接近 TTL 电平标准，常用于 5V 系统中接收较低高电平，但输出仍受自身供电影响。

报告项	应记录的内容	判断依据
物料与版本	芯片完整型号、逻辑系列、电源电压	可回到数据手册复查电气条件

连接约定	VCC/GND、复位、置位、输出使能、未用输入	没有悬空控制脚和危险负载
时钟与复位	时钟来源、频率范围、复位拉入和释放方式	状态只在边界变化，复位后无伪事件
异步输入	原始引脚、消抖、施密特整形、同步级数	后级只使用同步后的信号
观测结果	LED、逻辑分析仪或示波器看到的关键节点	现象能解释到具体边界和信号名
故障处理	失败现象、定位步骤、修复前后对比	修复不是偶然重插线，而有明确原因

这些实验把“状态、时钟与同步边界”的概念应用到真实芯片上。完成实验后，读者应能为每个小电路写出一页完整的实验记录：使用了哪些芯片，电源和默认电平如何保证，哪些输入是异步的，进入了几级同步器，复位默认状态是什么，哪些输出只是观察 LED，哪些信号绝不能直接驱动危险负载。这样的记录比单纯画出逻辑关系更接近真实工程。

因此，本章的最终目标不是让读者背出触发器、寄存器、计数器和状态机的定义，而是能为一个同步小系统留下**可复查的设计记录**：状态表、复位表、时序预算、跨边界处理方式和异常恢复策略。后续 CPU 的 PC、寄存器堆、控制器、异常入口和设备中断，都将重复使用这套记录格式。

专题：setup/hold 不是公式，而是采样窗口 触发器的 setup 和 hold 约束经常被背成两个时间参数，但它们本质上是在说明采样窗口。时钟沿到来前，D 输入要提前稳定一段时间；时钟沿之后，D 输入还要继续保持一小段时间。若输入在窗口内变化，触发器内部再生过程可能无法快速决定 0 或 1，输出延迟变长，甚至进入亚稳态。

时序参数	原理含义	违反后果
t_{setup}	时钟沿前 D 必须稳定的最短时间	采样值可能错误或输出延迟异常
t_{hold}	时钟沿后 D 必须保持的最短时间	新数据可能穿透到本周期采样
t_{clkq}	时钟沿后 Q 输出稳定所需时间	后级组合逻辑可用时间减少
t_{pd}	组合逻辑最坏传播延迟	决定下一级 setup 是否满足
skew	不同触发器实际看到时钟沿的差异	可能同时影响 setup 和 hold 裕量

单周期路径的基本时序关系可以写成：

```
clk_period >= t_clkq_max + t_logic_max + t_setup + skew + margin
hold_margin = t_clkq_min + t_logic_min - skew - t_hold
```

第一条约束决定频率上限，第二条约束决定短路径是否太快。很多初学者只关心“逻辑太慢”，却忽略“逻辑太快”也可能破坏 hold。若同一时钟沿从前级触发器出来的数据太快到达后级，而后级仍处在 hold 窗口内，就可能把本该下一拍才采样的数据提前采进去。解决 hold 的方法通常不是降低频率，而是调整时钟树、增加最小延迟或改变寄存器布局。

专题：时序预算的一个完整数值例

把采样窗口的两条不等式套到一组具体数字上。某设计目标频率 50MHz，即时钟周期 20ns。关键路径参数：源触发器 $t_{\text{CQ_max}}=3\text{ns}$ 、 $t_{\text{CQ_min}}=1.5\text{ns}$ ，组合逻辑最坏延迟 9ns，目标触发器建立时间 $t_{\text{SU}}=2\text{ns}$ 、保持时间 $t_{\text{H}}=0.5\text{ns}$ ，时钟偏斜按最不

利方向取 1ns。时序分析没有更多公式，只有一条纪律：建立检查用“最坏组合”，所有延迟取最大；保持检查用“最好组合”，所有延迟取最小——因为两个问题的敌人方向相反，一个怕数据来得太晚，一个怕数据来得太早。

先做建立检查。数据最晚在时钟沿后 3ns 离开源触发器，再花 9ns 穿过组合逻辑，第 12ns 才到达目标 D 端；目标要求它比下一个时钟沿（第 20ns）至少早 t_{SU} 加偏斜共 3ns 就绪，即第 17ns 之前。12ns 早于 17ns，检查通过，余量 5ns。

```
setup check (worst corner):
arrival   = tCQ_max + t_comb_max = 3 + 9       = 12ns
required  = T - tSU - skew       = 20 - 2 - 1   = 17ns
slack     = 17 - 12              = +5ns (pass)
```

再做保持检查。数据最快在时钟沿后 1.5ns 就可能离开源触发器；若这条路径几乎没有组合逻辑（例如移位寄存器的直接相邻位），新数据第 1.5ns 就到达目标 D 端。而目标的保持窗口要求旧数据至少维持到时钟沿后 t_H 加偏斜共 1.5ns。1.5ns 对 1.5ns，余量恰好是 0：纸面上不违例，工程上叫边缘。

```
hold check (best corner):
earliest  = tCQ_min + t_comb_min = 1.5 + 0     = 1.5ns
required  = tH + skew            = 0.5 + 1      = 1.5ns
margin    = 1.5 - 1.5            = 0ns (marginal)
```

余量为 0 意味着任何一点工艺波动、温度漂移，或实际时钟树偏斜略大于估计值，都会把它推成违例。更要紧的是，降频救不了它：把 50MHz 降到 25MHz，建立余量从 5ns 变成 25ns，保持余量却仍是 0ns——保持窗口跟着时钟沿走，不跟着周期走。修复只有三条路：在数据路径上增加受控的最小延迟（插缓冲器）、减小时钟偏斜（修时钟树）、或换用 t_{CQ_min} 更大的触发器。这就是“降频治不了保持问题”的数值含义。

检查项	代入组合	计算	结论与余量
建立时间	最坏组合： t_{CQ_max} 、组合最大延迟、 t_{SU} 、不利偏斜	$3+9+2+1=15ns$ ，周期 20ns	通过，余量 +5ns
保持时间	最好组合： t_{CQ_min} 、组合最小延迟约 0、 t_H 、不利偏斜	1.5ns 对 $0.5+1=1.5ns$	边缘，余量 0ns，建议加最小延迟
降频效果	周期从 20ns 改为 40ns	建立余量升至 25ns，保持余量仍 0ns	只治建立，不治保持

静态时序分析工具对设计中每一条寄存器到寄存器路径都做这两遍计算，报告里的 setup slack 与 hold slack 就是上面的 +5ns 和 0ns。手工算一遍的价值在于：看到报告时知道每个数字对应哪一段物理延迟，也知道该改逻辑、改时钟树还是改目标频率。

亚稳态只能降低概率，不能被完全消灭 异步输入、跨时钟信号和机械按键都可能在触发器采样窗口内变化。触发器内部的再生电路可能暂时停在中间电平，随后随机解析为 0 或 1，这就是亚稳态。同步器不能保证“永不亚稳”，只能让亚稳态有更多时间衰减，使错误传播到后级的概率低到工程可接受。

设计选择	作用	仍需注意
两级同步器	给第一级亚稳态多一个周期衰减	只能同步单 bit 电平，不能同步多 bit 总线值
握手协议	用 valid/ready 或请求/应答跨边界	吞吐降低，但语义清楚

异步 FIFO	用双端口存储和格雷码指针跨时钟	指针同步和空满判断必须严格验证
脉冲拉宽	让短事件至少覆盖目标时钟周期	过窄脉冲可能完全漏采

多 bit 总线不能简单给每一位各接两级同步器后直接使用，因为各位解析时间可能不同，接收端会看到不存在的混合值。正确做法通常是先在源时钟域保持数据稳定，再用握手同步“数据可用”事件；或者使用异步 FIFO 让写指针和读指针通过受控编码跨域。

复位要区分上电初始化、错误恢复和运行控制 复位不只是“把所有寄存器清零”。有些寄存器必须复位到安全值，有些数据寄存器可以不复位，有些外设需要按电源和时钟顺序释放，有些跨时钟域需要各自同步释放。把复位当成普通控制信号乱用，会造成时钟域之间状态不一致、异步释放毛刺或组合路径被复位扇出拖慢。

复位对象	建议策略	检查要点
控制状态机	复位到明确安全状态	复位释放后第一拍输出是否安全
PC/入口状态 数据寄存器	装入复位向量或启动状态 可按需要不复位，由控制路径 保证无效时不使用	第一条取指地址是否确定 是否有 valid 位防止读旧值
外设接口	先复位设备内部，再开放总线 响应	复位期间是否会误写 MMIO
跨域复位	异步断言、同步释放或每域独 立处理	释放边界是否满足目标时钟 域时序

良好的复位规格应写清楚：复位信号有效电平、最短保持时间、时钟是否必须运行、复位释放后第一个可接受输入的时间、输出是否需要保持安全值。没有这些边界，系统 bring-up 时会把电源、时钟、复位、状态机和总线问题混成一团。

状态机验证要覆盖非法状态和输出时序 状态机图只画正常转移是不够的。真实触发器可能因复位不完整、毛刺、单粒子翻转或设计错误进入未列出的编码。若非法状态没有恢复路径，控制器可能卡死或输出危险控制线。状态机输出还要区分 Moore 型和 Mealy 型：前者只依赖当前状态，后者还依赖输入，可能更快但更容易受输入毛刺影响。

验证项	要检查什么	失败风险
复位覆盖 非法状态恢复	所有关键状态寄存器进入已知状态 未使用编码能回到安全状态或错误 入口	上电随机进入危险路径 状态机锁死或误写设备
输出默认值	每个状态未显式赋值的控制线有安 全默认	锁存器推断或保持旧控 制线
输入同步	Mealy 输入是否来自同步边界	毛刺直接影响输出控制 线
进度性	等待状态是否有超时或退出条件	外设无响应时永久等待

这套检查表会在 CPU 控制器、内存等待、DMA 描述符、总线仲裁和中断控制器中反复出现。第二章的状态机不是孤立数字逻辑练习，而是后面所有“硬件会按阶段推进”的基础。

去抖和边沿检测要分成两件事 机械按键输入常常同时暴露三个问题：异步、抖动和事件检测。异步意味着它不按本机时钟变化，需要同步；抖动意味着一次按下会产生多个跳变，需要滤除；事件检测意味着电平稳定后还要决定什么时候产生一个周期宽的“按下事件”。把三者混在一起，常会导致一次按键被计数多次，或按住不放时不断触发。

```
button pipeline:
  raw_pin -> synchronizer -> debounce counter -> stable_level
  stable_level + previous_level -> one_cycle_press_event
```

阶段	原理	预期现象
同步	两级触发器降低亚稳态传播概率	raw 改变时内部不会产生多 bit 混合状态
去抖	输入持续稳定 N 个周期才接受新电平	抖动期间 <code>stable_level</code> 不变化
边沿检测	当前稳定电平与上一稳定电平比较	一次按下只产生一个事件脉冲
事件消费	状态机或计数器只看事件脉冲	长按不会重复触发，除非规格要求连发

这个小例子把第二章与第一章连接起来：第一章能用示波器看到按键抖动，第二章用同步状态机把抖动变成稳定事件，后面整机章再把该事件映射成 GPIO 状态位或中断请求。

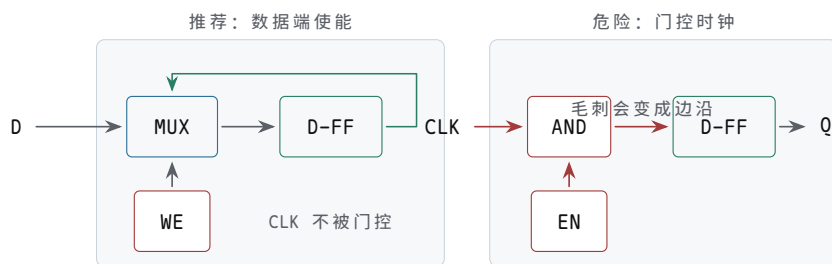
章末练习

本节练习要求把状态边界写清楚。答案必须说明哪个值在组合路径上传播，哪个值在时钟沿保存；若涉及外部输入、复位或跨时钟边界，还必须说明同步和默认安全状态。

任务	要完成的产物	预期结果
入门：画 Q 波形	给定一段 D 和 CLK 波形，分别画出 D 锁存器 (LE=CLK) 和 D 触发器 (上升沿) 的 Q	锁存器 Q 在 CLK=1 全程跟随 D，触发器 Q 只在上升沿变化
入门：填 SR 真值表	补全 $\overline{S}/\overline{R}$ 四种组合下 Q 的值，并标出禁行	禁行说明互补被破坏，不是“存了 1”；释放顺序决定恢复结果
入门：画计数波形	从初值 9 开始画 6 个周期的 Q[3:0] 总线带和 Q0 波形	每个上升沿整体 +1，Q0 恰好是 CLK 二频
入门：主从结构	给定 D 与 CLK，填出主锁存器 M 和输出 Q 在各阶段的值	M 只在 CLK=0 跟随 D，Q 只在上升沿复制 M
触发器路径时序分析	为一条寄存器到寄存器路径列出 t_{CQ} 、组合延迟、连线延迟、建立时间、偏斜和余量	能判断目标周期是否通过建立约束
保持违例	构造一条最小延迟过短的路径，并写出修复方法	能区分建立问题和保持问题
锁存器透明窗口	说明两个锁存器共用 enable 时可能出现的穿透风险	能解释为什么大系统更偏好清晰时钟边界
复位释放	为一个三状态控制器写复位表和释放策略	复位期间危险输出关闭，释放在同步边界发生
两级同步	说明外部按钮或中断线进入时钟域的处理流程	原始异步信号不能直接驱动状态机

MTBF 指标	说明为什么同步器只能降低亚稳态风险，并列出影响 MTBF 的因素	能把时钟频率、事件频率、恢复时间和器件特性联系起来
外部事件链路	为一个按钮或传感器写出从原始引脚到状态机消费的完整信号链	能区分电气处理、同步、边沿检测和事件保持
事件保持	设计 <code>pending/ack</code> 结构，说明何时置位、何时清除	窄事件不能因状态机暂时不可接收而丢失
多 bit 跨域	说明为什么 8-bit 总线不能逐位两级同步后直接使用	给出握手、FIFO 或格雷码方案
脉冲同步	设计一个不会漏采窄脉冲的跨域事件方案	能说明事件保持到被确认
时钟门控	比较写使能、数据 mux 保持、专用时钟门控和普通门控时钟	能说明为什么普通门电路不应直接切时钟
复位域	为两个相关时钟域写复位释放和 <code>ready</code> 等待策略	能说明复位跨域也需要同步和安全默认值
状态编码	比较二进制、one-hot 和格雷编码的用途与风险	能为未使用编码写安全恢复规则
波形验证	列出验证一个同步小系统必须观察的信号	能从波形判断写使能、复位、 <code>pending/ack</code> 是否正确
CPU 连接	说明 PC、流水线寄存器、控制器和中断入口分别对应本章哪个概念	能把第二章边界模型映射到后续 CPU
按钮同步器实作	用 74HC14 和 74HC74 写出按钮到同步电平的连接方案	能区分原始引脚、消抖整形、第一级同步和第二级同步
计数器实作	用 74HC161 或 74HC163 设计 4-bit LED 计数器	能说明时钟、复位、计数使能、并行装载和输出负载
移位输出实作	用 74HC595 说明串行输入、移位时钟、锁存时钟和输出使能	能解释为什么八个输出在锁存边界一起更新
写使能实作	用 74HC157 和 74HC74 搭一个可保持的 1-bit 寄存器	能说明为什么数据 mux 保持优于普通门控时钟
实验板物料	为本章低速逻辑实验列出最小 BOM 和用途	能覆盖电源、去耦、默认电平、观察负载和测量工具
接线检查	写出上电前检查表，覆盖低有效脚、未用输入、电压域和输出负载	能防止悬空、误复位、输出短路和电平不兼容
故障定位报告	针对一个失败现象写出定位步骤和修复验证	能从现象追到具体信号边界，而不是只重插线
数据手册读取	给出 74HC 实验前必须查的参数表和用途	能区分绝对最大额定值、推荐工作条件、输入输出电平和时序参数
电压与扇出	判断一个输出能否可靠驱动后级输入、LED 或多路负载	能用 <code>VOH/VOL/VIH/VIL</code> 和输出电流说明余量

时钟与面包板限制	说明按钮、NE555、晶振模块和面包板布线的边界	能解释最小脉宽、边沿质量、寄生效应和低速验证范围
非法状态恢复	为状态机未使用编码指定下一状态和输出	非法状态不产生危险使能
状态机输出	比较只依赖状态的输出和依赖输入的输出	能说明稳定性与响应速度的取舍
芯片案例	用 74HC74、74HC161 或移位寄存器说明一个本章概念	绑定到真实引脚、复位、时钟和输出负载
纹波与同步计数器	比较 4-bit 纹波计数器与 4-bit 同步计数器的时钟接法、延迟来源和译码风险	能说明纹波逐级延迟累积、译码有毛刺窗口，同步计数器把代价移到组合进位链
LFSR 前八态	4 位 LFSR 每拍 Q3 取旧 Q2、Q2 取旧 Q1、Q1 取旧 Q0、Q0 取旧 Q3 与旧 Q2 的异或，从 0001 起列出前 8 个状态	序列正确，并能说明 0000 为何必须排除
模 12 计数器	用 4-bit 寄存器与加一逻辑设计模 12 计数器 (0 到 11 循环)	归零发生在同步边界，无瞬时非法态
Moore 与 Mealy 区分	判断交通灯控制器与 1101 序列检测器各属哪类，并说明输出稳定性差异	分类依据写成“输出是否依赖当前输入”
交通灯转移表	按东西绿 30s、黄 5s、全红 2s、南北绿 30s、黄 5s、全红 2s 的需求补全状态转移表与输出表	六状态闭环，timer_done 为唯一转移条件，输出只依赖状态
格雷码相邻性	核对交通灯六状态格雷编码 000、001、011、010、110、100 的每对相邻状态（含首尾闭环）是否只差一位	六次转移逐一核对，全部只差一位



左边只改变 D 端选择，时钟树保持干净；右边把 EN 放进时钟路径，EN 的毛刺可能被触发器当作真实时钟沿。

图示：写使能 vs 门控时钟：写使能在数据路径上选择；门控时钟可能把 EN 的毛刺变成额外时钟沿。

参考解答与要点

任务	参考解答	必须保留的要点
入门：画 Q 波形	锁存器 Q 在 CLK=1 期间完全复制 D 的每次变化；触发器 Q 只在每个上升沿复制当时刻的 D，其余时间保持。	触发器不是“更快的锁存器”，采样时刻完全不同。
入门：填 SR 真值表	0,0 禁用；0,1 置 1；1,0 置 0；1,1 保持。从 0,0 同时释放到 1,1 时结果不可预测。	禁行是非法输入；可靠设计不产生 0,0 或不同时释放。

入门：画计数波形	Q[3:0] 总线带：9、10、11、12、13、14 逐沿跳变；Q0：1、0、1、0、1、0。	总线带整体跳变，不是逐位爬；Q0 每周翻转。
入门：主从结构	M：CLK=0 期间等于 D，CLK=1 期间保持上升沿时刻的 D；Q：每个上升沿等于 M 当时值。	D 在 CLK=1 期间的变化只影响 M 保持的旧值，不影响 Q。
触发器路径时序分析	示例： $T_{clk} \geq 0.8ns + 3.6ns + 0.7ns + 0.4ns + 0.5ns = 6.0ns$ 。若目标周期为 5.0ns，则建立约束失败。	使用最大延迟和不利偏斜，不得只用典型门延迟。
保持违例	示例：最快路径 0.13ns 到达，而目标保持窗口要求 0.28ns 后才允许变化，存在保持违例。修复可插入保持缓冲、调整布局或修正时钟树。	降频不能可靠修复保持违例。
锁存器透明窗口	enable 有效期间，前一级输入变化可能穿过第一级锁存器、组合逻辑和第二级锁存器，使一次更新跨越多个预期阶段。	必须指出透明期间没有清晰提交边界。
复位释放	控制状态复位到 IDLE 或 RESET_WAIT，写使能和危险输出为 0；复位可异步拉入，但在本时钟域用触发器同步释放。	复位值、安全输出、释放边界三者都要写明。
两级同步	外部按钮先经过电气默认值和消抖，再进入两级触发器，状态机只使用 button_sync。	两级同步降低亚稳态扩散概率，但不替代消抖。
MTBF 指标	MTBF 表示亚稳态导致可见失败的平均时间尺度。目标时钟越快、异步事件越频繁，风险越高；第一级到第二级之间恢复时间越长，风险越低；器件恢复参数也会影响结果。	同步器是可靠性设计，不是绝对功能保证。
外部事件链路	示例链路：raw -> filtered -> pressed -> meta -> sync -> event -> pending -> FSM。其中 raw/filtered 属于电气与消抖，meta/sync 属于时钟域同步，event/pending 属于目标时钟域事件协议。	每一层信号名应对应一个可测量、可观察的边界。
事件保持	start_event 置位 start_pending，状态机在 IDLE 且安全条件满足时产生 start_ack 清除 pending；复位或故障可按安全策略清除或拒绝事件。	单周期事件不应直接作为危险动作使能。
多 bit 跨域	8-bit 总线逐位同步会采到混合时刻的 bit。若是数据传输，应使用 valid/ready 握手或异步 FIFO；若是计数指针，可用格雷码减少一次变化的 bit 数。	多 bit 数据必须有整体有效性约定。
脉冲同步	可把源域窄脉冲转换为保持电平或翻转事件位，目标域同步后检测变化，再用确认信号让源域清除事件。	事件必须持续到目标域可见，不应仅依赖单周期脉冲宽度。

时钟门控	写使能让时钟继续到达触发器，只控制是否保存新值；专用时钟门控用于功耗控制并要求 enable 无毛刺和安全锁存；普通 AND 门切时钟会产生短脉冲、额外边沿和不可控偏斜。	普通数据通路优先用写使能，不应随意组合门控时钟。
复位域	每个时钟域内部同步释放复位；跨域依赖通过 ready、同步器或状态握手确认。同步器、pending 位和危险输出在复位期间应进入不会伪触发的默认值。	复位释放顺序属于设计约定，不能假设所有域同时恢复。
状态编码	二进制编码节省触发器但译码较复杂；one-hot 译码直接但触发器多；格雷编码适合相邻状态或跨域指针。无论哪种编码，未使用编码都应导向安全状态并关闭危险输出。	状态编码选择要绑定资源、速度、跨域和安全要求。
波形验证	至少观察 <code>clk/reset/source_q/dest_d/write_enable/state/sync1/sync2/pending/ack</code> ，检查 D 是否在采样沿前稳定、写使能是否只在目标周期有效、复位释放是否产生伪事件。	只看最终寄存器值不足以证明同步边界正确。
CPU 连接	PC 是寄存器，流水线寄存器是阶段边界，控制器是状态机，写回使能是状态控制线，中断入口来自同步后的外部或内部事件。	后续 CPU 仍遵守“组合准备、边界提交”的模型。
按钮同步器实作	按钮节点用上拉或下拉给出默认电平，经过 RC 和 74HC14 得到整形后的 <code>button_pressed</code> ；再接入 74HC74 两级触发器，后级只使用第二级 Q。复位脚进入不会伪触发的默认状态。	必须写清楚每一级信号名，不能把 raw 引脚直接送入状态机。
计数器实作	74HC161/74HC163 的 CP 接低速时钟，计数使能接受控有效电平，并行装载保持无效或由实验开关控制，Q0 到 Q3 通过限流电阻接 LED。复位极性按数据手册连接。	计数器体现同一时钟边界上的 4-bit 状态更新。
移位输出实作	74HC595 的 DS 在 SHCP 边沿前稳定，8 次移位后再给 STCP 边沿，Q0 到 Q7 一起更新；MR_n 和 OE_n 保持在确定电平。	必须区分移位寄存器内部状态和输出锁存边界。
写使能实作	74HC157 在旧 Q 和 <code>new_bit</code> 之间选择，输出送入 74HC74 的 D；时钟持续送入触发器， <code>write_enable=0</code> 时写回旧 Q， <code>write_enable=1</code> 时写入新值。	不应使用普通与门直接门控时钟作为正式方案。
实验板物料	最小 BOM 应包含稳定 3.3V 或 5V 电源、面包板、目标 74HC/74HCT 芯片、0.1uF 去耦电容、上拉/下拉电阻、LED 与限流电阻、按钮、RC 滤波电容、杜邦线和万用表；逻辑分析仪或示波器用于观察边沿和短脉冲。	物料清单必须对应实验中的电源、输入、状态输出和测量需求。

接线检查	上电前检查 VCC/GND、低有效复位和输出使能、未用输入默认值、LED 限流、输出是否短接、电压域是否兼容。发现芯片发热或输出异常时应立即断电复查。	检查表应能解释常见硬件失败，而不是只列器件名称。
故障定位报告	示例：按一次计数器跳多格时，先观察按钮 raw，再看 74HC14 输出，最后看计数器 CP；若 raw 抖动而整形后仍有多个边沿，应增加消抖或避免按钮直接作为时钟。报告要写出修复前后观测差异。	失败分析必须对应到可观察信号和修复后的波形。
数据手册读取	必须先查推荐工作条件确认 VCC、电压和输入边沿，再查 VIH/VIL/VOH/VOL 判断电平兼容，查输出电流判断 LED 和负载，查传播延迟、最高频率与最小脉宽判断时钟是否合格；绝对最大额定值只用于损伤边界。	不得把绝对最大额定值当作正常工作条件。
电压与扇出	若驱动器 VOH_min 小于接收器 VIH_min 加余量，高电平不可靠；若 LED 电流过大，输出电平会被拉坏；一个输出驱动多个 CMOS 输入时还要考虑电容负载和边沿变慢。	判断必须同时看电压余量、电流限制和负载电容。
时钟与面包板限制	按钮单步必须消抖和施密特整形，NE555 要检查占空比和脉宽，晶振模块要检查频率和电压域；面包板只适合低速概念验证，不能证明高频 CPU 级可靠性。	状态边界必须来自合格边沿，不来自慢爬升或多次抖动。
非法状态恢复	未使用编码的下一状态导向 IDLE、FAULT 或复位等待态，输出逻辑默认关闭写使能、驱动使能和片选。	非法状态的输出也要安全，不应只写下一状态。
状态机输出	Moore 输出只依赖状态，通常更稳定；Mealy 输出还依赖输入，响应更快但可能把输入毛刺传到控制线。危险输出宜寄存或只由稳定状态产生。	取舍必须结合后级是否采样或驱动危险对象。
芯片案例	以 74HC74 为例，D 是待保存数据，CLK 是采样边界，Q 是保存结果，异步复位/置位必须按数据手册极性连接；未用输入不能悬空。	真实芯片要检查引脚、电源、复位极性、时钟边沿和负载。
纹波与同步计数器	纹波计数器用低位 Q 作高位时钟，第 k 位比第 0 位晚 k 个 tCQ 翻转，4 级最坏相差 4 个 tCQ；翻转窗口内译码器会看到瞬时错值，输出毛刺真实存在。同步计数器所有位共用同一时钟沿、同时更新，代价是进位链（高位使能等于低位全 1）随位数变长，成为组合关键路径。	纹波的风险在时钟不同步，同步的代价在进位逻辑深度。

LFSR 前八态	0001、0010、0100、1001、0011、0110、1101、1010。逐拍按规则计算：高三位左移接收邻位，Q0 取旧 Q3 与旧 Q2 的异或。全 0 态的反馈恒为 0，一旦进入便永远锁死，故初态必须非零。	每拍只由旧态决定新态；0000 是陷阱态。
模 12 计数器	现态为 1011（即 11）时，下一状态逻辑选 0000 而非加一：用比较器译码 1011 控制数据 mux 即可。若改用异步清零计数器（如 74HC161），需译码 1100 去拉清零脚，会产生短暂的 1100 毛刺态，且各位清零时刻不一致；同步清零（如 74HC163）译码 1011 后在时钟沿干净归零。	归零必须发生在时钟边界；异步清零方案要指出毛刺态。
Moore 与 Mealy 区分	交通灯的灯色只由当前状态决定，属 Moore 型，输出在整个状态期间稳定；1101 检测器在 S3 且输入为 1 的转移上给出输出，属 Mealy 型，同拍响应但输入毛刺可直达输出。	分类看输出函数的自变量，与状态个数无关。
交通灯转移表	EW_GREEN 到 EW_YELLOW 到 ALL_RED_EW 到 NS_GREEN 到 NS_YELLOW 到 ALL_RED_NS 再回到 EW_GREEN，转移条件均为 timer_done=1，装载值依次为 5、2、30、5、2、30 秒；输出依次为绿/红、黄/红、红/红、红/绿、红/黄、红/红。timer_done=0 时保持原状态。	状态数、转移条件、输出必须与需求逐条对应。
格雷码相邻性	000 到 001 差最低位；001 到 011 差中间位；011 到 010 差最低位；010 到 110 差最高位；110 到 100 差中间位；100 到 000 差最高位。六次转移均只翻转一位。	闭环的最后一边（100 回到 000）也必须核对，不能只看前五次。

本章的标注对象是“边界”和“状态转移表”。仅说明某个值会变化并不充分，必须说明哪个时钟边界提交，提交前后的组合路径是什么。

- 纸上实验：画出一个 D 触发器前后各接一段组合逻辑，标出当前周期的旧 Q、组合路径上的 D、下一个时钟沿后的新 Q。
- 时序分析：为一条路径写出 $T_{clk} \geq t_{CQ} + t_{comb} + t_{setup} + t_{skew}$ ，再解释每一项对应哪个硬件现象。
- 复位设计：复位要让控制状态进入已知安全点，但释放复位也要有同步边界，不能让状态机半边先跑。
- 状态结构：寄存器是一组并排触发器；计数器是寄存器输出经过加法器反馈到输入；移位寄存器是固定连线选择了相邻 bit。
- 控制结构：状态机输出可以只依赖状态，也可以依赖状态和输入；后者反应快，但更容易把输入毛刺传给控制线。
- 检查题：若状态编码里出现未使用编码，电路上电或受干扰进入它时，下一状态逻辑应该怎样让机器回到安全状态？参考答案：未使用编码不应保

持未定义，也不应产生危险输出；下一状态逻辑应把它导向复位、IDLE、FAULT 或其他安全诊断状态，同时关闭写使能、功率使能等危险控制线。若安全要求更高，还应记录错误或触发复位流程。

经典系统教材通常会把组合逻辑和状态逻辑分开建模：组合逻辑计算下一值，状态元件在边界保存下一值。后面 CPU 的 PC、寄存器、状态机和控制器都遵守这个模型。

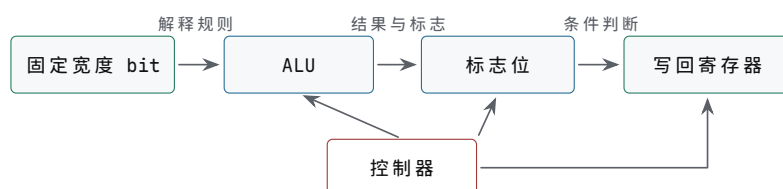
第 3 章

数值、ALU、数据通路与控制 器

前两章已经建立了两条基础事实：bit 不是抽象符号，而是带噪声余量的电平；状态不是文字变量，而是在时钟边界被触发器保存下来的电荷和电压。本章讨论第一部分的第三块内容，集中回答 CPU 出现之前必须解决的一组连续问题：固定宽度 bit 如何解释成数，算术逻辑单元如何产生结果和标志，寄存器、选择器、总线和写回路径如何组成数据通路，控制器又如何在每个周期输出一组一致的控制信号。

本章的写法遵循一个具体任务：让硬件完成 $R3 = R1 + R2$ ，再根据结果是否为 0 决定是否改变下一步动作。这个任务看似简单，却已经包含了计算机硬件的核心约定。首先， $R1$ 和 $R2$ 里保存的只是固定宽度 bit，必须说明它们按无符号、补码、定点还是其他格式解释；其次，ALU 要选择加法路径并产生结果、进位、零标志和溢出标志；再次，数据通路要把两个旧寄存器值送入 ALU，再把新结果写回目标寄存器；最后，控制器要保证读、算、写和下一状态选择发生在正确周期，不能把尚未稳定的组合输出提前提交。

经典体系结构教材讲 ALU 和数据通路时，很少把它们当作孤立名词处理，而是反复追问三件事：输入 bit 的解释是什么，组合路径的输出是什么，状态在什么边界提交。本章也按这个顺序阅读。若能为一条动作写出“源寄存器、选择器、ALU 操作、标志位、写回来源、写使能、下一状态”的 trace（执行轨迹），就已经具备走读最小 CPU 的基本能力。



本章不是先讲“加法公式”，而是把 bit 解释、组合计算、标志生成和时钟提交连成同一条连贯路径。

图示：ALU 章节的总图。箭头表示信息流向：固定 bit 先被解释，再进入组合计算，最后由控制器决定是否提交。

一、固定宽度 bit 的数值解释

硬件只保存 bit。所谓整数、正负号、小数点、十进制数字、溢出、大小比较，都是对同一组 bit 模式的解释规则。若不先讲清楚表示，后面的加法器、减法器、比较器和 ALU 就会像在做抽象数学；实际上它们处理的始终是固定宽度的一组电线。位宽一旦确定，硬件能保存的模式数量也就确定：4-bit 有 16 种模式，8-bit 有 256 种模式，32-bit 有 2^{32} 种模式。显示成十进制、十六进制还是二进制只是记法，不能改变硬件位宽。

4-bit 无符号数 $b_3 b_2 b_1 b_0$ 的值是：

$$\text{value} = b_3 \times 8 + b_2 \times 4 + b_1 \times 2 + b_0$$

因此 1011 表示 11，因为它等于 $8+0+2+1$ 。无符号数没有负数概念，所有 bit 都贡献非负权重。4-bit 能表达的范围是 0 到 15，8-bit 能表达的范围是 0 到 255。一个 4-bit 加法器只能输出 4 个结果 bit，再加上可能的最高位外进位。若

15 加 1，低 4 位回到 0000，最高位外产生进位。这个进位不是“可有可无的第 5 位”，而是无符号运算超出 4-bit 范围时硬件给出的明确信号。

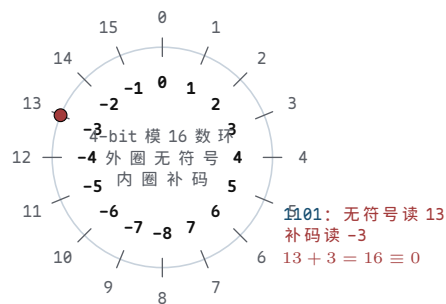
位宽	模式数量	无符号范围	典型硬件含义
4-bit	16	0 到 15	一片 4-bit 加法器、一个十六进制数字
8-bit	256	0 到 255	早期微处理器常见数据宽度、一个字节
16-bit	65536	0 到 65535	8086 通用寄存器和 ALU 的核心宽度
32-bit	2^{32}	0 到 $2^{32} - 1$	80386 扩展后的通用寄存器与地址计算宽度

补码表示让负数也能走同一个加法器。以 4-bit 为例，最高位不再只是 8，而是解释成负权重 -8：

$$\text{value} = b_3 * (-8) + b_2 * 4 + b_1 * 2 + b_0$$

于是 1111 的值是 $-8+4+2+1=-1$ ，1110 的值是 -2，1000 的值是 -8。4-bit 补码范围是 -8 到 7。补码范围不对称，因为 0000 已经表示 0，负数一侧多出一个最小值。这个细节很重要：在固定宽度内，最小负数没有对应的正数。例如 4-bit 的 -8 取相反数仍会得到 1000，这不是数学规则失效，而是硬件位宽不足。

为什么负权重成立？补码不是人为规定“最高位算负数”，而是模运算的自然结果。4-bit 加法器算的是模 16 加法：结果只保留低 4 位，超出 16 的部分被丢弃。在模 16 的世界里，-3 和 13 是同一个数，因为 $13 + 3 = 16 \equiv 0$ 。所以“存 -3”就是“存 13”，即 1101。同一根线上的 bit 模式于是有了两种读法：无符号读作 13，有符号读作 -3。把最高位解释成 -8 而不是 +8，两种读法就对上了： $8+4+1=13$ ， $-8+4+1=-3$ ，而 $13 \equiv -3 \pmod{16}$ 。负权重不是新算术，只是给同一个模运算换了一个名字区间：0 到 15 换成 -8 到 7。



图示：补码的数环读法：同一个 bit 模式，外圈是无符号读数，内圈是补码读数。加法器只按模 16 进位，两种读法共享同一条进位链——这就是为什么加法和减法能共用一套硬件。

“取反加一”也不再是口诀，而是一行证明：对任意 x ，都有 $x + \sim x = 1111 = -1$ (模 16 下各位互补，相加必得全 1)，所以 $-x = \sim x + 1$ 。硬件只需要一排 NOT 门和一条加法进位链，减法就被归并成了加法。

求一个补码数的相反数，可以按“取反加一”理解：

```
x      = 0011 // 3
~x     = 1100
~x + 1 = 1101 // -3 in 4-bit two's complement
```

这个规则适合硬件，因为取反可以用一排 NOT 或 XOR 门完成，加一可以交给加法器。于是减法可以变成加法：

$$a - b = a + (\text{NOT } b) + 1$$

这条公式是 ALU 设计的关键。硬件不必为加法和减法各放一套完全独立的算术电路，只要让控制信号同时决定 B 操作数是否反相，以及最低位进位输入是否为 1，就能在同一条进位链上完成 ADD 和 SUB。第一章已经讲过受控电流路径，本章把这个思想推进到算术层面：**控制信号改变的是电路连接和输入选择，不是纸面公式。**

原码、反码、补码是三种被真实采用过的负数表示 补码不是唯一被试过的方案。早期计算机还认真使用过两种更直觉的表示：原码把最高位专门用作符号位，其余位保存绝对值；反码把正数逐位取反得到对应负数；补码则在反码基础上再加一。三种方案用 8-bit 都能表示 -5，但硬件代价差别很大。先把同一个负数的三种写法并排摆开：

表示法	负数规则	-5 的写法	零的写法
原码	最高位记符号，其余位记绝对值	1000 0101	0000 0000 与 1000 0000 两个零
反码	正数逐位取反	1111 1010	0000 0000 与 1111 1111 两个零
补码	逐位取反再加一	1111 1011	唯一的 0000 0000

补码最终胜出，三条原因都直接对应门电路的开销。第一，加法会自然处理符号。原码做加减前必须先比较两数符号和绝对值大小，决定实际执行加法还是减法，再单独确定结果符号——符号位不能参与进位，需要一条独立的逻辑路径。补码把符号位当成普通位一起进位： $1111\ 1011 + 0000\ 0101 = 1\ 0000\ 0000$ ，最高位进位被位宽丢弃，低 8 位就是 0， $(-5)+5$ 在普通加法器上一次算对，不需要任何符号特例。第二，零唯一。原码和反码都有 +0 和 -0 两种模式，“结果是否为零”要检测两种 bit 模式，判零电路多一级组合逻辑；补码只有一种零，Zero 标志就是对所有结果 bit 做一次 NOR。第三，取负便宜且单向。反码取负确实只要一排 NOT 门，但它的加法在产生最高位进位时必须把进位绕回最低位再加一次，这叫循环进位 (end-around carry)，进位链因此多绕一圈；补码取负是“取反加一”，NOT 加已有进位链即可完成，进位始终朝一个方向走。

循环进位的差别用 $(-5)+7$ 演算一次。反码： $1111\ 1010 + 0000\ 0111 = 1\ 000\ 0\ 0001$ ，最高位的进位 1 必须加回最低位， $0000\ 0001 + 1 = 0000\ 0010$ ，才得到 2。补码： $1111\ 1011 + 0000\ 0111 = 1\ 0000\ 0010$ ，进位直接丢弃，低 8 位 $0000\ 0010$ 就是 2。一次加法里反码要等进位回绕，补码只走一趟。

这三条性质的意义超出表示法本身：补码把符号处理、判零和取负全部归并到同一条进位链上，加减法、比较和取负共用一套硬件。历史上 UNIVAC 1100 系列等机器沿用反码，而 x86、ARM、RISC-V、MIPS 的有符号整数一律是补码。前面“减法复用加法器”的全部便利，本质上都来自这张表最后一行。

同一组 bit 可以按无符号解释，也可以按补码解释。因此溢出判断也不同。

解释方式	关注点	例子
无符号	是否产生最高位外的进位	15 + 1 超出 4-bit 无符号范围
补码有符号	正正得负或负负得正	7 + 1 得到 1000，被解释为 -8

以下 4-bit 加法给出同一结果在两种解释下的差异：

$$0111 + 0001 = 1000$$

若按无符号解释，这是 $7+1=8$ ，仍在表示范围内。若按补码解释，0111 是 7，0001 是 1，结果 1000 是 -8，正数加正数却得到负数，所以发生有符号溢出。硬件不会替使用者决定哪种解释成立；它只提供结果 bit、最高位外进位、符号位变化和溢出标志。程序、指令语义和控制器必须约定当前采用哪一种解释。

溢出判定演算：四个 8-bit 例子覆盖全部标志组合 把“正正得负、负负得正”推进到 8-bit，用四个例子走完整数值过程。8-bit 补码范围是 -128 到 127，无符号范围是 0 到 255：

```
+100 + +50: 0110 0100 + 0011 0010 = 1001 0110
              signed: +150 read as -106   WRONG (overflow)
              Cout = 0, Overflow = 1

-100 + -50: 1001 1100 + 1100 1110 = 1 0110 1010
              signed: -150 read as +106   WRONG (overflow)
              Cout = 1, Overflow = 1

+100 + -50: 0110 0100 + 1100 1110 = 1 0011 0010
              signed: +50                  correct
              Cout = 1, Overflow = 0

-100 + +50: 1001 1100 + 0011 0010 = 1100 1110
              signed: -50                  correct
              Cout = 0, Overflow = 0
```

判定规则可以总结成一句：**两个同号数相加，得到异号结果，即补码溢出**。写成布尔式，若 a_7 、 b_7 、 r_7 是两个输入和结果的符号位，则 $V = a_7 b_7 \bar{r}_7 + \bar{a}_7 \bar{b}_7 r_7$ 。硬件里还有一个更便宜的等价判法：进入符号位的进位 C_7 与最高位向外的进位 C_{out} 不一致就是溢出， $V = C_7 \oplus C_{out}$ 。用第一个例子验证：逐位进位 (c_1 到 c_8) 是 0,0,0,0,0,1,1,0，进入符号位的进位是 1、向外进位是 0，两者不同， $V = 1$ ，与符号判法一致。

为什么异号相加永不溢出？一个正数加一个负数，结果必然落在两个加数之间：和的绝对值不超过两数绝对值中较大的那个，而那个加数本身就在 -128 到 127 之内。范围内加范围内得到范围外，只有在两个加数同号、绝对值同向叠加时才可能发生。减法的判定也归并到同一条规则：把它看成加上取负后的数，于是“异号相减且结果符号与被减数不同”就是溢出。

Carry 和 Overflow 是两个相互独立的判断，上面四个例子恰好把 (C_{out}, V) 的四种组合 (0,1)、(1,1)、(1,0)、(0,0) 都走了出来。Carry 报告的是“bit 模式超出无符号范围”，即模 2^8 回绕发生；Overflow 报告的是“补码真值超出有符号范围”。第二个例子里两者同时为 1：无符号读是 $156 + 206 = 362$ 超界，补码读是 $-100 + (-50) = -150$ 超界。第三个例子里 Carry=1 但 $V=0$ ：无符号读 $100 + 206 = 306$ 回绕，补码读 +50 完全正确。硬件不替程序选择哪种读法，所以两个标志都必须产生。

演算	输入符号	结果符号	Cout	V	读法结论
+100 + +50	同号 (正)	负	0	1	补码溢出；无符号 150 仍正确
-100 + -50	同号 (负)	正	1	1	补码溢出；无符号 362 也超界
+100 + -50	异号	正	1	0	补码正确；无符号发生回绕
-100 + +50	异号	负	0	0	两种读法都正确

两个标志的用途也因此分开：多字长加法把低字节的 Carry 送进高字节的 C_{in} ，有符号范围检查看 Overflow。x86-64 把两者分别放在 CF 和 OF，条件转移各走各

的：

```
; x86-64, Intel syntax
mov al, 100
add al, 50      ; AL = 0x96, CF = 0, OF = 1
jo  too_big    ; signed path checks OF

mov al, 200
add al, 100     ; AL = 0x2C, CF = 1, OF = 0
jc  carry_in   ; unsigned path checks CF
```

这段分工的意义在于：标志位不是“结果好或坏”的笼统指示，而是两种解释规则各自的边界信号。控制器和编译器按当前约定取其中一个，另一个照常产生但可以被忽略。

位宽变化也要有明确规则。把 4-bit 值送入 8-bit 数据通路时，若它是无符号数，应做零扩展；若它是补码有符号数，应做符号扩展。把 8-bit 值截断成 4-bit 时，高位被丢弃，剩余低 4 位不一定仍表示原值。

动作	输入解释	输出规则	示例
零扩展	无符号	高位补 0	1011 -> 00001011
符号扩展	补码有符号	高位复制符号位	1011 -> 11111011
截断	固定低位	只保留低位	10011011 -> 1011
饱和	特定 DSP/多媒体语义	超出范围钳到最大/最小	需要额外比较和选择

符号扩展不是显示格式，而是真实硬件路径。若 4-bit 的 1011 按补码解释为 -5，扩展到 8-bit 后必须成为 11111011，仍然表示 -5；若错误地零扩展为 00001011，它就变成 11。这个错误在地址偏移、短立即数、分支位移和加载有符号字节时都可能出现。8086、80386 这类处理器的指令语义中会明确区分零扩展、符号扩展和普通截断，因为这些规则直接决定 ALU 输入。

字节、字和字节序把数值解释放进内存 体系结构教材常从“一个对象在内存中占哪些字节”开始训练机器级思维。硬件寄存器里可以一次保存 16-bit、32-bit 或 64-bit 值，但主存通常按字节编址。于是一个 32-bit 数值放进内存时，必须规定四个字节按什么顺序排列。若数值为 0x12345678，大端序把最高有效字节放在最低地址，小端序把最低有效字节放在最低地址。两者保存的是同一个 32-bit 模式，只是地址到字节车道的映射不同。

字节序	低地址到高地址	硬件含义
大端序	12 34 56 78	低地址保存最高有效字节，适合按书写顺序观察十六进制
小端序	78 56 34 12	低地址保存最低有效字节，低位字节扩展和多精度运算方便

字节序不是“显示问题”，而是 LOAD/STORE、字节使能和总线车道的硬件约定。CPU 执行 32-bit LOAD 时，内存系统返回四个相邻字节，处理器内部再按本机字节序把它们拼成寄存器值。CPU 执行 8-bit LOAD 时，只取其中一个字节；若目标寄存器更宽，还要按指令语义做零扩展或符号扩展。若把这三件事混在一起，调试时会出现非常典型的错误：内存窗口看见的字节顺序和寄存器中显示的十六进制值不一致，看起来像“读错了”，实际可能只是字节序解释不同。

```
memory bytes at increasing addresses:
```

```

0x1000: 78
0x1001: 56
0x1002: 34
0x1003: 12

little-endian 32-bit load from 0x1000 -> 0x12345678
8-bit load from 0x1000                -> 0x78

```

不同宽度的访问有各自必须写清的规则。字节 LOAD 选择一个字节车道，再扩展到寄存器宽度——零扩展还是符号扩展必须由指令规定；半字 LOAD 选择两个相邻字节并按字节序拼接，地址低位和对齐规则必须满足；字 LOAD 选择四个字节拼成 32-bit 值，小端/大端、字节使能和未对齐处理都要写清；字节 STORE 只打开一个字节写使能，绝不能破坏同一字中的其他字节。

内存对象和指针本质上也是位模式 从硬件角度看，指针不是神秘类型，而是一个被解释为地址的固定宽度 bit 模式。数组、结构体、栈帧和缓冲区最终都归结为“起始地址 + 偏移”的地址形成路径。若数组元素是 4 字节整数，访问 $a[i]$ 时地址形成部件通常要计算 $base + i*4$ ；若结构体字段在对象内偏移 12 字节，LOAD/STORE 的地址就是 $base + 12$ 。这与前面的定点数类似：bit 本身不变，解释约定决定它是整数、地址、偏移还是控制字段。

把几类对象的硬件路径分开看：标量整数是固定位宽 bit 模式，存在寄存器或内存字节里，经 ALU 解释；指针是地址宽度 bit 模式，会进入地址形成、译码、cache/TLB 或总线；数组元素的地址是基址加缩放下标，由加法器、移位器或地址生成单元算出；结构体字段的地址是基址加固定偏移，通常由立即数扩展和 ALU 地址加法完成；栈对象的地址是栈指针加偏移，依赖 SP/RSP 类寄存器和调用约定。

这个视角能解释为什么越界访问危险。硬件不会自动知道“这个地址还属于数组”。若没有保护机制， $a[10]$ 只是另一次地址加法和一次 LOAD/STORE，它可能指向相邻变量、返回地址、MMIO 窗口或未映射区域。后续章节讨论 80386 分页、IOMMU 和异常时，本质上就是把“哪些地址可访问、按什么权限访问”提升为硬件可检查的约定。

小数不是新电路，而是新的位权解释 整数位的权重是 1, 2, 4, 8, ...，小数位只是把二进制小数点右侧的权重继续向下分成 $1/2, 1/4, 1/8, \dots$ 。例如：

```

10.1012 = 1*2 + 0*1 + 1*(1/2)
          + 0*(1/4) + 1*(1/8)
          = 2.625

```

这里的“小数点”不是一根额外电线。若寄存器里只保存 10101，硬件本身看见的仍然是五个 bit；把它解释成 21_{10} 、 10.101_2 ，还是某种编码字段，取决于电路和指令约定。这个事实会直接影响硬件设计：二进制小数的加法仍可以复用整数加法器，但必须保证两个操作数的小数点位置相同；若小数点位置不同，必须先对齐。

二进制小数和十进制小数并不总能互相有限表示。 0.5 可以写成 0.1_2 ，因为它正好是 $1/2$ ； 0.25 可以写成 0.01_2 ，因为它正好是 $1/4$ 。十进制的 0.1 则不能用有限个二进制小数位精确表示，只能形成无限循环展开。计算器、编译器和 FPU 不是“不认真”，而是受位宽限制：寄存器只有固定数量的 bit，必须在某个位置舍入。

十进制值	二进制表示	是否有限	硬件含义
0.5	0.1_2	有限	一位小数即可精确保存
0.25	0.01_2	有限	权重为 $1/4$ 的 bit 置 1
0.625	0.101_2	有限	$1/2 + 1/8$

0.1	无限循环	不能有限表示	固定位宽内必须近似和舍入
-----	------	--------	--------------

定点数把小数点位置变成硬件约定 定点数把“小数点在第几位”固定下来。常见写法是 Q 格式（沿用早期 DSP 手册用字母 Q 标记小数位数的习惯）：Q4.4 表示 8-bit 总宽度中有 4 个整数相关位和 4 个小数位，实际值等于原始整数除以 2^4 。例如 00011000 的无符号整数值是 24；若按 Q4.4 解释，它的实际值是 $24/16 = 1.5$ 。同一串 bit 并没有改变，改变的是缩放约定。

原始 bit	原始整数	定点解释	说明
00011000	24	Q4.4 中为 1.5	小数位数为 4，除以 16
00000110	6	Q1.7 中为 0.046875	小数位数为 7，除以 128
11110000	-16	有符号 Q4.4 中为 -1.0	先按补码得 -16，再除以 16
01111111	127	有符号 Q1.7 中接近 0.9921875	最大正值小于 1

定点加减最适合用现有整数 ALU。只要两个数采用相同 Q 格式，硬件可以直接把原始 bit 送入加法器；结果仍按同一个缩放因子解释。例如 Q4.4 中的 $1.5 + 0.25$ 可以看成原始整数 $24 + 4 = 28$ ，结果 00011100 解释为 $28/16 = 1.75$ 。因此许多音频、控制、电机、传感器和早期图形场景会优先使用定点数：整数加法器、移位器和乘法器即可承担主要工作。

定点乘法需要多一步处理。若两个 Q4.4 数相乘，原始整数相乘后缩放因子变成 2^8 ，因为两个 2^4 相乘得到 2^8 。要把结果放回 Q4.4，通常要右移 4 位，并决定被丢弃的低位是截断、四舍五入还是参与饱和判断。

```
Q4.4 value 1.5  -> raw 24
Q4.4 value 0.5  -> raw 8
raw product      -> 24 * 8 = 192
rescale to Q4.4  -> 192 >> 4 = 12
Q4.4 result      -> 12 / 16 = 0.75
```

这个例子说明定点不是“比浮点低级”的临时方案，而是一种明确的硬件选择。它牺牲动态范围，换来较低延迟、较小面积和更容易预测的执行时间。若电路要实时控制电机电流、滤波音频采样或处理传感器数据，固定缩放因子往往比复杂浮点更合适。

许多微控制器并不一定带硬件 FPU，或者即使带 FPU，也会在中断、功耗和实时性严格的路径上使用整数和定点运算。DSP 芯片则常见乘加单元，也就是 MAC 路径，用于反复执行“乘法、对齐、累加、饱和”。这些芯片的手册通常会明确给出 Q 格式、饱和模式、舍入模式和累加器宽度，因为它们决定滤波器、控制环和音频处理是否稳定。

定点乘法先做整数乘，再把小数点移回约定位置 两个 Q4.4 数相乘时，硬件看见的仍然只是两个 8-bit 整数。完整走一遍 1.5×2.25 ：

```
1.5   -> raw 24 = 0001 1000    (24 / 16 = 1.5)
2.25  -> raw 36 = 0010 0100    (36 / 16 = 2.25)

integer product: 24 * 36 = 864
16-bit accumulator: 0000 0011 0110 0000
read as Q8.8: 864 / 256 = 3.375
```



```
rescale to Q4.4: 864 >> 4 = 54 = 0011 0110
check: 54 / 16 = 3.375 = 1.5 * 2.25
```

为什么乘完必须移动小数点？每个操作数的 raw 值都是“真值 $\times 2^4$ ”，整数乘积天然携带 $2^4 \times 2^4 = 2^8$ 的缩放。若把 864 直接按 Q4.4 读，得到的是 54，恰好是真值的 16 倍。小数点在电路里不是实体，而是“缩放因子是多少”的约定；乘法把两个约定相乘，所以必须右移 4 位把约定改回 Q4.4。若目标是更宽的累加器，也可以不立即移位，按 Q8.8 继续参与后续乘加，等写回存储前再统一缩放——缩放时机是设计选择，缩放动作不可省略。

被右移丢弃的低位是真实的信息损失，处理方式必须写清。直接 $>> 4$ 是截断： 1.1875×1.1875 的 raw 乘积是 $19 \times 19 = 361$ ， $361 >> 4 = 22$ ，即 1.375，而真值是 1.41015625。若要在 Q4.4 内就近舍入，右移前先加半个最低位的权重，即加 1000₂： $(361 + 8) >> 4 = 23$ ，即 1.4375，距真值更近。两种做法不是对错问题，而是误差分布的约定：控制环在意单向误差累积，音频在意舍入噪声，芯片手册会把所选模式写明。

溢出检查同样在移位这一步完成。两个 8-bit 数相乘必须用 16-bit 暂存；右移 4 位后，若只取低 8 位作为 Q4.4 结果，则 bit 15 到 bit 8 必须全是 bit 7（新符号位）的复制，否则值已经超出 8-bit Q4.4 的范围。例如 7.9375×7.9375 ：raw 乘积 $127 \times 127 = 16129 = 0011\ 1111\ 0000\ 0001$ ，右移 4 位得 $0000\ 0011\ 1111\ 0000$ ；此时 bit 7 为 1，而 bit 15 到 bit 8 是 $0000\ 0011$ ，不是全 1，判定溢出——真值约 63.004 远超有符号 Q4.4 的上限 7.9375。此时要么置溢出标志交给软件，要么按饱和约定钳到 $0111\ 1111$ (7.9375)。

步骤	数值或位模式	缩放约定与检查点
解释输入	raw 24 与 raw 36	各带 2^4 缩放，两数小数位数必须一致
整数相乘	$24 \times 36 = 864$	缩放变为 2^8 ，需要双倍位宽暂存
右移还原	$864 >> 4 = 54$	回到 Q4.4；右移前加 8 可就近舍入
溢出检查	高位必须只是符号复制	超界时置标志，或饱和到最大/最小值

真实 DSP 把这条路径做成了专门部件。MAC（乘加）单元内部保留比输出格式宽得多的累加器，例如 32-bit 或 40-bit，让一连串“乘、加、乘、加”在中间过程不右移、不丢位，只在最终写回时做一次缩放、舍入和溢出检查。累加器多出来的高位常叫保护位（guard bits），作用正是推迟溢出判定：中间和允许暂时超出输出范围，只要最终结果能装回目标格式。这再次说明定点不是“简化版浮点”，而是一套把缩放时机、舍入模式和溢出策略全部显式写出来的工程约定。

十进制与 BCD 解决的是输入输出和精确十进制表示 二进制适合硬件算术，但人们日常习惯使用十进制。BCD，即 binary-coded decimal，用 4 个 bit 保存一个十进制数字。最常见的 8421 BCD 中，0000 到 1001 表示 0 到 9，1010 到 1111 不是合法十进制数字。两位十进制 59 可以保存为 0101 1001，而不是普通二进制整数 59 的 00111011。

表示	示例	适用问题
普通二进制	59 为 00111011	ALU 运算紧凑，位宽利用率高
8421 BCD	59 为 0101 1001	每个十进制位可直接显示和校正
非法 BCD	1010 到 1111	需要检测，不能当作十进制数字

BCD 加法的关键在于十进制校正。若一个半字节相加结果超过 9，或者低半字节产生了十进制意义上的进位，需要加 0110 把结果调整回合法 BCD。例如 $8 + 7 = 15$ ，普通二进制低 4 位是 1111，这不是合法 BCD 数字；加 0110 后得到 1 0101，表

示十进制进位 1 和低位数字 5。早期处理器保留辅助进位、十进制调整等机制，往往就是为了兼容十进制输入输出、计算器和商业数据处理。

8086 继承了面向 BCD 的十进制调整指令和辅助进位标志，原因不在于 BCD 比二进制 ALU 更快，而在于早期商业计算、显示和键盘输入大量使用十进制数字。现代通用 CPU 的主路径通常不会为 BCD 牺牲整数流水线，但十进制浮点、金融数据库和显示驱动仍会在特定边界上关心十进制精确性。

浮点数把范围和精度分开 定点数的小数点位置固定，动态范围有限。浮点数采用另一种约定：用一个符号位表示正负，用指数字段表示尺度，用有效数字字段表示精度。可以把它理解成硬件版科学计数法。一个常见规格化二进制浮点数可写成：

$$\text{value} = (-1)^{\text{sign}} * 1.\text{fraction} * 2^{(\text{exponent} - \text{bias})}$$

其中 **bias** 是阶码偏置，用来让指数字段以无符号 bit 保存正负指数。有效数字前面的 1 通常不显式保存，这叫隐藏位。浮点格式还要为特殊值保留编码：正负零、非规格化数、正负无穷大和 NaN。非规格化数用于靠近 0 的渐进下溢；无穷大用于表示超出范围或除以 0 一类结果；NaN 表示“不是一个数”的异常传播，例如 0/0 或无效运算。

把这条公式落实到最常见的格式上。IEEE 754 单精度用 32 个 bit，分成三段：

$$\text{值} = (-1)^s \times 1.f \times 2^{e-127}$$

s	阶码 e (8 位, bit 30-23)	尾数 f (23 位, bit 22-0)
---	-----------------------	-----------------------

按无符号保存, bias=127: e=127 表示 2^0

规格化时前导 1 隐藏, 不占 bit

图示：IEEE 754 单精度的 32-bit 布局：1 位符号、8 位阶码、23 位尾数。阶码偏置 127，所以 e=1 表示 2^{-126} ，e=254 表示 2^{127} ；e=0 和 e=255 被保留给非规格化数、零、无穷大和 NaN。

两个完整的解码演算，胜过十遍公式：

```
0x3F800000:
s = 0
e = 0111 1111 = 127 -> exponent = 127 - 127 = 0
f = 000...0 -> significand = 1.0
value = +1.0 * 2^0 = 1.0
```

```
0x40490FDB:
s = 0
e = 1000 0000 = 128 -> exponent = 1
f = 100 1001 0000 1111 1101 1011
    significand = 1.5707963...
value = 1.5707963 * 2^1 = 3.1415926...
```

对阶加法也用一个真实位宽走一遍。计算 $2^{10} + 2^{-14}$ ：两个数指数相差 24，较小数的尾数必须右移 24 位才能对齐；但单精度尾数只有 23 位，被移出的最低那个 1 落在保留位宽之外，只能按舍入规则处理——结果舍入回 2^{10} 。 2^{-14} 这次加法在硬件里真实发生了，却被格式位宽“吃掉”了。

```
2^10 + 2^-14 in binary32:
align: shift small significand right by 24
23-bit field cannot keep the shifted-out bit
result rounds back to 2^10
```

四个字段各自的硬件动作也要分清：符号位进入符号处理和比较路径，不能直接等同整数的最高位；指数字段决定数值尺度，加减前要对阶、乘除时参与指数加减，

风险是范围溢出或下溢；有效数字是精度来源，进入尾数加法、乘法和规格化，风险是位数有限导致舍入；特殊值（零、非规格化、Inf、NaN）需要旁路和异常规则，比较、传播和转换时都要查清语义。

阶码的两个端点被保留给零、次正规数、Inf 和 NaN 单精度阶码 1 到 254 留给规格化数，全 0 和全 1 两个端点另有约定： $e = 0$ 表示零和次正规数， $e = 255$ 表示无穷大和 NaN。于是一个 32-bit 模式先按阶码分五类，再看尾数细分：

类别	阶码 e	尾数 f	值	十六进制例
+0	全 0	全 0	+0	0x00000000
-0	全 0	全 0	-0 ($s=1$)	0x80000000
次正规数	全 0	非 0	$(-1)^s \times 0.f \times 2^{-126}$	0x00000001 为 2^{-149}
$\pm\text{Inf}$	全 1	全 0	$(-1)^s \times \infty$	0x7F800000 为 $+\infty$
NaN	全 1	非 0	不是一个数	0x7FC00000

次正规数 (subnormal, 也叫非规格化数) 处理下溢一侧的边界。规格化数依赖隐含的前导 1, 能表示的最小正数是 $e = 1, f = 0$ 的 1.0×2^{-126} (0x00800000); 比这更小的数装不下隐含位。于是标准约定 $e = 0$ 时隐含位变成 0, 指数固定为 -126, 值为 $0.f \times 2^{-126}$ 。f 的 23 个 bit 里最小非零值是末位的 1, 对应 $2^{-23} \times 2^{-126} = 2^{-149}$, 这就是单精度最小正数。关键在于过渡方式: 从 2^{-126} 继续向下, 表示能力不是悬崖式掉到 0, 而是有效位一位一位减少、精度逐渐丢失——这称为渐进下溢 (gradual underflow)。典型产生场景是两个接近的极小规格化数相减, 差落在规格化范围之下。

无穷大覆盖另一端的边界, 产生路径有两条。一条是真除零: IEEE 语义下 $1.0/0.0$ 得 $+\infty$ 、 $-1.0/0.0$ 得 $-\infty$, 同时置除零标志, 默认不停机。另一条是溢出: 结果的指数超过单精度上限 (约 3.4×10^{38}), 例如 $10^{38} \times 10$, 按当前舍入模式变成 Inf 或最大有限值。Inf 参与运算有定义好的结果: $1/\infty = 0$, $\infty + x = \infty$, 比较时大于一切有限值。流水线因此不必为超界结果插入特例停顿。

NaN 表示“没有定义”的运算结果: $0/0$ 、 $\infty - \infty$ 、 $0 \times \infty$ 、对负数开平方都产生 NaN。硬件必须实现它的两条语义。一是传播: NaN 参与几乎所有运算, 结果仍是 NaN (尾数字段通常原样传下去), 异常不会在长算式中途中被静默吞掉。二是比较: NaN 与任何值比较都为不相等, 包括它自己, 所以 $x == x$ 为假即可检出 NaN——这条性质被编译器和数学库直接利用。还要注意零的符号: $+0$ 与 -0 比较相等, 但 $1/(+0) = +\infty$ 、 $1/(-0) = -\infty$, 符号位在除法中仍然可观察; 负数下溢向零舍入时也会产生 -0 。

特殊值的意义在于让浮点成为封闭系统: 任意 32-bit 输入、任意运算, 都有定义好的 32-bit 输出, 控制逻辑无需为异常组合停下流水线。代价是 FPU 必须为阶码全 0 和全 1 各准备一条旁路, 译码、比较、类型转换和舍入都要先查这两类模式——这正是前面“特殊值需要旁路和异常规则”的具体内容。

浮点加法比整数加法复杂得多。两个整数相加, 只需对齐同名 bit 并沿进位链传播; 两个浮点数相加, 通常要先比较指数, 把较小数的有效数字右移对齐, 再做加减, 随后规格化结果、舍入, 并检查异常标志。若一个很大的数和一个很小的数相加, 小数可能在对阶右移时被移出保留位宽之外, 结果看起来像“小数被吃掉”。这不是软件显示问题, 而是 FPU 内部位宽和舍入规则的结果。

```
large = 1.0 * 2^30
small = 1.0
large + small may round back to large
when the significand has too few bits.
```

一次浮点加法要走完对阶、相加、规格化、舍入四步 FPU 的加法器不是一条进位链, 而是一条固定次序的小流水线。用 $1.5 + 0.078125$ 把四步完整走一遍。先把两

个十进制数写成规格化二进制，再写成 binary32:

```
1.5      = 1.1_2 * 2^0
          s=0  e=0111 1111 (127)  f=100 0000 ... 0000
          -> 0x3FC00000
0.078125 = 1.01_2 * 2^-4
          s=0  e=0111 1011 (123)  f=010 0000 ... 0000
          -> 0x3DA00000
```

第一步，对阶。两数指数不同，尾数不能直接相加：小数点位置不同的两个值必须先对齐，这与前面定点加法的前提完全一样。指数差是 $127 - 123 = 4$ ，规则是小阶向大阶对齐：把 0.078125 的有效数字右移 4 位，它的指数随之补到 127。为什么不让大阶向小阶对齐？那需要把大数尾数左移，最高的隐含位会移出保留位宽，丢的是最高有效位；右移小数丢的是最低有效位，损失最小。对阶后的 bit:

```
1.5:      1.1000 0000 0000 0000 0000 0000 * 2^0
0.078125: 0.0001 0100 0000 0000 0000 0000 * 2^0
          (significand shifted right by 4)
```

实际硬件在尾数右侧还会多留 guard、round、sticky 三种附加位，接住被移出的 bit，供最后一步舍入查询。

第二步，尾数相加。对齐后的两个 24-bit 有效数字（含隐含位）走普通整数进位链：

```
1.1000 0000 0000 0000 0000 0000
+ 0.0001 0100 0000 0000 0000 0000
= 1.1001 0100 0000 0000 0000 0000
```

第三步，规格化。和已经是 $1.x$ 形态，前导 1 在位，无需移动。（若和大于等于 2，例如 $1.5 + 1.5 = 11.0_2$ ，要右移 1 位并把指数加 1；若和小于 1，例如两个相近数相减，要左移直到前导 1 重新出现，指数相应减小。）

第四步，舍入。本例 23 位尾数恰好装下 1001 0100 ...，被舍弃部分为 0，不发生舍入，结果直接拼回三段：

```
s=0  e=0111 1111  f=100 1010 0000 ... 0000
-> 0x3FCA0000 = 1.578125 = 1.5 + 0.078125
```

舍入规则换一个必然触发它的例子看： $2^{24} + 1$ 。 2^{24} 的尾数是 1.000 ... 000，指数 24；1 对阶时要右移 24 位，它唯一的那个 1 被移到 23 位尾数之外。此时和的真值是 $2^{24} + 1$ ，恰好落在两个可表示值 2^{24} 和 $2^{24} + 2$ 的正中间——这个指数下尾数末位的权重 (ulp) 是 2。默认舍入模式是 round to nearest even：就近舍入，平局时取尾数末位为偶数的那个。 2^{24} 的尾数末位是 0，为偶，于是结果舍回 2^{24} ：这次加法真实执行了，加上的 1 却被格式位宽吃掉。若加的是 3，离 $2^{24} + 2$ 更近，结果就是 $2^{24} + 2$ 。

这个机制解释了为什么大数加小数容易“吃掉”小数：尾数只有 23 位，ulp 随指数放大，小数的全部有效位可能都在 ulp 之下，对阶右移后被舍入规则抹平。这也意味着浮点加法的每一步都可能按当时的指数重新舍入。

由此也能理解浮点加法通常不满足严格结合律。 $(a + b) + c$ 和 $a + (b + c)$ 可能在不同中间步骤舍入，最终得到不同的最低几位。工程上不能把浮点数当成实数域的无限精确元素；必须知道格式位宽、舍入模式、异常处理和比较策略。

8087 x87 协处理器把浮点执行从 8086 主整数路径旁边分离出来，说明浮点硬件从一开始就有明显的复杂度边界。后来 x86 通过 SSE、AVX 等 SIMD 浮点/向量扩展，把多个浮点操作组织成并行数据通路；现代高性能 CPU 还会用

流水线、转发、乱序调度和宽向量单元降低吞吐延迟。相反，许多小型 MCU 为了面积、功耗和确定性，仍然选择没有 FPU 或只提供较窄 FPU，由软件库、整数或定点路径承担一部分数值任务。

数值格式决定硬件代价和时延 同样是“加法”，整数、定点、BCD 和浮点的硬件路径并不相同。整数加法的核心是位对齐后的进位链；定点加法在格式一致时复用整数加法器；BCD 加法需要半字节合法性检测和十进制校正；浮点加法则增加对阶、尾数运算、规格化、舍入和特殊值处理。格式越灵活，硬件要承担的解释工作越多。

四种格式的取舍也因此不同。无符号/补码整数靠加法器、移位器和比较标志工作，简洁、快速、面积低，代价是范围固定、不能直接表示小数；定点用整数 ALU 加缩放、舍入和饱和，实时性好、适合 DSP 和控制场景，代价是尺度要人工管理；BCD 用二进制加法加十进制校正，十进制输入输出直接，代价是位宽利用率低、校正逻辑额外增加；浮点靠对阶、尾数、规格化、舍入和异常处理工作，动态范围大、通用科学计算方便，代价是面积、功耗、验证和延迟全面上升。

现代芯片能把很多数值运算做得很快，不是因为物理定律允许“没有延迟”，而是因为工程上同时用了多种手段。第一，常见路径被专门做成短关键路径，例如整数加法器采用超前进位或前缀结构，避免所有进位逐位等待。第二，复杂运算被流水线化，单次结果仍有若干周期延迟，但每个周期都能接收新操作，提高吞吐。第三，SIMD/向量单元把多个相同格式的数据并排处理，摊薄取指、译码和控制成本。第四，缓存、寄存器重命名、旁路转发和调度逻辑尽量让数据在核心附近流动，减少等待内存和长线传输的时间。

因此，“低时延”必须分清两种说法：**单次操作延迟**指一个结果从输入到可用需要多久，**吞吐率**指稳定状态下每个周期能完成多少个操作。现代浮点乘加单元可能有多个周期的单次延迟，但由于流水线很深，可以在每个周期发射新的乘加；整数 ALU 可能单次延迟更短，但若数据依赖形成长链，仍会受到前一条结果可用时间限制。读芯片资料时，应同时看 latency、throughput、数据格式、向量宽度、异常模式和是否支持饱和/舍入。

移位器也是算术部件。左移一位通常相当于乘以 2，但前提是移出去的 bit 没有携带需要保留的信息。右移更容易出差别。逻辑右移在高位补 0；算术右移保留符号位，用于补码有符号数。

```
logical right shift: 1000 >> 1 = 0100
arithmetic right shift: 1000 >> 1 = 1100
```

若 1000 按 4-bit 补码解释为 -8，算术右移得到 1100，也就是 -4；逻辑右移得到 0100，也就是 4。两个结果差异巨大。原因不是移位器“出错”，而是控制信号选择了不同硬件规则。循环移位又是第三种规则：移出的 bit 从另一端回填，并且常常与进位标志一起参与多字长位操作。

8086 的 FLAGS 寄存器把 Carry、Zero、Sign、Overflow 等标志暴露给分支、条件转移和多字长运算。80386 把通用寄存器和地址计算扩展到 32-bit 后，仍保留同类标志语义。这个延续说明：位宽可以扩大，执行单元可以复杂化，但 CPU 仍需要一组可复查的**结果标志**，把 ALU 的组合输出交给控制流和后续指令使用。

若想把本节内容搭到面包板上，可以先做一个 4-bit 数值解释实验：用 4 个拨码开关给出 b3..b0，用 4 个 LED 显示原始 bit，再用纸面表格分别写出无符号值和补码值。此时不需要任何复杂芯片，关键是训练一个习惯：LED 只显示 bit，数值来自解释规则。后面接入 74HC283 加法器、74HC86 反相控制和 74HC157 选择器时，仍然要保留这张解释表，否则很容易把结果 bit 与数学整数混为一谈。

二、从加法器到可复用 ALU

ALU 是算术逻辑单元。它不是孤立出现的大型部件，而是因为加法、减法、按位逻辑、移位和比较反复出现，硬件需要把这些能力集中到一个受控制的组合逻辑块中。ALU 的本质是：多个候选计算电路并排存在，控制信号选择本周期采用的结果，并同时产生可供控制器判断的标志位。

最小 ALU 可以写成接口：

```
ALU(a, b, op) -> result, flags
```

a 和 b 是两个操作数， op 是选择信号， $result$ 是结果， $flags$ 是结果附带的标志信息。这里的接口不是软件函数，而是一组并行电线：两个操作数总线进入 ALU，控制线进入选择网络，结果总线和标志线从 ALU 输出。

op	运算	硬件来源
ADD	$a + b$	加法器
SUB	$a - b$	加法器、B 反相控制、最低位进位
AND	$a \& b$	按位 AND 门阵列
OR	$a b$	按位 OR 门阵列
XOR	$a \text{ xor } b$	按位 XOR 门阵列
SHIFT	逻辑/算术/循环移位	移位器和补位选择逻辑
CMP	比较	通常复用减法路径和标志位

加法器从半加器开始。半加器接受两个 1-bit 输入，输出和位 sum 与进位 $carry$ ：

```
sum   = a xor b
carry = a and b
```

完整加法器还要接受来自低位的进位 cin ：

```
sum   = a xor b xor cin
cout  = (a and b) or (cin and (a xor b))
```

把多个完整加法器串起来，就得到串行进位加法器。最低位先计算，再把进位送到下一位。这个结构规则、容易画出、容易搭电路，也最能说明关键路径：最高位结果必须等待低位进位逐级传播。位宽越宽，最坏延迟越大。

这条层次链值得记住：半加器用 XOR 和 AND 解释一位相加的和位与进位；完整加法器在半加器上加入进位合成，构成多位串行进位链；四个完整加法器级联就是一片 4-bit 加法器，对应 74HC283/74LS283 一类芯片；ALU 加法路径则再加上输入选择和标志生成，支持 ADD、SUB、地址计算和比较。第一章的门级实验、本章的 ALU 和后面 CPU 的数据通路，走的都是这一条路。

ALU 里最值得细看的是 SUB。若单独做减法器，会增加面积和连线。补码让它复用 ADD 的加法器：

```
if op = ADD:
    b_to_adder = b
    carry_in   = 0

if op = SUB:
    b_to_adder = NOT b
    carry_in   = 1

result = a + b_to_adder + carry_in
```

实际电路里，B 的每一位前面可以接 XOR 门：当 $sub=0$ ， $b \text{ xor } sub = b$ ；当

$\text{sub}=1$, $b \text{ xor } \text{sub} = \text{NOT } b$ 。最低位进位输入也由 sub 控制。这样 ADD 和 SUB 共用同一条进位链。

sub	B 输入处理	Cin0	实际运算
0	B 原样通过	0	$A + B$
1	B 每位取反	1	$A + (\sim B) + 1 = A - B$

这个小结构可以直接搭成可观察实验。实验不需要先做完整 CPU，只要把几个连接点分开搭建、逐一观察，就已经包含了 ALU 复用的核心：A/B 输入用拨码开关给出两个 4-bit 操作数，原始 bit 要和纸面数值解释一致；B 反相用 74HC86 的四个 XOR 门处理 B 的四位， $\text{sub}=0$ 时原样通过、 $\text{sub}=1$ 时逐位取反；进位输入把同一个 sub 接到最低位 Cin，加法时 $\text{Cin}=0$ 、减法时 $\text{Cin}=1$ ；结果输出让 74HC283 的输出接 LED 或逻辑探针， sub 切换时加法和减法结果都要可复现。

74HC283/74LS283 是常见 4-bit 二进制全加器，适合观察多位进位链；74HC86 提供四个二输入 XOR 门，适合做 B 输入的可控反相；74HC157 提供四路二选一选择器，可用于在逻辑结果和加法结果之间选择。若再接上 74HC173 或 74HC574 保存操作数和结果，就能组成一个可单步观察的 4-bit ALU 实验台。实验重点不是追求速度，而是把 sub 、Cin、结果 LED 和标志 LED 之间的关系看清楚。

ALU 内部可以并行产生多个候选结果，再用多路选择器按 op 选出最终 result 。

```
add_result    = adder(a, b_to_adder, carry_in)
and_result    = a and b
or_result     = a or b
xor_result    = a xor b
shift_result  = shifter(a, shift_amount, shift_kind)

result = mux(op, add_result, and_result,
             or_result, xor_result, shift_result)
```

这不表示所有运算按软件顺序“算一遍”。组合逻辑是并行传播的：加法器、按位逻辑、移位器都可能同时产生自己的候选输出。最终哪一路被后级看见，由 mux 决定。代价也随之出现。候选电路越多，ALU 面积越大；mux 输入越多，选择延迟越大；若某个候选结果来自长进位链，它可能成为 ALU 的关键路径。

ALU 常见标志位包括：

标志位	来源	用途
Zero	结果所有 bit 做 OR 后再取反	相等判断、循环结束、条件选择
Carry	最高位外进位或借位约定	无符号溢出、多字长加减、无符号比较
Overflow	补码有符号运算的符号异常	有符号范围检查、有符号比较
Negative/Sign	结果最高位	补码符号判断、条件分支输入
Parity/辅助标志	特定低位统计或半字节进位	某些 ISA、BCD 或兼容性需求

Zero 可以由所有结果 bit 做 OR 后再取反得到。Negative 通常直接来自结果最高位。Carry 来自加法器最高位外的进位。Overflow 则要看有符号加减法中符号是否出现不可能的变化。对于加法，若两个输入符号相同而结果符号不同，则发生补码溢出；对于减法，若被减数和减数符号不同而结果符号与被减数不同，也发生

补码溢出。标志位不是额外装饰，而是后续控制的重要输入。

动作	Carry/Borrow 解释	Overflow 解释	常见用途
ADD	最高位外进位，表示无符号超界	同号相加得到异号	无符号加法、多字长加法、有符号溢出检查
SUB	由 ISA 约定表示无借位或借位	异号相减且结果符号异常	无符号大小比较、有符号大小比较
CMP	通常执行减法但不写回结果	与 SUB 相同，只保留比较标志	条件分支、条件选择、循环判断
INC/DEC	有些 ISA 不更新 Carry，或按专门规则更新	仍按补码边界判断	地址递增、循环计数，必须查清 ISA 约定

这张表强调一个容易被忽略的事实：标志位不是全局数学真理，而是**指令语义与硬件标志的约定**。同样执行 $A-B$ ，相等判断只关心 Zero，无符号小于关心借位或 Carry 约定，有符号小于要结合 Sign 和 Overflow；判断 A 是否为 0，直接检查 A 或执行传递操作，看 Zero 标志即可。早期 x86、6502、Z80 等处理器都把这些标志放进状态寄存器，但具体命名、是否更新、条件码如何解释并不完全相同。读芯片手册或写控制器时，必须先弄清“本条动作会更新哪些标志”。

多字长运算展示了 Carry 的实用价值。若只有 8-bit ALU，却要做 16-bit 加法，可以先加低 8 位，保存低字节结果和进位；再加高 8 位，并把低字节产生的 Carry 作为高字节加法的 Cin。早期 8-bit 处理器经常依靠这种方式完成更宽整数运算。软件看到的是 16-bit 加法，硬件实际执行的是多个周期内的低字节、高字节和进位链传递。

```
low_sum, carry0 = add8(A_low, B_low, 0)
high_sum, carry1 = add8(A_high, B_high, carry0)
result = high_sum:low_sum
```

把这个观察放到经典芯片中，可以看到同一问题的不同规模。6502、Z80、8086、80386 都必须解决 ALU 如何支持加法、减法、逻辑运算、移位和标志位。早期 8-bit 处理器更强调用较小数据通路和多周期控制完成动作；8086 把 16-bit 数据通路、段偏移地址形成和标志控制组织在更复杂的执行部件里，ALU 不只做算术，也服务地址计算和条件判断；80386 扩展到 32-bit 后，位宽、标志、地址计算和控制路径都随之加重，进位、布线、译码和时序压力同时上升；现代核心则把 ALU 复制到多个执行单元，用旁路、重命名和乱序调度喂养它们，但每条执行路径仍要满足时序约束。不同芯片的实现细节不同，但抽象不变：候选计算、选择器、进位路径、标志生成和时钟边界共同形成 ALU 行为。

超前进位按层级展开，进位选择用复制换等待 串行进位的瓶颈前面已经看过：16-bit 加法器的 C16 要等 16 段进位接力。超前进位 (carry-lookahead) 把每一位的进位方程 $C_{i+1} = G_i + P_i C_i$ 逐位代入，直到右边只剩 G 、 P 和 C0。4-bit 完整展开：

```
G_i = a_i AND b_i      (generate)
P_i = a_i XOR b_i      (propagate)

C1 = G0 + P0*C0
C2 = G1 + P1*G0 + P1*P0*C0
C3 = G2 + P2*G1 + P2*P1*G0 + P2*P1*P0*C0
C4 = G3 + P3*G2 + P3*P2*G1 + P3*P2*P1*G0 + P3*P2*P1*P0*C0
```

右边只依赖输入位和 C_0 ，四个进位因此可以同时开算：一串代入被换成一片与-或两级逻辑。代价也写在式子里——项数和扇入随位宽增长， C_4 的最后一项要 5 输入 AND，进位 OR 要收 4 项；位宽到 8 以上，单级展开的门扇入就不现实。所以更宽的加法器把同一条递推分层使用。

16-bit 的做法是 4 组 4-bit CLA 再加一层组间逻辑。每组内部按上面的展开并行产生组内进位；每组再输出两个组级信号：组产生 GG 表示“本组不管低位进位是什么，自己一定向外产生进位”，组传播 GP 表示“低位来什么进位，本组原样传出去”。

$$GG = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0$$

$$GP = P_3 \cdot P_2 \cdot P_1 \cdot P_0$$

第二级用完全相同的递推处理四个组的 GG/GP ，一次算出全部组间进位：

$$C_4 = GG_0 + GP_0 \cdot C_0$$

$$C_8 = GG_1 + GP_1 \cdot GG_0 + GP_1 \cdot GP_0 \cdot C_0$$

$$C_{12} = GG_2 + GP_2 \cdot GG_1 + GP_2 \cdot GP_1 \cdot GG_0 + GP_2 \cdot GP_1 \cdot GP_0 \cdot C_0$$

$$C_{16} = GG_3 + GP_3 \cdot GG_2 + GP_3 \cdot GP_2 \cdot GG_1 + GP_3 \cdot GP_2 \cdot GP_1 \cdot GG_0 + GP_3 \cdot GP_2 \cdot GP_1 \cdot GP_0 \cdot C_0$$

延迟由此重排。串行 16-bit 是 16 段进位接力；两级超前的进位路径只剩约 4 段：位级 G/P 一级门，组间进位与-或两级，组内进位与-或两级，末位求和 XOR 一级。按级门延迟估算，串行约 $1 + 2 \times 16 = 33$ 级，两级超前约 8 级——接力段数从 16 降到约 4。层级化的含义就是： $C = G + PC$ 这条递推在组内用一次、在组间再用一次；经典板级方案里，74182 超前进位发生器承担的正是组间这一层。

进位选择加法器 (carry-select) 走另一条路。把 16-bit 分成 4 段，每段并排摆两套 4-bit 加法器：一套假设段进位输入为 0，另一套假设为 1，两份和同时算完。真实的段间进位到达时，不再穿过段内进位链，只驱动一层 mux，从两份候选里选一份。等待时间变成“第一段完整加法”加“其后每段一级 mux”，约等于 4-bit 加法加三级 mux，远小于 16 段接力。

代价同样直接：段内加法器接近两倍，再加一层与位宽成正比的 mux 阵列，面积显著上升。为什么仍值得？因为串行结构里“等进位”的那段时间被利用起来，把两种可能的分支都提前算完——选择代替了等待。**提前算出所有候选，再用真实信号选择**是硬件里反复出现的换时手段：超前进位用逻辑深度换时间，进位选择用电路复制换时间；位宽继续加大时两者常常组合使用（组内超前、组间选择），32-bit 和 64-bit ALU 的主加法器很少是单一结构。

16-bit 结构	进位等待	额外代价	适用场景
串行进位	16 段接力，约 33 级门	几乎无额外面积	教学实验、窄位宽
两级超前	约 4 段，约 8 级门	大扇入与-或逻辑、组间 PG 网络	ALU 主加法路径
进位选择	一段加法加每段一级 mux	段内加法器近两倍加 mux 阵列	高主频宽位加法

这三种结构后面还会与移位、比较、乘法一起再比较一次；这里先记住一点：加法器的位宽不是简单复制全加器，进位组织方式本身就是设计变量。

三、进位、移位、比较与乘法的硬件取舍

加法器是 ALU 的核心路径之一，因为减法、地址计算、数组下标、栈指针更新和分支目标计算都会反复使用它。最直观的结构是串行进位加法器：第 0 位产生的

进位送到第 1 位，第 1 位再送到第 2 位，直到最高位。它面积小、结构规则，但最坏延迟随位宽增加而增长。4-bit 时容易接受，32-bit 或 64-bit 时就可能成为关键路径。

可以把每一位的进位逻辑归结为两类信号：

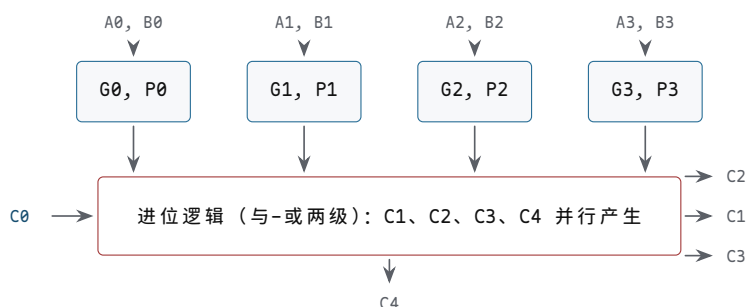
```
generate_i = a_i and b_i
propagate_i = a_i xor b_i
carry_out_i = generate_i or (propagate_i and carry_in_i)
```

若本位的 A 和 B 都为 1，本位会**产生**进位；若 A 和 B 有且仅有一个为 1，本位会**传递**低位进位。快速加法器的思想，就是用额外组合逻辑提前计算这些产生和传递关系，减少“等低位一位一位传上来”的时间。这说明速度不是免费获得的：更短的延迟通常换来更多门、更复杂连线 and 更严格布局。

把“提前计算”真正展开一次。以 4-bit 为例，把 $C_{i+1} = G_i \vee (P_i C_i)$ 逐位代入，每个进位都改写成只含 G、P 和 C_0 的形式：

```
C1 = G0 + P0*C0
C2 = G1 + P1*G0 + P1*P0*C0
C3 = G2 + P2*G1 + P2*P1*G0 + P2*P1*P0*C0
C4 = G3 + P3*G2 + P3*P2*G1 + P3*P2*P1*G0 + P3*P2*P1*P0*C0
```

注意右边只依赖 A、B 和 C_0 ：C4 不再等 C3，四个进位可以同时开算。这就是“超前进位”的全部含义——不是让进位消失，而是把进位链的串行代入改成并行的与或逻辑。



图示：4-bit 超前进位结构：四个 G/P 单元同时算出产生/传播信号，进位逻辑用与-或两级把 C1-C4 一次性算出来，不再串行等待。

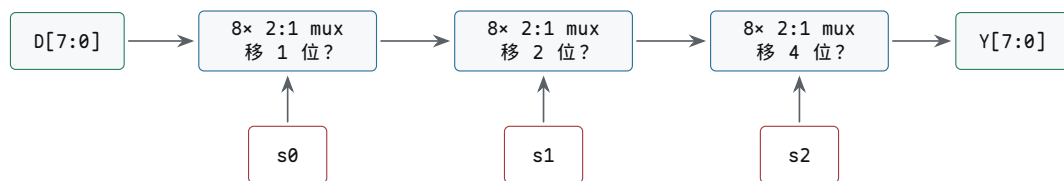
延迟对比按“级门延迟”估算（数量级示意，不代表具体工艺）：4-bit 串行进位，G/P 1 级加每位约 2 级进位逻辑再乘 4 级接力，最坏约 9 级；4-bit 超前进位，G/P 1 级、进位逻辑与-或 2 级、求和 XOR 1 级，最坏约 4 级。位宽越大差距越夸张：64-bit 串行要约 $1 + 2 \times 64 = 129$ 级，而树形前缀结构能把进位逻辑压到 $\log_2 64 = 6$ 层左右。这就是 64-bit ALU 不能只按“64 个全加器排队”估算性能的原因——真实高性能加法器都在用前缀或分组结构买这几级延迟。

几种结构的取舍由此而来：串行进位面积小、布线规则、容易扩展，代价是进位逐位传播、最坏延迟随位宽增长；超前进位预先计算产生/传递关系、缩短进位等待，代价是组合逻辑和布线更复杂；分组进位在面积和速度之间折中，代价是要设计分组边界和跨组进位；前缀加法器适合高性能宽位加法，代价是布线密集、功耗和布局压力较大。教学机可以先用串行进位把 trace 跑通，追求主频时再换超前或前缀结构。

现代芯片为什么能降低时延，不能简单归结为“晶体管更快”。在真实处理器里，低时延来自一组共同措施：把常用路径做短，把宽路径拆分成前缀或分组结构，把长组合逻辑放进流水线，把结果通过旁路网络尽快送给后续执行单元，把缓存和寄存器堆放在物理上更近的位置，并让版图工具控制连线长度、扇出和时钟偏斜。晶体管速度、门级结构、物理布局、流水线边界和调度策略共同决定最终周期时间。每项措施都有代价：更快的标准单元可能增加面积、功耗和漏电；分组和前缀进位让

布线更密、布局更难；流水线切分把延迟折成周期数，控制和旁路更复杂；结果旁路要增加选择网络和冲突判断；物理邻近要求模块复制和更强的布局约束；专用单元则要求常见运算有足够高的利用率才值得定制。

移位器也有类似取舍。若每次只移动一位，硬件简单，但一次移动 13 位就要多周期完成。桶形移位器允许在一个组合路径中按控制位选择移 1、2、4、8 等距离，多个阶段叠加得到任意移位量。以 8-bit 为例，移位量的低三位可以控制三层 mux：第一层决定是否移 1 位，第二层决定是否再移 2 位，第三层决定是否再移 4 位。这样能在一个周期内完成多位移位，但 mux 层数和连线负载会进入关键路径。



例：移位量为 5 (1 0 1) 时，s2=1 先移 4 位，s1=0 不动，s0=1 再移 1 位，合计 5 位，一次组合传播完成。

图示：8-bit 桶形移位器：三层 mux 分别由移位量的三个 bit 控制，每一层选择“直送”或“移 2^k 位”。任意 0~7 位移位量都能在一个组合路径内完成，代价是三层 mux 的延迟和大量交叉连线。

几种移位的硬件规则也要分清：逻辑移位在一端补 0，控制线选错会改变数值解释；算术右移在高位补符号位，必须区分有符号和无符号解释；桶形移位用多层 mux 按 1/2/4/8 位选择，mux 延迟和长连线容易成为关键路径；循环移位把移出位从另一端回填，标志位和数据位的关系必须写清楚。

比较器通常可以复用减法路径。判断相等时，执行 $A-B$ 后检查 Zero；判断无符号大小时，关注借位或 Carry 的解释；判断补码有符号大小时，不应仅看最高位，还要结合 Overflow。例如 1000 和 0111 在补码中分别是 -8 和 7，在无符号中分别是 8 和 7。同一组 bit 的大小关系随解释变化，因此比较器必须和控制器约定当前比较类型。

```
eq_unsigned_or_signed = (A - B).Zero
lt_unsigned            = borrow_from_subtract
lt_signed              = result.Sign xor result.Overflow
```

这里的公式表达的是常见标志组合，具体处理器对 Carry 与 Borrow 的命名可能不同。重要的是不要把“最高位为 1”误当成所有比较场景里的“小于”。在补码有符号比较中，结果符号需要用 Overflow 修正；在无符号比较中，符号位没有负数含义。

乘法器展示了另一类工程折中。最朴素的二进制乘法可以生成部分积，再把部分积相加。若一次性展开成组合阵列，延迟和面积都较大，但吞吐可以高；若用移位加法多周期完成，面积小，但一次乘法需要多个周期。早期处理器常常用多周期或微码方式处理复杂运算，现代处理器则可能提供专用乘法单元、流水化乘法器或多个执行单元。抽象上看，乘法仍然是“部分积生成、对齐、求和、保存结果”，只是硬件把这些步骤压缩、展开或流水线化。

几种实现的取舍：移位加法每周检查乘数一位、必要时累加被乘数移位值，面积小、周期数多，适合简单控制器；组合阵列同时生成多个部分积并通过加法网络求和，面积大、延迟大，但吞吐高；流水化乘法器在部分积压缩和求和之间插入寄存器，吞吐高但单次结果跨多个周期；Booth 编码和压缩树减少部分积数量或加快求和，代价是控制和布局复杂度增加。

阵列乘法器把乘法展开成部分积矩阵

二进制乘法和十进制竖式乘法结构相同：用乘数的每一位去乘被乘数，把得到的行按位权错开，再全部相加。差别只在于二进制里“一位乘一个数”格外简单——乘数位是 1，部分积就是被乘数本身；乘数位是 0，部分积就是全 0。所以“生成一个部分积位”在硬件上就是一个 AND 门： $p_{ij} = a_i \wedge b_j$ 。

以 4×4 位无符号乘法 $A \times B$ 为例, $A = a_3a_2a_1a_0$, $B = b_3b_2b_1b_0$, 先把 16 个部分积位排成矩阵, 每一行是被乘数与乘数一位的逐位与, 行与行之间按乘数位的权重错开一位:

	a3	a2	a1	a0				
x	b3	b2	b1	b0				

	a3b0	a2b0	a1b0	a0b0	<- 第 0 行, 权 2^0			
	a3b1	a2b1	a1b1	a0b1	<- 第 1 行, 权 2^1			
	a3b2	a2b2	a1b2	a0b2	<- 第 2 行, 权 2^2			
a3b3	a2b3	a1b3	a0b3		<- 第 3 行, 权 2^3			

p7	p6	p5	p4	p3	p2	p1	p0	<- 乘积, 共 8 位

代入具体数值 0111×0101 ($7 \times 5 = 35$), 逐行移位累加:

```

      0 1 1 1
    x 0 1 0 1
    -----
      0 1 1 1      (b0=1, 部分积 = 被乘数)
     0 0 0 0        (b1=0, 部分积 = 0)
    0 1 1 1         (b2=1, 左移两位)
   0 0 0 0          (b3=0)
   -----
  0 0 1 0 0 0 1 1 = 35

```

阵列乘法器就是把这张竖式直接铺成硬件： n^2 个 AND 门一次生成全部部分积位，再用 $n-1$ 行 n 位加法器逐行累加——第 0 行与第 1 行相加，结果再与第 2 行相加，依此类推。4×4 位乘法因此需要 16 个 AND 门和 3 行加法器，共约 12 个全加器。

延迟来自“逐行累加”这条链。每行加法本身有一次进位传播，行与行之间又是串行的，所以阵列乘法器的最坏延迟大致随位宽线性增长，且每一级都比同位宽加法器多出一行进位链。面积的增长更快：AND 门和全加器的数量都按 n^2 走，连线规模也是 n^2 量级。位宽翻一倍，加法器只长大一倍，乘法器却大约长大三倍（变为四倍）。

位宽 n	AND 门（部分积位）	全加器（ $n(n-1)$ ，逐行累加）	关键路径
4	16	12	3 行加法链
8	64	56	7 行加法链
16	256	240	15 行加法链
32	1024	992	31 行加法链

这就解释了为什么硬件乘法器比加法器贵得多：加法器是 $O(n)$ 个全加器排成一条链，乘法器是 $O(n^2)$ 个门铺成一片阵列，面积、延迟、功耗全面吃亏。所以小型控制器常用“每周期加一行”的移位加法方案，用时间换面积；只有乘法频繁出现的处理器才值得放一块完整阵列。现代高性能处理器进一步用 Booth 编码把部分积行数减半、用压缩树（如 Wallace 树）把逐行串行累加改成树形并行压缩，但“部分积矩阵加求和网络”这个基本结构没有改变。

Booth 编码用“一减一加”代替逐位累加

逐位移位加法的累加次数等于乘数中 1 的个数，最坏情况下 n 位乘法要做 n 次。Booth 编码的观察是：一串连续的 1 可以整体改写。例如乘数中出现 011110，按位权展开是 $2^4 + 2^3 + 2^2 + 2^1 = 30$ ；但它也等于 $2^5 - 2^1 = 32 - 2$ ，也就是 $100000 - 000010$ 。原本 4 次加法的区段，改写成“高端加一次、低端减一次”就够了。二进制减法同样由加法器完成（取补相加），所以这种改写不引入新硬件，只减少累加次数。

两位 Booth 编码把这个想法系统化：从最低位开始，每次看乘数的当前位 b_i 和它右边的相邻位 b_{i-1} （最低位右边补一个虚拟的 0），两位组合决定本轮对被乘数 M 做什么：

$b_i b_{i-1}$	本轮动作	含义
0 0	+0	一串 0 的中间，无事可做
0 1	+ M	一串 1 的开始（低端），把被乘数加上
1 0	- M	一串 1 的结束（高端），把被乘数减掉
1 1	+0	一串 1 的中间，已经折算过，跳过

直观记忆：从右往左扫乘数，0→1 是“1 串的入口”，做一次加；1→0 是“1 串的出口”，做一次减；00 和 11 都在串的内部，不累加。每轮做完后乘数右移一位，把下一对 bit 送上来。

以 4-bit 例子 0011×0110 ($3 \times 6 = 18$) 走一遍。被乘数 $M = 0011$ ，乘数 $B = 0110$ ，右侧补 0 后逐对考察：

轮次	考察位 $b_i b_{i-1}$	动作	对累加结果的贡献
1	0 0	+0	0
2	1 0	- M	$-0011 \times 2^1 = -6$
3	1 1	+0	0 (1 串中间)
4	0 1	+ M	$+0011 \times 2^3 = +24$

合计 $-6 + 24 = 18$ ，结果正确。普通逐位法对乘数 0110 要做 2 次加法，这个例子里 Booth 也是一加一减，看似没有节省。差别在 1 串变长时才显现：乘数若是 011110 (3 个连续 1)，逐位法要 3 次加法，Booth 仍只要“低端一加、高端一减”共 2 次动作；乘数若是全 1 的 1111，逐位法要 4 次加法，Booth 只在最低位遇到一次 1→0（实际是考察位 (1,0)），做一次减就结束。

Booth 编码的意义不在于硬件更精巧，而在于把“乘法要做几次累加”从“取决于乘数里有几个 1”变成“取决于乘数里有几段 1”。它还附带一个好处：这套规则天然兼容补码有符号乘数，不需要像原码乘法那样单独处理符号位。现代乘法器普遍采用按两位、三位分组的改进 Booth (radix-4、radix-8)，把部分积行数减到 $n/2$ 或 $n/3$ 行，再配合压缩树求和——这就是乘法器能在一个流水段里算完的起点。

恢复余数除法：一次一位地试出商

除法是乘法的逆过程，但硬件上它比乘法更难安排：乘法每一步做什么由乘数位直接决定，除法每一步要先“试一试”才知道。恢复余数除法 (restoring division) 是最直白的方案，和笔算长除法同构：每轮把部分余数左移一位，试减除数；够减，这一位商就是 1，余数保留差值；不够减，这一位商是 0，把余数恢复成试减前的值，进入下一轮。

以 4-bit 无符号除法 $13 \div 3$ 为例：被除数 1101，除数 0011。用一个 8-bit 寄存器 R 存放部分余数，高 4 位初始为 0，低 4 位初始为被除数，即 $R = 00001101$ ；除数始终与高 4 位对齐比较。每轮三步： R 整体左移一位；高 4 位试减 0011；差不为负则保留差并把最低位置 1，否则恢复高 4 位、最低位置 0。

轮次	左移后的 R	高 4 位试减 0011	商位	本轮结束的 R
初始	—	—	—	0000 1101
1	0001 1010	0001 - 0011，不够减，恢复	0	0001 1010
2	0011 0100	0011 - 0011 = 0000，够减	1	0000 0101
3	0000 1010	0000 - 0011，不够减，恢复	0	0000 1010

实验阶段	新增器件或连线	预期现象
4-bit 加法	74HC283、拨码开关、LED	0011+0101=1000, Carry 正确
加/减复用	74HC86 处理 B, sub 接 Cin	sub=1 时 A-B 正确
逻辑候选	AND/OR/XOR 门阵列	候选结果与真值表一致
结果选择	74HC157 多路选择器	op 改变时只有被选结果进入 LED
标志输出	OR 合成 Zero, 最高位和进位生成标志	结果为 0 时 Zero=1, 溢出案例可复现

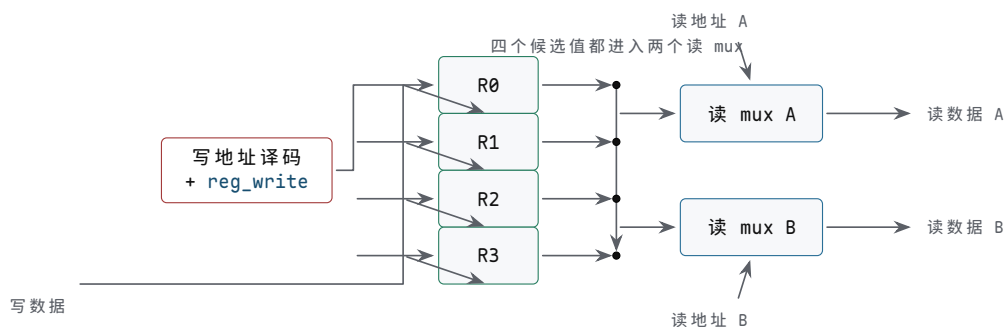
四、数据通路让值从旧状态走到新状态

数据通路回答一个问题：值从哪里来，经过哪里，最后保存到哪里。孤立的寄存器和 ALU 还不能完成连续动作。必须把寄存器阵列、选择器、ALU、内部连线和写回路接成一个可控制的闭环。若 ALU 是“能算什么”，数据通路就是“值能走哪条路”。

一个寄存器只能保存一个值。若硬件需要同时保存多个中间值，就需要寄存器阵列。寄存器阵列可以理解为一组编号寄存器，加上读写选择电路。

端口	作用	类型
读地址 A/B	选择两个源寄存器	输入
读数据 A/B	输出两个源值	输出
写地址	选择目标寄存器	输入
写数据	提供要保存的新值	输入
写使能	决定是否在时钟沿写入	输入

读地址进入译码和选择网络，决定哪两个寄存器的值出现在读数据端口。写地址进入译码器，写使能有效时，只允许被选中的那个寄存器在时钟沿接收写数据。这样，一组寄存器就能被编号访问，而不是每个寄存器单独拉一套控制线。真实处理器中的寄存器堆常常有多个读端口和写端口，原因也很直接：ALU 常常需要同时读两个源操作数，并在周期末写回一个目标。



图示：4 寄存器堆的内部结构：写地址经译码（并与 `reg_write` 相与）只打开一个目标的写入；写数据广播到所有寄存器的 D 端，但只有被选者提交；两个读 mux 按读地址各选一路输出。读是组合选择，写是时钟沿提交——这条不对称是寄存器堆和“一组变量”的本质区别。

ALU 的两个输入不一定总来自同一处。一个输入可能来自寄存器阵列，也可能来自 PC、计数器或固定 0；另一个输入可能来自寄存器、常量或扩展后的字段。因此 ALU 输入前通常有 mux。

```
alu_a = mux(a_sel, reg_a, pc, zero, latch_a)
alu_b = mux(b_sel, reg_b, immediate, one, offset)
```

以寄存器 R1 和 R2 相加、结果写入 R3 为例。数据通路需要让读地址 A 选择 R1，读地址 B 选择 R2；`a_sel` 选择 `reg_a`，`b_sel` 选择 `reg_b`；`alu_op` 选择 ADD。

此时 ALU 输出就是 $R1+R2$ 。

写回数据也不一定只来自 ALU。它可能来自 ALU，也可能来自存储器读口、外部输入、PC+4 或专用状态寄存器。写回前也需要 mux。

```
write_data = mux(wb_sel, alu_result,
                memory_data, input_data, pc_plus_4)
```

在 $R3=R1+R2$ 的动作中，wb_sel 选择 alu_result，写地址选择 R3，reg_write 为 1。等时钟沿到来，R3 才真正保存结果。若 reg_write 为 0，即使 ALU 算出了正确结果，寄存器阵列也不会改变。这个边界来自第二章的同步设计原则：组合逻辑可以在周期内传播和抖动，状态只在受控时钟沿提交。

当多个部件都可能把值送到同一个目的地时，不能让它们同时强行驱动同一条线。要么用 mux 明确选择一路，要么用总线协议规定同一时刻只有一个驱动者。ALU 输出、存储器输出和外部输入如果直接接在一起，一旦两个源同时输出不同电平，就会形成冲突。mux 让这件事变成明确选择：当前周期只允许一路成为 write_data，适合芯片内部数据通路和 FPGA 逻辑，代价是输入多时选择器延迟和面积增加。三态总线也能共享连线，适合板级总线和部分分立芯片实验，但必须保证同一时刻只有一个输出使能有效，多个驱动同时打开就是冲突。点到点连线适合高频、固定方向的数据路径，代价是连线数量增加、灵活性降低。在初学实验和 FPGA 设计中，优先使用 mux 更容易检查和调试。

三态总线型数据通路：一条线上轮流说话

第一章讲三态门时说过：三态输出除了 0 和 1，还有高阻态 Z——输出端像被“拔掉”一样，既不驱高也不驱低。三态总线正是利用这一点组织数据通路：把多个寄存器的输出、ALU 的输出、存储器读出都接到同一组公共线上，每个源前面放一个三态缓冲器，各自由输出使能控制。任何时刻最多一个使能有效，总线上就只有这一家在“说话”，其余源都处在高阻态，对总线电平没有影响。写入侧则反过来：总线上的值广播到所有寄存器的输入端，写使能决定谁在时钟沿接收。

与 mux 型数据通路对照，差别在“选择发生在哪里”。mux 方案里每条数据通路独立布线，由接收端的选择器挑一路——选择是集成的、点对点的，几路输入同时驱动各自的线互不相干。总线方案里，选择分散在每个驱动者的输出使能上，依赖一条全局约定：同一时刻只有一个驱动者。这条约定一旦被破坏，两个源同时驱动总线，一个驱高一个驱低，线上就是不可预期的电平，还可能流过过大的电流。

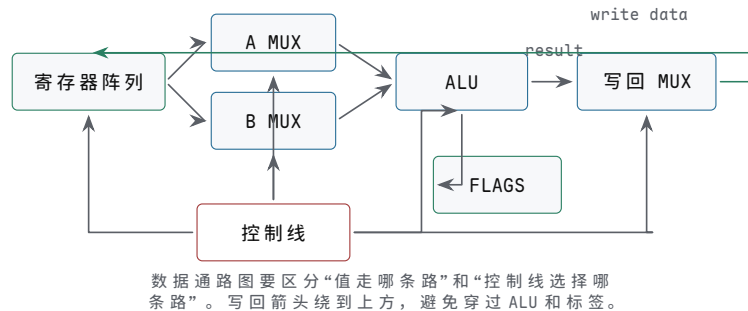
对照点	三态总线型	mux 型
连线数量	每个位宽只有一组公共线，源再多也不加线	每个源到每个接收点独立走线，随源数增长
面积代价	省布线、省选择器，适合布线紧张场合	多路 mux 本身占面积，输入越多级数越深
出错模式	使能冲突（多驱）或全高阻（无驱），影响是全局性的	选择信号错即选错路，故障局限在单点
时序分析	总线电平依赖各使能的时序，要分析所有潜在驱动者	路径固定，逐条路径算延迟即可
调试入口	先查“此刻谁在驱动”	直接观察选择信号和对应路径

早期小型机和微处理器爱用总线型，原因很实际：当时芯片内部布线层数少、布线资源紧张，分立搭建时电路板上的走线和插接件更是寸土寸金；一条公共总线让寄存器、ALU、存储器、I/O 都挂上来，扩展一个新部件只需再挂一个三态驱动器，不必为每个新源重铺一组线。PDP-11 的 Unibus、各类 8-bit 微处理器的地址/数据总线，都是这个思路。代价是把“谁在驱动”的正确性推给了控制设计：使能信号必须由译码逻辑保证互斥，切换驱动者时最好留一拍全高阻的间隔，避免旧驱动尚未关闭、新驱动已经打开的瞬间冲突。正因为这条约定难以形式化检查，现代芯片

内部数据通路大多回到 mux 型；三态总线更多留在板级互连和存储器接口这类“真正共享一条物理线”的地方。

一个最小数据通路闭环可以写成：

```
register file -> mux -> ALU -> mux -> register file
```



图示：最小数据通路闭环。源值从寄存器阵列读出，经过输入选择和 ALU，最后由写回选择器在时钟边界写回。

它能在一个周期内读取旧值、选择输入、完成组合计算，并在时钟沿保存结果。这个闭环已经能表达很多动作，但它还不会自己决定“当前周期做什么”。哪些地址送入寄存器阵列，ALU 做加法还是减法，写回选择哪一路，写使能是否打开，都需要控制器统一产生。

如果把这个闭环搭成分立电路，器件可以按功能分组选择。74HC574 或 74HC173 可以保存寄存器值；74HC138 可以把写地址译码成某个寄存器的写使能；74HC157 可以选择 ALU 输入或写回来源；74HC283/74HC86 组合成加减核心；LED 或逻辑分析仪用于观察寄存器输出、ALU 输出和写使能。这个实验的关键不是一次做完整指令集，而是让 R1、R2、R3 的值在时钟沿前后清晰可见。

这个 4 寄存器堆有一个重要限制：若用普通 74HC574，没有专门的输出三态控制和写使能逻辑，就要额外设计时钟使能或 D 端保持路径；若用 74HC173 这类带使能和输出控制的寄存器，实验更容易分阶段观察。无论选哪种器件，原则相同：写地址只决定“谁有资格写”，全局 `reg_write` 决定“本周期是否写”，时钟沿决定“何时真正写”。读 mux 是组合选择，写入是时序提交，两者不能混在一起。

把各连接点的器件和检查要点再落实一层：R0 到 R3 的存储用 74HC173 (4-bit 寄存器) 或 74HC574 (8-bit 成组保存)，复位后初值确定、时钟沿前后值可观察；写地址译码用 74HC138 把两位 `write_addr` 展开成四路写使能候选，同一周期只允许一个目标写入；全局 `reg_write` 与译码输出相与后送入各寄存器，`reg_write=0` 时所有寄存器保持；读端口 A/B 各用一组 4-bit mux 按 `read_addr_a`、`read_addr_b` 独立选择，A 端口变化不应改写寄存器内容，A/B 可同时读同一寄存器或不同寄存器；写回数据经写回 mux 进入所有寄存器 D 端，未选寄存器即使 D 端变化也不能写入。

真实芯片把这些部件压缩进同一颗硅片。8051 把 CPU、片内 RAM、特殊功能寄存器、定时器、串口和 I/O 口组织在一个单片机内部；对程序员来说，某些外设像“寄存器地址”一样被读写；对硬件来说，这是数据通路、地址译码、外设寄存器和控制器共同工作的结果。8086 把总线接口单元和执行单元分工，执行单元内部仍要围绕寄存器、ALU、标志和控制路径组织动作。80386 继续扩大寄存器和地址计算规模，并引入更复杂的保护与寻址硬件。规模不同，问题相同：值必须从旧状态沿受控路径走到新状态。

从 8051 看，单片机不能简化为“小型软件平台”，而是 CPU 数据通路、片内存储器、I/O 口、定时器和中断控制的硬件组合。第一章的输入调理和逻辑门，第二章的状态边界，本章的寄存器与控制线，都在单片机内部或引脚边界重新出

现。后面整机章讨论 MMIO、设备和中断时，会继续使用这条线索。

数据通路的时序分析应至少包含四项：源状态何时稳定，组合路径最坏延迟是多少，目标触发器建立/保持时间是否满足，哪些写使能在时钟沿有效。若一个单周期 $R3=R1+R2$ 路径太长，可以把动作拆成多周期：第一周期读 $R1/R2$ 并锁存到 A/B ，第二周期 ALU 计算并锁存到 $ALUOut$ ，第三周期把 $ALUOut$ 写回 $R3$ 。这样周期时间可以变短，但一次动作需要更多周期。速度、面积和控制复杂度再次形成取舍。

三种方案的取舍：单周期让读寄存器、ALU 计算、写回都在一个周期内完成，控制简单，但周期必须容纳最长路径；多周期把读、算、写分到多个状态和多个时钟沿，周期可短，代价是控制状态增加；流水线让多条动作在不同阶段重叠推进，吞吐提高，代价是冒险、旁路和冲刷更复杂。本章先用单周期和多周期把控制 trace 跑通，流水线留到最小 CPU 之后再展开。

五、控制器输出一组一致的控制线

数据通路提供很多可能路径，但同一周期不能所有路径都随意打开。控制器的任务是根据当前状态、指令字段和条件输入，产生一组控制信号，让数据通路执行一个明确动作。控制器不是写在纸上的命令列表，而是一块输出控制电线的硬件。它可以由组合逻辑、状态机、ROM、PLA（可编程逻辑阵列：把若干 AND 项和 OR 项做成规则阵列的组合逻辑，适合把译码表直接烧进硬件）、微码表或这些方法的混合实现。

控制信号通常是一组 0/1 或多 bit 选择值：

```
alu_op
a_sel
b_sel
wb_sel
read_addr_a
read_addr_b
write_addr
reg_write
flags_write
mem_read
mem_write
next_state_sel
```

这些信号直接接到 ALU、mux、寄存器阵列、标志寄存器和存储器接口。控制信号给错，数据通路就会走错。例如 `reg_write` 本应为 0 却变成 1，某个寄存器会在时钟沿被意外覆盖；`wb_sel` 本应选择 ALU，却选了外部输入，写回值就会错；`flags_write` 本应关闭却打开，旧标志会被无关结果污染，后续条件分支会误判。

若动作规则简单，可以用组合逻辑直接从输入生成控制信号。例如输入 `mode` 为 0 时做加法，为 1 时做按位 AND：

```
mode = 0 -> alu_op = ADD
mode = 1 -> alu_op = AND
```

硬连线控制速度快，结构也直接。缺点是规则复杂后，组合逻辑会变大，修改一个动作可能影响多条控制线。若一个动作无法在一个周期内完成，就需要状态机。状态机保存“当前走到哪一步”，每个状态输出一组控制信号，并决定下一状态。

控制器可以分成两个层次理解。第一层是**译码**：把 opcode、功能位、条件位和当前状态翻译成控制线。第二层是**时序安排**：决定这些控制线在哪个周期有效，哪些状态元件在该周期写入。若指令简单、动作少，硬连线译码可以直接生成控制信号；若指令复杂、步骤多，控制器可能使用微操作表或微码 ROM，把一条指令拆成多行控制字逐步执行。

几种控制方式的取舍：硬连线控制适合动作规则固定、追求低延迟的路径，代价

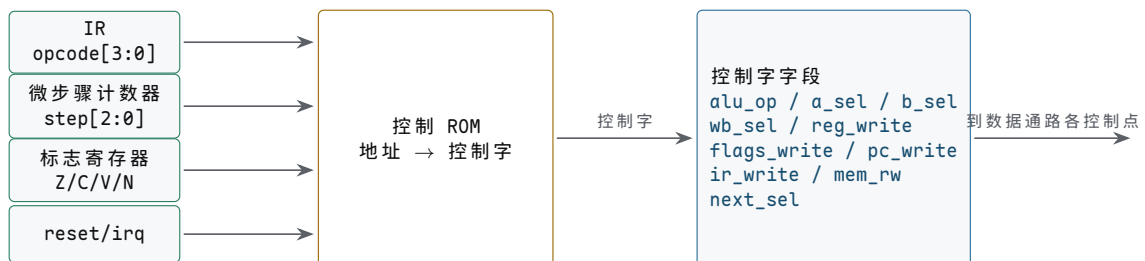
是逻辑复杂后修改困难、一个错误影响多条控制线；状态机控制适合多周期动作和明确阶段推进，代价是状态编码、非法状态和输出毛刺都要处理；微码控制适合指令复杂、步骤可表格化的场景，代价是微码存储、入口选择和异常中断点更复杂；混合控制让常见简单路径走硬连线、复杂路径走微操作，代价是必须统一提交边界和异常恢复规则。

一行控制字可以像下面这样理解：

```
state EXEC_ADD:
  read_addr_a = rs1
  read_addr_b = rs2
  a_sel       = REG_A
  b_sel       = REG_B
  alu_op      = ADD
  wb_sel      = ALU_RESULT
  write_addr  = rd
  reg_write   = 1
  flags_write = 1
  next_state  = FETCH
```

这并非软件语句，而是对硬件控制线的命名描述。控制字中的每一位最终都要连接到 mux、ALU、寄存器写使能、标志寄存器、存储器接口或下一状态逻辑。调试控制器时，不应只看“当前指令是什么”，还要看当前状态、控制字、条件输入和本周期会提交哪些状态。

控制器也可以用分立芯片做出教学模型。一个 74HC161 计数器可以充当微步骤计数器，74HC138 可以把微步骤译码成若干状态脉冲，小容量 EEPROM 或 ROM 可以存放控制字。地址线接 opcode、微步骤和标志位，数据线输出 `alu_op`、`a_sel`、`b_sel`、`reg_write` 等控制信号。这种做法的好处是非常直观：修改控制表就能改变机器动作；代价是控制存储器访问延迟、控制字宽度和异常入口都要进入时序设计。



图示：控制 ROM 的结构：指令 opcode、微步骤、标志位和复位/中断请求拼成 ROM 地址，地址内容就是本周期全组控制线。改机器动作等于改 ROM 内容表——这就是微码控制”指令是“可编程表格”的含义。

控制 ROM 地址位	来源	作用
<code>opcode[3:0]</code>	指令寄存器中的操作码	选择当前指令类型，如 ADD、SUB、LOAD、BRANCH
<code>step[2:0]</code>	微步骤计数器或状态寄存器	区分取指、译码、执行、写回等阶段
<code>Z/C/V/N</code>	标志寄存器输出	条件分支或条件写回的输入
<code>reset/irq</code>	复位或外部请求同步后的状态	强制进入复位入口或异常入口

控制 ROM 的数据输出就是控制字。为了便于搭实验机，可以先把控制字设计成固定宽度字段，而不是零散电线。

控制字段	连接对象	含义
<code>alu_op[2:0]</code>	ALU 函数选择	指定 ADD、SUB、AND、OR、XOR、PASS 等动作
<code>a_sel[1:0]</code>	ALU A 输入 mux	选择寄存器 A、PC、0 或临时寄存器
<code>b_sel[1:0]</code>	ALU B 输入 mux	选择寄存器 B、立即数、1 或偏移
<code>wb_sel[1:0]</code>	写回 mux	选择 ALUOut、存储器数据、PC+1 或外部输入
<code>reg_write</code>	寄存器堆写使能	决定目标寄存器是否在时钟沿写入
<code>flags_write</code>	标志寄存器写使能	决定 ALU 标志是否提交
<code>pc_write</code>	PC 写使能	决定下一 PC 是否提交
<code>ir_write</code>	指令寄存器写使能	决定本周期是否锁存取到的指令
<code>mem_read/write</code>	存储器接口	发起读周期或写周期
<code>next_sel[1:0]</code>	下一状态逻辑	选择顺序微步骤、取指、分支目标或异常入口

这个表可以直接变成 EEPROM 内容表。例如 `opcode=ADD, step=EXEC` 的控制字会选择寄存器 A/B 作为 ALU 输入, `alu_op=ADD`, 打开 `flags_write`, 并把下一状态设为写回; `opcode=BRANCH, step=EXEC` 的控制字则可能关闭 `reg_write`, 读取 Zero 标志, 通过 `next_sel` 选择顺序 PC 或分支目标。判断控制 ROM 方案是否可行, 关键不在“表看起来合理”, 而在每一位输出都能追到一个实际器件引脚。

微程序控制器：把控制字存进一块存储器

前面控制 ROM 的做法其实已经触到了微程序控制 (microprogrammed control) 的核心: 与其用组合逻辑从 `opcode` 现场译出控制线, 不如把“每条指令在每个周期该输出什么”事先写成一张表, 存进一块专门的控制存储器。表里的每一行叫一条微指令, 一台机器全部微指令合起来叫微程序。一条机器指令不再对应一段组合逻辑, 而对应控制存储器里的一段微指令序列—取指之后跳到这段微程序, 逐条执行, 走完这段微程序, 这条机器指令的所有周期也就走完了。

一条微指令通常包含三类字段:

字段	作用	例子
控制线字段	本周期送往数据通路的全部控制位, 每位接一条控制线	<code>alu_op</code> 、 <code>a_sel</code> 、 <code>reg_write</code> 、 <code>mem_read</code>
下一微地址字段	指出正常情况下下一条微指令的地址	<code>next</code> = 微地址 17
条件选择字段	指定本周期是否查看条件、查看哪个条件, 条件成立时改走哪个地址	看 Zero: 成立去分支微地址, 不成立用下一微地址

负责读这张表的小硬件叫微序列器 (microsequencer)。它每个周期做同一件事: 按当前微地址取出一条微指令, 把控制线字段直接送往数据通路, 同时按条件选择字段决定下一个微地址—顺序、跳到指定地址、或按条件二选一。机器指令译码在这里退化成一次跳转: 取指结束后, 用 `opcode` (必要时拼上功能位) 查一张入口表, 得到这条机器指令微程序的首地址, 交给微序列器接着走。条件分叉也用同一机制: 条件跳转指令的微程序里安排一条指定“看 Zero 标志”的微指令, Zero 为 1 时下一微地址指向“更新 PC”的微指令, 为 0 时指向“直接回取指”的微指令。

```
; 微程序示意: ADD 与条件跳转 BEQ 各对应一段微指令
FETCH:      mem_read=1, ir_write=1, next=DISPATCH
DISPATCH: 按 opcode 查入口表 -> ADD_U0 / BEQ_U0 / ...
ADD_U0:     a_sel=REG_A, b_sel=REG_B, alu_op=ADD,
```

```

        reg_write=1, flags_write=1, next=FETCH
BEQ_U0:  a_sel=PC, b_sel=OFFSET, alu_op=ADD,
        条件选择=Zero: Z=1 -> BEQ_U1, Z=0 -> FETCH
BEQ_U1:  wb_sel=ALU_RESULT, pc_write=1, next=FETCH

```

与普通组合逻辑译码（硬连线控制）对照，差别不在“控制线从哪来”——最终每根控制线都要在正确的周期输出正确的电平——而在“规则写在哪里”。硬连线方案把规则固化在与或门阵列里，改一条指令的行为就要改逻辑、重新布线；微程序方案把规则写成存储器内容，改指令行为只改微码，硬件本身一根线不动。这个差别在工程上影响深远：指令集复杂、寻址方式多的机器，比如 8086 这类变长指令、多周期动作的处理器，用硬连线译码会让组合逻辑大到难以设计和检查，微程序把复杂度转移成“填表”；机器出厂后发现指令行为有问题，有时只更新微码就能修正，不必更换芯片。代价同样明确：每条微指令多一次控制存储器访问，周期下限受存储器延迟约束；微地址、条件选择、入口表又是一层要调试的状态。所以规则简单的现代核心回到硬连线快速路径，把微码留给复杂指令和罕见情况——两层控制器并存，正是这一节两种方案在真实芯片里的最终分工。

在分立实验中，每个部件都有对应实现和各自的检查点：状态寄存器用 74HC161 计数器或 D 触发器组，检查每个时钟沿状态是否按预期推进；状态译码用 74HC138 或门阵列，检查是否只有当前状态对应的控制线有效；控制存储用 EEPROM、ROM 或拨码开关矩阵，检查同一地址是否输出稳定控制字；条件选择让 Zero/Carry/Overflow 进入下一状态逻辑，检查条件成立和不成立是否走向不同状态；复位逻辑用复位按钮加同步释放或明确初始化，检查上电后是否进入确定状态。

控制错误往往比 ALU 错误更隐蔽，因为 ALU 本身可能算对了，但错误选择线把结果送错地方，或者错误写使能在错误时钟沿提交状态。

排查要按控制线分类：`alu_op` 错会算错函数、标志位也可能错，先查 opcode 译码、功能位和状态选择；`a_sel/b_sel` 错让 ALU 输入不是预期来源，先查操作数字段、mux 控制和临时寄存器；`wb_sel` 错让正确结果没有被写回，先查写回 mux 和控制状态；`reg_write` 错让目标寄存器未写或被意外覆盖，先查写使能门控和时钟边界；`flags_write` 错让条件分支使用错误标志，先查标志寄存器写入条件和指令语义；`mem_write` 错让存储器或设备被误写，先查地址译码、写周期和保护条件。

8086 常被用来说明控制复杂度。它的指令长度不固定，寻址方式丰富，许多动作需要拆成多个内部步骤；因此不能把控制器想象成“opcode 直接连到 ALU op”这么简单。80386 进一步加入 32-bit 模式、保护机制和更复杂的地址转换相关检查，控制路径需要处理更多条件。现代处理器又在硬连线快速路径、微操作、预测、乱序调度和异常恢复之间做更复杂的工程分解。无论规模如何，控制器最终仍要回答同一个问题：本周期哪些电线有效，哪个状态会被提交。

六、用控制 trace 走读一次动作

复杂动作可以拆成微操作：读、选择、计算、保存、更新状态。本节所称微操作，指一个周期内同时打开的一组硬件控制信号。以下先以 R1 和 R2 相加、结果写入 R3 为例。为了让时序更容易观察，采用三周期版本：S0 读源寄存器并锁存，S1 计算并锁存 ALU 输出，S2 写回目标寄存器。

```

S0: read_sel_a=R1, read_sel_b=R2
    A_write=1, B_write=1
    reg_write=0, ALU_out_write=0

S1: a_sel=A, b_sel=B, alu_op=ADD
    ALU_out_write=1
    flags_write=1
    reg_write=0, A_write=0, B_write=0

S2: wb_sel=ALU_out, write_sel=R3
    reg_write=1
    A_write=0, B_write=0, ALU_out_write=0

```

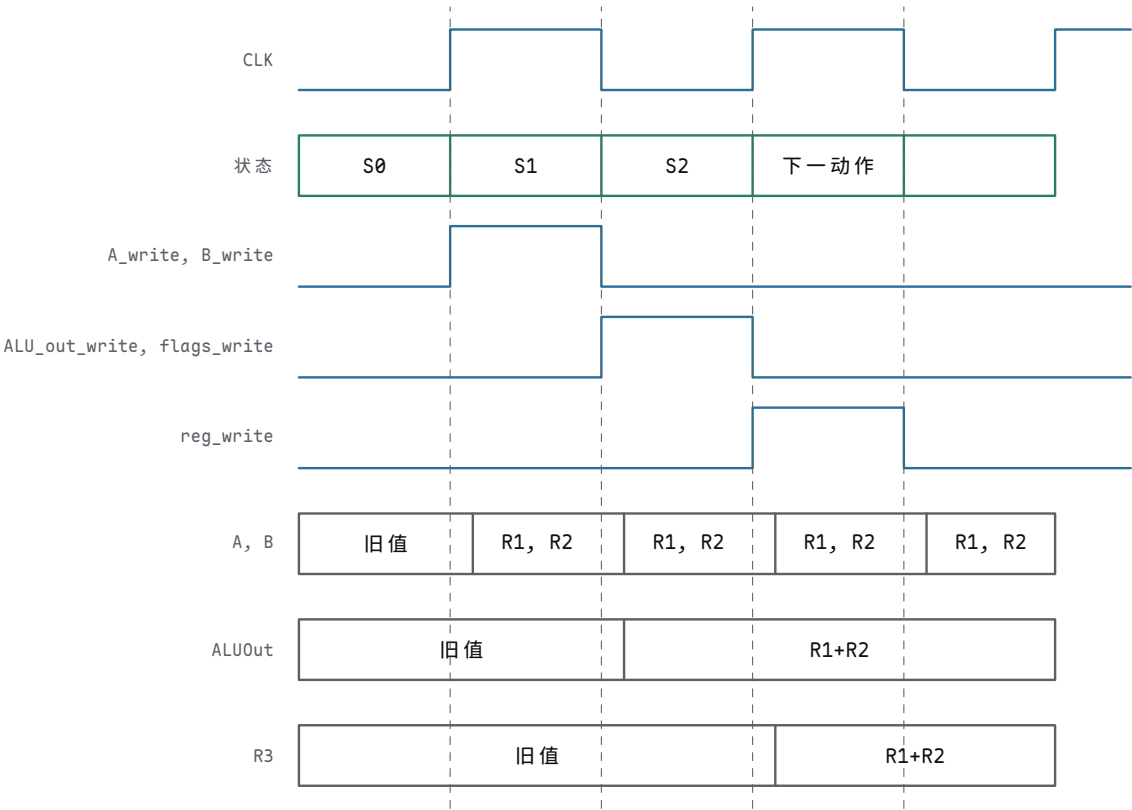
第一步 S0 需要落实为具体控制信号。`read_sel_a` 选中 R1 后，寄存器阵列的第一个读端口会把 R1 当前保存的 bit 组合驱动到读数据线上；`read_sel_b` 选中 R2 后，第二个读端口同理输出 R2。等这些组合路径稳定后，时钟沿到来，`A_write` 和 `B_write` 让 A、B 锁存器保存这两个值。这个周期里 `reg_write` 必须为 0，因为此时写回 mux 上的值并不代表最终结果。

第二步 S1 负责 ALU 计算。A、B 锁存器的输出作为 ALU 输入，`alu_op=ADD` 让 ALU 内部选择加法路径。加法电路不是瞬间得到稳定结果，低位进位会影响高位，输出线上可能经历一小段变化过程。控制器在这个周期打开 `ALU_out_write`，让时钟沿把已经稳定的 ALU 输出锁存下来；同时 `reg_write` 仍然保持 0。若本动作需要后续条件判断，`flags_write` 也在这个周期打开，把 Zero、Carry、Overflow 和 Negative 保存到标志寄存器。

第三步 S2 负责写回。`wb_sel` 选择 ALUOut 锁存器，`write_sel` 选择 R3，`reg_write` 在这个周期置 1。到时钟沿时，寄存器阵列把写数据端口上的值保存到 R3。至此，R1 和 R2 没有被改动，R3 得到 R1+R2 的结果，控制器回到空闲、取指或下一动作状态。

状态	打开的路径	周期末提交	必须关闭
S0	R1/R2 到 A/B 锁存器	A、B	<code>reg_write</code> 、 <code>ALU_out_write</code>
S1	A/B 到 ALU，ALU 到 ALUOut	ALUOut、Flags	<code>reg_write</code>
S2	ALUOut 到写回端口，写地址 R3	R3	A/B 写使能、 ALUOut 写使能

把这张表画成对齐波形后，“打开/提交/关闭”的关系就能逐项检查：



图示：三周期 $R3=R1+R2$ 的对齐波形：控制线只在规定周期为高；状态元件只在上升沿提交。注意三条控制线的互斥关系——任何一拍 `reg_write` 提前变高，R3 都会被未成熟的值污染；任何一拍 `ALU_out_write` 忘记关闭，下一次计算边界就会混入旧值。

该例也说明控制错误的风险。若 S2 的 `wb_sel` 错选成 B 锁存器，R3 写入的就是 R2，而不是相加结果；若 S1 提前打开 `reg_write`，R3 可能在 ALU 输出尚未稳定时被覆盖；若 S0 忘记关闭 `ALU_out_write`，旧的计算边界会被无关读数污染。微操作表的价值在于：每一步都能检查哪些线打开、哪些线关闭、哪个状态元件会在时钟沿改变。

再看一个条件动作：若 `R1-R2` 的结果为 0，则下一状态跳到 `TARGET`，否则继续 `NEXT`。硬件上这仍然是数据通路和控制器配合：ALU 执行 SUB，Zero 标志保存比较结果，下一状态逻辑读取 Zero 并选择目标。

```
S0: read_sel_a=R1, read_sel_b=R2
    A_write=1, B_write=1

S1: a_sel=A, b_sel=B, alu_op=SUB
    flags_write=1

S2: if Zero then next_state=TARGET
    else next_state=NEXT
    reg_write=0
```

这个 trace 说明比较不是神秘动作。相等比较可以由减法和 Zero 标志实现；条件跳转不是 ALU 输出直接“跳走”，而是控制器根据标志选择下一状态或下一 PC。后面进入最小 CPU 章节时，分支、跳转、访存、I/O 和中断都可以用类似 trace 分解。

七、把第三章接到最小 CPU

到这里，CPU 的几个必要部件已经在概念上齐备。寄存器堆保存通用值，ALU 产生算术逻辑结果和标志位，数据通路用 mux 和总线把源、结果和写回连起来，控制器在每个周期输出一组控制线。最小 CPU 还需要 PC、指令寄存器、指令译码、存储器接口和复位入口，但这些部件并不是凭空出现的：PC 是带加法和选择的寄存器，下一章要给它加上顺序取指、分支目标和异常入口；指令寄存器是保存当前指令 bit 的状态元件，下一章要解释字段和 opcode 译码；译码器是把 bit 字段翻译成控制线的组合逻辑；存储器接口是另一条受控数据通路，下一章要处理总线等待、对齐、MMIO 和异常。

把这些部件真正拼成最小 CPU 时，接口清单比概念名称更重要。下面这张表可以作为下一章的硬件检查单：每个部件不只要有名称，还要有输入、输出和写入边界。

部件	关键输入/输出	必要控制线
PC	输出取指地址，输入顺序 PC 或分支目标	<code>pc_write</code> 、 <code>pc_sel</code>
指令寄存器 IR	输入存储器返回的指令 bit，输出 opcode 和寄存器字段	<code>ir_write</code>
寄存器堆	输出两个源操作数，输入写回数据	<code>read_addr_a/b</code> 、 <code>write_addr</code> 、 <code>reg_write</code>
立即数扩展器	输入指令字段，输出零扩展或符号扩展值	<code>imm_kind</code> 或译码规则
ALU	输入两个操作数，输出结果和标志	<code>a_sel</code> 、 <code>b_sel</code> 、 <code>alu_op</code> 、 <code>flags_write</code>
存储器接口	地址、写数据、读数据、等待信号	<code>mem_read</code> 、 <code>mem_write</code> 、 <code>addr_sel</code>
控制器	输入 opcode、状态、标志和等待信号，输出控制字	<code>next_sel</code> 、复位入口、异常入口

若这张接口表无法填写完整，最小 CPU 就还只是部件堆叠，而不是可执行机器。特别要注意两条线：第一，PC、IR、寄存器堆和标志寄存器都是状态元件，必须有明确写使能；第二，存储器接口可能比 CPU 内部组合路径慢，因此控制器必须能等待、重试或保持状态。后面讨论整机硬件时，设备寄存器、总线等待和中断入口都会沿用同一套接口分析方法。

本章也给出了调试整机硬件的基本方法。不要先问“CPU 为什么不工作”，而要按 trace 缩小范围：PC 是否在时钟沿更新，指令寄存器是否保存了正确 bit，译码是否输出了预期控制字，ALU 输入是否来自正确源，ALU 输出和标志是否正确，写回 mux 是否选对，写使能是否只在目标周期打开。这个方法从 4-bit 面包板实验到 8086、80386 这样的经典处理器分析，再到现代核心的流水线调试，层次不同，思路一致。

八、从机器级表达式读回硬件动作 到这一章末尾，读者已经能看见一条机器级表达式背后的硬件路径。高级语言里的 `x+y`、`a[i]`、`p->field`、`x & mask` 看起来属于软件语法，但映射到硬件上，它们都要变成寄存器读、立即数扩展、移位、加法、逻辑运算、LOAD/STORE、标志位和写回。若教材只讲 ALU 真值表，不讲这些表达式如何变成地址和控制线，读者会知道“加法器能加”，却不知道程序为什么会变成一连串硬件动作。

把常见表达式逐个拆开：`x+y` 读两个寄存器，ALU 加法，结果写回，对应 `alu_op=ADD`，并产生 Carry、Overflow、Zero 标志；`x-y` 让 B 取反加一、复用加法器，`sub` 同时控制 XOR 和最低位进位；`x<<k` 由移位器按移位量选择各位来源，逻辑移位、算术移位和溢出规则要分开；`x & mask` 走 ALU 逻辑路径保留部分 bit，常用于字段提取、权限位和状态位；`a[i]` 由基址加缩放下标形成地址再 LOAD/STORE，经过移位器或乘法路径、地址加法和字节使能；`p->field` 是指针基址加固定字段偏移，依赖结构体布局、对齐和访问宽度。

以 `a[i] = a[i] + 1` 为例，若 `a` 是 32-bit 整数数组，机器动作至少包含五步。第一步下标缩放：`i<<2` 或地址生成单元计算 `i*4`，元素大小必须由类型布局决定。第二步地址形成：基址与缩放下标相加，要说明加法按地址宽度截断还是产生异常。第三步读取旧值：发起 32-bit LOAD，字节序、对齐、cache/MMIO 属性都要清楚。第四步加一：ALU 执行 `old+1`，要说明是否更新条件码、溢出如何解释。第五步写回内存：发起 32-bit STORE，字节使能必须正确，不能破坏相邻元素。每一步都能追到本章已有部件。

```
addr = base(a) + i * 4
old  = MEM[addr]
new  = old + 1
MEM[addr] = new
```

如果把 `a[i]` 错看成“数组对象知道自己的长度”，就会误解越界访问。硬件只看到基址、下标、缩放和加法。除非后续保护机制或编译器插入检查，否则 `i` 超出范围仍然会形成一个地址，并继续发起 LOAD 或 STORE。这个地址可能指向同一栈帧中的另一个变量、堆对象的元数据、返回地址、MMIO 区域，或未映射页。后面讨论异常和分页时，正是把“地址是否允许访问”提升为硬件可检查的约定。

结构体布局把字段名变成偏移 结构体字段名不是硬件对象。编译后，字段访问会变成“对象起始地址加字段偏移”。字段偏移由字段顺序、字段宽度和对齐规则决定。对齐不是排版习惯，而是为了让硬件访问落在更自然的字节边界上，减少拆分传输、简化字节使能和提高 cache 行利用率。

```
struct Packet {
    uint8_t type;
    uint8_t flags;
    uint16_t length;
    uint32_t address;
};
```

```
p->address -> LOAD32 [base(p) + 4]
```

字段	宽度	可能偏移	硬件访问
type	1 字节	0	字节 LOAD，通常零扩展
flags	1 字节	1	字节 LOAD，可与掩码配合
length	2 字节	2	半字 LOAD，受字节序影响
address	4 字节	4	字 LOAD，要求字节使能覆盖四个车道

若为了节省空间强行取消对齐，某些字段可能跨越自然边界。硬件仍可通过多个字节车道或多个总线周期完成访问，但代价会增加；某些架构会直接规定未对齐访问异常，要求软件拆成多个字节访问。教学阶段要把这件事写清楚：类型布局不是只影响内存大小，它会进入地址形成、字节选择、cache line 命中和总线传输。

位掩码是最小的机器级抽象工具 许多硬件状态寄存器不是把每个状态放进独立地址，而是把多个 1-bit 字段压在同一个字中。程序读出一个状态寄存器后，常用掩码提取某些位；写控制寄存器时，也常用掩码只改指定字段。掩码操作本质上是 ALU 的 AND、OR、XOR 和移位路径。

```
status = MMIO[UART_STAT]
ready  = (status & 0x01) != 0
error  = (status & 0x0E) >> 1

ctrl = ctrl | 0x04    // set bit 2
ctrl = ctrl & ~0x04   // clear bit 2
ctrl = ctrl ^ 0x04    // toggle bit 2
```

四种位操作各走一条 ALU 路径：AND mask 保留掩码为 1 的位、清除其他位，用于提取状态位、对齐地址和清除字段；OR mask 把掩码为 1 的位置 1，用于设置控制位和构造权限字段；XOR mask 翻转掩码为 1 的位，用于切换状态和构造取反路径；SHIFT 改变字段位置或完成缩放，用于字段对齐和数组下标乘元素宽度。

位掩码也能解释为什么设备寄存器规格必须写清楚读写语义。有些位是只读状态位，有些位写 1 清除，有些位写 0 无效，有些位一写就启动设备动作。若用普通“读-改-写”序列处理一个带副作用寄存器，可能在读时清掉事件，或在写回时误改其他字段。硬件书在这里要提醒读者：同样是 bit，普通 RAM 字节和设备寄存器字段的约定完全不同。

整数溢出要同时给出数学结果和机器结果 固定宽度整数运算的结果总是某个位宽内的 bit 模式。数学上的结果可能超出范围，此时硬件仍然给出低位截断结果和若干标志位。若程序把 INT_MAX+1 当作普通正数继续使用，后续比较、数组下标和地址形成都会被污染。硬件不会自动替程序选择“报错、饱和、回绕、陷入”哪一种语义，必须由 ISA、编译器、语言规则或显式指令规定。

这个表能把第三章和后面 CPU 章节连接起来。若 ISA 提供普通 ADD、带陷入 ADD、饱和 ADD 或宽乘法指令，控制器和 ALU 就必须输出不同的控制线和标志处理路径。若 ISA 只提供普通 ADD，软件就需要用比较和条件分支组合出额外检查。所谓“语言整数语义”，最后仍要体现为硬件结果、标志位、异常入口和控制流。

四种溢出语义各适合一类场景：回绕只保留低 N 位并由标志位记录越界，适合地址计数器、哈希和无符号模运算；陷入在溢出时进入异常或错误路径，适合安全语言运行时和调试检查；饱和把超范围结果钳到最大或最小值，适合 DSP、音频、图像和控制系统；扩大位宽用更宽临时结果保存数学值，适合乘法、累加和编译器优化的中间值。

案例：解析一个二进制包头 系统程序经常要解析来自网络、磁盘或外设 FIFO 的二进制包头。这个任务看起来属于软件格式处理，实际上集中使用了第三章的全部机器级工具：字节序、字段偏移、位掩码、整数扩展、对齐访问和溢出检查。假设某设备返回如下 8 字节包头：

```
offset  size  field
0       1    type
1       1    flags
2       2    length  // little-endian
4       4    address // little-endian
```

解析过程可以写成：

```
type  = LOAD8 [base + 0]
flags = LOAD8 [base + 1]
len_lo = LOAD8 [base + 2]
len_hi = LOAD8 [base + 3]
length = len_lo | (len_hi << 8)
addr   = LOAD32 [base + 4]
ready  = (flags & 0x01) != 0
error  = (flags & 0x0E) >> 1
```

解析动作	硬件路径	需要确认的问题
读取 <code>type</code>	字节 LOAD 后零扩展	目标寄存器高位是否清零
读取 <code>length</code>	两个字节按小端拼接	字节序是否和设备规格一致
读取 <code>address</code>	32-bit LOAD 或四次字节 LOAD 组合	对齐、字节使能和异常规则是否清楚
提取 <code>ready</code>	AND 掩码后比较零	状态位是否被读操作清除
提取 <code>error</code>	AND 后右移	字段位置和移位方向是否正确
检查长度	与缓冲区容量比较	无符号比较和溢出检查不能混用

若 `length` 来自外部设备，不能直接用它做 `base + length`。地址加法可能回绕，长度可能超过缓冲区，字段可能被设备错误写入。可靠代码需要先做范围检查，再执行后续 LOAD/STORE。硬件层面，这些检查仍然是比较器、标志位和分支。

```
if length > buffer_capacity:
    reject packet
if base + length < base:
    reject packet  // unsigned wraparound check
payload_end = base + length
```

检查项	机器级机制	漏掉后的后果
长度上限	无符号比较 <code>length <= capacity</code>	后续 STORE 越过缓冲区
地址回绕	<code>base+length < base</code> 说明无符号回绕	访问低地址或错误对象
字段合法性	掩码后剩余位应为 0 或规定值	未定义设备状态被当成合法状态
对齐要求	<code>address & (align-1)==0</code>	未对齐异常或拆分传输代价
权限属性	地址不能落入 MMIO 或只读区域	普通数据访问变成设备副作用或保护异常

这个案例是 CSAPP 式机器级思维和本书硬件 trace 的交汇点。读者不只要会写 C 代码，还要能指出每一步需要哪些 LOAD、移位、掩码、比较、标志和分支；更要能说明这些动作失败时，会表现为第五章讨论的总线错误、DMA 可见性问题，或第六章讨论的页错误和保护异常。

案例：多字长整数为什么要保存 Carry 很多低成本 CPU 或教学 CPU 的 ALU 宽度较小，但程序需要处理更宽整数。例如用 16-bit ALU 实现 32-bit 加法，必须先加低 16 位，再把低位加法产生的 Carry 加入高 16 位。这里 Carry 不是附属显示灯，而是下一步运算的真实输入。

```
// 32-bit A+B on a 16-bit ALU
lo = A_lo + B_lo
carry = carry_out(lo)
hi = A_hi + B_hi + carry
result = [hi:lo]
```

减法、多精度计数器、地址加法和校验和也会用到类似链路。若控制器在低位加法后错误关闭 `flags_write`，或在高位加法前让另一条指令覆盖 Carry，多字长结果就会错。这个例子帮助读者理解为什么真实 ISA 会提供 ADC、SBB 这类“带进位加/减”指令，也解释了为什么标志位是程序员可见状态的一部分。

把这条链路的控制线逐拍列清：低位加法用 `alu_op=ADD`、`carry_in=0`，预期低位结果和 CarryOut 正确；保存 Carry 用 `flags_write=1`，预期标志寄存器在下一周期仍可读；高位带进位加法用 `carry_in=Carry`，预期高位结果包含低位进位；写回组合结果时写回低位和高位目标寄存器，预期两个半字都在正确的时钟边界提交。

案例：32-bit 加法拆进 8-bit ALU 上一个案例用 16-bit ALU 分两次完成 32-bit 加法。若数据通路只有 8-bit 宽——早期微处理器和许多单片机正是如此——同一个 32-bit 加法就要拆成四步，从最低字节开始逐字节推进。每步的差别只有两处：第一步的最低字节没有进位输入，用普通 ADD；后面三步都必须把上一步的 Carry 当作本次加法的第三个输入，用带进位加 ADC。拆字长不是把同一个动作简单重复四遍，而是一条穿过四个字节的进位接力。

以 $A = 0x1FF34A80$ 、 $B = 0x0E21C590$ 为例，逐字节 trace 如下：

步骤	本字节计算	结果字节	Carry 输出与去向
1	$0x80+0x90$ (ADD, 进位输入为 0)	0x10	Carry=1, 送入步骤 2
2	$0x4A+0xC5+1$ (ADC, 加上一步 Carry)	0x10	Carry=1, 送入步骤 3
3	$0xF3+0x21+1$ (ADC)	0x15	Carry=1, 送入步骤 4
4	$0x1F+0x0E+1$ (ADC)	0x2E	Carry=0, 全和无第 33 位

四步之后从高到低拼出 $0x2E151010$ 。逐步核对数值：步骤 1 的 $128+144=272$ ，结果字节只保留 0x10，超出的 256 变成 Carry；步骤 2 的 $74+197+1=272$ ，同样保留 0x10 并把 1 交给下一步；步骤 3 的 $243+33+1=277$ ，保留 0x15；步骤 4 的 $31+14+1=46$ ，恰好不再越界。若步骤 3 漏掉来自步骤 2 的 Carry，本字节就差 1，错误还会沿进位链继续向高字节放大。这条链路上的每个 Carry 都不是显示信息，而是下一步的真实数据输入：任何一步算完后 Carry 被意外覆盖，后面所有字节的和都不可信。

这就是 ADC 指令存在的理由。普通 ADD 的进位输入恒为 0，无法表达“把上一步的进位加进来”；ISA 若只提供 ADD，软件就要用额外的比较和条件分支手工重建进位，代价远高于一条直接读取 Carry 标志的加法指令。因此从 8080、8086 到 x86-64，指令集都把 ADD 与 ADC 成对提供：做超出寄存器宽度的加法时，最低位用 `add`，更高位依次用 `adc`，且两条指令之间不能插入会改写标志的第三条指令。硬件

层面，ADD 与 ADC 往往共享同一条进位链，差别只在 `carry_in` 这一根线接 0 还是接标志寄存器的 Carry 输出：

```
byte0: alu_op=ADD, carry_in=0,   flags_write=1
byte1: alu_op=ADC, carry_in=CF,  flags_write=1
byte2: alu_op=ADC, carry_in=CF,  flags_write=1
byte3: alu_op=ADC, carry_in=CF,  flags_write=1
```

第四章处理更宽的地址计算、多字长计数器和宽字段时，还会反复用到这条链：宽数据被切成机器字长的若干片，片与片之间靠标志寄存器里的一位 Carry 衔接。多字长运算的正确性，最终就取决于这一位能否在两次 ALU 动作之间原样存活。

案例：把十进制字符串转成二进制整数 程序从串口、文件或命令行读到的数字是一串字符，不是数值。字符 '0' 到 '9' 在 ASCII 中的编码是 `0x30` 到 `0x39`，连续排列，所以“字符减 `0x30`”就得到该数字的数值 0 到 9。剩下的事情是按十进制位权组装：每读到一个新数字，已累积的结果扩大十倍，再加上新数值，即 `result = result*10 + digit`。以字符串 "1234" 为例：

读到字符	减 <code>0x30</code> 得数值	累积结果 <code>result*10+digit</code>
'1' = <code>0x31</code>	1	$0 \times 10 + 1 = 1$
'2' = <code>0x32</code>	2	$1 \times 10 + 2 = 12$
'3' = <code>0x33</code>	3	$12 \times 10 + 3 = 123$
'4' = <code>0x34</code>	4	$123 \times 10 + 4 = 1234$

四步之后得到二进制整数 `1234 = 0x04D2`。循环里唯一的算术是“乘十加”，而乘十不必动用乘法器： $10x = 8x + 2x$ ，乘 8 是左移 3 位，乘 2 是左移 1 位，两次移位加两次加法即可。移位器和加法器是 ALU 里已有的路径，乘法器则是面积大、延迟长的独立部件；编译器把常数乘法改写成移位加组合，正是强度削减优化的典型例子。对应的 x86-64 汇编（Intel 语法）如下，设 `rsi` 指向字符串、遇到第一个非数字字符结束：

```
    xor    eax, eax           ; result = 0
.next:
    movzx  ecx, byte [rsi]    ; 读当前字符并零扩展
    sub    ecx, 0x30          ; '0' 的编码是 0x30, 得数值 0..9
    cmp    ecx, 9
    ja     .done              ; 无符号大于 9: 非数字, 结束
    mov    rdx, rax
    shl    rdx, 3              ; rdx = result * 8
    shl    rax, 1              ; rax = result * 2
    add    rax, rdx            ; rax = result * 10
    add    rax, rcx            ; rax = result * 10 + digit
    inc    rsi
    jmp    .next
.done:
```

这段代码值得用本章的眼光再读一遍。`sub` 与 `cmp` 共用 ALU 的减法路径并更新标志；`ja` 是无符号条件跳转，依据 Carry 标志—字符小于 '0' 时减法产生借位、结果成为很大的无符号数，与大于 9 的情形被同一次比较一并拦下，所以一次无符号比较就完成了上下两个边界的检查。`shl` 走桶形移位器路径，`add` 走进位链，整个循环没有任何一条指令需要乘法器。字符串解析听起来是软件话题，拆到底仍然是本章的加法器、移位器和标志位。

专题：从门级代价看 ALU 为什么这样组织 ALU 表面上像一个会做很多运算的黑盒，内部通常更像若干条专用组合路径加一个结果选择器。加法/减法依赖进位链，逻辑运算依赖逐位门，移位依赖多级选择网络，比较往往复用减法路径和标志。理解这一点之后，读者就不会把 `ADD`、`AND`、`SLL`、`SLT` 当成同等代价的“软件操作”。

各项功能的来源和代价也不同：加法来自全加器链、超前进位或前缀进位网络，进位传播决定关键路径，位宽越大越明显；减法让 `B` 逐位取反并令最低位 `carry_in=1`，Carry 和 Overflow 的解释必须按减法规则重新定义；按位逻辑是每一位独立的 AND/OR/XOR/NOT，延迟短，但结果选择 mux 会增加负载；无符号小于看减法后的借位或 Carry 标志，不能直接看符号位；有符号小于由减法结果符号与 Overflow 共同决定，`0x8000-1` 这类边界最容易出错；移位用桶形移位器或多周期逐位移位，单周期桶形移位器面积和 mux 级数较大。

如果教学 CPU 的目标是先跑通控制 trace，可以先用纹波进位加法器和多周期移位；如果目标是提高主频，就要把加法器换成 carry-lookahead、carry-select 或 prefix 结构。这里的选择没有绝对优劣，只有约束不同：面积、功耗、最高频率、验证复杂度和 ISA 对单周期完成的承诺不同。

进位优化的核心是把“等待前一位进位”变成“并行计算哪些位会产生或传播进位”。对第 i 位，常用：

$$G_i = A_i B_i, \quad P_i = A_i \oplus B_i, \quad C_{i+1} = G_i \vee (P_i C_i)$$

其中 G 表示本位必定产生进位， P 表示本位会传播输入进位。前缀加法器会把这些关系按树形组合，缩短最长逻辑深度，但布线和门数会上升。书中不要求读者手推完整 Kogge-Stone 网络，但要求能说明：为什么一个 64-bit ALU 不能只按“64 个全加器排队”等比例粗略估计性能。

专题：ALU 状态标志的生成电路 Zero、Carry、Overflow、Negative 四个标志容易被初学者想象成“算完结果之后，再拿结果去分析一遍”的后置步骤。硬件上恰恰相反：四个标志都是同一次运算的副产物，与结果共用同一批内部信号，在同一个周期内与结果一起稳定。它们不是事后的评论，而是进位链和结果线上早已存在的消息，各自由一小段专用组合电路引出。

四个标志的来源各不相同。Negative 最简单，就是结果最高位那根线本身，补码解释下它天然指示正负，不经过任何门。Zero 是结果全部位经 OR 树归并后再取反：任何一位为 1 都会让 OR 树输出 1，只有结果全 0 时 Zero 才为 1，其树深随位宽按对数增长。Carry 来自进位链的最后一级——加法时是最高位向外的进位，减法时按约定改读为借位；它通常是标志中稳定最晚的一个，因为进位链本来就是 ALU 的关键路径。Overflow 是符号位两处进位的异或：记最高位的进位输入为 C_{n-1} 、进位输出（即 Carry）为 C_n ，则 $V = C_{n-1} \oplus C_n$ ——两个正数相加时，若数值部分向符号位进了位而符号位没有向外进位，符号就被异常翻转，异或正好检出这种“两处进位不一致”。

标志	生成电路	延迟特点
Negative	结果最高位直接引出，不经过任何门	与结果线同时稳定
Zero	全部结果位的 OR 树加一级取反	树深随位宽对数增长
Carry	进位链末级的进位输出	与进位链同长，常是关键路径
Overflow	最高位进位输入与进位输出相异或	在 Carry 之后只多一级 XOR

理解“同周期产生”对控制器设计有直接影响。前面三周期 trace 的 S1 拍里，`flags_write` 与 `ALU_out_write` 在同一个时钟沿把标志和结果分别锁存进标志寄存器和 ALUOut，二者谁也不等谁；S2 拍的条件分支读到的标志，与写回的数据严格属于同一次运算。若把标志想象成下一周期才慢慢算出来的东西，就会错误地在控制器里多加等待拍，或者反过来在标志尚未稳定的周期提前采样。真实时序里要照顾的只是各标志延迟不同：Carry 和 Overflow 沿进位链走，稳定最晚，标志寄存器的建立时间预算必须按它们来定。

专题：乘法、除法和规格化为什么常常多周期 乘法和除法不是“更大的加法按钮”。一个 n 位乘法可以看成 n 个部分积的移位求和；一个除法可以看成反复试减、

恢复或不恢复余数；浮点运算还要经历对阶、尾数运算、规格化和舍入。若所有步骤都做成单周期组合逻辑，面积和延迟会迅速失控。

每种运算的拆分方式和控制方式也不同：无符号乘法判断乘数最低位、条件加被乘数、移位、计数，用多周期移位加法或阵列/树形乘法器实现；补码乘法要处理符号扩展和负权部分积，Booth 编码能减少连续 1 的部分积数量；无符号除法左移余数、试减除数、决定商位、必要时恢复余数，用多周期状态机逐位产生商；定点乘法在原始整数相乘后调整缩放位数，乘积位宽翻倍，截断和舍入必须规定；浮点加法要对阶、尾数加减、规格化、舍入、异常标志，分流水级或由微码/协处理器完成。

以 8-bit 乘法为例，若采用移位加法，硬件只需要一个加法器、若干寄存器和一个计数器。每拍检查乘数最低位：若为 1，把当前被乘数加到累加器；然后被乘数左移，乘数右移。8 拍后得到 16-bit 乘积。这个实现慢，但面积小，特别适合早期处理器或低成本控制器。

```
product = 0
for i in 0..7:
    if (multiplier[i] == 1):
        product = product + multiplicand
    multiplicand = multiplicand << 1
    multiplier = multiplier >> 1
```

除法值得给一个完整例子。以 4-bit 恢复余数除法计算 $7 \div 2$ ：余数寄存器 R 初值为 0，被除数寄存器 Q 初值为 0111 (7)。每一拍先把 (R,Q) 整体左移一位，再试减除数；试减结果为负说明“这一位商不上”，商位记 0 并把余数加回去（恢复），否则商位记 1：

拍	(R,Q) 左移后	试减 R-2	商位	动作
1	0000,1110	$0 - 2 < 0$	0	恢复 R=0000
2	0001,1100	$1 - 2 < 0$	0	恢复 R=0001
3	0011,1000	$3 - 2 = 1$	1	保留 R=0001
4	0011,0001	$3 - 2 = 1$	1	保留 R=0001

四拍后 $Q=0011=3$, $R=0001=1$ ： $7 \div 2 = 3$ 余 1。每一步只用移位、减法和一次条件恢复——这就是为什么除法天然是多周期状态机：每拍都有“移位、试减、判负、恢复”四个微操作，商逐位产生，没有一条能把所有位一次算完的组合捷径。

浮点的难点更隐蔽。两个指数不同的浮点数相加时，较小数的尾数要右移对齐；右移时丢掉的位会进入 guard、round、sticky 三个附加位——guard 是被移出的最高一位，round 是次一位，sticky 是其余被移出位按 OR 合并出的“是否还有 1”，舍入电路用这三个附加位决定末位该不该进 1。尾数相加后可能需要左规或右规；最后按舍入模式提交。如果读者只把浮点看成“整数加法多一个小数点”，就无法解释为什么 $(a+b)+c$ 和 $a+(b+c)$ 可能不同，也无法解释 NaN、Inf、非规格化数和异常标志。

专题：规格化左移右移的代价 浮点加减法把尾数对齐并相加之后，工作还没有结束：和或差不一定仍保持规格化形式 $1.f \times 2^e$ 。尾数相加可能产生向整数位的进位，例如 $1.110 + 1.011 = 11.001$ ，结果大于等于 2；尾数相减可能让前导 1 消失，例如 $1.001 - 1.000 = 0.001$ ，整数位变成 0。两种情况都必须把结果移回“小数点前恰好一个 1”的形式，并同步调整指数，这一步叫规格化。前一种情形右移一位、指数加一，称为右规；后一种情形左移若干位、指数减去相应位数，称为左规。

两种规格化对应两条不同的硬件路径。右规的情形简单：进位只可能多出一位，硬件只需要一条固定右移一位的通路，把多出的进位放回最高位、把末位挤进 guard 位，同时给指数加一。左规的情形麻烦得多：相减造成的抵消可能抹掉任意多个前导位，最坏情况下（两个几乎相等的数相减）需要左移几十位。因此左规路径必须先在尾数上做前导零计数——用优先级编码器找出第一个 1 的位置 k ——再按 k 左移，并从指数中减去 k 。这条“先探测移位量、再移位”的链是浮点加法器特有的组合逻辑，

定点 ALU 里并不存在。

移位量不固定正是需要桶形移位器的原因。若用逐位移位寄存器，左规最坏要花费与尾数位宽相等的周期数，单精度是 24 位、双精度是 53 位，而且延迟随数据变化，流水级无法按固定节拍安排。桶形移位器用 $\log_2 n$ 层 mux 按二进制分解移位量：24 位尾数用移 1、2、4、8、16 五层，每层由移位量的一个 bit 控制，任意 0 到 23 位的左移都在一轮组合逻辑内完成，延迟固定且可预测。代价是面积和布线：每层都要把每个输出位接到多个候选输入之一， n 位 $\log_2 n$ 层共需约 $n \log_2 n$ 个二选一开关；对阶右移和左规两条路径还各占一套。

规格化还反过来影响舍入。右规把一位挤进 guard，左规把 guard、round、sticky 里的信息带回尾数低位，因此对阶时算好的舍入依据在规格化之后必须重新评估，舍入电路要拿规格化之后的三个附加位再做一次判断。把对阶移位、尾数相加、前导零计数、规格化移位和舍入全部塞进一个周期，关键路径会相当长；真实浮点加法器把它们分到不同流水级，这正是上一个专题说浮点运算“常常多周期”的具体原因。

专题：ALU 测试向量要覆盖边界值 ALU 的测试最怕只使用随机正常值。随机测试当然有用，但边界值才会逼出进位、溢出、符号扩展、移位宽度和比较解释错误。一张合格的测试向量表至少应覆盖零、最大正数、最小负数、全 1、单 bit、交替 bit、进位跨字节、符号位翻转和移位距离边界。

向量类型	示例	主要检查点
零值	0+0, 0-0	Zero 标志、无额外进位、写回不遗漏
无符号进位	0xFF+0x01	结果回绕和 CarryOut 同时正确
有符号溢出	0x7F+0x01	Sign 为 1 不等于“合法负数”，Overflow 必须置位
最小负数	0x80-0x01, -128/-1	补码非对称范围和除法溢出
比较混用	0xFF < 0x01	无符号为假，有符号为真
移位边界	1<<0, 1<<7, 1<<8	移位距离截断或非法规则必须明确
多字进位	0x00FF+0x0001	低字节进位进入高字节

调试时建议把每个向量写成五列：输入、期望结果、期望标志、经过的 ALU 路径、提交时钟沿。这样才能区分“组合逻辑算错”“标志解释错”“控制线选错路径”和“写回时机错”。这也是后续 CPU trace 的基础。

章末练习

本节练习要求把 bit 解释、硬件路径和控制信号联系起来思考。答案不应仅写算术结果，还要说明该结果来自哪条组合路径，在哪个时钟边界被保存，哪些控制线必须同时成立。

任务	要完成的产物	完成要求
补码解释	分别按无符号和补码解释 1000、1111、0111+0001	能区分结果 bit 与解释规则
位宽扩展	说明 1011 从 4-bit 扩展到 8-bit 时零扩展和符号扩展的差异	能说明扩展规则属于硬件路径
二进制小数	计算 10.101_2 的十进制值，并说明十进制 0.1 为什么不能有限二进制表示	能把小数位权和固定位宽联系起来
定点 Q 格式	解释 00011000 在 Q4.4 中的值，并说明两个相同 Q 格式数相加为何可复用整数加法器	能说明小数点位置是格式约定
有符号定点	解释 11110000 在有符号 Q4.4 中的值，并指出符号扩展是否仍然必要	能先按补码解释，再应用缩放因子
定点乘法	用 Q4.4 计算 1.5×0.5 的原始整数过程，说明为什么结果要右移 4 位	能说明乘法后缩放因子变化

BCD 表示	写出十进制 59 的 8421 BCD 表示, 并说明 1010 为什么不是合法 BCD 数字	能区分普通二进制和十进制编码
BCD 校正	解释 BCD 中 $8+7$ 为什么需要加 0110 校正	能说明辅助进位和十进制调整的硬件意义
浮点字段	说明浮点数的符号位、指数字段、有效数字字段分别解决什么问题	能区分范围、精度和符号处理
浮点特殊值	说明零、非规格化数、Inf、NaN 为什么需要保留编码	能说明浮点不是普通整数拼接
浮点舍入	说明“大数加小数”为什么可能舍入回大数, 以及这对结合律有什么影响	能把对阶、位宽和舍入联系起来
格式选型	在 MCU 传感器滤波、金融十进制显示、科学计算三种场景中选择整数/定点/BCD/浮点方案	能根据芯片约束和数值约定选型
减法复用	写出 $A-B$ 如何由加法器、XOR 反相和最低位进位输入完成	控制信号与数据路径一致
标志位	为一次加法和一次减法说明 Carry、Overflow、Zero、Negative 的来源	不把无符号进位和有符号溢出混用
进位路径	比较串行进位、超前进位和前缀加法的速度、面积、布线取舍	能指出关键路径来自哪里
桶形移位	说明 8-bit 桶形移位器怎样用 1/2/4 位阶段完成任意移位	能解释 mux 层数和延迟代价
比较器	用 $A-B$ 说明相等、无符号小于、有符号小于分别依据哪些标志	比较类型必须绑定解释规则
乘法器	比较组合阵列乘法和移位加法多周期乘法	能说明面积、延迟和周期数取舍
分立 ALU	列出用 74HC283、74HC86、74HC157 搭 4-bit 加/减/选择电路的连接要点	每个控制信号有可观察输出
74181 级联	说明两片 74181 如何组成 8-bit ALU, 以及片间进位如何连接	能区分操作数、函数选择、进位输入和进位输出
寄存器堆	说明读地址、写地址、写数据、写使能如何配合完成 $R3=R1+R2$	写回发生在时钟沿
4 寄存器实验	用 74HC173/574、74HC138、mux 设计 4 个 4-bit 寄存器的读写方案	同一周期只写一个目标, A/B 两个读端口可独立选择
总线冲突	解释为什么 ALU 输出和存储器输出不能随意直接并联	能说明 mux 或三态使能的必要性
控制器	比较硬连线控制、状态机控制和微码控制	能说明适用场景和风险
控制 ROM	给出一个控制 ROM 地址格式和控制字段, 说明 ADD 与条件分支各看哪些输入	每个控制字位都能追到实际控制线
控制 trace	为三周期 $R3=R1+R2$ 列出 S0/S1/S2 的打开路径和提交状态	能定位每个周期的写使能
最小 CPU 接口	列出 PC、IR、寄存器堆、ALU、存储器接口和控制器之间的关键输入输出	能说明哪些是组合路径, 哪些在时钟沿提交
错误定位	给出 alu_op、wb_sel、reg_write 错误各自导致的现象	能按 trace 排查, 而不是盲改 ALU

补码溢出判定	对 8-bit 运算 $01111111+00000001$ 与 $11111111+00000001$ 分别判定 Carry 和 Overflow	进位与溢出是两个独立结论，不能互相替代
浮点加法四步	按对阶、相加、规格化、舍入计算 $1.101_2 \times 2^3 + 1.101_2 \times 2^3$ ，尾数保留三位小数	能说明每步做什么、指数何时变化
CLA 展开	用 G_i 、 P_i 把 C_2 展开成只含输入位与 C_0 的表达式	能说明展开式为什么比逐级等待进位快
Booth 编码	对乘数 0110 做 Booth 重编码，写出对应的部分积组合	能说明连续 1 串如何变成一加一减
除法演算	用 4-bit 恢复余数法演算 $13 \div 3$ ，逐拍列出 (R,Q)、试减与商位	每拍的移位、试减、判负、恢复完整
微程序读法	说明由 opcode、step 和标志拼成的地址如何读出控制字并驱动控制线	能把查表动作和实际电线对应起来

参考解答与答题要点

任务	参考解答	必须答到的要点
补码解释	1000 无符号为 8，4-bit 补码为 -8； 1111 无符号为 15，补码为 -1； $0111+0001=1000$ 无符号为 8，补码中是 7+1 得到 -8，发生有符号溢出。	同一 bit 模式可有不同解释，硬件只给出结果和标志。
位宽扩展	1011 零扩展为 00001011 ，按无符号仍为 11；符号扩展为 11111011 ，按 8-bit 补码为 -5。若把补码负数误做零扩展，数值语义会改变。	扩展规则要由指令语义和控制线决定。
二进制小数	$10.101_2 = 2 + 1/2 + 1/8 = 2.625$ 。十进制 0.1 不能由有限个 $1/2, 1/4, 1/8, \dots$ 权重精确相加得到，因此固定位宽二进制格式只能保存近似值。	小数点不是新电线，而是位权解释规则。
定点 Q 格式	00011000 的原始整数为 24；Q4.4 的小数位数为 4，所以实际值为 $24/16 = 1.5$ 。两个相同 Q 格式数相加时缩放因子一致，原始整数可直接送入加法器，结果仍按同一缩放因子解释。	定点加减复用整数 ALU，但格式必须一致。
有符号定点	11110000 按 8-bit 补码为 -16；在 Q4.4 中实际值为 $-16/16 = -1.0$ 。扩展到更宽路径时仍需符号扩展，否则负值会变成正的大数。	先补码解释，再除以缩放因子。
定点乘法	Q4.4 中 1.5 的原始值为 24，0.5 的原始值为 8，乘积原始值为 192。两个 Q4.4 相乘后缩放因子为 2^8 ，要回到 Q4.4 需右移 4 位得到 12，解释为 $12/16 = 0.75$ 。	乘法后必须重缩放，并决定截断、舍入或饱和。
BCD 表示	十进制 59 的 8421 BCD 为 $0101\ 1001$ ，每个半字节表示一个十进制位。 1010 到 1111 不对应 0 到 9，属于非法 BCD 数字。	BCD 是十进制数字编码，不是紧凑二进制整数。

BCD 校正	8+7 的低半字节二进制和为 1111，不是合法 BCD。加 0110 后得到 1 0101，表示十进制进位 1 和低位数字 5。	十进制校正需要额外检测和加法路径。
浮点字段	符号位决定正负；指数字段决定尺度和动态范围；有效数字字段决定保留多少精度。规格化数通常解释为 $(-1)^s \times 1.f \times 2^{e-bias}$ 。	浮点把范围和精度分开管理。
浮点特殊值	零需要正负零编码；非规格化数让接近 0 的下溢更平滑；Inf 表示超出范围或特定除零结果；NaN 表示无效运算并可传播异常状态。	特殊编码必须由 FPU、比较器和转换路径共同遵守。
浮点舍入	浮点加法要先对阶，小数的有效数字可能在右移时被移出保留位宽之外，所以大数加小数可能舍入回大数。不同括号顺序会改变中间舍入位置，因此浮点加法通常不满足严格结合律。	对阶、规格化和舍入是硬件路径，不是显示问题。
格式选型	MCU 传感器滤波常选整数或定点，便于控制延迟、功耗和饱和；金融十进制显示可在边界使用 BCD 或十进制格式，避免十进制小数展示误差；科学计算通常使用浮点或向量浮点，换取大动态范围和通用数学库支持。	数值格式要跟芯片资源、实时性、精度约定和输入输出形式绑定。
减法复用	$A-B = A+(\text{NOT } B)+1$ 。每个 B 位前接 XOR，sub=1 时反相；最低位 Cin=sub；sub=0 时执行普通加法。	sub 必须同时控制 B 反相和最低位进位。
标志位	Zero 来自结果各位 OR 后取反；Negative 来自最高位；Carry 来自最高位外进位或借位约定；Overflow 来自补码符号变化规则。	标志位要绑定运算类型和解释方式。
进位路径	串行进位面积小但最坏延迟随位宽增长；超前或前缀结构用更多门和布线提前计算进位，缩短关键路径；前缀结构适合宽位高性能路径但布局压力更大。	速度换来面积、功耗和布局复杂度。
桶形移位	8-bit 移位量三位可控制三层 mux：按需移 1、2、4 位，组合后得到 0 到 7 位移位。逻辑右移补 0，算术右移补符号位。	一周周期多位移位会增加 mux 延迟和连线负载。
比较器	相等看 A-B 的 Zero；无符号小于看借位或 Carry 约定；有符号小于要结合结果符号和 Overflow，例如常见组合为 Sign xor Overflow。	不能用同一标志解释所有比较。
乘法器	阵列乘法展开部分积并求和，速度和吞吐较好但面积大；移位加法每周周期处理一部分，面积小但周期数多；流水化乘法器提高吞吐但增加控制边界。	乘法仍由部分积、对齐、求和和保存构成。
分立 ALU	A/B 来自拨码开关或寄存器；B 先经 74HC86 与 sub XOR；74HC283 完成加/减；逻辑候选由门阵列产生；74HC157 选择最终输出；LED 显示结果和标志。	控制线、候选结果和最终输出要分开观察。

74181 级联	低 4 位 74181 接 bit0 到 bit3, 高 4 位 74181 接 bit4 到 bit7; 两片共享函数选择控制; 低片 C_{n+4} 接高片 C_n 可形成串行跨片进位, 若追求更短延迟可加入超前进位器。	级联后位宽扩大, 但片间进位会占用时序预算。
寄存器堆	<code>read_a=R1, read_b=R2, a_sel=reg_a, b_sel=reg_b, alu_op=ADD, wb_sel=ALU, write_addr=R3, reg_write=1</code> 。若采用多周期, 还需 A/B 和 ALUOut 写使能。	写回必须等 ALU 输出稳定并在时钟沿提交。
4 寄存器实验	四个寄存器保存 R0 到 R3; 写地址经译码产生四路候选写使能, 再与 <code>reg_write</code> 相与; 读端口 A/B 分别用 mux 按读地址选择一路输出; 写回数据并接到所有寄存器 D 端但只有被选寄存器写入。	读是组合选择, 写是时钟沿提交。
总线冲突	两个输出若同时驱动同一线且电平不同, 会形成冲突和大电流。应使用 mux 明确选择一路, 或用三态总线并保证同一时刻只有一个输出使能有效。	共享连线必须有唯一驱动器。
控制器	硬连线控制适合简单高频路径; 状态机适合多周期阶段; 微码适合复杂指令分解; 混合方案让常见路径快、复杂路径可管理。	所有方案最终都要输出具体控制线。
控制 ROM	地址可由 <code>opcode</code> 、微步骤 <code>step</code> 、标志位和复位/中断请求组成; 数据输出可包含 <code>alu_op</code> 、 <code>a_sel</code> 、 <code>b_sel</code> 、 <code>wb_sel</code> 、 <code>reg_write</code> 、 <code>flags_write</code> 、 <code>pc_write</code> 、 <code>ir_write</code> 、 <code>mem_read/write</code> 、 <code>next_sel</code> 。ADD 看 <code>opcode</code> 与 <code>step</code> , 条件分支还要看 Zero 等标志。	ROM 每个输出位都必须连接到一个真实控制对象。
控制 trace	S0 打开 R1/R2 到 A/B 的路径并提交 A/B; S1 打开 A/B 到 ALU 的路径并提交 ALUOut/Flags; S2 选择 ALUOut 写回 R3 并打开 <code>reg_write</code> 。	每周期要写清打开路径、提交状态和关闭信号。
最小 CPU 接口	PC 输出取指地址并由 <code>pc_write/pc_sel</code> 更新; IR 锁存指令并输出 <code>opcode</code> /寄存器字段; 寄存器堆输出操作数并接收写回; ALU 产生结果和标志; 存储器接口处理地址、读写数据和等待; 控制器输入 <code>opcode</code> 、状态、标志和等待信号并输出控制字。	PC、IR、寄存器堆和标志是状态元件, 必须有写使能。
错误定位	<code>alu_op</code> 错会算错函数; <code>wb_sel</code> 错会写回错误来源; <code>reg_write</code> 错会漏写或误写寄存器。应按控制 trace 检查状态、选择线和写使能。	先区分计算错误、选择错误和提交错误。

补码溢出判定	$01111111+00000001=10000000$: 最高位外无进位, Carry=0; 补码读作 $127+1$ 得 -128, 正加正得负, Overflow=1。 $11111111+00000001=00000000$: 最高位外有进位, Carry=1; 补码读作 $-1+1=0$, 结果合法, Overflow=0。两例各占一种组合, 说明两个标志来源不同、结论独立。	进位看最高位外, 溢出看符号异常, 各管无符号与补码边界。
浮点加法四步	对阶: 两数指数相同, 无需移位。相加: $1.101+1.101=11.010$。规格化: 和不少于 2, 右规一位得 1.1010, 指数加一变为 2^4。 舍入: 保留三位小数, 移出位为 0, 直接舍去, 结果 1.101×2^4, 即十进制 26。	四步顺序固定, 右规后指数必须同步加一。
CLA 展开	$C_1 = G_0 + P_0C_0$; 代入得 $C_2 = G_1 + P_1C_1 = G_1 + P_1G_0 + P_1P_0C_0$。展开式只含各位输入和最初进位, 经两级与或门即可算出, 不必等 C_1 稳定。	用额外门数换进位延迟与位宽脱钩。
Booth 编码	乘数 0110 从低位起看相邻两位, 最低位右侧补 0: $(0,0) \rightarrow 0$, $(1,0) \rightarrow -1$, $(1,1) \rightarrow 0$, $(0,1) \rightarrow +1$。即第 1 位处减一次被乘数、第 3 位处加一次被乘数: $8M - 2M = 6M$。连续的两位 1 被编码成串尾之后 -1、串首 $+1$, 部分积由逐位相加变成一加减组合。	部分积数量由 1 的个数变成 1 串的个数。
除法演算	$13 = 1101$, 除数 $3 = 0011$, R 初值 0000。 拍 1: (R,Q) 左移为 $(0001,1010)$, $1-3 < 0$, 商 0 并恢复 R; 拍 2: 左移为 $(0011,0100)$, $3-3=0$, 商 1, 得 $R=0000$、$Q=0101$; 拍 3: 左移为 $(0000,1010)$, $0-3 < 0$, 商 0 并恢复; 拍 4: 左移为 $(0001,0100)$, $1-3 < 0$, 商 0 并恢复。最终 $Q=0100=4$, $R=0001=1$, 验算 $3 \times 4 + 1 = 13$。	商逐位产生, 每拍都是移位、试减、判负、条件恢复。
微程序读法	把 opcode、微步骤 step 和要检查的标志拼成 ROM 地址; 该地址读出的数据行就是本周控制字, 各字段直接接 alu_op、a_sel、wb_sel、reg_write、$next_sel$ 等控制线。例如条件分支在 EXEC 步且 $Z=1$ 时, 控制字让 $next_sel$ 选择分支目标而非顺序状态。	微指令不是被执行的软件语句, 而是被查出的电平表。

本章的经典读法是 trace: 从 bit 解释, 到 ALU 候选计算, 到数据通路值流, 再到控制信号时间表。

- 位级机制: 补码取负可写成按位取反再加一; 减法可复用加法器执行“B 取反、最低位进位为 1”。
- 标志规则: Carry 适合无符号边界, Overflow 适合补码有符号边界; 不能把两者混成同一个“溢出”。
- ALU 层面: 候选计算并行产生, 选择器选出一个结果, 标志位保存结果的特征。

- 数据通路层面：为 $R3 = R1 + R2$ 写读地址、ALU 输入选择、`alu_op`、写回选择、写地址、写使能。
- 控制层面：每个周期哪些写使能为 1，哪些 mux 选择哪一路，ALU 做什么，都要能列成 trace。
- 检查题：若 ALU 结果已经正确，但写回 mux 选择线接错，寄存器结果会怎样？参考答案：寄存器会保存被错误选择的来源，例如旧寄存器值、内存数据或外部输入，而不是 ALU 结果。若示波器、LED 或仿真已经证明 ALU 输出正确，排查重点应转向 `wb_sel`、写回 mux、写地址、`reg_write` 和控制器状态，而不是继续修改 ALU。

经典教材会把控制器和数据通路并排讲：数据通路说明“能走哪些路”，控制器说明“本周期允许哪条路”。两者合起来，才是可执行动作。

最小 CPU 与整机硬件

本部分导读

前面已经有了计算、状态、选择和控制。本部分先把这些部件合成一台可走读 trace 的最小 CPU，再把 CPU 接到寄存器、cache、TLB、DRAM、SSD/硬盘、MMIO、PCIe 设备、中断控制器和 DMA，最后通过 8086、80386、80486/Pentium、PC 主板、显卡和现代 SoC 观察真实硬件如何把这些边界逐步扩展到整机平台。

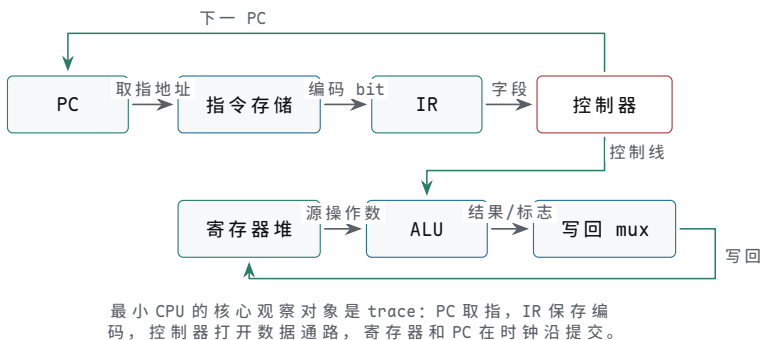
最小 CPU 与控制 trace

前面已经具备构造处理器的必要部件：组合逻辑、状态元件、时钟边界、ALU、数据通路和控制器。本章只做一件事：把这些部件合成为一台能够取指、译码、执行、写回和更新 PC 的最小 CPU。主存、总线、设备和经典芯片会在后续章节展开，本章先把 CPU 内部的状态边界和控制 trace（执行轨迹）写清楚。

CPU 可以视为一组同步硬件闭环：PC 指出下一条编码地址，取指路径取回指令 bit，译码逻辑生成控制信号，数据通路读取和计算，状态元件在时钟边界提交结果。只要能为一条指令写出 PC、IR、寄存器堆、ALU、写回 mux、标志位和控制器的完整 trace，就已经能判断这台最小 CPU 是否真的会执行程序。

核心思想

最小 CPU 不是“ALU 加寄存器”的粗线图，而是一组按周期提交的状态转换。每条指令都必须说明：本周期读哪些旧状态，组合路径经过哪些部件，哪些控制线有效，时钟沿会提交哪些新状态，异常或等待是否会阻止提交。



图示：最小 CPU 总图。箭头表示一条指令从取指到写回的完整路径，反馈线表示 PC 和寄存器堆在时钟边界更新。

一、从动作编码到控制字组织

若希望同一套数据通路在不同时间做不同动作，就需要把“这一次做什么”也表示成 bit。动作编码可以先写成最小形式：

```
opcode | dst | src_a | src_b
```

`opcode` 表示动作类型，例如 ADD、SUB、AND；`dst` 表示写入哪个寄存器；`src_a` 和 `src_b` 表示读取哪两个寄存器。控制器读取这些字段后，把它们译成 ALU op、读地址、写地址、写使能和写回选择。编码本身仍然是硬件输入，不是文字命令。

若有一组动作编码连续放在存储结构中，硬件还需要知道当前取哪一条。这个位置由 PC 保存。PC 本质上也是寄存器，只是它保存的是“下一条动作编码的地址”。

部件	作用
PC	保存下一条编码地址
指令存储器	根据 PC 输出当前编码
控制器	根据编码字段产生控制信号
寄存器阵列	保存通用数据
ALU	做整数和逻辑运算

选择器网络

选择 ALU 输入、写回来源和下一 PC

顺序执行时，下一 PC 可以由加法器产生：

```
pc_next = pc_current + instruction_size
```

如果每条编码占 4 个字节，`instruction_size` 就是 4；如果是可变长度指令，下一 PC 的计算会更复杂。关键事实是：顺序执行不是抽象的“下一行”，而是 PC 这个状态寄存器保存了新的地址。

最小单周期模型可以这样走：

阶段	硬件动作	边界
取指	PC 送入指令存储器，得到编码	组合路径
译码	控制器产生 mux、ALU、写使能信号	组合路径
执行	寄存器阵列读值，ALU 计算	组合路径
写回	结果在时钟沿写入寄存器或 PC	状态边界

用一条普通运算编码走读：当前位置 PC=100，指令存储器在地址 100 处保存的是：

```
ADD R1, R2 // R1 = R1 + R2
```

周期开始时，PC 输出稳定为 100。这个地址进入指令存储器，存储器经过地址译码后把这条编码输出。控制器看到 `opcode=ADD`，于是把 `alu_op` 设为 ADD，把寄存器阵列的两个读地址分别设为 R1 和 R2，把写地址设为 R1，把 `wb_sel` 设为 ALU，把 `reg_write` 设为 1，同时让下一 PC 选择 PC 加指令长度。

这些控制信号稳定后，寄存器阵列的两个读端口输出 R1 和 R2 当前的值。它们经过 ALU 输入选择器进入 ALU，加法路径产生结果。结果再经过写回选择器，到达寄存器阵列的写数据端口。另一边，PC 加法器准备下一条指令地址，PC 选择器把它送到 PC 的下一值输入。只有到时钟沿，两个状态变化才同时提交：R1 保存 ALU 结果，PC 保存下一条地址。

把这次走读填上具体数值，抽象的路径描述就变成了可逐项核对的数值关系。设 R1=5、R2=7、指令长 2 字节：

时刻	路径上的值	核对要点
周期开始	PC=100	取指地址稳定
取指后	IR = ADD R1,R2	编码来自 MEM[100]
译码后	alu_op=ADD，读地址 R1/R2，控制线与 opcode 一致 写地址 R1，wb_sel=ALU， reg_write=1	
读寄存器	read_a=5，read_b=7	寄存器堆当前值
执行后	ALU_out=12，写回 mux 输出 12	加法路径和写回选择正确
时钟沿	R1 <- 12，PC <- 102	两个状态同时提交，R2 不变

任何一格对不上，故障范围就锁死了：`read_a` 不是 5，查寄存器堆读地址；`ALU_out` 不是 12，查 `alu_op` 和进位链；12 没到写数据口，查 `wb_sel`；R1 没变或 PC 没变，查 `reg_write`、`pc_write` 和时钟沿本身。这就是 trace 比“检查一下”可靠的地方。

分支也可以归结为下一 PC 选择。顺序情况下，下一 PC 是 PC 加指令长度；条件分支则由 ALU 比较结果和选择器决定下一 PC 来源：

```
branch_taken = condition_met AND branch_op
next_pc = mux(branch_taken, pc_sequential, branch_target)
```

分支的硬件本质是改变 PC 这个寄存器的下一值。下一周期取到哪条编码，完全由这个新 PC 决定。

把指令周期写成逐阶段的执行过程 经典教材讲 CPU 时，常把一条指令拆成取指、译码、执行、访存、写回和下一 PC 更新。这个拆分不是为了背阶段名称，而是为了把每个阶段对应到真实硬件路径。最小单周期 CPU 可以在一个长周期内完成所有路径；多周期 CPU 可以把这些路径分到多个时钟；流水线 CPU 可以让多条指令在不同阶段重叠推进。三者的共同点是：每个阶段都要说明源状态、组合路径、目标状态和异常边界。

阶段	主要硬件动作	关键控制信号	提交边界
IF 取指	PC 送到指令存储器或 I-cache，取回指令 bit	pc_out、imem_read、ir_write	IR 或取指缓冲
ID 译码	opcode、寄存器号、立即数字段被解释成控制线	decode、imm_kind、read_addr	可组合，也可锁存
EX 执行	ALU 做加减、逻辑、比较或地址形成	alu_op、a_sel、b_sel	输出和标志
MEM 访存	对数据存储器、cache 或 MMIO 发起读写事务	mem_read、mem_write、byte_en	MDR 或外部响应
WB 写回	ALU 结果或读回数据写入寄存器堆	wb_sel、reg_write	目标寄存器
PC 更新	选择顺序 PC、分支目标、跳转目标或异常入口	pc_sel、pc_write	PC

这张表也可以直接用作调试时的排查清单。若某条加载指令读到了错误数据，不应只问“内存为什么错”，而要逐项检查：PC 是否取到正确编码，译码是否识别为加载，ALU 是否形成正确地址，MEM 阶段是否访问正确目标，读回数据是否进入 MDR (memory data register，存储器数据寄存器—访存返回数据在进入写回路径之前的暂存，写回 mux 的“存储器数据”候选就是从它引出)，WB 阶段是否选择了 MDR 而不是 ALUOut。这样的排查方式比凭感觉改控制器可靠得多。

控制字把“会执行指令”变成可接线信号 教学 CPU 最容易写成口号：取指、译码、执行、写回。真正能搭出来的版本必须有控制字。控制字不是高级软件结构，而是一组在某个周期稳定的控制线集合。它决定寄存器是否写入、ALU 选哪个操作、存储器是否读写、PC 从哪里取下一值。若控制字没有定义清楚，CPU 即使图上连通，也无法稳定执行程序。

控制对象	ADD	MOV（加载）	MOV（存储）	JZ
ALU 操作	加法或指定算术	基址加偏移	基址加偏移	比较或测试零
寄存器读	两个源寄存器	基址寄存器	基址和数据寄存器	条件源寄存器
存储器读	无	有	无	无
存储器写	无	无	有	无
写回选择	ALU 结果	存储器数据	无	无
PC 选择	顺序 PC	顺序 PC	顺序 PC	条件目标或顺序 PC

若把这些控制对象编码成 bit，可以得到类似下面的教学控制字。具体 bit 位不重要，重要的是每一位都能对应到实际连线或选择器输入。

```
control word fields:
  pc_sel      // sequential, branch_target, jump_target, exception
```



```
reg_write // enable register-file write port
wb_sel    // ALU, memory, pc_plus_size
alu_op    // add, sub, and, or, compare
alu_b_sel // register or immediate
mem_read  // request data read
mem_write // request data write
byte_en   // selected byte lanes
flags_write // update Zero/Carry/Overflow/Sign
```

把控制字应用到这台具体 ISA 上，每条指令一行，全部 7 条覆盖：

指令	alu_op	a_sel	b_sel	wb_sel	reg_w	mem_r	mem_w	pc_sel
ADD	ADD	寄存器	寄存器	ALU	1	0	0	顺序
MOV (立即数)	直送	立即数	—	ALU	1	0	0	顺序
MOV (加载)	ADD	寄存器	off4 扩展	存储器	1	1	0	顺序
MOV (存储)	ADD	寄存器	off4 扩展	—	0	0	1	顺序
JZ	判零	寄存器	—	—	0	0	0	Z? 目标：顺序
JMP	—	—	—	—	0	0	0	目标
SHL	左移	寄存器	imm4	ALU	1	0	0	顺序

这张表还能进一步写成布尔式。以 `reg_write` 为例，它只在四条指令下为 1——`ADD`、立即数 `MOV`、加载 `MOV` 和 `SHL`；多条 `MOV` 在布尔式中用 `opcode` 值区分：

```
reg_write = (op == ADD) OR (op == 0010) OR (op == 0011) OR (op == SHL)
mem_read  = (op == 0011) // MOV load
mem_write = (op == 0100) // MOV store
pc_sel     = (op == JMP) OR ((op == JZ) AND Zero) ? target : sequential
```

译码器到此就没有黑箱了：输入 `opcode` 的 4 个 bit，输出这张表里的每一格，且每一格都能在这张真值表和数据通路图之间互相指认。若某条指令行为异常，先在这张表上查控制字哪一格错了，再回数据通路图查那条线——排查顺序永远是先控制字、后路径。

硬连线控制与微程序控制是两种组织控制字的方法 控制器可以直接由 `opcode`、状态位和当前周期译码出控制线，这称为硬连线控制。它速度快，结构适合简单指令集和高速流水线，但修改指令行为需要改组合逻辑。另一种方法是把控制字存在控制存储器中，由微程序计数器逐条取出微指令；每条机器指令对应一段微操作序列。这种方法更像“硬件内部的小程序”，适合组织复杂指令、多周期总线访问和异常入口，但控制存储器和微序列会增加延迟。

控制方式	适合场景	关键问题
硬连线控制	指令格式规整、周期少、目标频率高	<code>opcode</code> 、状态位和时序状态是否唯一决定控制线
微程序控制	指令复杂、多周期动作多、需要复用数据通路	微地址如何选择、分支微指令如何跳转、异常如何打断
混合方式	常见简单指令走快路径，复杂指令走微序列	快路径和微序列是否共享一致的提交边界

```

microinstruction sketch:
alu_op | src_a | src_b | dst | mem_cmd | pc_cmd | next_micro_pc

load sequence:
u0: IR <- MEM[PC],      next = u1
u1: A <- R[base],       next = u2
u2: ADR <- A + imm,     next = u3
u3: MDR <- MEM[ADR],    next = u4 or wait
u4: R[dst] <- MDR, PC <- PC+4, next = fetch

```

微程序写法有一个重要好处：它迫使作者说明每个周期保存什么中间状态。若某一步需要等待外部存储器，`next_micro_pc` 可以停留在当前微地址；若访问失败，微序列可以跳到异常入口。这样，等待、错误和普通写回不再混在一句“执行加载”里。

多周期 CPU 用微操作缩短最长组合路径 单周期 CPU 概念清楚，但周期必须长到容纳取指、译码、ALU、访存和写回的最坏组合路径。多周期 CPU 把同一条指令拆成若干微操作，让每个周期只完成一小段界限清楚的动作。例如加载指令可以拆为：PC 取指、译码读寄存器、ALU 形成地址、存储器读、写回寄存器。每一步之间用 IR、MDR、ALUOut 或临时寄存器保存中间结果。

周期	微操作	打开的硬件路径	提交对象
T0	<code>IR = MEM[PC]</code>	PC 到指令存储器，读响应到 IR	IR
T1	<code>A = R[rs], B = R[rt]</code>	指令字段到寄存器堆读地址	A/B 临时寄存器
T2	<code>ALUOut = A + imm</code>	ALU 做地址形成或比较	ALUOut/Flags
T3	<code>MDR = MEM[ALUOut]</code>	地址到数据存储器，读响应到 MDR	MDR
T4	<code>R[rd] = MDR, PC = PC+4</code>	写回 mux 和 PC 加法器	寄存器、PC

这个 trace 说明，多周期设计不是“慢版单周期”，而是把过长的硬件路径切成可计时、可观察、可插入等待的边界。若数据存储器慢，可以只让 T3 等待；若分支比较已经在 T2 得到结果，可以在 T2 或 T3 改写 PC；若发生异常，可以把 PC 选择器改到异常入口，而不是继续提交错误写回。

微操作记法把每条指令写成可核对的寄存器传输序列 前面的微指令草图和多周期 trace 已经用过 `IR = MEM[PC]` 这类写法，现在把这套记法定死，称为寄存器传输级记法。核心约定只有三条：第一，箭头左侧是时钟沿要提交的状态元件，右侧读取的全部是本周期的旧值；第二，写在同一行里的多个传输在同一个时钟沿同时提交，行内没有先后；第三，行与行之间才是时间边界——多周期 CPU 里一行对应一拍，单周期 CPU 里整条指令写进一行，表示这些传输在同一拍内相继稳定、同一沿提交。于是 `R[d] <- R[d]+R[b]` 读作“时钟沿到来时，d 获得 d 与 b 旧值之和”，`MEM[R[a]+off] <- R[s]` 读作“时钟沿把 s 的旧值写入地址 R[a]+off 处”。

为什么值得为一层记法专门立约定：自然语言的“先取指、再执行”说不清哪些动作并发、哪些动作分拍，而硬件恰恰在这一点上不允许含糊。把指令写成传输序列之后，单周期与多周期之争就变成同一序列的两种排布——是一行写完，还是拆成五行、行间插入 IR、MDR、ALUOut 这些暂存元件——两种写法描述的是同一台机器。

写法	含义	核对要点
<code>R[x]</code>	寄存器堆中编号 x 的当前值	读到的是旧值
<code>MEM[a]</code>	地址 a 处的存储字	地址须先形成
<code>A <- B</code>	时钟沿把 B 提交给 A	右侧全是旧值

同一行多个传输	同一时钟沿同时提交	行内无先后
cond ? x : y	条件成立取 x，否则取 y	条件须提前稳定

用这套记法，本章 ISA 的四条代表指令各写成一行。注意本章 ROM 按字编址，顺序执行的下一 PC 是 PC+1：

```
ADD Rd, Rs:      IR <- ROM[PC]; R[d] <- R[d]+R[b]; PC <- PC+1
MOV Rd, [Ra+off4]: IR <- ROM[PC]; R[d] <- MEM[R[a]+sign_ext(off4)]; PC <- PC+1
MOV [Ra+off4], Rs: IR <- ROM[PC]; MEM[R[a]+sign_ext(off4)] <- R[s]; PC <- PC+1
JZ Rr, +off:      IR <- ROM[PC]; PC <- (R[r]==0) ? PC+1+sign_ext(off8) : PC+1
```

每一行都能翻成数值核对。以 PC=2、R1=0xF000 执行 MOV R2, [R1+4] (ROM 字 0x3450)：本沿读旧值 R[a]=0xF000，地址形成 0xF000+4=0xF004，提交 R2 <- MEM[0xF004] 与 PC <- 3。以 PC=3、R2=0 执行 JZ R2, +3 (0x5403)：条件成立，提交 PC <- 3+1+3=7；若 R2=1，同一行记法给出 PC <- 4。多周期写法只是把这些传输拆到 T0-T4 五行、行间用暂存元件保存中间值，与上一张多周期 trace 表逐行对应；单周期写法则把同一行内的传输理解为一长组合路径上相继稳定的各段。

同一程序在三种组织下的拍数对照 单周期、多周期和流水线不是三台不同的机器，而是同一组硬件路径的三种时间安排。比较它们必须先固定前提：三种组织使用同一批部件—同一个指令存储、同一个寄存器堆、同一个 ALU、同一个数据存储；设每段组合路径（取指、译码、执行、访存、写回）延迟相同，记为 1 个时间单位，并忽略流水寄存器自身的建立延迟。在此前提下，单周期机的时钟周期必须容纳最坏指令（加载：五段串行），即 5 个单位；多周期和流水线的周期由最长一段决定，即 1 个单位。

考察下面这个 6 条指令的小程序（取自下一节 ROM 程序的前 6 条，设按键已按下，GPIO_IN 读出 1，因此 JZ 条件不成立、顺序执行）：

```
0: MOV R1, 0xF0      // R1 = 0x00F0
1: SHL R1, 8         // R1 = 0xF000 (MMIO base)
2: MOV R2, [R1+4]    // R2 = MEM[0xF004], GPIO_IN reads 1
3: JZ R2, +3         // R2 != 0, not taken
4: MOV R3, 1         // R3 = 1
5: MOV [R1+0], R3    // MEM[0xF000] = 1, LED on
```

单周期组织最直截：每条 1 拍，共 6 × 1 = 6 拍，每拍 5 个时间单位，总时间 6 × 5 = 30。

多周期组织按指令分流。公共的取指、译码各占 1 拍；立即数 MOV 与 SHL 再走执行、写回，各 4 拍；加载多一拍访存，共 5 拍；JZ 在执行拍完成判零并改写 PC，共 3 拍；存储在执行拍形成地址后再用一拍写入，共 4 拍。总拍数 4+4+5+3+4+4 = 24，总时间 24 个单位，平均每条 4 拍。

理想 5 级流水(取指、译码、执行、访存、写回各一级)先看无冒险情形：5+6-1 = 10 拍—前 4 拍是填充，此后稳态每拍完成 1 条。再逐对检查相关：SHL 读 R1 紧跟立即数 MOV，加载的基址又读 R1，这两对靠 ALU 出口到入口的转发解决，不停顿；JZ 读 R2，而 R2 来自上一条加载，数据要到访存拍末才就绪，这是 load-use 冒险，即使有转发也要停 1 拍；JZ 不跳且按“猜不跳”预取，猜中无冲刷—若 R2=0 真跳，已在取指、译码级的两条预取作废，要再加 2 拍。本场景总拍数 10+1 = 11，总时间 11 个单位。

组织	总拍数	折合时间	关键问题
单周期	6 拍	6 × 5 = 30	周期被最慢的加载路径拖长
多周期	24 拍	24 × 1 = 24	周期短，但每条仍要排满 3-5 拍，平均每条 4 拍

理想流水 11 拍 $11 \times 1 = 11$ 填充 4 拍、load-use 停 1 拍；
程序越长平均拍数越接近 1

三个数字 30、24、11 的含义要说准确：它们不是说流水线永远快两倍以上，而是说明吞吐率来自哪里。单周期把整条路径拉长成一拍，多周期把拍切短但每条指令仍要排满几拍，流水线则让多条指令同时占据不同段。真实机器里各段延迟并不相等、一次访存可能等几十拍、流水寄存器本身也有延迟，但只要比较的前提仍是同一组硬件路径，“切短周期、重叠执行”这个方向性结论就稳定成立。

二、极小 ISA 与可接线数据通路

先规定一个极小 ISA，程序才能进入 ROM。为了让整机实验真正运行，需要把“指令”编码成固定 bit。下面是一种 16-bit 教学编码：高 4 bit 是 opcode，中间字段选择寄存器，低位可解释为寄存器号、立即数或分支偏移。寄存器字段 3 bit，即 R0-R7，其中 R0 固定为 0。它不是任何真实 ISA 的复制品，但足以把编码、译码、控制字和 ROM 内容连接起来。

指令	编码格式	语义	需要的硬件路径
ADD R d, Rs	0001 d a b ---	$R[d]=R[d]+R[b]$	双读端口、ALU、写回
MOV R d, imm8	0010 d x imm8	$R[d]=\text{zero_extend}(\text{imm8})$	立即数扩展、写回
MOV R d, [Ra+o ff4]	0011 d a off4 --	$R[d]=\text{MEM}[R[a]+\text{off4}]$	地址形成、读存储、写回
MOV [Ra+o ff4], Rs	0100 s a off4 --	$\text{MEM}[R[a]+\text{off4}]=R[s]$	地址形成、写数据、字节使能
JZ	0101 r x off8	若 $R[r]=0$, $\text{PC}=\text{PC}+1+\text{sign_extend}(\text{off8})$	比较、符号扩展、PC 选择
JMP	0110 off12	$\text{PC}=\text{PC}+1+\text{sign_extend}(\text{off12})$	目标形成、PC 写入
SHL R d, imm4	0111 d a imm4 --	$R[d]=R[d]<<\text{imm4}$, 低位补 0	移位器、写回

两个语义细节必须先定死，后面的程序才跑得上。第一，跳转偏移都相对**下一条指令的地址**计算：跳转目标 = 本条地址 + 1 + 偏移，偏移按补码解释，可正可负。第二，立即数形式的 MOV 只能装 8 位立即数，高地址必须靠移位拼出来：MOV R1, 0xF0 得到 $R1=0x00F0$ ，再 SHL R1, 8 左移 8 位得到 $R1=0xF000$ 。SHL 的字段布局与 ADD 相同（d、a、低 4 位立即数），所以译码路径几乎免费。与 x86-64 的二操作数约定一致，ADD Rd, Rs 和 SHL Rd, imm4 的目的寄存器同时是源操作数，编码中 a 字段固定等于 d。另外，JZ R2, +3 把判零与跳转融合为一条指令，这是教学子集的简化；在 x86-64 中，同样的动作对应 test/cmp 与 jz 两条指令。

这个编码能直接写入 ROM。若 GPIO_IN=0xF004、GPIO_OUT=0xF000，程序先用 MOV+SHL 拼出 MMIO 基址 0xF000，再循环读取按键、条件跳转、写 LED。下面左栏是 ROM 中的真实 16-bit 字，右栏是对应汇编和语义：

地址	内容	汇编	语义
0	0x22F0	MOV R1, 0xF0	$R1 = 0x00F0$
1	0x7260	SHL R1, 8	$R1 = 0xF000$ (MMIO 基址)
2	0x3450	MOV R2, [R1+4]	$R2 = \text{MEM}[0xF004]$, 读 GPIO_IN
3	0x5403	JZ R2, +3	$R2==0$ 时跳到地址 7 (3+1+3)
4	0x2601	MOV R3, 1	$R3 = 1$

5	0x4640	MOV [R1+0], R3	MEM[0xF000] = 1, 点亮 LED
6	0x6FFB	JMP -5	跳回地址 2 (6 + 1 - 5)
7	0x2600	MOV R3, 0	R3 = 0
8	0x4640	MOV [R1+0], R3	MEM[0xF000] = 0, 熄灭 LED
9	0x6FF8	JMP -8	跳回地址 2 (9 + 1 - 8)

这个程序有三个可以直接核对的事实。第一，分支偏移逐项吻合：地址 3 的 `JZ +3` 跳到地址 7，地址 6 的 `JMP -5` 和地址 9 的 `JMP -8` 都跳回地址 2。第二，`0xF004` 这个地址在 ISA 里是可以形成的——不是注释里的愿望，而是 `0x00F0 << 8 + 4` 的真实计算。第三，ROM 里每个 16-bit 字都能被译码器逐字段读出：拿 `0x7260` 核对，高 4 位 `0111` 是 `SHL`，`d=001`、`a=001`、`imm4=1000`=8。

定长编码与变长编码的取舍 本章用的是定长编码：每条指令恰好占一个 16-bit 字，取指地址每次加 1，字段位置一成不变。代价直接可见——`ADD` 有 3 位空置，立即数 `MOV` 空 1 位且只能装 8 位立即数，`JMP` 的范围被 12 位偏移限制在 ± 2048 个字。把 ROM 程序十条的空位加总：3 条立即数 `MOV` 各空 1 位，`SHL` 与 2 条访存 `MOV` 各空 2 位，`JZ` 空 1 位，共 12 位，占全部 160 位的 7.5%。收益同样直接：译码器是对固定字段的纯组合查表，任何一条指令的边界无需计算，这为以后每拍同时取多条、译多条留了路。

x86-64 在另一极：指令长 1 到 15 字节，长度由前缀字节、opcode、ModRM 字段逐层决定——不先译出本条长度，就不知道下一条从哪个字节开始，边界本身构成串行依赖。变长换来编码密度：常用指令短，`push rbp` 仅 1 字节，`ret` 仅 1 字节；长格式只在需要时出现。一个函数序言 `push rbp; mov rbp, rsp; sub rsp, 32` 实际占 8 字节 (1 + 3 + 4)；若按最长格式定长 8 字节排布，同样三条要占 24 字节。

为什么 RISC 传统坚持定长、x86 坚持变长，原因都在各自约束里。RISC 的目标是把译码做窄做快：定长让字段位置固定、多条指令的边界天然已知，译码逻辑短，才有可能每拍译出多条并维持高频率。x86 背负 8086 以来的兼容：历史二进制里充满变长编码，高密度编码也确实省指令存储和取指带宽。它的工程补偿是预取加预译码：8086 的总线接口单元带 6 字节预取队列，在执行单元忙碌时提前把后续字节取回排队，把“按字节块取指”和“按条译码执行”的速率差解耦；分支跳转时队列作废重填，这正是变长世界里控制冒险代价的一部分。现代 x86 走得更远：按 16 字节块取指，预译码逻辑标出每条指令的边界再送进指令 cache，译出的微操作还可以进微操作 cache，把变长译码的成本从“每条指令每次执行”摊薄到“每条指令一次”。

维度	本章 16-bit 定长	x86-64 变长 (1-15 字节)
下一指令边界	PC+1, 无需译码	须先译出本条长度
译码逻辑	固定字段查表, 纯组合	前缀、opcode、ModRM 逐层判定
编码密度	低, 短指令也占满 16 位	高, 常用指令 1-3 字节
取指方式	按字一次一条	按字节块预取, 队列缓冲
每拍多条	边界已知, 容易扩展	靠预译码标记和微操作 cache 补偿

指令字段放在哪里也是设计决策 本章编码里每个字段的位置都有一条理由。把它们逐字段摆出来，就能看出编码格式不是随手排布，而是译码路径的形状。

opcode 固定占最高 4 位 (bit 15-12)。理由是分级译码：第一级只看这 4 位就分出大类，决定低位如何解释——是寄存器号、立即数还是偏移。位置固定意味着这 4 位到译码器的连线与指令无关，取指一完成查表就开始。这个约定对人同样友好：ROM 十六进制字的第一个数字就是 opcode，`0x22F0` 的 2 是立即数 `MOV`，`0x7260` 的 7 是 `SHL`，人工核对与硬件译码走同一顺序。

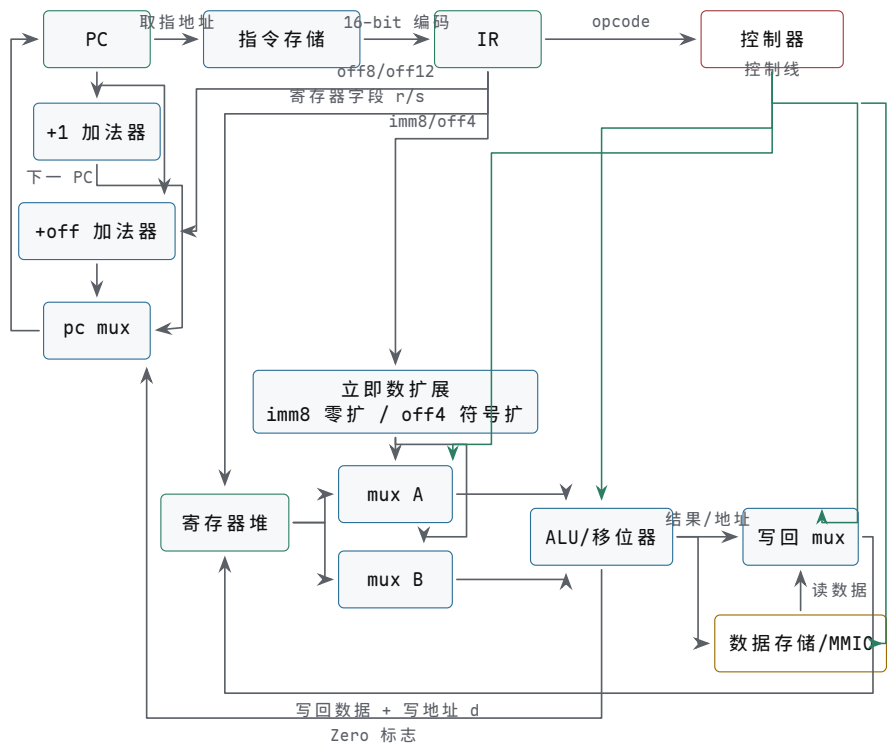
寄存器字段位置固定。 d (以及 JZ 的 r 、存储的 s) 恒在 bit 11-9, a 恒在 bit 8-6。理由是寄存器堆的读地址可以直接由 IR 的固定位驱动, 与 opcode 译码并行启动——读出来的值若用不上, 丢弃即可, 没有副作用。若字段位置随格式漂移, 读地址之前就必须先插一级按格式选择的多路选择器, 译码路径条数加倍, 寄存器读还被推迟到译码完成之后, 恰好撞上关键路径。 $0x3450$ 里 $d=010$ (R2)、 $a=001$ (R1), 与 $0x4640$ 里 $s=011$ (R3)、 $a=001$, 占的是完全相同的位置——这不是巧合, 是让读地址“免译码”的刻意安排。

立即数和偏移一律向低位对齐。 $imm4/off4$ 从 bit 5 起向下占 4 位, $imm8/off8$ 占 bit 7-0, $off12$ 占满 opcode 以外的 bit 11-0。理由是扩展逻辑单一: 各种长度的立即数共享从 bit 0 起的低位段, 零扩展或符号扩展只需按格式决定从哪一位开始复制符号、向上补多少位, 低位部分原样旁路; 指令字的 bit 0 就是立即数的 bit 0, 数值意义与位置一致。 JMP 把 opcode 之外全部 12 位留给偏移, 符号位在 bit 11, 范围 ± 2048 个字, $0x6FFB$ 的低 12 位 $0xFFB$ 即 -5 ——这是“低位全给数值、拿长度换范围”的极端情形。

字段	位置	这样放的理由	本章实例
opcode	bit 15-12	先出大类, 第一级译码只接 4 位	$0x22F0$ 首位 2 = 立即数 MOV
$d/r/s$	bit 11-9	第一个寄存器号位置恒定, 读地址免选择	$0x5403$ 的 $r=010$ 即 R2
a	bit 8-6	基址/源读地址恒定, 可与译码并行启动	$0x3450$ 的 $a=001$ 即 R1
b 、 $imm4/off4$	bit 5 起向下	第二源与短立即数共用低位段	$0x7260$ 的 $imm4=1000=8$
$imm8/off8$	bit 7-0	长立即数低字节对齐, 扩展路径单一	$0x5403$ 的 $off8=+3$
$off12$	bit 11-0	opcode 之外全给跳转范围	$0x6FFB$ 低 12 位 = -5

字段位置定下来之后, 下面的数据通路图里那些从 IR 直接引出的线才有依据: 寄存器字段到寄存器堆读地址的竖线、立即数扩展器的输入、偏移到 $+off$ 加法器的路径, 都对应这里的固定位置。

把这台 ISA 画成可接线的数据通路 有了确定指令集, 数据通路就不能再停在“ALU 加寄存器”的粗框上。下面这张图把取指、执行、写回和 PC 更新全部画成可检查的路径, 每条指令都可以在上面走一遍:



图示：这台最小 ISA 的完整数据通路。实线是数据路径，虚线是控制线。左侧一列是 PC 更新：顺序值和分支目标都先算好，由 pc mux 按 Zero 标志选择；中间是执行路径；右侧写回 mux 在 ALU 结果和存储器数据之间选择。任何一条指令的 trace，都应能在这张图上逐段指出来。

用六条指令走查这张图，可以覆盖全部指令类型：ADD 走“寄存器堆双读 → ALU → 写回 mux 选 ALU 结果”；立即数 MOV 走“imm8 零扩展 → mux A → ALU 直送 → 写回”；加载 MOV 走“基址寄存器加 off4 符号扩展 → ALU 形成地址 → 数据存储 → 写回 mux 选存储器数据”；存储 MOV 复用加载的地址路径，但数据从寄存器堆流向数据存储；JZ 的 Zero 决定 pc mux 选 +1 加法器还是 +off 加法器；SHL 走“mux A → ALU 内置移位路径 → 写回”。没有一条指令需要图外的部件——这是数据通路完整性的基本要求。

最后把 ROM 程序放进这台机器跑一整圈。设初态 PC=0、按键未按下（GPIO_IN 读 0），第 8 个时钟沿前按键被按下（GPIO_IN 读 1）：

时钟沿	PC	IR	本沿提交的状态
1	0	0x22F0	R1=0x00F0
2	1	0x7260	R1=0xF000
3	2	0x3450	R2=0（读 GPIO_IN）
4	3	0x5403	无（Zero=1，pc_sel 选目标：3+1+3=7）
5	7	0x2600	R3=0
6	8	0x4640	MEM[0xF000]=0，灯灭
7	9	0x6FF8	无（跳回 2）
8	2	0x3450	R2=1（按键已按下）
9	3	0x5403	无（Zero=0，顺序执行）
10	4	0x2601	R3=1
11	5	0x4640	MEM[0xF000]=1，灯亮
12	6	0x6FFB	无（跳回 2）

这一圈执行过程值得逐行核对：偏移算术（沿 4 的 3+1+3、沿 7 和 12 的回跳）、MMIO 地址形成（沿 3 和 8 的 0xF000+4）、条件跳转两次走向相反（沿 4 跳、沿 9 不跳）、以及程序效果本身——LED 跟随按键状态循环亮灭。能写出这张表，才说明编码、译码、数据通路、控制字和 PC 更新在同一个执行模型中互相吻合；任

何一格对不上，回到控制真值表和数据通路图逐格反查即可。

为这台 ISA 增加一条指令要走完一圈改动 以新增 `SUB R d, Rs`（语义 $R[d] \leftarrow R[d] - R[b]$ ，按结果更新 Zero）为例，把“加一条指令”拆成必须同时完成的几处改动。漏掉任何一处，机器都会出现一种特征明确的故障。

第一步，定义语义与编码。语义必须连标志一起写清：结果写回 `d`，Zero 按结果更新，因为 `JZ` 依赖它。编码要找一个空 opcode：0001 到 0111 已被占用；0000 故意保留一空 ROM 字、越界取指都会读出全零，让它译成“非法指令”比译成某条真指令更容易暴露问题。于是取 1000，字段布局与 `ADD` 完全相同：1000 `d a b ---`，二操作数约定同样是 `a=d`。

第二步，改译码真值表。控制字表新增一行：`alu_op=SUB`，两个源都选寄存器，`wb_sel=ALU`，`reg_w=1`，`pc_sel=顺序`；布尔式里 `reg_write` 多析取一项（`op == 1000`）。

第三步，确认数据通路已有减法路径。减法是加补码： $R[d] - R[b] = R[d] + (\text{按位取反 } R[b]) + 1$ 。本机 ALU 的加法器与异或网络（整机实验中由 74HC283 与 74HC86 构成）天然带这条路径：`SUB` 控制线让 B 输入走异或取反、进位输入置 1 即可，不需要任何新部件。这一步的工作量是“确认”，不是“新建”。

第四步，更新写回与标志通路。写回 mux 对 `SUB` 选 ALU 结果，与 `ADD` 共用同一条写回线；Zero 在同一个时钟沿随结果提交，保证下一条 `JZ` 读到的是新值，而不是上一条指令的旧标志。

第五步，补 trace 案例。先在纸面上核对编码：`SUB R1, R2` 拼出 1000 001 0 01 010 000 = 0x8250；再放进一段小程序走数值：

时钟沿	PC	IR	本沿提交的状态
1	0	0x2209	<code>R1=9 (MOV R1, 9)</code>
2	1	0x2404	<code>R2=4 (MOV R2, 4)</code>
3	2	0x8250	<code>R1=9-4=5, Zero=0 (SUB R1, R2)</code>
4	3	0x8490	<code>R2=0, Zero=1 (SUB R2, R2)</code>

沿 4 之后 `Zero=1`，若下一条是 `JZ R2, +off` 将按新标志跳转——这一步同时检验了第四步的标志更新时序。把五步归并一下，ISA 扩展的本质是四处一起改：编码表、译码真值表、数据通路、控制字（含写回与标志）；前两处决定“新指令是谁”，后两处决定“新指令做什么”，trace 案例则把四处改动放进同一个执行模型互相印证。

改动处	关键问题	漏改时的故障
编码表	有没有空 opcode，字段布局能否复用	新编码无处安放，或与保留字冲突
译码真值表	新行与布尔式是否同步加上	新指令译成非法，或译成别的指令
数据通路	现有部件能否完成新运算	<code>alu_op=SUB</code> 没有对应连线
控制字与写回	写回选择、写使能、标志更新是否正确	结果写不回去，或 <code>JZ</code> 读到旧 Zero

三、可见状态、条件码与调用约定

程序员可见状态不是全部硬件状态 机器级程序通常只直接看见一小组状态：PC、通用寄存器、条件码或标志、可寻址内存，有些 ISA 还显式暴露栈指针、链接寄存器或状态寄存器。CPU 内部还会有 IR、MDR、ALUOut、流水线寄存器、旁路网络和微程序计数器，但这些通常不是普通程序可以直接读写的对象。区分这两类状态很重要：ISA 规定的是程序必须可依赖的边界，微结构内部状态则服务于实现速度、面积和功耗。

状态	程序员视角	硬件实现视角
PC	下一条或当前执行位置	取指地址、分支选择、异常入口和返回路径
通用寄存器	保存整数、地址、临时值和返回值	寄存器堆端口、旁路、写回优先级
条件码	最近运算留下的比较结果	ALU 标志生成、 <code>flags_write</code> 和分支条件译码
内存	按地址访问的字节对象	cache、TLB、总线、MMIO 和权限检查
栈指针	当前栈顶位置	普通寄存器加受约束的加减和加载/存储序列

这种划分能解释为什么同一套 ISA 可以有不同实现。一个简单教学 CPU 可以用多周期状态机执行加载指令；一个高性能 CPU 可以把它拆进流水线、cache 和乱序队列。只要最终提交到程序员可见状态的结果相同，内部路径可以不同。反过来，若内部优化改变了 PC、寄存器、内存或条件码的可见顺序，就不再只是实现细节，而是破坏了 ISA 约定。

条件码把 ALU 结果变成控制流 条件分支不是“看高级语言的 if”，而是看硬件已经保存的标志。ALU 做加法、减法或比较时，可以同时产生 Zero、Sign、Carry、Overflow 等标志。控制器把这些标志与指令中的条件字段组合，决定下一 PC 选择顺序地址还是分支目标。比较指令常常等价于一次不写回结果的减法：结果本身丢弃，但标志被保留下来供后续分支使用。

条件	常见硬件标志	容易犯错的解释
相等	A-B 的 Zero 为 1	只适合判断 bit 模式相等，不说明大小
无符号小于	减法借位或 Carry 约定	不能直接用结果最高位判断
有符号小于	Sign 与 Overflow 的组合	最高位受溢出影响，必须修正
结果为负	Sign 标志为 1	只在补码解释下表示负数
有符号溢出	同号相加得到异号，或减法符号规则触发	与无符号进位不是同一件事

```
CMP R1, R2      // flags = R1 - R2, result is not written
JE equal_path   // branch if Zero == 1
JL less_path    // signed branch uses Sign xor Overflow
JB below_path   // unsigned branch uses borrow/carry convention
```

这个例子说明，分支条件必须绑定解释规则。同一对寄存器 R1 和 R2 的 bit 模式，按无符号数、有符号补码数或地址偏移解释，可能得到不同的“小于”。硬件不会自动知道程序员想要哪一种比较，必须由 opcode 或条件字段告诉控制器使用哪组标志逻辑。

条件码的使用模式：比较只设标志，分支只读标志 x86-64 把一次“判断”拆成两条指令配合完成。`cmp` 与 `test` 做一次减法或按位与，运算结果本身被丢弃，不写回任何寄存器或内存，只留下 Zero、Sign、Carry、Overflow 等标志；随后的 `jz`、`jnz`、`jc`、`jnc`、`jl` 等条件跳转不再访问通用寄存器，只读标志决定是否改写 PC。两条指令之间唯一的通信通道就是标志寄存器——这正是 x86-64 把标志归入程序员可见状态的原因：离开它，`cmp` 的结果无处可去。

三个最常用的配对覆盖绝大多数分支。第一，`cmp` 后接 `jz/jnz` 判断相等或不等：两操作数的 bit 模式相同则差为零，ZF 置 1。

```
cmp rax, rbx    // 计算 rax - rbx, 只写标志, 不写寄存器
jz equal_path   // ZF=1 时跳转: rax 与 rbx 相等
```

第二, `cmp` 后接 `jg/jl` 判断有符号大小: `jl` 的成立条件是 `SF` 与 `OF` 不同, `jg` 的成立条件是 `ZF=0` 且 `SF` 与 `OF` 相同。之所以要两个标志组合, 是因为减法一旦溢出, 结果最高位就不再可靠, 必须用溢出标志修正符号位。

```
cmp rax, rbx    // 同一对操作数, 同一组标志
jl less_path    // SF 与 OF 不同: 按补码解释 rax < rbx
jg greater_path // ZF=0 且 SF 与 OF 相同: rax > rbx
```

第三, `test` 后接 `jz` 判断一个寄存器是否为零: `test rcx, rcx` 计算 `rcx` 与自身的按位与, 结果必等于 `rcx` 本身, `ZF` 于是恰好表示“`rcx` 是否为 0”, 同时 `Carry` 与 `Overflow` 被清零。这是编译器生成“判零”时的惯用写法, 比 `cmp rcx, 0` 更短。

```
test rcx, rcx    // rcx AND rcx: ZF 恰好表示 rcx==0
jz loop_done     // ZF=1 时跳转: 计数器已减到 0
```

配对	判断依据	关键问题
<code>cmp + jz/jnz</code>	<code>ZF</code> , 差是否为零	只比较 <code>bit</code> 模式是否相同, 有符号与无符号通用
<code>cmp + jg/jl</code>	<code>ZF</code> 与 <code>SF</code> 、 <code>OF</code> 的组合	只适用于补码有符号比较; 无符号大小要换 <code>ja/jb</code> , 改读 <code>Carry</code>
<code>test + jz</code>	按位与结果的 <code>ZF</code>	操作数与自身相与, 等价于判零, 附带清 <code>Carry/Overflow</code>

回头看本章的教学 `ISA`, `JZ Rr, +off` 把“读寄存器、与零比较、条件改写 `PC`”三件事融合在一条指令里, 数据通路因此省掉独立的标志寄存器和 `cmp/test`, 分支可以在一个周期内完成。这是有意的教学简化, 代价同样清楚: 每条分支都要重新读一次寄存器, 而且只能判零。真实 `ISA` 把比较与跳转分开, 多付一条指令和一组必须在异常、中断时一并保护的标志状态, 换来的是一次比较可供多条后续分支复用, 判断条件也不限于零。

调用、返回和栈帧是受约束的地址事务 函数调用可以从软件语法降到三件硬件事: 保存返回地址, 跳到被调用函数入口, 为局部对象和保存寄存器预留内存。返回时再恢复必要状态, 把 `PC` 改回返回地址。很多 `ISA` 用专门的 `CALL/RET` 指令完成其中一部分; 精简教学 `CPU` 也可以用普通的存储、加载、加减栈指针和跳转指令组合出来。

```
CALL target, stack grows downward:
    RSP = RSP - word_size
    MEM[RSP] = PC + instruction_size    // push return address
    PC = target

RET:
    PC = MEM[RSP]                      // pop return address
    RSP = RSP + word_size
```

栈帧对象	机器级表示	硬件动作
返回地址	一个位宽等于 <code>PC</code> 的地址值	<code>push</code> 压栈保存, <code>RET</code> 时 <code>pop</code> 到 <code>PC</code>
保存寄存器	调用约定规定要保留的寄存器内容	进入时压栈, 返回前弹出恢复

局部变量 实参溢出区	RSP 或帧指针加固定偏移 寄存器放不下的参数位置	地址形成后普通加载/存储 调用方或被调用方按约定访问
对齐空洞	为总线宽度、ABI 或向量访问 保留的字节	调整 RSP，避免未对齐访问 代价或异常

把调用栈看成地址事务后，许多系统错误会变得具体。栈越界写不是抽象的“内存不安全”，而是一次存储指令形成了错误地址，可能覆盖相邻局部变量、保存寄存器、返回地址或未映射页。若返回地址被覆盖，RET 只是按约定把栈顶字节装进 PC；CPU 并不知道这个地址原本是否可信。后续章节讨论保护模式、页表、异常和执行权限时，就是在给这些地址事务加上硬件可检查的边界。

从一条语句贯通到硬件事务 为了避免本章变成若干概念并列，可以用一条简单语句贯通整机：`if (MMIO[GPIO_IN] != 0) MMIO[GPIO_OUT] = 1;`。在软件层面它像一次条件判断；在硬件层面，它至少包含取指、译码、地址形成、MMIO 读、条件分支、MMIO 写和 PC 更新。每一步都可以写成可逐项核对的硬件事务。

层次	发生的动作	需要观察的信号
取指	PC 指向 ROM 或 I-cache 中的 MOV/JZ 等指令编码	PC、指令存储片选、IR 内容
译码	opcode 被译成 <code>mem_read</code> 、 <code>pc_sel</code> 、 <code>mem_write</code> 等控制线	控制字和寄存器读地址
地址形成	基址寄存器与偏移形成 <code>GPIO_IN</code> 或 <code>GPIO_OUT</code> 地址	ALU 输入、ALUOut、地址总线
MMIO 读	GPIO 控制器返回已同步按键状 态	GPIO 片选、读使能、返回数 据稳定
条件判断	比较结果决定顺序 PC 或分支 目标	Zero 标志、分支控制、下一 PC
MMIO 写	输出锁存器接收写数据并驱动 LED	写使能、字节使能、锁存器输 出

这个例子也说明，编译后的指令顺序与外设副作用不能随意重排。普通内存访问可以被 cache、写缓冲和预取优化；但 `GPIO_OUT` 写入是外部可见动作，必须按设备属性和顺序规则执行。若系统把 MMIO 当作普通 cacheable 内存，LED 可能不会在预期时刻改变，甚至写入被合并或延迟到错误位置。

五类基本指令覆盖了 CPU 的主要通路 最小 CPU 不必一开始支持复杂指令，但至少能解释五类动作：寄存器到寄存器运算、加载、存储、条件分支和无条件跳转。它们共同覆盖寄存器堆、ALU、数据存储器、PC 选择器和标志位。

指令类	典型路径	必须打开的控制	写回对象
ADD/SUB	寄存器读出，经 ALU，结果 写回寄存器	<code>alu_op</code> 、 <code>wb_sel=ALU</code> 、 <code>reg_write</code>	寄存器、 Flags
MOV（加载）	基址寄存器加立即数形成地 址，存储器读回数据	<code>alu_op=ADD</code> 、 <code>mem_read</code> 、 <code>wb_sel=MEM</code>	寄存器
MOV（存储）	基址寄存器加立即数形成地 址，源寄存器写入存储器	<code>alu_op=ADD</code> 、 <code>mem_write</code> 、 <code>store_data_sel</code>	存储器 或 MMIO
BRANCH	比较源操作数或标志位，条 件成立则选择分支目标	<code>alu_op=CMP</code> 、 <code>pc_sel</code> 、 <code>pc_write</code>	PC
JUMP	立即数、寄存器或链接地址 决定下一 PC	<code>target_sel</code> 、 <code>pc_sel</code> 、可 选 <code>reg_write</code>	PC，可 选链接 寄存器

以加载指令为例，`MOV R3, [R1 + 8]`（即 $R3 = \text{MEM}[R1 + 8]$ ）不是单纯“读内存”。硬件要先读出 `R1`，把立即数 8 符号扩展或零扩展到 ALU 输入宽度，ALU 做地址加法，地址进入数据存储路径，读响应回来后经过写回 mux，最后在时钟沿写入 `R3`。若目标地址落入普通 RAM，读回的是存储阵列内容；若目标地址落入 MMIO 区，读回的是设备寄存器当前状态。地址相同形式，语义由地址译码决定。

```
MOV R3, [R1 + 8]
EX:  addr = R1 + sign_extend(8)
MEM:  read target selected by addr
WB:   R3 = read_data
```

存储指令的风险更高，因为它有副作用。`MEM[R1 + 8] = R3` 若落在 RAM，表示保存数据；若落在设备命令寄存器，可能启动设备、清除中断或改变输出引脚。因此存储指令必须明确字节使能、写数据来源、地址空间属性和完成条件。普通内存写响应表示写事务被接受；设备命令写响应通常不表示设备动作完成。

```
MOV [R1 + 8], R3
EX:  addr = R1 + sign_extend(8)
MEM:  write_data = R3, byte_en = access_size
      target receives write command
WB:   no register writeback
```

BRANCH 和 JUMP 则说明 PC 是数据通路的一部分。条件分支一般要先得到比较标志，例如 Zero、Sign、Carry 或 Overflow，再由控制器决定 `pc_sel`。无条件跳转可以直接选择目标地址；带链接跳转还会把返回地址写入寄存器。此时同一条指令可能同时写 PC 和一个通用寄存器，控制器必须明确两个写边界是否在同一时钟沿提交。

基本块和循环把 PC 更新变成可读结构 机器级程序可以先按基本块理解。一个基本块内部只有顺序执行，入口只有一个，出口通常是分支、跳转、返回或异常边界。硬件不认识“基本块”这个软件概念，但基本块对读 trace 很有用：块内每条指令都让 PC 顺序前进，块尾那条指令才选择下一 PC。循环就是某个块尾分支把 PC 改回较低地址。

```
loop:
  mov eax, [rdi]
  add rsi, rax
  add rdi, 4
  sub rcx, 1
  jnz loop
```

程序结构	PC 动作	硬件表现
顺序指令	$PC = PC + \text{size}$	取指地址递增， <code>pc_sel=sequential</code>
条件分支	根据标志选择顺序 PC 或目标 PC	标志位、条件译码、目标加法器
循环回边	分支目标小于当前 PC 附近地址	同一组编码被重复取指
函数调用	保存返回地址并选择函数入口	链接寄存器或栈写入，PC 改为目标
返回	从寄存器或栈恢复 PC	<code>RET</code> 或间接跳转选择返回地址

用这个视角看循环，性能问题也会具体起来。循环体中的加载可能 cache miss，加法依赖加载结果，分支依赖计数器更新，取指路径又要处理回跳。简单 CPU 会逐

条完成；流水线 CPU 需要处理 load-use 冒险和分支控制冒险；带预测的 CPU 会猜测回跳是否成立。无论复杂度如何，循环仍然是 PC、寄存器、标志和内存状态反复经过同一批硬件路径。

调用约定把寄存器和栈的责任写成明确约定 硬件只提供寄存器、加载/存储、PC 改写和少数专门指令；函数调用是否能可靠嵌套，还需要调用约定。调用约定规定参数放在哪些寄存器或栈位置，返回值放在哪里，哪些寄存器由调用者保存，哪些由被调用者保存，栈指针如何对齐。它不是纯软件礼节，因为每一条规则都会变成具体寄存器写回和内存访问。

调用约定对象	机器级作用	硬件动作
参数寄存器	前几个参数直接进入寄存器	调用前写寄存器，被调用函数读寄存器
返回值寄存器	函数结果放在约定寄存器	返回前写回，调用者随后读取
调用者保存寄存器	调用后不保证保持旧值	调用者必要时先压栈保存
被调用者保存寄存器	函数返回前必须恢复	被调用者入口压栈，出口弹出恢复
栈对齐	保证局部对象、返回地址和向量访问边界	调整 RSP，可能插入填充字节

```
caller:
    push r10                // caller saves if it needs r10 later
    mov edi, arg0
    mov esi, arg1
    call target
    pop r10
    use eax                 // return value

callee:
    push rbx                // callee-saved register
    ...
    pop rbx
    ret
```

这个 trace 能帮助读者理解为什么栈损坏常常表现成“返回后失控”。返回地址、保存寄存器和局部数组可能同在一段连续内存中。若局部数组越界写覆盖了保存寄存器，错误会在函数返回后才显现；若覆盖了返回地址，RET 会把被污染的字节装入 PC。CPU 执行 RET 时并不知道这个地址是否来自真实 CALL，它只执行“从约定位置取一个地址并写入 PC”的硬件动作。

叶子函数与非叶子函数的保存策略不同 编译器和手写汇编都按同一条规则决定函数序言的规模：这个函数还会不会调用别人。不再调用任何函数的函数称为叶子函数。它不会触发新的 CALL，栈顶不会被再压入一层返回地址，参数寄存器、返回值寄存器这类调用者保存寄存器也可以放心使用——没有下游调用来破坏它们。本章的 sum2 正是叶子函数：函数体只用 `eax`、`edi`、`esi`，按 System V AMD64 约定它们都属于调用者保存（调用后不必保持原值），所以单就计算而言它连栈帧都不必建立，三条指令足够。后面案例 trace 中那对 `push rbx/pop rbx` 是为了展示序言与尾声而保留的，最小的 sum2 长这样：

```
sum2:                // 叶子函数：不调用别人
    mov eax, edi      // 只用调用者保存寄存器
    add eax, esi
    ret               // 没有 push，也没有 sub rsp
```

一旦 `sum2` 改成自己也要调用别人——例如把加法换成调用 `sat_add` 做饱和加——它就不再是叶子，至少要补上三类动作。第一，保护会被下游调用破坏的现场：`edi/esi` 要腾出来给新调用传参，`call` 之后还需要的旧值必须先挪进被调用者保存寄存器或栈上。第二，凡是自己要用的被调用者保存寄存器，入口按约定保存、出口按相反顺序恢复。第三，维护栈对齐：x86-64 约定 `call` 执行前 `rsp` 是 16 的倍数，而函数入口因返回地址压栈变成 16 的倍数减 8，非叶子函数通常用一次 `push` 或 `sub rsp, 8` 把对齐调回来。

```
sum2:
    push rbx           // 保存被调用者保存寄存器，顺带对齐 rsp
    mov ebx, edi       // edi 要腾给新调用，旧值先进 rbx
    mov edi, esi       // 为 sat_add 准备参数
    call sat_add       // 栈顶再压一层返回地址
    add eax, ebx       // call 之后仍能取回第一个参数
    pop rbx           // 按相反顺序恢复
    ret
```

对比项	叶子函数	非叶子函数
可用寄存器	调用者保存寄存器可随意使用	跨过 <code>call</code> 还要用的值必须放被调用者保存寄存器或栈
栈帧	可以不建， <code>rsp</code> 全程不动	通常用 <code>push</code> 或 <code>sub</code> 建帧，出口反向拆除
返回地址	只在栈顶出现一层	每层内层 <code>call</code> 再压一层，自己的帧不能遮挡栈顶
对齐责任	无下游调用，无对齐负担	每次 <code>call</code> 前必须保证 <code>rsp</code> 按 16 字节对齐

这个区分直接决定序言与尾声的指令数，也就决定小函数的调用开销。叶子函数省下的不只是几条 `push` 和 `pop`，还有对应的栈读写事务；非叶子函数少做一次保存，错误不会立刻显现，而会推迟到某次深层调用之后，以“寄存器值莫名改变”的形式暴露。调用约定的意义正在这里：保存责任按“谁会破坏、谁还需要旧值”预先分配，每个函数只管自己那一段，调用链才能任意嵌套。

栈对齐是约定的一部分，不是性能洁癖 x86-64 的 System V 约定要求：`call` 执行的一瞬间，`rsp` 必须是 16 的倍数；`call` 压入 8 字节返回地址后，被调用函数入口处的 `rsp` 就是 16 的倍数减 8。这条规则写进约定而不是留给各人随意，原因有三层。第一层是压栈粒度：一次 `push` 或 `pop` 固定移动 8 字节，栈上每一格都是整字；若 `rsp` 不以 8 为边界，相邻两格就会错位重叠，读写互相覆盖。第二层是指令的硬性要求：`movaps` 这类对齐传送指令要求内存地址 16 字节对齐，编译器会把栈上某些局部对象按 16 字节放置，入口对齐一旦被破坏，这些指令直接触发通用保护异常。第三层是异常路径：中断与异常入口按约定位置保存现场，若干平台的异常栈帧同样假设栈指针对齐，对齐失守会让处理程序自身再出异常。

对齐的算术只有一条规律：每一次 8 字节的 `push` 或 `call`，都把“`rsp` 模 16”的余数翻转一次。入口时余数为 8，做奇数次 `push`（或一次 `sub rsp, 8`）后余数回到 0，可以安全 `call`；做偶数次则余数仍是 8，再 `call` 就把错位传给下游。上一条目里非叶子 `sum2` 用一条 `push rbx` 同时完成寄存器保存与对齐调整，正是这个算术的应用。

对齐规则	机制理由	失守后果
<code>rsp</code> 以 8 字节为粒度移动	<code>push/pop</code> 按整字压栈、读栈	相邻栈格错位重叠，数据互相覆盖
<code>call</code> 前 <code>rsp</code> 是 16 的倍数	给 <code>movaps</code> 等 16 字节对齐访问留边界	对齐传送指令触发通用保护异常

入口处余数为 8 由 call 压栈造成 普通访问允许未对齐	序言用奇数次 push 或 sub 把余数调回 0 一次访问可能横跨两个 cache 行甚至两页	错位沿调用链向下游传 播 拆成两次总线事务，变 慢且失去单次事务的原 子性
--------------------------------------	---	---

未对齐的后果因此分两档：普通整数加载/存储未对齐时，x86-64 硬件大多照常执行，只是付出额外事务的代价；对齐传送指令、严格平台以及打开 Alignment Check 的 x86 则直接抛对齐异常。这与第五章总线事务的字节使能互为表里：总线按字传输时，字节使能标记本次事务实际写哪些字节，地址未对齐意味着同一数据要动用两个总线周期、两组字节使能。对齐约定把这种代价在编译期就消掉，而不是留给硬件每次补救。

四、异常、中断与精确异常

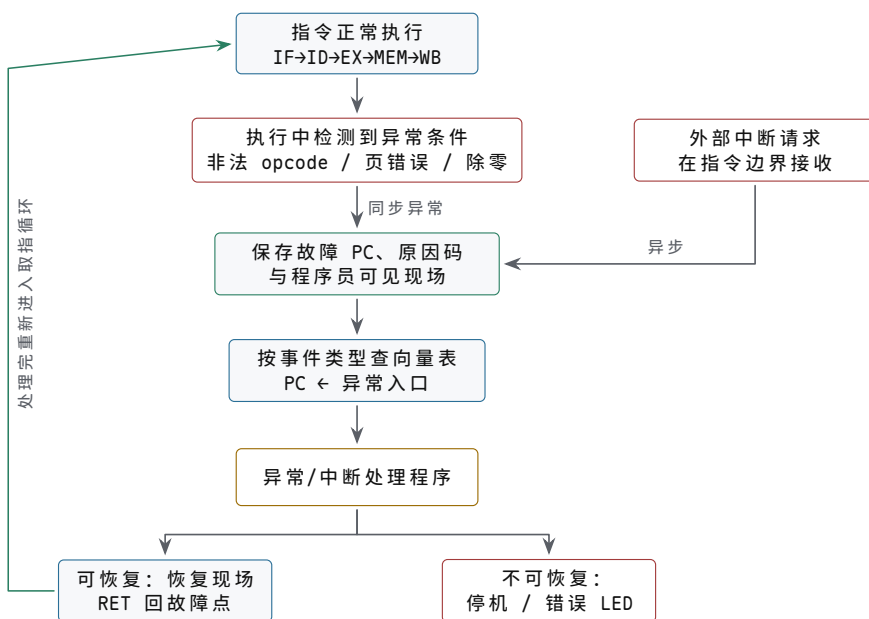
异常和中断是受控的非顺序 PC 更新 异常、中断和系统调用都可以视为特殊控制流。普通分支由指令显式请求；异常由当前指令执行过程中发现的条件触发，例如非法 opcode、除零、未对齐访问、页错误或总线错误；中断来自外部设备或定时器。三者最终都要让 CPU 保存足够的旧状态，并把 PC 改到一个规定入口。

事件	触发来源	入口动作
同步异常	当前指令导致，如非法指令、 页错误、除零	保存故障 PC 和原因，跳异常向量
外部中断	设备、定时器或中断控制器请求	在可接收边界保存 PC/Flags，跳中断入口
系统调用	指令显式请求进入特权服务	切换权限和入口，保存返回位置
复位	电源或复位逻辑强制初始化	PC 装入复位向量，控制状态清零或设默认值

异常路径最重要的是提交边界。若一条加载指令因页错误失败，目标寄存器不应保存随机数据；若一条存储指令访问设备前发生权限错误，设备不应看到写命令；若一条除法发生除零异常，后续指令不应像已经成功执行那样提交。简单 CPU 可以在每条指令结束时才接收中断；复杂流水线 CPU 则需要记录哪些指令已经可以提交，哪些仍可能被冲刷。

```
faulting load:
  IF:  fetch instruction
  ID:  decode MOV
  EX:  form address
  MEM: address translation fails
  WB:  suppressed
  PC:  exception_vector
```

这个例子说明，异常不是“多跳一次分支”这么简单。它还要阻止错误写回，把失败原因带给入口，并保证重试或终止时系统能判断发生了什么。第六章讨论 80386 页错误时，会看到硬件如何保存故障线性地址和错误码；第四章这里先建立统一模型：异常是硬件检测到无法继续普通提交后，选择另一条受控 PC 更新路径。



图示：异常与中断共用同一条入口骨架：保存现场、查向量表、跳转处理程序。区别只在触发来源和接收时机——同步异常必须禁止当前指令的错误写回，异步中断只在指令边界接收。处理完后沿虚线回到取指循环；裸机程序若无法恢复，至少要把系统停在一个可诊断的状态，而不是带着错误现场继续跑。

中断向量把异常号变成入口地址 异常或中断被接收之后，硬件面对的问题很具体：PC 应该改成多少。普遍做法是把所有入口预先存成一张向量表——本质是一个“异常号映射入口地址”的数组。事件类型被编码成异常号（中断号、向量号），硬件拿它做下标查表：表项地址 = 表基址 + 异常号 × 表项宽度，一次访存就得到入口，CPU 不需要逐条询问设备。

慢的对照方案是轮询式。多个设备共享一条中断请求线时，CPU 进入公共处理程序，逐个读各设备的状态寄存器，读到哪家的请求标志置位就处理哪家。若共享 5 个设备，读一次状态寄存器算一次 20 个周期的总线事务，最坏情况下来源排在最后，仅确认来源就要 $5 \times 20 = 100$ 个周期，真正的处理还没开始；设备数增加，这笔开销线性上升。向量式让设备在中断响应周期直接把向量号送上总线（8086 系统中由 8259 中断控制器完成这个动作），确认来源的开销与设备数无关。

8051 是固定向量的例子：向量表从程序存储器地址 0 开始，每个入口间隔 8 字节——复位用 0x0000，外部中断 0 用 0x0003，定时器 0 用 0x000B，外部中断 1 用 0x0013，定时器 1 用 0x001B，串行口用 0x0023。8 字节只够放一条跳转指令加一点收尾，装不下完整处理程序，所以表项里通常就是一条跳往真正处理程序的跳转指令。8086 则是数组式向量表的完整形态：内存最低端 0x00000–0x003FF 的 1 KB 放 256 项，每项 4 字节（2 字节偏移加 2 字节段地址），中断号为 n 的表项地址就是 $4n$ 。以 int 21h 为例，中断号 0x21 即十进制 33， $4 \times 33 = 132$ ，也就是 0x84，它的入口存在 0x00084 开始的 4 个字节里。

机器	表项定位	表项内容	组织特点
8051	固定地址，间隔 8 字节	通常是一条跳转指令	入口少；同一入口有多个来源时仍要软件再分辨
8086	$4n$ (n 为中断号)	偏移加段地址共 4 字节	256 项统一索引，软中断与硬件中断同表
教学模型	表基址 + 异常号 × 项宽	处理程序入口地址	本章异常入口沿用同一查表动作

向量表把“谁请求”翻译成“去哪里”，此后的入口动作——保存现场、切换权限、阻止错误写回——对各类事件是公共的，这正是上一主题那张异常骨架图中“查向量表”

一格的具体内容。还要留意一个边界：向量表本身也是内存，表项若被写坏，事件到来时 PC 会得到一个无意义地址。保护模式下的 x86 把向量表（IDT）的基址与界限交给特权寄存器管理，正是为这张表加上访问边界。

精确异常要求程序员可见状态像顺序执行 流水线和乱序执行会让多条指令同时处在不同阶段。若较晚指令已经写回，较早指令随后发现异常，程序员可见状态就会变得难以恢复。精确异常要求异常发生时，状态看起来像按程序顺序执行到某条指令：异常之前的指令都已提交，异常指令和之后的指令都没有提交。教学 CPU 可以通过按序完成天然满足；现代 CPU 需要提交队列、重排序缓冲或类似机制。

机制	解决的问题	代价
按序流水线	后一条指令不越过前一条提交	吞吐有限，长延迟指令会拖住后续
提交队列	执行可乱序，提交按程序顺序	需要记录目的寄存器、异常原因和结果
寄存器重命名	消除部分假依赖，允许更多并行	需要映射表、空闲表和恢复机制
冲刷流水线	分支预测错或异常时丢弃错误路径	浪费已经做的工作，需要恢复 PC 和状态

虽然本书不展开乱序核心设计，但提前理解“执行”和“提交”的区别很重要。ALU 算出结果只是执行完成，结果进入程序员可见寄存器才是提交。加载指令从 cache 取回数据只是访存完成，目标寄存器被写回才是提交。存储更复杂：它可能先进入 store buffer，等确认没有异常、顺序允许、地址属性允许后才对 cache、内存或设备可见。这个区分把第四章 CPU trace、第五章内存顺序和第六章平台异常连成一条线。

程序时间等于指令数乘 CPI 再乘时钟周期 评价一台 CPU 跑一个程序有多快，只需一个乘法：

$$\text{程序时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期} = \text{指令数} \times \text{CPI} / \text{主频}$$

指令数由程序、编译器和 ISA 决定；CPI（每条指令的平均周期数）由微结构与访存行为决定；时钟周期由工艺和最长组合路径决定。三个因子各自独立地进入乘积，任何优化都必须先说清楚自己动的是哪一个。

第一组演算：同一个程序（同一份二进制，100 万条指令）分别跑在两台 100 MHz 的机器上，时钟周期都是 10 ns。机器甲是理想单周期，CPI=1；机器乙是多周期，CPI=4。

机器	CPI	周期	程序时间
甲（单周期）	1	10 ns	$10^6 \times 1 \times 10 \text{ ns} = 10 \text{ ms}$
乙（多周期）	4	10 ns	$10^6 \times 4 \times 10 \text{ ns} = 40 \text{ ms}$

周期相同的前提下，时间差恰好等于 CPI 差：乙比甲慢 4 倍，多花的 30 ms 全部来自每条指令多用的 3 个周期。但不能反推出“CPI 低就一定快”。多周期设计敢于把周期做短，正是因为每个周期只做一小段：若甲按单周期实现，最长组合路径要求 40 ns 的周期，甲的实际时间变成 $10^6 \times 1 \times 40 \text{ ns} = 40 \text{ ms}$ ，与乙持平。只看 CPI 或只看主频都会误判，三个因子必须一起算。

第二组演算：cache 命中率对有效 CPI 的影响。cache 全命中时基础 CPI 为 1；设平均每条指令引起 1.2 次存储器访问（取指 1 次，约 20% 的 load/store 指令再贡献 0.2 次），miss 率 2%，每次 miss 罚 50 个周期，则：

$$\begin{aligned} \text{有效 CPI} &= \text{基础 CPI} + \text{每条指令访存次数} \times \text{miss 率} \times \text{miss 代价} \\ &= 1 + 1.2 \times 0.02 \times 50 = 1 + 1.2 = 2.2 \end{aligned}$$

98% 的命中率听起来很高，却因为 50 周期的 miss 代价把有效 CPI 抬到 2.2，程序时间变成理想情形的 2.2 倍。miss 代价越大，命中率小数点后的每一位就越值钱，这也是存储层次要在本书后文独占整章的原因。

提速途径	典型手段	付出的代价
减少指令数	更好的算法、编译器优化、表达力更强的 ISA	单条指令更重，CPI 或周期可能变长
降低 CPI	流水线、cache、旁路、分支预测	控制变复杂，冒险与精确异常都要额外机制
缩短周期	更深流水、更短组合路径、更先进工艺	级间寄存器开销增加，CPI 可能回升

三条途径共用同一个乘积，所以彼此牵制：复杂指令集用一条指令做更多事来减少指令数，往往抬高 CPI；深流水缩短周期，却让分支预测错误的冲刷更昂贵。本章的最小 CPU 在三条路上都走得很保守—指令数不省、CPI 接近 1、周期却必须包容最长路径—它存在的意义是把乘积里的每一项都暴露出来，让后续每个性能相关的章节都能对号入座。

五、整机实验、调试顺序与案例 trace

最小整机实验要先有明确的观察点 若在一块低速实验板上实现本节内容，不必立即追求完整 ISA。可以先定义四条动作：**ADD**、**MOV**（加载）、**MOV**（存储）、**JZ**。指令 ROM 提供编码，4 个寄存器保存通用值，74HC283/74HC86 构成加减核心，SRAM 或寄存器阵列模拟数据存储器，若干 LED 和拨码开关映射成 MMIO。实现要点是每一步都可观察：PC LED 显示当前取指地址，IR LED 或逻辑分析仪显示当前指令，控制线可单步观察，写回和 PC 更新只在时钟沿发生。

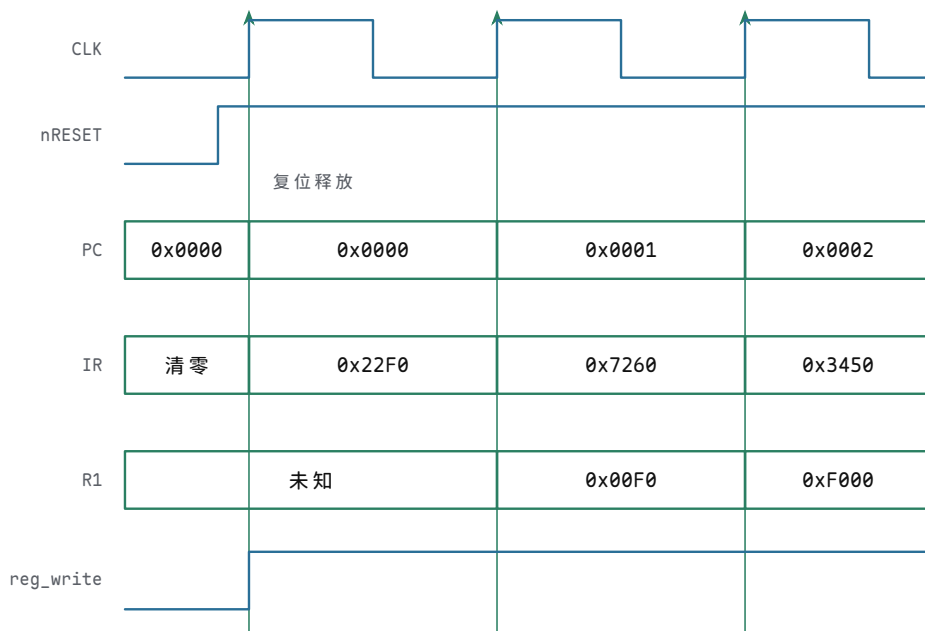
实验对象	可用器件或实现	可观察行为
指令 ROM	EEPROM、Flash 或预置拨码/仿真 ROM	复位后固定地址输出第一条编码
PC/IR	74HC161、74HC574 或触发器组	PC 每步按控制更新，IR 保存本周期指令
数据 RAM	SRAM、寄存器阵列或仿真存储器	加载/存储的地址、数据、写使能可观察
地址译码	74HC138、比较器或门阵列	RAM、LED、按键寄存器不会同时响应
MMIO LED	锁存器驱动 LED	写指定地址后 LED 状态在边界改变
MMIO 按键	上拉/下拉、施密特、同步器	读指定地址返回已同步状态，不直接用生按键

最小整机的单步调试顺序 搭建教学 CPU 时，最可靠的调试顺序不是一次性运行程序，而是逐个检查每个状态边界。先让复位把 PC 固定到入口地址，再单步确认 ROM 输出第一条指令；随后确认 IR 能锁存该指令，译码能产生正确控制线；再用一条 ADD 验证寄存器读、ALU 和写回；最后才加入加载、存储、分支和 MMIO。

调试阶段	操作	预期行为
复位	拉低/拉高复位并给一个时钟	PC 固定为入口，写使能均处于安全值
取指	单步一个取指周期	ROM 片选有效，IR 保存预期编码

ADD	预置 R1/R2, 执行 <code>ADD R1,R2</code>	R1 在时钟沿变为正确和, PC 前进
MOV (加载)	RAM 某地址预置数据, 执行加载	地址、读使能、写回来源均正确
MOV (存储)	执行写 RAM 或写 LED 寄存器	只有目标片选有效, 非目标不被改写
分支	改变 Zero 条件重复执行	PC 在顺序地址和分支目标之间正确选择

这种逐步观察的方式也适用于 FPGA。逻辑分析仪、仿真波形或片上 ILA 应至少抓取 PC、IR、控制字、ALU 输入输出、内存地址、读写使能、写回数据和寄存器写地址。若只看最终 LED 是否闪烁, 错误可能被循环掩盖; 若能看到每条指令的提交边界, 错误就能定位到具体周期。



图示: 复位后前三个周期的时序, 对照 § 二 ROM 程序开头三条编码 (`0x22F0=MOV R1,0xF0`、`0x7260=SHL R1,8`、`0x3450=MOV R2,[R1+4]`)。复位把 PC 保持在 `0x0000`、写使能保持在安全值; 释放后每个周期内组合路径完成取指、译码、执行, 周期末上升沿才提交新状态—`0x00F0` 在第二个上升沿、`0xF000` 在第三个上升沿写进 R1。这张波形图直观展示了“状态只在时钟沿改变”, 单步调试时应逐格核对。

流水线把吞吐率问题提前暴露出来 本节及其后的 scoreboard 专题属于扩写目录中“流水线、冒险和异常”一章的提前浓缩, 标记为选读: 第一遍读最小 CPU 时可以先跳过, 等单周期 trace 走通后再回来。一旦把取指、译码、执行、访存和写回拆成流水级, CPU 的平均吞吐率可以提高, 但控制复杂度也立刻上升。相邻指令可能争用同一个存储器端口, 后一条指令可能读取前一条尚未写回的寄存器, 分支可能让已经取到的指令变成错误路径。这些问题统称结构冒险、数据冒险和控制冒险。

冒险类型	例子	硬件处理
结构冒险	取指和加载同时争用单端口存储器	分离指令/数据存储, 或插入停顿
数据冒险	<code>ADD R1,...</code> 后立即 <code>SUB R2,R1,...</code>	旁路/转发, 或等待写回后再读
控制冒险	条件分支结果未出时已取后续指令	预测、延迟槽、停顿或冲刷流水线

异常冒险	较晚指令先完成，较早指令 发生异常	提交队列或按序提交保证精确 状态
------	----------------------	---------------------

因此，现代芯片低时延不只是“流水线更深”。流水线必须配套转发网络、停顿控制、冲刷控制和精确异常边界。否则，吞吐率提高会以错误状态提交为代价。学习最小 CPU 时提前看到这些冒险，可以防止把流水线误解成简单地把框图切成几段。

案例：从一个计数循环走到逐周期 trace 为了把本章从概念推进到可运行的工程，可以完整走读一个小程序。假设内存中从 `BASE` 开始有若干 32-bit 数，`rdi` 保存当前地址，`rcx` 保存剩余元素个数，`rsi` 保存累加和。程序每轮加载一个元素，加入累加和，地址加 4，计数减 1，若计数不为 0 就回到循环入口。

这个案例改用真实 x86-64 写法（Intel 语法，目的操作数在前）：32-bit 数据经 `eax` 中转，地址、计数与累加和使用 64-bit 寄存器，比前面的 16-bit 极小 ISA 宽一档，数值关系更醒目；两者的 trace 方法完全相同——读旧状态、经组合路径、在时钟沿提交。

```
loop:
    mov eax, [rdi]
    add rsi, rax
    add rdi, 4
    sub rcx, 1
    jnz loop
done:
    mov [RESULT], esi
```

这段程序覆盖了本章大部分通路：取指、译码、地址形成、数据加载、ALU 加法、立即数扩展、条件码或比较、条件分支、存储。若它能在单步下跑通，最小 CPU 的核心路径基本都被验证过。

指令	主要输入	组合路径	提交对象
MOV （加载）	<code>rdi</code> 、偏移 0	ALU 地址加法，数据存储器读	<code>eax</code> , PC
ADD	<code>rsi</code> 、 <code>rax</code>	ALU 加法，写回 mux	<code>rsi</code> , Flags, PC
ADD	<code>rdi</code> 、立即数 4	立即数扩展，ALU 加法	<code>rdi</code> , Flags, PC
SUB	<code>rcx</code> 、立即数 1	B 取反加一或减法路径	<code>rcx</code> , Zero, PC
JNZ	Zero 标志	条件判断，PC mux	PC
MOV （存储）	<code>rsi</code> 、 <code>RESULT</code> 地址	地址形成，数据存储器写	内存或 MMIO, PC

若采用五阶段流水线，前两条指令立即暴露 load-use 数据冒险。`mov eax, [rdi]` 读回的数据通常到 MEM 阶段末尾才可用，而下一条 `add rsi, rax` 在 EX 阶段就需要 `rax`。没有转发或停顿时，加法指令会读到 `rax` 的旧值。

周期	MOV	ADD	ADD	SUB
1	IF			
2	ID	IF		
3	EX	ID	IF	
4	MEM	stall 或 EX 等待	ID	IF
5	WB	EX 使用转发值	ID/EX	ID

这个表不是为了背流水线图，而是训练读者问三个问题：数据什么时候产生，下一条什么时候需要，是否有合法路径把新值送过去。若没有旁路网络，就必须插入停顿；若有从 MEM/WB 到 EX 的转发，控制器必须检测依赖并选择转发输入。否则程序在低速单周期模型中正确，在流水线模型中错误。

案例：条件分支和预测错误怎样恢复 同一个循环的 `JNZ` 还能展示控制冒险。取指单元通常希望每周期都取下一条指令，但分支是否成立要等比较结果出来。若等到结果再取指，流水线会空等；若提前猜测，就可能取错路径。教学 CPU 可以先选择“遇到分支就停顿”，再讨论预测。

策略	硬件动作	代价
停顿直到分支结果	分支进入 EX 前不继续提交后续路径	控制简单，循环每次损失周期
总是假设不跳	继续取顺序 PC，若实际跳转则冲刷	适合少跳分支，循环回边常错
总是假设跳转	提前取目标 PC，若实际不跳则冲刷	循环体内常较好，退出时错一次
动态预测	根据历史选择方向和目标	需要预测表、目标缓存和恢复机制

预测错误恢复必须同时处理两类状态。第一，已经取到但不该执行的指令要被冲刷，不能写寄存器、内存或设备。第二，PC 要恢复到正确目标，取指从那里重新开始。若错误路径上有存储或 MMIO 写已经对外可见，就无法简单恢复；因此流水线通常要把真正对外可见的提交推迟到确认顺序正确之后。

```
branch predicted not taken:
  fetch sequential instruction
  branch resolves taken
  squash wrong-path instruction
  PC = branch_target
  restart fetch
```

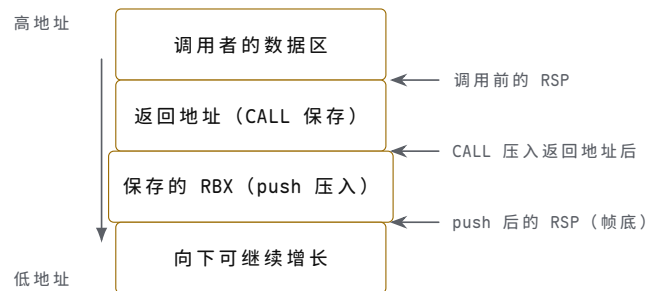
案例：函数调用的完整栈帧 trace 再用一个更接近系统程序的例子。函数 `sum2(a,b)` 返回两个整数之和。调用者把参数放进 `edi/esi`，`call` 把返回地址压栈并跳转；被调用者用 `push` 保存需要保留的寄存器，计算结果放进 `eax`，`ret` 恢复 PC。

```
caller:
  mov edi, 7
  mov esi, 5
  call sum2
  mov [RESULT], eax

sum2:
  push rbx
  mov eax, edi
  add eax, esi
  pop rbx
  ret
```

调用阶段	程序员可见状态变化	硬件边界
准备参数	<code>edi=7, esi=5</code>	<code>MOV</code> 写回寄存器，PC 顺序前进

CALL	返回地址压栈，PC=sum2 入口	栈指针减小，返回地址写入栈顶，再写 PC
建立栈帧	push 保存 RBX，RSP 减小	栈指针减小，写数据到栈地址
计算返回值	eax=edi+esi	ALU 加法，eax 写回
销毁栈帧	pop 恢复 RBX，RSP 加回	读栈、栈指针加回、寄存器写回
RET	PC= 返回地址	从栈顶读地址，写 PC



图示：函数调用案例对应的栈帧内存布局。CALL 先把返回地址压入，RSP 下移一格；sum2 的序言用 push 把 RBX 保存到新帧里；尾声 pop 按相反顺序恢复，RSP 加回。RET 硬件只认“返回地址”这一格：若某条越界写改写了它，RET 就把被污染的值装进 PC，程序表现为“返回后失控”。

这个例子把“调用约定”和“硬件状态”联系在一起。若 RSP 初始化错误，第一条 push 就可能写坏数据区；若 CALL 压入返回地址的位置错误，RET 会跳到不可预测地址；若被调用者忘记恢复 RBX，调用者看到的 RBX 就被破坏。所谓 ABI 兼容，最终表现为这些寄存器、栈位置和 PC 更新都按同一约定执行。

案例：异常插入到调用链中 若 sum2 内部执行加载时发生页错误或总线错误，异常入口必须知道故障发生在哪条指令、如何保存返回上下文、是否能恢复。简单裸机可能直接停机或点亮错误 LED；带操作系统的机器会把故障地址、错误码和当前 PC 保存到异常栈，再进入处理程序。

异常保存对象	为什么需要	恢复或诊断用途
故障 PC	知道哪条指令失败	修复后重试，或报告故障位置
状态/标志	保留中断允许、条件码、特权状态	返回时恢复执行环境
通用寄存器	处理程序可能使用寄存器	避免破坏被打断程序
错误原因	页不存在、权限错误、总线超时等	选择缺页、杀进程、复位设备或停机
栈指针/特权栈	异常处理本身需要可靠栈	防止在坏用户栈上继续保存状态

因此，异常处理不是在程序旁边加一条旁路，而是 CPU、栈、向量表、权限和内存映射共同定义的第二套控制流。第四章到这里可以停在一个结论：最小 CPU 只要能顺序执行还不够，真正的机器还必须能在错误、等待、分支和外部事件中保持可恢复的状态边界。

案例：写回来源选错怎样定位到一根控制线 第一个调试案例发生在多周期整机第一次跑通 ROM 程序时。现象是：前两条指令完全正确—0x22F0 执行后 R1=0x00F0，0x7260 执行后 R1=0xF000；第三条 0x3450=MOV R2, [R1+4] 执行后，R2 应当是 MEM[0xF004] (GPIO_IN，按键未按下时读 0)，实测却是 0xF004。随后 JZ R2, +3 因为 R2 非零总是顺序执行，表现为按键没有按下灯也亮。先把这个事实本身看清楚：R2 拿到的值恰好等于本条指令刚形成的访存地址。加载结果一般不会等于自己的地址，这个“巧合”就是第一根线索。

定位沿着写回数据通路反向走。写回 mux 只有两个候选：ALUOut 和 MDR。R2 拿到 ALUOut，说明 wb_sel 选错了来源；但在下结论之前，还要排除“访存本身就错了”的可能。单步重跑这条加载，逐拍观察：

检查步骤	观察结果	可以排除或锁定的范围
核对写回值与地址	R2=0xF004，与 ALU 形成的地址相同	写回数据来自 ALU 通路，不是随机错误
单步查 MDR	MDR=MEM[0xF004]=0，读使能、地址均正确	访存路径正常，故障在 MDR 之后
查 wb_sel (T4 访存拍)	期望已切到“存储器”，实测仍为 ALU	锁定到这一根选择线
回查控制字来源	T3 地址形成把 wb_sel 置为 ALU，T4 没有切换	控制字按拍切换缺失

根因是控制字在访存拍没有切换到存储器来源：wb_sel 在 T3 被“地址形成”这一拍置成 ALU（对地址加法本身无害，因为 T3 不写回），但控制器没有在 T4 把它改成“存储器”，于是 T5 写回拍沿用旧值，写回 mux 输出 ALUOut 而非 MDR。修复办法是把 wb_sel 改为按（状态，opcode）译码：加载指令从 T4 起 wb_sel= 存储器，T5 提交时写回 mux 自然输出 MDR。这个案例的方法比结论更重要：症状（寄存器值错）先归到路径（mux 输出错），再归到控制线（wb_sel），最后归到控制字（某一拍的某一格）——每一环都可观察、可单步复核。调试控制器时，永远先问“这个错误值是数据通路上哪一个 mux 能产生的”，再问“谁在这一拍驱动它的选择线”。

案例：PC 更新顺序错怎样让分支被撤销 第二个调试案例发生在同一台多周期整机上。正确设计约定 PC 只在一条指令的最后一个状态（T5）由 pc_sel 一次性提交；某次实现中，设计者让 JZ 在 T3 判零成立时就把目标写进 PC，理由是“早一拍跳转可以省时间”，同时又保留了所有指令在 T4 位置的“顺序 PC 提交”——PC←PC_seq，其中 PC_seq 在 T1 由 PC 加一锁存。两处改动各自看起来都合理，合在一起就出错了。

现象要对照正确行为看。按键未按下时 R2=0，地址 3 的 JZ R2, +3 成立，目标为 3+1+3=7，正确机器跳过地址 4、5、6，执行 7、8、9 的灭灯路径；实测地址 4 的 MOV R3, 1 仍然执行——连续两轮按键采样里它提交了两次，而正确行为下第一轮它根本不该出现，地址 7 的 MOV R3, 0 则一次也没有出现，灯不随按键熄灭。单步观察 PC，问题立刻显形：

时刻	PC	IR	说明
JZ 的 T1 末	3	0x5403	取指；PC_seq 锁存为 3+1=4
JZ 的 T3 末	7	0x5403	判零成立，PC 被提前改写为目标 3+1+3=7
JZ 的 T4 末	4	0x5403	顺序提交 PC←PC_seq=4，目标被覆盖
下一拍 T1 末	4	0x2601	IR 装入地址 4 的 MOV R3, 1——本应被跳过

根因是 PC 写使能的时机与指令提交边界重叠：同一条指令的生存期内 PC 被写了两次，T3 的分支写先发生，T4 的顺序提交后发生，后者覆盖前者，分支于是被静默撤销。这类错误尤其危险，因为它不产生任何异常、不停机、不点错误灯，只是“跳转偶尔不生效”，很容易被当成程序逻辑错误。修复办法是取消 T3 的直接写：分支目标在 T3 只装入 pc_next 寄存器，PC 只在一条指令的最后一个状态写入一次，由 pc_sel 在提交拍统一选择顺序值或目标。PC 是全机共享的状态，它在每一拍至多有一个写入者——这条纪律一旦破坏，分支、调用返回和异常入口会一起失去

保证。

本章到此只考察 CPU 内部 trace 最小 CPU 章节到这里应停在内部状态边界：PC、IR、寄存器堆、ALU、写回 mux、控制字、等待状态和流水线冒险。只要读者能把每条指令写成“读旧状态、经过组合路径、在时钟沿提交新状态”的 trace，本章目标就已经完成。

下一章再把这个 CPU 接到主存、总线、MMIO、DMA 和中断，讨论地址译码、等待、错误、cache 可见性和设备副作用。第六章再用 6502、Z80、8051、8086、80386、80486/Pentium 和现代 SoC 观察真实芯片如何把这些边界逐步扩展到整机平台。这样分章后，CPU 内部 trace、整机事务、经典芯片三类问题不会混在同一节里。

专题：译码器怎样从指令字段产生控制线 指令不是被 CPU “理解”后才执行，而是字段被译码成一组控制选择。操作码决定大类，寄存器字段连接寄存器堆读写地址，立即数字段进入符号扩展或零扩展路径，功能字段进一步选择 ALU 操作。最小 CPU 的译码器可以是组合逻辑，也可以是 ROM/PLA 或微码入口，但它的产物都应体现为可检查的控制线。

指令字段	进入的硬件路径	典型错误
opcode	主控制器，决定 mem_read / reg_write/pc_sel 等	未定义 opcode 被当成合法指令执行
rs/rt	寄存器堆读地址	字段位置接反导致读错源寄存器
rd	寄存器堆写地址或写回 mux 的目标选择	存储或分支指令错误写寄存器的目标选择
imm	符号扩展、零扩展、左移、地址加法器	立即数扩展规则与 ISA 不一致
funct	ALU 控制的二级译码	ADD/SUB/SLT 等同 opcode 下混淆

译码必须明确非法指令策略。教学机器可以停机并点亮错误码；带异常的机器可以保存 PC 并进入非法指令入口；极简硬件也可以规定未定义 opcode 为 NOP，但这种做法会掩盖程序损坏。无论选择哪种策略，都不能让非法组合悄悄打开 mem_write 或跳到不可预测 PC。

专题：多周期控制 FSM 的完整状态表 前面把一条指令拆成取指、译码、执行、访存、写回五拍，§ 一还用 T0-T4 的编号给出过加载指令的微操作序列。本专题把同一拆分改记为 T1-T5（从 1 起编号），并补齐所有指令的完整状态表。机制上只需记住三件事：控制器里有一个状态寄存器保存当前拍；每一拍输出的控制字由（状态，opcode）共同译出，而不是只随 opcode；每一拍末状态寄存器按“下一状态”规则前进，一条指令走完回到 T1。T4 只被加载、存储两条 MOV 使用，因为只有它们需要数据存储器事务；PC 只在 T5 提交一次，这正是前面 PC 更新顺序案例的修复原则写进状态表的样子。

状态	本拍动作	控制线	下一状态
T1 取指	IR←MEM[PC]；PC 本拍不改变	pc_out=1、 imem_read=1、 ir_write=1	T2（所有指令）
T2 译码	A←R[a]、B←R[b]； imm8/off4/off8 按指令扩展； 按 opcode 预置后续控制字	读地址选择、 imm_kind；本拍无状态 提交	T3（所有指令）

T3 执行	ADD: $ALUOut \leftarrow A+B$; SHL: $ALUOut \leftarrow A \ll imm4$; MOV 立即数: $ALUOut \leftarrow imm8$; 加载/存储: $ALUOut \leftarrow A+off4$; JZ: $Zero \leftarrow (R[r]==0)$; JMP: 备好目标	alu_op、a_sel、b_sel 按指令; JZ 时 $\rightarrow T4$; 其余 $\rightarrow T5$ flags_write=1	加载/存储 $\rightarrow T4$; 其余 $\rightarrow T5$
T4 访存	加载: $MDR \leftarrow MEM[ALUOut]$; 存储: $MEM[ALUOut] \leftarrow R[s]$; ready=0 时停留本拍	加载: mem_read=1, wb_sel 预选存储器; 存储: mem_write=1、byte_en	T5
T5 写回	寄存器提交 (仅 ADD、MOV 立即数、加载、SHL); $PC \leftarrow pc_sel$ 输出并提交	reg_write 按指令; wb_sel= ALU 或 寄存器; pc_write=1	T1 (下一条)

从这张表可以直接读出哪些指令跳过 T4: ADD、MOV 立即数、SHL、JZ、JMP 五种非访存指令在 T3 之后由次态逻辑直接送往 T5，每条只占 4 拍；加载和存储占满 5 拍。同理，JZ、JMP 和存储在 T5 的 reg_write=0，它们只提交 PC。次态逻辑因此有两个输入：当前状态和 opcode—T3 之后去 T4 还是 T5，正是靠 opcode 区分的唯一一处分叉。慢存储器也很好安放：ready=0 时次态保持 T4 不变，等待只拖住访存拍，不影响其他指令的其他拍。

这张状态表同时是硬连线控制与微程序控制的交点。用硬连线实现，它是“状态触发器加次态组合逻辑再加输出译码”；用微程序实现，每一行就是一条微指令，“本拍动作”是微指令的数据通路字段，“下一状态”就是下一微地址字段，T3 之后的分叉对应一条按 opcode 选择微地址的条件微指令。第三章讨论微程序控制器时说过，微序列器每个周期只做一件事：按微地址取微指令、送出控制线、再按条件选下一微地址—本表就是那台微序列器在这台教学 CPU 上的全部表格内容。两种实现的差别只在规则写在哪里，每一拍该出现的控制线一根也不多、一根也不少。

专题：scoreboard 和冒险检测的最小逻辑（选读） 本专题与前面的流水线冒险同属“流水线、冒险和异常”一章的提前浓缩，选读。流水线把多条指令重叠执行后，控制器必须知道“某个结果现在是否已经可用”。最简单的办法不是先设计复杂乱序执行，而是在流水级寄存器里保留目标寄存器号、是否写回、结果来自 ALU 还是存储器等信息，再由冒险检测逻辑比较当前指令的源寄存器。

冒险	检测条件	处理策略
ALU-ALU RAW	前一条会写 rd，后一条读取同一寄存器	转发 EX/MEM 或 MEM/WB 结果
load-use	加载的目标寄存器被下一条立即读取	停顿一拍，等待数据从存储阶段回来
存储数据	存储指令的写数据来自尚未写回的指令	转发到存储数据输入，或停顿
分支控制	分支结果尚未决定但后续指令已取入	冲刷流水线中错误路径指令
结构冲突	取指和访存争用同一存储端口	停顿，或分离指令/数据存储器

最小冒险检测可写成硬件式条件，而不是口头描述。例如：

$$stall = IDEX.memRead \wedge ((IDEX.rd = IFID.rs) \vee (IDEX.rd = IFID.rt))$$

这表示当前处于执行级的加载指令还没有数据，而取指/译码级指令马上要读这个目标寄存器。停顿时 PC 和 IF/ID 不更新，ID/EX 注入 bubble，避免错误指令带着旧数据前进。

专题：目标寄存器 pending 表—scoreboard 检测冒险的最小逻辑（选读） 上一专题把冒险检测写成布尔式，工程上更常用的最小结构是一张“目标寄存器 pending 表”：每个可写寄存器配一个触发器，置 1 表示“已有一条在途指令以它为目标、结果尚未提交”。规则只有两条：译码拍末，凡 `reg_write=1` 的指令把自己的目标寄存器置位；写回拍末，对应位清除。译码拍内，把本指令的全部源寄存器号与 pending 表比对，任一为 1 就停顿—PC、IR 保持，指令滞留译码拍，且不做登记。比对时要按本章编码认清谁是源：ADD、SHL 是二操作数约定，d 字段既是目标也是源；存储 MOV 的 s 字段（写数据）是源；JZ 的 r 字段是源；MOV 立即数没有源寄存器。R0 固定为 0、永不作写目标，它的 pending 位恒为 0，比对可以直接省掉。

下面用四条指令把这张表走一遍。约定五级流水（IF/ID/EX/MEM/WB）逐拍推进、无转发、写回在 WB 拍末提交、译码用当拍沿前的 pending 值比对；序列为 `MOV R2, [R1+4]` (I1)、`ADD R3, R2` (I2)、`MOV R4, 1` (I3)、`SHL R4, 4` (I4)：

拍	译码拍内容	pending 表事件	判定与动作
2	I1 译码	源 R1 未置位	放行；拍末置 P2=1
3-5	I2 译码	源 R2 的 P2=1	停顿三拍，滞留译码拍，不登记
5 末	I1 写回 R2	清 P2=0	I2 本拍仍见旧值，下一拍再比对
6	I2 再次译码	P2=0	放行；拍末置 P3=1
7	I3 译码	无源寄存器	放行；拍末置 P4=1
8-10	I4 译码	源 R4 的 P4=1	停顿三拍，待 I3 写回
10 末	I3 写回 R4	清 P4=0	I4 本拍仍见旧值
11	I4 再次译码	P4=0	放行；拍末再置 P4=1 (SHL 的目标也是 R4)

这张表有两条边界约定必须写死。第一，置位与清除都发生在拍末（时钟沿），比对用拍内的当前值，所以写回当拍的目标寄存器仍算 pending—多停一拍是保守但安全的约定。第二，pending 表只回答“能不能走”，不回答“值是多少”：它维护提交顺序，不维护数据本身；想减少停顿，需要上一专题的转发网络把未提交的值直接送往 ALU 输入。意义在于规模：8 个触发器加几组比较器，就把“某个结果现在是否可用”变成可查表的硬件事实；上一专题 `IDEX.memRead` 那条布尔式只是这张表“只看最近一条在途指令”的特例。理解了 pending 表，后面再看记分牌调度、寄存器重命名时，认出的都是同一思想的更细版本。

专题：同一条加载指令在三种 CPU 组织中的对照 为了避免“单周期、多周期、流水线”只停留在名词层面，可以用同一条 `MOV R1, [R2+8]` 对照三种组织。它们完成的语义相同：读 R2，加立即数形成地址，访问存储器，把数据写回 R1。区别在于资源是否复用、状态保存在哪里、关键路径由谁决定。

组织方式	加载指令的执行形态	主要代价
单周期	取指、译码、寄存器读、地址加法、时钟周期被最慢指令拉长 存储读、写回全部在一拍完成	
多周期	IF、ID、EX 地址形成、MEM、WB 分拍完成	控制状态机复杂，但硬件可复用
流水线	不同加载指令位于不同阶段，阶段间用寄存器隔开	需要处理冒险、冲刷和阶段平衡

单周期设计最适合教学入门，因为一条指令的 trace 很直观；多周期设计更像早期现实 CPU 的控制方式，因为它允许慢步骤独占多个周期；流水线设计提高吞吐量，但不会让单条加载的物理工作消失。读者应能把三种设计画成状态边界，而不是只背“流水线更快”。

专题：控制器检查要同时覆盖正常路径和错误路径 CPU 设计是否扎实，不能只跑一段加法程序来判断。控制器必须面对非法 opcode、未对齐访问、慢存储器、分支冲刷、中断屏蔽、异常返回和复位。否则机器在正常程序上看似正确，一旦接入真实总线或设备就会暴露状态边界不清的问题。

场景	必须观察的状态	合格表现
非法 opcode	PC、IR、错误码或 halt 状态	不写内存，不错误提交寄存器
慢速加载	地址、mem_read、ready、MDR	等待期间请求稳定，只提交一次
分支命中 异常入口	IF/ID、ID/EX、PC 选择 故障 PC、状态寄存器、异常向量	错误路径指令被冲刷 可诊断、可恢复或明确停机
复位	PC、寄存器堆、控制状态机	无旧控制线残留，第一条取指确定

这一类检查会迫使实现者回答：哪个周期算提交？哪些控制线在等待时保持？错误路径是否会产生副作用？这些答案比“程序最后算出 42”更接近 CPU 工程。

章末练习

本章练习不要求先追求完整 ISA，而要求把每条指令写成可检查的硬件 trace。答案必须同时说明控制线、数据路径和提交边界。

任务	要完成的产物	检查点
PC 取指	写出 PC、指令存储器、IR 和 pc_write/ir_write 的周期关系	能说明哪一拍保存指令
ADD trace	为 ADD R1,R2 (即 R1=R1+R2) 列出读地址、ALU 操作、写回选择和写使能	写回只在目标时钟沿发生
加载 trace	把加载指令拆成地址形成、存储读、MDR 保存、寄存器写回	能处理存储器未 ready
存储 trace	说明地址、写数据、字节使能、写控制和无寄存器写回	能区分 RAM 写和 MMIO 副作用
JZ trace	说明 Zero 标志如何影响下一 PC	分支目标和顺序 PC 选择明确
条件码比较	分别解释有符号小于和无符号小于使用哪些 ALU 标志	不把 Carry、Sign 和 Overflow 混用
CALL/RET trace	把调用和返回拆成保存返回地址、更新栈指针、改写 PC 和恢复 PC	能指出栈访问本质上是加载/存储事务
控制字	设计一个包含 alu_op/wb_sel/reg_write/mem_read/mem_write/pc_sel 的控制字	每一位能追到实际电路
微程序	为加载指令写出 5 个微步骤和等待状态处理	慢存储器只拖住访存阶段
流水线冒险	说明结构、数据、控制三类冒险各破坏什么边界	能提出停顿、转发或冲刷策略
SHL 微操作	为 0x7260=SHL R1, 8 写出 T1-T5 各拍的微操作、控制线与提交对象	不经过 T4，写回只在 T5 提交

三种组织拍数	统计 ROM 前三条 (0x22F0、0x7260、0x3450) 在单周期、五拍多周期、理想五级流水下各用多少拍	拍数与周期长度分开讨论
新增 SUB	为 SUB Rd, Rs (R[d]=R[d]-R[b]) 写控制字行和逐拍流程, 说明数据通路是否要加新部件	复用既有 ALU 减法路径
cmp+jcc 配对	把 JZ R2, +3 改写成 x86-64 的 test/jz 两条指令, 说明标志位在两条指令之间的角色	标志是前写后读的约定状态
CPI 演算	某程序动态执行 100 条指令: 加载 20、存储 10、其余 70 条为 ADD/MOV 立即数/SHL/JZ/JMP; 按多周期拍数求总拍数、CPI 与 100 MHz 时钟下的执行时间	CPI 是动态混合的平均
pending 表读法	对 MOV R2,[R1+4]、ADD R3,R2、MOV R4,1、SHL R4,4 序列 (五级流水、无转发、写回在 WB 拍末提交) 写出 pending 表逐拍变化并标出停顿	置位在译码拍末、清除在写回拍末

参考答案与要点

任务	参考答案	必须保留的要点
PC 取指	PC 输出取指地址, 指令存储器返回编码, ir_write 在取指边界保存编码; 下一 PC 可由 pc+size、分支目标或异常入口选择。	PC 和 IR 都是状态元件。
ADD trace	读端口选择 R1/R2, ALU 选择 ADD, 写回 mux 选择 ALUOut, 写地址为 R1, reg_write=1 只在写回边界有效。示例: R1=5, R2=7 时 read_a=5、read_b=7、ALU_out=12, 时钟沿 R1<-12、PC=100->102。	算对不等于写对。
加载 trace	EX 阶段形成地址, MEM 阶段发起读, 目标未 ready 时保持地址和控制, 响应后保存 MDR, WB 阶段写回目标寄存器。	等待不能污染其他状态。
存储 trace	存储指令使用 ALU 形成地址, 源寄存器提供写数据, 打开字节使能和写控制, 不打开寄存器写回。若地址命中 MMIO, 副作用由设备规格决定。	存储是有副作用的事务。
JZ trace	比较或 ALU 结果产生 Zero, 控制器根据 Zero 选择顺序 PC 或分支目标; PC 只在提交边界更新。示例 (本章 ROM 程序): 地址 3 的 JZ +3 在 R2=0 时 PC=3+1+3=7; 同一指令在 R2=1 时顺序取地址 4。	条件输入必须在采样前稳定。
条件码比较	无符号小于看减法后的借位/Carry 约定 (结果可能为正仍算小于); 有符号小于看 Sign XOR Overflow (如 0x80-1: 结果为负但无溢出, 有符号小于成立)。相等判断只看 Zero。	标志必须绑定解释规则。

CALL/RET trace	CALL: 返回地址先压栈保存 (栈指针减小后写入栈顶), 再 $PC=目标$; RET: 从栈读回返回地址, 恢复栈指针, $PC=返回地址$ 。栈访问本质就是加载/存储事务, 栈指针是普通寄存器加约定。	调用链是事务序列, 不是语法。
控制字	控制字字段应覆盖 PC、IR、寄存器堆、ALU、存储器和下一状态逻辑; 任何字段没有连接对象都不是硬件控制。	控制字是电线集合, 不是注释。
微程序	加载指令可写为取指、读寄存器、地址形成、访存等待、写回; 访存未完成时微地址保持在等待状态, 错误时跳异常入口。	微步骤提供清晰的观察点。
流水线冒险	结构冒险来自资源争用, 数据冒险来自结果未写回, 控制冒险来自下一 PC 尚未确定。停顿保护边界, 转发减少等待, 冲刷丢弃错误路径。	提高吞吐不能破坏提交顺序。
SHL 微操作	T1: $IR \leftarrow MEM[PC]$, 取回 0x7260, $imem_read/ir_write=1$; T2: $A \leftarrow R1=0x00F0$, $imm4$ 读出 8; T3: $alu_op=左移$ 、 $b_sel=imm4$, $ALUOut \leftarrow 0x00F0 \ll 8 = 0xF000$; 跳过 T4; T5: $wb_sel=ALU$ 、 $reg_write=1$, 时钟沿 $R1 \leftarrow 0xF000$ 、 $PC \leftarrow PC+1$ 。	非访存指令不走 T4。
三种组织拍数	单周期 3 拍: 每条一拍, 但周期长度按最慢的加载路径定。五拍多周期: MOV 立即数 4 拍、SHL 4 拍、MOV 加载 5 拍, 共 $4+4+5=13$ 拍。理想五级流水 (无冒险、每拍流入一条): 第 5 拍完成第一条, 共 $3+5-1=7$ 拍。	比较组织要同时看拍数和周期长度。
新增 SUB	控制字与 ADD 同行, 仅 $alu_op=SUB$: $a_sel/b_sel=寄存器$ 、 $wb_sel=ALU$ 、 $reg_w=1$ 、 $mem_r/mem_w=0$ 、 $pc_sel=顺序$ 。流程同 ADD: T1 取指, T2 双读, T3 ALU 做减 (B 取反加一), 跳过 T4, T5 写回并 $PC+1$ 。数据通路不加新部件, 译码真值表加一行即可。	新指令优先复用既有路径。
cmp+jcc 配对	x86-64 写法: $test\ ecx, ecx$ (或 $cmp\ ecx, 0$) 按结果置 ZF, 再由 $jz\ target$ 读 ZF 决定是否跳转。标志寄存器是两条指令之间“前一条写、后一条读”的约定状态; 教学 JZ 把比较与跳转内部化为一条。两者的偏移都相对下一条指令计算。	标志位必须绑定解释规则。
CPI 演算	总拍数 $20 \times 5 + 10 \times 5 + 70 \times 4 = 100 + 50 + 280 = 430$ 拍; $CPI = 430/100 = 4.3$; 100 MHz 时钟下 执行时间为 $430/10^8$ 秒 $= 4.3\ \mu s$ 。	CPI 依赖指令混合, 不是硬件常数。

pending 表读 拍 2 I1 译码置 P2；拍 3-5 I2（读 R2）同拍比对用清除
法 停顿三拍，拍 5 末 I1 写回清 P2，拍 6 前的值。
I2 放行置 P3；拍 7 I3 译码置 P4；拍
8-10 I4（读 R4）停顿三拍，拍 10 末
I3 写回清 P4，拍 11 I4 放行并再置
P4。共停 6 拍，四条指令 14 拍完成；
无冒险理想流水为 8 拍。

最小 CPU 的经典读法是逐周期 trace。不要只画“PC 连到内存、ALU 连到寄存器”的框图；要写清楚每条指令在哪个周期读旧状态、在哪条组合路径上传播、在哪个时钟沿提交新状态。后续主存、总线、I/O 和经典芯片只是在这个 trace 外面继续增加事务边界。

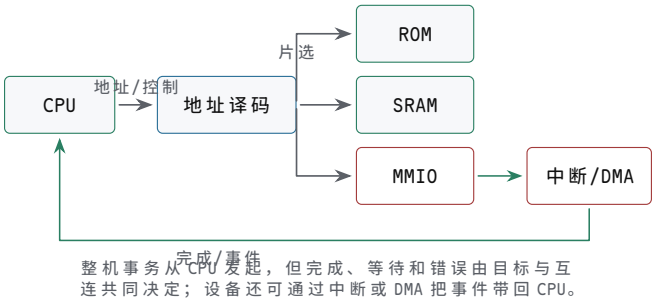
主存、总线、I/O 与 DMA

前面已经解释了最小 CPU 如何在内部取指、译码、执行和提交状态。若停在这里，读者只能得到一台“会在寄存器之间搬数”的小机器。真正的整机还需要大容量主存、可寻址设备、外部事件、等待、错误和批量数据搬运。本章专门补上这条桥：CPU 怎样把一个地址变成一次存储器或设备事务，设备怎样把完成事件通知 CPU，DMA 又怎样在不让 CPU 逐字搬运的情况下读写主存。

本章的目标不是背总线协议名字。无论是 8086 板级总线、微控制器片内总线、PCIe 设备，还是现代 SoC 互连，工程问题都相同：谁发起请求，谁响应地址，数据由谁驱动，什么时候采样，目标慢了怎样等待，目标不存在怎样失败，设备完成后怎样通知。把这些问题写清楚，才能从最小 CPU 走到可启动、可调试、可扩展的整机。

核心理念

整机硬件不是“CPU 加一块内存”的粗线图，而是由一组定义明确的事务组成。每次访问都必须说明地址、方向、字节选择、目标片选、握手、完成、错误和副作用。只有这些边界明确，CPU 才能可靠地取指、访存、访问设备和处理中断。



图示：从最小 CPU 到整机事务。地址译码决定目标，握手和事件决定本次访问何时真正完成。

一、主存不是抽象数组

软件把内存看成按地址编号的字节序列，硬件却必须用地线、译码器、存储阵列、数据线和控制信号实现这件事。一次读访问至少包含四个事实：发起者给出地址，译码逻辑选中目标，目标经过内部访问后驱动数据，发起者在规定边界采样。一次写访问则要求地址、写数据、字节使能和写控制在目标采样窗口内稳定。

对象	硬件含义	关键问题
地址	指出本次事务目标位置	高位译码是否只选中一个目标
数据	读时由目标返回，写时由发起者提供	同一时刻是否只有一个驱动器
读写控制	区分本次事务方向和类型	输出使能和写使能是否互斥
字节使能	指明哪些 8-bit 车道有效	字节写不会破坏相邻数据
ready/valid	指明请求或响应是否真正完成	慢目标能否插入等待而不丢地址和数据

ROM、SRAM 和 DRAM 的读写不是同一种动作 ROM 或 Flash 适合保存启动代码，掉电后内容仍在；SRAM 接口直观，地址稳定后在访问时间内返回数据，适合教学板、cache 和小容量片上 RAM；DRAM 密度高，但单元需要刷新，访问还要经过 bank、行、列、预充电和控制器调度。把这些对象都叫“内存”没有错，但在硬件设计中必须区分它们的时序和控制边界。

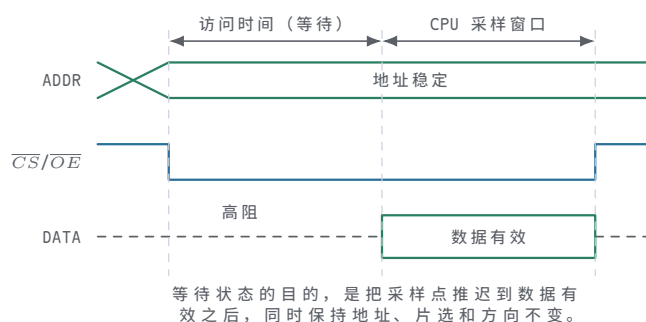
存储类型	典型用途	关键边界
ROM/Flash	复位入口、固件、常量表	复位地址必须映射到有效内容，读时序满足取指
SRAM	cache、实验板 RAM、小容量片上存储	CS/OE/WE 互斥，地址和数据满足建立时间
DRAM	大容量主存	控制器负责 activate、read/write、precharge、refresh
寄存器文件	CPU 内部近距离状态	多读端口、写端口、旁路和同周期读写规则明确

异步 SRAM 是最好的第一块主存 教学整机可以先使用低速异步 SRAM。读周期中，地址和片选稳定后，打开输出使能，SRAM 在访问时间后驱动数据线；写周期中，输出使能关闭，发起者驱动数据线，在写使能有效窗口内把数据写入目标字节。这个模型足够简单，也足以暴露硬件最常见错误：片选重叠、输出争用、写使能过早、字节使能错误。

```

SRAM read:
  address stable
  CS active, OE active, WE inactive
  wait access time
  sample data

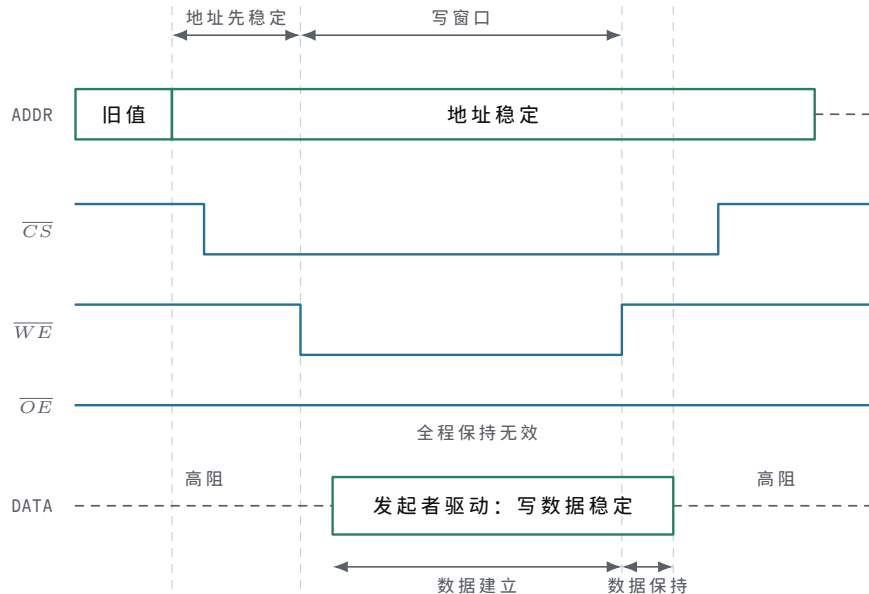
SRAM write:
  address and write_data stable
  CS active, OE inactive, WE active for write window
  selected byte lanes store data
  
```



图示：异步 SRAM 读周期：地址先稳定， $\overline{CS}/\overline{OE}$ 拉低后存储器需要一段访问时间才把数据驱动到总线上；CPU 只能在数据有效后采样。三条线各有明确的翻转时刻——读时序图读的就是这些边沿，而不是三条平直线。

SRAM 写周期的驱动权与写窗口 写周期与读周期使用同一组地址线、数据线和片选，差别在数据线的驱动方向和写使能窗口。读周期中 $\overline{OE}$ 有效，由 SRAM 驱动数据线，发起者只负责在数据有效后采样；写周期中 $\overline{OE}$ 必须保持无效，SRAM 的输出驱动器关闭，改由发起者把写数据驱动到数据线上——同一组数据线在任何时刻只能有一个驱动者。写使能 $\overline{WE}$ 拉低形成写窗口，规格书围绕它规定三条边界：地址必须在窗口打开前已经稳定，写数据必须在窗口关闭前满足建立时间，窗口关闭后还要继续保持一段保持时间，窗口本身也有最小宽度。这些要求来自同一个事实：

存储阵列在 $\overline{WE}$ 回升、窗口关闭的边沿附近把数据线上的值写入选中单元，窗口内任何一次地址或数据翻动，都可能把错误的值写进错误的单元。因此写周期的固定顺序是：先给地址和数据，再打开写窗口，先关窗口，最后才撤数据。

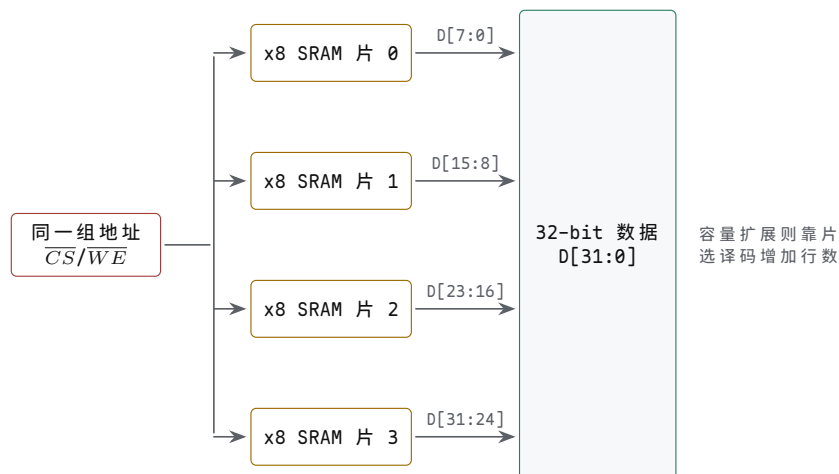


图示：异步 SRAM 写周期：地址与片选先稳定， $\overline{OE}$ 全程无效，发起者在写窗口内驱动数据线。存储阵列在 $\overline{WE}$ 回升、窗口关闭的边沿把数据线上的值写入选中单元，因此数据必须在窗口关闭前满足建立时间、关闭后满足保持时间，地址在整个窗口内不能翻动——与读周期对照，差别就在数据线由谁驱动、哪个边沿是真正的写入点。

DRAM 需要控制器而不是直接接地址线 DRAM 的容量来自更密集的存储单元，代价是接口复杂。控制器要先打开某个 bank 的某一行，再读写列；换行前需要预充电；即使没有普通访问，也要周期性刷新；高速 DRAM 还需要训练采样窗口。CPU 看到的是“主存读写”，但实际路径经过控制器、队列、bank 调度、行命中或行冲突。这也是为什么现代 CPU 必须靠 cache 把常用数据放近。

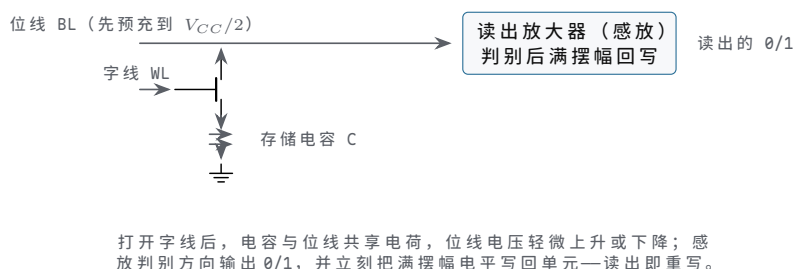
DRAM 动作	硬件含义	时延来源
Activate	打开某个 bank 中的一行	行译码和感放建立
Read/Write	在已打开行内选择列传输数据	列访问、突发传输和总线排队
Precharge	关闭当前行，准备访问另一行	位线恢复到可再次访问状态
Refresh	周期性恢复存储单元电荷	控制器必须暂停或调度普通访问

存储芯片的位宽扩展与容量扩展 单片存储芯片的位宽和容量都有限，整机用两组方向相反的手段补齐。位宽扩展解决“字不够宽”：把若干片芯片并接同一组地址线、片选和读写控制，每片只接数据线的的一个字节车道——四片 x8 SRAM 并接后，一次 32-bit 访问四片同时动作，各自收发 D[7:0]、D[15:8]、D[23:16]、D[31:24]，对外表现为一片 32-bit 宽的 SRAM。容量扩展解决“行不够多”：数据线和低位地址全部并接，新增的高位地址经译码器产生多路互斥片选，任一地址最多选中一片，行数随片数倍增。两种扩展经常同时使用，但边界必须分清：位宽扩展中各片同时被选中、各管一段数据线；容量扩展中各片互斥选中、数据线全部共用。混淆二者的直接后果，是两片同时驱动同一个字节车道的总线争用。



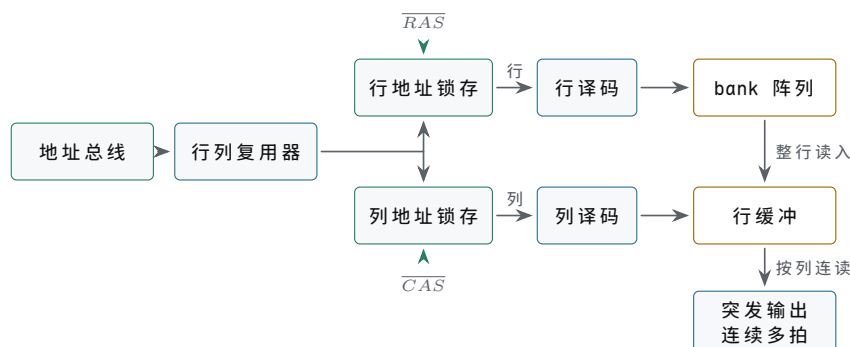
图示：位宽扩展：四片 x8 SRAM 并接同一组地址、 $\overline{CS}$ 和 $\overline{WE}$ ，同时动作、各管一个字节车道，对外是一片 32-bit SRAM。容量扩展方向相反：数据线全部并接，高位地址译码出互斥片选来增加行数，任一地址最多一片被选中。

DRAM 单元用电容存电荷，读出后必须重写 DRAM 的一个 bit 由一个访问晶体管和一个存储电容构成 (1T1C)。位线先预充到 $V_{CC}/2$ ；字线打开晶体管后，电容与位线共享电荷，位线电压略升或略降，读出放大器（感放）判别方向并输出 0/1。两个后果决定了 DRAM 的全部特殊性：其一，读出是破坏性的，感放必须把满摆幅电平经位线写回单元，否则原值丢失；其二，电容持续漏电，电荷在几十毫秒内就衰减到无法判别，控制器必须周期性刷新——逐行打开并重写。规格书里的 Activate、Precharge、Refresh 和刷新周期（典型要求 64 ms 内把全部行刷新一遍），都直接来自这两件事。



图示：1T1C 存储单元与读出放大器。刷新并不是额外功能，而是控制器替所有行周期性重复这个“读出-回写”过程；这也是 DRAM 不能像 SRAM 那样直接把地址线接上去用的原因。

DRAM 的行列地址复用与突发传输 DRAM 的地址引脚不随容量等比增长，靠的是行列地址复用：同一组地址引脚先送行地址、再送列地址。一个 2^{20} 字的阵列直接接地址需要 20 根地址线，复用后 10 根就够—— $\overline{RAS}$ 有效时把引脚上的值锁存为行地址， $\overline{CAS}$ 有效时把同一组引脚上的值锁存为列地址。省下的引脚直接降低封装成本，代价是接口从“给地址等数据”变成一串带时序要求的命令。行地址译码后打开整行，整行数据被感放读出并留在行缓冲中；此后的列访问只是从行缓冲里选一段送到数据总线。突发传输利用的正是这一点：给一个起始列地址，DRAM 按内部列计数器连续多拍输出相邻数据，后续各拍不再付出行打开的代价。行缓冲命中是突发便宜的根本原因，也是内存控制器尽量把相邻访问调度到同一行的理由。



同一组地址引脚在 $\overline{RAS}$ 、 $\overline{CAS}$ 两拍分别送出行、列地址；整行先读入行缓冲，列访问只是在缓冲里选段，行命中后的连续突发因此不必重开行。

图示：行列地址复用与突发传输。 $\overline{RAS}$ 拍锁存行地址并打开整行， $\overline{CAS}$ 拍锁存列地址；行缓冲命中后，连续列地址直接从行缓冲多拍输出，比重开一行便宜得多。

行打开、列传输和预充电之间的最小间隔由一组时序参数规定，规格书把它们写成以时钟拍为单位的数字。DDR3-1600 的内部时钟为 800 MHz，一拍 1.25 ns；1600 MT/s 的数据速率来自同一时钟的双沿传输，时序参数仍按时钟拍计。常见标注 11-11-11-28 就是下表四个数字。

参数	含义	DDR3-1600	纳秒换算
t_{RCD}	Activate 打开行之后，到第一个列读/写命令的最小间隔	11 拍	13.75 ns
t_{CL}	列读命令发出后，到第一个数据出现在总线上（CAS 延迟）	11 拍	13.75 ns
t_{RP}	Precharge 关闭当前行之后，到允许再次 Activate 的最小间隔	11 拍	13.75 ns
t_{RAS}	一次 Activate 到允许 Precharge 的最小行打开时间	28 拍	35 ns

换算很直接：拍数乘 1.25 ns。同一行内连续读，列延迟 13.75 ns 后就能紧跟突发数据；访问另一行则要先等 t_{RP} 再重新 Activate，一次行冲突至少多花 $t_{RP} + t_{RCD} = 22$ 拍，约 27.5 ns。这个数字就是内存调度器在乎行命中、软件在乎访问局部性的原因。

SDRAM 的突发长度与多 bank 交错 Activate 打开一行之后，控制器可以对同一行连发多个列读写命令；每个列命令传的也不是一个数据，而是按约定的突发长度 (burst length) 连续传一串——DDR3 固定为 8 (BL8)，也允许切成 4。DDR 的“双倍数据率”指时钟的上升沿和下降沿各传一次数据，而不是把时钟翻倍：DDR3-1600 的内部时钟是 800 MHz，数据速率 1600 MT/s，所以一次 BL8 突发占 4 个时钟拍、交 8 个数据，这期间数据总线被这一串独占。

bank 是 DRAM 内部可独立 Activate、独立 Precharge 的子阵列，各有自己的行缓冲。一个 bank 在等 t_{RCD} 、等数据的时候，命令总线可以对另一个 bank 发命令，把后者的行打开时间藏进前者的传输里，这就是多 bank 交错。沿用上一张表的 DDR3-1600 参数（1 拍 1.25 ns， $t_{RCD} = t_{CL} = t_{RP} = 11$ 拍， $t_{RAS} = 28$ 拍），比较“读两个不同行”在单 bank 与双 bank 下的时间轴：

单 bank，两次访问落在同一 bank 的不同行：

- 拍 0 ACT bank0 row0
- 拍 11 RD bank0 col0 (t_{RCD} 到期)
- 拍 22 数据出现在总线，BL8 占用拍 22-25
- 拍 28 PRE bank0 (t_{RAS} 到期才允许关行)
- 拍 39 ACT bank0 row1 (t_{RP} 到期)
- 拍 50 RD bank0 col1 (t_{RCD} 到期)
- 拍 61 第二组数据出现，共等 61 拍

双 bank 交错，两行分别在 bank0、bank1:

```

拍 0  ACT bank0
拍 1  ACT bank1      (不同 bank, 行打开并行进行)
拍 11 RD bank0       (数据占拍 22-25)
拍 15 RD bank1       (数据占拍 26-29, 恰好接上)
拍 26 第二组数据出现, 共等 26 拍

```

第二组数据从拍 61 提前到拍 26, 省 35 拍、约 43.75 ns。省下的不是任何一条时序参数—单 bank 必须串行等完 $t_{RP} + t_{RCD}$ 才能再读, 双 bank 让 bank1 的 Activate 与 bank0 的全过程并行, 这段等待被完全覆盖; 拍 15 才发第二条读命令, 是因为数据总线仍由两个 bank 共享, bank0 的突发占着拍 22-25, bank1 的数据最早只能接在拍 26。这就是内存条标注 bank 数、控制器把相邻地址散到不同 bank、调度器重排请求的原因: 行命中决定一次访问多快, bank 并行决定等待能不能被藏起来。

对照点	单 bank 顺序访问	双 bank 交错
第二组数据出现	拍 61	拍 26 (省 35 拍, 约 43.75 ns)
数据总线占用	拍 22-25、61-64, 中间大量气泡	拍 22-29 连续无气泡
关键等待	换行时 $t_{RP} + t_{RCD}$ 串行暴露	被另一 bank 的并行 Activate 覆盖
检查点	PRE 与 ACT 都卡在同一个 bank 上	命令错开, 数据在共享总线上排队衔接

组相联 cache 的替换策略是精度与代价的取舍 前面说过, 主存的时延是结构性的, 整机的应对是把热数据留在近处的 cache 里。本章后半部分会拆开 cache 的地址字段、写策略和一致性, 这里先回答放进 cache 之后必然面对的问题: 新行取回时组内已无空路, 牺牲哪一路? 直接映射没有这个问题, 每个地址只有唯一槽位; 组相联把它交给了替换策略。

精确 LRU (最久未使用) 记录组内各路的年龄顺序, 牺牲最老的一路, 命中率最好, 代价随路数急剧膨胀。2 路组相联时每组只需 1 位: 这一位指出哪一路更久未用, 每次命中取反即可。4 路要区分完整的年龄顺序, 共 $4! = 24$ 种排列, 每组需 $\lceil \log_2 24 \rceil = 5$ 位, 且每次命中都要重排年龄; 8 路则需 $\lceil \log_2 40320 \rceil = 16$ 位。伪 LRU 把 4 路组织成两层二叉树, 3 个内部节点各 1 位, 每组 3 位, 命中时沿路径翻转节点位; 它选出的不保证是最老路, 但命中率损失通常很小。FIFO 只记进入顺序, 命中时不更新任何状态; 随机替换连状态位都不要。

高组相联之所以普遍放弃精确 LRU, 原因有两头: 状态位按 $n!$ 增长还是小事, 更难办的是每次命中都要更新年龄, 路数越多更新逻辑的组合路径越长, 直接拖慢命中时间; 而 8 路、16 路的组里, “精确选最老”换来的命中率收益早已递减, 随机替换与 LRU 的差距常常小到测量噪声量级。工程上的结论因此很清楚: 2 路用 1 位精确 LRU, 4 路用 3 位伪 LRU, 更高组相联用伪 LRU 或随机。

策略	每组状态位 (4 路)	命中时是否更新	关键问题
精确 LRU	5 位 ($4! = 24$ 种年龄序)	每次命中重排年龄	路数增多后状态位和更新路径急剧膨胀
伪 LRU	3 位 (两层树节点)	沿树路径翻转节点位	选出的不保证是最老路, 命中率略降
FIFO	2 位 (轮流指针)	不更新	不看访问热度, 热行可能被轮到

随机

0 位

不更新

实现最简，大组相联下
命中率损失很小

硬件预取与软件预取都是拿带宽换命中 替换策略决定牺牲谁，预取则试图让牺牲更少发生：顺序扫描数组时，用完一行 cache 行，下一行几乎马上要用，空间局部性让“将来要访问什么”变得可预测。硬件预取器监视 miss 流的地址规律：next-line 预取在用到第 i 行时把第 $i+1$ 行取回；步长预取识别出固定步长后沿步长提前取回；预取流通常在 4 KiB 页边界停下——跨页地址可能未映射，为一次猜测触发缺页代价太大。

软件预取由指令显式给出提示。x86-64 提供 `prefetcht0`、`prefetchnta` 等预取指令：它们不改变任何架构可见状态，地址无效时也不触发缺页异常，只是提示硬件“这个地址很快会被读”。编译器或手写汇编常在循环里提前若干次迭代发出提示：

```
; rdi 指向当前元素，rsi 是数组结尾，每元素 8 字节
scan_loop:
    prefetcht0 [rdi + 512]    ; 提前 8 行(64 B 行)提示
    add     rax, [rdi]
    add     rdi, 8
    cmp     rdi, rsi
    jb      scan_loop
```

预取不是免费的：取错行会占住 cache 槽位、挤掉有用数据，每次预取本身也消耗内存带宽，过激的预取反而把真正需要的访问排在后面。可以粗算一笔账：顺序扫描大数组，行 64 B、元素 8 B，一行装 8 个元素；miss 时延按 200 拍、每个元素处理 4 拍计。无预取时每行一次 miss，每行花 $200+32=232$ 拍；预取把“取下一行”与“处理当前行”重叠，若提前量达到 $200\div32\approx7$ 行，处理完当前行时下一行已经在 cache 里，稳态每行只剩 32 拍，miss 被完全隐藏，吞吐提高约 $232/32\approx7.3$ 倍。若预取只能提前一行，32 拍的处理盖不住 200 拍的取回，每行仍暴露约 168 拍。提前量必须匹配“处理一行的耗时”与“取回一行的耗时”之比，这正是预取器设计的核心数字。

预取方式	触发时机	关键问题
next-line 硬件预取	用到第 i 行时取第 $i+1$ 行	顺序流效果好，随机访问纯属浪费
步长硬件预取	识别出固定地址步长后沿步长取	步长误判会持续污染 cache
软件预取指令	程序在用到数据之前显式提示	提示距离要折算成拍数，地址须确实会用
跨页停止	预取流在 4 KiB 页边界停下	猜测性访问不得触发缺页副作用

上电自检的走步与棋盘格各查一类故障 主存焊上板不等于可信：某位固定读成 0 或 1 (stuck-at)、相邻数据线或地址线短路、写入后状态维持不住（数据保持故障），都可能让“写一个值再读回来”这种最简单的测试侥幸通过——它只验证了大部分位在此刻、对这组值是可写的。上电自检用图案化的写入-读回序列把故障类别分开：走步 (walking 1/0) 每次只让一位与众不同，棋盘格 (checkerboard) 让相邻位恒取反值。

走步 1 先写全 0 背景，再对同一单元依次写 0x01、0x02、.....、0x80，每步读回比对，并复查其余单元仍是背景值。某位固定 0 会在轮到它走 1 时露馅，固定 1 会在背景检查中露馅；两位短路时走步值会变形——写 0x01 可能读回 0x03，直接指认相邻两位被短接。走步 0 把背景换成全 1、逐位走 0，对称覆盖另一半故障方向。棋盘格把 0x55 (0101 0101) 写满读回，再换成 0xAA (1010 1010)：任何相邻两位恒取反，相邻短路或位间耦合会让图案当场变形；保持故障则靠“写入后停一段

再读”暴露。这些图案便宜、确定、能在无操作系统的裸机上运行，查不出所有故障，但恰好覆盖焊接与走线最常出的几类，因此是板级 bring-up 的固定节目。

以 4 字节小内存 (addr0-addr3, 8-bit 字节) 为例，一次走步 1 加棋盘格的完整步骤如下。

步骤	操作	预期结果	检查点
1	addr0-addr3 全写 0x00，逐字节读回	全部 0x00	无固定 1，背景可写可读
2	addr0 依次写 0x01、0x02、.....、0x80，每步读回	读回恒等于写入	每个数据位都能独立置 1
3	步骤 2 每步之后读 addr1-addr3	仍为 0x00	无地址串扰、无相邻线短路
4	addr1-addr3 重复步骤 2、3	同上	逐字节覆盖全部单元
5	全写 0x55 读回，再全写 0xAA 读回	图案不变形	相邻位恒反，查相邻短路
6	背景改 0xFF，逐位走 0 (0xFE、0xFD、.....)	读回恒等于写入	对称覆盖固定 0 故障

校验位与 ECC 把位错从灾难降级为事件 主存的位会错：宇宙射线、电压毛刺、保持时间失守都可能翻转一个 bit。最便宜的对策是奇偶校验—每字节加 1 个校验位，使连同校验位在内的 1 的个数恒为偶数（偶校验）。数据 1011 0001 已有 4 个 1，校验位记 0；读回时若某位翻转，1 的个数变奇，硬件立刻知道出错了。奇偶校验的边界同样明确：只能发现奇数个位错，两位同时翻转就漏检；即使发现，也不知道错在哪一位，只能报错，不能纠正。

汉明码用多个校验位换取定位能力。(7,4) 汉明码把 4 个数据位和 3 个校验位排成 7 位，位置从 1 编号；校验位放在编号为 2 的幂的位置 (1、2、4)，数据位占据 3、5、6、7。把位置编号写成 3 位二进制，第 i 个校验位负责所有编号第 i 位为 1 的位置。每个校验组各做一次偶校验，哪几组失败，其编号拼起来恰好就是出错位置的编号—这个把“哪些组错”翻译成“哪里错”的数称为校正子 (syndrome)。

位置编号	存放内容	参与校验的校验位	说明
1	p_1	p_1 自身	编号 001，最低位为 1
2	p_2	p_2 自身	编号 010
3	d_1	p_1 、 p_2	编号 011
4	p_4	p_4 自身	编号 100
5	d_2	p_1 、 p_4	编号 101
6	d_3	p_2 、 p_4	编号 110
7	d_4	p_1 、 p_2 、 p_4	编号 111

算一遍：数据位 (位 3、5、6、7) 依次为 1、0、1、1。 p_1 覆盖位 1、3、5、7，其中数据 1、0、1 共两个 1， $p_1 = 0$ ； p_2 覆盖位 2、3、6、7，数据 1、1、1 共三个 1， $p_2 = 1$ ； p_4 覆盖位 4、5、6、7，数据 0、1、1 共两个 1， $p_4 = 0$ 。发出码字 (位 1 至 7) 为 0 1 1 0 0 1 1。若位 6 在途中翻成 0： p_1 组 (0,1,0,1) 共两个 1，通过； p_2 组 (1,1,0,1) 共三个 1，失败； p_4 组 (0,0,0,1) 共一个 1，失败。校正子按 $p_4p_2p_1$ 读出 (1,1,0) = 6，直接指向位 6，翻转回来即完成纠正。

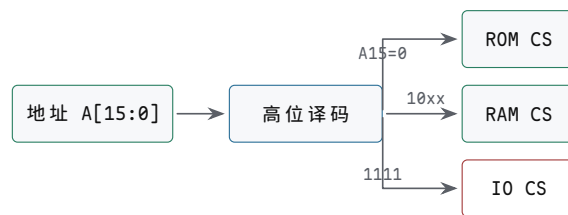
实用的 ECC 内存把同一思想做到 64 位数据加 8 位校验、72 位宽，实现单错纠正、双错检测 (SEC-DED)。贵在三处：阵列宽了 1/8，每条内存要多挂芯片；控制器对每次读做译码纠正，对部分写先读-改-写再重算校验，多一层逻辑和时延；平台还要为纠错路径做完整验证。对长时间运行、数据不能错的机器这是便宜的保险，桌面机则普遍省掉—这也是普通内存条与 ECC 内存条不能随意混插的原因。

二、地址译码定义整机地图

CPU 输出的是地址数值，整机必须规定这个数值由哪个目标响应。这个规定叫地址映射。一个简单教学板可以把低地址映射到 ROM，中间映射到 RAM，高地址小窗口映射到 GPIO、定时器和串口等设备寄存器。地址译码器把高位地址和访问类型变成片选信号。

example 16-bit address map:

```
0x0000-0x7FFF  boot ROM
0x8000-0xEFFF  SRAM
0xF000         GPIO_OUT
0xF004         GPIO_IN
0xF010         TIMER_COUNT
0xF014         TIMER_CTRL
```



设计目标：任意一个地址最多选中一个目标；
未映射区域也要有错误、固定值或超时规则。

图示：地址译码不是软件表格，而是片选电路。片选重叠会造成总线争用或误写设备。

映射对象	响应规则	典型错误
ROM	复位入口和只读代码地址命中	复位后 CPU 取不到第一条指令
RAM	普通数据读写地址命中	写使能误打到 ROM 或设备窗口
GPIO	设备寄存器地址命中并产生副作用	地址译码过宽，普通内存写误触发外设
未映射区	返回错误、超时或固定空值	访问悬空，CPU 永久等待或读到随机总线值

片选表达式就是硬件约定 若地址空间为 64 KiB，可以用高位地址产生粗粒度片选。例如 $A_{15}=0$ 选 ROM， $A_{15}=1, A_{14}=0$ 选 RAM， $A_{15}=1, A_{14}=1, A_{13}=1, A_{12}=1$ 选 I/O 窗口。无论用门电路、译码器、CPLD 还是 FPGA 实现，都必须证明任意合法周期最多只有一个目标片选有效。

```
rom_cs = !A15
ram_cs = A15 && !A14
io_cs  = A15 && A14 && A13 && A12
```

这三个表达式看起来像逻辑练习，实际上就是整机安全边界。若 ROM 和 RAM 可同时片选，读周期可能出现两个芯片同时驱动数据线；若 RAM 和 GPIO 片选重叠，普通 STORE 可能既改内存又改外设输出；若未映射地址没有超时或错误返回，CPU 可能等待一个永远不会响应的目标。

时序预算决定是否需要等待状态 地址译码需要时间，存储器访问需要时间，数据到达 CPU 输入端后还要满足建立时间。若这些时间总和超过时钟周期，就必须降低频率、换更快器件，或插入等待状态。等待状态不是软件延时，而是硬件事务延长：地址、片选、读写方向和数据驱动关系必须在等待期间保持可解释。

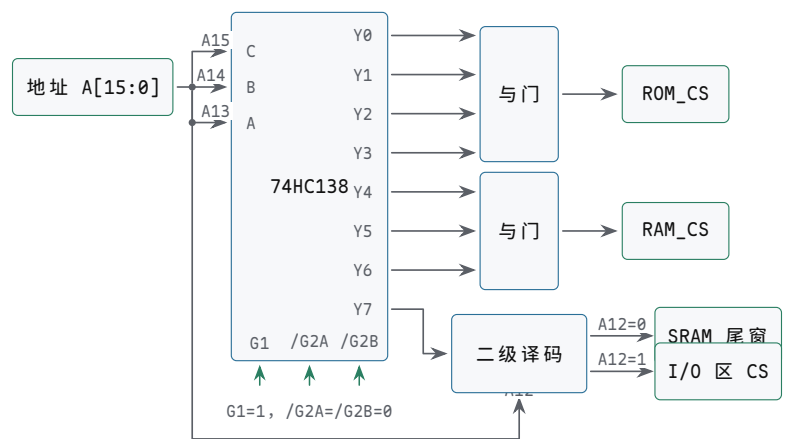
```
read_budget =
```

```
cpu_address_output_delay
+ address_decode_delay
+ memory_access_time
+ input_setup_time
+ board_margin
```

若 `read_budget` 大于一个周期，控制器就不能在下一拍采样数据。慢 ROM、慢外设和长板级走线都可能需要 READY/WAIT 机制。正式规格还应说明最大等待长度和超时行为，避免坏设备把整机永久挂住。

处理方式	适合场景	代价
降低时钟	全系统都慢，教学板容易观察	所有操作都变慢
更快器件	SRAM 或译码器成为瓶颈	成本、功耗或电平兼容性变化
插入等待	只有部分目标较慢	控制器必须保持事务状态
超时返回错误	目标可能不存在或失效	需要错误状态和恢复入口

案例：用 74HC138 译出整机片选 本节开头给出了 16-bit 地图：`0x0000-0x7FFF` 是 boot ROM (32 KiB, `A15=0`)，`0x8000-0xEFFF` 是 SRAM (28 KiB)，`0xF000` 起是 I/O 区 (4 KiB)。现在给出这张地图的引脚级实现。最经典的器件是 3-8 译码器 74HC138：它有三个地址输入 C、B、A 和八个低有效输出 `Y0-Y7`，任一时刻只有一个输出为低。把 `A[15:13]` 依次接上 C、B、A，每个输出就恰好管辖一个 8 KiB 窗口，窗内偏移由 `A[12:0]` 决定；使能端 `G1` 接高电平、`/G2A` 与 `/G2B` 接低电平，译码器才工作——这两个使能端也常改接总线读写选通，让片选只在存储周期内出现。输出低有效带来一个方便的性质：把若干相邻窗口合并成一个片选只需一个与门，任何一个被选中的输出变低，与门输出即为低。于是 `Y0-Y3` 经四输入与门合成 `ROM_CS`，正好覆盖 `A15=0` 的整个低半空间；`Y4-Y6` 经三输入与门合成 `RAM_CS`；`Y7` 则送入二级译码，由 `A12` 把 `0xE000-0xFFFF` 再分成 SRAM 尾窗和 I/O 区，合起来正好复现上面这张地图。



图示：引脚级地址译码。`A[15:13]` 经 74HC138 译成 8 个 8 KiB 窗口（输出低有效），与门把连续窗口合成 `ROM_CS` 与 `RAM_CS`，`Y7` 窗口由 `A12` 二级译码再分；使能端决定译码器何时工作。

输出	C B A (<code>A[15:13]</code>)	地址区间	去向
<code>Y0</code>	000	<code>0x0000-0x1FFF</code>	boot ROM 第 0 页
<code>Y1</code>	001	<code>0x2000-0x3FFF</code>	boot ROM 第 1 页
<code>Y2</code>	010	<code>0x4000-0x5FFF</code>	boot ROM 第 2 页
<code>Y3</code>	011	<code>0x6000-0x7FFF</code>	boot ROM 第 3 页
<code>Y4</code>	100	<code>0x8000-0x9FFF</code>	SRAM 第 0 页

Y5	101	0xA000-0xBFFF	SRAM 第 1 页
Y6	110	0xC000-0xDFFF	SRAM 第 2 页
Y7	111	0xE000-0xFFFF	二级译码：A12=0 归 SRAM 尾窗 (0xE000-0xEFFF)，A12=1 为 I/O 区 (0xF000-0xFFFF)

这套译码仍然是部分译码：I/O 区里只有 A12 和少数低位参与选择，A[11:5] 等地址位没有进译码器。结果是同一个设备寄存器会在整个 4 KiB 窗口里反复出现——0xF010 的定时器寄存器，在 0xF110、0xF210 等只改变未译码位的地址上同样可见，这些重复地址称为别名。SRAM 一侧同理：若实际芯片容量小于所属窗口，未译码的高位也会让同一个存储单元出现在多个地址上。部分译码用更少的门和更短的延迟换来足够大的窗口，在小系统里是合理的工程选择；但它把一道边界推给了规格：别名是否允许必须写明。允许别名，软件就不得用未译码位区分设备，调试时也不能从“访问了哪个地址”反推“选中了哪个设备”；不允许别名，就必须补足低位译码，让未映射组合返回错误或固定值，而不是把一次写错的访问悄悄变成对某个真实设备的读写。

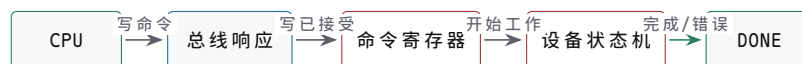
三、MMIO 把设备寄存器放进地址空间

设备寄存器可以像内存地址一样被 CPU 读写，但它们不是普通 RAM。读状态寄存器可能清除某些位，写命令寄存器可能启动外设动作，写数据寄存器可能进入 FIFO，读数据寄存器可能弹出一个元素。这类地址称为内存映射 I/O，简称 MMIO。MMIO 的关键不是地址长得像内存，而是每个寄存器都有副作用和顺序规则。

设备寄存器	读写语义	关键问题
DIR	设置 GPIO 引脚方向	输出引脚和输入引脚不会混用
OUT	写输出锁存器	写入只改变目标输出位
IN	读取同步后的外部输入	按键抖动和亚稳态不能直接进入 CPU
STAT	返回 ready、busy、error、irq 等状态	清除规则不会丢掉同时到来的新事件
DATA	数据收发寄存器或 FIFO 端口	空读、满写和溢出有明确定义

MMIO 写响应不等于设备已经完成 CPU 写一个命令寄存器，通常只表示设备已经接收命令。设备真正完成可能要等若干周期、若干微秒，甚至更久。完成可以通过状态位、轮询、DMA completion 或中断通知。若把 MMIO 写返回当作“设备动作完成”，软件会在设备尚未写完缓冲区、尚未刷新状态或尚未清除错误时继续执行，形成难复现的硬件错误。

```
device command sequence:
CPU writes COMMAND register
bus write response returns
device starts internal work
device sets DONE or ERROR
interrupt or polling tells CPU to inspect result
```



总线写响应只证明命令到达设备接口；真正完成必须看状态位、中断或 DMA completion。

图示：MMIO 写命令的两层完成边界。总线完成不等于设备动作完成。

端口 I/O 与 MMIO 是两种地址约定 8086 家族保留了独立 I/O 地址空间，IN 和 OUT 指令访问端口；许多 RISC 和单片机系统则把设备寄存器映射进普通地址空

间，用 LOAD/STORE 访问。两者都能控制设备，但硬件约定不同。端口 I/O 需要 I/O 周期和端口地址译码；MMIO 需要内存地址译码和内存属性规则，例如不可缓存、不可合并或必须保持顺序。

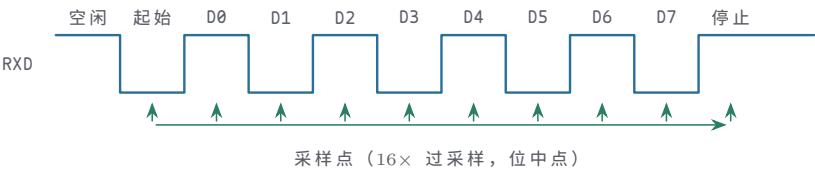
外部输入必须先调理和同步 按键、传感器、串口线和外部中断引脚都不按 CPU 时钟整齐变化。按键还会抖动，异步输入还可能让触发器进入亚稳态。教学板上的 GPIO 输入至少应经过电平保护、上拉/下拉、去抖策略和同步触发器，再进入设备寄存器。否则 CPU 读到的不是稳定硬件事实，而是未定义的边界行为。

不要把设备寄存器当作普通变量。普通 RAM 的读写目标是保存数据；MMIO 的读写目标是驱动硬件状态机。编译器、cache、写缓冲和乱序执行都可能改变普通内存访问的时机，但设备访问通常需要不可缓存属性、顺序约束和明确的完成检查。

从 GPIO、定时器和 UART 看设备寄存器 设备寄存器最好用具体布局理解。一个 GPIO 控制器可以有输出寄存器、输入寄存器、方向寄存器和中断状态寄存器；定时器可以有计数值、比较值、控制位和中断清除位；UART 可以有发送数据、接收数据、状态和波特率配置寄存器。CPU 访问这些寄存器时走的是地址事务，但设备内部看到的是控制状态机输入。

设备	寄存器边界	常见问题
GPIO	DIR/OUT/IN/IRQ_STAT	输入是否同步，输出是否只改目标位
定时器	COUNT/CMP/CTRL/IRQ_CLEAR	到期事件是否锁存，清除是否丢新事件
UART	TX/RX/STAT/BAUD	空读、满写、错误位和中断顺序是否清楚

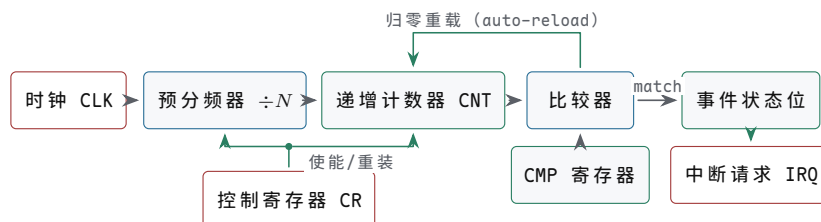
UART 帧格式与波特率约定 UART 没有时钟线，收发双方只靠“每一位持续同样的时间”这条约定保持对齐。线路空闲时停在高电平；发送方先用一个位时间的低电平打出起始位，再把 8 个数据位按 LSB 在前依次驱动上线，最后用至少一个位时间的高电平作为停止位。接收端用 16 倍于波特率的本地时钟对线路过采样：检测到下降沿后数 8 个采样周期，在起始位中点确认它仍是低电平，随后每隔 16 个采样周期在各位中点读取一次。中点采样让判决位置离前后边沿各半位，这正是异步串行能够容忍时钟误差的来源。



图示：UART 一帧（数据 0x55，LSB 在前）。空闲为高电平；起始位的下降沿给出帧起点，数据位在中点被逐位采样，停止位把线路带回高电平。

以 115200 baud 为例，每位宽度 $1/115200 \approx 8.68 \mu s$ 。接收端在起始位边沿对齐之后，最后一位（停止位）的采样点距该边沿 9.5 个位时间；若收发时钟的相对误差为 ϵ ，这个采样点会漂移 9.5ϵ 位，漂移超过半位即采到相邻位，所以理论上限约为 $0.5/9.5 \approx 5\%$ 。工程上还要从中扣除边沿检测的量化误差（16 倍过采样下最多 1/16 位）与电平翻转时间，留下的常用要求是每一侧波特率误差不超过约 2%，或者收发双方干脆从同一时钟源分频。误差超限时，出错的往往不是帧首而是帧尾——越早的位漂移越小，最后的停止位最先越界，表现为间歇性的帧错误或数据错位。

定时器的预分频、计数与比较匹配 定时器做的事只有一件：把固定频率的时钟变成软件可编程的周期间隔。它的数据路径很简单——预分频器先把输入时钟除以 N ，递增计数器 CNT 在每个分频后时钟上加一，比较器持续比较 CNT 与比较寄存器 CMP；当 CNT 数到 CMP，比较器输出 match。match 同时驱动两件事：置起事件状态位（若控制寄存器 CR 中的中断使能有效，同时拉起 IRQ），以及在 auto-reload 模式下把 CNT 清零，让计数立刻开始下一轮。于是中断周期严格等于（预分频 $\times$ (CMP+1)）个输入时钟——CMP 要加一，因为计数从 0 开始，数到 CMP 共经过 CMP+1 拍。以 1 MHz 输入、预分频 8 为例：分频后的计数时钟是 125 kHz，若 CMP=999，周期为 $8 \times (999 + 1) = 8000$ 个输入时钟，即 8 ms (125 Hz)；要得到 1 kHz 中断，应令 CMP=124，此时 $8 \times (124 + 1) = 1000$ 个时钟，正好 1 ms。事件状态位由软件按设备规则清除，清除顺序属于 § 四要讨论的中断约定。



图示：定时器数据路径：时钟经预分频驱动递增计数器，比较器在 CNT 等于 CMP 时产生 match，一路置事件状态位并按 CR 的中断使能拉起 IRQ，一路按 auto-reload 把 CNT 归零，开始下一周期。中断周期 = 预分频 $\times$ (CMP+1) 个输入时钟。

I2C 用两根开漏线支撑一条多设备总线 I2C 只有两根信号线：串行数据线 SDA 和串行时钟线 SCL。两根线都是开漏输出加上拉电阻——任何设备都只能把线拉低，不能主动驱动为高，线电平相当于所有设备输出的“线与”。空闲时两线靠上拉电阻停在高电平；主设备产生全部时钟，数据位只允许在 SCL 为低时改变，在 SCL 为高时被采样。事务以起始条件开头——SCL 保持高电平时 SDA 从高跳低；以停止条件结尾——SCL 保持高电平时 SDA 从低跳高。每个字节由 8 个数据位加 1 个应答位组成：第 9 个时钟里发送方释放 SDA，接收方把 SDA 拉低表示 ACK，保持高表示 NACK。寻址用 7 位设备地址，与 1 位读写方向合成地址阶段的一个字节。开漏结构正是这条总线能够共享的原因：多主同时发送时，发 0 的一方自然赢过发 1 的一方，竞争不需要额外仲裁线；从设备还可以在字节之间把 SCL 按住不放，强制主设备等待（clock stretching），慢设备因此不会丢数据。

UART 是点对点链路，没有地址概念；I2C 把多个设备挂在同一对线上，用起始条件和地址字节决定本次事务跟谁说话。下表对照二者的边界。

方面	I2C	UART
拓扑	一条共享总线挂多个主从设备	点对点，一发一收
时钟	主设备产生 SCL，收发同步于时钟沿	无时钟线，靠双方约定的波特率
寻址	起始条件后发送 7 位地址加读写位	无地址，线路上只有一个对端
确认	每字节后跟 ACK/NACK，主设备能发现无应答	无应答机制，只能靠上层协议
线数与速度	2 线；标准 100 kHz、快速 400 kHz	2 线收发交叉；常见 115200 baud

一次典型的“主发地址、寄存器号、再读回数据”事务如下，传感器和 EEPROM 的随机读几乎都是这个形状：

```

I2C register read transaction:
START
master sends [7-bit address + W] -> slave ACK
  
```

```

master sends [register number]    -> slave ACK
repeated START
master sends [7-bit address + R]  -> slave ACK
slave sends [data byte]          -> master NACK
STOP

```

中间的重复起始 (repeated START) 把“先告诉对方读哪个寄存器”和“再开始读”合并成一次不间断事务，避免其他主设备在两段之间插入；读方向下最后一个字节由主设备回 NACK 再发停止条件，通知从设备释放 SDA、结束发送。这次事务的开销可以直接算：一帧 9 位，共 4 帧 36 位，未计起始和停止条件的少量时间；标准模式 100 kHz 下每位 $10\mu\text{s}$ ，合计约 $36 \times 10 = 360\mu\text{s}$ ，快速模式 400 kHz 下约 $90\mu\text{s}$ 。也就是说，主设备每 0.36 ms 才能完成一次这样的寄存器读取，这正是 I2C 传感器数据更新率的量级来源；I2C 的意义不在于快，而在于只用两根线就把一整板低速设备纳入同一个地址空间。

SPI 用片选换速度，用移位环换全双工 SPI 是四线同步串行接口：SCLK 时钟、MOSI 主出从入、MISO 主入从出、CS 片选（低有效）。总线上没有地址阶段——主设备想跟谁通信就把谁的片选拉低，同一时刻只允许被选中的从设备驱动 MISO。数据通路是一对首尾相接的移位寄存器：每来一个时钟沿，主设备从 MOSI 移出一位，同时从 MISO 移入一位，收发在物理上永远同时进行，全双工是这个结构的自然结果而不是协议选项。时钟行为由两个参数规定：CPOL 决定空闲时 SCLK 停在高还是低，CPHA 决定在第几个时钟沿采样数据，组合出四种模式。通信双方必须事先约定同一种模式——这条协商写在芯片手册里，不在总线上进行。

模式	CPOL	CPHA	空闲电平	数据采样沿
0	0	0	低	第一个沿（上升沿）
1	0	1	低	第二个沿（下降沿）
2	1	0	高	第一个沿（下降沿）
3	1	1	高	第二个沿（上升沿）

以最常用的模式 0 读一个字节为例：主设备先拉低 CS；从设备把数据的最高位驱动到 MISO（发送方在时钟下降沿换位）；主设备发出 8 个时钟，在每个上升沿采样 MISO，依次得到 D7 到 D0；与此同时主设备把 MOSI 上的 8 位移给从设备，纯读时通常发全 0 占位字节；8 拍结束后主设备拉高 CS，事务结束。

```

SPI mode-0 byte read (MSB first):
CS low
for bit = D7 downto D0:
    slave changes MISO on falling edge of SCLK
    master samples MISO on rising edge of SCLK
    master shifts one dummy bit out on MOSI
CS high

```

SPI 没有 ACK，也没有起始停止开销，8 个时钟就是 8 位；代价写在引脚数量上，每多一个从设备多一根片选。与 I2C 对照，两者在速度、引脚和协议复杂度上各站一端：

方面	SPI	I2C
速度	常见数 MHz 到数十 MHz	标准 100 kHz，快速 400 kHz
引脚	4 线，且每个从设备一根片选	2 线，设备数量不占引脚
寻址	无地址，靠片选	7 位地址加读写位
确认与等待	无 ACK，无从设备等待机制	每字节 ACK/NACK，从设备可拉住 SCL 等待

协议复杂度	几乎只有移位，模式须事先约定	起始、停止、重复起始、应答状态机
-------	----------------	------------------

速度差距可以直接算：SCLK 取 10 MHz 时位时间 100 ns，读一个字节恰需 8 拍 = 0.8 μ s；I2C 快速模式 400 kHz 下位时间 2.5 μ s，一个字节加应答共 9 位 = 22.5 μ s，相差约 28 倍。Flash、显示屏、ADC 这类数据量较大的从设备多用 SPI，原因就在这里；而传感器这类低速、多节点、布线路径长的场合，I2C 的两线共享更省板级资源。可见两条总线没有优劣之分，只有“用引脚和约定换什么”的取舍不同。

USB 用枚举和描述符换来“通用”二字 UART、SPI、I2C 的共同前提是主设备事先知道对端是什么；USB 把这个前提取消了。总线严格以主机为中心，设备不能主动发起任何事务。插入检测靠电气约定：设备端在 D+ 或 D- 上接上拉电阻，全速设备拉 D+，低速设备拉 D-，集线器看到哪条线被拉高就知道来了什么速度的设备（高速设备先以全速现身，在复位期间再协商提速），并把端口状态变化上报主机。随后主机执行枚举：先复位该端口，设备以默认地址 0 应答；主机通过默认控制端点 0 读取设备描述符，再发 Set Address 给设备分配 1 到 127 之间的新地址；之后按新地址继续读取配置描述符、接口描述符、端点描述符和字符串描述符。描述符是一套标准格式的自描述数据：厂商 ID、产品 ID、设备类、需要多少端点、每个端点的类型和最大包长，操作系统据此匹配驱动。插拔能自动识别，不是因为总线聪明，而是因为每台设备都能用同一种格式回答“我是谁、我要什么”；“通用”二字，正来自枚举和描述符这套约定。

枚举完成后，数据按端点组织。端点是设备内部的单向通道，端点 0 固定留给控制传输，其余端点在描述符中声明自己的传输类型：

传输类型	语义	典型设备
控制	有应答、用于配置和命令，端点 0 专用	所有设备的枚举阶段
中断	主机按声明周期轮询，小数据量，出错重试	键盘、鼠标
批量	利用剩余带宽，无周期保证，出错重传	U 盘、打印机
等时	每帧预留带宽，不重传，容忍少量错误	音频设备、摄像头

把四种总线放在一起，复杂度阶梯很清楚：

总线	线数	寻址方式	设备自描述	典型定位
UART	2	无，点对点	无	调试口、模块间低速链路
SPI	4，每从设备加 1 片选	片选	无	板内高速从设备
I2C	2	7 位地址	无	板内多节点低速设备
USB	4（含电源和地）	枚举分配 7 位地址	标准格式描述符	机箱外可热插拔外设

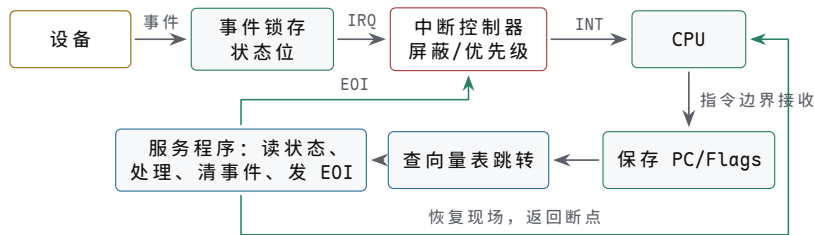
带宽数字能说明“通用”的另一面：高速模式 480 Mb/s 折合每微帧 125 μ s 约 7500 B，批量端点最大包 512 B，一个微帧最多容纳 13 个整包，理论上限约 $13 \times 512 \text{ B} \div 125 \mu\text{s} \approx 53 \text{ MB/s}$ 。同一条总线既要服务每 10 ms 被轮询一次、一次只发几个字节的键盘，又要服务成批搬数据的 U 盘，靠的正是把调度规则写进四种传输类型，而不是为每类设备各做一种接口。

四、中断和 DMA 把外部事件接回 CPU

轮询可以让 CPU 一直读状态寄存器，直到设备 ready。但若设备很慢，CPU 会浪费大量周期。中断提供另一种路径：设备在状态变化后请求中断，中断控制器汇总、屏蔽和排序请求，CPU 在可接受边界保存当前执行上下文，跳到中断入口，服务程序读取设备状态并清除事件。

对象	硬件职责	关键问题
设备	产生完成、错误或数据到达事件	状态位和中断请求是否同步一致
中断控制器	汇总、屏蔽、优先级和投递	屏蔽位、优先级、EOI 顺序是否正确
CPU 入口	保存边界状态并跳到入口	PC、标志和特权边界是否可恢复
服务程序	读状态、搬数据、清除事件	不丢新事件，不重复进入同一旧事件

中断链路从设备事件一直到服务程序返回 中断把“CPU 主动轮询”反转为“设备主动请求”。完整链路是：事件先在设备内锁存成状态位并拉起 IRQ；中断控制器按屏蔽位和优先级决定是否投递；CPU 只在指令边界采样 INT，接收后保存 PC/Flags，查向量表跳到服务程序；服务程序读状态、处理数据、按设备规则清事件，最后发 EOI 并恢复现场返回。链路上任何一环顺序错误，都会表现为丢中断或同一事件重复进入。



图示：中断的完整链路。实线是请求与服务路径，虚线是返回路径：EOI 让中断控制器恢复投递，恢复现场让 CPU 回到被打断的位置。服务程序必须按设备规定的顺序清事件—先清设备还是先 EOI，是每种芯片数据手册都会写明的细节。

中断清除顺序是硬件约定 若服务程序先清设备状态，再清中断控制器，或先通知中断控制器再清设备，效果可能不同。某些设备要求先读状态后读数据，某些要求写 1 清位，某些要求读数据寄存器自动清除 ready。教材级规格必须写清楚事件锁存、状态读取、清除、EOI 和重新开放中断的顺序。否则系统在低速测试中正常，在高频事件下丢中断或重复中断。

中断控制器用三张寄存器决定谁先被服务 设备多起来之后，“谁先被服务”不能靠先到先得。以 8259 可编程中断控制器为例，它用三张 8 位寄存器管理 8 条中断线：IRR（中断请求寄存器）锁存已到但尚未投递的请求；IMR（中断屏蔽寄存器）按位禁止对应线路参与竞争；ISR（在服务寄存器）记录当前正在被服务的请求。优先级裁决器在未屏蔽的 IRR 位中选出最高优先级的一位，与 ISR 中已在服务的最高位比较，只有新请求优先级更高时才向 CPU 拉起 INT。默认是固定优先级—IR0 最高、IR7 最低；控制器也可配置成轮转优先级，刚被服务的线路自动排到队尾，避免某条热线路长期独占。

寄存器	记录内容	关键问题
IRR	已锁存、待投递的请求位	请求是否真正被锁存，边沿触发与电平触发是否分清

IMR	每条线路的屏蔽位	屏蔽期间到来的请求是挂起还是丢失
ISR	正在服务的请求位	EOI 之前同级和低级请求一律等待

EOI 是这套机制里最要紧的动作。服务程序发出 EOI，控制器才清除对应的 ISR 位；ISR 位不清，同级和更低优先级的请求即使进了 IRR 也只能等待——优先级秩序正是靠 ISR 位维持的。若希望高级中断能打断正在运行的低级服务程序，即允许嵌套，低级服务程序必须在保存好现场之后重新打开 CPU 的中断允许位。嵌套时保存的内容必须完整，因为返回时要逐层恢复：CPU 接收中断时已把返回地址和标志寄存器压栈，服务程序还要保存自己用到的全部通用寄存器。每一层嵌套占用一帧栈，层数越深栈消耗越大，这是嵌套中断容易忽略的隐性成本。

```
isr_low:
    push rax                ; save registers this handler uses
    push rbx
    sti                    ; allow higher-priority nesting only now
    ; ... read device, handle data, clear device event ...
    cli                    ; block nesting on the return path
    mov al, 0x20           ; non-specific EOI command
    out 0x20, al           ; write 8259 command port, clears top ISR bit
    pop rbx
    pop rax
    iretq                  ; restore flags and return address
```

优先级机制带来的风险是反转与饿死。反转的典型情形：低级服务程序长时间关中断执行，高级请求虽已进 IRR 却无法投递，实际等待时间反而更长。可以量化：115200 baud 下串口一个字节连起始、停止位共 10 位，到达间隔约 $86.8\mu\text{s}$ ；接收缓冲只有一字节深时，若某个低级服务程序关中断超过这个时间，串口数据就会被下一字节覆盖，表现为溢出错误——优先级更高的串口实际上输给了优先级更低的服务程序。饿死的典型情形则相反：固定优先级下高级中断密集到来时，低级请求在 IRR 里长期得不到服务，轮转优先级正是为这种情况准备的。工程上的通用对策有两条：服务程序尽量短，把重活推迟到中断返回之后；必须关中断的窗口要给出时间上界，而不是随手一关了事。

DMA 解决的是另一类浪费。若磁盘、网卡、显示控制器或音频设备需要搬运大量数据，CPU 不应逐字从设备读出再逐字写入内存。DMA 让设备或 DMA 控制器成为总线发起者，按描述符或寄存器配置直接读写主存。CPU 负责准备缓冲区和描述符，设备负责搬运，完成后用状态位或中断通知 CPU。

```
DMA receive sketch:
CPU allocates buffer
CPU writes descriptor address and length
CPU ensures descriptor is visible to device
device reads descriptor and writes buffer
device posts completion
CPU handles interrupt and inspects buffer
```

DMA 控制器按通道组织，每个通道就是地址加计数 8237 是理解 DMA 控制器的经典样本。它提供 4 个独立通道，每个通道对应一条设备请求线 DREQ；通道的核心状态只有两类 16 位寄存器——当前地址寄存器指出下一个要访问的内存地址，每传一字节自动加一，当前计数寄存器记录还剩多少字节，每传一字节自动减一。计数写入值比真实字节数少一：写 0 表示传 1 字节，写 `0xFFFF` 表示传 65536 字节，计数从 0 再减一回绕时产生终止计数 TC。模式寄存器决定传输方向（设备到内存或内存到设备）和传输方式。由于 16 位地址最多覆盖 64 KiB，PC 还给每个通道配了一个外部页寄存器，锁存物理地址的高 8 位，合成 24 位地址；由此带来一条

经典限制：一次传输中低 16 位地址在页内推进，页寄存器保持不变，缓冲区不得跨越 64 KiB 页边界。

寄存器组	记录内容	关键问题
地址寄存器	下一个内存单元的低 16 位地址	初值是否指向缓冲区起点，方向是否为递增
计数寄存器	剩余字节数，写入值等于实际字节数减 1	忘记减 1 会多传一个字节，覆盖缓冲区之后的内容
模式/命令	传输方向、单字/块/请求方式、通道号	方向写反会把设备数据当内存数据搬
页寄存器	物理地址高 8 位（外部锁存）	缓冲区是否跨 64 KiB 页边界
屏蔽/请求	通道使能与软件请求	配置期间通道必须屏蔽，配完再开放

传输方式回答“拿到总线后干多久”。单字 (single) 模式每传一个字节就把总线还给 CPU，等设备下一次 DREQ 再申请；块 (block) 模式一旦拿到总线就连传到计数结束，其间 CPU 无法使用总线；请求 (demand) 模式随 DREQ 电平走走停停。慢设备配单字模式时，DMA 只是零星借用本属于 CPU 的总线周期，这称为周期窃取 (cycle steal)：CPU 不被告知、不切换状态，只在恰好要用总线的那一拍多等一拍。块模式是另一个极端——它把 CPU 完全关在总线之外，换取最短的总搬运时间。

以软盘控制器常用的通道 2 为例，把 8192 字节从设备搬到物理地址 0x23450 的完整配置序列如下。页号 0x02、页内偏移 0x3450，结束地址 0x2544F，不跨页边界；计数写 $8192 - 1 = 8191 = 0x1FFF$ 。

```
; 8237 ch2 block transfer: device -> memory, 8192 B at phys 0x23450
mov al, 0x06      ; mask ch2 while programming
out 0x0A, al      ; single-channel mask register
mov al, 0x86      ; mode: block, write-to-memory, ch2
out 0x0B, al      ; mode register
out 0x0C, al      ; clear byte-pointer flip-flop (value ignored)
mov al, 0x50      ; address low byte (0x3450)
out 0x04, al
mov al, 0x34      ; address high byte
out 0x04, al
mov al, 0xFF      ; count low byte (0x1FFF = 8192-1)
out 0x05, al
mov al, 0x1F      ; count high byte
out 0x05, al
mov al, 0x02      ; page register -> A16..A23
out 0x81, al
mov al, 0x02      ; unmask ch2
out 0x0A, al
; device raises DREQ; 8237 requests the bus and moves bytes;
; TC (count wraps through 0) ends the transfer and raises EOP/IRQ
```

序列里有两处容易出错。其一，16 位寄存器经同一个 8 位端口分两次写入，内部字节指针触发器决定下一次写的是低字节还是高字节，配置前必须先发清除命令，否则高低字节会写反。其二，屏蔽必须在配置之前、开放必须在配置之后，否则设备可能在地址只写了一半时就开始搬运。周期窃取的比例可以估算：软盘数据率约 62.5 KB/s，即每 $16\mu\text{s}$ 来一个字节；设总线周期 250ns、一次单字搬运占 4 拍共 $1\mu\text{s}$ ，DMA 占用总线的时间比例约 $1/16 \approx 6\%$ ，其余 94% 的总线时间仍归 CPU。若改用块模式，8192 字节约 8.2ms 内总线全部归 DMA，搬运更快，但 CPU 在这期间取不到指令。选哪种模式，本质上是在搬运总时间与 CPU 停顿之间做权衡。

DMA 最容易错在可见性和所有权 DMA 让 CPU 和设备共享主存，也引入 cache

一致性、顺序和缓冲区所有权问题。CPU 写好的描述符必须先对设备可见；设备写入的缓冲区必须在 CPU 读取前变得可见；CPU 不能在设备仍拥有缓冲区时重用它；设备不能越过分配的地址范围。现代系统还会用 IOMMU 限制 DMA 能访问哪些物理页，避免坏设备或错误驱动破坏任意内存。



图示：DMA 缓冲区所有权。完成通知、cache 可见性和所有权回收必须按顺序发生。

边界	必须说明	失败后果
描述符可见	CPU 写描述符后如何让设备看到	设备读到旧地址、旧长度或旧标志
数据可见	设备写缓冲区后 CPU 如何看到新数据	CPU 读到 cache 中的旧内容
所有权	缓冲区何时归 CPU，何时归设备	重用正在 DMA 的缓冲区，数据被覆盖
地址权限	设备能访问哪些物理页或 I/O 虚拟地址	DMA 写坏无关内存或触发 IOMMU fault
完成顺序	completion 到达是否代表所有数据已可见	收到中断后仍读到未完成数据

五、从教学总线到现代互连

早期微机的并行总线比较直观：地址线、数据线、读写控制、片选和等待信号都能在逻辑分析仪上看到。现代芯片内部和 PCIe 这类外部互连更复杂，可能使用分层 packet、请求队列、completion、错误报告和流控。但抽象问题没有改变：事务由发起者提出，经互连路由到目标，目标返回数据或完成状态，错误路径必须被记录 and 上报。

互连对象	旧总线中的类比	现代形式
地址阶段	地址线和片选	请求 packet、BAR、路由 ID、地址译码
数据阶段	数据线和字节使能	payload、byte enable、burst 或 cache line
等待	READY/WAIT	credit、ready/valid、队列背压
完成	采样数据或写周期结束	completion packet、状态位、中断或 MSI
错误	总线错误或超时	poison、fault、AER、IOMMU fault、设备 error

总线仲裁：多个主设备谁先用总线 共享总线上可以有多个能发起事务的主设备：CPU、DMA 控制器、显示控制器都可能同时要占用地址线和数据线。同一时刻只能有一个主设备驱动总线，因此必须有一种决定“这一轮给谁”的机制，这就是总线仲裁（bus arbitration）。仲裁要同时回答三个问题：裁决花多少时间，谁拥有优先权，低优先级请求会不会永远等不到。经典做法有三种，它们在线数、延迟和公平性之间做出不同取舍。

集中式仲裁器为每个主设备拉一对独立的请求线（request）和授权线（grant），全部连到一个中央仲裁器。所有请求并行到达，仲裁器在一两个周期内按优先级逻辑选出获胜者并抬起对应的 grant。这是最快的做法，优先级还可以做成可编程寄

存器；代价是线数随主设备数线性增长， N 个主设备需要 $2N$ 根专用线，仲裁器本身也成为单点。

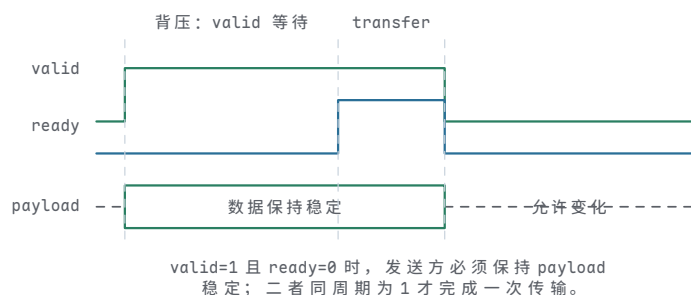
菊花链 (daisy chain) 只拉一条授权线，从仲裁器出发依次穿过所有主设备。设备没有请求时把授权信号向下一级传递，有请求时把授权截留下来占用总线。线数与设备数量无关，结构最简单；但优先级由设备在链上的物理位置写死——离仲裁器越近永远越优先，链尾设备可能长期等不到授权，而且授权要逐级传播，链越长延迟越大。

轮询计数 (polling) 用一小组设备号线代替一对一连线：仲裁器驱动 $\lceil \log_2 N \rceil$ 根线，轮流扫描设备编号，被扫到且有请求的设备抬起 busy 占用总线。线数最少，公平性也最灵活——计数器若每次从零开始，编号小的设备优先；若从上一次授权的设备之后继续扫描，就得到轮转公平。代价是平均要扫过半个编号表才能命中请求者，是三者中最慢的。

做法	专用线数	优先级	授权速度	关键问题
集中式仲裁器	$2N$ (每设备一对请求/授权)	由仲裁器逻辑决定，可编程	最快，并行比较	线多，仲裁器是单点
菊花链	与设备数无关 (一条授权链)	由物理位置固定，链首最高	逐级传播，链长则慢	链尾可能等不到，优先级不可改
轮询计数	$\lceil \log_2 N \rceil$ 根扫描线	由计数起点决定，可轮转公平	最慢，平均扫半表	扫描本身占用总线时间

现代片上互连并没有消灭仲裁，而是把它从“整条共享线上的全局问题”拆小，做进了每一个交换节点。NoC 路由器的每个输入端口要仲裁来自不同方向的 flit，crossbar 要为每个输出端口在多个输入之间仲裁，PCIe switch 要在端口和虚通道之间仲裁。共享并行总线退化成点到点链路之后，“谁先用”的问题依然存在于每个交换机内部——只是裁决范围从整块板卡缩小到一个端口。

valid/ready 是理解现代片上互连的最小模型 许多片上接口都可以抽象成 valid/ready 握手。发送方在 valid 为 1 时保持请求或响应稳定，接收方在 ready 为 1 时表示可以接收，只有二者同周期为 1，事务片段才真正完成。请求通道和响应通道可以分开，读请求和读数据也可以隔很多周期。这个模型比“总线一拍完成”更接近现代硬件。

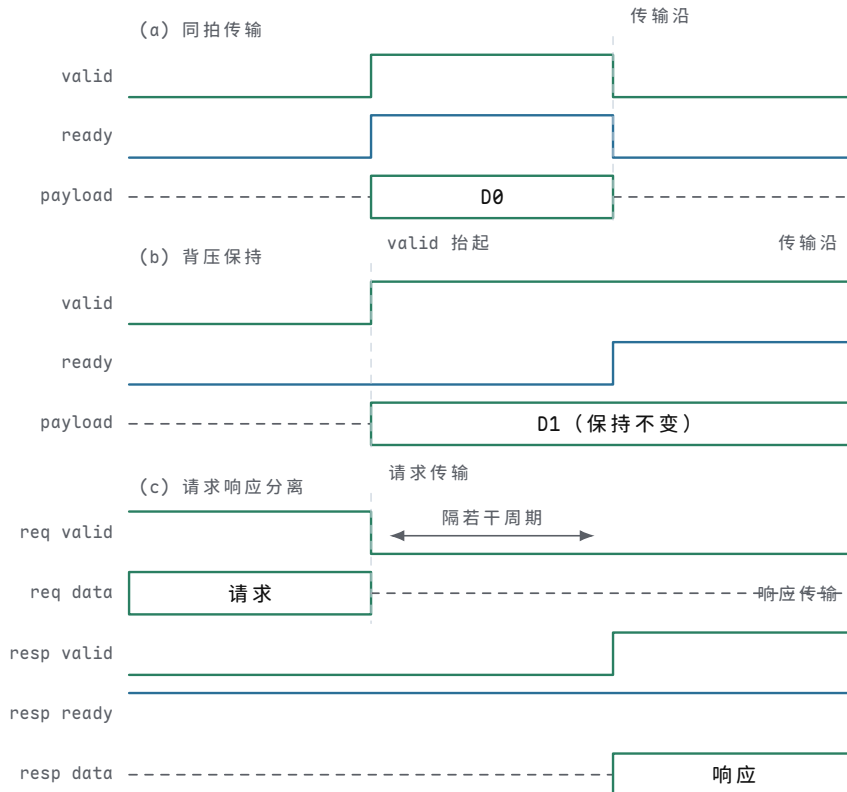


图示: valid/ready 握手。发送方先抬 valid 并保持数据稳定；接收方在背压期间保持 ready=0；两条线在 transfer 周期同时为 1，事务片段才完成。背压不是错误，而是接收方暂时不能接受。

handshake rule:

```
transfer happens only when valid == 1 and ready == 1
while valid == 1 and ready == 0:
    sender keeps payload stable
    receiver may apply backpressure by deasserting ready
```


三种 valid/ready 时序情形 valid/ready 的全部行为可以归纳成一条采样规则和三种常见情形。采样规则是：只在某个采样沿上 valid 和 ready 同时为 1，该通道才完成一次传输。三种常见情形是：双方同拍就绪，当拍完成；接收方拉低 ready 施加背压，发送方必须保持 valid 和 payload 不变；请求通道和响应通道各自独立握手，两次传输之间可以隔任意多个周期。这三种情形都不是异常，而是流控协议的正常组成部分。



图示：三种情形一条规则：只在 valid 与 ready 同拍为 1 的沿上传输。(a) 双方同拍就绪，当拍完成；(b) ready 为 0 的期间 valid 与 D1 保持不变，这是接收方施加的背压，不是错误；(c) 请求在一个沿完成，响应通道隔若干周期后才完成自己的传输，两个通道互不等待。

写缓冲和内存顺序解释了现代芯片为什么看起来更快 写入比读取更容易被隐藏。CPU 可以先把写事务放进写缓冲，让执行流水线继续推进；写缓冲再把相邻写合并、排队发送到 cache 或互连。这样常见 STORE 的表面延迟很低，但代价是可见性变复杂：其他核心、DMA 或设备什么时候看到这次写，取决于 cache 一致性、内存类型和屏障规则。普通 RAM、MMIO 和 DMA 描述符不能使用同一套可见性假设。

总线错误和超时必须成为规格的一部分 未映射地址不能只写成“不要访问”。真实系统必须规定访问未映射区域时会发生什么：返回固定值、保持等待直到超时、产生总线错误，或进入异常入口。设备无响应、DRAM 控制器报错、ECC 检测失败、外设权限不允许，也都应该形成可观察错误状态。没有错误约定的整机，调试时会把硬件故障误判为程序死循环。

cache 行和局部性把常见访问留在近处 主存容量大但离 CPU 远，寄存器离 CPU 近但容量小，cache 处在二者之间。cache 不是“更快的内存副本”这么简单，而是按 cache line 搬运的一组近处 SRAM。CPU 读取某个地址时，cache 会用地地址中的 tag 判断目标行是否已经在近处；若命中，数据直接从 cache 返回；若未命中，cache 控制器向下一级 cache 或 DRAM 请求整行，再把需要的字节送回 CPU。

对象	硬件含义	关键问题
cache line	一次填充和替换的最小数据块	行大小、字节偏移和未对齐访问是否清楚

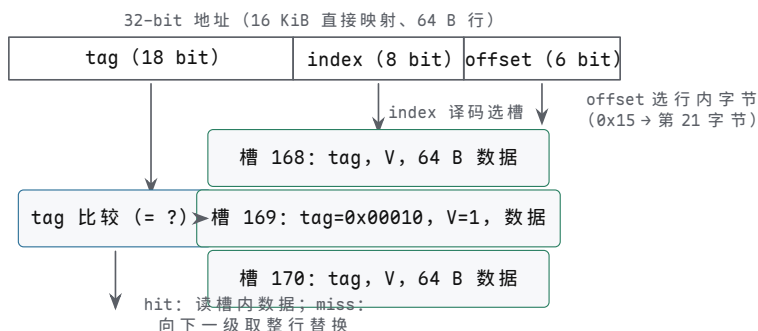
tag	判断某个 cache 槽保存的是哪段地址	命中比较是否覆盖高位地址和有效位
index	选择 cache 中的候选槽或集合	映射冲突是否可能造成反复 miss
valid/dirty	表示行是否有效、是否被修改过	替换时是否需要写回下一级
miss	近处没有所需行，需要向下一级请求	miss 期间 CPU 停顿、继续执行或乱序等待的边界

局部性解释了 cache 为什么有效。时间局部性表示刚访问过的数据很可能再次访问，例如循环变量和栈顶对象；空间局部性表示访问某个地址后，很可能访问相邻地址，例如顺序扫描数组。cache line 正是利用空间局部性：一次 miss 不只取一个字节，而是取一整段相邻字节。代价也随之出现：若程序跨很大步长访问数组，或者两个热数据反复映射到同一 cache 集合，硬件仍会不停搬行，表面上只是 LOAD 很慢，实质上是事务层反复 miss 和替换。

32-byte cache line example:

```
address bits = [ tag | index | byte_offset ]
byte_offset selects one byte within the line
index selects a cache set
tag comparison decides hit or miss
```

案例：一次 cache 命中的完整分解 cache 的查找可以拆成三个字段各管一件事：offset 选行内字节，index 选槽，tag 判身份。以 32-bit 地址、16 KiB 直接映射、64 B 行为例：offset = $\log_2 64 = 6$ bit；行数 = $16 \text{ KiB} \div 64 \text{ B} = 256$ ，index = $\log_2 256 = 8$ bit；tag = $32 - 8 - 6 = 18$ bit。任何地址到来时，硬件先按 index 找到唯一的槽，再用 tag 和 valid 位判断“槽里这行是不是这段地址”。



图示：直接映射 cache 的查找。地址拆成 tag/index/offset 三段：index 译码选出唯一的槽，槽内 tag 与地址 tag 比较决定 hit/miss，offset 再从 64 B 行内选出目标字节。

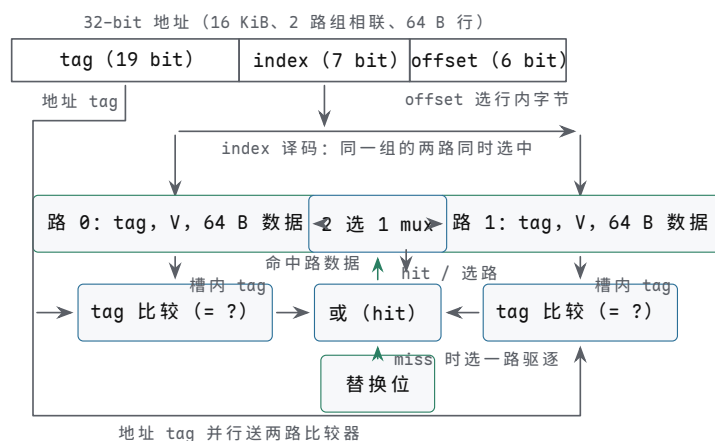
下表把四次访问的分解和结果走一遍。注意第 3、4 次展示了直接映射最典型的失效模式：

次序	地址	tag / index / offset	结果
1	0x00042A40	0x00010 / 0xA9 / 0x00	槽 169 的 V=0 → miss；从 DRAM 取回 64 B 整行填入，tag 写 0x00010、V 置 1，返回第 0 字节
2	0x00042A40	0x00010 / 0xA9 / 0x00	tag 相等且 V=1 → hit，直接从槽内返回数据

3	0x00082A55	0x00020 / 0xA9 / 0x15	槽 169 的 tag 是 0x00010, 不等 → 冲突 miss; 取回新行替换旧行 (旧行若 dirty 先写回 DRAM), 返回第 21 字节
4	0x00042A40	0x00010 / 0xA9 / 0x00	槽 169 刚被第 3 次访问换掉 → 又 miss

两个热地址映射到同一个槽时, 直接映射会反复 miss——这正是组相联存在的理由: 同一个槽放多路, 几路 tag 并行比较, 任何一路相等都算 hit, 常用数据就不必互相驱逐。

两路组相联让热行各占一路 直接映射的冲突来自“一个槽只能放一行”。把同样 16 KiB、64 B 行的 cache 改成 128 组、每组 2 路: offset 仍是 6 bit, index 从 8 bit 变成 7 bit, tag 从 18 bit 变成 19 bit。重走前面的四次访问: 0x00042A40 和 0x00082A55 的 index 都是 0x29, tag 分别是 0x00021 和 0x00041。第 1 次 miss, 填入路 0; 第 2 次 tag 相等, hit; 第 3 次两路 tag 都不等 (路 1 的 V 仍为 0), 仍然 miss, 但新行填入路 1, 不再驱逐路 0; 第 4 次再读 0x00042A40, 路 0 的 tag 相等, 变成 hit。直接映射下第 3、4 次的互相驱逐, 在 2 路组相联下只剩第 3 次那次无法避免的首次 miss。代价是每组多一个 tag 比较器、一个数据选择 mux 和若干替换位。



图示: 2 路组相联的查找。index 译码选中整个组, 组内两路的 tag 同时与地址 tag 比较; 任一路相等, 或门即输出 hit, 并用比较结果控制 mux 选出命中路的数据。miss 时按组内替换位选一路驱逐、填入新行。

写策略决定 STORE 什么时候真正离开 CPU 读 miss 的动作比较直观: 向下级取回数据。写入更复杂, 因为 CPU 可以先改 cache, 再决定什么时候把修改传播到下一级。写穿策略每次写命中都继续写下一级, 行为直接但带宽压力大; 写回策略只在 cache 中标脏位, 替换时再写回下一级, 常见情况下更快, 但可见性和错误恢复更复杂。无论哪种策略, 字节写都必须尊重 byte enable, 不能把同一 cache line 中无关字节改坏。

写策略	优点	风险和边界
写穿	下一级更快看见新值, 调试直观	带宽压力大, 常配合写缓冲隐藏延迟
写回	多次写同一行可合并, 减少外部事务	dirty 行替换、掉电和 DMA 可见性更难处理
写分配	写 miss 时先把整行取入 cache 再修改	小写入可能触发整行读取
非写分配	写 miss 直接向下级写, 不装入 cache	后续读同地址可能仍 miss

这也是 MMIO 不能随意缓存的根本原因。写 LED 寄存器、清中断状态或启动 DMA 命令时，程序需要的是一次外设可见动作，而不是某条 cache line 中的普通字节更新。若 MMIO 被写回 cache 吞掉，设备可能永远看不到命令；若读状态寄存器被 cache 命中，CPU 可能一直看到旧状态。因此地址属性必须区分普通可缓存内存、不可缓存设备内存、写合并 framebuffer、DMA coherent 区域等不同事务约定。

cache miss 是事务暂停，不是抽象的慢 当 LOAD miss 时，流水线不能凭空得到数据。简单 CPU 会停在访存阶段，保持地址和控制状态，直到下级返回数据；高性能 CPU 可能让独立指令继续执行，把这条 LOAD 放在未完成队列中，等数据回来再唤醒依赖指令。二者复杂度不同，但都必须维护同一个可见性约定：依赖这次 LOAD 的指令不能使用错误旧值，异常和中断也不能让程序看见半提交状态。

miss 类型	事务路径	典型处理
L1 miss/L2 hit	L1 请求 L2 返回 cache line	几到十几周期，流水线可能短暂停顿
最后级 cache miss	请求内存控制器和 DRAM	上百周期级别，常需乱序、预取或多线程隐藏
TLB miss	地址转换缓存缺失，要查页表或进异常	页表 walker、权限检查和可能的页错误
写回替换	脏行被换出，需先写下一级	替换队列、写缓冲和错误报告
设备/DMA 冲突	cache 中旧副本与设备写入不一致	coherent 互连、cache 维护或不可缓存映射

低时延来自近处、缓存、并行和卸载 现代整机降低时延，不是让所有路径都变成零等待。L1/L2 cache 让常见数据走近处 SRAM；TLB 让近期地址转换不必每次查页表；预取和分支预测提前准备可能需要的路径；流水线、旁路和乱序调度让独立工作重叠；DMA、队列和中断聚合把批量 I/O 从 CPU 指令流中卸载。失败路径仍然存在：cache miss、TLB miss、DRAM 排队、设备背压、DMA fault 和错误中断都会把访问拉回慢路径。

六、存储层次、地址转换和设备驱动 trace（执行轨迹）

前面已经讲了单次总线事务、MMIO、中断、DMA 和 cache。真正的系统程序运行时，这些对象不会孤立出现。一次普通 LOAD 可能先经过 TLB 查找，再查 L1 cache，未命中后查 L2/L3，最后进入内存控制器；一次设备接收数据可能由驱动准备描述符，设备 DMA 写缓冲区，cache 一致性或维护操作让 CPU 看见新数据，中断再把完成事件带回来。若教材只讲“内存比寄存器慢”，就漏掉了真正的硬件路径。

访问层次	常见命中路径	失败或慢路径
寄存器	直接从寄存器堆读写	端口冲突、旁路失败或流水线停顿
L1 cache	近处 SRAM 命中，几个周期内返回	L1 miss，向 L2 请求整条 cache line
L2/L3 cache	片上较大 cache 命中	最后级 miss，访问内存控制器和 DRAM
TLB	地址转换缓存命中	page walk、页错误或权限异常
DRAM	控制器调度行命中和突发传输	行冲突、刷新、排队、ECC 错误
设备/MMIO	设备寄存器响应读写	背压、超时、错误状态或中断完成

TLB 把地址转换也纳入近处缓存 虚拟地址或线性地址到物理地址的转换若每次都查页表，访存会变得很慢。TLB 可以理解为“地址转换的 cache”：它保存近期虚拟页到物理页框的映射及权限位。LOAD/STORE 形成虚拟地址后，先用虚拟页号查 TLB；命中时得到物理页框和权限信息，再与页内偏移组合成物理地址；未命中时，硬件 page walker 或软件异常处理会去内存里的页表结构查找。

```
virtual address = [ virtual_page_number | page_offset ]
TLB hit:
    physical_address = TLB[virtual_page_number].frame | page_offset
TLB miss:
    walk page tables or enter miss handler
```

TLB 字段	作用	关键问题
虚拟页号 tag	判断当前地址是否命中某个转换项	进程切换、地址空间标识和刷新规则是否清楚
物理页框	与页内偏移组合成物理地址	页大小、对齐和物理地址宽度是否一致
权限位	读写、执行、用户/特权等访问属性	访问失败是权限异常，不是普通 cache miss
valid/global	表示转换是否有效、是否跨进程保留	页表修改后如何让旧 TLB 项失效

TLB miss 和 cache miss 都是“近处没有”，但语义不同。cache miss 只是数据不在近处，通常可以向下级取回；TLB miss 可能只是转换缓存缺失，也可能说明页不存在或权限不允许。若页表项不存在，硬件不能继续访问内存数据；若权限不允许，访问必须转入异常。这个差别解释了为什么地址转换、保护和异常在系统硬件中是同一条路径的不同分支。

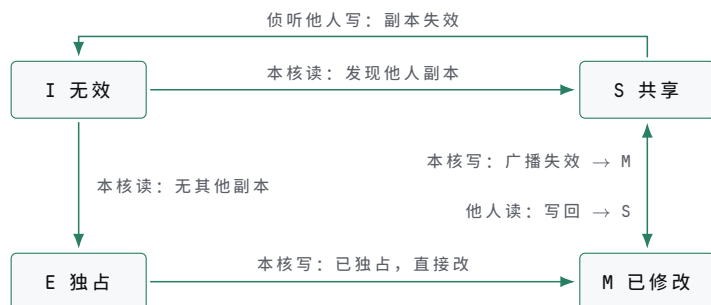
多核和 DMA 让 cache 一致性成为硬件约定 单核教学机里，cache 可以先理解为 CPU 与主存之间的近处副本。多核或 DMA 加入后，问题变成“多个观察者如何看见同一地址的值”。一个核心的 cache 中可能有某条 line 的旧副本，另一个核心或设备刚刚写入了新值；如果没有一致性协议、cache 维护或不可缓存映射，CPU 读到的就可能不是系统真实最新值。

场景	风险	常见硬件/软件边界
核心 A 写，核心 B 读	B 的 cache 中可能有旧行	cache coherence、失效消息、共享/独占状态
CPU 写描述符，设备读	描述符可能还在 CPU cache 或写缓冲中	coherent DMA、flush、写屏障
设备写缓冲区，CPU 读	CPU cache 中可能有旧数据	invalidate、coherent 互连、不可缓存映射
MMIO 状态寄存器	cache 命中会隐藏设备新状态	设备内存属性、禁止普通缓存

一致性协议细节可以很深，但教学阶段先抓住三件事。第一，同一地址可能在多个近处缓存中有副本。第二，写入必须让其他观察者的旧副本失效或更新。第三，设备并不总是自动参与 CPU cache 一致性；若平台不支持 coherent DMA，驱动必须在交出缓冲区前后做 cache 维护。否则 DMA 完成中断到达后，CPU 仍可能读到旧 cache line。

MESI 用四个状态约定谁拿着最新值 这套状态存在的意义，是让任何时刻、任何地址在所有观察者眼里只有一份“最新值”。想写某一行，核必须先成为唯一持有者：从 S 升级要广播失效，把其他核的副本打成 I；只有处于 M 或 E 时才能直接改。想读可以共享：多个核同时持有 S，读到的都是同一个干净副本，但共享期间

谁也不能直接写。于是两个核不会各自拿着不同的新值—要么一个核独占、其余无效，要么大家一起只读。参与一致性互连的 DMA 设备也被同一套状态约束；不参与的 platform，驱动就要用 cache clean/invalidate 手工扮演“写回”和“失效”的角色。



图示：MESI 一致性状态图（教学简化版）。本核读把 I 变成 E（无其他副本）或 S（他人有副本）；本核写必须先独占：S 到 M 要广播失效，E 到 M 直接改；总线侦听到他人读时 M 写回并退到 S；侦听到他人写时副本失效（E、M 同样失效，M 先写回）。

案例：MESI 在两个核之间的完整走查 状态图只有放进具体访问序列才有意义。下面用两个核、一个地址 X 把上一专题的状态机完整走一遍。设初始时核 A、核 B 的 cache 中 X 行均为 I（无效），内存持有 X 的最新值。先说明一个细节：按严格 MESI，核 A 第一次读入 X 时若没有发现任何其他副本，应记为 E（独占）；本走查假设 X 此前已被第三个核读过，或采用“读入即共享”的教学约定，因此第一步记为 S。这不影响后续步骤的转换逻辑。

步骤	动作	核 A	核 B	互连上发生的事
0	初始	I	I	X 的最新值在内存中
1	核 A 读 X	S	I	A 的读 miss，经互连从内存取回整行，记为共享副本
2	核 B 读 X	S	S	B 的读 miss，从内存取行；A 侦听到这次读，保持 S，两核持有同一份干净副本
3	核 A 写 X	M	I	A 先广播失效消息，B 的副本作废（S → I）；确认后 A 独占修改本行（S → M），内存中的 X 从此过期
4	核 B 读 X	S	S	B 的读 miss；A 侦听到读，把脏行写回内存（或直接转发给 B），自己降为 S；B 得到最新数据的共享副本

逐步看两处关键转换。第 3 步是“写之前先独占”：A 不能只改自己手里的 S 副本，否则 B 会继续读到旧值，同一地址就出现了两个“最新值”；广播失效把其他副本全部打成 I 之后，A 才升级成 M 并修改。第 4 步是“最新值跟着 M 走”：此时内存里的 X 是旧数据，B 的读不能由内存直接回答，必须由持有 M 行的 A 先把数据交出来—写回内存或 cache 之间直接转发—之后两核各持 S，内存恢复最新。

这张表就是一致性协议的 trace：每一行对应状态图上的一条边，每一步结束都满足同一条不变式—X 的最新值在全局唯一，要么在某核的 M 行里（其余全为 I，内存过期），要么在所有 S 副本与内存里保持一致。调试多核程序时若怀疑数据不一致，最有效的办法正是把可疑的访问序列按这种方式列成表，逐步检查哪一步破坏了唯一性。

store buffer 和屏障把“执行完成”与“可见”分开 STORE 对流水线来说可以很早“执行完成”：地址和数据已经形成，写事务进入 store buffer，后续独立指令可以继续执行。但这个 STORE 什么时候对其他核心、DMA 或设备可见，要看 store buffer 排队、cache 状态、内存类型和屏障规则。多核内存模型本身超出本书范围，

这里只需要与 DMA 直接相关的一条规则：普通 RAM 写可以合并和延迟；MMIO 写通常要求更强顺序；DMA 描述符写在通知设备前必须已经对设备可见。

```
prepare DMA descriptor:
    descriptor.addr = buffer
    descriptor.len  = length
    memory_barrier()
    MMIO[DEVICE_DOORBELL] = descriptor_index
```

这里的屏障不是“让 CPU 慢一点”的软件仪式，而是要求硬件在门铃 MMIO 写对设备可见之前，先让描述符内容按平台规则可见。若顺序反了，设备可能看到门铃，却读到旧描述符或半写入描述符。许多难复现的设备错误，本质上就是这个可见性边界没有写清楚。

对象	可见性要求	失败表现
普通数据	由语言、编译器和内存模型共同约束	多核读到旧值或乱序观察
锁变量	获取/释放顺序必须约束临界区数据	进入临界区后仍看见旧数据
DMA 描述符	通知设备前必须对设备可见	设备搬错地址或长度
MMIO 门铃	前面的配置写必须先到设备接口	设备按旧配置启动
中断完成	completion 到达后数据必须已可读取或可同步	中断处理读到旧缓冲区

案例：一个内存屏障挡住什么 屏障的作用可以用一个经典的双核序列看清。两个核共享变量 x 和 y ，初值都为 0：核 A 先把 1 写入 x ，再读 y 到 $r1$ ；核 B 先把 1 写入 y ，再读 x 到 $r2$ 。直觉上，两次写和两次读无论怎样交错，总有一个读应该看到对方的新值 1。但在带 store buffer 的真实机器上， $r1=0$ 且 $r2=0$ 的结果确实可能出现。

```
core A          core B
mov qword ptr [x], 1   mov qword ptr [y], 1
mov rax, [y]          mov rbx, [x]
```

原因就在上一专题的可见性划分： $x=1$ 对核 A 来说“执行完成”只表示写事务进入了 A 的 store buffer，并不意味着其他核已经能看见它。两个核的 store 都可能还排在各自的 store buffer 里，两个 load 就已经从内存（或对方尚未更新的 cache 副本）里取回了 0。

次序	核 A	核 B	内存中可见的 x / y
1	$x=1$ 进入 A 的 store buffer	—	0 / 0
2	—	$y=1$ 进入 B 的 store buffer	0 / 0
3	$r1=y$ ，读到 0	—	0 / 0
4	—	$r2=x$ ，读到 0	0 / 0
5	store buffer 排空， $x=1$ 全局可见	store buffer 排空， $y=1$ 全局可见	1 / 1

这个结果不违反任何一条单核规则——每个核自己看来程序都按序执行了——但它违反了“所有核看到同一个全局顺序”的直觉。x86 的 `mfence` 指令正是为此存在：它强制本核的 store buffer 排空之后，后续的 load 才允许执行，也就是说本核

此前的写必须先对其他核可见。

core A	core B
mov qword ptr [x], 1	mov qword ptr [y], 1
mfence	mfence
mov rax, [y]	mov rbx, [x]

加入屏障后, $r1=0$ 且 $r2=0$ 被禁止。推理只需直觉: 若 A 读到了 $y=0$, 说明 B 的 $y=1$ 在那一刻尚未对 A 可见; 而 B 的 `mfence` 保证 $y=1$ 全局可见之后 B 才会执行 $r2=x$, 到那时 A 的 $x=1$ 早已因 A 自己的 `mfence` 而对所有核可见, 所以 B 读 x 必得 1。屏障挡住的正是“写还留在 store buffer 里、读却已经出发”这段窗口; 它没有让写变得更快, 只是把可见性顺序钉死。

描述符环把 DMA 变成持续事务流 高速设备通常不会每次只处理一个缓冲区。网卡、磁盘控制器、USB 控制器常使用描述符环或队列: CPU 准备一批描述符, 设备按头尾指针消费, 完成后写回状态或 completion 队列, 并用中断或轮询通知 CPU。这样可以减少 MMIO 次数, 让 CPU 和设备并行工作。

```
receive ring:
CPU owns descriptor N
CPU fills buffer address and length
CPU marks descriptor ready for device
CPU updates tail pointer or rings doorbell
device writes packet into buffer
device writes completion status
CPU regains ownership and processes buffer
```

环队列字段	硬件含义	关键问题
描述符地址	设备要读写的主存位置	地址是否经过 IOMMU 或物理地址转换
长度	本次 DMA 允许访问的字节数	设备不能越界写出缓冲区
所有权位	当前由 CPU 还是设备拥有	所有权不重叠, 状态切换有屏障
头尾指针	CPU 和设备约定的生产/消费位置	指针更新顺序和环绕规则清楚
completion	设备报告完成、长度、错误码	中断到达不等于所有 cache 可见性自动正确

这个模型也说明驱动程序为什么必须理解硬件。驱动不是“调用设备 API”的普通软件, 它要写 MMIO 寄存器、分配对齐缓冲区、建立 DMA 映射、维护 cache 可见性、处理中断、回收描述符和处理错误。每一步都对应本章的硬件对象: 地址、权限、字节数、所有权、完成和错误。

描述符环不动、指针动 描述符环的关键是“环不动、指针动”: 描述符数组在内存中的位置固定, CPU 只推进 SW 指针, 设备只推进 HW 指针, 走到末尾就回绕; 两个指针的差值就是环中未完成事务的数量。doorbell 不携带数据, 只是告诉设备“指针又动了”, 设备即使错过一次 doorbell, 靠指针差也能补读。反过来, 某个描述符是否完成, 不能凭 doorbell 或中断猜测, 必须以设备写回描述符状态位 (或 completion 队列项) 为准; CPU 回收缓冲区前, 还要确认这次写回对自己可见。



图示：描述符环。设备按 HW 指针顺序消费，CPU 按 SW 指针顺序生产，描述符 7 之后回到 0；CPU 更新 SW 后写一次 doorbell 提示设备。完成与否以设备写回描述符状态位为准，doorbell 只是“有新描述符”的提示。

驱动 bring-up 应分层验证 调试设备时，最差的方法是直接运行完整驱动并等待它“能不能用”。更可靠的顺序是先验证配置空间或设备 ID，再验证 MMIO 映射，再验证单个寄存器读写，再验证中断，再验证一个最小 DMA 描述符，最后才跑持续队列。每一层都要有可观察的结果。

bring-up 阶段	操作	预期结果
枚举/识别	读取设备 ID、版本或能力寄存器	地址映射正确，读值稳定可复现
复位	写复位位并轮询 ready/busy	状态位按规格变化，不永久 busy
MMIO 回读	写可回读配置寄存器再读回	字节序、位域和写掩码正确
中断	人工触发或配置定时事件	中断线/MSI 到达，清除顺序正确
单次 DMA	一个描述符、一个缓冲区、一个 completion	地址、长度、cache 可见性和错误码正确
持续队列	多描述符环和批量 completion	所有权、环绕、背压和中断聚合正确

这张表也适用于教学板。即使没有 PCIe，只用一个 GPIO、定时器或简化 DMA 控制器，也应该按同样顺序验证：先确认地址译码，再确认寄存器语义，再确认事件通知，再确认数据搬运。这样本书的硬件 trace 就能从最小 CPU 直接延伸到现代设备驱动。

案例：一个接收包怎样进入内存 以网卡接收一个数据包为例。CPU 先分配若干缓冲区，把每个缓冲区的地址和长度写入接收描述符环；随后保证描述符对设备可见，再写 MMIO 门铃或尾指针告诉设备可以接收。设备收到外部数据后，作为 DMA 发起者把数据写入某个缓冲区，再写 completion 或描述符状态，最后触发中断。CPU 处理中断，确认 completion，按一致性规则让缓冲区数据对 CPU 可见，然后把缓冲区交给上层软件。

阶段	硬件事务	观察要点
准备缓冲区	CPU 在 RAM 中分配对齐缓冲区	物理地址或 I/O 虚拟地址有效，长度足够
写描述符	CPU STORE 地址、长度、所有权位	描述符 cache line 已按平台规则可见
通知设备	CPU 写 MMIO tail/doorbell	MMIO 不被普通 cache 合并或延迟越界
设备 DMA 写	设备发起内存写事务	IOMMU 权限、字节数、completion 顺序正确
投递完成	设备写状态并触发中断	completion 可读，中断不会早于数据可见

CPU 回收 中断处理读取状态和缓冲区

cache invalidate 或
coherent 规则完成

receive path:

```

CPU: fill rx_desc[N] with buffer address and length
CPU: barrier, then MMIO[RX_TAIL] = N
DEV: reads rx_desc[N]
DEV: writes packet bytes into buffer
DEV: writes completion status
DEV: raises interrupt
CPU: acknowledges interrupt and reads completion
CPU: synchronizes buffer visibility
CPU: consumes packet

```

这条路径里至少有三次“完成”。第一，CPU 写 MMIO 门铃完成，只表示设备接口看到了通知；第二，设备 DMA 写缓冲区完成，表示数据已经到达内存系统的某个可见点；第三，中断处理完成，表示 CPU 已经确认状态并回收所有权。把这三者混在一起，是驱动和硬件协同时最常见的错误来源。

案例：发送路径为什么要区分数据和描述符 发送数据包时方向相反。CPU 先把待发送数据写入缓冲区，再写发送描述符，最后通知设备。设备读描述符，再 DMA 读缓冲区，把数据发出。这里最关键的可见性顺序是：缓冲区内容必须先对设备可见，描述符必须描述正确地址和长度，门铃必须最后到达。否则设备可能发出旧数据、半写数据或错误长度的数据。

发送对象	可见性要求	失败表现
数据缓冲区	CPU 写入的数据先对设备可见	设备发送旧包或未初始化字节
描述符	地址、长度、标志位完整可见	设备读错缓冲区或长度
门铃寄存器	在数据和描述符之后对设备可见	设备提前启动，读到旧状态
completion	设备确认已不再读取缓冲区	CPU 过早重用缓冲区导致数据损坏

发送 completion 到达前，CPU 不应重用缓冲区。即使 MMIO 写门铃已经返回，设备可能还没有读描述符；即使设备已经读描述符，可能还没有读完数据；即使数据已经发出，completion 可能还在队列里等待 CPU 处理。因此所有权位和 completion 队列不是形式主义，而是让 CPU 和设备不同时修改同一块内存的硬件约定。

案例：块设备读请求的队列化 磁盘、NVMe、SD 控制器或简化块设备也可以用同一模型理解。CPU 准备命令描述符：逻辑块地址、目标缓冲区、长度、读写方向；设备读取命令后，在内部介质或控制器中完成实际读写，再通过 DMA 把数据写到主存，最后投递 completion。区别只在于设备内部工作可能更慢、队列更深、错误码更复杂。

块请求字段	含义	硬件问题
LBA 或块号	设备内部介质位置	越界、坏块、介质错误如何报告
缓冲区地址 长度	DMA 写入或读取的主存位置 传输字节数或块数	对齐、IOMMU、cache 可见性 不能越过缓冲区和描述符许可范围
方向	读设备到内存，或写内存到设备	DMA 方向决定 cache 维护方式

命令 ID	completion 对应哪个请求	队列乱序完成时仍能匹配请求
状态码	成功、超时、校验错误、权限错误	驱动恢复或上报错误

这个案例能把“设备很慢”讲得更具体。CPU 发出块读请求后，并不会逐字等待数据从设备口读出。它可以切换去做别的工作；设备和 DMA 在后台推进；中断或轮询 completion 时，CPU 再取回结果。硬件性能来自异步队列和批量搬运，而不是让单条 LOAD 直接等磁盘或网络。

案例：一个未映射访问如何成为可诊断错误 再看失败路径。CPU 执行 `MOV R1, [0xDEAD0000]`，地址译码没有任何目标响应。没有错误约定时，CPU 可能读到悬空总线值，或永久等待 ready。可靠平台应规定超时、总线错误或异常入口。若有 MMU，还可能在页表阶段就发现该地址未映射，不会走到外部总线。

失败位置	可观察现象	正确处理
页表不存在	地址转换阶段触发页错误	保存故障地址和错误码，进入页错误入口
权限不允许	转换存在但读写/特权不满足	触发保护异常，不发起外部访问
地址译码无目标	总线事务无人响应	超时或 bus error，而不是无限等待
目标返回错误	设备或内存控制器报告失败	错误状态进入异常或驱动恢复路径
ECC 校验失败	数据返回但校验错误	更正、重读、上报或机器检查

这张表对硬件调试尤其有用。现象“程序卡住”可能来自页表、总线译码、设备 ready、时钟、复位或错误入口任何一层。把失败位置写成层级，就能决定先看页表项、地址线、片选、ready、错误状态，还是异常向量。

读任何硬件平台，都可以用同一张事务表约束理解：发起者、目标、地址、方向、数据宽度、握手、完成、错误、副作用和可观察状态。早期 8086 总线、微控制器片内外设、PCIe 设备和现代 SoC 互连的规模不同，但这十项问题不变。

专题：DRAM 控制器为什么要排队和调度 SRAM 可以近似看成“地址稳定后读出数据”的存储器，DRAM 则必须先打开行、读写列、预充电、刷新。主存控制器面对的不是单个 LOAD，而是一串来自 CPU、DMA、显示控制器和其他主设备的请求。它要在正确性约束内排序请求，尽量利用行命中和 bank 并行，同时保证刷新不丢失数据。

DRAM 概念	含义	对系统的影响
row activate	把某一行搬到行缓冲中	同一行后续列访问较快
row conflict	下一个请求落在不同的行	需要预充电再打开新行，延迟增加
bank	多组可相对独立工作的阵列	交错访问可隐藏部分等待
refresh	周期性恢复电容电荷	刷新窗口内部分请求会被延后

ECC	用冗余位检测或纠正错误	增加位宽和延迟，但提高可靠性
-----	-------------	----------------

因此，主存延迟不是一个固定常数。一次 cache miss 到达内存控制器后，可能遇到行命中、行冲突、刷新、写队列排空或高优先级 DMA。系统软件看到的是“同样的 LOAD 有时快有时慢”，硬件工程师看到的是请求队列、调度策略和时序参数。教学时不必展开 DDR 全部命令，但必须让读者知道：主存不是无限宽、无限快、无排队的数据。

专题：cache 设计参数怎样改变命中率和时延 cache 的基本问题是把大而慢的主存切成较小的行，并保存最近使用的数据。地址会被拆成 tag、index、offset：offset 选择 cache 行内字节，index 选择组，tag 判断这一组里哪一路是目标行。不同组织改变冲突概率、比较器数量、替换策略和命中时延。

组织	优点	代价或风险
直接映射	查找简单、速度快、面积小	两个常用地址若映到同一行会反复冲突
组相联	多路候选降低冲突 miss	多个 tag 比较器和路选择 mux 增加延迟
全相联	任意行可放任意位置，冲突最少	硬件代价高，通常用于 TLB 或小 cache
写直达 写回	内存总是较新，恢复简单 多次写合并到 cache 行，带宽效率高	写流量大，常需写缓冲 dirty 行替换时必须写回，失电/一致性更复杂

评价 cache 不能只看“第二次访问变快”。还要看替换、脏行写回、字节写使能、未对齐访问、MMIO 不可 cache、DMA 可见性和异常路径。例如设备把数据 DMA 到主存后，CPU 若继续读旧 cache 行，就会看到过期数据；CPU 写好发送缓冲区后，若脏行没有写回，设备也可能读到旧内容。这就是为什么驱动要有 cache clean/invalidate 或一致性互连。

专题：MMIO 寄存器位语义要像数据手册一样写 MMIO 寄存器不是普通变量。每一位都可能副作用：读清、写 1 清、只读、保留位必须写 0、写后自动启动、硬件置位软件清除。若教材只写“把某地址当寄存器读写”，读者会在真实芯片上犯危险错误。

位语义	例子	软件访问要求
R0	输入电平、设备 ID	写入无效或保留，不应读改写破坏其他位
W0	FIFO 写口、启动命令	读值可能无意义，不能依赖回读
RW W1C	配置模式、使能位 中断状态位	修改前要明确复位默认值 写 1 清除对应事件，写 0 保持
RC reserved	读状态自动清除 未来扩展或工厂测试位	调试打印也可能改变设备状态 按手册写固定值，通常为 0

一个严谨的 GPIO 寄存器描述至少要写清：方向寄存器决定引脚驱动还是输入；输出寄存器只影响输出模式；输入寄存器来自同步后的外部电平；中断状态由边沿检测置位；清除中断采用 W1C；保留位读值未定义且写 0。这样驱动代码才能避免“读状态时把事件清掉”“读改写时误清中断”“把保留位写成 1 后芯片行为改变”等问题。

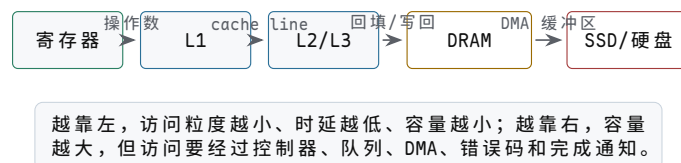
专题：总线错误要能定位到层级 当一次访问失败时，工程上最重要的问题不是

“错了”，而是错在地址转换、权限、互连、目标设备还是数据校验。若每层都只把错误折叠成一个笼统异常，调试会非常困难。更好的设计是保留故障地址、访问方向、主设备 ID、错误来源和目标返回码。

记录字段	作用	调试价值
fault address	失败访问的地址	区分空指针、越界和 MMIO 映射错误
access type	读、写、取指、原子操作	判断权限和副作用风险
master ID	CPU、DMA、GPU 或调试器	找到真正发起者
decode result	命中哪个目标或未命中	定位地址地图错误
response code	OKAY、SLVERR、DECERR、超时等	区分目标报错和无人响应

这也是为什么后续平台 bring-up 日志要记录总线层的事务信息。单看 C 代码中的一行 `*ptr` 无法知道失败来自页表、cache、IOMMU、互连还是设备时钟未开；完整错误记录能把问题缩小到可测量的硬件边界。

专题：内存层次要同时看容量、时延、带宽和所有权 计算机硬件不是只有 CPU。CPU 的每一次有用工作，几乎都要和某一级存储或 I/O 交换信息。寄存器最快但数量极少；L1/L2/L3 cache 容量逐级增大、时延逐级上升；DRAM 容量大但需要控制器、队列和刷新；SSD/硬盘容量更大，却只能通过控制器、队列、DMA 和中断间接参与执行。把这些对象都称为“存储”会掩盖关键差异：有些层按字节随机访问，有些层按 cache line 交换，有些层按页或块传输，有些层只能通过命令队列返回 completion。



图示：内存层次不是一条“更大的数组”，而是一组访问粒度和完成语义不同的层。

层次	常见粒度	谁直接访问	主要硬件问题
寄存器	word 或向量 lane	执行单元	端口数、旁路、写回冲突
L1 cache	cache line	当前核心 load/store 单元	命中时延、虚实地址、别名和写策略
L2/L3 cache	cache line	一个核心或多个核心	容量、共享、替换、一致性
DRAM	burst/cache line	内存控制器	bank、行命中、刷新、ECC、调度
SSD/硬盘	块、页或扇区	设备控制器和 DMA	命令队列、坏块、磨损、错误恢复
网络/显卡等设备	描述符、包、帧、buffer	DMA 引擎和设备内部队列	所有权、completion、IOMMU、cache 可见性

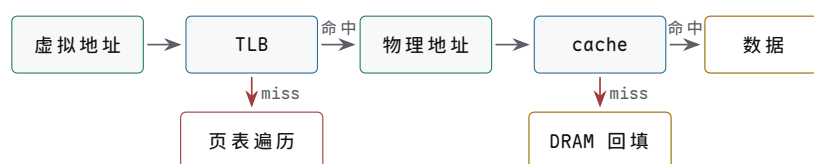
从程序角度看，`load x` 只是取一个变量；从硬件角度看，它可能命中寄存器旁路、命中 L1、命中 L2、命中共享 L3、访问 DRAM、触发页表遍历、发生缺页，甚至等待块设备把页面读回主存。CSAPP 强调的 *locality* 就是让常见访问停留在左侧短路径。硬件书不能只说“内存很快/很慢”，而要说明每一级的访问粒度、所有者、完成条件和失败路径。

专题：内存别名与同源不同名的 cache 问题 上一专题的层次表把“别名”列为 L1 cache 的关键问题之一，这里把它展开。TLB 的存在意味着同一个物理页可以被多个虚拟页映射：共享库、进程间共享内存、同一页在用户态和内核态各有别名，都是日常情形。这带来一个只在“用虚拟地址索引 cache”时才出现的问题：两个不同的虚拟地址指向同一物理行，若 cache 按虚拟地址的低位选槽，而两个别名的虚拟页号不同，index 就可能不同，同一物理行会被存进两个不同的槽。之后程序经别名一写入，经别名二读到的却是另一个槽里的旧副本——同一数据在 cache 里有了两份不一致的拷贝。

看一个小例子。页大小 4 KiB，cache 为 32 KiB 直接映射、64 B 行：offset 为 6 bit，行数 = $32\text{ KiB} \div 64\text{ B} = 512$ ，index 取地址的 bit 14..6 共 9 bit。页内偏移只有 12 bit (bit 11..0)，因此 index 的高 3 位 (bit 12..14) 来自虚拟页号——别名地址的差异正好在这几位里。设虚拟地址 0x1040 和 0x2040 都映射到物理地址 0x35040：前者的 index 是 65，后者的 index 是 129，同一物理行会被分别存进槽 65 和槽 129。经 0x1040 写入后，槽 65 是新值，槽 129 仍是旧值，两次读到同一物理地址却得到不同数据。

两种硬件解法基于同一条思路：让同一物理行的所有别名映到同一个槽，再由物理 tag 确认身份。第一种是物理索引 (PIPT)：index 直接取自物理地址，0x1040 和 0x2040 经 TLB 翻译后都是 0x35040，index 都是 321，必然同一个槽，tag 也比较物理地址，别名问题根本不会出现；代价是要先等 TLB 翻译完成才能开始查 cache，命中路径变长。第二种是限制 index 位数 (VIPT 的常见约定)：把 cache 设计成 index 加 offset 不超过页内偏移的 12 bit，例如 4 KiB 直接映射，或 32 KiB、8 路组相联——组数 = $32\text{ KiB} \div 64\text{ B} \div 8 = 64$ ，index 只需 6 bit，index 加 offset 正好 12 bit。此时 index 全部来自页内偏移，而任何别名的页内偏移都与物理地址相同：上例中两个别名与物理地址的 bit 11..6 都等于 1，必然映到同一组。查找与 TLB 翻译并行进行，拿到物理页号后再与组内各路的物理 tag 比较，第二次访问命中同一个行，两份副本无从产生。现代处理器的 L1 大多采用这种“虚拟索引、物理 tag、index 不越出页内偏移”的安排，既保留并行查找的速度，又让别名退化为普通命中。

专题：cache 与 TLB 是两套不同的近路 cache 缓存数据或指令内容，TLB 缓存虚拟页到物理页的地址转换。二者经常同时出现在一次 LOAD 路径上，但回答的问题不同：TLB 问“这个虚拟地址能否翻译成哪个物理地址”；cache 问“这个物理位置的内容是否已经在近处”。把二者混在一起，会导致读者无法理解为什么 TLB miss 不等于 cache miss，页错误也不等于总线错误。



图示：TLB miss 发生在地址转换阶段，cache miss 发生在数据回填阶段。二者都可能让 LOAD 变慢，但失败位置不同。

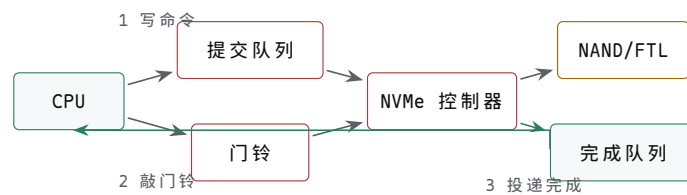
事件	真实含义	后续动作
TLB hit + cache hit	地址转换和数据都在近处	LOAD 快速返回
TLB miss + 页表命中	转换不在 TLB，但页表有效	硬件或软件页表遍历后填 TLB
页不存在	页表没有有效映射	进入页错误异常，可能由 OS 分配或从磁盘读页
cache miss	地址有效，但数据不在当前 cache	从下一级 cache 或 DRAM 回填 cache line

权限错误 转换存在但访问类型不允许 报保护异常，不能提交数据

专题：SSD 和硬盘不是慢内存，而是块设备 硬盘和 SSD 都不能像 SRAM 那样把地址线接上就读一个字节。CPU 要通过控制器提交命令：读哪个逻辑块、读多长、放到哪个主存缓冲区、完成后写哪个 completion。硬盘的内部约束来自机械寻道和旋转；SSD 的内部约束来自 NAND flash 页读写、块擦除、FTL 映射、磨损均衡、垃圾回收和掉电保护。它们对 CPU 暴露的共同形态，是队列化块设备。

对象	内部结构重点	对系统可见的边界
机械硬盘	盘片、磁头、磁道、扇区、缓存	随机访问受寻道和旋转影响，顺序访问更友好
SATA SSD	NAND flash、FTL、通道、擦除块	写放大、垃圾回收、坏块管理、掉电保护
NVMe SSD	PCIe、多队列、doorbell、completion queue	并发队列深度高，CPU 通过 MMIO 和 DMA 协作
SD/eMMC	控制器隐藏 flash 细节，命令集较简单	嵌入式启动和存储，延迟波动明显

理解 SSD 时要避免两个误解。第一，SSD 没有机械寻道，不等于每次访问都一样快；内部垃圾回收、擦除、映射表缓存和热控都会让尾延迟上升。第二，逻辑块地址连续，不等于物理 NAND 页连续；FTL 会把逻辑写重映射到新位置，再在后台回收旧块。因此块设备的正确抽象不是“巨大数组”，而是“命令队列 + DMA 缓冲区 + completion + 错误恢复”。



图示：NVMe 的基本形态：CPU 写队列和门铃，设备 DMA 读写主存，completion 才说明请求完成。

专题：读硬件结构时要区分数据路径和控制路径 CPU、cache、DRAM、SSD、网卡和显卡都可以用同一个提问框架阅读：数据从哪里到哪里，控制由谁发起，完成由谁报告，错误由谁保存。低质量描述常把这些混成“设备把数据给 CPU”。更好的描述应写出两条路径：数据可能经 DMA 在设备和主存之间移动；控制通常由 CPU 写 MMIO、设备写 completion、CPU 读状态或处理中断。

路径	典型方向	必须说明
控制路径	CPU 到 MMIO/doorbell，设备到状态/中断	命令何时被接收，完成如何通知
数据路径	设备 DMA 与主存缓冲区之间	缓冲区地址、长度、cache 可见性、权限
错误路径	设备或互连到错误码/异常	超时、校验、越界、权限和重试策略
恢复路径	驱动或固件到 reset/retry/rebuild	哪些状态保留，哪些队列丢弃或重放

章末练习

任务	要求	预期结果
----	----	------

SRAM 读写周期	画出地址、CS、OE、WE、数据线和采样窗口	能指出谁驱动数据线，谁采样
地址映射	设计 64 KiB ROM/RAM/GPIO/定时器映射	任意地址最多一个片选有效
等待状态	为慢 ROM 和慢 I/O 设计 READY/WAIT 规则	等待期间地址和方向稳定，有超时策略
MMIO 寄存器	设计 GPIO 的 DIR/OUT/IN/STAT 语义	外部输入经过同步，状态清除规则明确
中断顺序	写出设备 ready 到 CPU 服务程序返回的 trace	不丢事件，不重复清旧事件
DMA 顺序	写出描述符、缓冲区、completion 和 cache 可见性边界	CPU 和设备所有权不重叠
valid/ready	用 valid/ready 描述一次读请求和读响应	能说明背压和 completion 的作用
cache 行	用 tag/index/offset 解释一次 cache 命中和一次 miss	能说明 cache line、valid、dirty 和替换边界
MMIO 属性	说明为什么设备寄存器通常不能按普通 cacheable 内存处理	能指出副作用、顺序和可见性要求
未映射访问	规定未映射地址的超时、错误或固定返回值	CPU 不会永久等待未知目标
存储层次图	画出寄存器、L1/L2/L3、DRAM、SSD/硬盘的数据粒度和完成语义	能区分 cache line、页、块和 completion
TLB/cache 区分	写出一次 LOAD 中 TLB hit/miss、页错误、cache miss 的不同路径	不把地址转换错误和数据回填混为一谈
块设备队列	为 NVMe 或简化 SSD 写出 submission、doorbell、DMA、completion 顺序	能说明 CPU 写门铃不等于 I/O 完成
DRAM 时序计算	给定 tRCD、tCL、时钟周期和突发长度，估算一次随机读时延，并与行命中情形对比	三段时延拆分正确，量级为几十 ns
UART 帧分析	给定 RX 波形和标称波特率，数出起始位、数据位、停止位，还原字节并核对波特率误差	帧结构和误差判断都有依据
组相联查找	用本章的四次访问序列，分别在直接映射和 2 路组相联下走一遍	能指出哪几次由 miss 变 hit 及原因
描述符环顺序	写出 CPU 提交两个描述符到设备完成的 SW/HW 指针、doorbell 与完成写回顺序	doorbell 不当作完成，写回可见性明确

bank 交错拍数	4 个 bank 按 cache line 轮转交错，单 bank 一次访问占用 4 拍，bank 间每拍可启动一个新请求；计算连续读 8 条相邻 cache line 的总拍数，并与单 bank 串行对比	交错 11 拍、单 bank 32 拍，能写出每行的启动与完成拍
替换策略	2 路组相联、LRU，某组初始为空，依次访问 A、B、A、C、A、B；逐步写出命中/失效和组内容，并与同 index 直接映射对比	2 路共 3 次 miss，直接映射 6 次全 miss
预取演算	顺序读 16 条 cache line，每次 miss 延迟 30 拍、每行处理 8 拍；计算无预取与 next-line 预取的总拍数，并求能完全隐藏预取的最小处理节拍	608 拍对 488 拍，处理节拍不小于 30 才能完全隐藏
I2C 事务序列	写出主机从 7 位地址 0x50 的从机寄存器 0x10 读 2 字节的完整序列：START、地址字节、读写位、ACK/NACK、repeated START、STOP	0xA0/0xA1 正确，最后一字节 NACK，repeated START 位置正确
仲裁对照	用一句话分别说明集中式仲裁器、菊花链、轮询计数的线数、优先级和公平性，并指出片上互连中仲裁的位置	三种做法的取舍清楚，仲裁做进交换节点
MESI 走查	两核初始均为 I，按“A 读 X、B 读 X、A 写 X、B 读 X”写出每步两核状态和互连消息	每步最新值位置唯一，失效与写回/转发正确

参考解答与要点

任务	参考解答	必须保留的要点
SRAM 周期	读时地址和片选稳定，OE 有效、WE 无效，由 SRAM 驱动数据；写时 OE 无效，由发起者驱动数据，WE 在地址和数据稳定后有效。	输出使能互斥，写窗口满足建立/保持时间。
地址映射	例如低半区 ROM，高半区中一块 RAM，最高 4 KiB 为 I/O；用高位地址产生互斥片选，未映射区返回错误或超时。	片选互斥和未映射行为必须写入规格。
等待状态	慢目标在 ready 之前保持事务未完成，控制器保持地址、片选和方向；超过最大等待则返回错误。	等待状态是硬件事务延长。
MMIO	GPIO 输出写入锁存器，输入来自同步后的外部引脚，状态位说明 ready/error/irq，清除规则避免新旧事件混淆。	MMIO 有副作用，不是普通变量。
中断	设备锁存事件并请求中断，中断控制器投递，CPU 进入入口，服务程序读状态、处理数据、按设备规则清事件，再发送 EOI 或恢复中断。	清除顺序决定是否丢事件。

DMA	CPU 准备描述符和缓冲区并保证对设备可见，设备搬运数据并投递 completion, CPU 在 completion 后按一致性规则读取缓冲区。	描述符可见、数据可见和缓冲区所有权必须分开。
valid/ready	只有 valid 和 ready 同时为 1 才传输；ready 为 0 时发送方保持 payload；读请求和读响应可隔多个周期。	背压不等于错误，completion 才是完成标志。
未映射访问	未映射区域可以返回固定值、产生错误或等待到超时，但规则必须固定且可观察；不能让总线悬空成为随机值。	错误路径也是硬件规格。
cache 行	先算 $\text{offset}=\log_2(\text{行大小})$ 、 $\text{index}=\log_2(\text{行数})$ 、tag= 剩余位；命中要求槽 valid 且 tag 相等；miss 时取整行填入并更新 tag/valid, dirty 位决定替换时是否要先写回下一级。	三个字段的分工和 valid/dirty 语义不能丢。
MMIO 属性	设备寄存器有副作用：写是命令、读可能清状态；若被 cache 缓冲或合并，设备可能永远看不到命令，CPU 也可能一直读到旧状态，所以设备内存要映射为不可缓存、不可合并、顺序保持。	副作用、顺序、可见性三条都要答到。
存储层次图	寄存器（字，当拍完成）、L1/L2/L3 (cache line, 命中即返回)、DRAM (行/页，控制器调度与刷新)、SSD/硬盘 (块, I/O 请求和 completion); 越往下粒度越大，完成语义越依赖显式通知而不是当拍返回。	粒度与完成语义要对应，不能只画容量和速度。
TLB/cache 区分	TLB miss 发生在地址转换阶段：硬件查页表，缺页则走页错误异常；cache miss 发生在拿到物理地址之后：向下一级回填整行，CPU 只是停顿或乱序等待。页错误是控制流改道，cache miss 只是数据慢。	转换错误与数据回填是两条独立路径。
块设备队列	CPU 把描述符写进提交队列，写 doorbell 通知设备；设备 DMA 取走描述符、搬完数据后写完成队列，再（可选）发中断。doorbell 只表示“有新请求”，不代表 I/O 完成，完成以完成队列项为准。	submission 与 completion 分离，doorbell 不等于完成。
DRAM 时序	一次需要开行的随机读约为 tRCD（开行）加 tCL（列出数）加突发传输，常见 DDR3/DDR4 器件合计约 30-40 ns 量级；行命中可省去 tRCD，行冲突还要先 precharge。	时延由行、列、突发三段组成，不是固定常数。
UART 帧	先找起始位下降沿，再按位时间中点逐位采样：1 起始位、8 数据位（低位在前）、1 停止位；还原字节后核对实测位时间与标称波特率的偏差，双方合计误差一般不超过约 2%。	位边界、采样点和误差都要核对。

组相联	index 选中整个组，组内两路 tag 并行与地址 tag 比较，任一路相等且 $V=1$ 即 hit；两个热地址各占一路后不再互相驱逐；miss 时按替换位选一路驱逐。	并行比较、mux 和替换位缺一不可。
描述符环	CPU 只推进 SW 指针，设备只推进 HW 指针，末尾回绕；doorbell 只提示有新描述符，不代表完成；完成以设备写回状态位或 completion 项为准，回收前确认写回可见。	指针分离、doorbell 非完成、写回可见性。
bank 交错	第 1..4 拍依次启动第 1..4 行（各占一个 bank），第 i 行在第 i 拍启动、第 $i+3$ 拍完成；第 5 行在第 5 拍回到 bank 1 时前一次访问刚结束，无需等待。第 8 行在第 11 拍完成，共 11 拍。单 bank 串行需 $8 \times 4 = 32$ 拍。	加速来自启动流水化，单 bank 本身没有变快。
替换策略	2 路 LRU: A miss 得 {A,-}; B miss 得 {A,B}; A hit (B 成为最久未用); C miss 驱逐 B 得 {A,C}; A hit; B miss 驱逐 C 得 {A,B}, 共 3 次 miss。直接映射下同 index 只有一个槽，每次访问都驱逐上一行，6 次全 miss。	LRU 驱逐“最久未用”，不是“最早进入”。
预取演算	无预取：每行 miss 30 拍加处理 8 拍， $16 \times 38 = 608$ 拍。next-line 预取：第 1 行第 30 拍就绪，处理第 i 行的 8 拍只能隐藏预取的一部分，第 i 行数据在第 $30i$ 拍就绪，稳态每行 30 拍，总 $30 \times 16 + 8 = 488$ 拍。处理节拍 $T \geq 30$ 时预取被完全隐藏，总拍数降为 $30 + 16T$ 。	预取隐藏延迟但不消除首次 miss；处理太慢时仍受 miss 间隔限制。
I2C 事务	START $\rightarrow 0xA0$ ($0x50$ 左移一位、读写位 0 表示写) $\rightarrow$ 从机 ACK $\rightarrow 0x10$ (寄存器号) $\rightarrow$ ACK $\rightarrow$ repeated START $\rightarrow 0xA1$ (读写位 1 表示读) $\rightarrow$ ACK $\rightarrow$ 读第 1 字节 $\rightarrow$ 主机 ACK $\rightarrow$ 读第 2 字节 $\rightarrow$ 主机 NACK $\rightarrow$ STOP。	先写寄存器地址，repeated START 转读；最后字节 NACK 通知从机结束。
仲裁对照	集中式：每主设备一对请求/授权线，仲裁器并行比较，最快但线数为 $2N$ 且有单点；菊花链：授权逐级传递，线数与设备数无关，但优先级由位置固定、传播慢；轮询计数： $\lceil \log_2 N \rceil$ 根扫描线，从上次授权点继续则轮转公平，但平均扫半表最慢。片上互连把仲裁做进每个交换节点的端口。	三者在线数、延迟、公平性之间取舍；仲裁没有消失。
MESI 走查	步骤 1: A 由 I 变 S, B 保持 I (读入共享副本); 步骤 2: B 由 I 变 S, A 保持 S; 步骤 3: A 广播失效, B 由 S 变 I, A 由 S 变 M, 内存过期; 步骤 4: B 读 miss, A 写回或转发后由 M 降为 S, B 得 S, 内存恢复最新。	每步最新值唯一: S 期间在内存, M 期间在 A 的 cache。

经典芯片、启动与平台演进

前面已经把最小 CPU 和整机事务分开讲清楚。本章再回到真实芯片：6502、Z80、8051、8086、80386、80486、Pentium 和现代 SoC 不只是历史名称，而是硬件边界逐步扩展的样本。读这些芯片时，不应只背年份、位宽或指令集，而要问：状态保存在哪里，地址怎样形成，外部事务怎样发起，异常和等待怎样改变下一步，哪些功能被逐步搬进芯片内部。

芯片	本章观察重点	对整机理解的贡献
6502/Z80	8-bit CPU、外部地址/数据总线、控制周期	看见 CPU、存储器和 I/O 如何靠板级总线协作
8051	CPU、片内存储、I/O、定时器和中断集成	看见单片机把一部分整机控制收进芯片
8086/8088	16-bit x86、20-bit 地址、复用总线和预取队列	看见早期微处理器如何在引脚和地址空间之间取舍
80386	32-bit、保护模式、分页和异常机制	看见 CPU 如何承担系统级地址和权限约定
486/Pentium	片上 cache、流水线、浮点和预测方向	看见低时延来自片上集成和执行结构重组
现代 SoC	CPU、cache、内存控制器、I/O、安全启动同片集成	看见整机边界继续向片内移动

本章采用经典系统教材常用的标注方式：不把 CPU、总线、cache、I/O 当作孤立名词，而把它们写成可追踪的事务。阅读任何一颗芯片时，都应同时回答五个问题：状态保存在哪里，组合路径经过哪些部件，控制信号在什么时候有效，外部事务由谁驱动和谁采样，异常或等待如何改变下一步。



演进主线不是“位宽越来越大”这么简单，而是地址形成、等待隐藏、异常诊断和外设事务逐步进入芯片内部。

图示：经典芯片观察线索。每一代都把前一章的事务边界移动、扩宽或缓存。

一、早期微处理器：从 6502、Z80 到 8086

在 8086 之前，8-bit 微处理器已经把“CPU 加外部总线”这台戏完整演过一遍。6502、Z80 和 8051 各自回答了一个整机问题：寄存器太少怎么办，上下文切换和 DRAM 刷新如何省掉总线事务，整机部件能不能收进同一颗芯片。

6502 用极少的寄存器和零页换取简单的数据通路 6502 的编程模型只有累加器 A、变址寄存器 X 和 Y、栈指针 SP、程序计数器 PC 和标志寄存器 P，没有一组可自由选用的通用寄存器；整颗芯片约 3500–4500 个晶体管，数据通路围绕累加器组织，没有乘除法指令。寄存器这样少，工作数据就必须经常放在存储器里，6502

的对策是把地址空间最低的第 0 页 (0x0000-0x00FF) 当作“伪寄存器堆”使用：零页寻址的指令只有 2 个字节，3 个时钟拍完成一次读写，比访问任意 16-bit 地址更短更快。栈则被硬件固定在第 1 页 (0x0100-0x01FF)，SP 因此只需保存 8-bit 页内偏移。数组处理依靠 (zp),Y 间接变址：从零页的一对字节取出 16-bit 基地址，再加上 Y 作为元素偏移，一次读用 5 拍，相加后若跨页再加 1 拍。

指令形式	拍数	对应的总线动作
LDA 立即数	2	取操作码，再把操作数字节直接读入 A
LDA 零页	3	取操作码，取零页地址，从第 0 页读数据
LDA 绝对地址	4	取操作码，取 16-bit 地址的两个字节，再读数据
LDA (zp),Y	5+1	取操作码，从零页读基地址两字节，加 Y 后读数据，跨页加 1 拍
STA (zp),Y	6	取操作码，从零页读基地址两字节，加 Y 形成地址后写入，写比读多一拍
JMP 绝对地址	3	取操作码，取目标地址两个字节并装入 PC

每条指令的拍数固定而且很小，总线在每个时钟拍上完成一个确定动作，因此 6502 程序可以手工逐拍 trace（执行轨迹）：哪一拍取指、哪一拍读零页、哪一拍形成地址，全能数清楚。6502 的便宜正来自这种简单：没有乘除法，没有多通用寄存器，数据通路只围绕累加器，芯片面积和售价才可能压到最低。

280 用双寄存器组和取指刷新换取系统效率 280 在软件上与 8080 兼容，并在此之上做了几处扩展。最显眼的是寄存器组：主寄存器组 AF、BC、DE、HL 之外还有整套备用组 AF'、BC'、DE'、HL'，EX 和 EXX 各用一条指令完成整组交换。中断响应时不必把寄存器逐个压栈再逐个恢复，现场切换不发生任何存储器访问。第二个扩展与 DRAM 刷新有关：R 寄存器保存刷新行地址，在每个 M1 取指周期的后半段——此时指令字节已经读入、地址总线正好空闲——自动把 R 输出到地址总线上完成一次刷新，随后自动加一；刷新被“借”进取指周期，外部不再需要单独的刷新周期。此外，I 寄存器配合中断模式 2 提供向量页地址，IX、IY 变址寄存器扩展了寻址方式。

280 机制	硬件做法	换取的收益
主/备寄存器组	AF/BC/DE/HL 与 AF'/BC'/DE'/HL'，EX/EXX 整组交换	中断现场切换不访问存储器
R 寄存器	M1 取指后半段输出刷新地址，随后自动加一	DRAM 刷新不占额外总线周期
I 寄存器与中断模式 2	I 提供向量页地址，设备提供页内偏移	中断入口可以成表组织
IX/IY 变址寄存器	变址基址加偏移形成有效地址	数据结构和栈帧访问更直接

280 的教学点有两条。第一，寄存器越多，上下文切换越少访问内存：一组备用寄存器换来的是中断响应里整段存储器往返的消失。第二，刷新是“借用已有事务”的设计：DRAM 刷新本来要占用专门的总线时间，280 把它塞进每个取指周期里必然出现的空闲半段，系统里就再也看不见它。

8051 把一部分整机收进同一颗芯片 8051 是单片机：CPU、4-8 KiB ROM/EPROM、128-256 B 片内 RAM、4 个 8-bit 端口、2-3 个定时器、UART 和中断控制器全部在同一颗芯片内。片内 RAM 分区使用：最低 32 字节是 4 组工作寄存器 R0-R7，由程序状态字 PSW 的 RS0、RS1 两位选组，一条改选组的指令就完成一次快速上下文切换；0x20-0x2F 的 16 字节是位寻址区，其中 128 个 bit 可以单独寻址、置位

和清零，标志位不必占用整个字节；其余部分是通用区。片内外设寄存器称为特殊功能寄存器（SFR），占用 `0x80-0xFF` 的直接地址。四个端口采用准双向结构：引脚锁存器先写入 1，该引脚才能作为输入被读取。

8051 的教学点直接连着第五章：把 ROM、RAM、地址译码、MMIO、定时器、串口和中断控制器收进一颗芯片之后，“总线”变成了片内连线，板级事务缩成片内信号，但语义没有改变—写 SFR 仍然是写设备寄存器，读端口仍然是 MMIO 读，定时器溢出仍然是一个中断事件。整机边界在这里第一次明显移动，而读者已经学过的所有约定原样保留。

8086 是理解早期 x86 体系的重要起点。它是 16-bit 微处理器，拥有 16-bit 通用寄存器和 16-bit 数据通路特征，同时通过 20-bit 地址能力访问最多 1 MiB 物理地址空间。它的历史意义不只在指令集，还在于展示了 CPU 芯片如何通过外部总线、地址锁存、控制信号和存储器/外设共同构成一台机器。

8086 的寄存器组包括 AX、BX、CX、DX、SP、BP、SI、DI，以及 CS、DS、SS、ES 等段寄存器。段寄存器和偏移地址共同形成物理地址：

```
physical_address = segment * 16 + offset
```

这个规则反映了一个时代的硬件取舍：内部仍以 16-bit 编程模型为主，但外部需要访问超过 64 KiB 的地址空间，于是通过段基址左移和偏移相加得到 20-bit 地址。分段不是高级抽象，而是地址形成硬件的一部分。

8086 的外部总线也值得注意。地址线和数据线存在复用，某些引脚在总线周期的不同阶段承担不同含义。硬件需要地址锁存器在地址有效阶段保存地址，随后这些引脚可以用于数据传输。一次总线访问不是“读内存”三个字，而是分阶段发生的事务：输出地址、锁存地址、给出读写控制、等待存储器或 I/O 响应、传输数据、结束总线周期。

8086 观察点	硬件含义
16-bit 寄存器	数据运算和编程模型以 16-bit 为核心
20-bit 地址形成	通过段寄存器和偏移访问 1 MiB 物理空间
地址/数据复用总线	芯片引脚有限，外部需要锁存和分阶段传输
存储器与 I/O 周期	访问目标由控制信号和地址空间规则区分
BIU/EU 分工	取指队列和执行部件可部分重叠工作

8086 的典型板级连接还需要外部支持器件。地址/数据复用引脚在 T1 阶段输出地址，在后续阶段承载数据，因此常用地址锁存器保存低位地址；时钟、复位和 ready 同步可由时钟发生器组织；最大模式下，总线控制信号还可由总线控制器从状态信号译出。读数据手册时，不应只看“8086 是 16-bit CPU”，还要看它把哪些系统责任交给了板级芯片。

信号或器件	硬件作用	关键问题
ALE	地址锁存允许，提示外部锁存低位地址	锁存器是否在地址有效窗口保存地址
AD0-AD15	复用地址/数据线	哪个阶段由 CPU 驱动，哪个阶段由存储器或 I/O 驱动
A16-A19/S3-S6	高位地址与状态复用	地址是否在访问开始阶段被正确保存
RD/WR	读写控制	目标芯片输出使能和写使能是否互斥
M/IO	区分存储器周期和 I/O 周期	地址译码是否把事务送到正确目标
READY	外部目标请求插入等待状态	慢存储器是否能可靠延长总线周期

INTR/NMI	可屏蔽和不可屏蔽中断入口	请求是否同步，响应和清除顺序是否明确
8284/8288	时钟/复位/ready 或总线控制辅助	外部控制信号是否满足时序和极性要求

读总线周期要按 T 状态逐段读 上一张信号表说明了每个引脚负责什么，但还不够：同一个总线周期里，同一组引脚在不同时间段做不同的事。8086 把一次总线访问切成 T1、T2、T3、T4 四个状态，必要时在 T3 与 T4 之间插入等待状态 Tw。读手册的正确方式是逐状态确认：这一拍谁在驱动地址/数据引脚，哪个控制信号有效，外部芯片应在哪个边沿锁存或采样。

T 状态	地址/数据引脚	控制信号	外部动作
T1	CPU 在 AD0-AD15 和 A16-A19/S3-S6 上输出地址	ALE 拉高	地址锁存器在 ALE 下降沿保存地址
T2	地址从复用脚撤出，引脚准备转入数据方向	RD（读）或 WR（写）有效	数据收发器按读/写确定传输方向
T3	读周期中由目标芯片驱动数据线	控制保持，READY 为低则插入 Tw	存储器或 I/O 把数据放到总线上，等待状态期间地址和控制保持不变
T4	CPU 采样读入的数据，或结束写周期	控制信号撤销	目标释放数据线，总线周期结束

写周期与读周期骨架相同，差别在数据方向：CPU 从 T2 起亲自驱动数据线，并保持到写窗口结束，目标芯片在写控制有效的窗口内采样数据。“读总线周期”读到最后，就是把每个 T 状态里谁驱动什么、谁采样什么写清楚；有了这张表，上一张信号表里的每个信号才有了时间轴上的位置。

8086 的存储体选择是理解 16-bit 总线的关键 8086 有 16-bit 数据总线，但外部存储器常按 8-bit 芯片组织成偶地址低字节存储体和奇地址高字节存储体。地址最低位 A0 与高字节使能 BHE 共同决定哪些字节参与本次传输。这样，16-bit 字访问在偶地址上可以一次完成；若字从奇地址开始，通常需要拆成两个总线周期。

A0	BHE	参与字节	典型含义
0	0	D0-D7 与 D8-D15	偶地址 16-bit 字访问
0	1	D0-D7	偶地址低字节访问
1	0	D8-D15	奇地址字节访问
1	1	无有效字节	正常访问中不应作为有效传输

这张表能解释为什么“16-bit CPU”不等于每次总线访问都简单搬 16 bit。存储芯片、地址译码器和数据收发器必须理解字节车道。若把两个 8-bit 存储体简单并在一起而不处理 A0 和 BHE，字节写入会破坏相邻数据，奇地址字访问也无法正确完成。

最小模式、最大模式和外部控制器 8086 可以在最小模式下直接输出较多总线控制信号，也可以在最大模式下把状态信息交给外部总线控制器生成控制信号。前者适合单处理器小系统，后者便于接入更复杂的总线环境。这个差别说明，CPU 引脚不是固定地“表示某个概念”，它们会随系统模式承担不同职责。

系统形态	典型外部对象	学习重点
------	--------	------

最小模式	地址锁存器、数据收发器、ROM/RAM、简单 I/O	CPU 直接给出读写、方向、使能等控制
最大模式	8288 总线控制器、总线仲裁、协处理或多主设备	状态信号被外部控制器译成总线命令
8088 小系统	8-bit 外部数据总线、较低成本板级存储器	编程模型相近，外部总线带宽和周期不同

一个 8086 存储器读周期可以按 T 状态走读。T1 中，CPU 把地址放到复用总线并拉出 ALE，外部锁存器保存地址；T2 中，CPU 改变总线方向并给出读控制；T3/Tw 中，存储器或 I/O 设备把数据放到数据线上，若 READY 不满足，CPU 插入等待状态；T4 中，CPU 采样数据并结束周期。这个过程把第一章的三态总线、第二章的时钟边界和第三章的控制 trace（执行轨迹）合到一起：总线上的每一段时间都必须有唯一驱动者和明确采样者。

8086 memory read sketch:

T1: address valid, ALE latches address
T2: RD active, bus turns around for target data
T3: target drives data, READY may insert wait states
T4: CPU samples data, control deasserts



复用总线不是一根线同时表示所有含义，而是在同一个读周期内按 T 状态依次承担地址、方向、数据和完成边界。

图示：8086 存储器读周期。地址锁存、总线转向、等待和采样必须分阶段观察。

8086 写周期、中断响应和总线让出也要按事务读 读周期只说明目标驱动数据线；写周期则相反，CPU 在合适阶段驱动数据线，外部存储器或 I/O 设备在写控制有效窗口内采样。若数据收发器方向或写使能时序错误，写入可能丢失，也可能造成 CPU 和外设同时驱动总线。8086 的中断响应周期又是另一类事务：CPU 不访问普通内存数据，而是响应中断控制器，取得中断类型或进入规定入口。

事务	总线动作	观察重点
存储器写	地址先被锁存，CPU 驱动数据， WR 有效，目标采样	数据方向、字节存储体和写窗口
I/O 写	地址或端口号选择外设寄存器，CPU 驱动命令或数据	M/I/O 或 I/O 周期控制、端口译码
中断响应	CPU 接受 INTR 后执行响应周期，中断控制器给出类型或向量信息	请求保持、优先级、响应和屏蔽状态
DMA 让总线	DMA 请求总线，CPU 完成当前边界后让出，总线主控权转移	HOLD/HLDA 类握手、三态释放、仲裁

8086 memory write sketch:

T1: address valid, ALE latches address
T2: CPU drives write data, WR becomes active
T3: target samples data, READY may insert wait states
T4: WR deasserts, bus returns to idle or next cycle

这组事务可以直接指导早期微机板级排错。若读 ROM 正常、写 RAM 失败，应重点查数据收发器方向、**WR** 极性、字节存储体选择和 SRAM 写入时间；若外设中断不

进入，应同时查外设请求、8259 屏蔽位、CPU 中断允许状态和中断响应周期；若 DMA 后内存内容异常，应查总线让出时 CPU 是否真正三态、DMA 地址计数是否正确、目标存储器是否满足时序。

8086 内部常被划分为总线接口单元和执行单元。总线接口单元负责取指、形成物理地址、访问外部总线，并维护预取队列；执行单元负责译码和执行指令。这个分工说明，即使没有现代深流水，CPU 内部也已经在尝试把“取指访问外部世界”和“执行内部运算”分开，以减少外部存储访问延迟对执行的影响。

从本书前面建立的硬件模型看，8086 可以视为一个更复杂的同步系统：PC 在 x86 语境中体现为指令指针和代码段组合；地址形成部件把段和偏移送入加法路径；总线接口把地址和控制信号变成外部事务；执行部件译码指令并驱动寄存器、ALU 和标志位。8086 不是脱离前面章节的新对象，而是前述部件在真实芯片中的组合。

8086 还说明“低时延”在早期处理器中已经是系统问题。预取队列试图在执行部件工作时提前从外部总线取回后续指令，减少执行部件等待取指的时间。这个机制不等于现代流水线和分支预测，但思想相通：把慢的外部访问与内部执行尽量重叠。若分支改变控制流，预取队列中已经取得的顺序指令可能失效；若外部总线慢于指令消费速度，执行部件仍会等待。这里可以看见第一章快慢路径表的早期版本。

围绕 8086/8088 的经典外围芯片同样值得提前认识。8255 可编程并行接口把若干引脚组织成可配置 I/O 口；8253/8254 定时器提供周期性计数和中断来源；8259 中断控制器把多个外设请求汇总给 CPU；8237 DMA 控制器让设备和内存之间进行批量搬运；8250/16450/16550 UART 把串行线上的位流转换为 CPU 可读写的寄存器和 FIFO。它们共同说明，整机不是一颗 CPU，而是一组围绕地址、数据、控制和完成事件协作的芯片。

经典芯片	解决的问题	与本章主题的连接
8255 PPI	把并行引脚配置成输入、输出或握手端口	GPIO 和设备寄存器的早期形态
8253/8254 PIT	用计数器产生周期、延时和中断节拍	定时器寄存器与完成事件
8259 PIC	汇总、屏蔽和优先级排序中断请求	中断控制器和 CPU 入口
8237 DMA	在设备和内存之间搬运数据	总线主设备、仲裁和 cache 可见性
8250/16550 UART	串行收发、状态位和 FIFO	设备状态、数据寄存器和中断通知

8088/8086 最小系统可以按模块逐步实现 若要把 8086 类系统搭成现实电路，不应从“跑 DOS”开始，而应从最小可运行系统开始。8088 常用于教学，因为外部数据总线为 8 bit，存储器接口更容易搭建；8086 则保留 16-bit 外部数据总线，需要同时处理奇偶字节存储体。无论选择哪一种，系统都要包括时钟/复位、地址锁存、数据收发、ROM、RAM、I/O 译码和至少一个可观察外设。

模块	典型器件或做法	第一轮检查
时钟/复位	8284 或等价时钟复位电路	复位保持期间写控制无效，释放后第一拍地址可预测
地址锁存	74LS373/8282 锁存 AD 线上低位地址	ALE 有效窗口内保存地址，数据阶段地址不丢失
数据收发	74LS245/8286 控制数据方向	读周期外设驱动，写周期 CPU 驱动，不发生总线争用

启动 ROM	EPR0M/Flash 映射到复位入口	复位入口读出跳转或初始化代码
SRAM	静态 RAM 与字节使能/写使能连接	写一个地址后再读回同一数据，邻近字节不变
I/O 译码	74LS138、比较器或 PAL/GAL	8255、8253、8259、UART 等端口互不重叠
可观察外设	LED 锁存器、按键、串口或逻辑分析仪探针	CPU 写端口后能看到确定副作用

这样的系统可以分三步 bring-up。第一步只让 CPU 从 ROM 取指，确认复位入口、地址锁存和读周期成立；第二步加入 SRAM，执行一个写后读回的小程序，确认写周期、数据方向和等待状态成立；第三步加入 8255 或简单 LED 锁存器，确认 I/O 周期、端口译码和外设副作用成立。每一步都应记录地址线、ALE、读写控制、数据总线方向和目标片选。若第一步没有通过，后续软件调试没有意义。

等待状态发生器是慢器件和 CPU 之间的约定 8086/8088 的 READY 信号把固定周期总线变成可延长事务。慢 ROM、慢 SRAM、外设端口或板级译码较深时，目标可以通过 READY 让 CPU 插入等待状态。等待状态不是“让程序慢一点”的软件概念，而是总线周期中的硬件暂停：地址、控制和方向必须保持可解释，目标在满足访问时间后再允许 CPU 采样或结束写周期。

对象	READY/等待的作用	关键问题
慢 ROM	延长取指或读常量周期	等待期间地址和片选是否保持稳定
慢 SRAM	延长数据读写窗口	写周期中数据是否覆盖完整写入窗口
I/O 外设	给外设状态机足够时间返回数据	外设未 ready 时 CPU 是否错误采样浮空总线
DMA/仲裁	让总线所有权变化落在安全边界	CPU 是否在让总线前完成当前事务

一个简单等待状态发生器可以由地址译码、访问类型和计数器组成：快 SRAM 区域不插等待，慢 ROM 区域插入一个等待，外设区域插入两个等待或等待外设 ACK。正式规格应说明等待的最大长度和超时行为。否则一个坏外设可能永久拉住 CPU，使整机看起来像死机。

PC/XT 到 PC/AT 是早期整机平台案例 8088、8086、80286 这些处理器进入实际计算机后，通常不会单独出现。以 PC/XT、PC/AT 风格的平台为例，CPU 周围有启动 ROM、DRAM、DMA 控制器、中断控制器、定时器、键盘控制、串口、并口、显示适配器和扩展总线。它们共同定义了“软件看到的机器”。因此，理解这类平台时，不能只问 CPU 支持哪些指令，还要问端口地址怎样分配、中断线怎样汇总、DMA 通道怎样仲裁、启动 ROM 映射在哪里、扩展卡如何把自己的寄存器和 ROM 挂进系统。

平台对象	硬件职责	预期行为
BIOS ROM	复位后提供第一段固件，初始化基本设备	复位入口命中 ROM，高地址别名或跳转正确
8259A PIC	汇总外设中断并向 CPU 投递	屏蔽位、优先级、EOI 和中断向量正确
8253/8254 PIT	提供周期节拍和计数事件	计数输入、模式字、输出和中断请求正确
8237A DMA	在设备和内存之间搬运块数据	通道、页寄存器、地址计数和终端计数正确

键盘控制器	接收键盘扫描码，也常参与复位/门控控制	输入缓冲、输出缓冲和命令应答顺序正确
UART/并口	提供串行或并行外设连接	状态位、FIFO 或握手线能解释数据何时可收发
扩展总线	允许插卡响应 I/O、内存窗口或中断线	片选互斥、总线方向、中断线和 DMA 请求不冲突

这个案例把本章许多概念放到同一台机器里。BIOS ROM 是启动约定；8259 是中断入口；8254 是定时事件；8237 是 DMA 总线主设备；UART 是状态寄存器与数据寄存器；扩展总线是地址译码、三态和仲裁的板级版本。若一台 PC/AT 风格机器无法启动，应按硬件路径排查：复位后 CPU 是否访问 BIOS 入口，ROM 是否驱动数据总线，时钟和 READY 是否满足，随后才检查键盘、显示、磁盘或操作系统。

端口地址表是平台规格的一部分 早期 x86 平台大量使用独立 I/O 空间。设备寄存器不一定映射在普通内存地址中，而是通过端口号和 I/O 周期访问。平台必须写清楚哪些端口归哪个设备，哪些中断线归哪个设备，哪些 DMA 通道归哪个设备。否则，即使 CPU 指令正确，两个设备也可能响应同一个端口，或一个中断请求被错误解释成另一个设备事件。

规格项	说明内容	典型错误
I/O 端口	控制、状态、数据寄存器的端口号和读写语义	端口译码过宽，多个设备同时响应
中断线	哪个设备接到哪条 IRQ，是否可屏蔽和共享	设备已请求但 PIC 屏蔽，或 EOI 顺序错误
DMA 通道	通道号、方向、地址和长度寄存器	8-bit/16-bit 通道混用或页寄存器错误
内存窗口	显存、Option ROM、缓冲区或 MMIO 区域	RAM、ROM 和设备窗口重叠
启动顺序	固件先初始化哪些控制器，再枚举哪些设备	设备尚未复位稳定就被访问

把端口地址表写成规格，有助于理解“平台兼容性”。处理器可能更换为更快的 80286、80386 或 80486，但许多 I/O 端口、中断和启动约定仍然保留，使旧软件能够找到键盘、定时器、串口和磁盘控制器。硬件的发展不是每一代从零开始，而是在保留平台约定的同时，把部分功能移入更高集成度的芯片组。

二、80386：32-bit、保护模式与地址转换

80386 把 x86 推入 32-bit 时代。它扩大了通用寄存器宽度，引入 EAX、EBX、ECX、EDX、ESP、EBP、ESI、EDI 等 32-bit 形式；地址和数据处理能力显著增强，并提供保护模式、分页支持和更完整的内存保护机制。相较 8086，80386 展示的不只是“位宽变大”，而是 CPU 开始承担更复杂的系统级硬件职责。

80386 的一个关键变化是 32-bit 线性地址。程序形成的有效地址先经过分段机制得到线性地址；在启用分页时，线性地址再经过页表转换为物理地址。可以用简化链路表示：

```
effective address -> segmentation -> linear address
linear address    -> paging          -> physical address
```

这条链路说明，地址不再只是“寄存器加偏移后送到总线”。CPU 内部必须有地址形成、权限检查、页表访问、异常触发等硬件机制。若页表项不存在、权限不允许、访问越界，CPU 不是简单得到一个错误数据，而是产生异常事件，控制流入规定入口。

保护模式把“谁能访问什么”变成硬件可检查的约定。段描述符、特权级、页表权

限等机制，使得不同程序或系统组件不能随意访问全部内存或执行全部指令。这些机制后来成为现代操作系统隔离、虚拟内存和系统调用的硬件基础。对本书来说，重点不是展开操作系统，而是看见硬件层面的扩展：CPU 不只执行 ALU 运算，还参与地址翻译、权限判断和异常入口选择。

80386 观察点	硬件含义
32-bit 通用寄存器	数据通路和编程模型扩展到 32-bit
32-bit 线性地址	地址空间显著扩大，地址形成更复杂
保护模式	CPU 硬件检查权限、界限和特权级
分页机制	线性地址可经页表映射到物理地址
异常机制	地址或权限错误进入硬件规定的控制入口

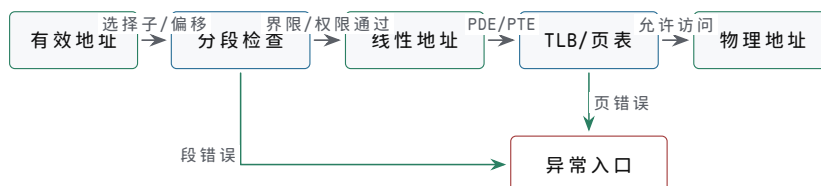
80386 的外部接口同样要按事务理解。32-bit 数据路径并不意味着每次访问都天然完整 32-bit 字。处理器需要用地址低位和字节使能说明本次访问哪些字节有效；外部总线、存储器阵列和 cache 控制器必须据此选择正确字节、处理未对齐访问或拆分周期。若只把 80386 理解成“寄存器变成 32 位”，就会漏掉字节使能、总线周期、异常和地址转换这些整机边界。

80386 对象	硬件含义	常见调试点
地址线与数据线	指出事务目标并传输 32-bit 数据片段	地址对齐、字节选择和目标译码是否一致
字节使能	表明本周期哪些字节有效	字节/半字/字访问不会破坏相邻字节
总线状态	区分读、写、取指、确认等周期语义	外部控制器是否按状态产生正确控制线
分页部件	查页目录和页表，形成物理地址	缺页、权限和存在位错误是否触发异常
TLB 思想	缓存近期地址转换结果	命中降低转换时延，失效和权限改变要刷新

分页可以用一次内存加载来观察。指令先形成有效地址，分段得到线性地址；分页开启时，线性地址被拆成页目录索引、页表索引和页内偏移。若转换结果已在 TLB 中，地址转换可以很快完成；若未命中，就要访问内存中的页表结构。页表项还带有存在位、读写权限、用户/内核权限、访问位和脏位等硬件标志。任何一步失败，这次访问不应返回任意数据，而应进入异常处理路径。

```

80386-style address walk:
effective address
-> segmentation checks
-> linear address
-> TLB lookup or page-table walk
-> permission checks
-> physical address or exception
  
```



80386 的一次内存加载在真正访问 cache 或外部总线前，已经可能因为段、页或权限检查失败而进入异常路径。

图示：80386 地址转换链路。地址转换和权限检查是访存事务的一部分，不是软件注释。

用一个线性地址拆开分页硬件 以线性地址 `0x00403ABC` 为例，80386 的 4 KiB 页式分页可把它拆成页目录索引、页表索引和页内偏移。高 10 bit 选择页目录项，中间 10 bit 选择页表项，低 12 bit 是页内偏移。该地址的页目录索引为 1，页表索引为 3，页内偏移为 `0xABC`。若 CR3 指向的页目录中第 1 项不存在，访问在真正读数据前就会变成异常；若页表项存在但权限不允许，也不能继续把物理地址送到 cache 或外部总线。

字段	示例值	硬件用途
页目录索引	1	选择页目录中的一个 PDE
页表索引	3	选择对应页表中的一个 PTE
页内偏移	<code>0xABC</code>	与物理页框基址相加形成最终地址
P/RW/US 等位	来自 PDE/PTE	判断存在性、读写权限和用户/特权访问

控制寄存器和描述符表是系统约定的入口 80386 的保护模式不能只理解为“地址变大”。处理器必须知道段描述符在哪里、异常入口在哪里、页目录在哪里、当前是否启用保护和分页。这些信息由控制寄存器、GDTR/IDTR 等状态保存。它们不是普通程序变量，而是 CPU 执行内存访问、异常和特权检查时自动使用的硬件约定。

对象	保存内容	硬件使用场景
CR0	保护模式、分页等控制位的一部分	决定地址和权限检查的总体模式
CR3	页目录基址	TLB 未命中或刷新后查页表
GDTR	全局描述符表位置和界限	装载段选择子、检查段属性
IDTR	中断/异常描述符表位置和界限	异常、中断、陷入入口选择
EFLAGS	条件码、方向、中断允许等状态	分支、算术结果和中断接收边界

从实模式进入保护模式是一段硬件约定切换 80386 复位后并不会自动处于完整分页保护环境。系统固件或早期启动代码必须先准备描述符表，再装载 GDTR，设置 CR0 中的保护模式相关位，并通过跳转刷新取指路径和段状态。分页还需要准备页目录、页表，把 CR3 指向页目录，再打开分页控制位。每一步都是硬件状态切换，顺序错误会导致取指、数据访问或异常入口无法解释。

阶段	硬件动作	错误后果
准备 GDT/IDT	在内存中放置段描述符和异常入口描述符	段装载或异常入口失败
装载 GDTR/IDTR	CPU 记录描述符表基址和界限	后续选择子无法被正确解释
设置 CR0	开启保护模式相关控制	旧的取指和段状态需要刷新
远跳转/重装段	让 CS/DS/SS 等进入新描述符语义	继续按旧语义执行会失控
准备页表并设置 CR3	给分页硬件页目录入口	TLB 未命中时无法 walk
开启分页	线性地址开始经页表转换	映射缺失会立即异常

描述符和异常错误码把失败原因带进硬件入口 保护模式中的段描述符不只是“基址和长度”。它还包含类型、存在位、特权级、可读写/可执行等属性。分页项也不只是物理地址，还包含存在、读写、用户/特权、访问和脏等位。异常发生时，CPU

需要把足够信息交给异常入口，使系统能够判断是不存在、权限错误、越界，还是其他硬件条件失败。

对象	关键字段	硬件问题
段描述符	base、limit、type、DPL、present	这个选择子是否存在、能否以当前特权使用
页表项	page frame、present、RW、US、accessed、dirty	这个线性页是否存在、能否按本次方式访问
异常入口	selector、offset、gate type、DPL	失败后 CPU 应进入哪段处理代码
错误码	选择子或页错误原因位	处理程序如何定位失败对象和原因

一次页错误不是“内存读失败”这么简单。CPU 已经形成了线性地址，检查 PDE/PTE 或权限时发现不能继续，于是放弃普通写回路径，保存足够的异常信息，并从 IDT 中选择页错误入口。若处理器允许一部分状态已经提交、一部分状态未提交，系统就难以恢复。因此异常路径必须与指令提交边界配合：错误指令不应像成功指令一样修改目标寄存器或设备状态。

页错误诊断要同时看地址、访问类型和权限位 80386 把页错误从“总线读不到数据”提升为可诊断的硬件事件。CPU 会保存导致故障的线性地址，并通过异常错误码给出若干原因位。学习阶段不必记住所有后续扩展，但必须掌握三个基本问题：页是否存在，访问是读还是写，访问来自用户态还是特权态。只有把这三项与 PDE/PTE 中的 present、RW、US 等位对照，才能判断是缺页、写只读页、用户态访问内核页，还是页表本身没有建立。

观察对象	含义	排查方向
故障线性地址	哪个线性地址无法完成转换或权限检查	页目录索引、页表索引和页内偏移是否落在预期区域
P 位	故障由不存在页还是保护违规引起	缺页处理、页表初始化或权限设置
W/R 位	本次访问是写还是读导致故障	写只读页、写保护代码段、copy-on-write 类机制
U/S 位	本次访问来自用户级还是特权级	用户程序越权访问内核映射或段/页权限不一致
IDT 入口	页错误应跳到哪个处理程序	异常入口描述符、栈切换和处理代码是否有效

这套诊断方法可以反过来指导最小保护模式实验。先建立一段恒等映射，让取指、栈和数据访问都可成功；再故意取消某个页表项的 present 位，确认访问该页会进入页错误入口；随后把某页设置为只读，执行写访问，确认错误码能区分权限失败和不存在页。若异常处理程序本身所在页没有映射，CPU 可能在处理一个错误时再次出错，最终表现为无法恢复的故障。因此保护模式 bring-up 必须先保证 GDT、IDT、栈、异常代码和页表所在内存可访问。

80386 总线的字节使能与 8086 一脉相承 80386 对外有 32-bit 数据总线。地址线通常给出 32-bit 对齐字的高位地址，字节使能信号指出本次传输中哪些 8-bit 字节有效。这样，一个字节写不会破坏同一 32-bit 数据片段中的其他字节；一个未对齐的 32-bit 写可能需要拆成多个传输，或由硬件和总线控制器处理额外周期。

访问类型	有效字节车道	关键问题
字节访问	一个 BE 低有效	相邻字节不能被写改

半字访问	两个相邻 BE 低有效	地址最低位和字节序必须一致
对齐字访问	四个 BE 低有效	一个总线数据片段可完成传输
未对齐字访问	跨越两个对齐片段	需要拆分周期或触发规定异常

从时延角度看, 80386 到 80486/Pentium 的演进可以读成“把常用慢路径搬近”。TLB 避免每次访存都走页表; 片上 cache 避免每次命中数据都走外部总线; 流水线把长执行路径切成多个边界; 分支预测减少控制流改变带来的等待。硬件并没有消灭延迟, 而是用缓存、预测、并行和更细边界, 让常见情况少走远路, 让少见情况通过异常、回填或冲刷流水线处理。

从 8086 到 80386, 可以看到 CPU 和整机边界的变化。8086 主要展示了早期微处理器如何通过外部总线接入存储器和 I/O; 80386 则展示了处理器如何把内存管理和保护机制纳入芯片内部。后来的处理器继续把 cache、TLB、流水线、分支预测、乱序执行、多核互连等机制纳入芯片, 但基本问题没有改变: 地址从哪里来, 权限如何检查, 数据经哪条路径返回, 状态在哪个边界提交。

80486 和 Pentium 可以作为 80386 之后的两个观察窗口。80486 把片上 cache、流水线和浮点方向的集成推到更显著的位置, 使常用指令和数据更容易走近路径; Pentium 进一步展示双流水线、分支预测和更复杂控制对吞吐率的影响。它们并没有改变“寄存器到组合逻辑到寄存器”的基本模型, 而是复制、切分和调度这些路径, 让常见工作更少等待远处存储器和长控制路径。

阶段	新增硬件重点	与前文概念的连接
80386	32-bit 地址、分页、保护和异常	地址路径和权限判断进入 CPU 内部关键结构
80486	片上 cache、流水线、浮点集成	常见数据靠近核心, 长组合路径被边界切分
Pentium	双流水线、预测和更复杂发射控制	多条候选执行路径需要统一调度和恢复
现代核心	乱序、重命名、TLB、多级 cache、多核互连	快慢路径、状态提交和一致性成为核心问题

三、整机启动与经典芯片后的演进

通电或复位后, CPU、主存控制器、互连和设备控制器都必须进入规定初始状态。PC 通常被置到固定启动地址, 那里连接 ROM、片上存储或其他启动介质。启动路径的第一件事, 是让硬件从可预测位置开始取编码。

整机启动不仅是 CPU 复位。时钟源要稳定, 复位树要按顺序释放, 主存控制器要完成训练或初始化, 互连要进入可转发请求的状态, 必要设备要处于安全默认值。若这些硬件条件没有满足, 即使 CPU 本身能取指, 也可能在第一次访问主存或设备时无法继续推进。

从复位向量走到第一条有效指令 启动可以写成一条硬件 trace。上电后, 电源管理电路等待电压进入允许范围; 时钟源或晶振稳定后, 复位电路保持 CPU 和关键外设安全状态; 复位释放时, PC 或取指地址逻辑装入固定复位向量; 地址译码把这个地址指向 ROM、Flash 或片上启动存储; 取指路径读回第一条编码; 控制器译码并开始执行初始化序列。任何一步没有定义, 整机都会表现为“CPU 不启动”, 但根因可能不在 CPU。

启动阶段	硬件动作	预期行为
电源爬升	电压达到允许范围, 去耦提供瞬态电流	欠压复位未提前释放
时钟稳定	晶振、PLL 或外部时钟进入稳定范围	时钟频率和占空比满足规格

复位保持	CPU、控制器、设备进入安全默认状态	危险输出关闭，写使能无效
复位释放	PC 装入复位向量或启动地址	第一拍地址可预测
取启动码	ROM/Flash/片上存储响应地址	数据总线返回有效编码
初始化	配置栈、RAM、时钟、设备和中断	每项配置有完成或错误状态



启动不是“CPU 开始跑”一句话，而是一条逐级成立的链路：电源和时钟先可靠，复位再释放，复位向量必须命中非易失启动代码。

图示：整机启动链路。第一条有效指令依赖电源、时钟、复位、地址映射和启动存储共同成立。

复位入口体现地址空间顶端和底端的不同设计 不同处理器的复位入口不相同，但背后的硬件问题一致：复位后 CPU 必须从非易失存储中的确定位置取得第一条可执行编码。6502 从固定向量中取入口地址，Z80/8051 常从 0 地址开始执行，8086 从 1 MiB 空间顶端附近开始取指，80386 及后续 x86 则把初始取指放到更高物理地址附近，以便系统固件映射在地址空间顶端。

处理器	典型复位入口	硬件含义
6502	从 0xFFFC/0xFFFD 取 16-bit 入口	顶端向量表必须由 ROM 响应
Z80/8051	从 0x0000 开始执行	低地址必须映射启动代码
8086	物理地址 0xFFFF0 附近	1 MiB 顶端 ROM 保存跳转或启动片段
80386 及后续 x86	高物理地址顶端附近	固件映射到高地址，早期代码再跳转到初始化区域
现代 SoC	片上 Boot ROM 或可配置启动介质	先验证/选择介质，再初始化 DRAM 和外设

在 8086 类系统中，复位后从规定的高地址入口开始取指，外部 ROM 必须映射到这个入口；在许多微控制器中，复位向量可能位于片内 Flash 的固定地址，前几个字保存初始栈指针和入口地址；在现代 SoC 中，片上 Boot ROM 可能先验证启动介质、配置内存控制器，再把控制权交给后续固件。形式不同，问题相同：复位向量指向哪里，谁在该地址响应，第一批指令执行前哪些硬件已经可靠。

整机实验可以从 ROM、RAM 和两个 MMIO 寄存器开始 一个可搭建的最小整机不需要复杂操作系统。可先规定 8-bit 或 16-bit 地址空间：低地址接 ROM，中间接 RAM，高地址映射 LED 输出寄存器和按键输入寄存器。CPU 或教学控制器复位后从 ROM 取指，执行一段程序：读取按键状态，若按下则把 LED 寄存器置 1，否则置 0。这个系统已经包含整机硬件的核心对象：取指、数据访存、地址译码、MMIO、副作用、外部输入同步和可观察输出。

```
minimal whole-machine program:
loop:
    r1 = MMIO[GPIO_IN]
    if r1 == 0 goto off
    MMIO[GPIO_OUT] = 1
    goto loop
off:
    MMIO[GPIO_OUT] = 0
    goto loop
```

这段程序的硬件检查不应只看 LED 是否跟随按钮。应同时确认 ROM 地址、RAM 地址和 MMIO 地址不会被同时片选；按钮已经同步后才被读；写 GPIO_OUT 只改变输出锁存器，不误写 RAM；分支改变的是 PC 下一值；循环运行时没有总线争用。若以后把 LED 换成电机、继电器或功率负载，还必须增加驱动级、续流保护和安全停机链路，不能把 MMIO 输出锁存器直接当成功率开关。

最小整机的检查应分成三层 第一层检查硬件静态条件：电源、复位、时钟、片选互斥和三态方向。第二层检查单条指令：ADD、MOV、JZ 等每条指令都能在单步下看到正确的控制线和提交边界。第三层检查小程序：ROM 中循环程序能够读按键、分支、写 LED，并能在异常输入下保持安全状态。

层次	具体检查	失败时优先排查
静态硬件	电源纹波、复位释放、时钟边沿、片选互斥	供电、复位树、译码极性
单指令	PC、IR、控制字、ALUOut、MDR、写回寄存器	控制器、寄存器堆、ALU、存储器时序
小程序	按键输入同步、条件分支、LED 输出锁存	MMIO 地址、分支目标、外设副作用
错误路径	未映射地址、慢设备等待、外设 error 位	超时、错误响应、异常入口

在实验报告中，应记录每个检查点的“预期值、实测值、观察工具和结论”。例如 PC 第 0 拍应为 0x0000，ROM 片选应有效，IR 应锁存第一条 MOV；执行写 GPIO_OUT 的指令时，RAM 片选应无效，LED 锁存器写使能应只出现一个有效窗口。这样的记录比“程序运行成功”更接近工程实践。

可采购器件清单要匹配教学目标 若目标是搭一台低速可观察整机，器件选择应服务于可测性，而不是追求最高频率。可以使用 74HC/74HCT 系列计数器、锁存器、三态缓冲器、加法器、比较器、EEPROM、SRAM 和 LED/按键模块；也可以用 CPLD/FPGA 把控制器和数据通路放进可编程逻辑，再把 ROM/RAM/MMIO 作为独立模块观察。

对象	可选器件	选择理由
PC/计数	74HC161/163 或 FPGA 寄存器	可清零、可装载、可单步观察
IR/锁存	74HC574/273	时钟沿保存指令或中间状态
ALU 原型	74HC283、异或门、比较器或 FPGA 逻辑	可先实现加法、零检测和简单逻辑
总线隔离	74HC245/244	控制数据方向，避免多源驱动
ROM	AT28C 系列 EEPROM 或片内 ROM	保存启动程序和指令编码
RAM	低速异步 SRAM 或 FPGA block RAM	接口直观，便于单步读写
I/O	LED 锁存器、按键同步器、UART 模块	形成可观察输入输出和串口调试

搭建顺序应从电源、时钟、复位、单个寄存器和单条总线开始。只有确认三态控制不会冲突、复位后所有写使能无效、时钟边沿干净，才应接入 ROM、RAM 和 I/O。若直接把所有模块连接后调试，任何错误都可能表现为“程序不跑”，排查成本会急剧上升。

整机 bring-up 要记录症状、信号和假设 真实硬件第一次上电时，常见问题不是“CPU 坏了”，而是复位、时钟、片选、总线方向、等待状态或启动 ROM 映射有误。把症状、相关信号和当前假设逐项记录下来，才能形成一条可逐步核对的排查链条，而不是笼统地说“检查硬件”。

症状	优先观察信号	可能原因
PC 不变化	复位、时钟、PC 写使能	复位未释放、时钟不稳定、控制器未进入取指
ROM 无输出	地址线、ROM 片选、输出使能	复位向量未映射到 ROM，译码极性错误
数据总线异常发热	CPU/存储器输出使能、方向控制	两个器件同时驱动总线
加载读错值	地址、字节使能、读响应、MDR	地址形成错误或存储器时序不满足
LED 乱闪	MMIO 片选、写使能、外设锁存时钟	地址译码过宽或写周期毛刺
中断重复进入	设备状态、中断控制器 ACK、CPU 入口	清除顺序错误或事件源未确认

经典微处理器的发展可以用若干硬件剖面概括。下表不是处理器史年表，而是说明每一代芯片把哪些硬件问题推到读者面前。

芯片或系列	典型特征	对硬件学习的意义
4004	4-bit 微处理器	微处理器把控制器、寄存器和算术部件集成到单芯片形态
6502/Z80	经典 8-bit 微处理器	外部总线、存储器、I/O 和多周期控制构成早期微机基本形态
8051	经典单片机	CPU、片内存储、I/O、定时器和中断控制展示片上整机控制
8080/8085	8-bit 微处理器	外部总线、地址空间、存储器和 I/O 开始形成通用微机结构
8086/8088	16-bit x86 起点	段：偏移地址、指令预取、复用总线和外部锁存器体现早期整机取舍
80286	保护模式前驱	分段保护和特权机制开始成为处理器硬件的一部分
80386	32-bit x86	32-bit 寄存器、线性地址、分页和异常机制把 CPU 推向系统级硬件
80486/Pentium	片上 cache 与更深执行结构	流水线、cache、浮点单元和更复杂控制逻辑成为性能核心

8051 的意义在于把“小整机”收进一颗芯片。它让读者看到，CPU 旁边可以直接布置片内 RAM、ROM、I/O 口、定时器、串口和中断控制。外部引脚不再只是地址/数据总线，也可能直接是可编程 I/O。对硬件学习而言，这说明整机边界可以移动：同样是“写设备寄存器改变输出”，在早期微机上可能通过板级地址译码访问外设，在单片机上则可能是写特殊功能寄存器改变某个引脚锁存器。

80486 的意义在于把片上 cache 和更清楚的流水化执行推到学习视野中。片上 cache 缩短了高频取指和数据访问路径，使常见访问不必每次走外部总线；流水线把取指、译码、执行等工作分段，使不同指令可以重叠推进。它同时提醒读者，性能不是单靠提高门速度，把常见数据放近和把长路径切段同样关键。

Pentium 的意义在于展示更强的并行执行方向。双流水线、分支预测和更复杂的发射控制，使处理器尝试在同一时间推进多条合适的指令。此时硬件问题不再只是“一个 ALU 算得多快”，还包括指令之间是否有依赖、分支方向是否预测正确、两条流水线是否争用资源、错误路径如何清除、异常如何保持精确边界。这些机制为现代乱序执行、寄存器重命名和提交队列铺路。

80486 把 cache、FPU 和流水线收进芯片 80386 时代，cache 是主板上的独立

芯片加标签存储器，浮点运算是另一颗 80387 协处理器，两者都通过板级总线与 CPU 通信。80486 把三样东西收进片内：8 KiB 统一 cache（指令和数据共用，采用写穿策略）、浮点单元和一条 5 级流水线。配套的还有总线接口的改变：486 引入突发总线周期，一次 cache 行填充用 4 拍连续传完 16 B，只有第一拍需要给出地址，后面三拍不再重复。收益很直接：cache 命中时取指和读数不再走板级总线，常见指令的吞吐接近每拍一条；代价同样明确：片内 cache 要处理一致性，写穿策略持续占用外部总线带宽，流水线的段间控制、数据相关和分支处理也比多周期实现复杂得多。

Pentium 用双发射和分支预测重组执行结构 Pentium 的性能提升不再主要来自时钟频率，而是来自执行结构的重组。它有 U、V 两条整数流水线，满足配对规则的简单指令可以在同一拍内双发射；分支目标缓冲（BTB）预测分支方向，猜错才冲刷流水线；指令 cache 和数据 cache 分开，各 8 KiB，取指和读数不再争同一个端口；外部数据总线扩到 64 bit，一次总线事务搬运的数据翻倍。Pentium Pro 又往前走了一步：x86 指令先被译成微操作，由乱序执行核心按数据就绪顺序执行，寄存器重命名消除假依赖，最后按程序顺序提交，从而在乱序执行下保持精确异常。执行可以和提交分离，是这一代留下的主线：第四章讲的执行/提交区分，在这里成为真实芯片的基本组织方式。

386/486 主板把 CPU、内存和外设分成平台层次 80386、80486 之后，整机越来越依赖主板芯片组。CPU 不再直接以简单板级总线面对所有外设，而是通过内存控制器、cache 控制器、总线桥和 I/O 控制器访问不同速度、不同协议的目标。早期 PC 主板可以粗略理解为：CPU 与高速本地总线连接，芯片组负责 DRAM、cache、ISA/EISA/VLB/PCI 等扩展路径，低速外设再挂到更慢的 I/O 控制器。后来的“北桥/南桥”说法，就是把高速内存/图形路径和低速 I/O 路径分开组织。

平台层次	负责对象	与本章概念的关系
CPU 本地总线	取指、数据访问、cache miss、写回	地址、数据、字节使能、等待和总线错误
内存/cache 控制	DRAM 行列访问、二级 cache、写缓冲	命中/未命中、回填、写回和时序预算
高速扩展路径	图形、磁盘控制、网络或总线桥	DMA、总线主设备、仲裁和中断
低速 I/O 控制	串口、并口、键盘、软驱、RTC 等	端口 I/O、状态寄存器、完成事件
固件层	BIOS/Option ROM、设备初始化和启动选择	复位向量、枚举、内存检测和启动介质

这个分层解释了为什么整机性能不能只看 CPU 主频。若 cache 未命中要走慢 DRAM，若 PCI 设备 DMA 被总线仲裁阻塞，若中断控制器或桥片配置错误，CPU 再快也会等待。另一方面，把 cache、内存控制器、图形控制、PCIe 根复合体等功能逐步移近 CPU，也正是现代平台降低时延的方向：常用路径减少芯片间往返，慢速外设通过桥接和队列异步处理。

从 PCI 到 PCIe，设备仍然是事务发起者和响应者 PCI/PCIe 这类现代扩展互连比早期 ISA 总线复杂得多，但本章的分析方法仍然适用。设备可以暴露配置空间、MMIO BAR、DMA 能力和中断机制；CPU 或固件先枚举设备，分配地址窗口和中断资源；驱动程序再通过 MMIO 配置设备，通过 DMA 描述符让设备读写内存，并通过 MSI/MSI-X 或传统中断接收完成事件。

PCI/PCIe 对象	硬件含义	关键问题
配置空间	设备身份、能力、BAR 和控制位	枚举是否能读到设备，BAR 是否分配到不冲突地址

BAR/MMIO	设备寄存器窗口映射到物理地址	访问属性是否不可缓存或按设备规则排序
DMA	设备作为总线发起者读写主存	地址权限、IOMMU、cache 一致性和 completion 顺序
MSI/MSI-X	设备通过内存写形式投递中断消息	消息地址/数据配置是否正确，是否会丢事件
错误报告	链路、权限、超时或设备内部错误	错误能否被记录、上报和恢复

这一段对学习现代硬件很重要：PCIe 不再是多块板卡共享一排并行地址线的简单总线，但它仍然要解决同样的问题。请求从哪里来，目标地址是谁，数据何时有效，完成如何返回，错误如何上报，中断如何投递，DMA 写入何时对 CPU 可见。只要把这些问题标出来，复杂协议就不会变成无法理解的名词堆。

SoC 把北桥、南桥和许多外设继续收进芯片 现代 SoC 的趋势是把 CPU 核、cache、内存控制器、GPU/NPU、视频编解码、PCIe/USB/SATA、网卡、显示控制、GPIO、定时器和安全启动逻辑放进一颗或少数几颗芯片。这样做的直接收益是缩短常见路径、降低引脚数量、减少板级功耗和提高集成度；代价是片上互连、一致性、时钟/电源域、调试和安全启动变得更复杂。

SoC 结构	低时延来源	新增边界
片上 L2/L3 cache	多个核心共享近处数据	一致性状态、替换、QoS 和带宽竞争
集成内存控制器	CPU 到 DRAM 控制路径更短	DRAM 训练、刷新、ECC 和功耗状态
片上互连	核、DMA、GPU、外设通过网络通信	仲裁、优先级、背压和死锁避免
集成外设	GPIO、UART、SPI、I2C、USB 等近距离访问	MMIO 属性、中断路由和复位/时钟门控
安全启动	Boot ROM 验证后续固件	密钥、签名、回滚保护和调试锁定

因此，“现代芯片为什么时延低”可以更完整地回答：不是所有路径都低时延，而是常见路径被集成到片上、被 cache/TLB 缓存、被预取和预测提前准备、被流水线和乱序执行并行化、被 DMA 和队列卸载；不常见路径则通过 cache miss、TLB miss、DRAM 排队、设备背压、错误报告和中断恢复来处理。低时延来自体系结构、微架构、互连、存储层级和平台集成共同作用。

从 8086、80386 向后看，处理器继续沿几条方向扩展。第一，内部执行路径更深，流水线把取指、译码、执行、访存、写回切成多个阶段。第二，存储层级更复杂，片上 cache 和 TLB 成为地址访问性能的关键。第三，控制逻辑更复杂，分支预测、乱序执行和异常恢复要求硬件保存更多内部状态。第四，多核和片上互连让“总线事务”扩展成核间一致性和缓存一致性协议。第五，外设从简单寄存器发展到 PCI、PCIe、USB、SATA 等分层协议，但本质仍然是地址、数据、控制、握手、完成和错误状态。

演进方向	保持不变的硬件问题
流水线	状态边界如何切分，错误路径如何清除
cache/TLB	地址如何命中、缺失、替换和回填
保护与异常	何时拒绝访问，如何进入规定处理入口
DMA 与设备	谁发起事务，谁完成事务，完成如何通知 CPU
多核一致性	多个观察者如何看到受控顺序的内存结果

到这里，本书的硬件链路闭合：电和器件形成可控开关，开关形成门，门形成组

合逻辑，反馈和时钟形成状态，寄存器和 ALU 形成数据通路，控制器组织动作，CPU 执行编码，主存、cache、互连、设备和启动路径把它扩展成整机。8086 和 80386 的意义在于提供了两个清晰历史剖面：一个展示早期微处理器如何连接总线和地址空间，一个展示处理器如何承担更复杂的地址转换、保护和异常机制。

四、从目标文件、固件到可启动机器

前面几章已经解释了 CPU 如何执行编码，主存和设备如何响应地址，经典芯片如何定义平台边界。还差一条常被硬件教材略过、却对系统理解非常关键的链路：程序怎样从目标文件进入内存，符号地址怎样变成机器地址，启动固件怎样把硬件带到可执行状态，异常表和页表怎样让失败路径可恢复。参考经典系统教材的读法，这一节不展开编译器细节，而是把链接、装载和启动都还原成硬件地址约定。

目标文件里的符号最终要变成地址 汇编器或编译器产生的目标文件通常还不知道所有最终地址。函数名、全局变量名、外部符号和跳转目标先以符号或重定位项表示。链接器把多个目标文件和库合并，决定代码段、只读数据段、数据段、BSS 段等放到什么地址，并把需要修补的位置改成最终地址或相对偏移。

对象	系统含义	硬件落点
代码段	机器指令编码	取指路径从这些地址读取 opcode
只读数据	常量、跳转表、字符串	普通加载读取，权限通常禁止写
数据段	已初始化全局变量	启动时从镜像复制到 RAM
BSS	零初始化全局变量	启动时清零，镜像中不必保存全量零字节
重定位项	需要链接器或装载器修补的地址位置	改写指令立即数字段、地址表或指针字

这说明“函数地址”不是抽象标签。取指硬件需要 PC 中的数值地址；CALL 需要目标地址或相对偏移；全局变量加载需要数据地址；中断向量需要入口地址。链接器和装载器的工作，就是把这些名字和段布局变成 CPU 能取指、加载、存储和跳转的地址。

```
source idea:
    call uart_putc
    load global_counter

after link/load:
    CALL 0x00102480
    MOV R1, [0x00203010]
```

装载器建立的是第一张可执行地址地图 在有操作系统的机器上，装载器把可执行文件映射进进程地址空间，建立代码、数据、堆、栈和共享库区域；在裸机或固件场景中，启动代码负责把镜像从 ROM/Flash 复制或映射到合适位置，清 BSS，设置栈指针，初始化全局数据，然后跳到入口函数。两者规模不同，但硬件问题一致：哪些地址可取指，哪些地址可读写，栈在哪里，异常入口在哪里，设备寄存器在哪里。

启动/装载动作	硬件意义	失败表现
设置栈指针	CALL/RET、局部变量和异常入口有可用内存	第一次函数调用或中断即破坏内存
复制数据段	RAM 中全局变量得到初始值	程序读到未初始化数据

清 BSS	零初始化对象符合语言和固件约定	状态机、计数器、指针初 值随机
建立向量表	中断和异常有入口地址	外部事件或错误无法处 理
初始化时钟/总 线	后续 RAM、设备、cache 路径满足 时序	访问慢器件超时或读写 失败
跳入口	PC 改到主程序第一条指令	程序真正进入可维护代 码路径

裸机启动常见伪代码如下。它表面像软件初始化，实质上每一步都在建立后续硬件事务的前提。

```
reset_handler:
  set SP to top_of_stack
  copy .data from flash image to SRAM
  zero .bss in SRAM
  initialize clocks and memory controller
  install vector table base
  initialize essential MMIO devices
  jump main
```

异常向量表是硬件失败路径的地址表 只要系统会处理中断、页错误、非法指令或设备事件，就需要某种入口表。早期处理器可能用固定地址或固定向量；保护模式处理器使用更复杂的描述符表；微控制器常在启动区域放置向量表。表的形式不同，但作用相同：当硬件事件发生时，CPU 能把事件编号转成处理程序入口，并保存足够状态以便恢复或诊断。

向量项	保存内容	硬件使用场景
复位向量	第一条启动代码地址或跳转指令	上电或复位后装入 PC
非法指令向量	无法译码或不允许执行时的入口	opcode 不合法或特权不允许
页错误/总线错误向量	地址转换或外部事务失败入口	加载/存储/取指不能完成
外设中断向量	设备完成、错误或数据到达入口	中断控制器投递事件
系统调用入口	用户代码请求特权服务入口	权限切换和受控进入内核或监控程序

向量表本身也必须位于可访问地址。若页表或地址译码没有覆盖异常入口，CPU 在处理第一个错误时会遇到第二个错误；若栈没有准备好，保存 PC/Flags 或错误码时会破坏未知内存；若中断处理程序所在代码被错误标成不可执行，事件到达后系统会在入口处再次失败。启动顺序必须先保证最小异常路径可靠，再打开复杂设备和中断。

页表把装载结果提升为可保护的运行时地图 链接器决定镜像中的相对布局，装载器把段放进内存，页表则把运行时地址空间变成可检查的约定。代码页可以标为只读和可执行，数据页可读写但不可执行，内核页禁止用户访问，设备 MMIO 页设置为不可缓存或强顺序。这样，同一条加载/存储/取指路径在真正访问 cache 或总线前，会先经过地址转换和权限检查。

区域	典型页属性	硬件效果
代码	只读、可执行	对代码页的写操作触发保护异常

只读数据	只读、不可执行或按平台规定	防止常量被误写或执行数据
普通数据/堆/栈	可读写、通常不可执行	数据写入允许，取指可被阻止
内核映射	特权访问，用户禁止	用户态越权访问触发异常
MMIO 窗口	不可缓存、设备顺序属性	读写按设备事务执行，不被普通 cache 吞掉

这张表把前几章内容合并起来：第三章的 bit 和地址解释，第四章的 PC 与加载/存储，第五章的 cache/MMIO 属性，第六章的 80386 分页和异常，在页表属性中汇合。现代系统的许多安全和可靠性机制，本质上就是让硬件在每次取指和访存前检查这些约定。

从固件到操作系统是平台所有权转移 固件启动阶段拥有硬件控制权：它配置时钟、内存控制器、基本 I/O、启动介质和必要安全检查。操作系统接管后，会重新建立页表、中断控制、设备驱动和调度机制。这个交接不是一句“加载内核”，而是一段平台所有权转移：谁拥有中断表，谁拥有页表，谁管理内存，谁接管设备，谁定义错误处理策略。

阶段	固件职责	交给后续系统的约定
早期启动 内存初始化	电源、时钟、复位、最小取指路径 DRAM 控制器、cache/TLB 初始策略	CPU 能从可信入口执行 可用内存范围和保留区域
设备发现	基本总线枚举、启动介质选择	设备地址窗口、中断和 DMA 能力
镜像验证/加载	读取、校验、解压或复制内核/程序	入口地址、参数和内存布局
控制权转移	设置寄存器、栈、页表或模式后 跳转	后续系统接管异常、中断 和设备

在微控制器上，固件和“应用”可能就是同一个镜像；在 PC 或服务器上，固件、引导加载器、内核和用户程序分层明显；在 SoC 上，还可能有 Boot ROM、一级引导、可信固件、安全监控和普通操作系统。层数不同，但每一层都要把可执行地址、栈、异常入口、内存属性和设备所有权交代清楚。

安全启动把第一条指令也纳入信任链 现代 SoC 常把 Boot ROM 固化在芯片内部，复位后先执行不可修改的 ROM 代码。Boot ROM 读取下一阶段固件，验证签名、版本和配置，再决定是否跳转。这样做不是密码学装饰，而是硬件启动约定的一部分：如果第一段可变固件不可信，后续页表、设备配置、密钥和操作系统都可能被篡改。

安全启动对象	硬件/固件动作	失败处理
Boot ROM	从固定复位入口执行，不可现场 改写	提供根信任和最小验证代 码
公钥/哈希 镜像签名	存在熔丝、ROM 或安全存储中 覆盖代码、配置和版本信息	用于验证下一阶段镜像 验证失败则停止、降级或 进入恢复模式
回滚保护	记录允许的最小版本	防止加载旧漏洞固件
调试锁定	限制 JTAG、串口下载或内存读出	防止启动后被外部接口改 写控制流

这也解释了为什么“硬件从零到整机”不能只讲电路能跑。整机一旦能取指、访存和访问设备，就必须回答谁能改代码、谁能访问内存、谁能发起 DMA、谁能接收中断、启动镜像是否可信、错误路径是否可恢复。经典芯片提供了这些问题的历史剖

面，现代平台则把它们推向更复杂的安全和系统集成边界。

案例：一个裸机镜像从链接地址跑到 main 假设一块教学板把 Flash 映射到 0x00000000，SRAM 映射到 0x20000000，GPIO 映射到 0x40000000。链接脚本规定向量表和代码放在 Flash，已初始化数据运行在 SRAM，但初始内容存放在 Flash 镜像中，BSS 在 SRAM 中清零，栈顶放在 SRAM 高地址。这个布局看起来是软件文件，实际上直接决定复位后每一次取指、加载和存储的地址。

```
memory map:
  0x00000000-0x0003FFFF Flash: vectors, code, const, data image
  0x20000000-0x20007FFF SRAM: data, bss, stack
  0x40000000-0x40000FFF GPIO MMIO
```

镜像区域	链接/装载含义	硬件事务
向量表	复位入口和异常入口地址	复位时取入口，异常时取向量
.text	指令编码放在 Flash	PC 取指访问 Flash 或映射后的代码区
.rodata .data 镜像	常量和只读表 初始值存放在 Flash	加载读取，通常禁止写 启动代码复制到 SRAM 运行地址
.bss	运行时应为 0 的对象	启动代码对 SRAM 执行写操作清零
栈	函数调用和异常保存上下文	SP 指向 SRAM 可写区域

复位后第一段代码不应依赖尚未初始化的全局变量，也不应调用需要完整栈和运行时的复杂函数。它要先建立最小可运行边界。

```
reset trace:
  PC = vector_table.reset_handler
  SP = _stack_top
  copy bytes from _sidata in Flash to _sdata in SRAM
  zero bytes from _sbss to _ebss
  configure clocks and essential wait states
  set vector table base if architecture supports it
  call main
```

启动动作	不能省略的原因	常见错误
设置 SP	后续 CALL、PUSH、异常保存需要栈	SP 指到 Flash 或未映射区
复制 .data	全局初值要出现在 RAM 运行地址	变量初值全错或仍为 Flash 内容
清 .bss	语言和固件约定零初始化	状态机初值随机，指针非零
配置等待状态	Flash、SRAM 或外设满足时序	高频下取指或访问偶发失败
设置向量表 跳 main	中断和异常能进入正确入口 进入应用主逻辑	第一场中断跳到错误地址 入口地址或模式错误导致程序失控

案例：保护模式最小启动清单 以 80386 风格机器为例，若要从实模式进入带分页的保护环境，必须先建立一组硬件可读的表和寄存器状态。这个过程常被写成启动代码技巧，但本质是把 CPU 的地址解释约定从旧模式切换到新模式。

步骤	硬件目的	预期行为
建立 GDT	给代码段、数据段、栈段描述符	描述符 base/limit/type/present 正确
建立 IDT	给异常和中断入口描述符	页错误、非法指令等入口 可达
加载 GDTR/IDTR	CPU 知道描述符表位置	表所在内存可读，界限正 确
打开保护模式位	改变段选择子解释方式	远跳转后 CS 使用新描述 符
重装段寄存器	DS/SS/ES 等进入新约定	数据和栈访问按新描述符 解释
建立页目录/页表	线性地址可转换到物理地址	取指、栈、异常代码均被 映射
加载 CR3 并开启分页	CPU 开始查页表和 TLB	缺失映射会触发页错误而 非随机访问

最小保护模式 bring-up 应先保守映射。把当前代码、栈、页表、GDT、IDT 和 VGA/串口调试 MMIO 映射好，再打开分页；确认能打印或点亮调试输出后，再逐步收紧权限。若一开始就建立复杂高半区映射、多个页大小、用户/内核分离和设备属性，任何一个地址错误都可能导致连续异常，难以定位。

```
protected-mode checklist:
GDT covers code/data/stack selectors
IDT covers at least fault handlers
stack is writable after mode switch
current instruction stream is mapped
page tables themselves are mapped
debug output MMIO is mapped uncached
fault handlers do not fault recursively
```

案例：从异常号定位启动失败 启动阶段最可怕的现象是“黑屏”或“无输出”。更有效的做法是把失败分层。若能进入非法指令异常，说明取指至少到了某条不合法编码；若能进入页错误，说明保护模式和 IDT 至少部分工作；若连续进入双重错误或复位，说明异常处理路径本身可能失败；若完全没有总线活动，可能是复位、时钟或复位向量映射问题。

现象	可能边界	优先观察
无取指总线活动	复位、时钟、复位向量、ROM 片选	示波器看时钟/复位/地址线
取指后立即异常	编码、模式、段描述符或权限错误	IR/PC、异常号、错误码
页错误循环	页表缺映射、权限错误、异常入口未映射	故障地址、PDE/PTE、CR3
中断后死机	栈、IDT、EOI 或清中断顺序错误	SP、向量号、中断控制器状态
DMA 后数据错	cache 可见性、IOMMU、缓冲区所有权	描述符、completion、cache 维护记录

这张表把整本书的排查方法统一起来：不要先猜“CPU 坏了”，先问硬件行为停在哪个边界。电源和时钟是第一章的边界，触发器和状态是第二章的边界，ALU 和控制线是第三章的边界，PC/IR/寄存器是第四章的边界，主存/总线/MMIO/DMA 是第五

章的边界，描述符、页表、固件和平台是第六章的边界。

案例：把教学板扩展成一台小系统 最后给一个可以落地的整机扩展路线。第一阶段只跑 ROM 中的 LED 循环；第二阶段加入 SRAM 和栈，能调用函数；第三阶段加入定时器中断，能周期性切换状态；第四阶段加入简化 DMA，把一段内存复制到另一段；第五阶段加入保护或至少地址错误检测，让未映射访问进入错误入口。每一阶段都对应书中一个硬件层次。

阶段	新增硬件/固件	预期行为
ROM LED	复位向量、ROM、GPIO MMIO	上电后固定节奏改写 LED 寄存器
SRAM 栈	SRAM 译码、SP 初始化、CALL/RET	函数调用和局部变量不破坏全局数据
定时器中断	定时器、向量表、中断清除	周期事件不丢失，主循环可继续运行
简化 DMA	描述符、源/目标地址、completion	DMA 完成后 CPU 读到正确内存内容
错误入口	未映射检测、总线超时或简化异常	错误地址进入可观察处理程序

这条路线比“一步到位跑操作系统”更适合硬件书。读者每完成一层，都能指出新增了哪些地址、哪些控制线、哪些状态边界、哪些错误路径。等这些边界稳定后，再看 8086、80386 或现代 SoC，就不是背芯片史，而是在识别同一组问题如何被真实平台放大和工程化。

综合项目：写一份最小整机规格 本章最后可以把全书内容收束成一份最小整机规格。规格不需要一开始很复杂，但必须让别人能按它搭出同一台机器，并能判断每个访问是成功、等待还是错误。下面给出一个建议模板，读者可以用分立逻辑、FPGA、微控制器外设模拟或软件仿真来实现。

规格项目	必须写清楚	预期行为
地址空间	ROM、RAM、MMIO、未映射区范围	任意地址最多一个目标响应
复位行为	PC 初值、SP 初值、控制线安全值	上电第一条取指可重复观察
指令集	编码、寄存器字段、立即数扩展和标志规则	每条指令能写成控制 trace
内存访问	字节序、对齐、字节使能、等待和错误	加载/存储不破坏相邻字节
设备寄存器	每个位的读写、副作用和清除规则	GPIO/定时器/UART 状态可验证
中断/异常	向量入口、保存状态、清除和返回规则	外部事件和错误能进入可诊断路径
DMA/队列	描述符格式、所有权、completion 和 cache 规则	CPU 与设备不同时拥有缓冲区

规格写完后，建议按下面的顺序实现，而不是同时接上所有模块。

里程碑	要实现的最小能力	预期行为
M1	复位后从 ROM 取指，PC 顺序前进	逻辑分析仪能看到连续取指地址
M2	ADD/SUB/JNZ 正确执行计数循环	寄存器值和分支方向符合 trace

M3	SRAM 加载/存储和栈可用	函数调用能返回，局部变量不乱
M4	GPIO MMIO 可读写	LED/按键通过地址事务控制
M5	定时器中断可进入和返回	中断不破坏主循环寄存器和栈
M6	简化 DMA 或块复制控制器可搬数据	completion 后内存内容正确
M7	未映射访问进入错误入口	错误地址和原因可观察

每个里程碑都应保留一个最小测试程序。不要只保留最终大程序，因为最终程序一旦失败，无法判断是取指、译码、ALU、存储器、MMIO、中断还是 DMA 出错。小测试程序就是硬件书里的单元测试。

```
M2 counter loop:
    MOV R1, 0
    MOV R2, 10
loop:
    ADD R1, 1
    SUB R2, 1
    JNZ R2, loop
    MOV [GPIO_OUT], R1
```

测试失败现象	优先怀疑	观察信号
PC 不变	复位、时钟、PC 写使能或取指等待	PC、reset、clock、ready
循环次数错	立即数扩展、SUB、Zero 标志或 JNZ	R2、Zero、pc_sel、branch_target
GPIO 不变	地址译码、MMIO 写使能、字节使能	address、io_cs、mem_write、write_data
函数返回错	SP、CALL/RET 返回地址、栈 RAM	SP、栈内容、PC、writeback
中断后错乱	保存/恢复寄存器、EOI、清状态顺序	vector、SP、saved PC、irq status
DMA 数据错	描述符、地址权限、cache 可见性、completion	desc、buffer、owner、done

这个综合项目的目的，是让读者把书中的概念变成可执行机器：第一章的电平和输出驱动，第二章的状态边界，第三章的 ALU 和控制字，第四章的 CPU trace，第五章的地址事务和 DMA，第六章的启动、异常和平台约定。能写出并验证这份规格，才算真正把“硬件从零到整机”连起来。

综合项目的实验报告模板 为了避免实验结果只停留在“好像能跑”，每个里程碑都应留下实验报告。报告不需要长篇叙事，但必须保存足够的观察记录，让另一个人能复现同样结论。建议每个实验至少记录以下字段：

报告字段	要记录的内容	为什么重要
硬件版本	原理图版本、器件型号、时钟频率、电源电压	不同器件和频率会改变时序结论
固件版本	ROM 镜像、链接地址、入口地址、编译选项	同一硬件跑不同镜像可能表现不同
地址地图	ROM/RAM/MMIO/未映射区域范围	后续访问错误可直接对照

测试程序	最小汇编或机器码列表	能定位具体指令和控制 trace
观察信号	PC、IR、控制字、地址、数据、ready、写使能	证明结果来自正确硬件路径
预期结果	预期寄存器、内存、设备和异常状态	避免把偶然现象当成通过
失败记录	失败现象、假设、修改和复测结果	保留完整调试记录

例如 M3“SRAM 栈可用”的实验报告应包含：复位后 SP 的值，CALL 前后的 PC，栈上保存的返回地址，函数内部局部变量地址，RET 后 PC 是否回到调用点，R0 或指定返回值寄存器是否正确。若只写“函数调用成功”，后续出现异常时无法判断是栈、返回地址、寄存器保存还是取指路径出了问题。

```
M3 trace example:
reset PC = 0x00000000
initial SP = 0x20008000
CALL writes return address 0x00000124 to [SP-4]
callee uses [SP-8] for local variable
RET loads PC = 0x00000124
R0 = expected return value
```

这类报告会把学习从“看懂章节”推进到“能维护机器”。硬件工程的核心不是一次点亮 LED，而是每次改动后仍能解释：哪个状态边界改变了，哪个事务完成了，哪个错误路径被覆盖了，哪些观察结果足以支持结论。

专题：从 8086 到 SoC 的“同一问题不同边界” 8086、80386、Pentium、微控制器和现代 SoC 看起来差别很大，但平台工程反复面对同一组问题：第一条指令从哪里来，地址如何到达目标，慢设备如何等待，错误如何上报，外设如何通知 CPU，DMA 与 cache 如何保持可见，调试器如何观察状态。演进改变的是边界位置，而不是问题本身。

平台问题	早期芯片形态	现代 SoC 形态
复位入口	外部 ROM 响应固定复位向量	片内 Boot ROM、熔丝、启动介质选择和安全检查
地址译码	板级译码器产生片选	片上互连、地址防火墙、IOMMU 和外设桥
等待状态	READY/WAIT 拉长总线周期	valid/ready、AXI response、QoS 和超时
中断	外部中断控制器或简单向量	GIC/APIC、优先级、亲和性、虚拟化中断
DMA	简单外设总线主控	多主设备、cache 一致性、SMMU/IOMMU 权限
调试	逻辑分析仪看总线引脚	JTAG/SWD、trace macrocell、性能计数器和固件日志

这张对照能帮助读者读芯片手册。看到 AXI、APB、GIC、IOMMU、Boot ROM、secure monitor 时，不应把它们当成孤立名词，而应问：它们分别把早期平台的哪一条裸露信号或板级功能搬进了芯片内部？搬进去之后，哪些行为变得更快、更安全，哪些调试信息变得更难直接观察？

专题：BIOS、Boot ROM 和 loader 的最小移交清单 启动链路不是“跳到下一段代码”这么简单。每一级启动代码都必须向下一级交出可用的执行环境：当前特权级、栈、内存可用范围、异常入口、时钟、串口或日志设备、镜像位置和校验结果。缺少任何一项，下一阶段可能运行一段时间后才以难以定位的方式失败。

移交项目	必须说明的内容	失败现象
入口地址	下一阶段第一条指令位置和执行状态	跳入错误模式或错误对齐地址
栈	栈顶、增长方向、可用范围	CALL/异常保存破坏数据或 ROM 区
内存地图	RAM、ROM、MMIO、保留区、安全区	loader 覆盖设备寄存器或固件数据
异常向量	向量基址、异常栈、默认处理器	早期错误无法诊断，直接死机
时钟/电源	CPU、总线、外设时钟是否打开	访问外设永久等待或超时
日志通道	UART、半主机、内存日志或调试口	失败时没有可观察的输出
镜像验证	长度、哈希、签名、加载地址	执行损坏或不可信代码

安全启动只是把这张移交清单加上信任约束：Boot ROM 信任芯片内固化公钥或熔丝状态，验证下一阶段签名，再把控制权交给经过验证的镜像。若验证失败，平台不能继续执行任意代码，而应进入恢复、锁定或错误报告路径。这样读者能把“安全启动”理解为启动链路中的状态机和逐级验证过程，而不是抽象口号。

专题：平台 bring-up 日志怎样定位到具体层 bring-up 的第一原则是每一步只引入一个新假设，并记录足够的观察结果。若一开始就运行完整系统，失败时无法知道是时钟、复位、内存、cache、异常、串口、设备树、驱动还是文件系统出错。更稳妥的办法是从复位向量开始逐层扩大可用机器。

阶段	建议日志	证明的边界
复位	PC、模式、第一条指令字	ROM 映射和取指可用
串口	时钟源、波特率、发送寄存器状态	MMIO、外设时钟和基本输出可用
SRAM	写读 walking bit、地址线测试	片内 RAM 和数据访问可用
DRAM	初始化参数、训练结果、块读写校验	外部内存控制器和板级连线可用
异常	故障地址、异常号、返回 PC	向量表和异常栈可用
cache/MMU	页表摘要、属性、TLB flush 记录	地址转换和一致性边界可控
中断	控制器配置、pending/active 状态	异步事件链路可用

每条日志都应能回答“这条记录来自哪里”。例如串口能打印字符，不只证明 C 函数被调用，还证明 CPU 能取指、写 MMIO、外设时钟打开、引脚复用正确、波特率近似正确。若后续打开 cache 后打印乱码，就可以把变化集中到 cache 属性、写缓冲、MMIO ordering 或时钟切换，而不是重新怀疑所有层。

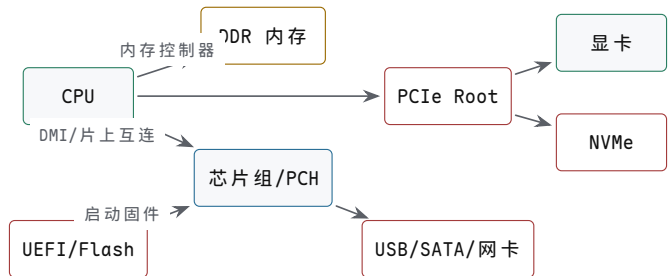
专题：把经典芯片放进同一张学习路线 本章不要求记住每颗芯片的全部引脚，而要求形成可迁移的读法。读 6502 时看复位向量、零页和简单总线；读 8086 时看段：偏移、复用总线和字节存储体；读 80386 时看保护模式、分页和异常；读 Pentium 时看流水线、cache 和总线协议；读 SoC 时看片上互连、启动 ROM、DMA、一致性和安全边界。

阅读对象	优先抓住的问题	可迁移能力
6502/Z80/8051	复位、寄存器、地址空间、简单外设	从最少状态读懂机器执行

8086	分段、外部总线、READY、板级译码	把 CPU 周期映射到平台信号
80386	特权级、描述符、分页、异常	理解操作系统为什么需要硬件保护
80486/Pentium	cache、流水线、总线事务	解释性能和一致性不再只是 ISA 问题
现代 SoC	Boot ROM、片上互连、DMA、安全、调试	把整机看成多主设备协作系统

这样安排后，经典芯片不是怀旧材料，而是复杂系统的分层样本。每前进一步都在回答同一件事：更多性能、更多外设、更多安全约束进入系统后，哪些状态必须被显式保存，哪些事务必须被确认完成，哪些错误必须变成可诊断信息。

专题：主板不是插槽集合，而是平台约定 主板把 CPU、内存、固件、电源、时钟、PCIe 设备、USB/SATA/NVMe、显卡、网卡和低速管理控制器连成一个可启动平台。它的核心职责不是“提供很多接口”，而是保证复位后每个设备处于可枚举、可供电、可配置、可报告错误的状态。CPU 能跑程序只是平台的一部分；内存训练、PCIe 枚举、电源时序、时钟分配、固件配置和热管理同样决定整机是否可靠。



图示：PC 主板的关键不是“很多线”，而是启动、枚举、供电、时钟和错误报告共同构成平台约定。

主板对象	负责的问题	典型失败表现
供电 VRM	把电源转换成 CPU/GPU/内存所需电压，并按时序上电	复位不释放、负载变化时掉压、机器重启
时钟源	给 CPU、内存、PCIe、USB 等提供参考时钟	链路训练失败、设备枚举不稳定
固件 Flash	保存 UEFI/BIOS、微码、平台配置	无法取第一阶段代码或配置错误
DDR 通道	连接内存颗粒或 DIMM，完成训练和 ECC	内存训练失败、随机数据错误
PCIe Root	枚举显卡、NVMe、网卡和扩展设备	设备消失、降速、AER 错误
低速管理	Super I/O、EC、传感器、风扇、RTC	键盘、风扇、温度或睡眠唤醒异常

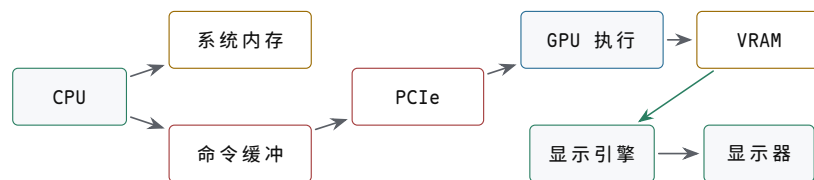
专题：PCIe 是分组互连，不是并行总线升级版 早期外部总线常暴露地址线、数据线、读写控制和 READY；PCIe 则把事务封装成包，在点对点串行链路上传输。CPU 或芯片组通过 root complex 发起配置、内存读写和消息事务；设备通过 BAR 暴露 MMIO 窗口，通过 MSI/MSI-X 发送中断，通过 DMA 访问主存。PCIe 的关键边界包括链路训练、配置空间、BAR 映射、事务层包、完成包和错误报告。

PCIe 概念	含义	本书对应概念
配置空间	枚举设备、读取 ID、配置 BAR 和能力	地址地图和平台发现

BAR	把设备寄存器映射到 CPU 地址空间	MMIO 副作用和访问属性
Memory Read/Write TLP	设备或 root 发起的内存事务包	总线事务和 completion
MSI/MSI-X	设备通过写内存样式消息触发中断	中断不一定是独立引脚
AER	高级错误报告	错误路径可诊断

把 PCIe 讲清楚后，显卡、NVMe、网卡都可以归入同一模型：先由平台枚举设备并分配地址窗口；驱动把命令写入队列或 MMIO；设备通过 DMA 读写主存；完成后写 completion 或发 MSI；错误通过状态寄存器、completion 状态或 AER 上报。这样读者不会把“插在主板上的设备”理解成 CPU 的附属品，而会把它看成能主动访问内存的总线主设备。

专题：显卡的核心不是显示器接口，而是并行处理器加显存系统 显卡既是显示设备，也是大规模并行计算设备。GPU 内部有许多执行单元、调度器、寄存器文件、L1/L2 cache、纹理/采样单元、光栅化和显示引擎；外部或封装上有高带宽显存，例如 GDDR 或 HBM。CPU 不会逐像素把画面写到显示器，而是准备命令缓冲区、资源描述、纹理和顶点数据，GPU 读取这些对象并在显存中生成帧缓冲，显示引擎再按扫描时序把帧送到显示接口。



图示：GPU 数据路径：CPU 提交命令，GPU 经 PCIe/内存读取资源，在显存中生成帧，显示引擎独立扫描输出。

显卡对象	硬件含义	常见误解
命令缓冲	CPU 写入、GPU 异步消费的工作队列	不是 CPU 直接调用 GPU 函数
VRAM	GPU 近处的大带宽存储系统	不是普通 DRAM 的别名，带宽和访问模式不同
着色/计算单元	大量相似线程并行执行	不是少数超快 CPU 核心
纹理/cache 单元	优化二维或三维局部访问和采样	不等于通用 cache 的简单复制
显示引擎	按固定时序读取帧缓冲并输出信号	渲染完成和显示扫描是两个边界

GPU 的性能问题也不同于 CPU。CPU 常追求低时延和强单线程控制流；GPU 常追求吞吐，用大量并行线程隐藏显存等待。分支发散、访存不合并、显存带宽不足、CPU/GPU 同步过多，都会让 GPU 性能掉下来。理解显卡时应问：命令何时提交，资源何时对 GPU 可见，GPU 何时完成，帧缓冲何时可显示，CPU 是否过早读回结果。

专题：整机案例：从打开程序到读取文件和显示图像 一个真实用户动作会穿过多种硬件。用户打开程序，操作系统可能从 SSD 读取可执行文件和页面；CPU 触发块设备队列；NVMe 控制器 DMA 数据到主存；页表和 cache 让 CPU 取指执行；程序提交图形命令；GPU 读取资源、渲染到 VRAM；显示引擎扫描帧缓冲。这里没有哪个部件可以单独解释全程，必须把 CPU、内存、SSD、PCIe、GPU、显示和中断连成 trace。

阶段	硬件动作	完成标志
缺页/读文件	OS 提交 NVMe 读请求，设备 DMA 到主存	块 I/O completion 和页状态有效
执行代码	CPU 取指、TLB/cache 查找、分支和写回	指令提交，异常路径为空
准备图形资源	CPU 写命令缓冲和资源描述符	命令队列对 GPU 可见
GPU 渲染	GPU 读取资源，执行 shader，写 VRAM	fence 或 completion 表示渲染完成
显示扫描	显示引擎按刷新时序读取帧缓冲	vblank、翻页或显示控制器状态

这个案例可以作为读完整本书后的纸面项目。要求不是实现操作系统和驱动，而是把每一步的机器行为写清楚：谁拥有缓冲区，谁发起 DMA，哪个地址空间有效，cache/TLB 在哪里参与，哪个 completion 才说明完成，哪个错误会阻止下一阶段。

专题：纸面项目应替代不可执行的大实验 如果没有硬件平台，仍然可以做高质量项目。项目形式应从“搭出来能跑”改成“规格写得可检查”。例如给出一台简化 PC：一个 CPU、两级 cache、DDR 控制器、PCIe root、NVMe、GPU、USB 控制器和固件 ROM。读者要写地址地图、启动顺序、设备枚举、一次文件读取、一次 GPU 命令提交、一次中断处理和一个错误路径。这样的纸面项目比空泛地说“了解主板”更有效。

纸面项目	必须交付	合格标准
简化 PC 地址地图	DRAM、固件、MMIO、PCIe BAR、未映射区	任意地址有明确目标或错误
NVMe 读路径	submission、doorbell、DMA buffer、completion、中断	CPU 不把门铃返回当成 I/O 完成
GPU 命令路径	命令缓冲、资源可见性、VRAM、fence、显示扫描	渲染完成和显示完成分开
主板启动路径	电源、时钟、复位、固件、内存训练、PCIe 枚举	每一步有失败后可诊断状态
错误注入	TLB miss、cache miss、PCIe 错误、SSD 超时、GPU hang	错误归属到具体层，不混成“程序卡住”

章末练习

任务	要求	学习目标
8086 地址形成	用段寄存器和偏移计算一个物理地址	能说明 20-bit 地址来自硬件加法路径
复用总线	写出 8086 读周期中地址、ALE、数据方向和 READY 的阶段	能指出何时锁存地址、何时采样数据
字节存储体	解释 A0 与 $\overline{\text{BHE}}$ 如何选择低/高字节	能处理奇地址字访问
80386 分页	把一个 32-bit 线性地址拆成目录索引、页表索引和页内偏移	能说明异常发生在真正访存前
保护模式入口	列出 GDTR/IDTR、CR0、CR3 和跳转刷新各自作用	能说明状态切换顺序

复位向量	比较 6502、Z80/8051、8086、80386 后续 x86 的复位入口	能说明第一条指令必须由非易失存储响应
平台分层	说明 CPU 本地总线、内存/cache 控制、高速扩展、低速 I/O 和固件层职责	能说明等待来自整个平台
SoC 演进	说明把内存控制器、I/O 和安全启动搬进片内的收益与代价	能指出片上互连和调试复杂度
主板结构	画出 CPU、DDR、PCH、UEFI Flash、PCIe、NVMe、GPU 的连接关系	能说明供电、时钟、复位和枚举不是 CPU 内部问题
PCIe 设备	说明配置空间、BAR、TLP、MSI 和 AER 各自解决什么问题	能把显卡/NVMe/网卡统一成总线主设备
显卡路径	写出 CPU 提交命令、GPU 读资源、VRAM 写帧、显示扫描的 trace	能区分渲染完成、DMA 完成和显示完成
6502 寻址与拍数	用零页寻址和 (zp),Y 间接变址写数组求和循环并逐拍计数	理解拍数就是总线事务序列
Z80 交换与刷新	说明 EXX 怎样加速中断响应、R 寄存器怎样把刷新藏进取指	寄存器组交换 = 不访存的上下文切换
486/Pentium 集成	列出 80486 和 Pentium 各自搬进片内的部件及对应收益	集成把等待从板级移到片内

参考解答与要点提示

任务	参考解答	必须掌握的要点
8086 地址形成	物理地址由 <code>segment*16+offset</code> 得到，内部编程模型保持 16-bit，外部地址能力扩展到 20-bit。	分段是地址形成硬件。
复用总线	T1 输出地址并用 ALE 锁存；T2 改变总线方向并给出读控制；T3/Tw 等待目标驱动数据且 READY 可延长；T4 采样并结束。	复用引脚必须分阶段解释。
字节存储体	偶地址低字节由 A0 选择，奇地址高字节由 BHE 参与选择；奇地址字访问可能拆成两个周期。	位宽不等于任意地址一次完成。
80386 分页	4 KiB 页下，高 10 bit 选 PDE，中 10 bit 选 PTE，低 12 bit 是页内偏移；PDE/PTE 不存在或权限不允许时触发异常。	地址转换也是硬件路径。
保护模式入口	进入保护模式前要准备描述符表、装载 GDTR/IDTR、设置 CR0、刷新取指路径；分页还需准备页目录/页表、装载 CR3、打开分页位。	状态切换顺序错误会让取指和异常入口失效。
复位向量	6502 从高端向量取入口，Z80/8051 常从 0 地址开始，8086 从 <code>0xFFFF0</code> 附近，后续 x86 从更高物理地址附近，SoC 可从 Boot ROM 进入。	启动入口是硬件规格。

平台分层	CPU 本地总线处理核心事务，内存/cache 控制处理 DRAM 和回填，高速扩展处理图形/磁盘/网络，低速 I/O 处理串口/键盘/RTC，固件负责初始化和启动选择。	主频之外还有平台等待。
SoC 演进	片上集成缩短常见路径、减少引脚和功耗，但引入片上互连、一致性、时钟/电源域、安全启动和调试复杂度。	低时延来自边界移动，不是所有路径变快。
主板结构	CPU 经前端总线连北桥，北桥负责内存和图形，南桥挂低速 I/O（串口、键盘、RTC），固件 Flash 上电先执行并完成初始化。	平台分层是把不同速度的路径分开组织。
PCIe 设备	固件读配置空间枚举设备并分配 BAR 地址窗口，驱动经 MMIO 配置队列，设备用 DMA 搬数据，MSI/MSI-X 投递完成中断，AER 报告错误。	枚举-配置-DMA-完成是统一模型。
显卡路径	CPU 写命令环并按 doorbell，GPU 读资源和命令渲染，结果写 VRAM，显示控制器按扫描时序读出；渲染完成、DMA 完成、显示完成是三件不同的事。	三种完成语义不能混。
6502 寻址与拍数	LDA 零页 3 拍，(zp),Y 间接变址 5 拍（跨页 6 拍）；求和循环每轮用 (zp),Y 读一个元素，接 INY、BNE，拍数可手工核对。	简单数据通路让逐拍 trace 可行。
Z80 交换与刷新	EXX 一条指令交换全部通用寄存器，省去压栈；M1 取指后半段把 R 输出到地址总线完成 DRAM 刷新。	交换和借用都是减少总线事务。
486/Pentium 集成	486 搬入 cache、FPU、流水线；Pentium 搬入双发射、BTB 预测、分离 cache 和 64-bit 数据总线。	每一代都在移动等待的位置。

读完后的检查表

读完本书后，至少应该能回答下面这些问题：

主题	能力要求
比特	能说明为什么计算机用两个稳定区间表示 0 和 1，而不是追求连续电压的精确值。
逻辑门	能从开关路径解释 NOT、AND、OR、NAND 的行为，并理解为什么 NAND 可以构造其他门。
组合逻辑	能把真值表转成门电路，解释半加器、全加器、多路选择器和译码器分别解决什么问题。
时序逻辑	能说明锁存器、触发器、寄存器和时钟边界的关系，知道为什么组合逻辑不能保存历史。
状态机	能把状态、输入、转移条件和输出分开，并说明状态机怎样控制一组硬件动作。
ALU	能把加法、减法、按位运算、比较和标志位放在同一个计算部件里解释。
最小 CPU	能画出 PC、指令存储器、寄存器堆、ALU、数据存储器 and 控制器之间的数据流。
整机硬件	能说明主存、总线、MMIO、DMA、中断控制器和复位启动怎样把最小 CPU 扩展成整机。
平台事务	能把一次内存、MMIO、DMA 或中断访问写成发起者、目标、地址、握手、完成、错误和副作用的 trace（执行轨迹）。
存储层次	能区分寄存器、cache、TLB、DRAM、SSD/硬盘的访问粒度、时延、带宽、完成方式和错误路径。
主板与扩展	能说明供电、时钟、固件、DDR、PCIe、NVMe、USB/SATA、网卡和显卡如何组成平台，而不是把整机理解为 CPU 附件。
显卡与显示	能写出 CPU 提交命令、GPU 读取资源、VRAM 写帧、显示引擎扫描输出之间的异步边界。
经典芯片	能用同一套 trace 方法解释 8086 的复用总线、80386 的地址转换、PC 平台分层和现代 SoC 的片上集成。

最小整机毕业设计

如果要把本书内容真正实现在一块教学板、FPGA 工程或仿真模型上，最后不要只满足于“程序能跑”的结论，而要保留一组可复查的实验记录。下面这条路线把全书链路压缩成一个最小整机：复位后从 ROM 取指，能执行 ADD、MOV、JZ，能访问 SRAM，能通过 MMIO 读按键并写 LED，能用中断或轮询处理一个外部事件。每一步都应能单步观察，失败时能定位到具体环节。

阶段	预期行为	观察点
电源/时钟/复位	电平稳定、时钟边沿可用、复位期间危险输出关闭	示波器截图或仿真波形，复位释放前写使能无效
取指	PC 指向复位入口，ROM 被正确片选，IR 锁存第一条编码	PC、地址线、ROM CS、数据线、IR 写使能
单条算术	寄存器读、ALU、标志位和写回边界正确	ADD R3, R1, R2 的源值、ALUOut、目标寄存器写入

访存	地址形成、SRAM 读写、字节使能和等待状态正确	地址、读写控制、MDR、写数据、ready/wait
分支	Zero 或比较结果能选择下一 PC	顺序 PC 和分支目标各跑一次，错误路径不提交
MMIO	普通 RAM 和设备寄存器不会混淆，副作用可观察	GPIO 地址片选、LED 锁存器写窗口、按键同步后的输入值
外部事件	轮询或中断能把设备状态带回 CPU	状态位锁存、清除顺序、服务程序入口和返回
错误路径	未映射地址、慢设备或设备错误有固定行为	超时、错误位、异常入口或固定返回值，不允许总线悬空

实验报告应包含的材料

高质量硬件报告不靠形容词，而靠可复查的记录。至少应包含下面几类材料：

材料	内容	常见不足
地址地图	ROM、RAM、MMIO、未映射区和复位入口	只有文字说明，没有片选表达式或互斥证明
指令 trace	每条核心指令的控制线、数据路径和提交边界	只列汇编程序，不说明硬件何时写状态
时序波形	取指、SRAM 读、SRAM 写、MMIO 写、中断或 DMA 完成	只截最终 LED 状态，没有关键控制信号
故障注入	故意访问未映射地址、制造慢 ready、触发错误状态	只验证成功路径，失败路径不可解释
可见性规则	DMA 或设备事件中 CPU 与设备何时拥有缓冲区和状态位	completion 到达前后 cache/缓冲区规则不清楚
复位记录	复位期间和释放后前几拍的 PC、片选、写使能	复位后第一条指令来源不明

排错入口

调试硬件时，不要从“代码可能写错了”开始。先把症状定位到具体环节，再选择观察信号。

症状	优先观察	常见根因
复位后无取指	复位、时钟、PC、ROM 片选、数据线	复位未释放、复位入口未映射、ROM 输出使能错误
第一条指令错误	地址锁存、ROM 内容、字节序、IR 写使能	地址线接错、程序镜像偏移、取指采样过早
算术偶发错误	源寄存器、ALU 输入、标志位、写回 mux	控制字位错误、组合路径太长、写回目标错
取数读到旧值	地址、片选、OE/WE、ready、MDR	读采样早于数据有效，或片选命中错误目标
存数破坏相邻数据	字节使能、写使能宽度、地址低位	字节车道选择错误或写窗口过宽
LED 或设备乱动作	MMIO 片选、写使能、设备状态机输入	地址译码过宽，普通内存写误触发设备
中断丢失或重复	设备状态、IRQ、控制器屏蔽、EOI、清除位	清除顺序错误，新旧事件没有分离
DMA 后数据不对	描述符、缓冲区地址、cache 可见性、completion	描述符对设备不可见，CPU 过早重用缓冲区

SSD 读请求卡住	submission queue、doorbell、控制器状态、completion queue、中断	把门铃写返回误认为块 I/O 完成，或 completion 未被消费
显卡画面不更新	命令缓冲、资源可见性、VRAM、fence、显示扫描	渲染完成、帧缓冲翻页和显示扫描三个边界混淆
PCIe 设备消失	链路训练、配置空间、BAR、MSI/AER、电源状态	平台枚举或链路错误被误认为驱动逻辑错误

最终检查不以“能不能复述章节标题”为准，而以能不能说清楚机器的实际行为为准。

- 给一个信号节点，能说出它何时被拉高、何时被拉低、何时悬空、何时冲突。
- 给一个组合电路，能列真值表、最长路径、可能毛刺和后级采样边界。
- 给一个状态电路，能指出旧状态、下一状态、时钟沿、建立/保持约束和复位行为。
- 给一个算术结果，能分别按无符号和补码解释，并说明 Carry 与 Overflow 的差别。
- 给一条最小 CPU 指令，能写出控制信号 trace、数据通路 trace 和 PC 下一值。
- 给一次内存或设备访问，能写出地址译码、总线/互连事务、响应返回和完成状态。
- 给一次 DMA 或中断，能写出缓冲区所有权、可见性、状态清除和完成通知顺序。

如果这些内容都能写出来，说明读者已经把“电平 -> 门 -> 组合逻辑 -> 状态 -> 数据通路 -> 控制器 -> CPU -> 整机”的链路连成了一台机器。